



McGRAW-HILL  
COMMUNICATIONS  
SERIES

**Man Young Rhee**

# **Error Correcting Coding Theory**

**DUQUESNE UNIVERSITY LIBRARY**







---

# Error-Correcting Coding Theory

John H. van Treebeek

Department of Mathematics

University of Toronto

McGraw-Hill Publishing Company

1221 Avenue of the Americas, New York, New York 10020-1095

London: 200, Old Street, London EC1Y 4EL, England

© 1995 by McGraw-Hill Publishing Company

Printed in the United States of America



---

# Error-Correcting Coding Theory

**Man Young Rhee**

*Hanyang University  
Seoul, Korea*

**McGraw-Hill Publishing Company**

New York St. Louis San Francisco Auckland Bogotá  
Caracas Hamburg Lisbon London Madrid Mexico  
Milan Montreal New Delhi Oklahoma City  
Paris San Juan São Paulo Singapore  
Sydney Tokyo Toronto

Library of Congress Cataloging-in-Publication Data

Rhee, Man Young

Error-correcting coding theory

Bibliography: p.

Includes index.

1. Error correcting codes (Information theory)

2. Data transmission systems. I. Title.

TK5102.5.R46 1989 005.7'2 88-37735

ISBN 0-07-052061-5

TK 5102.5

R46

1989

Copyright © 1989 by McGraw-Hill Inc. All rights reserved. Printed in the United States of America. Except as permitted under the United States Copyright Act of 1976, no part of this publication may be reproduced or distributed in any form or by any means, or stored in a data base or retrieval system, without prior written permission of the publisher.

1234567890 DOC/DOC 89432109

ISBN 0-07-052061-5

*The editors for this book were Theron Shreve and Rita T. Margolies, the designer was Naomi Auerbach, and the production supervisor was Dianne L. Walber. It was set in Century Schoolbook. It was composed by the McGraw-Hill Publishing Company Professional & Reference Division composition unit.*

*Printed and bound by R. R. Donnelley & Sons Company.*

Information contained in this work has been obtained by McGraw-Hill, Inc., from sources believed to be reliable. However, neither McGraw-Hill nor its authors guarantees the accuracy or completeness of any information published herein and neither McGraw-Hill nor its authors shall be responsible for any errors, omissions, or damages arising out of use of this information. This work is published with the understanding that McGraw-Hill and its authors are supplying information but are not attempting to render engineering or other professional services. If such services are required, the assistance of an appropriate professional should be sought.

*For more information about other McGraw-Hill materials, call 1-800-2-MCGRAW in the United States. In other countries, call your nearest McGraw-Hill office.*

---

# Contents

Preface   vii

|             |                                                                             |     |
|-------------|-----------------------------------------------------------------------------|-----|
| Chapter 1.  | Introduction and Overview                                                   | 1   |
| Chapter 2.  | Elements of Abstract Algebra for Coding                                     | 15  |
| Chapter 3.  | Linear Binary Block Codes                                                   | 35  |
| Chapter 4.  | Cyclic Codes                                                                | 89  |
| Chapter 5.  | Short-Random-Error Correction by Error-Trapping Techniques                  | 115 |
| Chapter 6.  | Burst-Error Correction for Block Codes                                      | 145 |
| Chapter 7.  | Bose-Chaudhuri-Hocquenghem Codes and Reed-Solomon Codes                     | 167 |
| Chapter 8.  | Majority-Logic Decodable Codes                                              | 255 |
| Chapter 9.  | Convolutional Codes and Their Structures                                    | 311 |
| Chapter 10. | Maximum-Likelihood Decoding and Sequential Decoding for Convolutional Codes | 351 |
| Chapter 11. | Threshold Decoding for Convolutional Codes                                  | 387 |

|                   |     |
|-------------------|-----|
| Bibliography      | 427 |
| List of Symbols   | 435 |
| Glossary of Terms | 441 |
| Index             | 453 |

MAY 27 1993





---

# Preface

The digital coded systems for either data transmission or data storage have much in common. Since the channel or storage medium is subject to various types of noise, distortion, and interference, the output of the channel or storage medium differs from its input because they are both sensitive to the errors that can result from impaired transmission. The theory and practice of error-correction coding is concerned with protection of digital information against the errors that occur during data transmission or storage. Many ingenious error-correction techniques based on a vigorous mathematical theory have been developed and have many important and frequent applications. The current problem with any high-speed data communication system is how to control the errors that occur during data transmission through a noisy channel. In order to achieve reliable communications, designers should develop good codes and efficient decoding algorithms. Knowledge of error-correction coding is in great demand and becomes an important asset for many practicing engineers and computer scientists who are involved in the design of large digital systems. The amount of coding research directed at error control will continue to grow because the recent developments in integrated circuit chip technology have advanced so much.

The history of error-correction coding begins in 1948 with the publication of a landmark paper by Claude Shannon. Since Shannon's work, a great deal of effort has been devoted to the encoder-decoder implementation for controlling errors in noisy environments. The pioneering work of coding in the early 1950s concentrated on a theory that required extensive mathematical rigor in the domain of abstract algebra and probability theory. During the 1960s much effort was devoted to finding structures for classes of good codes that could produce an arbitrarily small probability of error, but little progress was made. In the 1970s, coding research began to focus on designing families of codes with larger code lengths and better performance. However, further advances are still required to overcome practical limitations. As the era of the 1980s opened, the emphasis in coding research shifted dramatically from theory to practical applications.

This book is intended to serve as an introduction to error-correction coding theory. The primary objective is to present how the basic

concepts and techniques of error control are applied to digital transmission and storage systems so that the underlying theory can be better understood. The book progresses systematically from block codes to convolutional codes, through material that lies at the forefront of modern coding theory. In studying the construction and properties of error-correcting codes, one needs to make extensive use of the notions associated with the theory of vector spaces and to understand the concepts and operations of abstract algebra.

The mathematical prerequisite introduced in Chapter 2 provides a theoretical format for the remainder of the text. Each succeeding chapter then builds upon this theory, applying it to a specific area within coding research. However, mathematical sophistication has been kept at the lowest level possible. More than 134 examples with detailed solutions and many theorems with proofs appear in the text, both to provide a greater understanding of the subject matter and to induce a more flexible approach to the solution of the problems at the end of each chapter. A solution manual is available to instructors through the publisher.

Chapter 1 presents an introduction and overview of error-control coding for digital communications and computer storage systems. Since coding is an extremely mathematical subject, Chapter 2 presents a brief survey of the powerful structure of abstract algebra. This algebraic framework provides the tools needed by the reader to understand the theory of error-correcting codes. Although the mathematical treatment in this chapter is somewhat rigorous, it is limited in scope to materials that will be useful in succeeding chapters.

Chapters 3 to 8 are devoted to the analysis and design of block codes that control errors in a noisy environment. The fundamentals of linear block codes are thoroughly presented in Chapter 3. Cyclic codes, as they are used in practice, form an important subclass of block codes. These codes are attractive because encoding and syndrome computation can easily be implemented by employing shift-register circuits or digital logic circuits. The basic structure and properties of cyclic codes are presented in Chapter 4.

Chapter 5 is mainly concerned with coding techniques for noisy channels on which transmission errors occur independent of digit position, i.e., random errors. A simple way of decoding for some cyclic codes, known as error-trapping decoding, is covered in this chapter. Chapter 6 covers cyclic codes that are very effective for burst-error correction. Many theorems and proofs that have relevance to burst-error correction are also included. There are extensive discussions of burst-error-trapping decoding using Fire codes.

Chapter 7 provides detailed coverage of BCH codes for multiple error correction. Iterative algorithms for finding the error-location polyno-

mial are presented in great detail. Numerous examples are presented, and various hardware configurations are discussed. Hardware implementations for the syndrome computation circuit and the searching unit for error-location numbers are presented. Software for the error-location polynomial is included, and discussion of nonbinary Reed-Solomon codes for concatenated coding systems is also presented. Chapter 8 presents the subject of majority-logic decodable codes. Majority-logic decoding techniques for correcting random errors are fully covered.

Chapters 9 through 11 are devoted to the principles, structures, and encoding-decoding methods of convolutional codes. Chapter 9 introduces the fundamental properties of convolutional codes, coupled with the encoder state diagram that serves as the basis for studying tree or trellis code structure and distance properties. Chapter 10 presents probabilistic decoding—that is, maximum-likelihood decoding and sequential decoding. Maximum-likelihood decoding using the Viterbi algorithm for hard and soft decisions, as well as sequential decoding using both the Fano and the ZJ stack algorithms, are intensively covered in this chapter. Performance analysis based on code distance properties is also included.

Chapter 11 presents threshold decoding of convolutional codes by Massey's majority-logic decodable principle. Convolutional codes for correcting longer bursts can be obtained by interleaving. The interleaved codes will correct not only burst errors but also many patterns of random errors. In this chapter, a code developed specifically for correcting burst errors is covered in detail. Since this book presents the application of error-correction coding in the context of communications system design, the emphasis throughout is placed on the design and implementation of encoders and decoders.

The scope of the book is adequate to span a two-semester graduate-level sequence, and the material has been organized with such a course in mind. The book can also be used as reference for electronic engineers and computer scientists in industry who are interested in studying the fundamentals of coding and how to apply the principles to the design of error-control digital systems.

A natural division of the material is to cover Chapters 1 through 7 in a first-semester graduate course, and Chapters 8 through 11 in the second. Alternatively, portions of the book can be used for a one-semester course, at the instructor's choice, but it is recommended that the instructor cover Chapters 3, 4, 6, 7, 9, and 10, which include the fundamentals of both block codes and convolutional codes. Chapters 3, 4, 5, 7(partly), 9, and 10 should be studied by senior students who are seriously interested in coding theory.

In most books, the notation for block codes often differs from the



notation for convolutional codes. Therefore, I have given careful consideration to choosing notation that brings clarity and internal consistency to the book. The symbol  $d$  is used for the information data;  $c$  for the code word or transmitting vector;  $e$  for the error pattern; and  $r$  for the received word. This notation is used consistently not only with block codes, but with convolutional codes as well.

This book is the outgrowth of my teaching and research efforts in coding over the last 20 years at the university and in industry, coupled with my supervision of many graduate students at both the M.S. and Ph.D. levels. I thank them all, even if not by name. The books and papers that had a large influence on this book, either directly or indirectly, are listed in the bibliography at the end of the text. However, it is impossible to mention all those people who influenced me a great deal in writing this book. My sincere gratitude goes to them.

I am grateful to the Ministry of Communications, Republic of Korea, Korea Telecommunication Authority, the Electronics and Telecommunications Research Institute, and the Korea Science and Engineering Foundation for their continuing support of my interest and research in the coding field. Some of the research papers for those organizations are contained in this book.

I would like to express my special appreciation to Rita Margolies, Editing Supervisor, who made numerous corrections and suggestions for improvements of this book. I also wish to thank composition and production people for their dedication and perseverance in preparing this beautiful book.

*Man Y. Rhee*



# Introduction and Overview

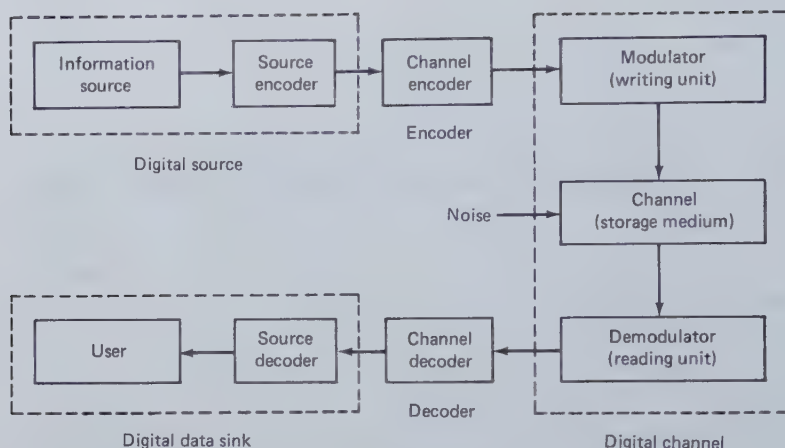
Digital data transmission through a physical channel in a communication system and data recording on a storage medium in a computer system have much in common. In both cases, digital data are transferred from an information source to a destination. Since the channel or storage medium is subject to various types of noise, distortion, and interference, the output of the channel or storage medium differs from its input because of the errors that can result from impaired transmission. Therefore, the need for error control arises from the massive amount of data processed in communication and storage systems.

Good codes and coding algorithms are available to meet this need. In addition, the rapid advances in gate array integrated circuit chip technology have made possible the design and implementation of encoder-decoder pairs which use these coding algorithms.

Error-control coding is a special subject with its own history and its own arithmetic systems. The theory and the practice of error-control coding relate to protection of digital information against the errors that occur during data transmission or storage. Many ingenious error-correction techniques have been developed based on rigorous mathematical theory and have then become important subjects with frequent applications. A major concern of the designer is the control of errors so that reliable reproduction of information and data can be obtained.

## 1.1 Model of Digital-Coded Systems

The block diagram in Fig. 1.1 illustrates the basic elements for transmission or storage of digital information through a coded system. For this system, all information transferred between blocks must be in digital form if error control is to be utilized.



**Figure 1.1** Block diagram for digital-coded system.

Data, which enter the communication (or storage) system from the information source, are first processed by a source encoder that is designed to convert the source information into coded form. The source encoder usually transforms the source output into a sequence of binary digits (bits) called the information sequence  $\mathbf{d}$ . The source output can be either a continuous waveform or a sequence of discrete symbols. In the case of analog output, the source encoder must possess an analog-to-digital (A/D) conversion capability, for example, pulse-code modulation (PCM).

The channel encoder transforms the information sequence  $\mathbf{d}$  into a binary-coded sequence  $\mathbf{c}$  called a code word. The channel code word is a new, longer sequence that contains redundancy of parity-check symbols. Each symbol in the code word might be represented by a bit or, perhaps, by a group of bits. In general, binary digits are not suitable for transmission over the waveform channel or for recoding on a digital storage medium.

The binary digits in a code word from the channel encoder are fed into a modulator (or writing unit) that transforms each bit into an elementary signal waveform. Thus, the modulator must convert each bit of the channel code word into a suitable waveform of duration  $T$  seconds in order for the bits to transmit. Binary phase shift keying (PSK) or frequency shift keying (FSK) are commonly used as signaling waveforms for transmitting the code word. This waveform enters the channel (or storage medium) and is corrupted by noise. The waveform channel consists of all the hardware and physical media that the waveform passes through in going from the output of the modulator (or writing unit) to the input of the demodulator (or read-

ing unit). In coherent systems, the binary modulation scheme of PSK is often used. With this binary transmission, a 1 is represented by the waveform  $s_1(t) = \sqrt{2P} \cos \omega_0 t$ , while a 0 is represented by the antipodal signal  $s_0(t) = -s_1(t) = \sqrt{2P} \cos(\omega_0 t + 180^\circ)$ , where the power in the waveform is  $P = E_b/T$ . In noncoherent PSK systems, the demodulation of the waveform sign is not possible, so typically a pair of signaling tones,  $s_0(t) = \sqrt{2P} \sin(\omega_0 t + \theta)$  and  $s_1(t) = \sqrt{2P} \sin(\omega_1 t + \theta)$ , is used to represent the transmitted digits 0 and 1, respectively, and to pass the received signal through parallel bandpass matched filters. An alternate approach is to use a correlator with the received signal multiplied by a locally generated version of the transmitted waveform. Typical examples of waveform channels are telephone lines, microwave links, high-frequency radio links, telemetry links, and satellite links. Typical storage media include core memories, magnetic tapes, drums, disk files, and optical memory units. Each of these examples is subject to various types of noise disturbances. Some random or burst noise is usually added to the channel waveform during transmission. Distortion may be present as a result of heavy filtering or multiple signal paths. The disturbance may cause signal suppression which could in turn cause the amplitude of the received signal to vary, or the channel itself may be time-varying. The disturbance may generally be modeled as an additive gaussian process, or it may be urban noise of various kinds, or it may result from intentional jamming by an unfriendly party. Channels are also defined in another way. For memoryless channels, the noise affects each transmitted bit independently. Hence transmission errors occur randomly in the received word  $\mathbf{r}$ . Typical examples of memoryless random-error channels are most line-of-sight transmission channels, many satellite channels, and deep-space channels. For memory channels, the noise does not occur independently in the channel and the transmission errors occur in bursts. Examples of burst-error channels are (1) radio channels where the error bursts are caused by signal fading as a result of multipath transmission, (2) telephone lines which are affected by impulsive switching noise and crosstalk from other lines, and (3) magnetic recordings, which are subject to tape dropouts due to surface defects and dust particles. Of course, there are some channels which contain a combination of both random and burst errors.

Next, the resulting received signal is processed first by the demodulator and then by the channel decoder. The demodulator (or reading unit) makes a decision for each received signal of duration  $T$  seconds to determine which of the two possible digits, 1 or 0, was transmitted. This is called a hard decision. The output of the demodulator is called the received word  $\mathbf{r}$ . This  $\mathbf{r}$  may not match the transmitted code word

$\mathbf{c}$  as a result of transmission error. Each demodulated digit is a best estimate of the transmitted digit, but the demodulator makes some errors because of channel noise. The probability that this estimate is correct depends upon the signal-to-noise ratio in the data bandwidth, the amount of signal distortion due to filtering and nonlinear effects, and the detection scheme being used.

The channel decoder transforms the received sequence  $\mathbf{r}$  into a binary sequence  $\hat{\mathbf{d}}$ , or the estimated information sequence. Since the noise may cause some decoding errors, the channel decoder must be implemented to minimize the probability of decoding error. The channel decoder uses the syndrome of a received code word  $\mathbf{r}$  to correct the errors in the received word and then produces an estimate of the information sequence  $\mathbf{d}$ . If all errors are corrected, the estimated information sequence  $\hat{\mathbf{d}}$  matches the original source information  $\mathbf{d}$ .

The source decoder transforms the estimated sequence  $\hat{\mathbf{d}}$  into an estimate of the source output and delivers this estimate to the user. Thus the source decoder performs the inverse operation of the source encoder and delivers its output to the data sink.

This book aims to present the analysis, design, and implementation of the channel encoder and decoder, which is a subject known as error-correction coding. The data compression or compaction functions performed by the source encoder and the source decoder are not discussed here. The information source and source encoder are combined into a digital source with output  $\mathbf{d}$ ; the modulator (or writing unit), the waveform channel (or storage medium), and the demodulator (or reading unit) are combined into a coding channel with input  $\mathbf{c}$  and output  $\mathbf{r}$ ; and the source decoder and user are combined into a digital sink with input  $\hat{\mathbf{d}}$ . Thus, in this book, the channel encoder and decoder will be referred to simply as the encoder and decoder.

## 1.2 Coding Classification

Coding is largely classified into two different types: (1) coding for block codes and (2) coding for convolutional codes. All data of interest in a binary block code can be represented as a binary information sequence consisting of 1s and 0s. The encoder for a block code divides the information sequence into information blocks of  $k$  bits. Thus, each information block is represented by the binary  $k$ -tuple  $\mathbf{d} = (d_0, d_1, \dots, d_{k-1})$  simply called an information sequence. There are a total of  $2^k$  different possible kinds of information sequences. The encoder transforms each  $k$ -bit information sequence  $\mathbf{d}$  independently into an  $n$ -tuple  $\mathbf{c} = (c_0, c_1, \dots, c_{n-1})$  called a code word. Thus, a binary block code consists of a set of code words of length  $n$ . The elements of a code



word are selected from an alphabet of  $q$  elements. When  $q = 2$  for 0 and 1, the code is called a binary code, whereas when  $q > 2$ , the code is nonbinary. There are  $2^n$  possible code words in a binary block code of length  $n$  ( $k < n$ ). However, from these  $2^n$  code words, we select only  $2^k$  code words to form a code. Thus a block of  $k$  information bits is mapped into a code word of length  $n$  at the encoder output. This set of  $2^k$  code words of length  $n$  is called an  $(n, k)$  block code. The ratio  $R = k/n$  is known as the code rate. Since the  $n$ -bit code word  $\mathbf{c}$  depends only on the corresponding  $k$ -bit information sequence,  $\mathbf{d}$ , the encoder is memoryless and can be implemented with a combinational logic circuit. Since  $R \leq 1$  for a binary code,  $n - k$  redundant bits can be added to each  $k$ -bit information sequence to form a code word. These redundant bits provide the code with the capability of combating the channel noise. These  $n - k$  redundant bits are called parity-check bits.

Besides its code rate  $R$ , an important parameter of a code word is its weight, which is simply the number of nonzero elements that it contains. In general, each code word has its own weight. The set of all weights in a code constitutes the weight distribution of the code. When all the  $2^k$  code words have equal weight, the code is called a constant-weight code. The distance between the two code words is defined as the number of corresponding elements or positions in which they differ. This measure is called the Hamming distance. The smallest value of the set of Hamming distances for the  $2^k$  code words is called the minimum distance and is denoted by  $d_{\min}$ . An  $(n, k)$  block code with the minimum distance  $d_{\min}$  guarantees the correction of all the error patterns of  $t = \lfloor (d_{\min} - 1)/2 \rfloor$  or fewer errors ( $\lfloor x \rfloor$  denotes the largest integer contained in  $x$ ). The performance characteristics obtained by coding will depend on a number of code parameters such as the code rate, the number of code words in the code, the distance properties of the code, the coding bound, and the coding gain.

A number of elementary concepts from linear algebra play a central role in coding theory. Expressed in terms appropriate for  $(n, k)$  block codes, the vector space  $V$  consists of the  $2^n$  distinct  $n$ -tuples over the field of two elements  $\{0, 1\}$ . An  $(n, k)$  linear code is a set of  $2^k$   $n$ -tuples called code words which forms a subspace  $S$  of vector space  $V$  over Galois field  $GF(2)$ . If we select a set of  $k$  linearly independent vectors from  $V$  and from this set construct the set of all linear combinations of these vectors, the resulting set forms a subspace  $S$  of dimension  $k$ . If the set of vectors in  $V$  is orthogonal to every vector in  $S$ , this set of vectors is also a subspace of  $V$  and is called the null space of  $S$ . Since the dimension of  $S$  is  $k$ , the dimension of the null space is  $n - k$ . The null space of  $S$  is another linear code which consists of  $2^{n-k}$  code words of length  $n$  and  $n - k$  information bits. We call it an  $(n, n - k)$  dual



code. Furthermore, some topics that are related to the theory of error-correcting codes, such as groups, rings, and Galois fields in abstract algebra, will be discussed in Chap. 2.

The usual figure of merit for a coded system is the ratio of energy per information bit to noise spectral density  $E_b/N_0$  that is required to achieve a given probability of error. The coding gain is defined for the amount of improvement that is achieved when a particular coding scheme is used. The usual method of determining coding gain is to plot the probability of error  $P(E)$  versus  $E_b/N_0$  for both coded and uncoded operations and to read the difference in required  $E_b/N_0$  at a specified error rate. There are essentially two kinds of coding bounds, i.e., bounds on minimum distance and bounds on performance. The bounds on minimum distance are the Hamming bound and the Plotkin bound, which indicate the maximum possible minimum distance for a given code length and code rate, while the achievable lower bound on the minimum distance of the best code is the Gilbert-Varsharmov bound. When new codes are being developed, minimum distance bounds are often used to determine how close the code is to the best possible one. These bounds are fully discussed in Chap. 3. The bounds on performance indicate that the average performance of all block codes exhibits a probability of error that decreases exponentially with code length. These random coding bounds imply the existence of specific codes which do better than average. However, these bounds are not very useful for estimating the absolute performance of a code because good codes exhibit a probability of error that is considerably lower than that predicted by the bound. Chapters 5 through 8 are devoted to the analysis, design, and implementation of block codes for controlling errors in a noisy environment.

Modern coded systems are often designed to transmit at very high data rates. To protect such systems from error, designers often use an alternate coding scheme using convolutional codes in addition to block codes. A binary convolutional code can be generated by a linear finite-state machine connected to an  $m$ -stage shift register,  $n$  modulo-2 adders connected to some of the shift register stages, and a multiplexer that scans the output of the modulo-2 adders. This machine is called an  $(n, k)$  convolutional encoder with memory order  $m$ , sometimes denoted as an  $(n, k, m)$  convolutional encoder as a matter of convenience.

The input data to the convolutional encoder, called the information sequence, are shifted into and along the shift register  $k$  bits at a time, and the encoder output sequences are obtained by taking the convolution of the information sequence with the generator sequences of the code. Thus the multiplexer terminals use a process that scans in a serial fashion for the code word that is to be transmitted over the chan-

nel. The number of output bits for each  $k$ -bit input sequence is  $n$ . Since a convolutional encoder generates  $n$  encoded bits for every  $k$  information bits, the code rate is  $R = k/n$ , which is consistent with the definition of the code rate for a block code. Generally,  $k$  and  $n$  are small integers.

The generator matrix  $G$  of the code is a semi-infinite matrix which has an infinite number of rows and columns. This corresponds to the fact that the information and code sequences can be arbitrarily very large. However, for any practical application, there is a maximum allowable length  $L$  for which we often define the  $L$ th truncation of a convolutional code. Thus the information sequence consists of  $kL$  bits, and the code sequence is represented by  $n(m + L)$  bits. Therefore, the  $L$ th truncation of an  $(n, k)$  convolutional code with memory order  $m$  can be viewed as an  $(n(m + L), kL)$  linear block code. In this sense, convolutional codes may be thought of as a special class of linear block codes with superior properties which both facilitate decoding and improve performance. The constraint length is defined as  $n_A = (m + 1)n$  because it is the maximum number of encoder outputs that can be affected by a single information bit.

With block codes, algebraic properties have been very important in constructing good classes of codes and in developing decoding algorithms. However, this is not the case with convolutional codes in general. Decoding of convolutional codes is probabilistic decoding. Generally, maximum-likelihood decoding by the Viterbi algorithm and sequential decoding by the Fano algorithm or the stack algorithm are called probabilistic decoding. On the other hand, threshold or majority-log decoding is classified as algebraic decoding.

The Viterbi algorithm was proposed in 1967 and has been recognized as an attractive solution to maximum-likelihood decoding. Maximum-likelihood decoding is characterized as the determination of the shortest path through a topological structure called a code trellis; an efficient solution for that determination is the Viterbi algorithm. However, Viterbi decoding of convolutional codes is not practical for a long code with a larger constraint length  $n_A$  because the error probability increases exponentially with  $n_A$ . Since both complexity and decoding effort increase exponentially with  $n_A$ , convolutional codes with small constraint lengths must be used for Viterbi decoding because of the limitation of the encoder memory  $m$ . Another point related to the Viterbi algorithm is that  $2^m$  computations must be performed per data sequence even when the noise effect is negligible, which results in wasted decoding effort.

In 1961, Wozencraft devised a decoding technique called sequential decoding, which has been the subject of research interest for over 20 years. In 1963, Fano introduced a new version of sequential decoding

which is referred to as the Fano algorithm. Another version of sequential decoding, known as the stack or ZJ algorithm, was discovered independently by Zigangirov in 1966 and Jelinek in 1969. A major problem with sequential decoding is that the number of computations required to advance one node deeper into the code tree is a random variable. This characteristic strongly affects the complexity required to achieve a given level of performance. The performance of sequential decoding is slightly suboptimum, but its decoding effort is independent of the code constraint length. Thus sequential decoding can be used with codes of long constraint lengths. But its major drawback is that noise frames require a great deal of computation. Consequently, decoding times occasionally exceed some upper limit, causing data to be lost or erased.

In 1963, Massey introduced a less efficient but simpler-to-implement decoding method, called majority-logic or threshold decoding, which would be applicable to convolutional codes. The structure of convolutional codes is not algebraic but topological. But an algebraic approach can also be used for decoding convolutional codes. Conceptually and practically threshold decoding is closest to majority-logic decoding of block codes. Thus threshold decoding can eliminate the search aspects of Viterbi decoding and sequential decoding. Although threshold decoding is inferior in performance when compared to Viterbi or sequential decoding, decoder implementation is somewhat simpler.

### 1.3 Development of Coding Theory

The history of error-correction coding began in 1948 with the publication of a landmark paper by Claude Shannon. Since Shannon's work, a great deal of effort has been devoted to the problem of implementing encoder-decoder pairs for error control in noisy communications and digital computers. There are two different types of codes in common use today, i.e., block codes and convolutional codes.

During most of the 1950s, coding research was devoted primarily to block codes with a strong emphasis on algebraic approaches. In spite of diligent research, no better class of codes was found until the end of the decade. Hamming codes (1950) were the first class of linear block codes devised for single-error correction.

During the 1960s much effort was devoted to finding structures for classes of good codes that could produce an arbitrarily small probability of error, but not much progress was made. Meggitt (1961) devised a decoder that was applicable to any cyclic code, but refinements were necessary for practical implementation; the technique is known as error-trapping decoding. Although the idea of error-trapping decoding



was introduced by Meggitt, it was refined by Kasami, Mitchel, Rudolph, and others. However, error-trapping decoding becomes ineffective if it is applied to long, high-rate codes that have a greater error-correcting capability. Fire (1959) discovered a large class of burst-error-correcting cyclic codes. Fire codes comprise the first class of cyclic codes that can be used for correcting burst errors and that can be decoded by the error-trapping technique. Burton (1969) introduced a class of phased-burst-error-correcting cyclic codes. Bose and Chaudhuri (1960) and Hocquenghem (1959) found a large class of multiple-error-correcting codes, called the BCH codes. The discovery of these powerful BCH codes led to a search for a practical decoding algorithm, which was devised by Peterson (1960), refined by Gorenstein and Zierler (1961), and extended by Berlekamp, Massey, Chien, Forney, and Lin. In addition, Reed and Solomon (1960) found an extremely important and practical class of nonbinary BCH codes that could cope with bursts of errors. The Reed-Solomon codes are at present coming into widespread use in many communication and computer storage systems. The majority-logic decoder is another effective device for decoding certain classes of cyclic block codes. Massey (1963) was the first to present a unified treatment for majority-logic decoding algorithms. The maximum-length codes and the difference-set codes are two small subclasses of one-step majority decodable cyclic codes. Finite-geometry codes (euclidean geometry codes and projective geometry codes) have their own structure and rules of orthogonalization for the multiple-step majority decodable codes. Rudolph (1967) investigated finite geometry codes which were extended and generalized by many coding researchers.

Another remarkable achievement made in the 1960s was the discovery of convolutional codes. Since 1955 when convolutional codes were first introduced by Elias, Wozencraft (1961) proposed sequential decoding as a practical decoding method, Fano (1963) introduced the Fano algorithm as a new version of sequential decoding, and another version of sequential decoding, called the ZJ stack algorithm, was independently discovered by Zigangirov (1966) and Jelinek (1969). In 1967, Viterbi proposed the Viterbi algorithm for maximum-likelihood decoding. Since then Omura (1969) and Forney (1972–1974) proved that the Viterbi algorithm is the best maximum-likelihood decoding algorithm for convolutional codes with shorter constraint lengths. In 1963, Massey proposed a less powerful but easy to implement decoding method called threshold decoding. It differs from Viterbi decoding and sequential decoding in that the final decision made on a given information block is based on only one constraint length of received blocks rather than on the entire received sequence. Since Hagelbarger (1959) introduced the concept of correcting burst errors of recurrent codes, many coding theorists have developed techniques for providing more



efficient convolutional codes that are designed for burst-error correction. The main contributors in this area are Berlekamp (1964), Preparata (1964), Massey (1965), Iwadare (1968), Kohlenbarg and Forney (1968), Gallager (1968), Tong (1970), and Ferguson (1971).

In the 1970s, coding research began to settle down to the task of designing families of codes that have larger code lengths and better performance. However, further advances were still required to overcome practical limitations. Research on block codes as well as convolutional codes was again combined. Coding theorists brought new insights to these subjects, and substantial advances were made.

As the era of the 1980s opened, the emphasis in coding research shifted from theory to practical applications. Encoders and decoders began to appear frequently in newly designed digital communication systems and digital storage systems.

## 1.4 Overview of This Book

A brief description of each chapter follows.

Chapter 1 presents an overview of coding for error control in data transmission and storage systems. Chapter 2 develops the mathematical topics from abstract algebra that are necessary for understanding coding theory. These mathematical topics are needed to comprehend the materials in subsequent chapters.

Chapters 3 through 8 are devoted to detailed discussions of block codes and the associated decoding techniques for random-error correction as well as burst-error correction. From the standpoint of practical importance, the linear binary codes discussed in Chap. 3 constitute a very small subclass of all possible block codes. Fundamental concepts of linear codes (parity-check matrix, encoder implementation, coding bounds, minimum distance, syndrome, standard array, weight enumerators, probability of decoding error, etc.) are developed in this chapter, and several useful classes of codes are also presented. Most linear block codes used in practice are cyclic codes that possess algebraic and geometric structures. The basic structure and properties of cyclic codes are presented in Chap. 4. The Meggit decoder and error-trapping decoding for short random-error correction (three or fewer errors) are presented in Chap. 5. Chapter 6 covers almost exclusively block codes designed for the correction of burst errors. Cyclic codes such as Fire codes are used to correct long bursts of errors and are decoded using the error-trapping techniques. Also included is an extensive section on several theorems which are most helpful for correcting a single burst of errors. Chapter 7 presents a large class of the most powerful random-error-correcting codes, known as BCH codes, which occupy a prominent place in the theory and practice of multiple-

random-error correction. Decoding procedures for computations of syndrome, error-location polynomials, and error-location numbers are fully covered. Numerous examples are presented, and a discussion of hardware and software implementation of BCH codes is included. This chapter also covers Reed-Solomon (RS) codes for burst-error correction and application of RS codes to concatenated coding. Chapter 8 provides detailed coverage of majority-logic decoding and majority-logic decodable codes, including finite-geometry codes. The majority-logic decoders can always be implemented in a manner similar to Meggitt decoders and are characterized by a simple majority gate for examining the syndrome. Thus these codes are decoded rather simply in most cases but seem to be somewhat less powerful than the BCH codes in terms of their ability to correct errors for the most interesting values of code length and efficiency.

Chapters 9 through 11 are devoted to the presentation of the fundamentals of convolutional codes. Chapter 9 introduces convolutional codes, with the encoder state diagram serving as the basis for studying code structure and distance properties. Since a convolutional encoder is a finite-state machine, its operation can be described by a state diagram. The structure of convolutional codes can be delineated with the aid of the state, tree, and trellis diagrams. The distance properties and performance bounds for probabilistic decoding can be obtained from the state diagram. The performance of a convolutional code depends on the decoding algorithm employed and the distance properties of the code. The generating function approach to finding the weight distribution is fully covered in this chapter. Chapter 10 covers the Viterbi algorithm for maximum-likelihood decoding and also presents the stack and Fano algorithms for sequential decoding. A detailed performance analysis based on code distance properties is included, in addition to a discussion of the difficult problem of the computational performance of sequential decoding. Chapter 11 presents threshold decoding of convolutional codes. Several codes (self-orthogonal codes, trial-and-error orthogonalizable codes, uniform codes, etc.) for threshold decoding are analyzed in detail. Techniques that provide more efficient codes designed for burst-error correction of convolutional codes are discussed in this section. This chapter concludes with comparison of the performance of the three major decoding methods for convolutional codes.

## 1.5 Coding Applications

The research for error-correcting codes was motivated primarily by the problems arising in communications systems. For example, many communications systems have limitations on their transmitted power.

Error-correcting codes are an excellent means of reducing power needs because reliable communications can be achieved with the aid of codes even when the information is weakly received at its destination. These codes, however, have many other applications. Error-correcting codes are used to protect data in computer memories, on magnetic tape, or in disk storage systems and to protect against noise or circuit malfunction in digital logic circuits.

Block coding schemes are often used for error control in computer magnetic tape or disk storage systems. Convolutional coding schemes are widely used for error control in many terrestrial, satellite, and space communications systems.

Computer systems cannot tolerate even very low error rates in their data flows, because a single error can destroy a computer program. A class of single-error-correcting and double-error-detecting Hamming code has been effectively used (for example, in IBM systems 360 and 370) to improve the reliability of core memories. Semiconductor memories are faster than core memories, but they are less reliable because of their high density per chip and their exposure to radiation. As a result, efficient computer operation demands the use of error-correcting codes for improving semiconductor memory reliability. The most important features of these codes are fast encoding and error detection in the decoding process. Fire codes or RS codes are usually used for error control in magnetic disk storage systems. The most common errors on magnetic tapes are burst errors or erasures on tracks, which are caused by surface defects in the presence of dust particles. As with magnetic tape storage systems, burst errors predominate in magnetic disk storage systems. These errors are primarily caused by surface irregularities, such as defects or variations in head-media separation in the presence of dust particles. Block codes are also used for error control in magnetic drums.

Convolutional codes have found applications in many diverse systems. Convolutional coding with hard- and soft-decision Viterbi decoding has found application in many space and satellite communications systems. For example, the Voyager space mission in 1977 to Mars, Jupiter, and Saturn used an encoder-decoder pair designed by the Jet Propulsion Laboratory that operated with a (2, 1, 6) convolutional code. Viterbi decoding of convolutional codes can also be used in a concatenated coding system with a RS block code. The high reliability and modest complexity of concatenated coding systems make them prime candidates for many of the space and satellite applications of the 1980s. Convolutional coding with hard- and soft-decision sequential decoding has been applied in many space communications systems. NASA's Pioneer 9 solar orbit space mission, launched in 1968, was the first deep-space mission to make use of error-correcting codes.



The code used on Pioneer 9 was a (2, 1, 24) convolutional code. The Pioneer II Saturn fly-by mission, launched in 1973, operated at data rates of up to 2 kb/s. The Linkabit Corporation designed and built a number of hardware sequential decoders for NASA applications. A 50-Mb/s hard-decision Fano decoder built in 1970 remains the fastest sequential decoder ever constructed and is used on a NASA space-station-to-ground telemetry link. The Communication Satellite Corporation has used convolutional codes with threshold decoding in several different applications. In 1975, a threshold decoder for an (8, 7, 47) self-orthogonal code was designed and built for the 64.0-kb/s INTELSAT single-channel-per-carrier system. To correct burst errors on a troposcatter channel, the Linkabit Corporation in 1973 designed and built an interleaved convolutional encoder with a Viterbi decoder for the U.S. Army Satellite Communications Agency. This system uses the (2, 1, 6) convolutional code with either hard- or soft-decision decoding. Furthermore, block codes are used for many automatic-repeat-request (ARQ) systems with required high-rate codes used for error detection only. However, convolutional codes, which combine error correction with error detection, can also be used in a hybrid ARQ system that requires lower code rates.

The coding schemes discussed so far have been classified by their applications into block and convolutional codes. However, the current trend is toward a general blurring of this separation. Error-correcting codes and their encoder-decoder pairs can be applied to today's complex digital systems because they can handle massive amounts of data flow. Applications of error-control coding to communication problems are diverse. Time-division multiplex access systems, packet switching networks, digital autopilots, digital radar-signal processing systems, cellular mobile communications systems, spread-spectrum systems against enemy jamming, cryptographic cipher systems, etc., all are systems that involve digital data processing. In every case, error-control techniques ensure proper performance of the system. Therefore, error-correction coding becomes an even more important subject because it permits communication links to function reliably in the presence of noise, distortion, and interference.





# Elements of Abstract Algebra for Coding

The purpose of this chapter is to present briefly the mathematical topics from abstract algebra and the theorems that are necessary for an understanding of coding theory. These mathematical topics are highly valued by those who use them for coding and are needed to comprehend the subsequent chapters. The structures of groups, rings, and Galois fields will play an important role in coding analysis as well as in encoder-decoder implementations. Since it is assumed that the reader already has some familiarity with these topics, our exposition is neither systematic nor complete.

## 2.1 Groups

A set is a well-defined collection or an aggregate of objects. A set  $S$  is comprised of elements. If  $a$  is one of these elements, we use the notation  $a \in S$ , whereas if  $a$  is definitely not in  $S$ , the notation is  $a \notin S$ . A set with no elements is called an empty set and is denoted by  $\emptyset$ . A set  $B$  is called a subset of a set  $A$  if and only if every element of  $B$  is in  $A$ . The fact that  $B$  is a subset of  $A$  is often denoted by  $B \subset A$  which reads “ $B$  is contained in  $A$ .” For any set of  $A$ ,  $A$  itself and  $\emptyset$  are both subsets even though they are trivial cases; however they are considered to be improper subsets of  $A$ . Any other subset of  $A$  is a proper subset of  $A$ .

A binary operation  $*$  on a set  $S$  is a rule that  $a * b$  is the element assigned to the pair  $(a, b)$  by  $*$ . A binary operation  $*$  on a set  $S$  is commutative if  $a * b = b * a$  for all  $a, b \in S$ . The operation  $*$  is associative if  $(a * b) * c = a * (b * c)$  for all  $a, b, c \in S$ .

A group  $G$  is a set together with an operation  $*$  on  $G$  such that the following axioms are satisfied:

1. For every  $a, b \in G$ ,  $c = a * b$  is in  $G$  (closure).
2. For every  $a, b, c \in G$ ,  $a * (b * c) = (a * b) * c$  (associative law).
3. There is an element  $e \in G$  such that for every  $a$  in  $G$ ,  $a * e = e * a = a$  (identity).
4. For every  $a \in G$ , there is an element  $a^{-1}$  in  $G$  such that  $a * a^{-1} = a^{-1} * a = e$  (inverse).

A group that has a finite number of elements is called a finite group, otherwise it is an infinite group. The number of elements in a finite group  $G$  is called the order of  $G$ . Groups with the additional property that  $a * b = b * a$  for all  $a, b \in G$  are called commutative groups or abelian groups. A group  $G$  is called an additive group when the operation is additive (+), while a group  $G$  is called a multiplicative group under the operation of multiplication ( $\cdot$ ). In an abelian group with an additive operation, we have  $a + (-a) = (-a) + a = 0$ , where the inverse element of  $a$  is written as  $-a$ . In this case, the identity of element  $e$  is 0. Under the multiplicative operation, the identity element  $e$  is 1 and the inverse element of  $a$  is written as  $a^{-1}$ , so that  $a \cdot a^{-1} = a^{-1} \cdot a = 1$ .

**Theorem 2.1** In every group  $G$ , the identity element  $e$  in  $G$  is unique, and the inverse of each element in  $G$  is also unique.

**Proof** Suppose that  $e$  and  $e'$  in  $G$  are two identity elements. Then  $e = e * e' = e'$ . Since  $e = e'$ , it is unique. Hence there is one and only one identity element. Next, suppose that  $a$  and  $a'$  are two inverses for an element  $b$ . Then

$$a = a * e = a * (b * a') = (a * b) * a' = e * a' = a'$$

which implies  $a = a'$ . Therefore there is only one inverse for  $b$ . Since  $b^{-1}b = bb^{-1} = 1$ ,  $b$  is an inverse for  $b^{-1}$ . Hence  $(b^{-1})^{-1} = b$  because inverses are unique.

**Example 2.1** Consider the finite group  $G = \{0, 1, 2, 3, 4\}$  under modulo-5 addition. This is given by Table 2.1.

**TABLE 2.1** Modulo-5 Addition

| + | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 3 | 4 |
| 1 | 1 | 2 | 3 | 4 | 0 |
| 2 | 2 | 3 | 4 | 0 | 1 |
| 3 | 3 | 4 | 0 | 1 | 2 |
| 4 | 4 | 0 | 1 | 2 | 3 |

**Example 2.2** Consider the group  $G = \{1, 2, 3, 4\}$  under modulo-5 multiplication. This is illustrated in Table 2.2.

TABLE 2.2 Modulo-5 Multiplication

| • | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | ① | 2 | 3 | 4 |
| 2 | 2 | 4 | ① | 3 |
| 3 | 3 | ① | 4 | 2 |
| 4 | 4 | 3 | 2 | ① |

The inverse of an element  $a$ , where  $a$  is one of the group elements, is obtained by running along the  $a$ -row until ① is encountered and then running up the column to identify the inverse of  $a$ . That is, the inverse of 1 is 1, of 2 is 3, of 3 is 2, and of 4 is 4.

Let  $H$  be a subset of elements of a group  $G$ . Then  $H$  is called a subgroup of  $G$  if  $H$  itself satisfies all the axioms for  $G$  when the same operation  $*$  is performed on  $H$ . To prove whether  $H$  is a subgroup, it is only necessary to check that  $a * b$  is in  $H$  whenever  $a, b \in H$  (closure), and the inverse  $a^{-1}$  of each  $a$  in  $H$  is also in  $H$ . If  $H$  is closed under the group operation, the identity must exist also and the associative law must hold in  $H$ .

Given a finite group  $G$  and a subgroup  $H$  of  $G$ , an important rectangular array known as the coset decomposition of  $G$  can be constructed as shown below. Let the elements of  $G$  be denoted by  $g_1, g_2, g_3, \dots$  and the elements of  $H$  by  $h_1, h_2, h_3, \dots$ . Now we construct the array as follows: The first row consists of the elements of the subgroup  $H$ , with the identity at the left and every other element of  $H$  appearing once and only once. Choose  $h_1$  to be the identity element. The first element in the second row is any element of  $G$  not appearing in the first row, and the rest of the elements in the second row are obtained by multiplying each subgroup element by this first element on the left. Construct similarly a third, fourth, and fifth row, each time choosing a previously unused group element in the first column. Stop when all the group elements appear somewhere in the array because  $G$  is a finite group. Thus the array for a coset decomposition of  $G$  is

$$\begin{array}{ccccccc}
 h_1 = 1 & h_2 & h_3 & h_4 & \cdots & h_n \\
 g_2 * h_1 = g_2 & g_2 * h_2 & g_2 * h_3 & g_2 * h_4 & \cdots & g_2 * h_n \\
 g_3 * h_1 = g_3 & g_3 * h_2 & g_3 * h_3 & g_3 * h_4 & \cdots & g_3 * h_n \\
 \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\
 g_m * h_1 = g_m & g_m * h_2 & g_m * h_3 & g_m * h_4 & \cdots & g_m * h_n
 \end{array}$$

The element appearing in the first column is known as a coset leader. Each row in this array is called a left coset. If the coset decomposition is defined with the element of  $H$  multiplied on the right, the rows are



known as right cosets. However, when the group is abelian, the left coset is simply called a coset because every left coset is trivially a right coset. From the rectangular structure of the coset decomposition of  $G$ , it is easily seen that the number of elements in  $H$  times the number of cosets is equal to the number of elements in  $G$ . That is, the order of  $H$  divides the order of  $G$  such that

$$\text{Order of } H \times \text{number of cosets of } G \text{ with respect to } H = \text{order of } G$$

Let us show that an element cannot appear twice in the same coset or in two different cosets. Suppose that two elements in the same coset are equal such as  $g_i * h_j = g_i * h_k$ . Multiplying each by  $g_i^{-1}$  gives  $h_j = h_k$ . This is a contradiction because each element in  $H$  appears only once in the first row in the array. Next, suppose that two elements in two different cosets are equal, that is,  $g_i * h_j = g_k * h_l$  for  $k < i$ . Multiplying the right side by  $h_j^{-1}$  gives  $g_i = g_k * h_l * h_j^{-1}$ . Since  $h_l * h_j^{-1}$  is in the subgroup  $H$ , this indicates that  $g_i$  is in the  $k$ th coset. This contradicts the rule of array construction that coset leaders must be previously unused. Thus we have proved that every element of  $G$  appears once and only once in a coset decomposition of  $G$ .

A subgroup  $H$  of a group  $G$  is called normal if, for any element  $h$  of  $H$  and any element  $g$  of  $G$ ,  $g^{-1} * h * g$  is in  $H$ . If a subgroup  $H$  of a group  $G$  is normal, it is possible to define an operation on cosets that forms a new group for which the cosets are the elements. This group is called the factor group and is denoted by  $G/H$ . In other words, if  $H$  is a normal subgroup of a group  $G$ , the group of cosets of  $H$  under the induced operation is the factor group  $G/H$  of  $G \bmod H$ . The cosets are also known as residue classes of  $G \bmod H$ .

If  $G$  is a group and  $a \in G$ , then  $H = \{a^q \mid q \in \mathbf{Z}\}$  is a subgroup of  $G$ . This group is the cyclic subgroup of  $G$  generated by some fixed element  $a$ . Suppose that  $G$  is a cyclic group with  $q$  elements and generated by  $a$ . Let  $b \in G$  and  $b = a^s$ . Then  $b$  generates a cyclic subgroup  $H$  of  $G$  containing  $q/d$  elements, where  $d$  is the greatest common divisor of  $q$  and  $s$ , denoted by  $\text{GCD}(q, s)$ . Also, given a group  $G$  and an element  $a \in G$ , if  $G = \{a^q \mid q \in \mathbf{Z}\}$ , group  $G$  is cyclic. Thus, a cyclic group is a finite group in which every element can be written as a power of some element  $a$ . Every cyclic group can then be expressed in the form  $G = \{a^0, a^1, a^2, \dots, a^{q-1}\}$ , where  $q$  is the order of  $G$ ,  $a^0 = e$  is the identity element,  $a$  is a generator of  $G$ , and the inverse of  $a^i$  is  $a^{q-i}$ . It is also true that  $a^q = a^0$ . Summarizing, a cyclic subgroup of  $G$  containing  $a$  must contain all elements  $a^n$  (or  $na$  for an additive group) for all  $n \in \mathbf{Z}$ . In fact, every cyclic group is abelian. One typical cyclic group of  $q$  elements under addition modulo- $q$  is the cyclic group given by  $\mathbf{Z}_q = \{0, 1, 2, \dots, q-1\}$ . Consider  $\mathbf{Z}_{12}$  which has a generator  $a = 1$ . Since

$GCD(3, 12) = 3$ ,  $3 = 3 \cdot 1$  generates a subgroup of  $12/3 = 4$  elements, that is,  $\mathbf{Z}_4 = \{0, 3, 6, 9\}$ .

An important group is a permutation group. A permutation of a set is a rearrangement or mapping of the elements of the set. For a given set  $S$ , let  $\phi(S)$  be the set of mapping  $a$  of  $S$  onto itself such that each  $a$  gives a one-to-one correspondence of  $S$  with  $S$  itself. Then  $\phi(S)$  forms a group under the composition of mappings. This group is called the systematic group on  $S$ , and a subgroup of this group is called a permutation group on  $S$ . A mapping  $\phi$  of a group  $G$  into a group  $G'$  is called a homomorphism if  $\phi(ab) = \phi(a)\phi(b)$  holds for arbitrary  $a, b \in G$ . In this case,  $\{a \in G \mid \phi(a) = 1\}$  is known as the kernel of  $\phi$ . If the kernel consists only of the identity, then the homomorphism is called an injection. On the other hand, if  $\phi$  is a homomorphism of  $G$  onto  $G'$ , we say that  $\phi$  is a surjection. If a homomorphism  $\phi$  is injective and at the same time surjective, then we say that  $\phi$  is an isomorphism. If there is an isomorphism of a group  $G$  onto a group  $G'$ , then it is said that  $G$  is isomorphic to  $G'$ . An isomorphism of a group  $G$  to itself is called an automorphism of  $G$ . The set of all automorphisms of  $G$  is called the automorphism group of  $G$ . The set of all inner automorphisms of  $G$  forms a normal subgroup of the automorphism group of  $G$ . Two subsets  $H$  and  $K$  of  $G$  are said to be conjugate to each other if there is an element  $a$  of  $G$  such that  $a^{-1}Ha = K$ . A subgroup  $N$  of  $G$  is called a normal subgroup if  $N$  itself is the only conjugate of  $N$ . In this case,  $aN = Na$  for every  $a \in N$ , which is called the residue class of modulo- $N$ . Cayley's theorem states that every group is isomorphic to some group consisting of permutations under permutation multiplication. This is a nice and intriguing result, although we shall have no important use for it. However, the theorem is a classic in group theory and appears in almost all algebra texts.

Finally, consider the direct products of a finite number of groups. Let  $G_1, G_2, \dots, G_n$  be groups. The product set  $\{G_1, G_2, \dots, G_n\}$  is the set of all ordered  $n$ -tuples  $(a_1, a_2, \dots, a_n)$ , where  $a_i \in G_i$ . We define a multiplication by  $(a_1, a_2, \dots, a_n)(b_1, b_2, \dots, b_n) = (a_1b_1, a_2b_2, \dots, a_nb_n)$ . Then this product set forms a group where the identity is  $(1, 1, \dots, 1)$  and  $(a_1, a_2, \dots, a_n)^{-1} = (a_1^{-1}, a_2^{-1}, \dots, a_n^{-1})$ .

## 2.2 Rings

Another important algebraic structure is the ring. A ring  $R$  is a set that has two operations defined on addition and multiplication, such that the following axioms are satisfied:

1. The set  $R$  is an abelian group under addition.
2. For any  $a, b \in R$ , the product  $ab$  is in  $R$  (closure).

3. For any  $a, b, c \in R$ ,  $a(bc) = (ab)c$  (associative law).
4. For any  $a, b, c \in R$ ,  $a(b + c) = ab + ac$  and  $(b + c)a = ba + ca$  (distributive law).

The additive operation is always commutative in a ring, but the multiplicative operation need not be. A commutative ring is one in which multiplication is commutative such that  $ab = ba$  for all  $a, b \in R$ . A ring  $R$  with a multiplicative identity  $1$  such that  $1a = a1 = a$  for all  $a \in R$  is a ring with unity. Hence, a multiplicative identity in a ring is unity.

**Theorem 2.2** For any two elements  $a, b$  in a ring  $R$ , it follows that  $a0 = 0a = 0$ ,  $a(-b) = (-a)b = -(ab)$ , and  $(-a)(-b) = ab$ .

**Proof** For any  $a \in R$ ,  $a0 = a(0 + 0) = a0 + a0$  from axiom 4. Adding  $-a0$  to both sides gives  $0 = a0 + (-a0) = a0 + a0 + (-a0) = a0$ . Thus,  $a0 = 0$  by the cancellation law for the additive group. Similarly,  $0a = 0$ . Next,  $0 = a0 = a(b - b) = ab + a(-b)$ , so  $a(-b) = -(ab)$ . Likewise  $(-a)b + ab = (-a + a)b = 0b = 0$ , so  $(-a)b = -(ab)$ . Lastly  $(-a)(-b) = -([a(-b)]) = -(-(ab)) = ab$ .

It is evident that the set  $\{0\}$ , with  $0 + 0 = 0$  and  $(0)(0) = 0$ , gives a ring. Hence  $0$  acts as a multiplicative as well as an additive identity. However, there is no multiplicative inverse for  $0$ . Every element in a ring has the inverse under additive operation. Under the multiplicative operation, there need not be any inverses, but in a ring with identity, inverses may exist. A multiplicative inverse of an element  $a$  in a ring with unity is an element  $a^{-1} \in R$  such that  $aa^{-1} = a^{-1}a = 1$ . Just as for groups, a multiplicative inverse for an element  $a$  in  $R$  is unique.

### 2.2.1 Integral domains

An integral domain is a commutative ring with unity containing no divisors equal to zero. An element  $e$  of a ring  $R$  is called an identity if  $e$  is the identity of  $R$  under multiplication and if  $e \neq 0$ . If a ring  $R$  has an identity, then it is unique and is denoted by  $1$ . An element  $a$  of  $R$  is called a zero divisor if there is a nonzero element  $b$  such that  $ab = 0$ . An integral domain is a ring with unity (or identity) in which every nonzero element is not a zero divisor. An element  $a$  is called nilpotent if there is a natural number  $n$  such that  $a^n = 0$ . This  $a$  is a zero divisor. If  $F$  is an additive group for a positive integer  $n$  and an element  $a \in F$ ,  $na$  denotes the  $n$ -tuple sum  $a + a + \cdots + a$  of  $a$ . For a ring  $R$ , if a positive integer  $n$  exists such that  $na = 0$  for all  $a \in R$ , then the least such positive integer is the characteristic of the ring. If no such positive integer exists, then  $R$  is of characteristic 0. For example, an integral ring  $\mathbb{Z}_n = \{0, 1, 2, \dots, n - 1\} \bmod n$  is of characteristic  $n$ .

**Theorem 2.3** If  $R$  is a ring with unity  $1$ , then  $R$  has the characteristic  $n > 0$  if and only if  $n$  is the smallest positive integer such that  $n1 = 0$ .

**Proof** If  $R$  has the characteristic  $n > 0$ , then  $na = 0$  for all  $a \in R$ , so in particular  $n1 = 0$ . Conversely, suppose that  $n$  is a positive integer such that  $n1 = 0$ . Then for any  $a \in R$ , we have

$$na = a + a + \cdots + a = a(1 + 1 + \cdots + 1) = a(n1) = a0 = 0$$

## 2.2.2 Ideals, residue classes, and the quotient ring

Let  $R$  be a ring. Consider a partition of  $R$  into disjoint subsets. These subsets themselves can be viewed as elements in a ring where both addition and multiplication of the subsets are the induced operations from  $R$ . A subring  $I$  of a ring  $R$  satisfying  $rI \subseteq I$  and  $Ir \subseteq I$  for all  $r \in R$  is called an ideal of  $R$ . A subring  $I$  of  $R$  satisfying  $rI \subseteq I$  for all  $r \in R$  is a left ideal of  $R$ , and one satisfying  $Ir \subseteq I$  for all  $r \in R$  is a right ideal of  $R$ . Thus we can see that for any element  $a$  of  $I$  and any element  $r$  of  $R$ ,  $ar$  and  $ra$  are in an ideal  $I$ . Let us reiterate for emphasis that *an ideal is to a ring as a normal subgroup is to a group*.

If  $I$  is an ideal in a ring  $R$ , then the ring of cosets  $r + I$  is called the quotient ring, or factor ring, or residue class ring of modulo- $I$ , and is denoted by  $R/I$ . The cosets are called residue classes of modulo- $I$ . Since an ideal is a subgroup, the cosets can be formed. In this case the cosets are residue classes. The ideal forms the first row of an array, with the 0 element at the leftmost. Then any ring element not in the ideal can be chosen as the coset leader of the first residue class, and the rest of the cosets are formed by adding the leader to each of the elements of the ideal as follows:

$$\begin{array}{cccccc} 0 = a_1, & a_2, & a_3, & a_4, & a_5, & \dots \\ r_1 = r_1 + a_1, & r_1 + a_2, & r_1 + a_3, & r_1 + a_4, & r_1 + a_5, & \dots \\ r_2 = r_2 + a_1, & r_2 + a_2, & r_2 + a_3, & r_2 + a_4, & r_2 + a_5, & \dots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \end{array}$$

The first element in each row is a previously unused element of  $R$ . Since a ring  $R$  is an abelian group under addition, an ideal  $I$  is a normal subgroup such that the collection of cosets  $r + I$  for  $r \in R$  is a ring. Addition of the residue classes forms a factor group. Thus the residue classes of a ring with respect to an ideal form a ring called the residue class ring. It is also possible to define multiplication of residue classes.

## 2.2.3 Integer ring and division algorithm

Under the two operations of addition and multiplication, the integers (positive, negative, and zero) form an integer ring  $\mathbf{Z}$ . Within the ring of integers, division is not always possible. If  $r$ ,  $s$ , and  $a$  are integers



and  $ra = s$ , then it is said that  $s$  is divisible by  $r$  or that  $r$  divides  $s$  or that  $r$  is a factor of  $s$ . Whenever  $r$  divides  $s$  and  $s$  divides  $r$ , it follows that  $r = \pm s$ . This is because  $r = sa$  and  $s = rb$  for some integers  $a$  and  $b$ . Therefore  $r = rab$ , so  $ab$  must equal 1. Thus  $a$  and  $b$  must both be either 1 or  $-1$ , and  $a = b^{-1}$ .

An integer  $p \geq 1$  that is divisible only by  $\pm p$  or  $\pm 1$  is called a prime integer. A positive integer that is not prime is called composite. The greatest common divisor of two integers  $r$  and  $s$ , denoted by  $GCD(r, s)$ , is the largest positive integer that divides both of them. Two integers are said to be relatively prime if their greatest common divisor is 1. The least common multiple of two integers  $r$  and  $s$ , denoted by  $LCM(r, s)$ , is the smallest positive integer that is divisible by both of them. Since the integer ring is an integral domain, it is possible to execute cancellation and division with remainder.

**Theorem 2.4 (Division Algorithm)** For every pair of integers  $s$  and  $d$ , there is a unique pair of integers  $q$ , the quotient, and  $r$ , the remainder, such that

$$s = dq + r \quad \text{where } 0 \leq r < |d|$$

Given two positive integers  $r$  and  $s$ , their greatest common divisor can be computed by an iterative application of the division algorithm. This procedure is known as the euclidean division algorithm. When  $r < s$ , the euclidean algorithm takes the following iterative steps:

$$s = q_1 r + r_1$$

$$r = q_2 r_1 + r_2$$

$$r_1 = q_3 r_2 + r_3$$

$$\vdots$$

$$r_{n-2} = q_n r_{n-1} + r_n$$

$$r_{n-1} = q_{n+1} r_n$$

where the process stops when a remainder of zero is obtained. The last nonzero remainder  $r_n$  is the greatest common divisor, that is,  $r_n = GCD(r, s)$ . From the euclidean division algorithm, some very important algebraic properties of integers can be obtained. For any integers  $r$  and  $s$  there exist integers  $a$  and  $b$  such that

$$GCD(r, s) = ar + bs$$

An illustration of how the greatest common divisor is found using the euclidean algorithm will be shown by the following example.

**Example 2.3** Find the greatest common divisor of two integers 22,471 and 3266.

$$22,471 = (3266)6 + 2875$$

$$3266 = (2875)1 + 391$$

$$2875 = (391)7 + 138$$

$$391 = (138)2 + 115$$

$$138 = (115)1 + 23$$

$$115 = (23)5 + 0$$

Thus,  $GCD(22,471, 3266) = 23$ . Now we can express 23 as a linear combination of 22,471 and 3266 by starting at the bottom of the above sequence of equations as shown below.

$$\begin{aligned} 23 &= 138 - (115)1 \\ &= 138 - (391 - 138 \times 2) = (138)3 - 391 \\ &= (2875 - 391 \times 7)3 - 391 = (2875)3 - (391)22 \\ &= (2875)3 - (3266 - 2875 \times 1)22 = (2875)25 - (3266)22 \\ &= (22,471 - 3266 \times 6)25 - (3266)22 \\ &= (22,471)25 - (3266)172 \end{aligned}$$

Thus we have found that  $GCD(22,471, 3266)$  can be expressed as a linear combination of 22,471 and 3266 with coefficients from the integer ring. That is,  $GCD(22,471, 3266) = (22,471)25 - (3266)172$ .

For  $r, s \in \mathbf{Z}$ , we say that  $r$  is congruent to  $s \bmod d$ , denoted by  $r \equiv s \pmod{d}$ , if  $r - s$  is evenly divisible by  $d$ , so that  $r - s = dt$  for some  $t \in \mathbf{Z}$ . For example,  $17 \equiv 33 \pmod{8}$  because  $17 - 33 = 8(-2)$ . This means that  $r$  and  $s$  have the same remainder when divided by  $d$ , but  $r$  is not necessarily smaller than  $d$ .

## 2.3 Fields

A field  $F$  has the most powerful algebraic structure. The sets of all rational numbers, all real numbers, and all complex numbers are fields under the usual operations of addition and multiplication. They are called the rational number field, the real number field, and the complex number field, respectively, and are denoted by  $\mathbf{Q}$ ,  $\mathbf{R}$ , and  $\mathbf{C}$ , respectively. These fields all have an infinite number of elements. However, we are interested in fields with a finite number of elements.

A field  $F$  is defined as a set that has two operations, addition and multiplication, such that the following axioms are satisfied.

1. The set is an abelian group under addition.
2. The field is closed under multiplication, and the set of nonzero elements is an abelian group under multiplication.
3. The distribution law  $(a + b)c = ac + bc$  holds for all  $a, b, c$  in the field.

Under the additive operation, the identity element 0 exists and the additive inverse of  $a$  is denoted by  $-a$ . Under the multiplicative operation, the identity element 1 exists and the multiplicative inverse of  $a$  is denoted by  $a^{-1}$ . Moreover, there exist subtraction  $(a - b) = a + (-b)$  and division  $(a/b) = b^{-1}a$ . A field  $F$  with  $q$  elements is called a finite field, or a Galois field, and is denoted by  $GF(q)$ . The minimum number of elements a field can have is two, i.e., 0 and 1. This is called the binary field, which is denoted by  $GF(2)$ .

## 2.4 Polynomial Rings

Let  $R$  be a ring. A polynomial  $f(x)$  with coefficients in  $R$  is expressed by a formal sum

$$f(x) = f_0 + f_1x + \cdots + f_{n-2}x^{n-2} + f_{n-1}x^{n-1}$$

where  $x$  is an indeterminate, and the indices and exponents are integers. The degree of  $f(x)$  is the largest power of  $x$  in a term with a non-zero coefficient  $f_{n-1}$ . A polynomial is called monic if the coefficient of the highest power of  $x$  is 1.

For each field  $F$ , there is a ring  $F[x]$  called the ring of polynomials over  $F$ . A polynomial ring is analogous in many ways to an integer ring. The set of all polynomials over the field  $F$  forms a ring  $F[x]$  under the usual addition and multiplication of polynomials.

The sum of two polynomials  $f(x)$  and  $g(x)$  in  $F[x]$  is another polynomial in  $F[x]$  defined by

$$f(x) + g(x) = \sum_{i=0}^{n-1} (f_i + g_i)x^i \quad \text{degree of } g(x) \leq \text{degree of } f(x)$$

where the degree of the sum is not greater than the larger of these two degrees. The product of two polynomials in  $F[x]$  is another polynomial in  $F[x]$  such that

$$f(x)g(x) = \sum_i \left( \sum_{j=0}^i f_j g_{i-j} \right) x^i$$

The degree of a product of two polynomials is equal to the sum of the degrees of the two factors.

Within a ring of polynomials, subtraction is always possible, but division is not always possible. If there is a polynomial  $a(x)$  such that  $r(x)a(x) = s(x)$ , it is said that the polynomial  $s(x)$  is divisible by the polynomial  $r(x)$  or that  $r(x)$  divides  $s(x)$  or that  $r(x)$  is a factor of  $s(x)$ . A nonzero polynomial  $p(x)$  that is divisible only by  $p(x)$  is called an irreducible polynomial. A monic irreducible polynomial is called a primitive polynomial. Whenever  $r(x)$  both divides  $s(x)$  and is divisible by  $s(x)$ , then  $r(x) = s(x)a(x)$  and  $s(x) = r(x)b(x)$ . Thus  $r(x) = r(x)b(x)a(x)$  under the restriction that  $a(x)$  and  $b(x)$  must have zero degree. Therefore,  $r(x) = \alpha s(x)$ , where  $\alpha$  is an element of the field  $F$ .

The greatest common divisor of two polynomials  $r(x)$  and  $s(x)$ , denoted by  $GCD[r(x), s(x)]$ , is the monic polynomial of largest degree that divides both of them. Two polynomials are said to be relatively prime if their greatest common divisor is 1. The greatest common divisor of two polynomials  $r(x)$  and  $s(x)$  can always be expressed in the form

$$GCD[r(x), s(x)] = a(x)r(x) + b(x)s(x)$$

The least common multiple of two polynomials  $r(x)$  and  $s(x)$ , denoted by  $LCM[r(x), s(x)]$ , is the monic polynomial of smallest degree divisible by both of them.

**Theorem 2.5 (Division Algorithm for Polynomials)** For every polynomial  $f(x)$  and nonzero polynomial  $g(x)$ , there is a unique pair of polynomials— $q(x)$ , the quotient polynomial, and  $s(x)$ , the remainder polynomial—such that  $f(x) = g(x)q(x) + s(x)$  and the degree of  $s(x)$  is less than the degree of  $g(x)$ .

The remainder  $s(x)$  is called the residue of  $f(x)$  when divided by  $g(x)$ . The congruence  $s(x) \equiv f(x) \pmod{g(x)}$  means that  $s(x)$  and  $f(x)$  have the same remainder under division by  $g(x)$  but that the degree of  $s(x)$  is not necessarily smaller than that of  $g(x)$ .

**Theorem 2.6** An element  $a \in F$  is a root (or a zero) of  $f(x) \in F[x]$  if and only if  $x - a$  is a factor of  $f(x)$  in  $F[x]$ .

**Proof** Suppose that  $f(a) = 0$  for  $a \in F$ . By Theorem 2.5, there exists  $q(x), s(x) \in F[x]$  such that

$$f(x) = (x - a)q(x) + s(x)$$

where the degree of  $s(x)$  is less than 1. Then we must have  $s(x) = c$  for  $c \in F$ , so  $f(x) = (x - a)q(x) + c$ . Since  $f(a) = 0$  for  $a \in F$ ,  $0 = f(a) = 0q(a) + c$ . Hence it must be that  $c = 0$ . Thus  $f(x) = (x - a)q(x)$ , so  $x - a$  is a factor of  $f(x)$ . Conversely, if  $x - a$  is a factor of  $f(x)$  in  $F[x]$ , where  $a \in F$ , then we have  $f(a) = 0$ ,  $q(a) = 0$ .

**Theorem 2.7 (Euclidean Division Algorithm for Polynomials)** Given two polynomials  $r(x)$  and  $s(x)$  of  $F[x]$ , their greatest common divisor can be computed by an



iterative application of the division algorithm. If the degree of  $s(x)$  is greater than the degree of  $r(x)$ , the computation for  $GCD[r(x), s(x)]$  is

$$s(x) = q_1(x)r(x) + r_1(x)$$

$$r(x) = q_2(x)r_1(x) + r_2(x)$$

$$r_1(x) = q_3(x)r_2(x) + r_3(x)$$

$$\vdots$$

$$r_{n-1}(x) = q_{n+1}(x)r_n(x)$$

where the iterative process stops when a remainder of zero is obtained. Then the greatest common divisor of  $r(x)$  and  $s(x)$  is  $r_n(x) = GCD[r(x), s(x)]$ .

In Theorem 2.6, an important result is obtained by assuming a first-degree divisor  $x - a$  in the division algorithm. Since  $s(x)$  is of one degree less than the degree of the divisor, it must have degree 0. That is,  $s(x) = c$  for  $c \in F$ . Thus a polynomial of degree 0 is a field element and therefore has an inverse, but no nonzero polynomial of higher degree has an inverse. Substitution of  $a$  for  $x$  then shows that  $s(x) = c = f(a)$ . This is the remainder theorem. If  $f(a) = 0$ ,  $a$  is a root (or a zero) of  $f(x)$ . Then  $s(x) = c = 0$  and  $x - a$  is a factor of  $f(x)$ . This result is known as the factor theorem. Thus, for each distinct root of  $f(x)$  there is a distinct first-degree factor. Since the degree of a product of polynomials is the sum of the degrees of the factors, the degree of  $f(x)$  is at least as great as the number of roots of  $f(x)$ .

Suppose that we have  $F[x]$ , the ring of polynomials over the field  $F$ . If we choose any monic polynomial  $p(x)$  from  $F[x]$ , we can define the quotient ring by using  $p(x)$  as a modulus. The ring of polynomials modulo- $p(x)$  is the set of all polynomials with degree smaller than that of  $p(x)$  together with the results of polynomial addition and polynomial multiplication modulo- $p(x)$ . This ring is conventionally denoted by  $F[x]/p(x)$ .

**Example 2.4** Determine the elements of  $GF(4)$  over  $GF(2)$  generated by the primitive polynomial  $p(x) = 1 + x + x^2$ . The field elements are represented by the set of polynomials  $\{0, 1, \alpha, \alpha^2\} = \{0, 1, x, 1 + x\}$ , where  $\alpha$  is the primitive element of  $GF(4)$ . Notice that addition or multiplication in  $GF(4)$  is not modulo-4 addition or multiplication (see Table 2.3). Thus we can construct  $GF(4)$  over  $GF(2)$  by using the primitive polynomial  $p(x) = 1 + x + x^2$  as shown above.

The ring of polynomials modulo- $p(x)$  is a field if and only if  $p(x)$  is a primitive polynomial. Therefore, whenever a primitive polynomial  $p(x)$  of degree  $m$  over  $GF(q)$  is found, we can construct a Galois field  $GF(q^m)$  with  $q^m$  elements. The elements are represented by such  $q^m$  polynomials over  $GF(q)$  of degree less than  $m$ .

TABLE 2.3 Arithmetic Tables for  $GF(4)$  Structure\*

| +          | 0          | 1          | $\alpha$   | $\alpha^2$ | ·          | 0 | 1          | $\alpha$   | $\alpha^2$ |
|------------|------------|------------|------------|------------|------------|---|------------|------------|------------|
| 0          | 0          | 1          | $\alpha$   | $\alpha^2$ | 0          | 0 | 0          | 0          | 0          |
| 1          | 1          | 0          | $\alpha^2$ | $\alpha$   | 1          | 0 | 1          | $\alpha$   | $\alpha^2$ |
| $\alpha$   | $\alpha$   | $\alpha^2$ | 0          | 1          | $\alpha$   | 0 | $\alpha$   | $\alpha^2$ | 1          |
| $\alpha^2$ | $\alpha^2$ | $\alpha$   | 1          | 0          | $\alpha^2$ | 0 | $\alpha^2$ | 1          | $\alpha$   |

| + | 0 | 1 | 2 | 3 | · | 0 | 1 | 2 | 3 |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 3 | 0 | 0 | 0 | 0 | 0 |
| 1 | 1 | 0 | 3 | 2 | 1 | 0 | 1 | 2 | 3 |
| 2 | 2 | 3 | 0 | 1 | 2 | 0 | 2 | 3 | 1 |
| 3 | 3 | 2 | 1 | 0 | 3 | 0 | 3 | 1 | 2 |

\* $GF(4) = \{0, 1, 2, 3\}$  for  $\alpha = 2$  and  $\alpha^2 = 3$ .

## 2.5 Galois Fields

In this section, the aim is to determine the structure of all finite fields. Let  $E$  be an extension field of degree  $m$  over a finite field  $F$ . If  $F$  has  $q$  elements, then  $E$  has  $q^m$  elements. The original field  $F$  is called the ground field. Thus, the field formed by taking polynomials over a finite field  $F$  modulo  $p(x)$ , where  $p(x)$  is an irreducible polynomial of degree  $m$ , is called an extension field  $E$  of degree  $m$  over  $F$ . For any prime power  $q$  and positive integer  $m$ ,  $GF(q)$  is a subfield of  $GF(q^m)$  and  $GF(q^m)$  is an extension field of  $GF(q)$ .

Every Galois field  $GF(q)$  has at least one primitive element. A primitive element of  $GF(q)$  is an element  $\alpha$  such that every field element except zero can be expressed as a power of  $\alpha$ . Primitive elements are useful for constructing fields because we can obtain the field elements by multiplying powers of the primitive element  $\alpha$ . Since  $p(\alpha) = 0$ ,  $\alpha$  is a root of  $p(x)$ , and it is said that the extension field is obtained by adjoining a root of  $p(x)$  to the ground field. In addition,  $GF(q^n)$  is not a subfield of  $GF(q^m)$  if  $n$  does not divide  $m$ . The degree of the minimal polynomial over  $GF(q)$  of any element of  $GF(q^m)$  is a divisor of  $m$ .

An irreducible polynomial  $p(x)$  of degree  $m$  is said to be primitive if and only if it divides  $x^n + 1$  for no  $n$  less than  $2^m - 1$ . In fact, every binary primitive polynomial  $p(x)$  of degree  $m$  is a factor of  $x^{2^m-1} + 1$ . Primitive polynomials of every degree exist over every Galois field, and every Galois field has a primitive element  $\alpha$ . Table 2.4 gives a list of primitive polynomials over  $GF(2)$ .

The Galois field  $GF(q)$  is an extension field of the subfield  $GF(p)$  where  $p$  is a prime. For example, the binary field  $GF(2)$  is the subfield of the Galois field  $GF(2^m)$  of  $q = 2^m$  elements ( $m > 1$ ). The subfield  $GF(2)$  is also called the ground field of  $GF(2^m)$ . The subset  $\{0, 1\}$  forms

TABLE 2.4 Primitive Polynomials over  $GF(2)$

| $m$ | $p(x)$                          | $m$ | $p(x)$                          |
|-----|---------------------------------|-----|---------------------------------|
| 2   | $x^2 + x + 1$                   | 15  | $x^{15} + x + 1$                |
| 3   | $x^3 + x + 1$                   | 16  | $x^{16} + x^{12} + x^3 + x + 1$ |
| 4   | $x^4 + x + 1$                   | 17  | $x^{17} + x^3 + 1$              |
| 5   | $x^5 + x^2 + 1$                 | 18  | $x^{18} + x^7 + 1$              |
| 6   | $x^6 + x + 1$                   | 19  | $x^{19} + x^5 + x^2 + x + 1$    |
| 7   | $x^7 + x^3 + 1$                 | 20  | $x^{20} + x^3 + 1$              |
| 8   | $x^8 + x^4 + x^3 + x^2 + 1$     | 21  | $x^{21} + x^2 + 1$              |
| 9   | $x^9 + x^4 + 1$                 | 22  | $x^{22} + x + 1$                |
| 10  | $x^{10} + x^3 + 1$              | 23  | $x^{23} + x^5 + 1$              |
| 11  | $x^{11} + x^2 + 1$              | 24  | $x^{24} + x^7 + x^2 + x + 1$    |
| 12  | $x^{12} + x^6 + x^4 + x + 1$    | 25  | $x^{25} + x^3 + 1$              |
| 13  | $x^{13} + x^4 + x^3 + x + 1$    | 26  | $x^{26} + x^6 + x^2 + x + 1$    |
| 14  | $x^{14} + x^{10} + x^6 + x + 1$ | 27  | $x^{27} + x^5 + x^2 + x + 1$    |

a subfield  $GF(2)$  of  $GF(2^m)$ . The set  $GF(2^m) = \{0, 1, \alpha, \alpha^2, \dots, \alpha^{2^m-1}\}$  is a Galois field of  $2^m$  elements. The addition and multiplication defined on  $GF(2^m)$  imply modulo-2 addition and multiplication. The number of elements in the smallest subfield of  $GF(q)$  is called the characteristic of  $GF(q)$ . Each Galois field contains a unique smallest subfield, which has a prime number of elements. Here the characteristic of every Galois field is a prime number. Thus,  $GF(p^m)$  is a field of characteristic  $p$  for any choice of  $m$ . As an example, the characteristic of  $GF(2^m)$  is 2.

**Theorem 2.8** In a finite field  $GF(q)$  of characteristic  $p$ ,

$$(\alpha + \beta)^p = \alpha^p + \beta^p$$

for any two elements  $\alpha$  and  $\beta$  in  $GF(q)$ .

**Proof** Expansion of  $(\alpha + \beta)^p$  by the binomial theorem yields

$$(\alpha + \beta)^p = \sum_{i=0}^p \binom{p}{i} \alpha^i (\beta)^{p-i}$$

where the binomial coefficient

$$\binom{p}{i} = \frac{p!}{i!(p-i)!} = \frac{p(p-1)!}{i!(p-i)!}$$

where  $i$  is an integer and  $p$  is a prime. Therefore, the denominator divides  $(p-1)!$ , and  $\binom{p}{i}$  is a multiple of  $p$ . That is, the denominator does not contain  $p$ . Since  $\binom{p}{i} \equiv 0 \pmod{p}$  in  $GF(q)$ , we have

$$(\alpha + \beta)^p = \alpha^p + \beta^p$$

This fact can be generalized easily to the following lemma.

**Lemma 2.8.1** Assume that the characteristic  $p$  of a finite field  $GF(q)$  is a prime number. If  $\beta_1, \beta_2, \dots, \beta_n \in GF(q)$  and if  $q$  is a power of  $p$ , i.e.,  $q = p^m$ , then

$$(\beta_1 + \beta_2 + \dots + \beta_n)^q = \beta_1^q + \beta_2^q + \dots + \beta_n^q$$

**Theorem 2.9** Let  $\beta$  be a nonzero element of a finite field  $GF(q)$ . Then  $\beta^{q-1} = 1$ .

**Proof** Let  $\beta_1, \beta_2, \dots, \beta_n$  denote the  $q - 1$  nonzero elements of  $GF(q)$ . The  $x^{q-1} - 1 = (x - \beta_1)(x - \beta_2) \dots (x - \beta_{q-1})$ . Thus the set of nonzero elements of  $GF(q)$  is a finite group under the operation of multiplication. Let  $n$  be the order of  $\beta$ . That is,  $\beta^n = 1$ . From  $\beta^{q-1} = (\beta^n)^{(q-1)/n}$ , we have  $\beta^{q-1} = 1^{(q-1)/n} = 1$  because  $n$  divides  $q - 1$  (see Theorem 2.10). Thus  $\beta$  is a root of  $x^{q-1} + 1$ .

Since the primitive polynomial  $p(x)$  of degree  $m$  is a factor of  $x^{2^m-1} + 1$ ,  $x^{2^m-1} + 1 = q(x)p(x)$ . Replacing  $x$  by  $\alpha$  [a root of  $p(x)$ ], we have  $\alpha^{2^m-1} + 1 = q(\alpha)p(\alpha)$ . Since  $p(\alpha) = 0$ , it follows that  $\alpha^{2^m-1} + 1 = q(\alpha) \cdot 0 = 0$ . Adding 1 to both sides of this equation, it follows that  $\alpha^{2^m-1} = 1$ .

**Theorem 2.10** Let  $\beta$  be a nonzero element in a Galois field  $GF(q)$ . Let  $n$  be the order of  $\beta$ . Then  $n$  divides  $q - 1$ .

**Proof** Suppose that  $n$  does not divide  $q - 1$ . Dividing  $q - 1$  by  $n$ , yields  $q - 1 = kn + r$ , where  $0 < r < n$ . Hence,  $\beta^{q-1} = \beta^{kn+r} = (\beta^n)^k \cdot \beta^r$ . Since  $\beta^{q-1} = 1$  and  $\beta^n = 1$ , then it must be that  $\beta^r = 1$ . However, this is impossible because  $0 < r < n$  and  $n$  is the smallest integer such that  $\beta^n = 1$ . Therefore,  $n$  must divide  $q - 1$ .

Constructing the Galois field  $GF(2^m)$  requires a primitive polynomial  $p(x)$  of degree  $m$  and a primitive element  $\alpha$ , which is a root of  $p(x)$ . The powers of  $\alpha$  generate all the nonzero elements of  $GF(2^m)$ . Let  $n$  be the order of  $\alpha^{2^\lambda}$  where  $\lambda > 0$ . Then  $(\alpha^{2^\lambda})^n = 1$ . From Theorem 2.10,  $2^m - 1 = kn$ . Since  $\alpha$  is a primitive element of  $GF(2^m)$ , its order is  $2^m - 1$ ; thus  $\alpha^{2^m-1} = 1$  and  $2^\lambda n = 2^m - 1$ . Since  $2^\lambda$  and  $2^m - 1$  are relatively prime,  $n$  divides  $2^m - 1$ . Thus,  $\alpha^{2^\lambda}$  is also a primitive element of  $GF(2^m)$ .

Consider a polynomial  $f(x) = f_0 + f_1x + \dots + f_nx^n$  with the coefficients  $f_i = 0$  or  $1$ ,  $0 \leq i \leq n$ , from  $GF(2)$ . Using the fact that  $2 = 1 + 1 = 0$ , we obtain

$$\begin{aligned} f^2(x) &= (f_0 + f_1x + \dots + f_nx^n)^2 \\ &= f_0^2 + (f_1x + f_2x^2 + \dots + f_nx^n)^2 \\ &= f_0^2 + (f_1x)^2 + (f_2x^2 + f_3x^3 + \dots + f_nx^n)^2 \end{aligned}$$

Repeating this process for expansion, we eventually end up with

$$\begin{aligned} f^2(x) &= f_0 + f_1x^2 + f_2(x^2)^2 + \dots + f_n(x^2)^n \\ &= f(x^2) \end{aligned}$$



because  $f_i^2 = f_i$ . In general, we can thus see that  $[f(x)]^{2^\lambda} = f(x^{2^\lambda})$  for any  $\lambda \geq 0$ .

**Theorem 2.11** Let  $f(x)$  be a polynomial with coefficients from  $GF(2)$ . Let  $\beta$  be an element in  $GF(2^m)$ , an extension field of  $GF(2)$ . If  $\beta$  is a root of  $f(x)$ , then  $\beta^{2^\lambda}$  for any  $\lambda \geq 0$  is also a root of  $f(x)$ .

**Proof** Since  $[f(x)]^{2^\lambda} = f(x^{2^\lambda})$  and  $f(\beta) = 0$ , we have

$$[f(\beta)]^{2^\lambda} = f(\beta^{2^\lambda}) = 0$$

Hence,  $\beta^{2^\lambda}$  is also a root of  $f(x)$ .

The element  $\beta^{2^\lambda}$  is called a conjugate of  $\beta$ . If  $\beta$  is a root of  $f(x)$  over  $GF(2)$ , then all distinct conjugates of  $\beta$  are also roots of  $f(x)$ . These conjugates are elements in  $GF(2^m)$ . Generally, a single element can have more than one conjugate, in fact, as many as  $m$ .

**Theorem 2.12** The  $2^m - 1$  nonzero elements of  $GF(2^m)$  form all the roots of  $x^{2^m-1} + 1$ .

**Proof** Let  $\beta$  be a nonzero element in the field  $GF(2^m)$ . Then it follows that  $\beta^{2^m-1} = 1$ , or  $\beta^{2^m-1} + 1 = 0$ . This implies that  $\beta$  is a root of the polynomial  $x^{2^m-1} + 1$ . Hence, every nonzero element of  $GF(2^m)$  is a root of  $x^{2^m-1} + 1$ . Since the degree of  $x^{2^m-1} + 1$  is  $2^m - 1$ , the  $2^m - 1$  nonzero elements of  $GF(2^m)$  form all the roots of  $x^{2^m-1} + 1$ .

Since the zero element 0 of  $GF(2^m)$  is the root of  $x$ , Theorem 2.12 can be extended such that the elements, including the zero element, of  $GF(2^m)$  form all the roots of  $x^{2^m} + x$ . Since any element  $\beta$  in  $GF(2^m)$  is a root of the polynomial  $x^{2^m-1} + 1$ ,  $\beta$  must be a root of a polynomial over  $GF(2)$  with a degree less than  $2^m$ .

Consider a ground field  $GF(2)$  and an extension field  $GF(2^m)$ . Let  $\beta$  be any field element of  $GF(2^m)$ . The monic polynomial  $m(x)$  of smallest degree with coefficients in  $GF(2)$  such that  $m(\beta) = 0$  is called the minimal polynomial of  $\beta$ .

**Theorem 2.13** The minimal polynomial  $m(x)$  of any element  $\beta$  is irreducible.

**Proof** Suppose that  $m(x)$  is not irreducible. Then  $m(x) = m_1(x)m_2(x)$ , where  $m_1(x)$  and  $m_2(x)$  are nontrivial factors. Therefore, the degree of either  $m_1(x)$  or  $m_2(x)$  is lower than the degree of  $m(x)$ . Since  $m(\beta) = m_1(\beta)m_2(\beta) = 0$ , then at least one of the factors,  $m_1(\beta)$  or  $m_2(\beta)$ , must be zero. This contradicts the hypothesis that  $m(x)$  is the polynomial of the smallest degree such that  $m(\beta) = 0$ . Thus,  $m(x)$  cannot have nontrivial factors, and it must be irreducible.

**Theorem 2.14** If  $f(x)$  is a polynomial with coefficients from  $GF(2)$  and if  $f(\beta) = 0$ , then  $m(x)$ , the minimal polynomial of  $\beta$ , is a factor of  $f(x)$ .

**Proof** The euclidean division algorithm gives

$$f(x) = m(x)q(x) + r(x)$$

where the degree of  $r(x)$  is less than the degree of  $m(x)$ . Since  $f(\beta) = 0$  and  $m(\beta) = 0$ , it follows that  $r(\beta) = 0$ . Since  $m(x)$  is the minimal polynomial of  $\beta$  and the degree of  $r(x)$  is less than that of  $m(x)$ ,  $r(\beta) = 0$  only when  $r(x) = 0$ . Therefore,  $f(x) = m(x)q(x)$ , so  $f(x)$  is divisible by  $m(x)$ .

**Theorem 2.15** Let  $f(x)$  be an irreducible polynomial with coefficients from  $GF(2)$ ,  $\beta$  be an element in  $GF(2^m)$ , and  $m(x)$  be the minimal polynomial of  $\beta$ . Then  $m(x) = f(x)$  only when  $f(\beta) = 0$ .

**Proof** From Theorem 2.14,  $m(x)$  is a factor of  $f(x)$ , and it follows that  $m(x) = f(x)$  for  $m(x) \neq 1$ , because  $f(x)$  is irreducible.

**Theorem 2.16** Let  $p$  be the characteristic of  $GF(q)$ . Then for any polynomial  $f(x)$  over  $GF(q)$  and for any positive integer  $m$ ,

$$\left[ \sum_{i=0}^m f_i x^i \right]^p = \sum_{i=0}^m f_i^p x^{ip^m}$$

**Proof** The same reasoning will be used here as in the proof of Theorem 2.8. Let us proceed in small iterative steps, beginning with  $m = 1$ . From  $f(x) = f_1(x)x + f_0$ , it follows that

$$[f(x)]^p = \sum_{i=0}^p \binom{p}{i} [f_1(x)x]^i f_0^{p-i}$$

where a prime  $p$  does not appear in the denominator of

$$\binom{p}{i} = \frac{p!}{i!(p-i)!} = \frac{p(p-1)!}{i!(p-i)!}$$

unless  $i = 0$  or  $p$ . Hence, except for  $i = 0$  or  $p$ ,  $\binom{p}{i}$  is a multiple of  $p$ , and  $\binom{p}{i} = 0 \pmod{p}$ . Thus, we have

$$[f(x)]^p = [f_1(x)]^p x^p + f_0^p$$

Applying the same process to  $f_1(x)$  and continuing in this fashion, we have

$$[f(x)]^p = \sum_{i=0}^p f_i^p x^{ip}$$

Further, for  $m = 2$ ,

$$[f(x)]^{p^2} = [[f(x)]^p]^p = \left[ \sum_{i=0}^p f_i^p x^{ip} \right]^p = \sum_{i=0}^p f_i^{p^2} x^{ip^2}$$

Thus the theorem is proved by repeating this iterative process any number of times.

**Theorem 2.17** Let  $m(x)$  be the minimal polynomial over  $GF(q)$  of  $\beta$ , an element of  $GF(q^m)$ . Then  $m(x)$  is also the minimal polynomial of  $\beta^q$ .

**Proof** Using Theorem 2.16, we have

$$[m(x)]^q = \sum_i m_i^q (x^q)^i \quad 0 \leq i \leq \deg m(x)$$

Since  $m_i^q = m_i$ , it follows that

$$[m(x)]^q = \sum_i m_i (x^q)^i = m(x^q)$$

Knowing that  $m(\beta) = 0$ , we have  $0 = [m(\beta)]^q = m(\beta^q)$ . Therefore,  $\beta^q$  is also a root of  $m(x)$ . Since  $m(x)$  is irreducible,  $m(x)$  is the minimal polynomial of  $\beta^q$ . Thus, since two elements  $\beta$  and  $\beta^q$  of  $GF(q^m)$  share the same minimal polynomial  $m(x)$  over  $GF(q)$ , they are called conjugates.

The conjugates of an element  $\beta$  are easily obtained using Theorem 2.17. If  $m(x)$  is the minimal polynomial of  $\beta$ , then it is also that of  $\beta^q$ ,  $\beta^{q^2}$ ,  $\beta^{q^3}$ , ...,  $\beta^{q^{e-1}}$ , etc. Variable  $e$  is the smallest integer such that  $\beta^{q^e} = \beta$ . In fact, the conjugates are all roots of  $m(x)$ .

For a polynomial  $m(x)$  over a field  $GF(q)$ , an extension field  $GF(q^m)$  containing  $GF(q)$  is called a splitting field for  $m(x)$  if  $m(x)$  can be factored into linear factors, say, the product of some polynomials of degree 1 over  $GF(q^m)$ . Thus, if  $GF(q^m)$  is a splitting field of  $m(x)$ , the roots of  $m(x)$  generate  $GF(q^m)$ .

**Theorem 2.18** Let  $m(x)$  be the minimal polynomial of an element  $\beta$  in  $GF(q^m)$ . Let  $e$  be the smallest integer such that  $\beta^{q^e} = \beta$ . Then

$$m(x) = (x - \beta)(x - \beta^q) \cdots (x - \beta^{q^{e-1}})$$

**Proof** From Theorem 2.17, the minimal polynomial of  $\beta$  in  $GF(q^m)$  must certainly have all of these roots  $\beta$ ,  $\beta^q$ , ...,  $\beta^{q^{e-1}}$ . As stated in Theorem 2.12, all we need to show is that  $m(x)$  thus constructed has coefficients only in  $GF(q)$ . The set of nonzero elements of  $GF(q^m)$  that are roots of  $x^q - x$  forms the subfield  $GF(q)$ . First, let us find  $[m(x)]^q$ .

$$\begin{aligned} [m(x)]^q &= (x - \beta)^q (x - \beta^q)^q (x - \beta^{q^2})^q \cdots (x - \beta^{q^{e-1}})^q \\ &= (x^q - \beta^q)(x^q - \beta^{q^2})(x^q - \beta^{q^3}) \cdots (x^q - \beta^q) \end{aligned}$$

where we used  $\beta^{q^e} = \beta$ . Thus, we have

$$[m(x)]^q = m(x^q) = \sum_i m_i x^{iq}$$

whereas, using Theorem 2.16, it follows that

$$[m(x)]^q = \left[ \sum_i m_i x^i \right]^q = \sum_i m_i^q x^{iq}$$

Thus we can see easily that  $m_i^q = m_i$  for each  $i$ . Hence every  $m_i$  is in the subfield  $GF(q)$  as expected.

**Example 2.5** Consider the Galois field  $GF(2^5)$  given by Table 7.2. Let  $\alpha$  be a primitive element of  $GF(2^5)$  and let  $\beta = \alpha^3$ . Then the set of conjugates of  $\beta$  is

$\{\beta, \beta^2, \beta^{2^2}, \beta^{2^3}, \beta^{2^4}\} = \{\alpha^3, \alpha^6, \alpha^{12}, \alpha^{24}, \alpha^{17}\}$ . The minimal polynomial  $m(x)$  of  $\beta = \alpha^3$  over  $GF(2)$  is then

$$m(x) = (x + \alpha^3)(x + \alpha^6)(x + \alpha^{12})(x + \alpha^{24})(x + \alpha^{17})$$

which, when multiplied out, must only have coefficients in  $GF(2)$  such that

$$m(x) = 1 + x^2 + x^3 + x^4 + x^5$$

## PROBLEMS

**2.1** The group  $G = \{1, 2, \dots, p-1\}$  under modulo- $p$  multiplication is called a multiplication group. Construct a table for the group under modulo-3 multiplication.

**2.2** A prime field  $GF(p)$  is defined as a set  $\{0, 1, 2, \dots, p-1\}$  of order  $p$  under modulo- $p$  addition and multiplication. Construct tables for  $GF(7)$  under modulo-7 addition and multiplication.

**2.3** Prove that every cyclic group is abelian.

**2.4** Compute the greatest common divisor of the two integers 901 and 238 by using the euclidean division algorithm.

**2.5** For the two integers 901 and 238 given in Prob. 2.4, there exist integers  $a$  and  $b$  such that  $GCD(901, 238) = 901a + 238b$ . Find  $a$  and  $b$ .

**2.6** If  $r$  is congruent to  $s$  modulo  $d$ , it is denoted as  $r \equiv s \pmod{d}$ . When two integers  $r$  and  $s$  are given as 238 and 901, respectively, show that  $238 \equiv 901 \pmod{17}$ .

**2.7** Consider the set  $S = \{0, 1, 2\}$  with three elements.

*a* Construct the arithmetic tables.

*b* Is this a field?

*c* Does  $GF(2)$  contain  $GF(3)$ ?

**2.8** Consider a group with 24 elements such that  $\mathbf{Z}_{24} = \{0, 1, 2, \dots, 23\}$ .

*a* What is  $GCD(4, 24)$ ?

*b* Let 4 be a generator of an additive subgroup of  $\mathbf{Z}_{24}$ . Find the subgroup of six elements.

**2.9** Let  $r(x) = x^6 + x^5 + x^3 + 1$ , and let  $s(x) = x^8 + x^5 + x^4 + x + 1$ .

*a* Find  $GCD[r(x), s(x)]$  over  $GF(2)$ .

*b* Find  $a(x)$  and  $b(x)$  that satisfy  $GCD[r(x), s(x)] = a(x)r(x) + b(x)s(x)$ .

**2.10** Consider the prime field  $GF(11)$ .

*a* Construct addition and multiplication tables for this field.

*b* Find the set of all primitive elements.

*c* Determine the orders of all nonprimitive elements.



**2.11** Consider  $GF(2^4)$  based on the primitive polynomial  $p(x) = 1 + x + x^4$ . Let  $\alpha$  be a primitive element of  $GF(2^4)$ .

- a* Construct a table for  $GF(2^4)$  by showing power, polynomial, and vector notations of each field element.
- b* Find all the distinct conjugates of  $\alpha$  and  $\alpha^3$ .
- c* Find the minimum polynomials of  $\alpha$  and  $\alpha^3$ .
- d* Calculate the values of  $\alpha^5\alpha^{11}$ ,  $\alpha^4/\alpha^{10}$ , and  $1 + \alpha^3 + \alpha^{11}$ .

## Linear Binary Block Codes

From the point of view of practical importance, the linear binary codes discussed in this chapter constitute a very small subclass of all possible block codes. The linear binary codes have been defined over the binary field  $GF(2)$  of two elements  $\{0, 1\}$  using modulo-2 arithmetic. However, the binary codes can be generalized to the nonbinary codes like Reed-Solomon codes with code symbols from an extension field  $GF(2^m)$ , where  $m$  is the number of bits per symbol. The linear block codes are often referred to by the names of group codes or parity-check codes. A block code made up entirely of  $2^k$  code words of block length  $n$  is called a linear  $(n, k)$  code if and only if its  $2^k$  code words form a  $k$ -dimensional subspace of the vector space of all the  $n$ -tuples over the binary field  $GF(2)$ . A linear block code is a nonempty set of code words ( $n$ -tuples each) over  $GF(2)$  if and only if the modulo-2 sum of any two code words under componentwise vector addition is also a code word and the product of any code word and a field element is a code word. In any linear code, the all-zero word is always a code word because it can be generated by taking the sum of a code word and itself.

Another important subclass of linear block codes is cyclic codes, or polynomial-generated codes, which will be presented in the subsequent several chapters. This class of codes is defined by imposing a cyclic structural property that helps to make the encoders and decoders practical.

In this chapter, the linear codes are described in terms of their generator and parity-check matrices. Encoding by the generator matrix, coding bounds on the minimum distance, the syndrome for error correction, the standard array for table-lookup decoding, and weight enumerators and the probability of decoding error are intensively dis-

cussed. Finally, several noncyclic block codes that are frequently encountered in practice, including the Hamming code, the Golay code, the Hadamard code, the Reed-Muller code, and some product codes, are presented in this chapter.

### 3.1 Matrix Description of Linear Codes

The set of all  $n$ -tuples where the entries are chosen from  $GF(2)$  is a vector space. A linear code is a vector space, where each code word is a vector. Thus, a set of these vectors of length  $n$  is called a linear block code if and only if it is a subspace of the vector space of  $n$ -tuples.

Matrix representation of a code is an ideal way to describe the code compactly and completely. An  $(n, k)$  linear code is completely specified by a  $k \times n$  generator matrix  $\mathbf{G}$ . Each row of  $\mathbf{G}$  is an  $n$ -tuple over  $GF(2)$ , and each column is a  $k$ -tuple over  $GF(2)$ . Since the row space of  $\mathbf{G}$  is any set of basis vectors for the  $k$ -dimensional subspace, any code word  $\mathbf{c}$  is a linear combination of the rows of  $\mathbf{G}$  based on the information data  $\mathbf{d} = (d_0, d_1, \dots, d_{k-1})$  such that

$$\mathbf{c} = d_0 \mathbf{g}_0 + d_1 \mathbf{g}_1 + \dots + d_{k-1} \mathbf{g}_{k-1} \quad (3.1)$$

where  $d_i$ ,  $0 \leq i \leq k-1$ , denotes the information bits and  $\mathbf{g}_i$ ,  $0 \leq i \leq k-1$ , is any row vector of  $\mathbf{G}$ .

Consider a  $k$ -bit information sequence  $\mathbf{d} = (d_0, d_1, \dots, d_{k-1})$  that is to be encoded. The corresponding code word is denoted by the  $n$ -bit vector  $\mathbf{c} = (c_0, c_1, \dots, c_{n-1}) = (d_0, d_1, \dots, d_{k-1}) \cdot \mathbf{G}$ , where  $\mathbf{G}$  is a  $k \times n$  generator matrix. The systematic structure of a code word consists of the  $k$ -bit information part and the  $n-k$  parity-check part, which is given by the weighted modulo-2 sum of the information digits. Since the  $n-k$  parity-check digits are linear sums of the information digits, parity-check equations of the code can be expressed by

$$\gamma_j = \sum_{i=0}^{k-1} p_{i,j} d_i = p_{0,j} d_0 + p_{1,j} d_1 + \dots + p_{k-2,j} d_{k-2} + p_{k-1,j} d_{k-1} \quad 0 \leq j \leq n-k-1 \quad (3.2)$$

where  $p_{i,j} = 0$  or  $1$ .

If a code word  $\mathbf{c}$  is arranged by the systematic format such that

$$\begin{aligned} \mathbf{c} &= (c_0, c_1, \dots, c_{n-k-1}, c_{n-k}, \dots, c_{n-1}) \\ &= (\gamma_0, \gamma_1, \dots, \gamma_{n-k-1}, d_0, d_1, \dots, d_{k-1}) \end{aligned}$$

the rightmost  $k$  bits in a code word  $\mathbf{c}$  are the information digits and the leftmost  $n-k$  bits are the parity-check digits. A linear code with this arrangement is referred to as a linear systematic code. Thus, the

parity-check digits represented by Eq. (3.2) can be written as

$$\begin{aligned}
 c_0 &= \gamma_0 = p_{0,0}d_0 + p_{1,0}d_1 + \cdots + p_{k-2,0}d_{k-2} + p_{k-1,0}d_{k-1} \\
 c_1 &= \gamma_1 = p_{0,1}d_0 + p_{1,1}d_1 + \cdots + p_{k-2,1}d_{k-2} + p_{k-1,1}d_{k-1} \\
 c_2 &= \gamma_2 = p_{0,2}d_0 + p_{1,2}d_1 + \cdots + p_{k-2,2}d_{k-2} + p_{k-1,2}d_{k-1} \\
 &\vdots \\
 c_{n-k-1} &= \gamma_{n-k-1} = p_{0,n-k-1}d_0 + p_{1,n-k-1}d_1 + \cdots \\
 &\quad + p_{k-2,n-k-1}d_{k-2} + p_{k-1,n-k-1}d_{k-1}
 \end{aligned} \quad (3.3)$$

Since this parity-check portion is completely specified by the following form:

$$\begin{aligned}
 (c_0, c_1, c_2, \dots, c_{n-k-1}) \\
 = (d_0, d_1, d_2, \dots, d_{k-1}) \cdot \begin{bmatrix} p_{0,0} & p_{0,1} & \cdots & p_{0,n-k-1} \\ p_{1,0} & p_{1,1} & \cdots & p_{1,n-k-1} \\ p_{2,0} & p_{2,1} & \cdots & p_{2,n-k-1} \\ \vdots & \vdots & \ddots & \vdots \\ p_{k-1,0} & p_{k-1,1} & \cdots & p_{k-1,n-k-1} \end{bmatrix}
 \end{aligned} \quad (3.4)$$

the information portion is then

$$\begin{aligned}
 (c_{n-k}, c_{n-k+1}, \dots, c_{n-1}) &= (d_0, d_1, \dots, d_{k-1}) \cdot \begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \end{bmatrix}
 \end{aligned} \quad (3.5)$$

Thus, the  $n$ -tuple code word  $\mathbf{c}$  corresponding to the  $k$ -tuple information sequence  $\mathbf{d}$  is related by the following formula:

$$(c_0, c_1, \dots, c_{n-k-1}, c_{n-k}, \dots, c_{n-1}) = (d_0, d_1, \dots, d_{k-1}) \cdot [\mathbf{P}_{k \times (n-k)} : \mathbf{I}_k] \quad (3.6)$$

where  $\mathbf{P}_{k \times (n-k)}$  denotes the  $k \times (n-k)$  matrix in Eq. (3.4) that determines the  $n-k$  redundant bits and  $\mathbf{I}_k$  is the  $k \times k$  identity matrix in Eq. (3.5) that reproduces the  $k$  information bits.

### 3.1.1 The generator matrix

The encoding procedure of Eq. (3.6) may also be represented in matrix form as



$$\mathbf{c} = \mathbf{d} \cdot \mathbf{G} \quad (3.7)$$

where

$$\begin{aligned} \mathbf{G} &= [\mathbf{P}_{k \times (n-k)} \vdots \mathbf{I}_k] \\ &= \begin{bmatrix} p_{0,0} & p_{0,1} & \cdots & p_{0,n-k-1} & 1 & 0 & 0 & \cdots & 0 \\ p_{1,0} & p_{1,1} & \cdots & p_{1,n-k-1} & 0 & 1 & 0 & \cdots & 0 \\ \vdots & & & & & & & & \\ p_{k-1,0} & p_{k-1,1} & \cdots & p_{k-1,n-k-1} & 0 & 0 & 0 & \cdots & 1 \end{bmatrix} = \begin{bmatrix} \mathbf{g}_0 \\ \mathbf{g}_1 \\ \vdots \\ \mathbf{g}_{k-1} \end{bmatrix} \end{aligned} \quad (3.8)$$

is called the generator matrix of the systematic code.

By performing elementary row operations on  $\mathbf{G}$ , we obtain another matrix  $\mathbf{G}$  over  $GF(2)$  by interchanging any two rows in the row space of  $\mathbf{G}$  and adding one row to another. Therefore,  $\mathbf{G}$  is not unique because the set of basis vectors of  $\mathbf{G}$  is not unique. We also note that since the subspace has dimension  $k$ , the rank of  $\mathbf{G}$  is  $k$ .

An  $(n, k)$  code generated by  $\mathbf{G}$  that is not in the systematic form of Eq. (3.8) is called nonsystematic. However, such a generator matrix is equivalent to a generator matrix in systematic form in the sense that one can be obtained from the other by elementary row operations and column permutations. Thus, any linear  $(n, k)$  code is equivalent to a linear  $(n, k)$  systematic code.

**Example 3.1** Consider the  $(6, 3)$  linear code with a generator matrix in the nonsystematic form:

$$\mathbf{G} = \begin{bmatrix} 1 & 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} \mathbf{g}_0 \\ \mathbf{g}_1 \\ \mathbf{g}_2 \end{bmatrix}$$

By performing elementary row operations, we obtain the following generator matrix in systematic form:

$$\mathbf{G} = \begin{bmatrix} 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} \mathbf{g}_0 + \mathbf{g}_2 \\ \mathbf{g}_2 \\ \mathbf{g}_1 \end{bmatrix}$$

The block code given in Table 3.1 is a  $(6, 3)$  linear code whose code words are obtained from the two generator matrices given above.

**TABLE 3.1** Code Words Corresponding to Information Data for the (6, 3) Linear Code

| Information data | Code words    |   |   |            |   |   |
|------------------|---------------|---|---|------------|---|---|
|                  | Nonsystematic |   |   | Systematic |   |   |
| 0 0 0            | 0             | 0 | 0 | 0          | 0 | 0 |
| 0 0 1            | 0             | 1 | 1 | 0          | 0 | 1 |
| 0 1 0            | 1             | 1 | 0 | 0          | 0 | 1 |
| 0 1 1            | 1             | 0 | 1 | 0          | 1 | 1 |
| 1 0 0            | 1             | 1 | 0 | 1          | 1 | 0 |
| 1 0 1            | 1             | 0 | 1 | 1          | 0 | 0 |
| 1 1 0            | 0             | 0 | 0 | 1          | 1 | 1 |
| 1 1 1            | 0             | 1 | 1 | 1          | 0 | 1 |

For example, the information vector  $\mathbf{d} = (1\ 0\ 1)$  is encoded into the code words as follows:

$$\mathbf{c} = (1\ 0\ 1) \begin{bmatrix} 1 & 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 & 0 \end{bmatrix} = (1\ 0\ 1\ 1\ 0\ 0)$$

for a nonsystematic word, and

$$\mathbf{c} = (1\ 0\ 1) \begin{bmatrix} 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 & 1 \end{bmatrix} = (0\ 1\ 1\ 1\ 0\ 1)$$

for a systematic word.

As can be seen from Table 3.1, the one-to-one pairing between information data and code words depends on the choice of basis vectors that appear as rows of  $\mathbf{G}$ . However, the total set of code words is unaffected.

### 3.1.2 The parity-check matrix

Let  $C$  be an  $(n, k)$  linear code. Associated with any linear  $(n, k)$  code is the dual code  $C_D$  of dimension  $(n - k)$ . The dual code is a linear  $(n, n - k)$  code with  $2^{n-k}$  code words, which is the null space of the linear  $(n, k)$  code. The generator matrix for the dual code, denoted by  $\mathbf{H}$ , consists of  $(n - k)$  linearly independent code vectors such that any vector in the row space of  $\mathbf{G}$  is orthogonal to the rows of  $\mathbf{H}$ . This matrix  $\mathbf{H}$  is called a parity-check matrix of a linear code  $C$ . It is an  $(n - k) \times n$  matrix of rank  $n - k$ . An  $n$ -tuple  $\mathbf{c}$  is a code word if and only if it is orthogonal to every row vector of  $\mathbf{H}$ . That is,

$$\mathbf{c} \cdot \mathbf{H}^T = \mathbf{0} \quad (3.9)$$

where  $\mathbf{0}$  denotes an all-zero row vector with  $n - k$  elements. This provides one way to test whether a word is a legitimate code vector. Since  $\mathbf{c} \cdot \mathbf{H}^T = \mathbf{0}$  holds for every code word of an  $(n, k)$  code, it follows that:

$$\mathbf{G} \cdot \mathbf{H}^T = \mathbf{0}$$

where  $\mathbf{0}$  is now a  $k \times (n - k)$  matrix with all-zero elements.

If the generator matrix  $\mathbf{G}$  of an  $(n, k)$  systematic code is in the form of Eq. (3.8), the parity-check matrix  $\mathbf{H}$  may take the following form:

$$\mathbf{H} = [\mathbf{I}_{n-k} : \mathbf{P}_{(n-k) \times k}^T] = \begin{bmatrix} 1 & 0 & 0 & \cdots & 0 & p_{0,0} & \cdots & p_{k-1,0} \\ 0 & 1 & 0 & \cdots & 0 & p_{0,1} & \cdots & p_{k-1,1} \\ & & \vdots & & & \vdots & & \vdots \\ 0 & 0 & 0 & \cdots & 1 & p_{0,n-k-1} & \cdots & p_{k-1,n-k-1} \end{bmatrix} \quad (3.10)$$

where  $\mathbf{P}^T$  is the transpose of submatrix  $\mathbf{P}$  in Eq. (3.8). Thus, for any  $k \times n$  generator matrix  $\mathbf{G}$  whose rows  $\mathbf{g}_0, \mathbf{g}_1, \dots, \mathbf{g}_{k-1}$  are linearly independent, there exists an  $(n - k) \times n$  parity-check matrix  $\mathbf{H}$  whose rows  $\mathbf{h}_0, \mathbf{h}_1, \dots, \mathbf{h}_{n-k-1}$  are linearly independent such that the inner product of any row  $\mathbf{g}_i$  in  $\mathbf{G}$  and any row  $\mathbf{h}_j$  in  $\mathbf{H}$  must be zero, i.e.,  $\mathbf{g}_i \cdot \mathbf{h}_j = 0$  for  $0 \leq i \leq k - 1$  and  $0 \leq j \leq n - k - 1$ .

**Example 3.2** Consider the (7, 4) linear code with a generator matrix:

$$\mathbf{G} = \begin{bmatrix} 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} \mathbf{g}_0 \\ \mathbf{g}_1 \\ \mathbf{g}_2 \\ \mathbf{g}_3 \end{bmatrix}$$

The information vector  $\mathbf{d} = (0 \ 1 \ 1 \ 0)$  is systematically encoded into the code word either as

$$\mathbf{c} = \mathbf{d} \cdot \mathbf{G}$$

$$= (0 \ 1 \ 1 \ 0) \begin{bmatrix} 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 \end{bmatrix} = (1 \ 0 \ 1 \ 0 \ 1 \ 1 \ 0)$$

**TABLE 3.2** Code Words for the Linear (7, 4) Code

| Information bits | Code words    |
|------------------|---------------|
| 0 0 0 0          | 0 0 0 0 0 0 0 |
| 0 0 0 1          | 1 1 1 0 0 0 1 |
| 0 0 1 0          | 1 1 0 0 0 1 0 |
| 0 0 1 1          | 0 0 1 0 0 1 1 |
| 0 1 0 0          | 0 1 1 0 1 0 0 |
| 0 1 0 1          | 1 0 0 0 1 0 1 |
| 0 1 1 0          | 1 0 1 0 1 1 0 |
| 0 1 1 1          | 0 1 0 0 1 1 1 |
| 1 0 0 0          | 1 0 1 1 0 0 0 |
| 1 0 0 1          | 0 1 0 1 0 0 1 |
| 1 0 1 0          | 0 1 1 1 0 1 0 |
| 1 0 1 1          | 1 0 0 1 0 1 1 |
| 1 1 0 0          | 1 1 0 1 1 0 0 |
| 1 1 0 1          | 0 0 1 1 1 0 1 |
| 1 1 1 0          | 0 0 0 1 1 1 0 |
| 1 1 1 1          | 1 1 1 1 1 1 1 |

$$\begin{aligned}
 \text{or } \mathbf{c} &= \sum_{i=0}^3 d_i \mathbf{g}_i = 0 \cdot \mathbf{g}_0 + 1 \cdot \mathbf{g}_1 + 1 \cdot \mathbf{g}_2 + 0 \cdot \mathbf{g}_3 \\
 &= (0 \ 1 \ 1 \ 0 \ 1 \ 0 \ 0) + (1 \ 1 \ 0 \ 0 \ 0 \ 1 \ 0) \\
 &= (1 \ 0 \ 1 \ 0 \ 1 \ 1 \ 0)
 \end{aligned}$$

Thus, as listed in Table 3.2, there are 16 code words corresponding to  $2^4 = 16$  information words.

According to Eq. (3.9), the parity-check matrix  $\mathbf{H}$  of this (7, 4) code is immediately obtained as

$$\mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 & 0 & 1 \end{bmatrix}$$

### 3.2 Encoder Implementation of Linear Block Codes

As stated in the previous section, any one-to-one correspondence of information data and code words can be used as an encoding procedure through the relation

$$\mathbf{c} = \mathbf{d} \cdot \mathbf{G}$$

where  $\mathbf{d}$  is a  $k$ -tuple information sequence to be encoded and  $\mathbf{c}$  is the corresponding  $n$ -tuple code word. When this encoding equation is used to define the encoder, the pairing between  $\mathbf{d}$  and  $\mathbf{c}$  depends on the choice of basis vectors appearing as rows of  $\mathbf{G}$ . Based on Eqs. (3.4) and



(3.5), the encoder for an  $(n, k)$  linear systematic code can be implemented easily. As a result of the systematic structure, the information sequence  $\mathbf{d} = (d_0, d_1, \dots, d_{k-1})$  to be encoded is shifted into the channel for reproduction of information bits and at the same time into the shift register where the  $n - k$  parity-check bits are formed. These redundancy bits are serialized and shifted into the channel, forming a complete code word in systematic form.

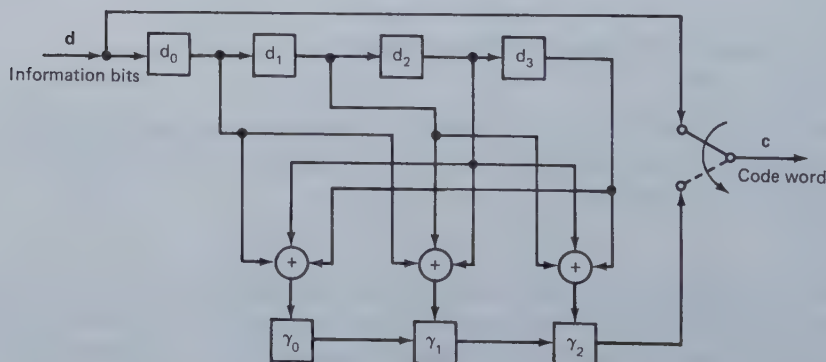
**Example 3.3** Consider the  $(7, 4)$  linear code with a generator matrix

$$\mathbf{G} = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Let  $\mathbf{d} = (d_0, d_1, d_2, d_3)$  be the information bits for encoding. According to Eq. (3.4), the  $n - k$  parity-check bits  $\gamma_i, i = 0, 1, 2$ , are expressed by

$$\begin{aligned} (\gamma_0, \gamma_1, \gamma_2) &= (d_0, d_1, d_2, d_3) \cdot \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix} \\ &= (d_0 + d_2 + d_3, d_0 + d_1 + d_2, d_1 + d_2 + d_3) \end{aligned}$$

Based on this equation, the encoder for the  $(7, 4)$  code is easily implemented as shown in Fig. 3.1. Thus, the systematic code word sent to the channel is  $\mathbf{c} = (\gamma_0, \gamma_1, \gamma_2, d_0, d_1, d_2, d_3)$ . For example, if the information sequence  $\mathbf{d} = (0 \ 1 \ 1 \ 0)$  is shifted into the shift register, then the  $n - k = 3$  parity-check digits are



**Figure 3.1** Encoder for the  $(7, 4)$  systematic code.

formed at the outputs of the  $n - k = 3$  modulo-2 adders. They are  $\gamma_0 = 1$ ,  $\gamma_1 = 0$ , and  $\gamma_2 = 0$ . Thus, the code word  $\mathbf{c} = (1\ 0\ 0\ 0\ 1\ 1\ 0)$  is formed.

### 3.3 Coding Bounds for Linear Block Codes

The topic of coding bounds has been addressed in numerous coding papers in the past. The coding bounds on minimum distance  $d_{\min}$  are important in assessing the capabilities of error-correcting codes. With an  $(n, k)$  block code of  $2^k$   $n$ -tuple words, it is possible to derive both upper and lower bounds on the largest minimum distance attainable. One pair of upper and lower bounds is obtained by considering two code words which differ by the minimum distance. The bounds on  $d_{\min}$  are obtained from structural aspects of the codes. The principal useful bounds of this type are the Hamming bound, the Plotkin bound, and the Gilbert-Varshamov bound.

The Hamming bound, Plotkin bound, and the Elias bound all provide upper bounds on  $d_{\min}$  for any  $(n, k)$  block code, either linear or nonlinear. The Hamming bound tends to provide the tightest bound for high-rate codes, while the Plotkin bound is more appropriate for low-rate codes. On the other hand, the Gilbert-Varshamov bound provides a lower bound for  $d_{\min}$  of the best code. At low rates, the Elias bound approaches the Plotkin bound. At medium rates, we may obtain a stronger bound by combining the Hamming bound and the Plotkin bound. These bounds on  $d_{\min}$  are often used when developing new codes to indicate how close the code is to the best possible one. The bounds on performance also indicate that the average performance of all block codes exhibits a probability of error that decreases exponentially with code length. Thus, the minimum distance is certainly one measure of a code's ability to correct errors. As a necessary prerequisite, the concept of minimum distance will be discussed in the following subsection.

#### 3.3.1 The minimum Hamming distance and weight

As discussed previously, an error-correcting code of length  $n$  and code rate  $R$  is judged by its minimum distance. Let  $\mathbf{c} = (c_0, c_1, \dots, c_{n-1})$  be any  $n$ -tuple code word. The Hamming weight of  $\mathbf{c}$ , denoted by  $w(\mathbf{c})$ , is defined as the number of nonzero components of  $\mathbf{c}$ . Let  $\mathbf{u}$  and  $\mathbf{v}$  be two binary  $n$ -tuple words. The Hamming distance between  $\mathbf{u}$  and  $\mathbf{v}$ , denoted by  $d(\mathbf{u}, \mathbf{v})$ , is defined as the number of positions in which they differ from each other. To elaborate, suppose that  $\mathbf{c}$  is the transmitted code word and  $\mathbf{r}$  is the received word at the output of the demodulator. If  $\mathbf{c} \neq \mathbf{r}$ , then an error pattern  $\mathbf{e}$  of  $t$  errors will result in  $\mathbf{r}$  which differs from  $\mathbf{c}$  in  $t$  positions. That is  $d(\mathbf{c}, \mathbf{r}) = t$ . Thus, given a linear  $(n, k)$

code, one can compute the Hamming distances between all possible pairs of code words. The smallest value of the Hamming distance for all pairs of code words is called the minimum distance of the code and is designated by  $d_{\min}$ . Hence, the minimum distance of a linear block code  $C$  is defined as

$$d_{\min} = \min\{d(\mathbf{u}, \mathbf{v}) : \mathbf{u} \neq \mathbf{v}; \mathbf{u}, \mathbf{v} \in C\} \quad (3.11)$$

For an  $(n, k)$  linear block code  $C$ , the minimum weight, denoted  $w_{\min}$ , is the smallest weight of any nonzero code word in  $C$ . Since the Hamming distance between two code words  $\mathbf{u}$  and  $\mathbf{v}$  in  $C$  is equal to the Hamming weight of the modulo-2 sum of  $\mathbf{u}$  and  $\mathbf{v}$ , it follows that

$$d(\mathbf{u}, \mathbf{v}) = w(\mathbf{u} + \mathbf{v}) = w(\mathbf{c}) \quad (3.12)$$

Therefore, determining the minimum distance of the code is equivalent to finding its minimum weight. From Eqs. (3.11) and (3.12), the minimum weight is defined as follows:

$$\begin{aligned} w_{\min} &= \min\{w(\mathbf{u} + \mathbf{v}) : \mathbf{u} \neq \mathbf{v}; \mathbf{u}, \mathbf{v} \in C\} \\ &= \min\{w(\mathbf{c}) : \mathbf{c} \neq \mathbf{0}, \mathbf{c} \in C\} \\ &= d_{\min} \end{aligned} \quad (3.13)$$

Let  $C$  be an  $(n, k)$  code whose code words are in systematic form. With a code such as  $C$ , there exist code words with only one nonzero information bit and  $n - k$  parity-check bits. Such a code word cannot have a Hamming weight that is larger than  $n - k + 1$ . Since the minimum distance in  $C$  is equal to the minimum weight,  $d_{\min}$  is upper-bounded as follows:

$$d_{\min} \leq n - k + 1 \quad (3.14)$$

This is a simple upper bound on the minimum distance of an  $(n, k)$  binary or nonbinary linear block code. However, this upper bound is extremely loose for binary codes. In the case where  $d_{\min} = n - k + 1$  for the largest possible distance, the code is called a maximum-distance-separable code.

The relation of the minimum distance of a code to its parity-check matrix  $\mathbf{H}$  can be used to prove Eq. (3.14) as shown in the following discussion. Let  $\mathbf{c} = (c_0, c_1, \dots, c_{n-1})$  be a code word of the Hamming weight  $w(\mathbf{c})$ . Then,  $\mathbf{c} \cdot \mathbf{H}^T$  represents a linear combination of the  $n$  columns of  $\mathbf{H}$ . Since  $\mathbf{c} \cdot \mathbf{H}^T = \mathbf{0}$ , if  $\mathbf{c}$  is a legitimate code word, the column vectors of  $\mathbf{H}$  are linearly dependent. If  $\mathbf{c}_j$  denotes the minimum weight code word whose nonzero components are  $c_{j1}, c_{j2}, \dots, c_{jm}$ , then the condition  $\mathbf{c}_j \cdot \mathbf{H}^T = \mathbf{0}$  must hold. Consequently, the condition can be written as

$$c_{j1}h_{j1} + c_{j2}h_{j2} + \cdots + c_{jm}h_{jm} = 0$$

where  $c_{j\lambda}$ ,  $1 \leq \lambda \leq m$ , are the  $m$  nonzero components of  $\mathbf{c}_j$ . Because  $c_{j1} = c_{j2} = \cdots = c_{jm} = 1$ , we must have

$$h_{j1} + h_{j2} + \cdots + h_{jm} = 0 \quad (3.15)$$

Hence, we note that the minimum weight of  $C$  is equal to the smallest number of columns of  $\mathbf{H}$  that sum to 0. Since  $w_{\min} = d_{\min}$ , it follows that  $d_{\min}$  columns of  $\mathbf{H}$  are linearly dependent. In other words, no more than  $d_{\min} - 1$  columns of  $\mathbf{H}$  are linearly independent. Since the rank of  $\mathbf{H}$  is at most  $n - k$ , then  $n - k \geq d_{\min} - 1$ , or  $d_{\min} \leq n - k + 1$ .

**Example 3.4** Consider the (7, 4) linear code with the parity-check matrix

$$\mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 & 0 & 1 \end{bmatrix} = [\mathbf{h}_0, \mathbf{h}_1, \mathbf{h}_2, \mathbf{h}_3, \mathbf{h}_4, \mathbf{h}_5, \mathbf{h}_6]$$

which is given in Example 3.2. With this parity-check matrix, no two or fewer columns sum to zero because all columns of  $\mathbf{H}$  are distinct and nonzero. Hence, the minimum weight of this code is at least 3. Since  $w_{\min} = d_{\min}$ , there must exist at least one subset of  $d_{\min}$  columns of  $\mathbf{H}$  that sum to zero. Thus, we see that, from  $\mathbf{H}$ , there exists at least one set of three columns, i.e.,  $\mathbf{h}_0$ ,  $\mathbf{h}_1$ , and  $\mathbf{h}_5$  that sum to zero. Therefore, this code must have a minimum distance of 3.

In order for an  $(n, k)$  linear code to correct  $t$  errors, we must find a code with a minimum distance that satisfies

$$d_{\min} \geq 2t + 1 \quad (3.16)$$

which means that  $d_{\min}$  must always be at least one more than twice the number of errors that are to be corrected. Any  $(n, k)$  linear code with minimum distance  $2t + 1$  is called a perfect code. With a coding system which achieves channel capacity, an  $(n, k)$  linear code that has minimum distance  $d_{\min}$  can correct all error patterns of weight

$$t = \lfloor (d_{\min} - 1)/2 \rfloor \quad (3.17)$$

where  $\lfloor x \rfloor$  denotes the largest integer contained in  $x$ . The interpretation of Eq. (3.17) is that the optimum decoder for a binary symmetric channel (BSC) will decode correctly if the  $t$  errors in a code word represent less than half the minimum distance  $d_{\min}$  of the code. Thus, we can conclude that any number of errors up to  $t = \lfloor (d_{\min} - 1)/2 \rfloor$  are always correctable. Hence, the minimum distance is certainly one measure of a code's ability to correct errors. The parameter  $t$  of Eq. (3.17) is called the random-error-correcting capability of the code. The code



in this sense is referred to as a  $t$ -error-correcting code. For example, the Golay (23, 12) code having  $d_{\min} = 7$  can correct three errors or less. Clearly, for given  $n$  and  $k$ , it is desirable to construct a block code that has the largest possible minimum distance.

### 3.3.2 The Hamming bound

Since the BSC is memoryless, the bit errors occur independently. The number of errors occurring in a  $n$ -tuple code word on a BSC has a binomial distribution with mean  $np$  and variance  $np(1 - p)$ , where  $p$  is the channel transition probability. Hence, the probability of  $i$  errors occurring in a block of  $n$  bits is

$$P_i = \binom{n}{i} p^i (1 - p)^{n-i} \quad (3.18)$$

If a  $t$ -error-correcting block code is used for error correction on a BSC with a transition probability  $p$ , the probability that the decoder fails to decode properly is upper-bounded by the expression:

$$P(E) \leq \sum_{i=t+1}^n \binom{n}{i} p^i (1 - p)^{n-i} \quad (3.19)$$

The equality in Eq. (3.19) holds if the linear block code is a perfect code. Since the code has  $2^{n-k}$  cosets, a perfect code has the property that its standard array (to be discussed in Sec. 3.5) has all the error patterns of  $t$  or fewer errors and no others as coset leaders. Hamming codes which have the parameters  $n = 2^{n-k} - 1$ ,  $d_{\min} = 3$ , and  $t = 1$  form a class of single-error-correcting perfect codes. But perfect codes are rare. Besides the Hamming codes, the (23, 12) Golay code having  $d_{\min} = 7$  and  $t = 3$  is the only nontrivial perfect code.

Suppose that we place a small sphere of radius  $t$  around each of the possible transmitted code words. In that case these spheres are increased in radius by the integer amounts until they cannot be made larger without causing some spheres to intersect. This value of the radius, called the packing radius of the code, is equal to the number of errors that can be corrected by the code. For a  $t$ -error-correcting block code, the number of code words in a sphere of radius  $t = \lfloor (d_{\min} - 1)/2 \rfloor$  is

$$1 + n + \binom{n}{2} + \cdots + \binom{n}{i} = \sum_{i=0}^t \binom{n}{i}$$

where  $\binom{n}{i}$  is the  $i$ th binomial coefficient, which designates that there are exactly  $n$  patterns that differ from the correct one in  $i$  positions. Since there are  $2^k$  possible transmitted code words, there are  $2^k$  nonoverlapping spheres, each having a radius  $t$ . The total number of code words enclosed in the  $2^k$  spheres cannot exceed the  $2^n$  possible

received code words. Thus, a  $t$ -error-correcting code must satisfy the following inequality:

$$2^n \geq 2^k \sum_{i=0}^t \binom{n}{i}$$

or

$$2^{n-k} \geq \sum_{i=0}^t \binom{n}{i} \quad (3.20)$$

This is called the Hamming bound, or sphere-packing bound. A perfect code satisfies the Hamming bound with equality. Equation (3.20) can also be expressed in terms of the amount of redundancy that is required for a fixed value of  $k$  and  $t$ :

$$n - k \geq \log_2 \left[ \sum_{i=0}^t \binom{n}{i} \right] \quad (3.21)$$

For nonbinary codes this bound generalizes to

$$n - k \geq \log_q \left[ \sum_{i=0}^t \binom{n}{i} (q - 1)^i \right] \quad (3.22)$$

where  $q$  is the number of symbols in the code alphabet.

### 3.3.3 The Plotkin bound

The Plotkin bound is also an upper bound on the minimum distance that can be achieved for a given  $n$  and  $k$ . The principle of the Plotkin bound is that the minimum distance between any pair of code words in a code is no larger than the average distance of all possible nonzero code words.

Consider an  $(n, k)$  code whose  $2^k$  code words are  $\mathbf{c}_0, \mathbf{c}_1, \mathbf{c}_2, \dots, \mathbf{c}_{2^k-1}$  in which  $\mathbf{c}_i = (c_{i0}, c_{i1}, \dots, c_{i,n-1})$ . In order to derive this bound, let us introduce a  $2^k \times n$  array whose rows are the code words as follows:

$$\begin{array}{ll} \mathbf{c}_0 & c_{00}, c_{01}, \dots, c_{0,n-1} \\ \mathbf{c}_1 & c_{10}, c_{11}, \dots, c_{1,n-1} \\ \vdots & \\ \mathbf{c}_i & c_{i0}, c_{i1}, \dots, c_{i,n-1} \\ \vdots & \\ \mathbf{c}_j & c_{j0}, c_{j1}, \dots, c_{j,n-1} \\ \vdots & \\ \mathbf{c}_{2^k-1} & c_{2^k-1,0}, c_{2^k-1,1}, \dots, c_{2^k-1,n-1} \end{array}$$

Taking the sum of all pairwise Hamming distances gives

$$\sum_{i=0}^{2^k-1} \sum_{j=0}^{2^k-1} d(\mathbf{c}_i, \mathbf{c}_j) = \sum_{\lambda=0}^{n-1} \sum_{i=0}^{2^k-1} \sum_{j=0}^{2^k-1} d(c_{i\lambda}, c_{j\lambda}) \quad (3.23)$$

where the bit distance between two digits  $c_{i\lambda}$  and  $c_{j\lambda}$  is designated as

$$d(c_{i\lambda}, c_{j\lambda}) = w(c_{i\lambda} + c_{j\lambda}) = \begin{cases} 0 & \text{when } c_{i\lambda} = c_{j\lambda} \\ 1 & \text{otherwise} \end{cases}$$

Now, if  $N(\lambda)$  designates the number of 0s in the  $\lambda$ th column, then  $N(\lambda) < 2^k$  must hold, because  $N(\lambda) = 2^k$  indicates all zero elements in the  $\lambda$ th column. For each column  $\lambda$ , there exists an  $\iota$ ,  $0 < \iota \leq 2^k - 1$ , for which a bit  $c_{\iota\lambda} = 1$ , whereas there is an  $N(\lambda)$  of  $m$  for which  $c_{m\lambda} = 0$ . Hence, we have

$$\sum_{m=0}^{2^k-1} d(c_{\iota\lambda}, c_{m\lambda}) = N(\lambda), \quad c_{\iota\lambda} \neq c_{m\lambda} \quad (3.24)$$

In addition, the number of 1s in the  $\lambda$ th column is  $2^k - N(\lambda)$  for which  $c_{\iota\lambda} = 1$ . Thus, we get

$$\sum_{\iota: c_{\iota\lambda}=1} \sum_{m=0}^{2^k-1} d(c_{\iota\lambda}, c_{m\lambda}) = [2^k - N(\lambda)]N(\lambda) \quad (3.25)$$

On the other hand, there is an  $N(\lambda)$  of  $\iota$  for which  $c_{\iota\lambda} = 0$  and consequently  $2^k - N(\lambda)$  of  $m$  for which  $c_{m\lambda} = 1$ . Thus, we obtain

$$\sum_{\iota: c_{\iota\lambda}=0} \sum_{m=0}^{2^k-1} d(c_{\iota\lambda}, c_{m\lambda}) = N(\lambda)[2^k - N(\lambda)] \quad (3.26)$$

Combining Eqs. (3.25) and (3.26), yields

$$\sum_{\iota=0}^{2^k-1} \sum_{m=0}^{2^k-1} d(c_{\iota\lambda}, c_{m\lambda}) = 2N(\lambda)[2^k - N(\lambda)] \quad (3.27)$$

But the factor  $N(\lambda)[2^k - N(\lambda)]$  has a maximum value at  $N(\lambda) = 2^{k-1}$ . Hence, Eq. (3.27) can be expressed as

$$\sum_{\iota=0}^{2^k-1} \sum_{m=0}^{2^k-1} d(c_{\iota\lambda}, c_{m\lambda}) \leq \frac{(2^k)^2}{2} \quad (3.28)$$

Comparing Eq. (3.28) with Eq. (3.23), we get

$$\sum_{\iota=0}^{2^k-1} \sum_{m=0}^{2^k-1} d(\mathbf{c}_\iota, \mathbf{c}_m) = \sum_{\lambda=0}^{n-1} 2N(\lambda)[2^k - N(\lambda)] \leq \frac{n(2^k)^2}{2} \quad (3.29)$$

Since  $d_{\min} = \min d(\mathbf{c}_i, \mathbf{c}_m)_{i \neq m} = \min [d(\mathbf{c}_i, \mathbf{c}_m) \mid \mathbf{c}_i \neq \mathbf{c}_m; \mathbf{c}_i, \mathbf{c}_m \in C]$ , we have

$$\begin{aligned} \sum_{i=0}^{2^k-1} \sum_{m=0}^{2^k-1} d(\mathbf{c}_i, \mathbf{c}_m) &= \sum_{i=0}^{2^k-1} \sum_{i \neq m} d(\mathbf{c}_i, \mathbf{c}_m) \\ &\geq 2^k(2^k - 1)d_{\min} \end{aligned} \quad (3.30)$$

Combining the inequalities (3.29) and (3.30), there exists a linear  $(n, k)$  block code over  $GF(2)$  with a minimum distance such that

$$d_{\min} \leq \left(\frac{n}{2}\right) \left(\frac{2^k}{2^k - 1}\right) \quad (3.31)$$

This is the Plotkin upper bound on minimum distance when the code is binary. Equation (3.31) can also be explained as follows. With a  $2^k \times n$  array whose rows are the code words, each nonzero element appears  $2^{k-1}$  times in each column of the array. There are  $n$  columns in the array, and the sum of the weights of all code words is  $n2^{k-1}$ . Since there are  $2^k - 1$  nonzero code words and since the weight of a minimum weight code word cannot be greater than the average weight, the following inequality holds:

$$d_{\min} \leq \frac{n2^{k-1}}{2^k - 1}$$

For nonbinary codes, the Plotkin bound generalizes to

$$d_{\min} \leq \frac{n(q-1)q^{k-1}}{q^k - 1} \quad (3.32)$$

### 3.3.4 The Gilbert-Varshamov bound

The Gilbert-Varshamov lower bound on linear codes is discussed in this subsection. In order to illustrate the existence of a linear  $(n, k)$  code with minimum distance  $d_{\min}$  over  $GF(2)$ , it is only necessary to show the existence of an  $(n - k) \times n$  matrix  $\mathbf{H}$  such that no set of  $d_{\min} - 1$  or fewer columns is linearly dependent. This concept certainly provides a means of establishing the minimum distance for a specific block code when the parity-check matrix  $\mathbf{H}$  is given. For finding such a matrix  $\mathbf{H}$ , we can choose the first column as any of  $2^{n-k} - 1$  nonzero  $(n - k)$ -tuples over  $GF(2)$ . The second column can be chosen as any nonzero  $(n - k)$ -tuple that is not a scalar multiple of the first column. The third column may be any  $(n - k)$ -tuple which is not a linear combination of the first two. Continue in this manner, choosing successive columns so that each new column is not a linear combina-



tion of any  $d_{\min} - 2$  or fewer previous columns. In general, suppose  $n - 1$  columns have been chosen so that any  $d_{\min} - 1$  of them are linearly independent. In the worst possible case, every one of these linear combinations is a distinct nonzero element. There are at most

$$\binom{n-1}{1} + \binom{n-1}{2} + \cdots + \binom{n-1}{d_{\min}-2}$$

distinct linear combinations of these  $n - 1$  columns taken  $d_{\min} - 2$  or fewer at a time. Provided this number is less than  $2^{n-k} - 1$ , we can add another column that is different from these linear combinations and keep the property that any  $d_{\min} - 1$  columns of the new  $(n - k) \times n$  matrix are linearly independent. Thus, there exists a binary linear code of length  $n$ , with at most  $n - k$  parity-check bits and a minimum distance of at least  $d_{\min}$ , provided that

$$\sum_{i=0}^{d_{\min}-2} \binom{n-1}{i} < 2^{n-k} \quad (3.33)$$

This is called the Gilbert-Varshamov lower bound on linear binary codes. A nonbinary version of the Gilbert-Varshamov bound can also be written as

$$\sum_{i=0}^{d_{\min}-2} \binom{n-1}{i} (q-1)^i < q^{n-k} \quad (3.34)$$

Then we get

$$\sum_{i=0}^{d_{\min}-2} \binom{n}{i} (q-1)^i \geq q^{n-k}$$

which simply gives us the inequality

$$n - k \leq \log_q \left[ 1 + \binom{n}{1}(q-1) + \binom{n}{2}(q-1)^2 + \cdots + \binom{n}{d_{\min}-2}(q-1)^{d_{\min}-2} \right] \quad (3.35)$$

**Example 3.5** From the (127, 113) BCH code with  $d_{\min} = 5$  and  $t = 2$ , find the largest possible value of  $k$  by making use of the coding bounds discussed above. The Hamming and Gilbert bounds will be used to estimate how close the code is to the best possible one. The Hamming bound states that

$$\sum_{i=0}^2 \binom{127}{i} \leq 2^{n-k}$$

or

$$8129 \leq 2^{n-k} \quad (3.36)$$

Thus, the smallest number of parity-check bits for which Eq. (3.36) holds is 13, which corresponds to  $k = 114$ . Next, the Gilbert lower bound from Eq. (3.35) for  $q = 2$  states that

$$\sum_{i=0}^3 \binom{127}{i} \geq 2^{n-k}$$

$$\text{or} \quad 333,502 \geq 2^{n-k} \quad (3.37)$$

The largest number of parity-check bits for which Eq. (3.37) holds is 18, which corresponds to  $k = 109$ . Thus, from the Hamming and Gilbert bounds, it follows that  $n = 127$ ,  $d_{\min} = 5$ , and  $t = 2$  codes exist with the information bits in the range of  $109 < k < 114$ . Therefore, it can be concluded that a (127, 113) BCH code is a fairly good code, even though a little improvement could be made with further searching.

As you can see from the above example, a number of bounds are known to determine the best possible error-correcting code of a certain information sequence which is unknown. No codes can exist beyond such bounds, and codes are sure to exist within such bounds. Therefore, a knowledge of these bounds certainly deepens one's understanding of coding theory.

### 3.4 The Syndrome

Let  $\mathbf{c} = (c_0, c_1, \dots, c_{n-1})$  be a code word to be transmitted and  $\mathbf{r} = (r_0, r_1, \dots, r_{n-1})$  be the received word at the output of the demodulator. Word  $\mathbf{r}$  may or may not be different from  $\mathbf{c}$  depending on the noise introduced by the channel. If  $\mathbf{r} \neq \mathbf{c}$ , we have  $\mathbf{r} = \mathbf{c} + \mathbf{e}$ , where

$$\begin{aligned} \mathbf{e} &= \mathbf{r} + \mathbf{c} \\ &= (e_0, e_1, \dots, e_{n-1}) \end{aligned} \quad (3.38)$$

and  $\mathbf{e}$  is called the  $n$ -tuple error pattern.

Since  $e_i = 1$ ,  $0 \leq i \leq n-1$ , for  $r_i \neq c_i$ , the 1s in  $\mathbf{e}$  are the number of error bits caused by the channel noise.

Upon receiving  $\mathbf{r}$ , the decoder starts to compute the syndrome, to locate the errors, and to correct them. The syndrome, denoted by  $\mathbf{s}$ , of  $\mathbf{r}$  is defined to be the following  $(n-k)$ -tuple row vector:

$$\begin{aligned} \mathbf{s} &= \mathbf{r} \cdot \mathbf{H}^T \\ &= (s_0, s_1, \dots, s_{n-k-1}) \end{aligned} \quad (3.39)$$

Since  $\mathbf{r}$  is the vector sum of  $\mathbf{c}$  and  $\mathbf{e}$ , it follows that

$$\mathbf{s} = (\mathbf{c} + \mathbf{e})\mathbf{H}^T = \mathbf{e} \cdot \mathbf{H}^T \quad (3.40)$$

because  $\mathbf{c} \cdot \mathbf{H}^T = \mathbf{0}$ . Equation (3.40) shows the relation between the syndrome and the error pattern. Thus, the parity-check matrix permits us to determine a quantity which depends only upon the error pattern. As shown in Eqs. (3.39) and (3.40), the syndrome  $\mathbf{s}$  is not only computed from the received word  $\mathbf{r}$ , but it also depends upon the actual error pattern  $\mathbf{e}$ . The fundamental importance of the syndrome stems from its use in error detection and error correction. Clearly,  $\mathbf{s} = \mathbf{0}$  if and only if  $\mathbf{e} = \mathbf{0}$ . Hence, the decoder accepts  $\mathbf{r}$  as the transmitted code word  $\mathbf{c}$ . When  $\mathbf{s} \neq \mathbf{0}$ ,  $\mathbf{r}$  is not a code word and the presence of errors has been detected.

**Example 3.6** For the code in Example 3.4, let  $\mathbf{r} = (1\ 1\ 1\ 0\ 0\ 1\ 0)$  be the received vector. Then, the syndrome is immediately obtained as

$$\mathbf{s} = (1\ 1\ 1\ 0\ 0\ 1\ 0) \cdot \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix} = (0\ 0\ 1)$$

In general, for the received word  $\mathbf{r} = (r_0, r_1, \dots, r_{n-1})$ , the syndrome is given by

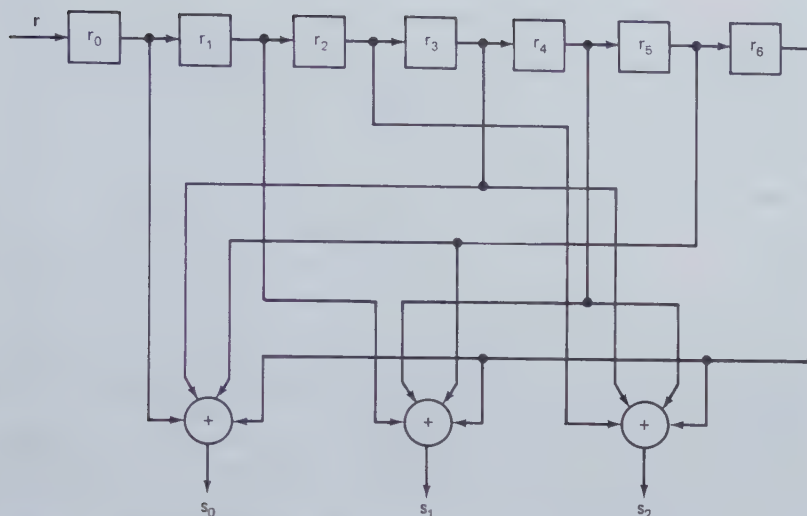
$$\begin{aligned} \mathbf{s} &= (s_0, s_1, s_2) \\ &= (r_0, r_1, r_2, r_3, r_4, r_5, r_6) \cdot \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix} \end{aligned}$$

Thus, the syndrome bits are

$$s_0 = r_0 + r_3 + r_5 + r_6$$

$$s_1 = r_1 + r_4 + r_5 + r_6$$

$$s_2 = r_2 + r_3 + r_4 + r_6$$



**Figure 3.2** Syndrome generator for the (7, 4) linear systematic code.

Thus, for the received bits  $r_0 = r_1 = r_2 = 1$ ,  $r_3 = r_4 = 0$ ,  $r_5 = 1$ , and  $r_6 = 0$ , the syndrome bits are  $s_0 = 0$ ,  $s_1 = 0$ , and  $s_2 = 1$ . The syndrome register, or generator, for this code is shown in Fig. 3.2.

**Example 3.7** Consider the (6, 3) linear code with the parity-check matrix

$$\mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 & 1 \end{bmatrix}$$

Let  $\mathbf{c} = (1 \ 1 \ 0 \ 0 \ 1 \ 0)$  be the transmitted code word and  $\mathbf{r} = (1 \ 1 \ 0 \ 1 \ 1 \ 0)$  be the received word. Upon receiving  $\mathbf{r}$ , the decoder first starts to compute the syndrome as

$$\mathbf{s} = \mathbf{r} \cdot \mathbf{H}^T = (1 \ 0 \ 1)$$

The decoder then attempts to determine the corresponding error pattern  $\mathbf{e} = (e_0, e_1, \dots, e_5)$ . Thus, it follows from Eq. (3.40) that the error bits are related to the syndrome bits by the following linear equations:

$$e_0 + e_3 + e_4 = 1$$

$$e_1 + e_4 + e_5 = 0$$

$$e_2 + e_3 + e_5 = 1$$

These  $n - k = 3$  equations have no unique solution but rather  $2^k = 8$  possible solutions. Assuming that  $e_3, e_4, e_5$  are chosen as shown in the first column of Table 3.3,  $e_0, e_1, e_2$  are immediately determined as shown in the second column.



TABLE 3.3 Eight Possible Error Patterns for the (6, 3) Linear Code

| $e_3$ | $e_4$ | $e_5$ | $e_0$ | $e_1$ | $e_2$ | $e_0$ | $e_1$ | $e_2$ | $e_3$ | $e_4$ | $e_5$ |
|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|
| 0     | 0     | 0     | 1     | 0     | 1     | 1     | 0     | 1     | 0     | 0     | 0     |
| 0     | 0     | 1     | 1     | 1     | 0     | 1     | 1     | 0     | 0     | 0     | 1     |
| 0     | 1     | 0     | 0     | 1     | 1     | 0     | 1     | 1     | 0     | 1     | 0     |
| 0     | 1     | 1     | 0     | 0     | 0     | 0     | 0     | 0     | 0     | 1     | 1     |
| 1     | 0     | 0     | 0     | 0     | 0     | 0     | 0     | 0     | 1     | 0     | 0     |
| 1     | 0     | 1     | 0     | 1     | 1     | 0     | 1     | 1     | 1     | 0     | 1     |
| 1     | 1     | 0     | 1     | 1     | 0     | 1     | 1     | 0     | 1     | 1     | 0     |
| 1     | 1     | 1     | 1     | 0     | 1     | 1     | 0     | 1     | 1     | 1     | 1     |

Accordingly, the  $2^3 (= 8)$  error patterns that satisfy the equations above are listed in the third column of the table.

If the channel is a BSC, the minimum-weight element is usually chosen as the true error pattern, i.e.,  $\mathbf{e} = (0\ 0\ 0\ 1\ 0\ 0)$  is the most probable error pattern. Once the error pattern  $\mathbf{e}$  has been found, the vector  $\mathbf{r} + \mathbf{e}$  is used as the actual transmitted code word. Using  $\mathbf{e} = (0\ 0\ 0\ 1\ 0\ 0)$  as the true error pattern, the decoder will decode  $\mathbf{r} = (1\ 1\ 0\ 1\ 1\ 0)$  into the following code word:

$$\begin{aligned}
 \mathbf{c} &= \mathbf{r} + \mathbf{e} \\
 &= (1\ 1\ 0\ 1\ 1\ 0) + (0\ 0\ 0\ 1\ 0\ 0) \\
 &= (1\ 1\ 0\ 0\ 1\ 0)
 \end{aligned}$$

which is the actual transmitted code word as expected.

### 3.5 The Standard Array

The standard array is a way of tabulating all of the  $2^n$   $n$ -tuples over  $GF(2)$ . This  $2^{n-k} \times 2^k$  rectangular array forms the conceptual basis for decoding linear block codes. The standard array can be used as a decoding table by simply locating the received  $n$ -tuple word in the table. This  $n$ -tuple in the  $i$ th column is equal to the sum of the code word heading the  $i$ th column and the correctable error pattern appearing in the same row in the first (leftmost) column. Unfortunately, the size of the array grows exponentially with the code length, so even for modest values of  $n$  and  $k$ , the direct use of the standard array is impractical for large codes. However, as a conceptual device, the decoding table is quite useful in illuminating some of the important properties of block codes. In group theory, the rows of this array are called cosets and the element in the leftmost column is called the coset leader.

Let  $C$  be a given  $(n, k)$  linear code in  $GF(2)$ , which we treat as a vector space. Let  $\mathbf{c}_1, \mathbf{c}_2, \dots, \mathbf{c}_{2^k}$  be the  $2^k$  code words of  $C$ . The factor space  $\{\mathbf{e}_i + C\}$ , where  $\mathbf{e}_i$  are coset leaders, has significance in decoding. The  $n$ -tuples of cosets are listed in the rows of an array. The following steps should be taken to form the standard array of Fig. 3.3. First,

| $\mathbf{c}_1 = \mathbf{0}$ | $\mathbf{c}_2$                        | ... | $\mathbf{c}_j$                        | ... | $\mathbf{c}_{2^k}$                        |
|-----------------------------|---------------------------------------|-----|---------------------------------------|-----|-------------------------------------------|
| $\mathbf{e}_2$              | $\mathbf{e}_2 + \mathbf{c}_2$         | ... | $\mathbf{e}_2 + \mathbf{c}_j$         | ... | $\mathbf{e}_2 + \mathbf{c}_{2^k}$         |
| $\mathbf{e}_3$              | $\mathbf{e}_3 + \mathbf{c}_2$         | ... | $\mathbf{e}_3 + \mathbf{c}_j$         | ... | $\mathbf{e}_3 + \mathbf{c}_{2^k}$         |
| $\vdots$                    | $\vdots$                              |     | $\vdots$                              |     | $\vdots$                                  |
| $\mathbf{e}_i$              | $\mathbf{e}_i + \mathbf{c}_2$         | ... | $\mathbf{e}_i + \mathbf{c}_j$         | ... | $\mathbf{e}_i + \mathbf{c}_{2^k}$         |
| $\vdots$                    | $\vdots$                              |     | $\vdots$                              |     | $\vdots$                                  |
| $\mathbf{e}_{2^{n-k}}$      | $\mathbf{e}_{2^{n-k}} + \mathbf{c}_2$ | ... | $\mathbf{e}_{2^{n-k}} + \mathbf{c}_j$ | ... | $\mathbf{e}_{2^{n-k}} + \mathbf{c}_{2^k}$ |

Figure 3.3 Standard array for an  $(n, k)$  linear code.

place the  $2^k$  code words of  $C$  in the first (top) row with the all-zero word  $\mathbf{c}_1 = (0\ 0\ \cdots\ 0)$  as the leftmost element. From the remaining  $2^n - 2^k$   $n$ -tuples, an  $n$ -tuple  $\mathbf{e}_2$  is chosen and is placed under the all-zero word  $\mathbf{c}_1$ . Now, the second row is formed by adding  $\mathbf{e}_2$  to each code word  $\mathbf{c}_i$ ,  $2 \leq i \leq 2^k$ , in the top row and placing the sum  $\mathbf{e}_2 + \mathbf{c}_i$  under  $\mathbf{c}_i$ . Having completed the second row, an unused  $n$ -tuple  $\mathbf{e}_3$  is chosen from the remaining  $n$ -tuples and is placed under  $\mathbf{e}_2$ . Then a third row is formed by adding  $\mathbf{e}_3$  to each code word in the top row and placing  $\mathbf{e}_3 + \mathbf{c}_i$  under  $\mathbf{c}_i$ . This process is continued until all the  $n$ -tuples are used up so that each possible  $n$ -tuple appears somewhere in the array. Thus, we have a rectangular array of rows and columns as shown in Fig. 3.3.

**Example 3.8** Consider the  $(6, 3)$  linear code generated by the following matrix:

$$\mathbf{G} = \begin{bmatrix} 0 & 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 & 1 \end{bmatrix}$$

This code with minimum distance  $d_{\min} = 3$  corrects one error. The standard array of this single-error-correcting code is shown in Table 3.4.

TABLE 3.4 Standard Array for the  $(6, 3)$  Single-Error-Correcting Code

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 000000 | 110001 | 101010 | 011011 | 011100 | 101101 | 110110 | 000111 |
| 000001 | 110000 | 101011 | 011010 | 011101 | 101100 | 110111 | 000110 |
| 000010 | 110011 | 101000 | 011001 | 011110 | 101111 | 110100 | 000101 |
| 000100 | 110101 | 101110 | 011111 | 011000 | 101001 | 110010 | 000011 |
| 001000 | 111001 | 100010 | 010011 | 010100 | 100101 | 111110 | 001111 |
| 010000 | 100001 | 111010 | 001011 | 001100 | 111101 | 100110 | 010111 |
| 100000 | 010001 | 001010 | 111011 | 111100 | 001101 | 010110 | 100111 |
| 001001 | 111000 | 100011 | 010010 | 010101 | 100100 | 111111 | 001110 |

As can be observed from Table 3.4, all the 6-tuples of weight  $t = \lfloor (d_{\min} - 1)/2 \rfloor = 1$  or less are used as coset leaders. However, there is at least one 6-tuple of weight  $t + 1 = 2$  in the leftmost column, but it cannot be used as a coset leader for minimum-distance decoding.

In general, an  $(n, k)$  linear code with minimum distance  $d_{\min}$  is capable of correcting all the error patterns of  $t = \lfloor (d_{\min} - 1)/2 \rfloor$  or fewer errors. However, if more than  $t$  errors occur, then errors may occur in the decoding; that is, the code cannot correct all the error patterns of weight  $t + 1$ . Each coset leader is a minimum weight vector of its coset. Essentially the coset leaders represent correctable error patterns.

If  $\mathbf{r}$  is the received  $n$ -tuple word,  $\mathbf{r} \cdot \mathbf{H}^T$  represents the  $(n - k)$ -tuple syndrome of  $\mathbf{r}$ . The syndrome plays an important role in decoding a received word  $\mathbf{r}$ . Consider the coset whose coset leader is  $\mathbf{e}_k$ . An element, or a received word, in this coset is the sum of  $\mathbf{e}_k$  and some code word  $\mathbf{c}_i$  in the top row of the array. The syndrome of this word is then

$$(\mathbf{e}_k + \mathbf{c}_i)\mathbf{H}^T = \mathbf{e}_k\mathbf{H}^T$$

Therefore, we see that the syndrome of any element in a coset is equal to the syndrome of the coset leader. Thus, all the  $n$ -tuples have the same syndrome if and only if they belong to the same coset. Thus, if an  $\mathbf{r}$  is received that has the error pattern  $\mathbf{e}_k$ , then  $\mathbf{r}$  and  $\mathbf{e}_k$  have the same syndrome. It follows, therefore, that for maximum-likelihood decoding of  $\mathbf{r}$ , one must choose a coset leader  $\mathbf{e}_k$  of minimum weight in the coset which contains  $\mathbf{r}$  and then decode  $\mathbf{r}$  as  $\mathbf{r} + \mathbf{e}_k$ .

Consider the case where  $\mathbf{e}_k$  and  $\mathbf{e}_l$  are the coset leaders of the  $k$ th and  $l$ th cosets, respectively. The syndromes for different cosets are not equal. To prove this: If the syndromes of these two cosets are assumed to be equal, then we have

$$(\mathbf{e}_k + \mathbf{e}_l)\mathbf{H}^T = \mathbf{0}$$

This implies that  $\mathbf{e}_k + \mathbf{e}_l$  must be a code word  $\mathbf{c}_j$ . Hence,  $\mathbf{e}_l = \mathbf{e}_k + \mathbf{e}_j$  implies that  $\mathbf{e}_l$  is in the  $k$ th coset, which contradicts the coset decomposition rule. Therefore, no two cosets have the same syndrome.

If the syndromes of the coset leaders are known, then the table-lookup decoding may be simplified as follows. For the received word  $\mathbf{r}$ , (1) calculate the syndrome  $\mathbf{s}$  and (2) find the coset leader  $\mathbf{e}_k$  with the same syndrome; then  $\mathbf{r} + \mathbf{e}_k$  is assumed to be the code word transmitted.

**Example 3.9** Consider the  $(6, 3)$  linear code with the parity-check matrix of

$$\mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 & 0 \end{bmatrix}$$

Since there exist  $2^{n-k} = 8$  cosets, we have eight coset leaders, or correctable error patterns. The coset leaders and their corresponding syndromes are obtained from the formula  $\mathbf{s} = \mathbf{eH}^T$ . They are listed in Table 3.5.

**TABLE 3.5 Coset Leader versus Syndrome Table for the (6, 3) Linear Code**

| Coset leaders (correctable error patterns) |       |       |       |       |       | Syndromes |       |       |
|--------------------------------------------|-------|-------|-------|-------|-------|-----------|-------|-------|
| $e_0$                                      | $e_1$ | $e_2$ | $e_3$ | $e_4$ | $e_5$ | $s_0$     | $s_1$ | $s_2$ |
| 0                                          | 0     | 0     | 0     | 0     | 0     | 0         | 0     | 0     |
| 1                                          | 0     | 0     | 0     | 0     | 0     | 1         | 0     | 0     |
| 0                                          | 1     | 0     | 0     | 0     | 0     | 0         | 1     | 0     |
| 0                                          | 0     | 1     | 0     | 0     | 0     | 0         | 0     | 1     |
| 0                                          | 0     | 0     | 1     | 0     | 0     | 1         | 0     | 1     |
| 0                                          | 0     | 0     | 0     | 1     | 0     | 0         | 1     | 1     |
| 0                                          | 0     | 0     | 0     | 0     | 1     | 1         | 1     | 0     |
| 1                                          | 0     | 0     | 0     | 1     | 0     | 1         | 1     | 1     |

Assume that  $\mathbf{r} = (1\ 0\ 1\ 1\ 0\ 1)$  is received when  $\mathbf{c} = (1\ 0\ 1\ 1\ 0\ 0)$  is transmitted. Note that a single error has occurred because of channel noise. For a single-error correction, the syndrome of  $\mathbf{r}$  can be calculated as follows:

$$\mathbf{s} = (1\ 0\ 1\ 1\ 0\ 1) \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 0 \end{bmatrix} = (1\ 1\ 0)$$

Using Table 3.5,  $(1\ 1\ 0)$  is the syndrome corresponding to the coset leader  $\mathbf{e} = (0\ 0\ 0\ 0\ 0\ 1)$ , which is assumed to be the error pattern caused by the channel noise. Accordingly,  $\mathbf{r}$  is decoded into

$$\begin{aligned} \mathbf{c} &= \mathbf{r} + \mathbf{e} \\ &= (1\ 0\ 1\ 1\ 0\ 1) + (0\ 0\ 0\ 0\ 0\ 1) \\ &= (1\ 0\ 1\ 1\ 0\ 0) \end{aligned}$$

which is the actual code word transmitted.

Next, let us suppose that the all-zero word  $\mathbf{c} = (0\ 0\ 0\ 0\ 0\ 0)$  is transmitted and  $\mathbf{r} = (0\ 0\ 1\ 0\ 1\ 0)$  is received. Note that double errors have occurred during the transmission of  $\mathbf{c}$ . Upon receiving  $\mathbf{r}$ , the decoder calculates the syndrome as

$$\mathbf{s} = \mathbf{rH}^T = (0\ 1\ 0)$$



From Table 3.5 we find the coset leader  $\mathbf{e} = (0\ 1\ 0\ 0\ 0\ 0)$  which corresponds to the syndrome  $\mathbf{s} = (0\ 1\ 0)$ . Thus,  $\mathbf{r}$  is decoded into the code word

$$\begin{aligned}\mathbf{c} &= \mathbf{r} + \mathbf{e} \\ &= (0\ 0\ 1\ 0\ 1\ 0) + (0\ 1\ 0\ 0\ 0\ 0) \\ &= (0\ 1\ 1\ 0\ 1\ 0)\end{aligned}$$

Since  $\mathbf{c}$  is not the actual code word transmitted, the error pattern is not correctable and a decoding error has been committed.

Now let us consider the case where  $\mathbf{c} = (1\ 0\ 1\ 1\ 0\ 0)$  is transmitted and  $\mathbf{r} = (1\ 0\ 1\ 0\ 0\ 1)$  is received. The syndrome of  $\mathbf{r}$  is  $\mathbf{s} = (0\ 1\ 1)$ . The error pattern corresponding to the syndrome  $\mathbf{s} = (0\ 1\ 1)$  is  $\mathbf{e} = (0\ 0\ 0\ 0\ 1\ 0)$ . As a result,  $\mathbf{c} = \mathbf{r} + \mathbf{e} = (1\ 0\ 1\ 0\ 0\ 1) + (0\ 0\ 0\ 0\ 1\ 0) = (1\ 0\ 1\ 0\ 1\ 1)$  is not the code word actually transmitted. Now, it is clear that the  $(6, 3)$  code is capable of correcting only a single error over a given code length. The decoder for this  $(6, 3)$  linear code is shown in Fig. 3.4.

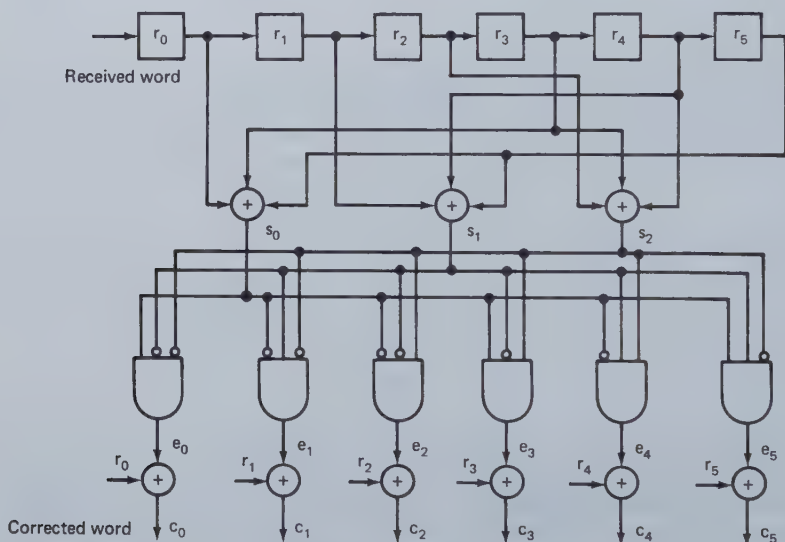


Figure 3.4 Decoder for the  $(6, 3)$  code given in Table 3.5.

### 3.6 Weight Enumerators and Probability of Decoding Error

Let  $C$  be an  $(n, k)$  linear code. The minimum distance of  $C$  is the weight of the minimum weight code word, which represents a critical parameter in the evaluation of  $C$ . However, in most practical coding analyses it is important to know the actual distribution for weights of the code words. Unfortunately, a complete weight structure for most classes of codes is not known.

Let  $A_i$  be the number of code words of weight  $i$  in  $C$ . The numbers  $A_0, A_1, \dots, A_n$  are called the weight distributions of  $C$ . Obviously, if the minimum distance is  $d_{\min}$ , then  $A_0 = 1, A_1, \dots, A_{d_{\min}-1}$  are all zero, and  $A_{d_{\min}}$  is not zero. In general, it is possible to determine the weight distribution of a linear code by simply examining all  $2^k$  code words, provided  $k$  is relatively small. But, if  $k$  is greater than  $n/2$ , then it may be easier to determine the weight structure of the dual code, which contains fewer words. For small  $n$ , the weight distribution can be found by a computer search, but for large  $n$  this quickly becomes impractical.

There exists an interesting relationship between the weight distribution of a linear code  $C$  and the weight distribution of its dual code  $C_D$ . The MacWilliams identity gives a relation between the weight enumerator of a linear code and that of its dual code. We define the weight enumerator  $A(z)$  of a code  $C$  to be

$$A(z) = \sum_{i=0}^n A_i z^i \quad (3.41)$$

if  $C$  contains  $A_i$  code words of weight  $i$ . The weight enumerator  $B(z)$  of its dual code  $C_D$  is formed from

$$B(z) = \sum_{i=0}^n B_i z^i \quad (3.42)$$

From MacWilliams (1962),  $A(z)$  and  $B(z)$  are related by the following identity:

$$\begin{aligned} B(z) &= 2^{-k} \sum_{i=0}^n A_i (1+z)^{n-i} (1-z)^i \\ &= 2^{-k} (1+z)^n A\left(\frac{1-z}{1+z}\right) \end{aligned} \quad (3.43)$$

$$\text{or} \quad A(z) = 2^{-(n-k)} (1+z)^n B\left(\frac{1-z}{1+z}\right) \quad (3.44)$$

These identities, Eqs. (3.43) and (3.44), are known as the MacWilliams identities, and it can be seen that the weight distribution of the code  $C$  can be determined if the weight distribution of its dual code  $C_D$  is known and vice versa. The weight distribution has a number of applications in the study of codes. The following example is a simple application for computing the weight distribution of an  $(n, k)$  code.

**Example 3.10** Consider the  $(7, 4)$  linear Hamming code given in Table 3.2. The set of weight distributions of this code is given by

$$\{A_0, A_1, A_2, \dots, A_7\} = \{1, 0, 0, 7, 7, 0, 0, 1\}$$

and thus, the weight enumerator of this code is

$$A(z) = 1 + 7z^3 + 7z^4 + z^7$$

By the MacWilliams identity, the weight enumerator of the (7, 3) dual code is given by

$$\begin{aligned} B(z) &= 2^{-4} (1+z)^7 A\left(\frac{1-z}{1+z}\right) \\ &= 2^{-4} [(1+z)^7 + 7(1-z)^3 (1+z)^4 + 7(1-z)^4 (1+z)^3 + (1-z)^7] \end{aligned}$$

which reduces to

$$B(z) = 1 + 7z^4$$

This is the weight enumerator of the (7, 3) dual code which has one code word of weight 0 and seven code words of weight 4.

There is, of course, another way of evaluating the weight enumerator for its dual code as shown in the following discussion. Again referring to Example 3.2, the dual of the (7, 4) code is generated by its parity-check matrix:

$$\mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 & 0 & 1 \end{bmatrix}$$

The eight code words generated by  $\mathbf{H}$  in the dual code are

$$\begin{array}{ll} (0 & 0 & 0 & 0 & 0 & 0 & 0) & (1 & 0 & 0 & 1 & 0 & 1 & 1) \\ (0 & 0 & 1 & 1 & 1 & 0 & 1) & (1 & 0 & 1 & 0 & 1 & 1 & 0) \\ (0 & 1 & 0 & 0 & 1 & 1 & 1) & (1 & 1 & 0 & 1 & 1 & 0 & 0) \\ (0 & 1 & 1 & 1 & 0 & 1 & 0) & (1 & 1 & 1 & 0 & 0 & 0 & 1) \end{array}$$

Judging from these code words, the weight distribution of the (7, 3) dual code is  $\{1, 0, 0, 0, 7, 0, 0, 0\}$ . Hence, the weight enumerator for this dual code is immediately obtained as  $B(z) = 1 + 7z^4$ .

Since a BSC is memoryless, the bit errors occur independently. Hence, the probability of  $i$  errors in a code length  $n$  is

$$P_i(E) = \binom{n}{i} p^i (1-p)^{n-i} \quad (3.45)$$

where  $p$  is the transition probability of a BSC.

A decoder that corrects all error patterns of weight  $t$  or less and corrects no error patterns of weight greater than  $t$  is called incomplete, but it is still useful. An incomplete decoder that does not decode be-

yond the packing radius  $t$  of a code will decode every received word to the closest code word provided it is within distance  $t$  of the code word. A decoder that corrects up to  $t$  errors of a code has the following probability of correctly decoding a word:

$$P(c) = \sum_{i=0}^t \binom{n}{i} p^i (1-p)^{n-i} \quad (3.46)$$

When more than  $t$  errors occur, the decoder may commit a decoding error. Hence, the upper bound on the probability that a  $t$ -error-correcting  $(n, k)$  linear code will commit a decoding error is given by

$$P_e(E) \leq \sum_{i=t+1}^n \binom{n}{i} p^i (1-p)^{n-i} \quad (3.47)$$

In other words, the lower bound on the probability of correct decoding by the best  $(n, k)$  code is given by Eq. (3.47). The equality in Eq. (3.47) holds only when an  $(n, k)$  linear code is a perfect code.

The probability expressions are given in terms of the weight distribution  $\{A_i\}$  of the code; thus they are only useful when  $\{A_i\}$  is known. Undetected errors occur only when the decoder fails to detect the presence of errors. If an  $(n, k)$  code is used only for error detection on a BSC, the probability of an undetected error  $P_{ud}(E)$  is computed from the weight distribution  $\{A_i\}$  of  $C$ . That is,

$$P_{ud}(E) = \sum_{i=1}^n A_i p^i (1-p)^{n-i} \quad (3.48)$$

Using the MacWilliams identity, the probability of an undetected error can also be computed from the weight enumerators  $A(z)$  for  $C$  or  $B(z)$  for  $C_D$ . From Eq. (3.48),  $P_{ud}(E)$  can be modified to give

$$\begin{aligned} P_{ud}(E) &= (1-p)^n \sum_{i=1}^n A_i \left( \frac{p}{1-p} \right)^i \\ &= (1-p)^n \left[ A \left( \frac{p}{1-p} \right) - 1 \right] \end{aligned} \quad (3.49)$$

for the weight enumerator of  $C$ ,  $A(z)$ ,  $z = p/(1-p)$ . The probability of an undetected error is also expressed in terms of the weight enumerator for the dual code  $C_D$ ,  $B(z)$ ,  $z = 1-2p$ . Referring to the MacWilliams identity of Eq. (3.44), we have

$$A \left( \frac{p}{1-p} \right) = 2^{-(n-k)} (1-p)^{-n} B(1-2p)$$

Hence, Eq. (3.49) becomes

$$\begin{aligned} P_{\text{ud}}(E) &= 2^{-(n-k)} B(1 - 2p) - (1 - p)^n \\ &= 2^{-(n-k)} \sum_{i=0}^n B_i (1 - 2p)^i - (1 - p)^n \end{aligned} \quad (3.50)$$

**Example 3.11** Consider again the (7, 4) Hamming code whose weight enumerator is  $A(z) = 1 + 7z^3 + 7z^4 + z^7$  (see Example 3.10). We know that the weight distribution of this code is  $A_0 = 1$ ,  $A_3 = A_4 = 7$ , and  $A_7 = 1$ . The probability of an undetected error is derived from Eq. (3.49) as follows:

$$\begin{aligned} P_{\text{ud}}(E) &= (1 - p)^7 \left[ A_3 \left( \frac{p}{1 - p} \right)^3 + A_4 \left( \frac{p}{1 - p} \right)^4 + A_7 \left( \frac{p}{1 - p} \right)^7 \right] \\ &= 7p^3(1 - p)^4 + 7p^4(1 - p)^3 + p^7 \end{aligned}$$

Of course, this probability can also be computed from Eq. (3.48) by using the weight distribution of the code itself. Since  $p \ll 1/2$  over a BSC, let us arbitrarily pick  $p = 10^{-2}$ . Then we compute  $P_{\text{ud}}(E) = 6.79 \times 10^{-6}$ , which tells us that out of 1 million transmitted code digits there are on average seven erroneous code digits that pass through the decoder undetected. On the other hand, since the weight enumerator of the (7, 3) dual code is  $B(z) = 1 + 7z^4$ , the weight distribution of this dual code is  $B_0 = 1$  and  $B_4 = 7$ . By making use of Eq. (3.50), the probability of an undetected error is also expressed by

$$P_{\text{ud}}(E) = 2^{-4} [1 + 7(1 - 2p)^4] - (1 - p)^7$$

For a given  $p = 10^{-2}$ , we have  $P_{\text{ud}}(E) = 6.79 \times 10^{-6}$  which agrees exactly with the result obtained above. The following table indicates that  $P_{\text{ud}}(E)$  decreases monotonously as  $p$  becomes smaller and smaller.

| $p$                | $10^{-1}$            | $10^{-2}$             | $10^{-3}$             | $10^{-4}$           |
|--------------------|----------------------|-----------------------|-----------------------|---------------------|
| $P_{\text{ud}}(E)$ | $5.1 \times 10^{-3}$ | $6.79 \times 10^{-6}$ | $6.98 \times 10^{-9}$ | $7 \times 10^{-12}$ |

### 3.7 Some Specific Linear Block Codes

Although the most important linear cyclic codes are not discussed in this section, several noncyclic block codes that are frequently encountered in practice are briefly described and their important parameters are listed. Five types of linear block codes, including the Hamming codes, the Golay codes, the Hadamard codes, the Reed-Muller codes, and some product codes are introduced here. However, most of them are equivalent to the cyclic codes and so are discussed in later chap-



ters. Chapters 4 through 8 are devoted to the cyclic codes and their decoding techniques.

### 3.7.1 The Hamming codes

The binary Hamming codes are a family of  $(n, k)$  codes of the form  $(2^m - 1, 2^m - 1 - m)$ , where  $m = n - k$  is the parity-check bit. As the simplest nontrivial example, we have the  $(7, 4)$  Hamming code for  $m = 3$ . The parity-check matrix  $\mathbf{H}$  of a Hamming code has  $n - k = m$  rows and  $n = 2^m - 1$  columns, except for the all-zero  $m$ -tuple column vector. No two columns of  $\mathbf{H}$  of a Hamming code are linearly dependent; otherwise the two columns would be identical. However, for  $m > 1$ , it is possible to find three columns of  $\mathbf{H}$  which add to zero. Consequently  $d_{\min} = 3$  is the minimum distance for an  $(n, k)$  Hamming code. Hence the Hamming codes are capable of correcting all single errors.

The Hamming codes are examples of the few known perfect codes. Referring to Eq. (3.20), the Hamming bound with equality is

$$2^{n-k} = 1 + n \quad \text{for } t = 1$$

from which  $n = 2^{n-k} - 1 = 2^m - 1$  is the code length for the Hamming codes. They are obviously perfect codes. Another unique property of the Hamming codes is that they are one of the few classes for which the complete weight structure is known. The weight enumerator for the Hamming codes of length  $n = 2^m - 1$  is

$$A(z) = \frac{1}{n+1} [(1+z)^n + n(1+z)^{(n-1)/2}(1-z)^{(n+1)/2}] \quad (3.51)$$

On the other hand, the  $(2^m - 1, m)$  dual Hamming codes have the following simple weight enumerator:

$$B(z) = 1 + (2^m - 1)z^{2^{m-1}} \quad (3.52)$$

which consists of the all-zero word and  $2^m - 1$  code words of weight  $2^{m-1}$ .

If one additional parity bit is amended to it, an  $(n, k)$  Hamming code can be modified to yield an  $(n+1, k)$  code with  $d_{\min} = 4$ . This new family of  $d_{\min} = 4$  codes is called the extended Hamming codes, which have the form of  $(2^m, 2^m - m - 1)$ . On the other hand, an  $(n, k)$  Hamming code may be shortened to  $(n - \lambda, k - \lambda)$  by removing  $\lambda$  rows of its generator matrix  $\mathbf{G}$  or, equivalently, by removing  $\lambda$  columns of its parity-check matrix  $\mathbf{H}$ . The cyclic versions of the Hamming codes will be fully discussed in Chap. 5. Table 3.6 shows parameters for some Hamming codes over  $GF(2)$ .

TABLE 3.6  $(n, k)$  for Some Hamming Codes over  $GF(2)$ 

| $m$ | $(n, k)$ | $m$ | $(n, k)$   |
|-----|----------|-----|------------|
| 3   | (7, 4)   | 6   | (63, 57)   |
| 4   | (15, 11) | 7   | (127, 120) |
| 5   | (31, 26) | 8   | (255, 247) |

**Example 3.12** Consider the (15, 11) Hamming code for  $m = 4$ . The dual of this code is generated by its parity-check matrix:

$$H = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 1 & 1 \end{bmatrix}$$

There are  $2^4 = 16$  code words in the (15, 4) dual Hamming code:

|                                 |                                 |
|---------------------------------|---------------------------------|
| (0 0 0 0 0 0 0 0 0 0 0 0 0 0 0) | (1 0 0 0 1 0 0 1 1 0 1 0 1 1 1) |
| (0 0 0 1 0 0 1 1 0 1 0 1 1 1 1) | (1 0 0 1 1 0 1 0 1 1 1 1 1 0 0) |
| (0 0 1 0 0 1 1 0 1 0 1 1 1 1 0) | (1 0 1 0 1 1 1 1 0 0 0 1 0 0 1) |
| (0 0 1 1 0 1 0 1 1 1 1 0 0 0 1) | (1 0 1 1 1 1 0 0 0 1 0 0 1 1 0) |
| (0 1 0 0 1 1 0 1 0 1 1 1 1 0 0) | (1 1 0 0 0 1 0 0 1 1 0 1 0 1 1) |
| (0 1 0 1 1 1 1 0 0 0 1 0 0 1 1) | (1 1 0 1 0 1 1 1 1 0 0 0 1 0 0) |
| (0 1 1 0 1 0 1 1 1 1 0 0 0 1 0) | (1 1 1 0 0 0 1 0 0 1 1 0 1 0 1) |
| (0 1 1 1 1 0 0 0 1 0 0 1 1 0 1) | (1 1 1 1 0 0 0 1 0 0 1 1 0 1 0) |

By observing the weight distribution of this dual code from the code words listed above, we see that it consists of one all-zero word and 15 code words of weight 8. Therefore, the weight enumerator of this dual code can be immediately written as

$$B(z) = 1 + 15z^8$$

which is also easily derived from Eqs. (3.52) or (3.43). That is,

$$B(z) = 1 + (2^4 - 1)z^{2^3} = 1 + 15z^8$$

or 
$$B(z) = 2^{-11} (1 + z)^{15} A\left(\frac{1-z}{1+z}\right) = 1 + 15z^8$$

We have learned that there are two ways to compute the probability of an undetected error for an  $(n, k)$  code: i.e., using  $A(z)$  or  $B(z)$ . How-

ever, since  $m = n - k = 4$  is smaller than  $k = 11$ , it is much easier to calculate  $P_{\text{ud}}(E)$  from the weight enumerator  $B(z)$  for the dual code by making use of Eq. (3.50). From Eq. (3.50), the probability of an undetected error is

$$P_{\text{ud}}(E) = 2^{-4}[1 + 15(1 - 2p)^8] - (1 - p)^{15} \quad (3.53)$$

On the other hand, the weight distribution of the (15, 11) Hamming code can be determined by computing the weight enumerator  $A(z)$  from Eq. (3.51). That is,

$$\begin{aligned} A(z) &= \frac{1}{16}[(1 + z)^{15} + 15(1 + z)^7(1 - z)^8] \\ &= 1 + 35(z^3 + z^{12}) + 105(z^4 + z^{11}) + 168(z^5 + z^{10}) \\ &\quad + 280(z^6 + z^9) + 435(z^7 + z^8) + z^{15} \end{aligned}$$

Thus, the weight distribution of this code is  $A_0 = 1$ ,  $A_3 = A_{12} = 35$ ,  $A_4 = A_{11} = 105$ ,  $A_5 = A_{10} = 168$ ,  $A_6 = A_9 = 280$ ,  $A_7 = A_8 = 435$ , and  $A_{15} = 1$ . Substituting these  $A_i$ 's into Eq. (3.49), we get  $P_{\text{ud}}(E)$  for the weight enumerator  $A(z)$  of the (15, 11) code. That is,

$$\begin{aligned} P_{\text{ud}}(E) &= 35p^3(1 - p)^3[(1 - p)^9 + p^9] + 105p^4(1 - p)^4[(1 - p)^7 + p^7] \\ &\quad + 168p^5(1 - p)^5[(1 - p)^5 + p^5] + 280p^6(1 - p)^6[(1 - p)^3 + p^3] \\ &\quad + 435p^7(1 - p)^7 + p^{15} \end{aligned} \quad (3.54)$$

Comparing Eq. (3.54) from  $A(z)$  with Eq. (3.53) from  $B(z)$ , we can easily see that computing  $P_{\text{ud}}(E)$  by Eq. (3.54) is rather more involved than computing it by Eq. (3.53). If  $p = 10^{-2}$  over a BSC, the probability,  $P_{\text{ud}}(E)$  given by either Eqs. (3.53) or (3.54), is approximately  $3 \times 10^{-5}$ ; that is, out of 100,000 transmitted code digits about three erroneous code digits pass through the decoder undetected.

### 3.7.2 The Golay codes

The Golay code occupies a unique and important position in the area of error-control codes. Four such important Golay codes are discussed in this subsection. They are (23, 12) and (24, 12) binary Golay codes and (11, 6) and (12, 6) ternary Golay codes.

The (23, 12) Golay code with  $t = 3$  is a binary linear code with  $d_{\min} = 7$  and has been used in several communications systems. Since the (23, 12) Golay code over  $GF(2)$  has the minimum distance  $d_{\min} = 7$ , the cosets of minimum weight  $\leq 3$  are disjoint. The number of such cosets is

$$1 + \binom{23}{1} + \binom{23}{2} + \binom{23}{3} = 2^{11} = 2^{23-12}$$

which represents the summed elements in the rows of the Pascal triangle, and an exact power of 2 is obtained. Therefore, the (23, 12) code is perfect because it includes all cosets of the code, and all spheres of packing radius  $t = 3$  around each code vector are disjoint. The extended (24, 12) Golay code is quite useful because it has  $d_{\min} = 8$  and its code rate is exactly  $R = 1/2$ . This code can be derived from the (23, 12) Golay code by adding an overall parity check to each code word. The (24, 12) extended code is self-dual and only quasi-perfect because all spheres of radius  $t$  about each code vector are disjoint, but every vector is at most a distance  $t + 1$  from some code vector.

The only known nonbinary perfect code is a ternary (11, 6) Golay code over  $GF(3)$  with  $d_{\min} = 5$ . It is a perfect double-error-correcting code. A Hamming sphere of radius  $t = 2$  over  $GF(3)$  contains

$$1 + 2\binom{11}{1} + 4\binom{11}{2} = 243 = 3^5 \text{ vectors}$$

which suggests that it might be possible to perfectly pack with  $729 = 3^6$  spheres (code words) of radius  $t = 2$ . Both the (23, 12) binary code and the (11, 6) ternary code are quadratic-residue codes that have the largest minimum distance of any known code with the same  $n$  and  $k$ , and this is what makes quadratic-residue codes attractive. Finally, the extended (12, 6) ternary Golay code is obtained by adding an overall parity check to the (11, 6) ternary code. This code is also unique and self-dual, and it has minimum distance  $d_{\min} = 6$ .

Table 3.7 is a summary of the weight distribution and number of code words of Golay codes. Through use of Table 3.7, the weight structure of these Golay codes can be determined.

TABLE 3.7 Weight Distribution and Number of Code Words of Golay Codes

| Binary codes |                  |                  | Ternary codes |                 |       |                 |
|--------------|------------------|------------------|---------------|-----------------|-------|-----------------|
|              | (23, 12)<br>code | (24, 12)<br>code |               | (11, 6)<br>code |       | (12, 6)<br>code |
| $i^*$        | $d_{\min} = 7$   | $d_{\min} = 8$   | $i^*$         | $d_{\min} = 5$  | $i^*$ | $d_{\min} = 6$  |
| 0            | 1                | 1                | 0             | 1               | 0     | 24              |
| 7            | 253              | 0                | 5             | 132             | 3     | 440             |
| 8            | 506              | 759              | 6             | 132             | 6     | 264             |
| 11           | 1288             | 0                | 8             | 330             | 12    | 1               |
| 12           | 1288             | 2576             | 9             | 110             |       | 729             |
| 15           | 506              | 0                | 11            | 24              |       |                 |
| 16           | 253              | 759              |               | 729             |       |                 |
| 23           | 1                | 0                |               |                 |       |                 |
| 24           | —                | 1                |               |                 |       |                 |
|              | 4096             | 4096             |               |                 |       |                 |

\* $i$  designates the weight.

The weight enumerator of the (23, 12) Golay code over  $GF(2)$  is

$$A(z) = 1 + 253(z^7 + 2z^8 + 2z^{15} + z^{16}) + 1288(z^{11} + z^{12}) + z^{23} \quad (3.55)$$

For the extended (24, 12) Golay code over  $GF(2)$ , the weight enumerator of this code is

$$A(z) = 1 + 759(z^8 + z^{16}) + 2576z^{12} + z^{24} \quad (3.56)$$

Finally, the (11, 6) ternary Golay code over  $GF(3)$  has a weight enumerator of

$$A(z) = 1 + 132(z^5 + z^6) + 330z^8 + 110z^9 + 24z^{11} \quad (3.57)$$

For the extended (12, 6) ternary Golay code over  $GF(3)$ , the weight enumerator is

$$A(z) = 24 + 440z^3 + 264z^6 + z^{12} \quad (3.58)$$

**Example 3.13** Consider the (23, 12) Golay code and its extended (24, 12) code over  $GF(2)$ . Given the respective weight enumerators of Eqs. (3.55) and (3.56), the probability of undetected error is

$$\begin{aligned} P_{ud}(E) = & 253[p^7(1-p)^{16} + p^{16}(1-p)^7] + 506[p^8(1-p)^{15} + p^{15}(1-p)^8] \\ & + 1288[p^{11}(1-p)^{12} + p^{12}(1-p)^{11}] \end{aligned} \quad (3.59)$$

for the (23, 12) binary code; and for the (24, 12) extended code, it is

$$P_{ud}(E) = 759[p^8(1-p)^{16} + p^{16}(1-p)^8] + 2576[p^{13}(1-p)^{11} + p^{24}] \quad (3.60)$$

The plot of  $P_{ud}(E)$  against  $p$  is shown in Fig. 3.5. The analysis shows that the performance of the (24, 12) extended code is somewhat better than that of the (23, 12) binary Golay code.

From the foregoing discussion, a perfect code satisfies the Hamming bound with equality. The packing radius  $t$  is equal to the number of errors that can be corrected by the code. The construction of the standard array stops just when the spheres of radius  $t$  are completed. Therefore, it is clear that the concept of a perfect  $t$ -error-correcting code must include the following element: The coset leaders are unique and contain exactly all vectors of weight  $t$  or less. Since there are few classes of such codes, the weaker concept of a quasi-perfect code is sometimes of interest. The quasi-perfect code is defined as one that for some  $t$  has all vectors of weight  $t$  or less, some vectors of weight  $t + 1$ , and no vectors of weight greater than  $t + 1$ . The only nontrivial perfect binary codes are (1) the Hamming codes which are single-error-correcting codes over  $GF(2)$  with  $n = 2^m - 1$  and  $k = 2^m - 1 - m$  as



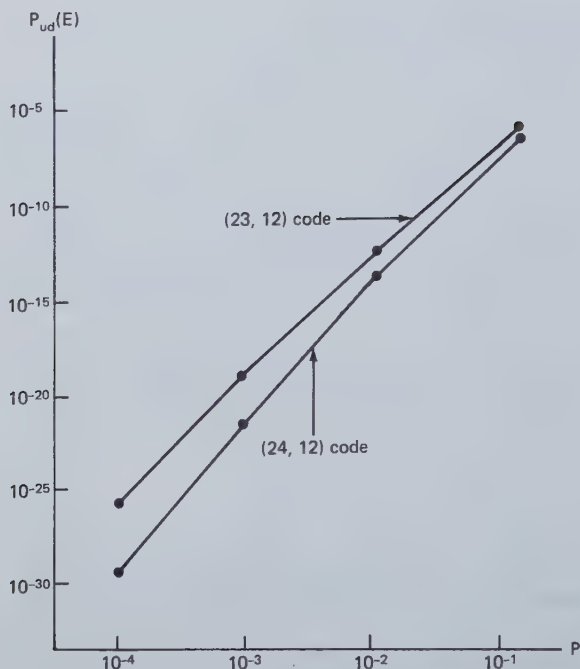


Figure 3.5 Performance curves for Golay codes over  $GF(2)$ .

discussed in Sec. 3.7.1; (2) the binary repetition codes with two all-zero and all-one code words; and (3) the two Golay codes which are the (23, 12) binary code of  $d_{\min} = 7$  and the (11, 6) ternary code of  $d_{\min} = 5$ .

In the following, we will briefly look into the interesting connection between Golay codes and  $\sigma$ -designs. A  $\sigma$ -design or a  $\lambda$ -( $\sigma$ ,  $\omega$ ,  $n$ ) design with  $\lambda = 1$  is called a Steiner system, which we denote by  $S(\sigma, \omega, n)$ . For  $\sigma = 4$  and  $\sigma = 5$ , the only known Steiner systems are the Mathieu systems, that is,  $S(4, 7, 23)$ ,  $S(5, 8, 24)$ ,  $S(4, 5, 11)$ , and  $S(4, 6, 12)$ . A  $\lambda$ -( $\sigma$ ,  $\omega$ ,  $n$ ) design is an ordered pair  $(A, B)$ , where  $A$  is any  $n$ -set (a set with  $n$  elements), and  $B$  is a family of  $\omega$ -subsets (called blocks) of  $A$  with the property that each  $\sigma$ -subset of  $A$  is contained in precisely one of  $\lambda$  blocks of  $B$ . Notice that we always assume  $0 < \sigma < \omega < n$  in this order, and that each  $\sigma$  falls roughly in  $0 < \sigma \leq 6$  for nontrivial  $\sigma$ -designs.

The (23, 12) Golay code over  $GF(2)$  is perfect and is a quadratic-residue code. The minimum distance  $d_{\min}$  must satisfy the inequality  $d(d-1) \geq 22$ . But because  $d$  is odd, we must choose  $d \geq 7$ . Therefore, this code has  $d_{\min} = 7$  because the number of code words belonging to the sphere of radius  $t = 3$  centered in an arbitrary code word is

$$\sum_{i=0}^3 \binom{23}{i} = 2^{11}$$

which is exactly identical to the number of cosets. In a  $\lambda$ -( $\sigma$ ,  $\omega$ ,  $n$ ) design, the total number of blocks is computed by

$$b = \binom{n}{\sigma} / \binom{\omega}{\sigma}$$

with  $\lambda = 1$ . Since the (23, 12) Golay code has

$$b = \binom{23}{4} / \binom{7}{4} = 253 \text{ blocks}$$

the code words of weight 7 cover

$$253 \binom{7}{4} = \binom{23}{4}$$

which also means that the code words of weight 4 and length 23 are covered by all code words of weight 7. Thus, the code words of weight 7 form the blocks of a Steiner system  $S(4, 7, 23)$ . These code words are sometimes called octads. The extended (24, 12) Golay code over  $GF(2)$  is self-dual, and all its weights are divisible by 4. Each code word of weight 8 covers  $\binom{8}{5}$  vectors of weight 5, which are all distinct. From the following identity:

$$759 \binom{8}{5} = \binom{24}{5}$$

we see that any binary word of weight 5 and length 24 is covered by exactly one code word of weight 8 in the (24, 12) code. Thus, the code words of weight 8 (called octads) in the extended (24, 12) Golay code form the blocks of a Steiner system  $S(5, 8, 24)$ .

The (11, 6) ternary Golay code is perfect and is a quadratic-residue code. There are

$$2 \left[ \binom{11}{1} + \binom{11}{2} \right] = 132$$

code words. Since the 132 blocks cover

$$132 \binom{6}{5} = \binom{12}{5}$$

distinct words of weight 5, every word of weight 5 is covered by exactly one block. Thus, the code words of weight 5 form the blocks of the Steiner system  $S(4, 5, 11)$ . The extended (12, 6) ternary Golay code is

self-dual, and all its weights are divisible by 3. Similarly, it follows that this code forms the Steiner system  $S(5, 6, 12)$ .

Thus, the minimum-weight code words in the extended  $(24, 12)$  and  $(12, 6)$  Golay codes form the Steiner systems  $S(5, 8, 24)$  and  $S(5, 6, 12)$ , while those of codes  $(23, 12)$  and  $(11, 6)$  form the Steiner systems  $S(4, 7, 23)$  and  $S(4, 5, 11)$ , respectively.

### 3.7.3 The Hadamard codes

A Hadamard matrix  $[\mathbf{H}]_n$  is an orthogonal  $n \times n$  matrix of the entries  $+1$  and  $-1$  with the property that any row differs from any other row in exactly  $n/2$  positions. One row of the matrix contains all  $+1$ s. The other rows contain evenly the  $+1$ s and  $-1$ s of  $n/2$  each. Furthermore, all the entries in the first row and the first column of  $[\mathbf{H}]_n$  have all  $+1$ s. The  $n \times n$  Hadamard matrices can exist only if  $n$  is a power of 2 (i.e.,  $n = 2^m$ ) or a multiple of 4. If  $n$  is not a power of 2, the code is nonlinear.

By changing the  $+1$ s to 0s and the  $-1$ s to 1s, the Hadamard matrix for  $n = 2$  can be expressed by

$$[\mathbf{H}]_2 = \begin{bmatrix} +1 & +1 \\ +1 & -1 \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} \quad (3.61)$$

If  $[\mathbf{H}]_n$  is an  $n \times n$  Hadamard matrix, we can generate the Hadamard matrix  $[\mathbf{H}]_{2n}$  according to the following relation:

$$[\mathbf{H}]_{2n} = \begin{bmatrix} [\mathbf{H}]_n & [\mathbf{H}]_n \\ [\mathbf{H}]_n & [\bar{\mathbf{H}}]_n \end{bmatrix} \quad (3.62)$$

where  $[\bar{\mathbf{H}}]_n$  denotes the complement (0s replaced by 1s and vice versa) of  $[\mathbf{H}]_n$ . Thus, by substituting Eq. (3.61) into Eq. (3.62), we obtain the Sylvester-type Hadamard matrix of order 4:

$$[\mathbf{H}]_4 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix} \quad (3.63)$$

Since the Hadamard matrix exists whenever the order  $n$  is a multiple of 4,  $[\mathbf{H}]_8$  can easily be obtained as follows:

$$[\mathbf{H}]_8 = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{bmatrix}$$

Since  $[\mathbf{H}]_n$  is a Hadamard matrix,  $[\mathbf{H}]_{2n}$  is also a Hadamard matrix. We have thus seen the possibility of doubling the minimum distance.

The code is constructed by taking the vectors  $\mathbf{c}_1, \mathbf{c}_2, \dots, \mathbf{c}_n$  corresponding to the rows of the matrix  $[\mathbf{H}]_n$  and the vectors  $-\mathbf{c}_1, -\mathbf{c}_2, \dots, -\mathbf{c}_n$ . This gives a set of  $2n$  binary vectors of length  $n$ , the minimum distance between any two of them being equal to  $n/2$ . Now, let us consider a code derived from the Hadamard matrix. By taking the complement of  $[\mathbf{H}]_4$  from Eq. (3.63), we get

$$[\bar{\mathbf{H}}]_4 = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 \end{bmatrix} \quad (3.64)$$

Thus, the rows of  $[\mathbf{H}]_4$  and  $[\bar{\mathbf{H}}]_4$  form a linear binary code of block length  $n = 4$  having  $2n = 8$  code words. The minimum distance of this code is  $d_{\min} = n/2 = 2$ . By repeated application of Eq. (3.62), we can generate Hadamard codes with block length  $n = 2^m$ ,  $k = \log_2 2n = \log_2 2^{m+1} = m + 1$ , and  $d_{\min} = n/2 = 2^{m-1}$ , where  $m$  is a positive integer. In addition to an important special case where  $n = 2^m$ , Hadamard codes of other block lengths are possible, but the codes are not linear. The second type of Hadamard matrix is generally called the Paley-type Hadamard matrix and is of any order  $n = p + 1$  for a multiple of 4 and of order  $n = p^m + 1$  if the quadratic residues in  $GF(p^m)$  are used. For this problem see MacWilliams and Sloane (1977), with detailed discussions.

### 3.7.4 The product codes

Channel errors sometimes occur in a mixed manner, i.e., random-burst-type errors. To combat these mixed errors, it is frequently desir-

able to design codes that are capable of correcting random errors as well as burst errors. In this subsection, we discuss one method of deriving from given component codes a new code which is useful for random-burst-error correction.

It is sometimes convenient to process code words as multidimensional arrays. The product code is the simplest kind of these multidimensional codes. Elias (1954) introduced the idea of the product code, constructing a two-dimensional array from two given component codes.

Let  $C_1$  and  $C_2$  be  $(n_1, k_1, d_1)$  and  $(n_2, k_2, d_2)$  linear component codes over  $GF(2)$ , respectively. The  $d_1$  denotes the minimum distance of  $C_1$ , and  $d_2$  denotes the minimum distance of  $C_2$ . The direct product  $C_1 \otimes C_2$  is a two-dimensional  $(n_1 n_2, k_1 k_2, d_1 d_2)$  code whose code words consist of all  $n_1 \times n_2$  arrays constructed as described below and shown in Fig. 3.6.

In a rectangular array of  $n_1$  columns and  $n_2$  rows, the rows are code words in  $C_1$ , and the columns are code words in  $C_2$ . The top right corner of the array contains the  $k_1 k_2$  information bits. The bits in the top left corner of this array are computed according to the parity-check rules for  $C_1$ , and the bits in the bottom right corner are computed according to the parity-check rules for  $C_2$ . The check bits in the bottom left corner yield  $(n_1 - k_1) \times (n_2 - k_2)$  checks on check bits, so it is possible to have simultaneous checks on all row code words in  $C_1$  and all column code words in  $C_2$ . Therefore, in order to encode the product

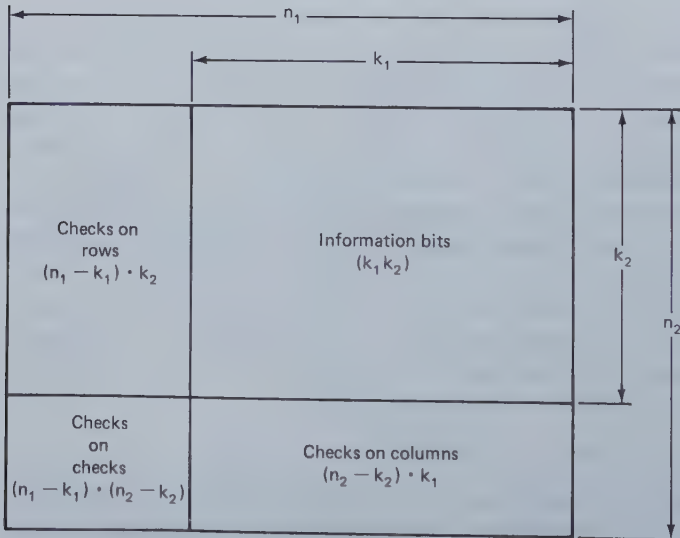


Figure 3.6 Code array for the product code  $C_1 \otimes C_2$ .



code  $C_1 \otimes C_2$ , we may first encode the first  $k_2$  rows with an encoder for  $C_1$  and then encode all  $n_1$  columns with an encoder for  $C_2$ . By a similar argument, we could obtain the same code word if the first  $k_1$  columns are first encoded with a  $C_2$  encoder, and then all the  $n_2$  rows are encoded with a  $C_1$  encoder.

**Example 3.14** Consider an  $(n, n - 1)$  even weight code. For  $n = 3$ , we get the  $(3, 2)$  code whose generator matrix and parity-check matrix are, respectively,

$$G = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix} \quad \text{and} \quad H = [1 \quad 1 \quad 1]$$

| Information bits | Code words |
|------------------|------------|
| 0 0              | 0 0 0      |
| 0 1              | 1 0 1      |
| 1 0              | 1 1 0      |
| 1 1              | 0 1 1      |

As you see from the table above, the minimum distance of this code has an even weight of 2. The direct product of the  $(3, 2, 2)$  even weight code with itself is the  $(9, 4, 4)$  code. Thus, the number of information arrays is  $2^4 = 16$  as shown below.

|     |     |     |     |
|-----|-----|-----|-----|
| 0 0 | 0 0 | 0 0 | 0 0 |
| 0 0 | 0 1 | 1 0 | 1 1 |
| 0 1 | 0 1 | 0 1 | 0 1 |
| 0 0 | 0 1 | 1 0 | 1 1 |
| 1 0 | 1 0 | 1 0 | 1 0 |
| 0 0 | 0 1 | 1 0 | 1 1 |
| 1 1 | 1 1 | 1 1 | 1 1 |
| 0 0 | 0 1 | 1 0 | 1 1 |

For demonstration purposes, let us take one information array

|     |
|-----|
| 1 0 |
| 0 1 |

marked by the solid square line. In order to encode the product code  $C_1 \otimes C_2 = (3, 2, 2) \otimes (3, 2, 2)$ , we first encode  $k_2 = 2$  rows with an encoder for  $C_1$ , obtaining  $(1 \ 1 \ 0)$  and  $(1 \ 0 \ 1)$ , and then encode all three columns with an encoder for  $C_2$ , obtaining  $(1 \ 1 \ 0)$ ,  $(1 \ 0 \ 1)$ , and  $(0 \ 1 \ 1)$ . Hence, the corresponding code word of the  $(9, 4, 4)$  product code is shown in the marked square in Table 3.8, or

TABLE 3.8 Encoded Arrays of  $(3, 2, 2) \otimes (3, 2, 2) \rightarrow (9, 4, 4)$  Product Code

|       |       |       |       |
|-------|-------|-------|-------|
| 0 0 0 | 0 0 0 | 0 0 0 | 0 0 0 |
| 0 0 0 | 1 0 1 | 1 1 0 | 0 1 1 |
| 0 0 0 | 1 0 1 | 1 1 0 | 0 1 1 |
| 1 0 1 | 1 0 1 | 1 0 1 | 1 0 1 |
| 0 0 0 | 1 0 1 | 1 1 0 | 0 1 1 |
| 1 0 1 | 0 0 0 | 0 1 1 | 1 1 0 |
| 1 1 0 | 1 1 0 | 1 1 0 | 1 1 0 |
| 0 0 0 | 1 0 1 | 1 1 0 | 0 1 1 |
| 1 1 0 | 0 1 1 | 0 0 0 | 1 0 1 |
| 0 1 1 | 0 1 1 | 0 1 1 | 0 1 1 |
| 0 0 0 | 1 0 1 | 1 1 0 | 0 1 1 |
| 0 1 1 | 1 1 0 | 1 0 1 | 0 0 0 |

|       |
|-------|
| 1 1 0 |
| 1 0 1 |
| 0 1 1 |

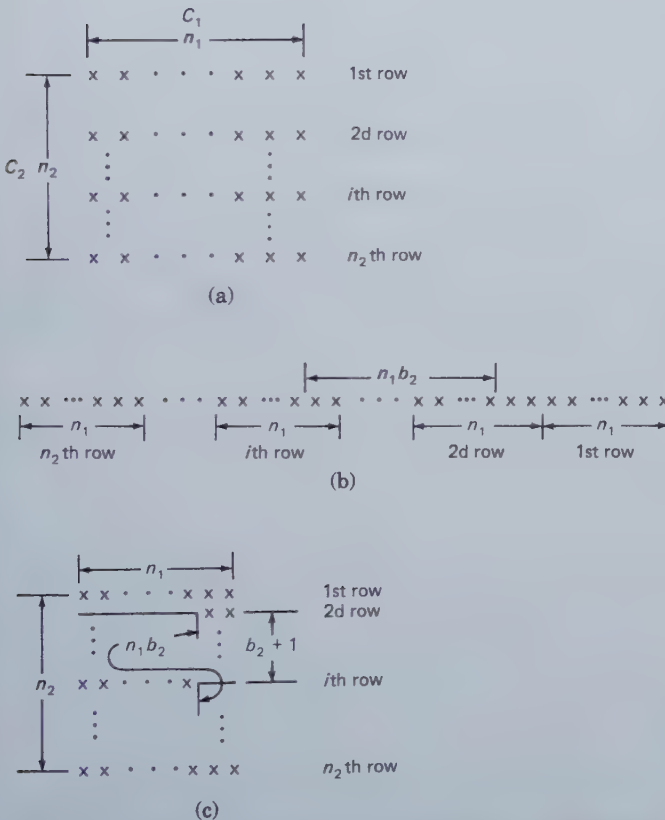
Thus, the 16 encoded arrays of the  $(9, 4, 4)$  product code are shown in Table 3.8.

If  $d_1$  is the minimum weight of  $C_1$  and  $d_2$  is the minimum weight of  $C_2$ , then a code word in the product code must have at least  $d_1$  nonzero elements in each row and at least  $d_2$  nonzero elements in each nonzero column. Thus, the minimum weight of the product code is exactly  $d_1 d_2$ . Therefore, at least one such code word exists.

One suggested way to correct random errors by decoding is column correction after row correction. It is called "simple correction on rows and columns," but it is not always effective. If the two codes can correct  $t_1 = \lfloor (d_1 - 1)/2 \rfloor$  and  $t_2 = \lfloor (d_2 - 1)/2 \rfloor$ , respectively, then the product code is capable of correcting any combination of  $t = \lfloor (d_1 d_2 - 1)/2 \rfloor = 2 t_1 t_2 + t_1 + t_2$  errors. However, like the product code of two Hamming single-error-correcting codes, you can see that it may not be correctable through simple correction on rows and columns. Consequently, it can be concluded that this theoretical capability  $t$  of the product code is at least worthless without a practical decoding algorithm. These decoding algorithms are applicable to an errors-only decoder for one component code and an errors-erasures decoder for the other component code. Thus, several techniques that decode a product code up to its guaranteed error-correction capability have been found so far. A decoder for correcting the random errors in a product of two BCH code words or in a product of two majority-logic decodable codes has been implemented, but it still remains an interesting subject for further study.

A burst of length  $b$  is defined as a bit string (its first and last bits are nonzero) whose nonzero components are confined to  $b$  consecutive bit positions. Let  $b_1$  and  $b_2$  be the burst-error-correcting capabilities of component codes  $C_1$  and  $C_2$ . Consider the word array of the product code shown in Fig. 3.7a. Suppose that a word array of the product code is transmitted row by row as shown in Fig. 3.7b. When the received sequence with any error burst of length  $b_2$  or less is rearranged back into an array on a row-by-row basis, it is clear that  $n_1 b_2$  will affect no more than  $b_2 + 1$  consecutive rows (see Fig. 3.7c).

Since each column is affected by a burst of length  $b_2$  or less, the burst will be corrected by column-by-column decoding. Hence, the burst-error-correcting capability of the product code is at least  $n_1 b_2$ . Similarly, consider that a word array of the product code is transmitted column by column and decoded row by row. It can be shown that



**Figure 3.7** The word array of the product code. (a) Word array composition. (b) Row-by-row transmission. (c) Reconstruction of a word array.

any error burst of length  $n_2 b_1$  or less can be corrected. Thus, the burst-error-correcting capability of  $b$  of the product code can be written as

$$b \geq \max \{n_1 b_2, n_2 b_1\} \quad (3.65)$$

Thus far, we have considered a product code for either random-error correction or burst-error correction independently. However, a product code could be used for simultaneous random-burst-error correction.

Burton and Weldon (1965) proposed the theorem for simultaneous random- and burst-error correction. The theorem states that "the cosets containing the random-error patterns of weight  $t = \lfloor (d_1 d_2 - 1)/2 \rfloor$  or less are disjoint from the cosets which contain bursts of length  $b = \max \{n_1 t_2, n_2 t_1\}$  or less." In order for these conditions to be met, it is sufficient to show that the product code  $C_1 \otimes C_2$  can correct any combination of  $t$  or fewer random errors and simultaneously correct any error burst of length  $b$  or less. If we assume  $n_1 t_2 > n_2 t_1$ , then  $b = n_1 t_2$ . Consider a burst of length  $n_1 t_2$  or less. As before, assume that the word array of the product code is arranged on a row-by-row basis. Then each column contains at most  $t_2$  errors in a burst of length  $n_1 t_2$  or less. If this burst-error pattern and some random-error pattern of weight not greater than  $t$  are in the same coset, then the sum of these two error patterns is a code array in the product code. Each column of such a code array has either weight zero or a weight of at least  $d_2$ . Since the sum of random and burst error digits in each column is no greater than  $d_2$ , each nonzero column must be composed of at least  $d_2 - t_2$  errors from the random-error pattern and at most  $t_2$  errors from the burst-error pattern. Since there are at most  $t$  random errors, these errors can be distributed among at most  $\lfloor t/(d_2 - t_2) \rfloor$  columns. Hence, the sum array contains at most  $\lfloor t/(d_2 - t_2) \rfloor t_2 + t$  nonzero digits. However,

$$\left\lfloor \frac{t}{(d_2 - t_2)} \right\rfloor t_2 + t \leq t \left( \frac{t_2}{d_2 - t_2} + 1 \right) \leq 2t \left( \frac{d_2}{d_2 + 1} \right) < 2t < d$$

Hence, the sum array contains fewer than  $2t < d_1 d_2$  nonzero digits and cannot be a code array in the product code. This contradiction implies that the two error patterns cannot be in the same coset of a standard array for the product code. If  $n_2 t_1 > n_1 t_2$ , then  $b = n_2 t_1$ . Thus, the same argument can be applied to rows instead of columns.

**Example 3.15** The direct product of the (6, 3, 3) shortened Hamming code with itself is the (36, 9, 9) product code which has  $t = 4$  and  $b = 6$ . In order to show that the (36, 9, 9) product code is capable of correcting any combination of  $t = 4$  or fewer random errors and simultaneously correcting any error burst of length  $b = 6$  or less, it is sufficient to show that an error burst of length  $b = 6$  or less

and a random error pattern of  $t = 4$  or fewer errors cannot be in the same coset. One of the encoded arrays of the (36, 9, 9) product code is

$C_1 = (6, 3, 3)$  code

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 1 | 1 | 0 | 1 | 1 | 0 |
| 1 | 0 | 1 | 0 | 1 | 0 |
| 1 | 0 | 1 | 1 | 0 | 1 |
| 0 | 0 | 0 | 1 | 1 | 1 |
| 0 | 1 | 1 | 0 | 1 | 1 |
| 0 | 1 | 1 | 1 | 0 | 0 |

$C_2 = (6, 3, 3)$  code

If this code array of the (36, 9, 9) code is transmitted row by row, then we get

011100, 011011, 000111, 101101, 101010, 110110

Since noise affects these transmitted code digits either randomly or in bursts, we can assume that the received word containing a random-error pattern of  $t = 4$  is

011100, 111011, 000111, 101101, 001110, 110010

and the received word containing the burst of length  $b = 6$  is

011100, 011110, 101111, 101101, 101010, 110110

respectively. Hence, the random-error pattern of weight  $t = 4$  is obtained as

000000, 100000, 000000, 000000, 100100, 000100

and the burst-error pattern is

000000, 000101, 101000, 000000, 000000, 000000

If these two error patterns are in the same coset, then their sum

000000, 100101, 101000, 000000, 100100, 000100

must be a code array in the product code as shown here:

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 1 | 0 | 0 |
| 1 | 0 | 0 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 |
| 1 | 0 | 1 | 0 | 0 | 0 |
| 1 | 0 | 0 | 1 | 0 | 1 |
| 0 | 0 | 0 | 0 | 0 | 0 |

Each column of this code array should have either weight  $d_2 = 0$  or weight  $d_2 = 3$ . But, since the minimum distances of the third column and the sixth column are both 1, this sum array cannot be a code array in the product code. On the other hand, there are at most  $t = 4$  random errors. These errors can be distributed among two columns. Hence, it can be concluded that the sum array contains at most  $(2)(1) + 4 = 6$  nonzero digits. However, the minimum distance of the (36, 9, 9) product code is 9. Therefore, we know that the sum array above is not a code array in the product code. Thus, it can be judged from the analysis



that the received word containing the random-error pattern of weight  $t = 4$  is disjoint from the received word containing the burst of length  $b = 6$ .

### 3.7.5 The Reed-Muller codes

The Reed-Muller codes were discovered by Muller (1954) and provided with a decoding algorithm by Reed (1954). The Reed-Muller (RM) codes are a class of binary codes connected with finite geometries that are easy to describe and can be decoded by a simple majority-logic technique.

The original  $r$ th-order RM codes were binary and noncyclic, and formed a special subclass of euclidean geometry (EG) codes. The  $r$ th-order binary RM code has the following parameters:

$$\begin{aligned} n &= 2^m \\ k &= 1 + \binom{m}{1} + \binom{m}{2} + \cdots + \binom{m}{r} \\ n - k &= 2^m - k = \sum_{i=0}^{m-r-1} \binom{m}{i} \\ d_{\min} &= 2^{m-r} \end{aligned}$$

where  $r < m$  for any integer  $m$ .

The dual of the  $r$ th-order RM code of length  $2^m$  is the  $(m - r - 1)$ th-order RM code. For  $m \geq 3$ , there exists a  $(2^m - 1, m)$  maximum-length code with  $d_{\min} = 2^{m-1}$ . The first-order RM codes are closely related to the maximum-length codes. Since the first-order RM code has the dimensions  $n = 2^m$ ,  $k = m + 1$ , and  $d_{\min} = 2^{m-1}$ , the first-order RM code is exactly the extended maximum-length code. The zeroth-order RM code is a  $(2^m, 1)$  repetition code which has the minimum distance of  $2^m$ .

Since a RM code is a linear code, it is defined by a procedure for constructing a nonsystematic generator matrix that will prove convenient for decoding. Let

$$\mathbf{u} = (\mathbf{u}_0, \mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_{n-1})$$

and

$$\mathbf{v} = (\mathbf{v}_0, \mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_{n-1})$$

be two vectors, respectively. The vector sum and the vector product of these two vectors are defined as

$$\mathbf{u} + \mathbf{v} = (\mathbf{u}_0 + \mathbf{v}_0, \mathbf{u}_1 + \mathbf{v}_1, \dots, \mathbf{u}_{n-1} + \mathbf{v}_{n-1})$$

and

$$\mathbf{uv} = (\mathbf{u}_0\mathbf{v}_0, \mathbf{u}_1\mathbf{v}_1, \dots, \mathbf{u}_{n-1}\mathbf{v}_{n-1})$$

respectively. Let  $\mathbf{v}_0$  be a vector whose  $2^m$  components are all 1s. Then  $\mathbf{v}_0$  is called an identity vector. Let  $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_m$  be the rows of a ma-

trix with all possible  $m$ -tuples as columns. We call them the basis vectors. The  $r$ th-order RM code is formed by using the basis vectors  $\mathbf{v}_0, \mathbf{v}_1, \dots, \mathbf{v}_m$  and all of their vector products  $r$  or fewer at a time. The generator matrix for the  $r$ th-order RM code of length  $2^m$  is then defined as the following array:

$$\mathbf{G} = \begin{bmatrix} \mathbf{v}_0 \\ \mathbf{v}_1 \\ \mathbf{v}_2 \\ \vdots \\ \mathbf{v}_m \\ \mathbf{v}_1\mathbf{v}_2 \\ \mathbf{v}_1\mathbf{v}_3 \\ \vdots \\ \mathbf{v}_{m-1}\mathbf{v}_m \\ \mathbf{v}_1\mathbf{v}_2\mathbf{v}_3 \\ \vdots \\ \mathbf{v}_{m-2}\mathbf{v}_{m-1}\mathbf{v}_m \\ \vdots \\ \mathbf{v}_1\mathbf{v}_2 \cdots \mathbf{v}_r \\ \vdots \\ \mathbf{v}_{m-r+1} \cdots \mathbf{v}_{m-1}\mathbf{v}_m \end{bmatrix} \quad (3.66)$$

where  $\mathbf{v}_i = (v_{i,0}, v_{i,1}, \dots, v_{i,n-1})$ .

**Example 3.16** Consider the second-order RM code for  $m = 4$ . The parameters of this code are then

$$n = 2^4 = 16$$

$$k = 1 + \binom{4}{1} + \binom{4}{2} = 1 + 4 + 6 = 11$$

$$n - k = 16 - 11 = 5$$

$$d_{\min} = 2^2 = 4$$

Hence, this is the (16, 11) linear code which is equivalent to the (15, 11) Hamming code expanded by an extra parity check. The random-error-correcting capability of this code is  $t = 2^{m-r-1} - 1 = 2^{4-2-1} - 1 = 1$ . The generator matrix

of the second-order RM code for  $m = 4$  is the  $11 \times 16$  matrix which is derived from Eq. (3.66) as shown below.

$$\mathbf{G} = \begin{bmatrix} \mathbf{v}_0 \\ \mathbf{v}_1 \\ \mathbf{v}_2 \\ \mathbf{v}_3 \\ \mathbf{v}_4 \\ \mathbf{v}_1\mathbf{v}_2 \\ \mathbf{v}_1\mathbf{v}_3 \\ \mathbf{v}_1\mathbf{v}_4 \\ \mathbf{v}_2\mathbf{v}_3 \\ \mathbf{v}_2\mathbf{v}_4 \\ \mathbf{v}_3\mathbf{v}_4 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} \quad (3.67)$$

Since every basis vector of  $\mathbf{G}$  has even weight, the product of two basis vectors must also have even weight. Because

$$k = \binom{m}{0} + \binom{m}{1} + \cdots + \binom{m}{r}$$

gives the information bits of the  $r$ th-order RM code of length  $2^m$ , the information vector is generally expressed as

$$\mathbf{d} = (d_0, d_1, \dots, d_m, d_{1,2}, \dots, d_{m-1,m}, \dots, d_{1,2,3}, \dots, d_{m-r+1}, \dots, m-1, m)$$

The encoding is then represented as a vector-matrix product as follows:

$$\mathbf{c} = \mathbf{d} \cdot \mathbf{G} = (d_0, d_1, \dots, d_m, d_{1,2}, \dots, d_{m-r+1}, \dots, m-1, m)$$

$$\begin{bmatrix} \mathbf{v}_0 \\ \mathbf{v}_1 \\ \vdots \\ \mathbf{v}_m \\ \vdots \\ \mathbf{v}_1\mathbf{v}_2 \cdots \mathbf{v}_r \\ \vdots \\ \mathbf{v}_{m-r+1}\mathbf{v}_{m-r+2} \cdots \mathbf{v}_m \end{bmatrix}$$

$$\begin{aligned}
 &= d_0 \mathbf{v}_0 + d_1 \mathbf{v}_1 + d_2 \mathbf{v}_2 + \cdots + d_m \mathbf{v}_m + d_{1,2} \mathbf{v}_1 \mathbf{v}_2 + d_{1,3} \mathbf{v}_1 \mathbf{v}_3 + \cdots \\
 &\quad + d_{m-1,m} \mathbf{v}_{m-1} \mathbf{v}_m + \cdots + d_{1,2,3} \mathbf{v}_1 \mathbf{v}_2 \mathbf{v}_3 + \cdots \\
 &\quad + d_{m-r+1, \dots, m-1, m} \mathbf{v}_{m-r+1} \cdots \mathbf{v}_{m-1} \mathbf{v}_m
 \end{aligned} \tag{3.69}$$

Thus, the code words of the (16, 11) RM code can be easily computed by using Eq. (3.69). If  $\mathbf{d} = (1 \ 0 \ 1 \ 0 \ 1 \ 1 \ 1 \ 0 \ 0 \ 0 \ 1)$  is the information vector to be encoded, its corresponding code word, according to Eq. (3.69), would be

$$\begin{aligned}
 \mathbf{c} &= \mathbf{d} \cdot \mathbf{G} \\
 &= (1 \ 0 \ 1 \ 0 \ 1 \ 1 \ 1 \ 0 \ 0 \ 0 \ 1) \cdot \mathbf{G} \\
 &= (1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1) \\
 &\quad + (0 \ 0 \ 1 \ 1 \ 0 \ 0 \ 1 \ 1 \ 0 \ 0 \ 1 \ 1 \ 0 \ 0 \ 1 \ 1) \\
 &\quad + (0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1) \\
 &\quad + (0 \ 0 \ 0 \ 1 \ 0 \ 0 \ 0 \ 1 \ 0 \ 0 \ 0 \ 1 \ 0 \ 0 \ 0 \ 1) \\
 &\quad + (0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 0 \ 1 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 0 \ 1) \\
 &\quad + (0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 1 \ 1 \ 1) \\
 &= (1 \ 1 \ 0 \ 1 \ 1 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 0 \ 1 \ 0 \ 0 \ 0)
 \end{aligned} \tag{3.70}$$

Based on Eq. (3.69), the encoding circuit for the (16, 11) RM code can be implemented easily. The encoder for this code is shown in Fig. 3.8, where  $\square$  denotes a ROM to store the identity vector  $\mathbf{v}_0$ , the basis vectors  $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_m$ , and all of their vector products  $r = 2$  at a time,  $\square$  denotes an AND gate, and  $\oplus$  denotes a modulo-2 adder. The encoding operation is very simple. The information  $\mathbf{d} = (1 \ 0 \ 1 \ 0 \ 1 \ 1 \ 1 \ 0 \ 0 \ 0 \ 1)$  to be encoded is multiplied by digits from ROM at AND gates and the outputs of the exclusive-OR gate are shifted into the buffer register and then shifted into the channel.

The Reed algorithm, designed specifically for RM codes, is unusual as compared to decoding algorithms for most codes. An important feature of the Reed algorithm is that the information is recovered directly from the received word without going through syndrome computations. This can best be explained by the following example.

**Example 3.17** Consider again the second-order RM code of length  $2^4 = 16$ . In this case  $r = 2$ ,  $k = 11$ , and  $n - k = 5$ . The corresponding basis will be the row vectors of  $\mathbf{G}$  in Eq. (3.67). The information bits of  $\mathbf{d} = (d_0, d_1, \dots, d_{10})$  are then encoded into the code word

$$\begin{aligned}
 \mathbf{c} &= (c_0, c_1, c_2, \dots, c_{15}) \\
 &= d_0 \mathbf{v}_0 + d_1 \mathbf{v}_1 + d_2 \mathbf{v}_2 + d_3 \mathbf{v}_3 + d_4 \mathbf{v}_4 + d_{1,2} \mathbf{v}_1 \mathbf{v}_2 + d_{1,3} \mathbf{v}_1 \mathbf{v}_3 \\
 &\quad + d_{1,4} \mathbf{v}_1 \mathbf{v}_4 + d_{2,3} \mathbf{v}_2 \mathbf{v}_3 + d_{2,4} \mathbf{v}_2 \mathbf{v}_4 + d_{3,4} \mathbf{v}_3 \mathbf{v}_4
 \end{aligned} \tag{3.71}$$

Substituting all basis vectors of  $\mathbf{G}$  into Eq. (3.71), we get

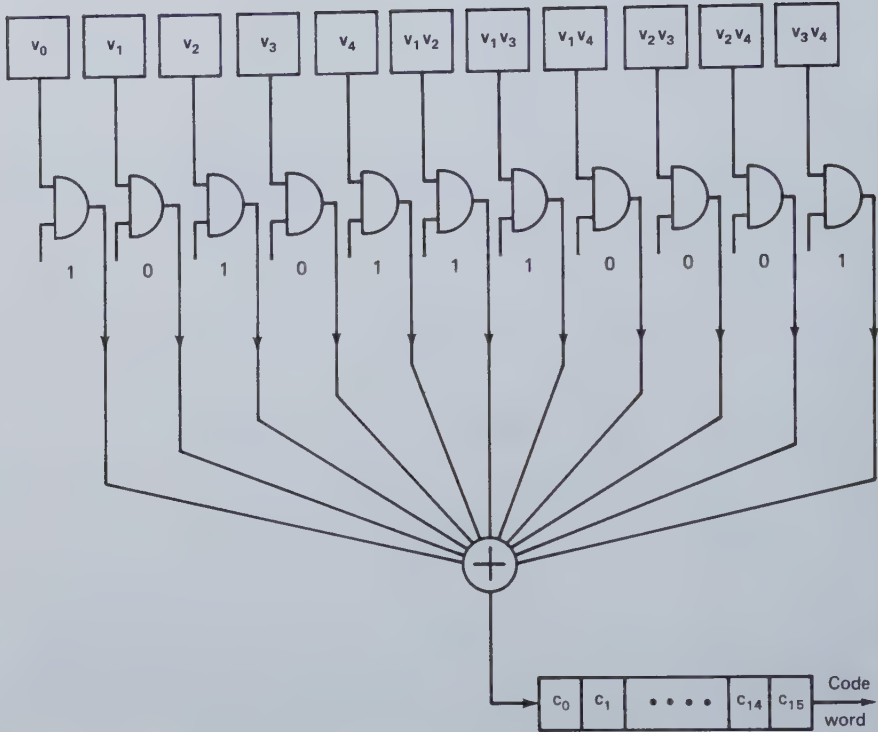


Figure 3.8 Encoding circuit for the second-order RM code with  $m = 4$ .

$$\begin{aligned}
 \mathbf{c} = & d_0(1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1) \\
 & + d_1(0 \ 1 \ 0 \ 1 \ 0 \ 1 \ 0 \ 1 \ 0 \ 1 \ 0 \ 1 \ 0 \ 1 \ 0 \ 1) \\
 & + d_2(0 \ 0 \ 1 \ 1 \ 0 \ 0 \ 1 \ 1 \ 0 \ 0 \ 1 \ 1 \ 0 \ 0 \ 1 \ 1) \\
 & + d_3(0 \ 0 \ 0 \ 0 \ 1 \ 1 \ 1 \ 1 \ 0 \ 0 \ 0 \ 0 \ 1 \ 1 \ 1 \ 1) \\
 & + d_4(0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1) \\
 & + d_{1,2}(0 \ 0 \ 0 \ 1 \ 0 \ 0 \ 0 \ 1 \ 0 \ 0 \ 0 \ 1 \ 0 \ 0 \ 0 \ 1) \\
 & + d_{1,3}(0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 0 \ 1 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 0 \ 1) \\
 & + d_{1,4}(0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 0 \ 1 \ 0 \ 1 \ 0 \ 1) \\
 & + d_{2,3}(0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 1 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 1) \\
 & + d_{2,4}(0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 1 \ 0 \ 0 \ 1 \ 1) \\
 & + d_{3,4}(0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 1 \ 1 \ 1)
 \end{aligned} \tag{3.72}$$

from which each bit of the code word can be expressed by the linear sum of information bits as shown below:



$$\begin{aligned}
 c_0 &= d_0 \\
 c_1 &= d_0 + d_1 \\
 c_2 &= d_0 + d_2 \\
 c_3 &= d_0 + d_1 + d_2 + d_{1,2} \\
 c_4 &= d_0 + d_3 \\
 c_5 &= d_0 + d_1 + d_3 + d_{1,3} \\
 c_6 &= d_0 + d_2 + d_3 + d_{2,3} \\
 c_7 &= d_0 + d_1 + d_2 + d_3 + d_{1,2} + d_{2,3} + d_{1,3} \\
 c_8 &= d_0 + d_4 \\
 c_9 &= d_0 + d_1 + d_4 + d_{1,4} \\
 c_{10} &= d_0 + d_2 + d_4 + d_{2,4} \\
 c_{11} &= d_0 + d_1 + d_2 + d_4 + d_{1,2} + d_{1,4} + d_{2,4} \\
 c_{12} &= d_0 + d_3 + d_4 + d_{3,4} \\
 c_{13} &= d_0 + d_1 + d_3 + d_4 + d_{1,3} + d_{1,4} + d_{3,4} \\
 c_{14} &= d_0 + d_2 + d_3 + d_4 + d_{2,3} + d_{2,4} + d_{3,4} \\
 c_{15} &= d_0 + d_1 + d_2 + d_3 + d_4 + d_{1,2} + d_{1,3} + d_{1,4} + d_{2,3} + d_{2,4} + d_{3,4}
 \end{aligned} \tag{3.73}$$

The information bits  $d_{i,j}$  in Eq. (3.73) are obtained as given below.

$$\begin{aligned}
 d_{1,2} &= c_0 + c_1 + c_2 + c_3 & d_{2,3} &= c_0 + c_2 + c_4 + c_6 \\
 d_{1,2} &= c_4 + c_5 + c_6 + c_7 & d_{2,3} &= c_1 + c_3 + c_5 + c_7 \\
 d_{1,2} &= c_8 + c_9 + c_{10} + c_{11} & d_{2,3} &= c_8 + c_{10} + c_{12} + c_{14} \\
 d_{1,2} &= c_{12} + c_{13} + c_{14} + c_{15} & d_{2,3} &= c_9 + c_{11} + c_{13} + c_{15} \\
 d_{1,3} &= c_0 + c_1 + c_4 + c_5 & d_{2,4} &= c_0 + c_2 + c_8 + c_{10} \\
 d_{1,3} &= c_2 + c_3 + c_6 + c_7 & d_{2,4} &= c_1 + c_3 + c_9 + c_{11} \\
 d_{1,3} &= c_8 + c_9 + c_{12} + c_{13} & d_{2,4} &= c_4 + c_6 + c_{12} + c_{14} \\
 d_{1,3} &= c_{10} + c_{11} + c_{14} + c_{15} & d_{2,4} &= c_5 + c_7 + c_{13} + c_{15} \\
 d_{1,4} &= c_0 + c_1 + c_8 + c_9 & d_{3,4} &= c_0 + c_4 + c_8 + c_{12} \\
 d_{1,4} &= c_2 + c_3 + c_{10} + c_{11} & d_{3,4} &= c_1 + c_5 + c_9 + c_{13} \\
 d_{1,4} &= c_4 + c_5 + c_{12} + c_{13} & d_{3,4} &= c_2 + c_6 + c_{10} + c_{14} \\
 d_{1,4} &= c_6 + c_7 + c_{14} + c_{15} & d_{3,4} &= c_3 + c_7 + c_{11} + c_{15}
 \end{aligned} \tag{3.74}$$

Now, consider decoding of the information bit  $d_{i,j}$ . It can be decoded by setting up  $2^{m-r} = 2^{4-2} = 4$  linear check sums in the  $2^m = 2^4 = 16$  received bits. Each check sum involves  $2^r = 4$  bits of the received word, and each received bit is used in only one check sum. Therefore, since there is only  $2^{m-r-1} - 1 = 1$  error, only one of these estimates is wrong. Thus a majority decision will provide  $d_{i,j}$ .

If we can once recover the information bits  $d_{i,j}$ , then we can compute their contribution to the received word and subtract this contribution. This reduces the problem to that of decoding a smaller code, i.e., a first-order RM code. The decoding procedure is simply a succession of majority decisions.

After these  $d_{i,j}$  have been determined,

$$d_{1,2}\mathbf{v}_1\mathbf{v}_2 + d_{1,3}\mathbf{v}_1\mathbf{v}_3 + d_{1,4}\mathbf{v}_1\mathbf{v}_4 + d_{2,3}\mathbf{v}_2\mathbf{v}_3 + d_{2,4}\mathbf{v}_2\mathbf{v}_4 + d_{3,4}\mathbf{v}_3\mathbf{v}_4$$

can be subtracted from the code word. This, in the absence of errors, would leave the following expression:

$$\begin{aligned} \mathbf{c}' &= (c'_0, c'_1, c'_2, c'_3, \dots, c'_{13}, c'_{14}, c'_{15}) \\ &= d_0\mathbf{v}_0 + d_1\mathbf{v}_1 + d_2\mathbf{v}_2 + d_3\mathbf{v}_3 + d_4\mathbf{v}_4 \\ &= d_0(1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1) \\ &\quad + d_1(0 \ 1 \ 0 \ 1 \ 0 \ 1 \ 0 \ 1 \ 0 \ 1 \ 0 \ 1 \ 0 \ 1 \ 0 \ 1) \\ &\quad + d_2(0 \ 0 \ 1 \ 1 \ 0 \ 0 \ 1 \ 1 \ 0 \ 0 \ 1 \ 1 \ 0 \ 0 \ 1 \ 1) \\ &\quad + d_3(0 \ 0 \ 0 \ 0 \ 1 \ 1 \ 1 \ 1 \ 0 \ 0 \ 0 \ 0 \ 1 \ 1 \ 1 \ 1) \\ &\quad + d_4(0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1) \end{aligned} \quad (3.75)$$

Consequently, from Eq. (3.75), the following set of equations are induced in terms of the modulo-2 sum of the information bits  $d_k$ ,  $k = 0, 1, \dots, 4$ .

$$\begin{aligned} c'_0 &= d_0 & c'_8 &= d_0 + d_4 \\ c'_1 &= d_0 + d_1 & c'_9 &= d_0 + d_1 + d_4 \\ c'_2 &= d_0 + d_2 & c'_{10} &= d_0 + d_2 + d_4 \\ c'_3 &= d_0 + d_1 + d_2 & c'_{11} &= d_0 + d_1 + d_2 + d_4 \\ c'_4 &= d_0 + d_3 & c'_{12} &= d_0 + d_3 + d_4 \\ c'_5 &= d_0 + d_1 + d_3 & c'_{13} &= d_0 + d_1 + d_3 + d_4 \\ c'_6 &= d_0 + d_2 + d_3 & c'_{14} &= d_0 + d_2 + d_3 + d_4 \\ c'_7 &= d_0 + d_1 + d_2 + d_3 & c'_{15} &= d_0 + d_1 + d_2 + d_3 + d_4 \end{aligned} \quad (3.76)$$

Thus, the next set of the information bits  $d_1, d_2, d_3$ , and  $d_4$  can be described by the matching components of  $\mathbf{c}'$  as follows:

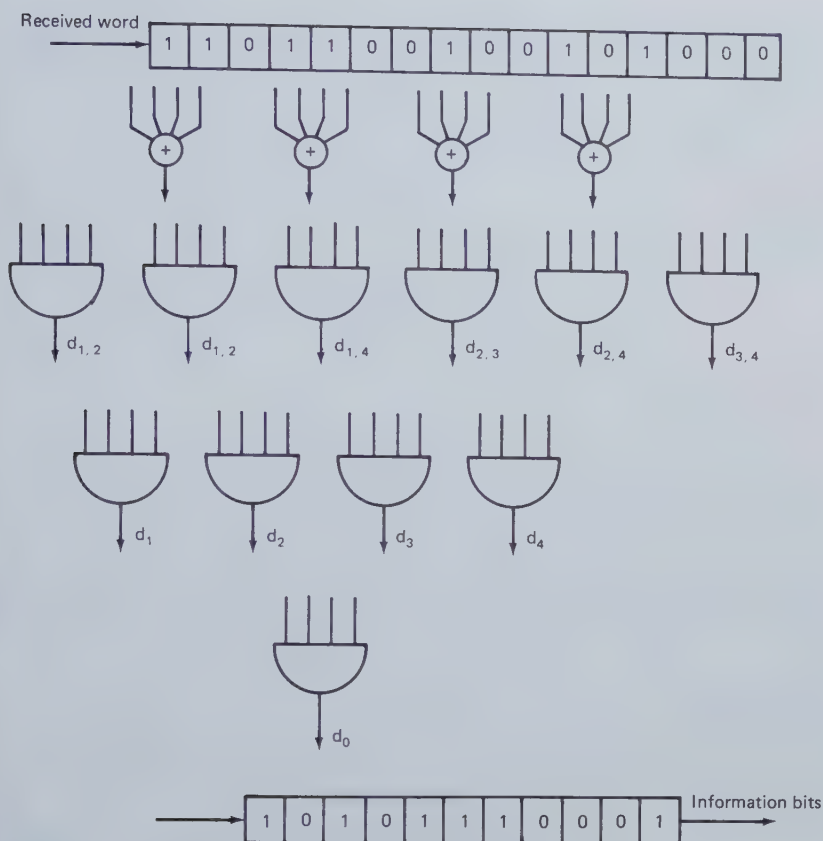


Figure 3.9 Decoding circuit for the (16, 11) RM code.

$$\begin{aligned}
 d_1 &= c'_0 + c'_1 = c'_2 + c'_3 = c'_4 + c'_5 = c'_6 + c'_7 = c'_8 + c'_9 \\
 &= c'_{10} + c'_{11} = c'_{12} + c'_{13} = c'_{14} + c'_{15} \\
 d_2 &= c'_0 + c'_2 = c'_1 + c'_3 = c'_4 + c'_6 = c'_5 + c'_7 = c'_8 + c'_{10} \\
 &= c'_9 + c'_{11} = c'_{12} + c'_{14} = c'_{13} + c'_{15} \\
 d_3 &= c'_0 + c'_4 = c'_1 + c'_5 = c'_2 + c'_6 = c'_3 + c'_7 = c'_8 + c'_{12} \\
 &= c'_9 + c'_{13} = c'_{10} + c'_{14} = c'_{11} + c'_{15} \\
 d_4 &= c'_0 + c'_8 = c'_1 + c'_9 = c'_2 + c'_{10} = c'_3 + c'_{11} = c'_4 + c'_{12} \\
 &= c'_5 + c'_{13} = c'_6 + c'_{14} = c'_7 + c'_{15}
 \end{aligned} \tag{3.77}$$

If only one error occurs in the received word  $\mathbf{r}$ , then the information bits  $d_k$ ,  $k = 1, 2, 3, 4$ , can be determined by the linear check sums of Eq. (3.77) using the majority-decision rule. The decoding algorithm first recovers  $d_{i,j}$  from  $\mathbf{c}$ . Then, it computes  $\mathbf{r}'$  by subtracting the  $d_{i,j}$  contributions from the received word  $\mathbf{r}$ :

$$\mathbf{r}' = \mathbf{r} - (d_{1,2}\mathbf{v}_1\mathbf{v}_2 + d_{1,3}\mathbf{v}_1\mathbf{v}_3 + d_{1,4}\mathbf{v}_1\mathbf{v}_4 + d_{2,3}\mathbf{v}_2\mathbf{v}_3 + d_{2,4}\mathbf{v}_2\mathbf{v}_4 + d_{3,4}\mathbf{v}_3\mathbf{v}_4) \quad (3.78)$$

Finally, the information digit  $\mathbf{d}$  can be determined in a similar manner. That is,

$$\begin{aligned} \mathbf{c}'' &= \mathbf{c}' - (d_1\mathbf{v}_1 + d_2\mathbf{v}_2 + d_3\mathbf{v}_3 + d_4\mathbf{v}_4) \\ &= d_0\mathbf{v}_0 \end{aligned} \quad (3.79)$$

Namely, it gives

$$d_0 = c_0'' = c_1'' = c_2'' = \dots = c_{15}'' \quad (3.80)$$

If this relation is applied to the received word, we get

$$\mathbf{r}'' = \mathbf{r}' - (d_1\mathbf{v}_1 + d_2\mathbf{v}_2 + d_3\mathbf{v}_3 + d_4\mathbf{v}_4) \quad (3.81)$$

Thus, the majority decision will recover  $d_0$  because  $\mathbf{r}''$  contains only one error. The decoding circuit for the second-order RM code with  $m = 4$  is shown in Fig. 3.9.

## PROBLEMS

**3.1** The generator matrix for a (6, 3) linear code over  $GF(2)$  is given by

$$\mathbf{G} = \begin{bmatrix} 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 1 & 1 & 0 \end{bmatrix}$$

- Express  $\mathbf{G}$  in the systematic form.
- Find the code word with (1 1 0) as information bits.
- List the code word generated by the matrix in both systematic and nonsystematic form.
- How many code words are there of Hamming weight 0, 1, 2, ..., 6?

**3.2** The generator matrix for a (7, 3) linear code over  $GF(2)$  is

$$\mathbf{G} = \begin{bmatrix} 0 & 0 & 1 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 & 1 & 0 & 0 \end{bmatrix}$$

- Express  $\mathbf{G}$  in the systematic form.
- Determine the parity-check matrix  $\mathbf{H}$  for the code.
- List the code words generated by the two generator matrices given above and, thus, demonstrate that these matrices generate the same set of code words.

- d Determine the minimum distance of this code.
- e Show that the code word corresponding to the information bits  $\mathbf{d} = (1\ 0\ 1)$  is orthogonal to  $\mathbf{H}$ .

**3.3** Construct an encoder for the (6, 3) code given in Prob. 3.1. Devise both the nonsystematic and systematic encoding circuits for this code.

**3.4** Devise a syndrome generator for the (7, 3) systematic code given in Prob. 3.2.

**3.5** Consider an (8, 4) systematic code whose parity-check matrix is

$$\mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 \end{bmatrix}$$

- a Express the syndrome bits in terms of the received word bits.
- b Construct the syndrome circuit for this code.
- c Set the 16 coset leaders, and determine their corresponding syndromes.
- d Construct the standard array of this code.
- e Obtain the syndrome of the received word  $\mathbf{r} = (1\ 0\ 0\ 1\ 1\ 1\ 1\ 0)$  when the code word  $\mathbf{c} = (1\ 0\ 0\ 1\ 1\ 0\ 1\ 0)$  is transmitted.
- f Find the coset leader (correctable error pattern) corresponding to the syndrome obtained in part e and decode  $\mathbf{r}$  into the actual code word  $\mathbf{c}$ .

**3.6** Based on the MacWilliams identity, the weight enumerator  $B(z)$  of the dual code  $C_D$  is expressed in terms of the weight enumerator  $A(z)$  of the code  $C$  by

$$B(z) = 2^{-k} (1 + z)^n A\left(\frac{1 - z}{1 + z}\right)$$

Derive  $A(z)$  in terms of  $B(z)$  as

$$A(z) = 2^{-(n-k)} (1 + z)^n B\left(\frac{1 - z}{1 + z}\right)$$

**3.7** Determine the weight distribution of the (8, 4) linear code given in Prob. 3.5. Compute the probability of an undetected error of this code when the transition probability of a BSC is  $p = 10^{-2}$ .

**3.8** The weight enumerator of the  $(2^m - 1, m)$  code, a dual code of a  $(2^m - 1, 2^m - m - 1)$  Hamming code, is given by

$$B(z) = 1 + (2^m - 1)z^{2^{m-1}}$$

Derive the weight enumerator  $A(z)$  of the Hamming code.

**3.9** A fairly good linear code of length  $n = 63$  with the minimum distance  $d_{\min} = 5$  is desired.



- a* Determine the range of parity-check bits.
  - b* Find the largest possible value of  $k$ .
  - c* Show that your estimated code is a (63, 51) BCH code.
- 3.10** Consider the (8, 4) linear code given in Prob. 3.7.
  - a* Is this code self-dual?
  - b* Is this code perfect or quasi-perfect?
  - c* Explain that a decoder for the (8, 4) code is either a complete decoder or an incomplete decoder.
- 3.11** Consider a (31, 26) Hamming code for  $m = 5$ .
  - a* Compute the weight enumerator of the (31, 5) dual code.
  - b* Compute the probability of an undetected error for a (31, 26) Hamming code on a BSC with  $p = 10^{-2}$ .
- 3.12** Consider a two-dimensional (36, 9, 9) product code.
  - a* What are the component codes?
  - b* What is the minimum distance of this product code?
  - c* Determine the error-correcting capabilities  $t$  and  $b$ .
  - d* Construct the encoded arrays of this product code.
- 3.13** Consider the second-order RM code for  $m = 3$ .
  - a* Determine the parameters of this RM code.
  - b* Find all the basis vectors of the generator matrix for this code.
  - c* Devise an encoder for this RM code.

# Cyclic Codes

Cyclic codes form a subclass of linear block codes. They possess algebraic and geometric structures which, if properly used, make possible the development of encoder-decoder circuits. A number of efficient encoding and decoding algorithms have been derived for cyclic codes, making it possible to implement long block codes with a large number of code words in practical communication systems. The search for effective error-correcting codes has been most successful through the more than 30 years of coding history.

## 4.1 Description of Cyclic Codes

An  $(n, k)$  linear code  $C$  over  $GF(q)$  is called a cyclic code if whenever a code word  $\mathbf{c} = (c_0, c_1, \dots, c_{n-2}, c_{n-1})$  is in  $C$ ,  $\mathbf{c}^{(1)} = (c_{n-1}, c_0, c_1, \dots, c_{n-3}, c_{n-2})$  is also in  $C$ . When dealing with cyclic codes, it is convenient to associate a code word  $\mathbf{c}$  with a code polynomial  $c(x)$  of degree  $n - 1$  or less:

$$c(x) = c_0 + c_1x + c_2x^2 + \dots + c_{n-1}x^{n-1} \quad (4.1)$$

Then we say that  $\mathbf{c}$  and  $c(x)$  have a one-to-one correspondence. Consider the polynomial,  $xc(x)$  multiplied by  $x$ :

$$xc(x) = c_0x + c_1x^2 + \dots + c_{n-1}x^n \quad (4.2)$$

Since this polynomial is of degree  $n$  when  $c_{n-1} = 1$ , Eq. (4.2) cannot represent a code word. However, if we divide  $xc(x)$  by  $x^n + 1$ , we then get

$$xc(x) = c_{n-1}(x^n + 1) + c^{(1)}(x) \quad (4.3)$$

where  $c^{(1)}(x) = c_{n-1} + c_0x + \dots + c_{n-2}x^{n-1}$  is the remainder. Note here

that the polynomial  $c^{(1)}(x)$  represents an  $n$ -tuple code word  $\mathbf{c}^{(1)} = (c_{n-1}, c_0, c_1, \dots, c_{n-2})$  which is just a cyclic shift of the code word  $\mathbf{c}$  by one place to the right. If  $xc(x)$  is divided by  $x^n + 1$ , the remainder  $c^{(1)}(x)$  is expressed as

$$c^{(1)}(x) = xc(x) \bmod (x^n + 1) \quad (4.4)$$

Similarly, if  $c(x)$  represents a code word in an  $(n, k)$  cyclic code, then  $x^i c(x) \bmod (x^n + 1)$  is also a code word of the cyclic code  $C$ , and it can be written as

$$x^i c(x) = q(x)(x^n + 1) + c^{(i)}(x) \quad (4.5)$$

where the remainder polynomial  $c^{(i)}(x)$  represents an  $n$ -tuple code word obtained by cyclically shifting  $\mathbf{c}$  by  $i$  places and  $q(x)$  is the quotient polynomial.

Let  $g(x)$  be the generator polynomial of degree  $n - k$  which is a factor of  $x^n + 1$ . This means that any factor of  $x^n + 1$  with degree  $n - k$  generates an  $(n, k)$  cyclic code. Since  $g(x)$  is a divisor of  $x^n + 1$ , it follows that

$$x^n + 1 = h(x)g(x) \quad (4.6)$$

where  $h(x)$  is the polynomial of degree  $k$  and has the following form:

$$h(x) = h_0 + h_1x + \dots + h_kx^k \quad (4.7)$$

with  $h_0 = h_k = 1$ . We call  $h(x)$  the parity polynomial of an  $(n, k)$  cyclic code. Furthermore, we define a polynomial  $d(x)$  of degree  $k - 1$  or less

$$d(x) = d_0 + d_1x + \dots + d_{k-1}x^{k-1} \quad (4.8)$$

as the information polynomial whose coefficient set  $\mathbf{d} = (d_0, d_1, \dots, d_{k-1})$  represents the  $k$ -bit information sequence. Clearly, the product  $d(x)g(x)$  is a polynomial which represents a code word and is of degree  $n - 1$  or less. Since each code polynomial  $c(x)$  is a multiple of  $g(x)$ , the encoding can be achieved by multiplying the information polynomial  $d(x)$  by  $g(x)$ . Thus an  $(n, k)$  cyclic code is completely specified by the monic polynomial

$$g(x) = 1 + g_1x + \dots + g_{n-k-1}x^{n-k-1} + x^{n-k} \quad (4.9)$$

which is called the generator polynomial of the code. Thus from Eq. (4.5) it is easily seen that

$$c^{(i)}(x) = [q(x)h(x) + x^i d(x)]g(x) = d_i(x)g(x) \quad (4.10)$$

which proves that if any code word is cyclically shifted  $i$  times, another code word in  $C$  is formed. Thus, we see that the  $2^k$  code words

possessing the cyclic property can be generated by the generator polynomial  $g(x)$ .

The dual code of an  $(n, k)$  cyclic code generated by  $g(x)$  is an  $(n, n - k)$  cyclic code generated by  $h(x)$ , which is called the parity polynomial of an  $(n, k)$  cyclic code. Since  $h(x)g(x) = x^n + 1$ , we have

$$h(x)g(x) = 0 \pmod{x^n + 1} \quad (4.11)$$

which implies that the polynomials  $g(x)$  and  $h(x)$  are orthogonal. Furthermore, the polynomials  $x^i g(x)$  and  $x^j h(x)$  are also orthogonal for all  $i$  and  $j$ .

**Example 4.1** Consider the  $(7, 4)$  cyclic code whose generator polynomial of degree 3 is a factor of  $x^7 + 1$ . Since the polynomial  $x^7 + 1$  can be factored as  $x^7 + 1 = (1 + x)(1 + x + x^3)(1 + x^2 + x^3)$ , we have two generator polynomials, that is,  $g(x) = 1 + x + x^3$  or  $g(x) = 1 + x^2 + x^3$ . Let  $\mathbf{d} = (1 \ 0 \ 1 \ 0)$  be the information sequence to be encoded. The corresponding information polynomial is then  $d(x) = 1 + x^2$ . If we use  $c(x) = d(x)g(x)$  to generate a code word that is in non-systematic form, the code polynomial becomes

$$\begin{aligned} c(x) &= d(x)g(x) = (1 + x^2)(1 + x^2 + x^3) \\ &= 1 + x^3 + x^4 + x^5 \end{aligned}$$

or the code word  $\mathbf{c} = (1 \ 0 \ 0 \ 1 \ 1 \ 1 \ 0)$ . The  $(7, 4)$  cyclic code generated by  $g(x) = 1 + x^2 + x^3$  is shown in Table 4.1, from which we see that this code has a minimum distance of 3 and is a single-error-correcting code.

TABLE 4.1 A  $(7, 4)$  Cyclic Code Generated by  $g(x) = 1 + x^2 + x^3$

| Information sequence $\mathbf{d}$ | $d(x)$              | $c(x) = d(x)g(x)$                     | Code word $\mathbf{c}$ |
|-----------------------------------|---------------------|---------------------------------------|------------------------|
| 0 0 0 0                           | 0                   | 0                                     | 0 0 0 0 0 0 0          |
| 0 0 0 1                           | $x^3$               | $x^3 + x^5 + x^6$                     | 0 0 0 1 0 1 1          |
| 0 0 1 0                           | $x^2$               | $x^2 + x^4 + x^5$                     | 0 0 1 0 1 1 0          |
| 0 0 1 1                           | $x^2 + x^3$         | $x^2 + x^3 + x^4 + x^6$               | 0 0 1 1 1 0 1          |
| 0 1 0 0                           | $x$                 | $x + x^3 + x^4$                       | 0 1 0 1 1 0 0          |
| 0 1 0 1                           | $x + x^3$           | $x + x^4 + x^5 + x^6$                 | 0 1 0 0 1 1 1          |
| 0 1 1 0                           | $x + x^2$           | $x + x^2 + x^3 + x^5$                 | 0 1 1 1 0 1 0          |
| 0 1 1 1                           | $x + x^2 + x^3$     | $x + x^2 + x^6$                       | 0 1 1 0 0 0 1          |
| 1 0 0 0                           | 1                   | $1 + x^2 + x^3$                       | 1 0 1 1 0 0 0          |
| 1 0 0 1                           | $1 + x^3$           | $1 + x^2 + x^5 + x^6$                 | 1 0 1 0 0 1 1          |
| 1 0 1 0                           | $1 + x^2$           | $1 + x^3 + x^4 + x^5$                 | 1 0 0 1 1 1 0          |
| 1 0 1 1                           | $1 + x^2 + x^3$     | $1 + x^4 + x^6$                       | 1 0 0 0 1 0 1          |
| 1 1 0 0                           | $1 + x$             | $1 + x + x^2 + x^4$                   | 1 1 1 0 1 0 0          |
| 1 1 0 1                           | $1 + x + x^3$       | $1 + x + x^2 + x^3 + x^4 + x^5 + x^6$ | 1 1 1 1 1 1 1          |
| 1 1 1 0                           | $1 + x + x^2$       | $1 + x + x^5$                         | 1 1 0 0 0 1 0          |
| 1 1 1 1                           | $1 + x + x^2 + x^3$ | $1 + x + x^3 + x^6$                   | 1 1 0 1 0 0 1          |

**Example 4.2** Consider the (7, 3) dual code of the (7, 4) cyclic code. Since the generator polynomial  $g(x)$  is of degree 4, this code has as its generator polynomial either  $g(x) = 1 + x + x^2 + x^4$  or  $g(x) = 1 + x^2 + x^3 + x^4$ . For  $\mathbf{d} = (0\ 1\ 1)$ , the corresponding information polynomial is  $d(x) = x + x^2$ . Thus the code polynomial gives

$$\begin{aligned} c(x) &= (x + x^2)(1 + x + x^2 + x^4) \\ &= x + x^4 + x^5 + x^6 \end{aligned}$$

or  $\mathbf{c} = (0\ 1\ 0\ 0\ 1\ 1\ 1)$ . The (7, 3) dual code generated by  $g(x) = 1 + x + x^2 + x^4$  is listed in Table 4.2. Notice that this code is not in systematic form. Since this code has  $d_{\min} = 4$ , the error-correcting capability is  $t = \lfloor (d_{\min} - 1)/2 \rfloor = 1$ , i.e., it is a single-error-correcting code.

**TABLE 4.2 A Dual (7, 3) Cyclic Code Generated by  $g(x) = 1 + x + x^2 + x^4$**

| Information sequence $\mathbf{d}$ | $d(x)$        | $c(x) = d(x)g(x)$       | Code word $\mathbf{c}$ |
|-----------------------------------|---------------|-------------------------|------------------------|
| 0 0 0                             | 0             | 0                       | 0 0 0 0 0 0 0          |
| 0 0 1                             | $x^2$         | $x^2 + x^3 + x^4 + x^6$ | 0 0 1 1 1 0 1          |
| 0 1 0                             | $x$           | $x + x^2 + x^3 + x^5$   | 0 1 1 1 0 1 0          |
| 0 1 1                             | $x + x^2$     | $x + x^4 + x^5 + x^6$   | 0 1 0 0 1 1 1          |
| 1 0 0                             | 1             | $1 + x + x^2 + x^4$     | 1 1 1 0 1 0 0          |
| 1 0 1                             | $1 + x^2$     | $1 + x + x^3 + x^6$     | 1 1 0 1 0 0 1          |
| 1 1 0                             | $1 + x$       | $1 + x^3 + x^4 + x^5$   | 1 0 0 1 1 1 0          |
| 1 1 1                             | $1 + x + x^2$ | $1 + x^2 + x^5 + x^6$   | 1 0 1 0 0 1 1          |

## 4.2 Systematic Code Structure

Given the generator polynomial  $g(x)$  of an  $(n, k)$  cyclic code, the code can be put into the following systematic form:

$$\mathbf{c} = (\gamma_0, \gamma_1, \dots, \gamma_{n-k-1}, d_0, d_1, \dots, d_{k-1}) \quad (4.12)$$

From Eq. (4.12), the information polynomial  $d(x)$  and the check polynomial  $\gamma(x)$  can be expressed as

$$d(x) = d_0 + d_1x + \dots + d_{k-1}x^{k-1} \quad (4.13)$$

$$\gamma(x) = \gamma_0 + \gamma_1x + \dots + \gamma_{n-k-1}x^{n-k-1} \quad (4.14)$$

respectively, where  $\gamma(x)$  is the remainder polynomial of degree  $n - k - 1$  or less and is obtained by dividing  $x^{n-k}d(x)$  by  $g(x)$ . Thus, we have

$$\begin{aligned} x^{n-k}d(x) &= q(x)g(x) + \gamma(x) \\ &= d_0x^{n-k} + d_1x^{n-k+1} + \dots + d_{k-1}x^{n-1} \end{aligned} \quad (4.15)$$

which represents the unaltered information sequence  $\mathbf{d}$  followed by



$n - k$  parity-check digits. These  $n - k$  parity-check bits are just the coefficients of  $\gamma(x)$  as denoted by Eq. (4.14). Thus from Eq. (4.15),

$$c(x) = \gamma(x) + x^{n-k}d(x) = q(x)g(x) \quad (4.16)$$

corresponds to the code polynomial of degree  $n - 1$  or less and is a multiple of  $g(x)$ . Clearly, this expression corresponds to the code word of Eq. (4.12) in systematic form.

**Example 4.3** Consider the (7, 4) cyclic code with the generator polynomial  $g(x) = 1 + x + x^3$ . Suppose that the information to be encoded is  $\mathbf{d} = (1 \ 1 \ 0 \ 0)$ . The corresponding information polynomial is  $d(x) = 1 + x$ . Since  $n - k = 3$ , we have  $x^3d(x) = x^3 + x^4$ . Dividing  $x^3 + x^4$  by  $g(x) = 1 + x + x^3$  yields the remainder polynomial  $\gamma(x) = 1 + x^2$ . Therefore the code polynomial expressed by Eq. (4.16) becomes  $c(x) = 1 + x^2 + x^3 + x^4$ , or  $\mathbf{c} = (1 \ 0 \ 1 \ 1 \ 1 \ 0 \ 0)$ . Thus all 16 code words, in either systematic or nonsystematic form, can be found as listed in Table 4.3.

**TABLE 4.3 A (7, 4) Cyclic Code Generated by  $g(x) = 1 + x + x^3$**

| Information sequence | Nonsystematic word | Systematic word |
|----------------------|--------------------|-----------------|
| 0 0 0 0              | 0 0 0 0 0 0 0      | 0 0 0 0 0 0 0   |
| 0 0 0 1              | 0 0 0 1 1 0 1      | 1 0 1 0 0 0 1   |
| 0 0 1 0              | 0 0 1 1 0 1 0      | 1 1 1 0 0 1 0   |
| 0 0 1 1              | 0 0 1 0 1 1 1      | 0 1 0 0 0 1 1   |
| 0 1 0 0              | 0 1 1 0 1 0 0      | 0 1 1 0 1 0 0   |
| 0 1 0 1              | 0 1 1 1 0 0 1      | 1 1 0 0 1 0 1   |
| 0 1 1 0              | 0 1 0 1 1 1 0      | 1 0 0 0 1 1 0   |
| 0 1 1 1              | 0 1 0 0 0 1 1      | 0 0 1 0 1 1 1   |
| 1 0 0 0              | 1 1 0 1 0 0 0      | 1 1 0 1 0 0 0   |
| 1 0 0 1              | 1 1 0 0 1 0 1      | 0 1 1 1 0 0 1   |
| 1 0 1 0              | 1 1 1 0 0 1 0      | 0 0 1 1 0 1 0   |
| 1 0 1 1              | 1 1 1 1 1 1 1      | 1 0 0 1 0 1 1   |
| 1 1 0 0              | 1 0 1 1 1 0 0      | 1 0 1 1 1 0 0   |
| 1 1 0 1              | 1 0 1 0 0 0 1      | 0 0 0 1 1 0 1   |
| 1 1 1 0              | 1 0 0 0 1 1 0      | 0 1 0 1 1 1 0   |
| 1 1 1 1              | 1 0 0 1 0 1 1      | 1 1 1 1 1 1 1   |

The generator matrix  $\mathbf{G}$  for a (7, 4) cyclic code can be constructed from a set of  $k = 4$  linearly independent code words. The basis vectors corresponding to the four single-weight information sequences are found by using Eq. (4.16) as illustrated in Table 4.4.

**TABLE 4.4 Basis Vectors for the (7, 4) Cyclic Code**

| $\mathbf{d}$ | $d(x)$ | $x^3d(x)$ | $\gamma(x)$   | $c(x) = \gamma(x) + x^3d(x)$ | $\mathbf{c}$  |
|--------------|--------|-----------|---------------|------------------------------|---------------|
| 1 0 0 0      | 1      | $x^3$     | $1 + x$       | $1 + x + x^3$                | 1 1 0 1 0 0 0 |
| 0 1 0 0      | $x$    | $x^4$     | $x + x^2$     | $x + x^2 + x^4$              | 0 1 1 0 1 0 0 |
| 0 0 1 0      | $x^2$  | $x^5$     | $1 + x + x^2$ | $1 + x + x^2 + x^5$          | 1 1 1 0 0 1 0 |
| 0 0 0 1      | $x^3$  | $x^6$     | $1 + x^2$     | $1 + x^2 + x^6$              | 1 0 1 0 0 0 1 |

Row vectors of the generator matrix  $\mathbf{G}$  consist of four linearly independent basis vectors as shown below.

$$\mathbf{G} = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 1 \end{bmatrix} = [\mathbf{I}_{4 \times 3} : \mathbf{I}_{4 \times 4}]$$

The parity-check matrix  $\mathbf{H}$  for the (7, 4) cyclic code with the generator matrix  $\mathbf{G}$  is easily obtained as

$$\mathbf{H} = [\mathbf{I}_{3 \times 3} : \mathbf{I}_{3 \times 4}^T] = \begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \end{bmatrix}$$

### 4.3 Generator Matrix and Parity-Check Matrix

For an  $(n, k)$  cyclic code with the generator polynomial  $g(x)$ , the generator matrix  $\mathbf{G}$  in systematic form can be found easily. Dividing  $x^{n-k+i}$  by the generator polynomial  $g(x)$  yields

$$x^{n-k+i} = q_i(x)g(x) + \gamma_i(x) \quad i = 0, 1, \dots, k-1 \quad (4.17)$$

$$\text{where} \quad \gamma_i(x) = \gamma_{i,0} + \gamma_{i,1}x + \gamma_{i,2}x^2 + \dots + \gamma_{i,n-k-1}x^{n-k-1} \quad (4.18)$$

is the remainder polynomial of degree  $n-k-1$  or less. By rearranging Eq. (4.17), we get

$$c_i(x) = q_i(x)g(x) = \gamma_i(x) + x^{n-k+i} \quad i = 0, 1, \dots, k-1$$

which are multiples of  $g(x)$ . Therefore they are  $k$  code polynomials as shown below.

| Code polynomials                         | Code words                                                                                          |
|------------------------------------------|-----------------------------------------------------------------------------------------------------|
| $c_0(x) = \gamma_0(x) + x^{n-k}$         | $\mathbf{c}_0 = (\gamma_{0,0}, \gamma_{0,1}, \dots, \gamma_{0,n-k-1}, 1 \ 0 \ 0 \dots 0)$           |
| $c_1(x) = \gamma_1(x) + x^{n-k+1}$       | $\mathbf{c}_1 = (\gamma_{1,0}, \gamma_{1,1}, \dots, \gamma_{1,n-k-1}, 0 \ 1 \ 0 \dots 0)$           |
| $\vdots$                                 | $\vdots$                                                                                            |
| $c_{k-1}(x) = \gamma_{k-1}(x) + x^{n-1}$ | $\mathbf{c}_{k-1} = (\gamma_{k-1,0}, \gamma_{k-1,1}, \dots, \gamma_{k-1,n-k-1}, 0 \ 0 \ 0 \dots 1)$ |

Arranging these  $k$  code words as rows of a  $k \times n$  matrix produces the generator matrix  $\mathbf{G}$  in systematic form for an  $(n, k)$  cyclic code.

$$\begin{aligned}
 \mathbf{G} = \begin{bmatrix} \mathbf{c}_0 \\ \mathbf{c}_1 \\ \vdots \\ \mathbf{c}_{k-1} \end{bmatrix} &= \begin{bmatrix} \gamma_{0,0} & \gamma_{0,1} & \cdots & \gamma_{0,n-k-1} & 1 & 0 & 0 \cdots 0 \\ \gamma_{1,0} & \gamma_{1,1} & \cdots & \gamma_{1,n-k-1} & 0 & 1 & 0 \cdots 0 \\ \vdots & \vdots & & \vdots & \vdots & \vdots & \vdots \\ \gamma_{k-1,0} & \gamma_{k-1,1} & \cdots & \gamma_{k-1,n-k-1} & 0 & 0 & 0 \cdots 1 \end{bmatrix} \quad (4.19) \\
 &= [\mathbf{\Gamma}_{k \times (n-k)} : \mathbf{I}_k]
 \end{aligned}$$

The corresponding parity-check matrix  $\mathbf{H}$  for an  $(n, k)$  cyclic code is

$$\begin{aligned}
 \mathbf{H} &= [\mathbf{I}_{n-k} : \mathbf{\Gamma}_{(n-k) \times k}^T] \\
 &= \begin{bmatrix} 1 & 0 & 0 \cdots 0 & \gamma_{0,0} & \gamma_{1,0} & \cdots & \gamma_{k-1,0} \\ 0 & 1 & 0 \cdots 0 & \gamma_{0,1} & \gamma_{1,1} & \cdots & \gamma_{k-1,1} \\ 0 & 0 & 1 \cdots 0 & \gamma_{0,2} & \gamma_{1,2} & \cdots & \gamma_{k-1,2} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 \cdots 1 & \gamma_{0,n-k-1} & \gamma_{1,n-k-1} & \cdots & \gamma_{k-1,n-k-1} \end{bmatrix} \quad (4.20)
 \end{aligned}$$

**Example 4.4** Consider again the  $(7, 4)$  cyclic code generated by  $g(x) = 1 + x^2 + x^3$ . The result of dividing  $x^{3+i}$  by  $g(x)$  is given in the following table.

| $i$ | $x^{3+i}$ | $\gamma_i(x)$ | $c_i(x) = \gamma_i(x) + x^{3+i}$ | $\mathbf{c}_i$ |
|-----|-----------|---------------|----------------------------------|----------------|
| 0   | $x^3$     | $1 + x^2$     | $1 + x^2 + x^3$                  | 1 0 1 1 0 0 0  |
| 1   | $x^4$     | $1 + x + x^2$ | $1 + x + x^2 + x^4$              | 1 1 1 0 1 0 0  |
| 2   | $x^5$     | $1 + x$       | $1 + x + x^5$                    | 1 1 0 0 0 1 0  |
| 3   | $x^6$     | $x + x^2$     | $x + x^2 + x^6$                  | 0 1 1 0 0 0 1  |

By using these four code words  $\mathbf{c}_i$  as rows of a  $4 \times 7$  matrix, we obtain the following generator matrix  $\mathbf{G}$  in systematic form:

$$\mathbf{G} = \begin{bmatrix} 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 1 \end{bmatrix}$$

The corresponding parity-check matrix  $\mathbf{H}$  in systematic form is

$$\mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 & 0 & 1 \end{bmatrix}$$

Consider a set of  $k$  linearly independent polynomials  $g(x), xg(x), \dots, x^{k-1}g(x)$ . If the  $k$   $n$ -tuples corresponding to these  $k$  polynomials are

used as the rows of an  $k \times n$  matrix, then we obtain the following generator matrix  $\mathbf{G}$  in nonsystematic form:

$$\mathbf{G} = \begin{bmatrix} g_0(x) & = & g(x) \\ g_1(x) & = & xg(x) \\ \vdots & & \\ g_{k-1}(x) & = & x^{k-1}g(x) \end{bmatrix} \quad (4.21)$$

$$= \begin{bmatrix} g_0 & g_1 & g_2 & \cdots & g_{n-k-1} & g_{n-k} & 0 & 0 & \cdots & 0 \\ 0 & g_0 & g_1 & g_2 & \cdots & g_{n-k-1} & g_{n-k} & 0 & \cdots & 0 \\ \vdots & & & & & & & & & \\ 0 & 0 & 0 & \cdots & g_0 & g_1 & \cdots & g_{n-k-1} & g_{n-k} \end{bmatrix}$$

where  $g_0 = g_{n-k} = 1$ .

**Example 4.5** The dual code of the (7, 4) cyclic code generated by  $g(x) = 1 + x^2 + x^3$  is a (7, 3) cyclic code with the generator polynomial  $h(x) = 1 + x^2 + x^3 + x^4$ . The row vectors of its generator matrix correspond to the polynomials  $x^i h(x)$ ,  $i = 0, 1, 2$ . Hence we have

$$\mathbf{G} = \begin{bmatrix} h(x) \\ xh(x) \\ x^2h(x) \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \end{bmatrix} \quad (4.22)$$

On the other hand, the generator matrix in nonsystematic form for the (7, 4) cyclic code with  $g(x) = 1 + x^2 + x^3$  is obtained from the polynomials  $x^i g(x) = x^i + x^{2+i} + x^{3+i}$ ,  $i = 0, 1, 2, 3$ . Thus the generator matrix for this (7, 4) cyclic code in nonsystematic form is given by

$$\mathbf{G} = \begin{bmatrix} g(x) \\ xg(x) \\ x^2g(x) \\ x^3g(x) \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 \end{bmatrix} \quad (4.23)$$

Now, from Eq. (4.11) we know that when two polynomials  $g(x)$  and  $h(x)$  are orthogonal, the two vectors corresponding to  $g(x)$  and  $h(x)$  are orthogonal only if the ordered elements of one of these vectors are reversed. Therefore, for the (7, 4) cyclic code with the generator matrix  $\mathbf{G}$  given by Eq. (4.23) the parity-check matrix can be obtained by taking the rows of Eq. (4.22) in reverse order, that is,

$$\mathbf{H} = \begin{bmatrix} 0 & 0 & 1 & 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 1 & 0 & 0 \end{bmatrix} \quad (4.24)$$

To verify that Eq. (4.24) is correct, use Eqs. (4.23) and (4.24) to show that  $\mathbf{G} \cdot \mathbf{H}^T = \mathbf{0}$ .

To transform Eq. (4.21) into a generator matrix that has the systematic form of Eq. (4.19), follow the steps given below.

1. Use  $g(x)$  as the first row.
2. To generate the second row, cyclically shift the first row one column to the right. This corresponds to the operation  $xg(x)$ . But the  $(n - k)$ th column entry must be 0. If this entry is 1, add the first row to it. Thus the second row is  $xg(x)$  if the coefficient of  $x^{n-k}$  in  $g(x)$  is 0, or  $xg(x) + g(x)$  if the coefficient is 1.
3. To generate the third row, repeat the same process, i.e., shift the entries in the second row one column to the right. Add  $g(x)$  if the  $(n - k)$ th column entry is not 0. Repeat this process for all the rows until the  $(k - 1)$ th row is reached.

It is apparent that the systematic form of  $\mathbf{G}$  is obtained from the nonsystematic form of  $\mathbf{G}$  by successive addition of rows using the rules described above. Notice that this operation does not change the code words in  $C$  but simply reorders them.

**Example 4.6** Consider the  $(7, 3)$  cyclic code where  $g(x) = 1 + x + x^2 + x^4$ . Using Eq. (4.21), the nonsystematic form of  $\mathbf{G}$  is

$$\mathbf{G} = \begin{bmatrix} g_0(x) = g(x) \\ g_1(x) = xg(x) \\ g_2(x) = x^2g(x) \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 1 \end{bmatrix}$$

To generate the systematic form of  $\mathbf{G}$ , we follow the rules above and find the systematic matrix as shown in Eq. (4.19).

$$\mathbf{G} = \begin{bmatrix} g_0(x) = g(x) \\ g_1(x) = xg_0(x) \\ g_2(x) = xg_1(x) + g_0(x) \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{bmatrix}$$

**Example 4.7** Consider the  $(7, 4)$  cyclic code with  $g(x) = 1 + x^2 + x^3$ . The generator matrix in nonsystematic form is

$$\mathbf{G} = \begin{bmatrix} g(x) \\ xg(x) \\ x^2g(x) \\ x^3g(x) \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 \end{bmatrix}$$



from which the systematic form of  $G$  is easily obtained as

$$G = \begin{bmatrix} g(x) \\ xg_0(x) + g_0(x) \\ xg_1(x) + g_0(x) \\ xg_2(x) \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 1 \end{bmatrix}$$

**Theorem 4.1** The  $(n, n-k)$  dual code of an  $(n, k)$  cyclic code generated by  $g(x)$  is also cyclic and is generated by the polynomial  $h^*(x) = x^k h(x^{-1})$ , where  $h(x) = (x^n + 1)/g(x)$ .

**Proof** Since  $g(x)h(x) = x^n + 1$ , this expression can also be written as  $g(x^{-1})h(x^{-1}) = x^{-n} + 1$  or  $x^n g(x^{-1})h(x^{-1}) = x^n + 1$ . If we rearrange it as  $x^{n-k}g(x^{-1}) \cdot x^k h(x^{-1}) = g^*(x)h^*(x) = x^n + 1$ , then  $h^*(x) = x^k h(x^{-1})$  is certainly a factor of  $x^n + 1$ . Since the degree of  $h(x)$  is  $k$ , its reciprocal polynomial  $h^*(x)$  must be of the same degree. Therefore it follows that

$$\begin{aligned} h^*(x) &= x^k(h_0 + h_1x^{-1} + h_2x^{-2} + \dots + h_kx^{-k}) \\ &= h_k + h_{k-1}x + h_{k-2}x^2 + \dots + h_1x^{k-1} + h_0x^k \end{aligned}$$

Thus we see that  $h^*(x)$  is a divisor of  $x^n + 1$  and generates an  $(n, n-k)$  cyclic code with the following generator matrix:

$$\begin{aligned} H &= \begin{bmatrix} h^*(x) \\ xh^*(x) \\ x^2h^*(x) \\ \vdots \\ x^{n-k-1}h^*(x) \end{bmatrix} \\ &= \begin{bmatrix} h_k & h_{k-1} & h_{k-2} & \dots & h_0 & 0 & 0 & \dots & 0 \\ 0 & h_k & h_{k-1} & h_{k-2} & \dots & h_0 & 0 & \dots & 0 \\ 0 & 0 & h_k & h_{k-1} & h_{k-2} & \dots & h_0 & \dots & 0 \\ \vdots & & \vdots & & \vdots & & \vdots & & \vdots \\ 0 & 0 & 0 & \dots & h_k & h_{k-1} & h_{k-2} & \dots & h_0 \end{bmatrix} \end{aligned} \quad (4.25)$$

Multiplying  $c(x) = d(x)g(x)$  by  $h(x)$  yields

$$\begin{aligned} c(x)h(x) &= d(x)g(x)h(x) \\ &= d(x)(1 + x^n) \\ &= d_0 + d_1x + \dots + d_{k-1}x^{k-1} + d_0x^n + d_1x^{n+1} + \dots + d_{k-1}x^{n+k-1} \end{aligned} \quad (4.26)$$

We can see that the powers  $x^k, x^{k+1}, \dots, x^{n-1}$  do not appear in Eq. (4.26). Thus if we expand the product  $c(x)h(x)$  on the left side of Eq. (4.26), the coefficients of  $x^k, x^{k+1}, \dots, x^{n-1}$  must be 0. Therefore we have

$$\begin{aligned} h_0c_k + h_1c_{k-1} + \dots + h_{k-1}c_1 + h_kc_0 &= 0 \\ h_0c_{k+1} + h_1c_k + \dots + h_kc_1 + h_{k+1}c_0 &= 0 \\ \vdots & \\ h_0c_{n-1} + h_1c_{n-2} + \dots + h_{n-2}c_1 + h_{n-1}c_0 &= 0 \end{aligned} \quad (4.27)$$

But since the highest coefficient of  $h(x)$  is  $h_k$ , all the higher coefficients  $h_{k+1}, h_{k+2}, \dots, h_n$  must be equal to 0. Consequently, Eq. (4.27) becomes

$$\begin{aligned} \sum_{i=0}^k h_i c_{k-i} &= 0 \\ \sum_{i=0}^k h_i c_{k-i+1} &= 0 \\ \vdots & \\ \sum_{i=0}^k h_i c_{n-i-1} &= 0 \end{aligned} \quad (4.28)$$

which further reduces to a single expression:

$$\sum_{i=0}^k h_i c_{n-i-j} = 0 \quad 1 \leq j \leq n-k \quad (4.29)$$

Equation (4.29) indicates that any code word  $\mathbf{c}$  in  $C$  is orthogonal to every row of matrix  $\mathbf{H}$ .

**Example 4.8** Consider the  $(7, 4)$  cyclic code generated by  $g(x) = 1 + x^2 + x^3$ . The parity polynomial is  $h(x) = (x^7 + 1)/g(x) = 1 + x^2 + x^3 + x^4$ . Hence the reciprocal polynomial is  $h^*(x) = x^4 h(x^{-1}) = 1 + x + x^2 + x^4$ . Since  $(x^7 + 1)/h^*(x) = 1 + x + x^3$ ,  $h^*(x)$  is also a factor of  $x^7 + 1$ . Thus the generator matrix  $\mathbf{H}$  for the  $(7, 3)$  dual code is

$$\mathbf{H} = \begin{bmatrix} h^*(x) \\ xh^*(x) \\ x^2h^*(x) \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 1 \end{bmatrix}$$

If we construct all the code words of the  $(7, 3)$  code generated by  $h^*(x) = 1 + x + x^2 + x^4$ , we find

$$\mathbf{c} = \mathbf{d} \cdot \mathbf{H} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 0 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 & 1 \end{bmatrix}$$

which is exactly identical to the listing of code words  $\mathbf{c}$  in Table 4.2. Since  $d_{\min} = 4$ , this code is capable of correcting any single error and detecting any combination of double errors.

#### 4.4 Encoder of Cyclic Codes

We have seen in Sec. 4.2 that the encoding of  $k$ -bit information sequences involves computing the parity-check bits as the remainder  $\gamma(x)$  obtained by dividing  $x^{n-k}d(x)$  by  $g(x)$ . Thus the encoding is accomplished by using a division circuit which is an  $(n - k)$ -stage shift register with feedback connections specified by the generator polynomial  $g(x)$ . In systematic encoding, which is generally used for high-rate codes, the information bits are transmitted without alteration as shown in Fig. 4.1. Note that shifting the information sequence  $d(x)$  into the register from the right end of the shift register is equivalent to multiplying  $d(x)$  by  $x^{n-k}$ . The total number of shifts required to form a code word is  $n$ .

**Example 4.9** Consider the  $(7, 4)$  cyclic code with  $g(x) = 1 + x^2 + x^3$ . Figure 4.2 shows the encoder specified by  $g(x)$ , which has the coefficients  $g_0 = 1$ ,  $g_1 = 0$ ,  $g_2 = g_3 = 1$ . Suppose that the information to be encoded is  $\mathbf{d} = (1 \ 0 \ 0 \ 1)$ . When the gate is on, the  $k$ -bit information is shifted into the register and is also sent

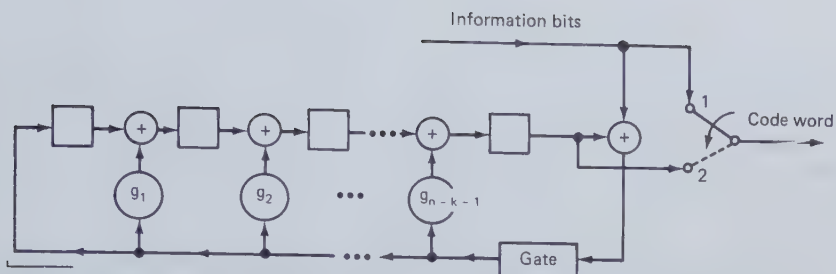


Figure 4.1 Encoder having the  $(n - k)$ -stage shift register for an  $(n, k)$  cyclic code.

to the channel. As soon as the information sequence has completely entered the register, the  $n - k = 3$  parity bits are formed in the register. When the gate is turned off, the parity-check bits are shifted into the channel to form a legitimate code word  $\mathbf{c} = (1\ 1\ 0\ 1\ 0\ 0\ 1)$ . The detailed operation for this encoder is shown in Table 4.5.

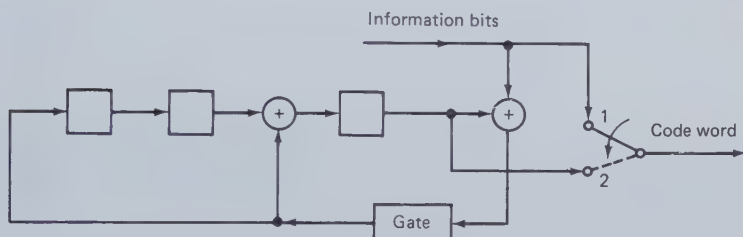
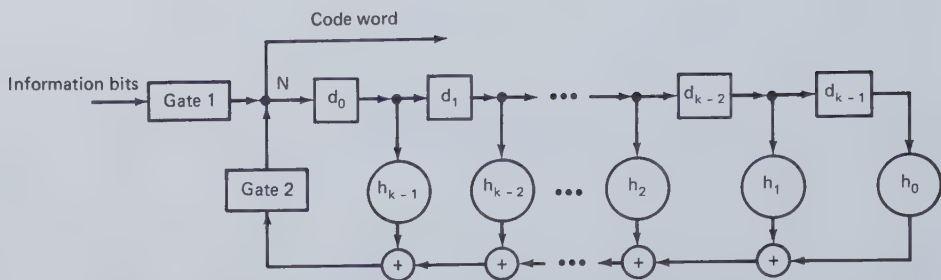


Figure 4.2 Encoder for the  $(7, 4)$  cyclic code generated by  $g(x) = 1 + x^2 + x^3$ .

TABLE 4.5 Encoding Operation for the  $(7, 4)$  Cyclic Encoder Shown in Fig. 4.2

| Shift no. $i$ | Gate | After $i$ th shift |               |
|---------------|------|--------------------|---------------|
|               |      | Register contents  | Output        |
| 0             | On   | 0 0 0              | 1             |
| 1             | On   | 1 0 1              | 0 1           |
| 2             | On   | 1 1 1              | 0 0 1         |
| 3             | On   | 1 1 0              | 1 0 0 1       |
| 4             | Off  | 1 1 0              | 0 1 0 0 1     |
| 5             | Off  | 0 1 1              | 1 0 1 0 0 1   |
| 6             | Off  | 0 0 1              | 1 1 0 1 0 0 1 |

The encoding of an  $(n, k)$  cyclic code can also be accomplished by using its parity polynomial  $h(x)$ . It is therefore possible, and sometimes more convenient, to implement the encoder for low-rate codes using a  $k$ -stage shift register. The  $k$  information bits are stored initially in the  $k$ -stage shift register, and the register is shifted  $n - k$  times to obtain



**Figure 4.3** Encoder employing the  $k$ -stage shift register for an  $(n, k)$  cyclic code generated by the parity polynomial  $h(x) = 1 + h_1x + h_2x^2 + \dots + h_kx^k$ .

the  $n - k$  parity-check bits as explained below. Since  $h_k = 1$ , Eq. (4.29) can also be written as

$$c_{n-k-j} = \sum_{i=0}^{k-1} h_i c_{n-i-j} \quad 1 \leq j \leq n - k \quad (4.30)$$

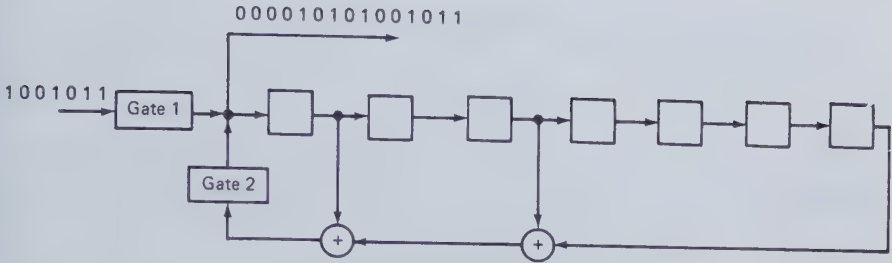
In a systematic code, the information bits in each code word represent  $c_{n-k}, c_{n-k+1}, \dots, c_{n-1}$ , whereas the  $n - k$  parity-check bits  $c_0, c_1, c_2, \dots, c_{n-k-1}$  are completely specified by Eq. (4.30). Using Eq. (4.30), we can then determine the  $n - k$  parity-check bits from the following formulas:

$$\begin{aligned} c_{n-k-1} &= \sum_{i=0}^{k-1} h_i c_{n-i-1} \\ c_{n-k-2} &= \sum_{i=0}^{k-1} h_i c_{n-i-2} \\ &\vdots \\ c_0 &= \sum_{i=0}^{k-1} h_i c_{k-i} \end{aligned} \quad (4.31)$$

An encoder based on Eq. (4.31) and employing a  $k$ -stage shift register is shown in Fig. 4.3. This type of encoder is illustrated by using the  $(15, 7)$  cyclic code in the following example.

**Example 4.10** Consider the  $(15, 7)$  cyclic code generated by  $g(x) = 1 + x^4 + x^6 + x^7 + x^8$ . The parity polynomial  $h(x)$  of this code is then  $h(x) = (x^{15} + 1)/g(x) = 1 + x^4 + x^6 + x^7$ . Since the coefficients of  $h(x)$  are  $h_0 = 1, h_1 = h_2 = h_3 = 0, h_4 = 1, h_5 = 0, h_6 = h_7 = 1$ , the encoding circuit based on  $h(x)$  is as shown in Fig. 4.4. Thus Eq. (4.30), which determines the parity-check bits, becomes





**Figure 4.4** Encoder for the (15, 7) cyclic code based on the parity polynomial  $h(x) = 1 + x^4 + x^6 + x^7$ .

$$c_{8-j} = \sum_{i=0}^6 h_i c_{15-i-j} \quad 1 \leq j \leq 8$$

Assume that we encode the information bits  $\mathbf{d} = (1\ 0\ 0\ 1\ 0\ 1\ 1)$ . Then we must have  $c_8 = c_{11} = c_{13} = c_{14} = 1$  and  $c_9 = c_{10} = c_{12} = 0$  in an encoded code word in systematic form. Now since  $n - k = 8$ , all the parity-check bits are obtained from Eq. (4.31) as described below. The first parity-check bit is

$$\begin{aligned} c_7 &= \sum_{i=0}^6 h_i c_{14-i} \\ &= h_0 c_{14} + h_1 c_{13} + h_2 c_{12} + h_3 c_{11} + h_4 c_{10} + h_5 c_9 + h_6 c_8 \\ &= 1 + 0 + 0 + 0 + 0 + 0 + 1 = 0 \end{aligned}$$

The second parity-check bit is

$$\begin{aligned} c_6 &= h_0 c_{13} + h_1 c_{12} + h_2 c_{11} + h_3 c_{10} + h_4 c_9 + h_5 c_8 + h_6 c_7 \\ &= 1 + 0 + 0 + 0 + 0 + 0 + 0 = 1 \end{aligned}$$

Similarly, the rest of the parity-check bits are

$$\begin{aligned} c_5 &= c_{12} + c_8 + c_6 = 0 + 1 + 1 = 0 \\ c_4 &= c_{11} + c_7 + c_5 = 1 + 0 + 0 = 1 \\ c_3 &= c_{10} + c_6 + c_4 = 0 + 1 + 1 = 0 \\ c_2 &= c_9 + c_5 + c_3 = 0 + 0 + 0 = 0 \\ c_1 &= c_8 + c_4 + c_2 = 1 + 1 + 0 = 0 \\ c_0 &= c_7 + c_3 + c_1 = 0 + 0 + 0 = 0 \end{aligned}$$

Thus the code word that corresponds to the information sequence  $\mathbf{d} = (1\ 0\ 0\ 1\ 0\ 1\ 1)$  is  $\mathbf{c} = (0\ 0\ 0\ 0\ 1\ 0\ 1\ 0\ 1\ 0\ 0\ 1\ 0\ 1\ 1)$ . For checking purposes, let us consider a code word  $c(x) = \gamma(x) + x^{n-k}d(x)$ . Dividing  $x^{n-k}d(x) = x^8(1 + x^3 + x^5 + x^6) = x^8 + x^{11} + x^{13} + x^{14}$  by  $g(x)$  yields the remainder  $\gamma(x) = x^4 + x^6$ . Thus the code

TABLE 4.6 Step-by-Step Operation for the (15, 7) Cyclic Encoder Shown in Fig. 4.4

| Shift<br>no. <i>i</i> | Gate status |           | After <i>i</i> th shift |                                |
|-----------------------|-------------|-----------|-------------------------|--------------------------------|
|                       | Gate<br>1   | Gate<br>2 | Register<br>contents    | Output                         |
| 0                     | On          | Off       | 0 0 0 0 0 0 0           | 1                              |
| 1                     | On          | Off       | 1 0 0 0 0 0 0           | 1 1                            |
| 2                     | On          | Off       | 1 1 0 0 0 0 0           | 0 1 1                          |
| 3                     | On          | Off       | 0 1 1 0 0 0 0           | 1 0 1 1                        |
| 4                     | On          | Off       | 1 0 1 1 0 0 0           | 0 1 0 1 1                      |
| 5                     | On          | Off       | 0 1 0 1 1 0 0           | 0 0 1 0 1 1                    |
| 6                     | On          | Off       | 0 0 1 0 1 1 0           | 1 0 0 1 0 1 1                  |
| 7                     | Off         | On        | 1 0 0 1 0 1 1           | 0 1 0 0 1 0 1 1                |
| 8                     | Off         | On        | 0 1 0 0 1 0 1           | 1 0 1 0 0 1 0 1 1              |
| 9                     | Off         | On        | 1 0 1 0 0 1 0           | 0 1 0 1 0 0 1 0 1 1            |
| 10                    | Off         | On        | 0 1 0 1 0 0 1           | 1 0 1 0 1 0 0 1 0 1 1          |
| 11                    | Off         | On        | 1 0 1 0 1 0 0           | 0 1 0 1 0 1 0 0 1 0 1 1        |
| 12                    | Off         | On        | 0 1 0 1 0 1 0           | 0 0 1 0 1 0 1 0 0 1 0 1 1      |
| 13                    | Off         | On        | 0 0 1 0 1 0 1           | 0 0 0 1 0 1 0 1 0 0 1 0 1 1    |
| 14                    | Off         | On        | 0 0 0 1 0 1 0           | 0 0 0 0 1 0 1 0 1 0 0 1 0 1 1  |
| 15                    | On          | Off       | 0 0 0 0 1 0 1           | *0 0 0 0 1 0 1 0 1 0 0 1 0 1 1 |

\*The first bit of next information block.

polynomial can then be expressed by  $c(x) = x^4 + x^6 + x^8 + x^{11} + x^{13} + x^{14}$ . The legitimate code word corresponding to this  $c(x)$  is  $\mathbf{c} = (0\ 0\ 0\ 0\ 1\ 0\ 1\ 0\ 1\ 0\ 0\ 1\ 0\ 1\ 1)$ , which was obtained previously. Finally the encoding operation for this code is shown in Table 4.6.

4.5 Syndrome and Decoding

An important step in decoding most cyclic codes is computing the syndrome of the received sequence. We recall that when decoding a linear code, the syndrome is the  $(n - k)$ -bit vector that is produced by multiplying the received sequence  $\mathbf{r}$  by the parity-check matrix, that is,  $\mathbf{s} = \mathbf{r} \cdot \mathbf{H}^T$ . For cyclic codes in systematic form, the syndrome polynomial  $s(x)$  of degree  $n - k - 1$  or less is the remainder that results from dividing the received polynomial  $r(x)$  by the generator polynomial  $g(x)$  such that

$$r(x) = q(x)g(x) + s(x)$$

or

$$s(x) = r(x) \bmod g(x)$$
$$= s_0 + s_1x + \cdots + s_{n-k-1}x^{n-k-1} \tag{4.32}$$

Thus the  $n - k$  coefficients of  $s(x)$  will form the syndrome  $\mathbf{s}$ . It is also clear that the remainder  $s(x) = 0$  only if  $r(x) = c(x)$ , which is a multiple of  $g(x)$ . If  $s(x) \neq 0$ , then  $r(x)$  is not a code polynomial  $c(x)$  and contains the transmission error  $e(x)$  which is due to the channel noise. Let

$e(x) = e_0 + e_1x + \cdots + e_{n-1}x^{n-1}$  be the error polynomial. Then the received polynomial is

$$r(x) = c(x) + e(x) \quad (4.33)$$

When coupled with Eq. (4.32),  $e(x)$  in Eq. (4.33) becomes  $e(x) = c(x) + q(x)g(x) + s(x)$ . Since  $c(x) = d(x)g(x)$  is a code polynomial which is a multiple of  $g(x)$ ,  $e(x)$  becomes

$$e(x) = [d(x) + q(x)]g(x) + s(x) \quad (4.34)$$

Hence we see that the syndrome  $s(x)$  is actually the remainder that results from dividing  $e(x)$  by  $g(x)$ . In fact,  $s(x) = 0$  if and only if  $e(x) = 0$ . The function of the decoder is to estimate the error polynomial  $e(x)$  from the syndrome  $s(x)$ . Therefore the detector has to estimate  $e(x)$  based on the syndrome  $s(x)$ . The syndrome computation can be accomplished by using the  $(n - k)$ -stage encoding circuit shown in Fig. 4.5, except that  $r(x)$  is shifted into the syndrome generator either from the left end or from the right end. When all of  $r(x)$  has been shifted into the syndrome register, the contents in the register will form the syndrome  $s(x)$ . Because of their cyclic structure, cyclic codes are extremely suitable for error detection and correction. The error pattern detector is just the syndrome generator with a single OR gate with the syndrome bits as inputs. If the syndrome is not 0, the output of the OR gate is 1 and errors have been detected. Hence the received bit  $r_{n-1}$  in the rightmost stage of the buffer register is erroneous and must be corrected. If the output of the error detector is 0, then the rightmost bit in the buffer is assumed to be correct and no correction is necessary.

Consider the case where the received sequence  $r(x)$  is shifted into the syndrome generator from the left end. Let  $r^{(1)}(x)$  be a cyclic shift of  $r(x)$ . From Eq. (4.3), it follows that

$$r^{(1)}(x) = r_{n-1}(x^n + 1) + xr(x) \quad (4.35)$$

Using Eqs. (4.6) and (4.32), Eq. (4.35) can be written as

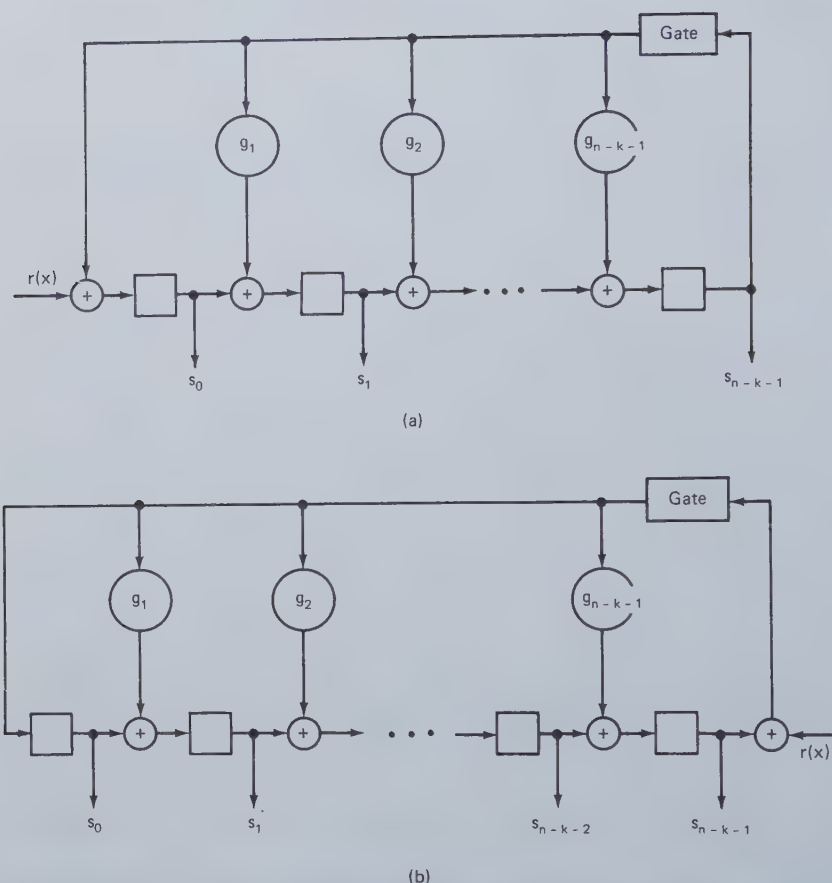
$$r^{(1)}(x) = r_{n-1}h(x)g(x) + x[q(x)g(x) + s(x)]$$

Let  $\gamma(x)$  be the remainder resulting from dividing  $r^{(1)}(x)$  by  $g(x)$ . Then it follows that

$$q_1(x)g(x) + \gamma(x) = [r_{n-1}h(x) + xq(x)]g(x) + xs(x)$$

$$\text{or} \quad xs(x) = [q_1(x) + r_{n-1}h(x) + xq(x)]g(x) + \gamma(x) \quad (4.36)$$

Equation (4.36) tells us that  $\gamma(x)$  is also the remainder resulting from dividing  $xs(x)$  by  $g(x)$ . Hence it holds that



**Figure 4.5** An  $(n - k)$ -stage syndrome generator with input from (a) the left end and (b) the right end.

$$\gamma(x) = s^{(1)}(x) \quad (4.37)$$

which is the syndrome of  $xr(x)$ . If the degree of  $s(x)$  is less than  $n - k - 1$ , then the degree of  $xs(x)$  is less than that of  $g(x)$ . Consequently, Eq. (4.36) becomes

$$xs(x) = \gamma(x) \quad (4.38)$$

This result coupled with Eq. (4.37) yields

$$xs(x) = s^{(1)}(x)$$

which is just  $s(x)$  shifted one place to the right. On the other hand, if the degree of  $s(x)$  is equal to  $n - k - 1$ , then the degree of  $xs(x) + \gamma(x)$  equals that of  $g(x)$ . Thus  $q_1(x) + r_{n-1}h(x) + xq(x)$  must be of degree 0.

Therefore Eq. (4.36) becomes  $xs(x) = g(x) + \gamma(x)$ . Since  $g(x)$  is monic, the coefficient of  $x^{n-k-1}$  in  $s(x)$  must be 1. Thus, shifting  $s(x)$  once in the syndrome register causes  $g(x)$  to be subtracted from the shifted syndrome which yields  $\gamma(x)$ .

Next, let us consider the case where the received sequence  $r(x)$  is shifted into the syndrome generator from the right end, as shown in Fig. 4.5b. In this case, the contents in the syndrome generator form the syndrome  $s^{(n-k)}(x)$  of  $r^{(n-k)}(x) = x^{n-k}r(x)$ . Dividing  $x^{n-k}r(x)$  by  $g(x)$  yields

$$x^{n-k}r(x) = q(x)g(x) + \gamma(x) \quad (4.39)$$

From Eq. (4.5), we get

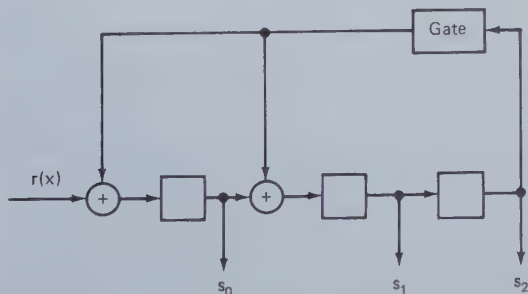
$$x^{n-k}r(x) = a(x)(x^n + 1) + r^{(n-k)}(x) \quad (4.40)$$

Combining Eqs. (4.39) and (4.40) and using Eq. (4.6) yields

$$r^{(n-k)}(x) = [q(x) + a(x)h(x)]g(x) + \gamma(x)$$

This means that  $\gamma(x)$  is also the remainder when  $r^{(n-k)}(x)$  is divided by  $g(x)$ . Therefore  $\gamma(x)$  is indeed the syndrome of  $r^{(n-k)}(x)$ .

**Example 4.11** Consider a syndrome circuit for the (7, 4) cyclic code generated by  $g(x) = 1 + x + x^3$ . This code has  $d_{\min} = 3$  and is capable of correcting any single error in  $e(x)$ . There are seven correctable single-error patterns. Suppose that the code word  $\mathbf{c} = (1\ 1\ 0\ 1\ 0\ 0\ 0)$  or  $c(x) = 1 + x + x^3$  is transmitted and  $\mathbf{r} = (1\ 0\ 0\ 1\ 0\ 0\ 0)$  or  $r(x) = 1 + x^3$  is received. This means that a single error has occurred at  $e(x) = x$ . Dividing this error pattern by  $g(x)$ , we obtain its corresponding syndrome  $\mathbf{s} = (0\ 1\ 0)$ . Word  $\mathbf{r}$  is shifted into the syndrome register shown in Fig. 4.6; the contents in that register are given in Table 4.7. At the end of the seventh shift, the register contains the syndrome  $\mathbf{s} = (0\ 1\ 0)$ . If the register is shifted once more with the input gate disconnected, the new contents will be  $\mathbf{s}^{(1)} = (0\ 0\ 1)$ , which is the syndrome of  $\mathbf{r}^{(1)} = (0\ 1\ 0\ 0\ 1\ 0\ 0)$ , a cyclic shift of the received word  $\mathbf{r}$ .



**Figure 4.6** Syndrome generator for the (7, 4) cyclic code with  $g(x) = 1 + x + x^3$ .



TABLE 4.7 Contents of the Syndrome Register Shown in Fig. 4.6

| Shift no. | Input | Register contents |   |   |
|-----------|-------|-------------------|---|---|
|           |       | $s_0, s_1, s_2$   |   |   |
| 0         | —     | 0                 | 0 | 0 |
| 1         | 0     | 0                 | 0 | 0 |
| 2         | 0     | 0                 | 0 | 0 |
| 3         | 0     | 0                 | 0 | 0 |
| 4         | 1     | 1                 | 0 | 0 |
| 5         | 0     | 0                 | 1 | 0 |
| 6         | 0     | 0                 | 0 | 1 |
| 7         | 1     | 0                 | 1 | 0 |
| 8         | 0     | 0                 | 0 | 1 |
| 9         | 0     | 1                 | 1 | 0 |
| 10        | 0     | 0                 | 1 | 1 |
| 11        | 0     | 1                 | 1 | 1 |
| 12        | 0     | 1                 | 0 | 1 |

Next let us consider the decoding of the (7, 4) cyclic code with  $g(x) = 1 + x + x^3$ . When  $r(x)$  has been simultaneously shifted into the syndrome register and the buffer register, the syndrome register contains (0 1 0). In general, when a single error occurs at  $e(x) = x^i$ , another  $6 - i$  shifts are required for the register contents to be (1 0 1). Since in this example  $e(x) = x$ , five more shifts are needed to obtain  $\mathbf{s} = (1\ 0\ 1)$  after all the received bits have been shifted into the syndrome generator. Thus the next received bit to come out of the buffer register is an erroneous bit. Therefore, only  $\mathbf{s} = (1\ 0\ 1)$  needs to be detected. The syndrome  $\mathbf{s} = (1\ 0\ 1)$  is read into the detector of a single AND gate. Since 1 appears at the output of the detector, the erroneous bit  $r_1$  in the rightmost stage of the buffer register must be corrected. The complete decoder for the (7, 4) cyclic code and an illustration of the decoding process is shown in Fig. 4.7.

In this section we have only presented the introductory concepts of decoding cyclic codes. In Chaps. 5 through 8, the decoding algorithms and techniques will be covered more extensively. The Meggit decoder and error-trapping methods suitable for decoding short random-error-correcting and burst-error-correcting cyclic codes are presented in Chaps. 5 and 6. The best-known algorithms developed for decoding long random-error-correcting BCH codes are presented in Chap. 7 and the decoding techniques for the majority-logic decodable codes are treated in Chap. 8.

#### 4.6 Modifications of Cyclic Codes

When devising codes for communications systems, it is often necessary to find values of  $n$  and  $k$  that accommodate data formats other

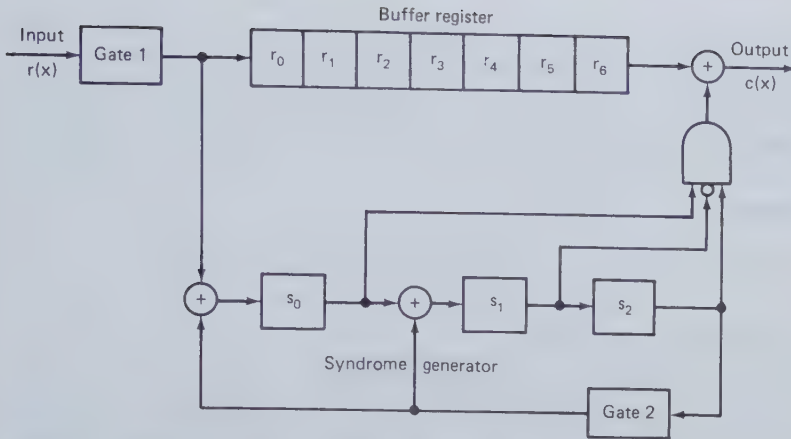


Figure 4.7 Decoder for the (7, 4) cyclic code generated by  $g(x) = 1 + x + x^3$ .

than those preferred by the coding designer. In such cases, the desired code is usually modified to fit the required format. This can be done by adding a column or a row of the generator matrix  $\mathbf{G}$ ; deleting a column or row; adding both a column and a row; or deleting both a column and a row. There are six such code modifications.

1. Any  $(n, k)$  cyclic code can be *extended* to an  $(n + 1, k)$  cyclic code by adding parity-check bits. For example, the (7, 4) Hamming code becomes an (8, 4) extended code whose minimum distance is 4. The (127, 112) BCH code recommended in the INTELSAT V era can also be extended to a (128, 112) code. If the original code has an odd minimum distance, the extended code will have an even minimum distance.
2. A code can be *punctured* by deleting parity-check bits. This process is just the inverse of extending a code. Thus, the (8, 4) extended code can be punctured to produce the (7, 4) Hamming code.
3. A code can be *expurgated* by decreasing  $k$  without changing  $n$ . Therefore, an expurgated code is obtained by discarding some code words. For example, the (7, 4) Hamming code with  $g(x) = 1 + x + x^3$  can be expurgated to give a new (7, 3) code with  $g(x) = (1 + x)(1 + x + x^3)$ .
4. A code can be *augmented* by increasing  $k$  without changing  $n$ . Thus an augmented code can be obtained by adding new code words to the code. This process is the inverse of expurgating a code. Given a code with  $g(x)$ , an augmented code is generated by  $g(x)/(1 + x)$  only when  $1 + x$  is a factor of  $g(x)$ .
5. A code can be *lengthened* by adding more information bits. The code is first augmented into the code generated by  $g(x)/(1 + x)$ . This

augmented code is then extended by adding an overall parity check.

6. A code can be *shortened* by dropping some information bits from each code word to obtain a shortened cyclic code. This corresponds to decreasing both dimensions of the generator matrix  $\mathbf{G}$  by the same amount.

The following is a discussion of shortened cyclic codes.

#### 4.6.1 Shortened cyclic codes

An  $(n - i, k - i)$  shortened code can be formed by dropping  $i$  information bits from each code word of an  $(n, k)$  cyclic code. There are  $2^{k-i}$  such code words of length  $n - i$ . To shorten an  $(n, k)$  cyclic code by  $i$  bits is the same as omitting the last  $i$  rows and columns of the generator matrix  $\mathbf{G}$ , or the last  $i$  columns of the parity-check matrix  $\mathbf{H}$ . However, a shortened code is no longer cyclic because the remainder of  $xc(x)$  divided by  $x^{n-i} + 1$  is not generally a code word when  $c(x)$  is a code word. However, it is possible to make such shortened codes cyclic if they are ideals in the ring of polynomials modulo  $f(x)$  for some polynomial  $f(x)$  of degree  $n - i$ . Such a code is called a pseudocyclic code.

If the original cyclic code has the minimum distance  $d_{\min}$ , then the shortened cyclic code also has at least the minimum distance  $d_{\min}$ . Since the number of cosets  $2^{n-k}$  remains unchanged by shortening, the shortened code can also correct any error pattern correctable by the original code even though the code is shortened. A cyclic code shortened by  $i$  bits can be decoded by the same circuitry used for decoding the original code provided only that each shortened received sequence is always prefixed with  $i$  zeros. However, decoding can be accomplished more quickly only when the syndrome of  $x^{n-k+i}r(x)$  is calculated. A decoder for an  $(n - i, k - i)$  shortened code will be discussed in Chap. 5.

**Example 4.12** Consider the  $(7, 4)$  cyclic code with  $g(x) = 1 + x + x^3$ , which was discussed in Example 4.3. By shortening this code by one bit, we obtain the resultant  $(6, 3)$  shortened code. The generator matrix  $\mathbf{G}'$  and the parity-check matrix  $\mathbf{H}'$  for the  $(6, 3)$  shortened code are obtained, respectively, by omitting the last row and column from  $\mathbf{G}$  of the original code and the last column of  $\mathbf{H}$ :

$$\mathbf{G}' = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 0 & 1 \end{bmatrix} \quad \mathbf{H}' = \begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 \end{bmatrix}$$

The third column in Table 4.3 represents the systematic code words  $c$ . The code words  $c'$  for the  $(6, 3)$  shortened code can be found directly from the code words  $c$  in the third column of Table 4.3 by omitting the last column of the code words (or the first bit of each code word) and then taking every other code word from the top row. Thus the code words of the  $(6, 3)$  shortened cyclic code are formed as follows:

$$c' = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 1 & 1 \end{bmatrix} \quad (4.41)$$

Observing  $c$  and  $c'$ , we see that a shortened code  $C'$  has at least the same minimum distance  $d_{\min} = 3$  as that of the original code  $C$ .

**Theorem 4.2** Every shortened cyclic code with  $d_{\min} > 2$  is a pseudocyclic code.

**Proof** Consider the shortened cyclic code of length  $n - i$  obtained from an  $(n, k)$  cyclic code generated by  $g(x)$ . Since  $g(x)$  is not a divisor of  $x^{n-i} + 1$  for the shortened code, it can be written as

$$x^{n-i} = q(x)g(x) + \gamma(x) \quad (4.42)$$

where the remainder  $\gamma(x)$  should not be 1 and its degree is less than that of  $g(x)$ . When the polynomial  $f(x) = x^{n-i} + \gamma(x)$  is of degree  $n - i$ , Eq. (4.42) becomes  $f(x) = q(x)g(x)$ , which is a multiple of  $g(x)$ . In fact, a unique monic polynomial  $q(x)$  is a factor of  $f(x)$ . Now, consider a code polynomial  $c(x)$  which is a multiple of  $g(x)$ . Let  $a(x)$  be any polynomial. By the euclidean division algorithm it follows that

$$a(x)c(x) = q'(x)f(x) + \gamma'(x) \quad (4.43)$$

Since  $g(x)$  divides both  $c(x)$  and  $f(x)$ ,  $\gamma'(x)$  is clearly a multiple of  $g(x)$ . Thus  $g(x)$  generates a pseudocyclic code that is an ideal in the algebra of polynomials modulo- $f(x)$ . As a result, whereas cyclic codes are ideals in the ring of polynomials modulo  $(x^n + 1)$ , shortened cyclic codes are ideals in the ring of polynomials modulo- $f(x)$  of degree  $n - i$ .

**Example 4.13** Consider again the  $(7, 4)$  cyclic code generated by  $g(x) = 1 + x + x^3$ . If this original code is shortened by 1 bit, the code words of the  $(6, 3)$  shortened cyclic code are given by Eq. (4.41). Let us arbitrarily pick a code word  $c_1 = (0 \ 1 \ 1 \ 0 \ 1 \ 0)$  which is found in the third row from the topmost all-zero word in  $c'$ . The corresponding code polynomial is  $c_1(x) = x + x^2 + x^4 = x(1 + x + x^3)$ , which is certainly a code word because it is divisible by  $g(x) = 1 + x + x^3$ . Cyclically shifting  $c_1(x)$  once gives

$$\begin{aligned}
c_2(x) &= xc_1(x) \\
&= x^2 + x^3 + x^5 \text{ i } c_2 = (0 \ 0 \ 1 \ 1 \ 0 \ 1) \\
&= x^2(1 + x + x^3)
\end{aligned}$$

which is also the code word found in the sixth row in  $\mathbf{c}'$ . Cyclically shifting  $c_2(x)$  once more produces

$$\begin{aligned}
c_3(x) &= xc_2(x) \\
&= 1 + x^3 + x^4 \bmod (x^6 + 1) \leftrightarrow c_3 = (1 \ 0 \ 0 \ 1 \ 1 \ 0) \\
&= (1 + x)(1 + x + x^3) + x^2
\end{aligned}$$

As you can see,  $g(x)$  is no longer a divisor of  $c_3(x)$ , because  $c_3(x)/g(x)$  has the remainder  $\gamma(x) = x^2$ . Hence,  $c_3(x)$  is not a code polynomial. Therefore, to make shorten code cyclic, we must seek some other polynomial that will divide  $f(x) = x^{n-i} + \gamma(x) = 1 + x^2 + x^6$ . Since  $c_3(x) = x^3 + x^4 + x^6 = 1 + x^2 + x^3 + x^4 \bmod f(x)$ ,  $\mathbf{c}_3 = (1 \ 0 \ 1 \ 1 \ 1 \ 0)$  is a code word. In a similar fashion, the rest of the shortened code words can be obtained as follows:

$$\begin{aligned}
c_4(x) &= x + x^3 + x^4 + x^5 \quad \leftrightarrow \mathbf{c}_4 = (0 \ 1 \ 0 \ 1 \ 1 \ 1) \\
c_5(x) &= 1 + x^4 + x^5 \quad \bmod f(x) \leftrightarrow \mathbf{c}_5 = (1 \ 0 \ 0 \ 0 \ 1 \ 1) \\
c_6(x) &= 1 + x + x^2 + x^5 \bmod f(x) \leftrightarrow \mathbf{c}_6 = (1 \ 1 \ 1 \ 0 \ 0 \ 1) \\
c_7(x) &= 1 + x + x^3 \quad \bmod f(x) \leftrightarrow \mathbf{c}_7 = (1 \ 1 \ 0 \ 1 \ 0 \ 0)
\end{aligned}$$

Now the reader can be convinced that all the code words of a shortened cyclic code can be constructed on the basis of some polynomial that is modulo  $f(x) = x^{n-i} + \gamma(x)$ .

## PROBLEMS

**4.1** Consider the  $(7, 3)$  cyclic code generated by  $g(x) = 1 + x + x^2 + x^4$ . Let  $\mathbf{c} = (0 \ 0 \ 1 \ 1 \ 1 \ 0 \ 1)$  be a code word of the  $(7, 3)$  code. Find all the code words by using the following shift equation:

$$[\mathbf{c}^{(i)}]^T = \mathbf{S}[\mathbf{c}^{(i-1)}]^T \quad i = 1, 2, \dots, n-1$$

where the shift matrix is

$$\mathbf{S} = \begin{bmatrix} 0 & 0 & 0 & \cdots & 1 \\ & & & & 0 \\ & \mathbf{I}_{(n-1) \times (n-1)} & & & 0 \\ & & & & \vdots \\ & & & & 0 \end{bmatrix}$$

**4.2** The generator polynomial  $g(x)$  for the  $(15, 4)$  cyclic code is of degree 11.

a Find all the generator polynomials of this code.

b What is the dual code of this  $(15, 4)$  code?



- c Is this dual code a cyclic Hamming code? If so, determine the generator polynomial of its dual code.

**4.3** Let  $g(x) = 1 + x + x^5 + x^6 + x^{10} + x^{11}$  be the generator polynomial of the (15, 4) cyclic code.

- Determine the parity polynomial  $h(x)$  of this code.
- Let  $G_1$  and  $G_2$  be the generator matrices in nonsystematic and systematic forms, respectively. Find  $G_1$  and  $G_2$  for this code.
- Derive the systematic form  $G_3$  from the nonsystematic form  $G_1$  obtained in part b above and compare  $G_3$  with  $G_2$ .
- Determine the parity-check matrix  $H$  of this code.

**4.4** Consider the (15, 4) dual code of the (15, 11) cyclic Hamming code, generated by  $g(x) = 1 + x + x^5 + x^6 + x^{10} + x^{11}$ .

- Find all the code words in systematic form for this (15, 4) code.
- Find all the code words in nonsystematic form.

**4.5** Show that all the code words generated by  $g(x) = 1 + x + x^5 + x^6 + x^{10} + x^{11}$  of the (15, 4) cyclic code are orthogonal to rows of the parity-check matrix  $H$ .

**4.6** Prove that the generator polynomial  $g(x)$  of an  $(n, k)$  cyclic code is a factor of  $x^n + 1$ .

**4.7** The (15, 11) dual code to the (15, 4) cyclic code is generated by the polynomial  $h^*(x) = x^m h(x^{-1})$ , where  $h(x) = (x^n + 1)/g(x)$ .

- Find  $h^*(x)$ .
- Find the generator matrix  $G$  in systematic form for this code.

**4.8** Consider the circuit which multiplies an input polynomial  $a(x) = 1 + x + x^3$  by the fixed polynomial  $b(x) = 1 + x + x^4$  over  $GF(2)$ .

- Devise this multiplication circuit by two different methods.
- Display the circuit operation by shifting each input digit into the circuit.

**4.9** Consider the circuit which divides an input polynomial  $a(x) = 1 + x^2 + x^4$  by the fixed polynomial  $b(x) = 1 + x + x^2$ .

- Devise this division circuit.
- Determine the output polynomial by direct division of  $a(x)/b(x)$ .
- Display the circuit operation in order to find the output polynomial, and compare the result with the polynomial obtained in part (b) above.

**4.10** A single shift-register circuit that multiplies by  $b(x)$  and then divides by  $c(x)$  can be constructed by combining the multiplication circuit and the division circuit.

- Devise the circuit for multiplying an input polynomial  $a(x) = 1 + x^2 + x^4$  by  $b(x) = 1 + x^2$  and dividing by  $c(x) = 1 + x + x^2$ .

- b* Display the contents of the shift register after each input digit is shifted into the circuit.

**4.11** The generator polynomial of the (15, 11) cyclic Hamming code is  $g(x) = 1 + x + x^4$ .

- a* Find the parity polynomial  $h(x)$  of this code.  
*b* The encoding of a cyclic code is accomplished by using its parity polynomial  $h(x)$ . Devise the encoder for this code.  
*c* Determine the parity-check bits, and then obtain the code word corresponding to the information sequence  $\mathbf{d} = (1\ 0\ 0\ 0\ 0\ 0\ 1\ 0\ 1\ 1)$ .

**4.12** Consider the (15, 6) double-error-correcting code generated by  $g(x) = 1 + x + x^4 + x^5 + x^6 + x^9$ .

- a* Devise the encoder for this code by using  $g(x)$  and  $h(x)$ , respectively.  
*b* Determine the parity-check bits, and then obtain the code word in systematic form corresponding to  $\mathbf{d} = (1\ 0\ 0\ 1\ 0\ 1)$ .  
*c* Determine the minimum distance of this code.

**4.13** By shortening the (15, 7) cyclic code by 3 bits, we obtain the (12, 4) shortened code. The generator polynomial of the (15, 7) code is  $g(x) = 1 + x^4 + x^6 + x^7 + x^8$ .

- a* Determine the generator matrix  $\mathbf{G}'$  and the parity-check matrix  $\mathbf{H}'$ , respectively, for this (12, 4) shortened code.  
*b* Using the generator matrix  $\mathbf{G}'$ , find all the code words of the (12, 4) shortened code.  
*c* What is the minimum distance of this shortened code?  
*d* Find the polynomial  $f(x) = x^{n-i} + \gamma(x)$  of degree  $n - i$ .  
*e* Construct the pseudocyclic code words in the polynomial modulo  $f(x)$ .

**4.14** Consider the (15, 7) cyclic code with  $g(x) = 1 + x + x^2 + x^4 + x^8$ .

- a* Devise a syndrome circuit for this code.  
*b* Let  $\mathbf{r} = (1\ 1\ 1\ 0\ 1\ 0\ 1\ 0\ 0\ 1\ 0\ 0\ 0)$  be a received word. Compute the syndrome of  $\mathbf{r}(x)$ .  
*c* Display the contents of the syndrome generator after each digit of  $\mathbf{r}$  has been shifted into the syndrome computation circuit.

## Short-Random-Error Correction by Error-Trapping Techniques

The Meggitt decoder and error-trapping decoding, which are used for the correction of three or fewer errors ( $t \leq 3$ ), are presented in this chapter. These techniques are suitable for decoding short, random-error-correcting cyclic codes. A decoder based on this type of decoding technique employs a simple error-pattern detector for error correction, which is done simply by adding the error pattern to the received word. For long cyclic codes where more extensive error-correction capability is needed ( $t > 3$ ), the Meggitt decoder becomes very ineffective, and the BCH and majority-logic-decodable codes are better suited.

### 5.1 The Meggitt Decoder

In principle, Meggitt decoders, introduced in 1961, are used to decode any cyclic code or shortened cyclic code. However, as the number of errors to be corrected increases, the complexity of Meggitt decoders grows proportionally. Therefore, in practice they are not used for correcting more than three random errors.

The basic function of a decoder is to associate the syndrome  $s(x)$  of the received polynomial  $r(x)$  to an error pattern  $e(x)$ . That is, the syndrome is used to identify correctable error patterns. The syndrome can be calculated by dividing the received polynomial  $r(x)$  by the generator polynomial  $g(x)$ . After  $r(x)$  has been shifted into the syndrome register, the register contains the syndrome. If the error pattern is shifted cyclically one place to the right, then the new syndrome is obtained by advancing the feedback shift register containing  $s(x)$  one place to the

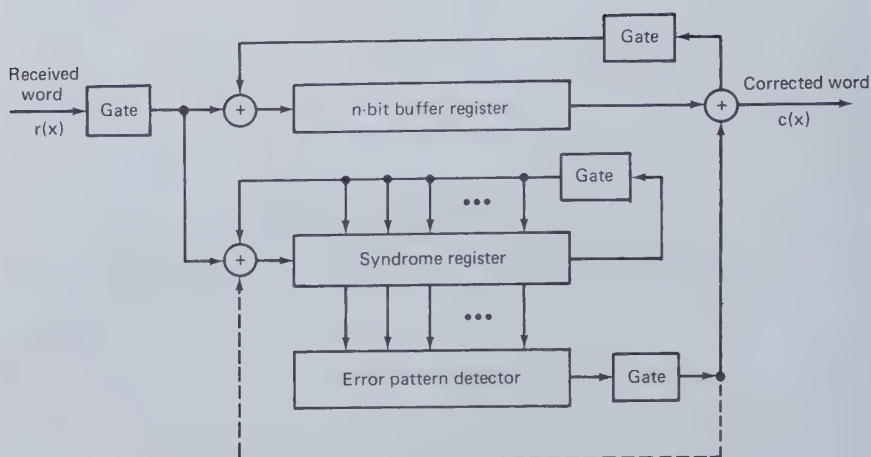


Figure 5.1 A Meggitt decoder for cyclic codes.

right. This implies a need to test a specific error pattern after each shift.

A general Meggitt decoder for an  $(n, k)$  cyclic code is shown in Fig. 5.1. It consists of a syndrome generator, an error-pattern detector, and a buffer register to store the received word. The syndrome is formed by shifting  $r(x)$  into the syndrome generator, and the buffer register stores  $r(x)$ , which comes from the left end. The syndrome is fed into the error-pattern detector whose output is 1 if and only if the syndrome corresponds to a correctable error pattern with an error  $r_{n-1}$  at the rightmost stage of the buffer register. If the detector output is 0, then no correction is necessary. The Meggitt decoder is sometimes modified in a slightly different manner. In Fig. 5.1, the connection between the detector output and the syndrome register (indicated by the dotted line) allows the error effect to be removed from the syndrome and results in a new syndrome being formed by the syndrome update. The considerations involved in building Meggitt decoders are illustrated in the following section by the examples of cyclic Hamming codes.

## 5.2 Cyclic Hamming Codes

A Hamming code was first constructed as a linear block code by Richard W. Hamming in his paper, which appeared in the *Bell System Technical Journal* in 1950. Since then, it has been discovered that Hamming codes can be arranged in cyclic form, and they have been

widely used for error-control purposes in computer storage systems for improving the reliability of core memories. Since the number of parity-check bits  $m$  is equal to the degree of the generator polynomial, the number of information bits  $k$  of a cyclic Hamming code of length  $n = 2^m - 1$  is  $k = n - m = 2^m - m - 1$ . Therefore, in general, a cyclic Hamming code is called a  $(2^m - 1, 2^m - m - 1)$  linear code.

It is called a primitive polynomial  $p(x)$  of degree  $m$  if and only if  $p(x)$  divides  $x^n + 1$  for no  $n$  less than  $2^m - 1$ . Thus a cyclic Hamming code is a code whose generator polynomial is a primitive polynomial  $p(x)$  of degree  $m$ . Fields of  $2^m$  symbols are called Galois fields  $GF(2^m)$  and are important in the study of cyclic codes. Let  $\alpha$  be a root of  $p(x)$ . Then  $\alpha$  is a primitive element of  $GF(2^m)$ . Thus the minimal polynomial of  $\alpha$  is also primitive. If we choose a primitive polynomial  $p(x)$  properly, the powers of  $\alpha$  up to  $2^m - 2$  will be different, and  $\{0, 1, \alpha, \dots, \alpha^{2^m-2}\}$  will be the set of  $2^m$  field elements. Each element can be expressed as a sum of some elements from the set  $\{1, \alpha, \alpha^2, \dots, \alpha^{m-1}\}$ . Thus a table of powers of  $\alpha$  is developed for the general case.

Consider the null space of

$$\mathbf{H} = [1, \alpha, \alpha^2, \dots, \alpha^{2^m-2}] \quad (5.1)$$

where  $\alpha, \alpha^2, \dots, \alpha^{2^m-2}, \alpha^{2^m-1} = 1$  are all the elements of  $GF(2^m)$ . If the powers of  $\alpha$  are arranged by column vectors of 1s and 0s, then every nonzero column of length  $m$  appears as a column of  $\mathbf{H}$ . Therefore, the null space of  $\mathbf{H}$  is a Hamming single-error-correcting code. In fact, each  $m$ -tuple column vector of  $\mathbf{H}$  also represents the syndrome vector corresponding to a single-error pattern.

**Example 5.1** A primitive polynomial  $p(x)$  of degree  $m$  over  $GF(2)$  is given in Table 5.1.

**TABLE 5.1** Primitive Polynomials of Degree  $m$ ,  $1 \leq m \leq 8$ , over  $GF(2)$

| $m$ | $p(x)$                      |
|-----|-----------------------------|
| 1   | $1 + x$                     |
| 2   | $1 + x + x^2$               |
| 3   | $1 + x + x^3$               |
| 4   | $1 + x + x^4$               |
| 5   | $1 + x^2 + x^5$             |
| 6   | $1 + x + x^6$               |
| 7   | $1 + x^3 + x^7$             |
| 8   | $1 + x^2 + x^3 + x^4 + x^8$ |

Let  $\alpha$  be a root of a primitive polynomial  $p(x)$  of degree  $m = 3$ . The elements of  $GF(2^3)$  generated by  $p(x) = 1 + x + x^3$  are shown in Table 5.2.



**TABLE 5.2** The Elements of  $GF(2^3)$  Generated by  $p(\alpha) = 1 + \alpha + \alpha^3 = 0$

| Power      | Polynomial              | Vector |
|------------|-------------------------|--------|
| 0          | 0                       | 0 0 0  |
| 1          | 1                       | 1 0 0  |
| $\alpha$   | $\alpha$                | 0 1 0  |
| $\alpha^2$ | $\alpha^2$              | 0 0 1  |
| $\alpha^3$ | $1 + \alpha$            | 1 1 0  |
| $\alpha^4$ | $\alpha + \alpha^2$     | 0 1 1  |
| $\alpha^5$ | $1 + \alpha + \alpha^2$ | 1 1 1  |
| $\alpha^6$ | $1 + \alpha^2$          | 1 0 1  |

**Example 5.2** Consider the (7, 4) cyclic code generated by  $g(x) = 1 + x + x^3$ . The parity-check matrix  $\mathbf{H}$  can be immediately written by using Table 5.2.

$$\mathbf{H} = [\alpha^0, \alpha^1, \alpha^2, \alpha^3, \alpha^4, \alpha^5, \alpha^6]$$

$$= \begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \end{bmatrix}$$

Let the received sequence be  $\mathbf{r} = (1 \ 0 \ 1 \ 1 \ 1 \ 0 \ 1)$ . Then the syndrome is computed as  $\mathbf{s} = (1 \ 0 \ 1)$  from  $\mathbf{s} = \mathbf{r} \cdot \mathbf{H}^T$ , whereas dividing the received polynomial  $r(x) = 1 + x^2 + x^3 + x^4 + x^6$  by the generator polynomial  $g(x) = 1 + x + x^3$  yields the syndrome (the remainder)  $s(x) = 1 + x^2$  or  $\mathbf{s} = (1 \ 0 \ 1)$ .

Consider a cyclic Hamming code of length  $2^m - 1$  generated by a primitive polynomial  $p(x)$  of degree  $m$ . The parity-check matrix  $\mathbf{H}$  consists of  $2^m - 1$  columns, and each column represents a nonzero  $m$ -tuple vector. Therefore,  $\mathbf{H}$  is an  $m \times (2^m - 1)$  matrix. If  $x^i$  for  $i = 0, 1, \dots, 2^m - 2$  is divided by the generator polynomial  $p(x)$ , the remainder  $\gamma_i(x)$  of degree  $m - 1$  or less results:

$$\gamma_i(x) = \gamma_{i,0} + \gamma_{i,1}x + \gamma_{i,2}x^2 + \dots + \gamma_{i,m-1}x^{m-1} \quad (5.2)$$

The coefficients of  $\gamma_i(x)$  form an  $m$ -tuple vector  $(\gamma_{i,0}, \gamma_{i,1}, \dots, \gamma_{i,m-1})$  which actually becomes a column of  $\mathbf{H}$ . Thus, for  $i = 0, 1, \dots, 2^m - 2$ , we obtain  $2^m - 1$  of these nonzero  $m$ -tuple vectors. These  $2^m - 1$   $m$ -tuple vectors compose columns of the parity-check matrix. Therefore the parity-check matrix  $\mathbf{H}$  of a cyclic Hamming code has the following form:

$$\mathbf{H} = [\mathbf{I}_m : \mathbf{P}_{m \times k}^T]$$

$$= (1, x, x^2, \dots, x^{m-1}, x^m, x^{m+1}, \dots, x^{2^m-3}, x^{2^m-2})$$

$$= \begin{bmatrix} 1 & 0 & 0 & \cdots & 0 & 0 & p_{m,0} & p_{m+1,0} & \cdots & p_{2^m-2,0} \\ 0 & 1 & 0 & \cdots & 0 & 0 & p_{m,1} & p_{m+1,1} & \cdots & p_{2^m-2,1} \\ 0 & 0 & 1 & \cdots & 0 & 0 & p_{m,2} & p_{m+1,2} & \cdots & p_{2^m-2,2} \\ \vdots & & & & & & & & & \\ 0 & 0 & 0 & \cdots & 0 & 1 & p_{m,m-1} & p_{m+1,m-1} & \cdots & p_{2^m-2,m-1} \end{bmatrix} \quad (5.3)$$

An  $(n, k)$  cyclic code of length 15 is generated by a factor of

$$x^{15} + 1 = (1 + x)(1 + x + x^2)(1 + x + x^4)(1 + x^3 + x^4) \\ \times (1 + x + x^2 + x^3 + x^4)$$

There are 30 such cyclic codes. Table 5.3 lists the generator polynomials of  $(n, k)$  cyclic codes,  $1 \leq k \leq 14$ , of code length 15. The factor of degree 4 generates a  $(15, 11)$  cyclic code.

TABLE 5.3 Parameters of  $(15, k)$  Cyclic Codes

| Information bits $k$ | Minimum distance $d_{\min}$ | Generator polynomial $g(x)$                                                                             |
|----------------------|-----------------------------|---------------------------------------------------------------------------------------------------------|
| 14                   | 2                           | $1 + x$                                                                                                 |
| 11                   | 3                           | $1 + x + x^4$                                                                                           |
| 10                   | 4                           | $1 + x^2 + x^4 + x^5$                                                                                   |
| 7                    | 5                           | $1 + x^4 + x^6 + x^7 + x^8$                                                                             |
| 6                    | 6                           | $1 + x + x^4 + x^5 + x^6 + x^9$                                                                         |
| 5                    | 7                           | $1 + x + x^2 + x^4 + x^5 + x^8 + x^{10}$                                                                |
| 4                    | 8                           | $1 + x^3 + x^4 + x^6 + x^7 + x^9 + x^{10} + x^{11}$                                                     |
| 2                    | 10                          | $1 + x + x^3 + x^4 + x^6 + x^7 + x^9 + x^{10} + x^{12} + x^{13}$                                        |
| 1                    | 15                          | $1 + x + x^2 + x^3 + x^4 + x^5 + x^6 + x^7 + x^8 + x^9 \\ + x^{10} + x^{11} + x^{12} + x^{13} + x^{14}$ |

**Example 5.3** From Table 5.3, we see that  $p(x) = 1 + x + x^4$  is the generator polynomial of a  $(15, 11)$  cyclic Hamming code. Using either Eq. (5.3) or the syndrome vectors in Table 5.4, the parity-check matrix  $\mathbf{H}$  of this code is obtained as

$$\mathbf{H} = (x^0, x^1, x^2, \dots, x^{13}, x^{14}) \\ = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 1 & 1 \end{bmatrix} \quad (5.4)$$

TABLE 5.4 Syndrome Versus Error Pattern of the (15, 11) Hamming Code Generated by  $g(x) = 1 + x + x^4$

| Error pattern $e_i(x)$ | Syndrome $s_i(x)$   | Syndrome bits |
|------------------------|---------------------|---------------|
| $x^0$                  | 1                   | 1 0 0 0       |
| $x^1$                  | $x$                 | 0 1 0 0       |
| $x^2$                  | $x^2$               | 0 0 1 0       |
| $x^3$                  | $x^3$               | 0 0 0 1       |
| $x^4$                  | $1 + x$             | 1 1 0 0       |
| $x^5$                  | $x + x^2$           | 0 1 1 0       |
| $x^6$                  | $x^2 + x^3$         | 0 0 1 1       |
| $x^7$                  | $1 + x + x^3$       | 1 1 0 1       |
| $x^8$                  | $1 + x^2 + x^3$     | 1 0 1 0       |
| $x^9$                  | $x + x^2 + x^3$     | 0 1 0 1       |
| $x^{10}$               | $1 + x + x^2$       | 1 1 1 0       |
| $x^{11}$               | $x + x^2 + x^3$     | 0 1 1 1       |
| $x^{12}$               | $1 + x + x^2 + x^3$ | 1 1 1 1       |
| $x^{13}$               | $1 + x^2 + x^3$     | 1 0 1 1       |
| $x^{14}$               | $1 + x^3$           | 1 0 0 1       |

The field elements of  $GF(2^4)$  are  $1, \alpha, \dots, \alpha^{14}$  whose 4-tuple vectors are the columns of  $H$ . Therefore, Eq. (5.4) can be checked by making a field table for  $GF(2^4)$  generated by  $p(x) = 1 + x + x^4$ . A table for field elements of  $GF(2^4)$  is left to the reader as an exercise.

The encoding circuit for the (15, 11) Hamming code generated by  $p(x) = 1 + x + x^4$  is shown in Fig. 5.2.

Since  $n = 15$ , there are 15 single-error patterns which are represented in polynomial form as

$$e_i(x) = x^i \quad i = 0, 1, 2, \dots, 14 \tag{5.5}$$

The syndrome  $s_i(x)$ ,  $0 \leq i \leq 14$  is the remainder resulting from dividing  $e_i(x)$  by  $g(x)$  as shown in Table 5.4.

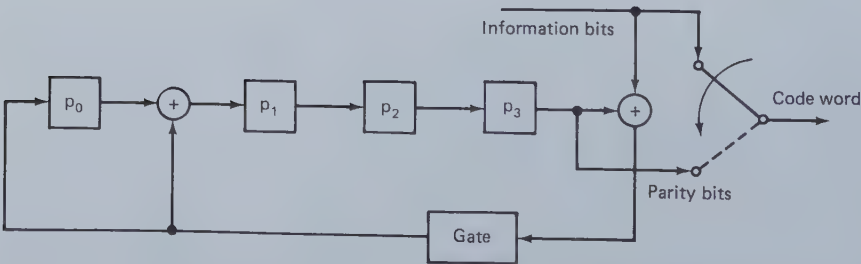


Figure 5.2 Encoder for the (15, 11) cyclic Hamming code generated by  $p(x) = 1 + x + x^4$ .

Notice that there is a one-to-one correspondence between a single-error pattern and its syndrome. If the syndrome is known, the single error can be located. Since the syndrome  $s(x)$  is computed from the received word  $r(x)$ , the function of the decoder is to estimate the error pattern from the syndrome.

**Theorem 5.1** Let the syndrome  $s(x)$  of  $r(x)$  be the remainder that results from dividing  $r(x)$  by  $g(x)$ . If all the single errors lie in the parity-check section of  $r(x)$ , then  $s(x)$  is equal to the error pattern  $e(x)$ .

**Proof** Since  $s(x)$  is the remainder resulting from dividing  $e(x)$  by  $g(x)$ , it follows that

$$e(x) = q(x)g(x) + s(x) \quad (5.6)$$

Because the errors of  $r(x)$  are confined to the parity-check section, the degree of  $e(x)$  is  $n - k - 1$  or less. However, since the degree of  $g(x)$  is  $n - k$ ,  $q(x)$  must be zero. Therefore it follows that

$$e_i(x) = s_i(x) \quad i = 0, 1, \dots, n - k - 1 \quad (5.7)$$

This proves that the error pattern confined to the  $n - k$  parity-check section and the corresponding syndrome of  $r(x)$  take the same values.

Referring to Table 5.4 for the (15, 11) cyclic Hamming code, we see that there are four single-error patterns which are confined to the parity-check section of  $r(x)$ . According to Theorem 5.1 we have

$$e_0(x) = x^0 = s_0(x)$$

$$e_1(x) = x^1 = s_1(x)$$

$$e_2(x) = x^2 = s_2(x)$$

$$e_3(x) = x^3 = s_3(x)$$

Recall that the code words of a systematic cyclic code are expressed as

$$\gamma_i(x) + x^{n-k+i} = q_i(x)g(x) \quad i = 0, 1, 2, \dots, k - 1 \quad (5.8)$$

The generator matrix  $\mathbf{G}$  is then obtained by using Eq. (5.8) to generate its rows. That is,  $\mathbf{G} = [\mathbf{P}_{k \times (n-k)}; \mathbf{I}_k]$ .

**Example 5.4** Consider the (15, 11) cyclic Hamming code generated by  $g(x) = 1 + x + x^4$ . Dividing  $x^{4+i}$  by  $g(x)$  yields the remainder  $\gamma_i(x)$ ,  $i = 0, 1, \dots, 10$ . Using Eq. (5.8), we obtain  $\gamma_i(x) + x^{4+i}$ ,  $i = 0, 1, \dots, 10$ , as shown in Table 5.5.

Thus the generator matrix  $\mathbf{G}$  of this code is

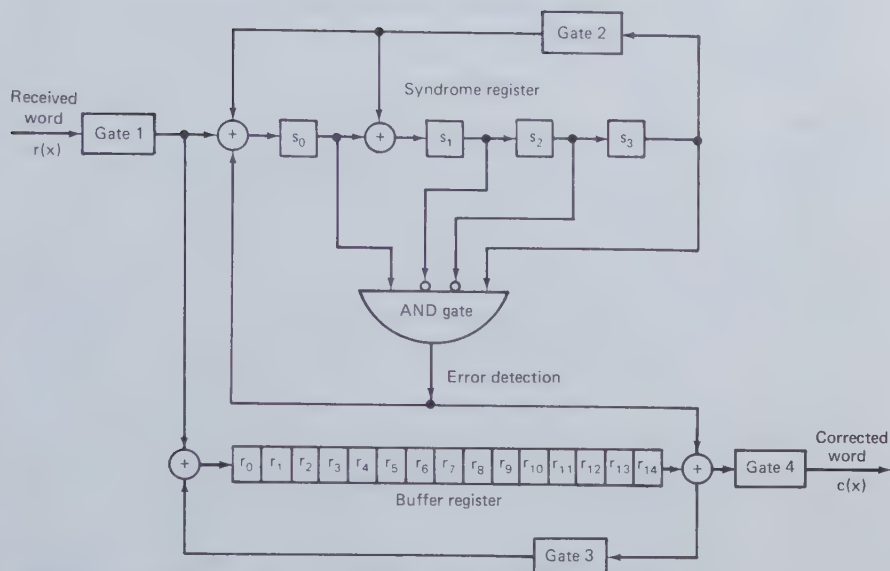
$$\mathbf{G} = \begin{bmatrix}
 1 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 0 & 1 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 0 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 1 & 1 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\
 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\
 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\
 0 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\
 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\
 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\
 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1
 \end{bmatrix} = [\mathbf{P}_{11 \times 4} : \mathbf{I}_{11}] \quad (5.9)$$

TABLE 5.5

| $i$ | $\gamma_i(x) + x^{4+i}$                                |
|-----|--------------------------------------------------------|
| 0   | $1 + x \quad \quad \quad + x^4$                        |
| 1   | $\quad x + x^2 \quad \quad \quad + x^5$                |
| 2   | $\quad \quad x^2 + x^3 \quad \quad \quad + x^6$        |
| 3   | $1 + x \quad \quad + x^3 \quad \quad \quad + x^7$      |
| 4   | $1 \quad \quad + x^2 \quad \quad \quad + x^8$          |
| 5   | $\quad x \quad \quad + x^3 \quad \quad \quad + x^9$    |
| 6   | $1 + x + x^2 \quad \quad \quad + x^{10}$               |
| 7   | $\quad x + x^2 + x^3 \quad \quad \quad + x^{11}$       |
| 8   | $1 + x + x^2 + x^3 \quad \quad \quad + x^{12}$         |
| 9   | $1 \quad \quad + x^2 + x^3 \quad \quad \quad + x^{13}$ |
| 10  | $1 \quad \quad \quad + x^3 \quad \quad \quad + x^{14}$ |

The consideration involved in building a Meggitt decoder is illustrated by a (15, 11) cyclic Hamming code for a single-error correction. Suppose that a single error occurs at  $x^i$ . When the received word  $r(x)$  has completely shifted into the syndrome register, the syndrome in the register corresponds to the error at  $x^i$ . Now, if we shift  $r(x)$  cyclically once in the syndrome and buffer registers at the same time, the error will be at the location  $x^{i+1}$  and the syndrome contents will be the syndrome corresponding to the error at  $x^{i+1}$ . If the buffer and syndrome registers are continuously shifted until the error bit moves to the rightmost stage of the buffer register, the error will be located at  $x^{14}$  and the contents in the syndrome generator will be the syndrome for the error at that location. When the syndrome corresponding to the error at  $x^{14}$  is detected, the erroneous bit coming out of the buffer register must be corrected by an exclusive-OR gate. Based on this reason-





**Figure 5.3** Decoder for the (15, 11) cyclic Hamming code generated by  $g(x) = 1 + x + x^4$ .

ing, a decoder for the (15, 11) cyclic Hamming code can be implemented as shown in Fig. 5.3.

**Example 5.5** Consider the decoder for the (15, 11) Hamming code generated by  $g(x) = 1 + x + x^4$ . Suppose that the code word  $c(x) = 1 + x + x^2 + x^3 + x^{12}$ , or  $\mathbf{c} = (1\ 1\ 1\ 1\ 0\ 0\ 0\ 0\ 0\ 0\ 1\ 0\ 0)$ , is transmitted and the error polynomial  $e(x) = x^9$  is due to a single error that occurred at location  $x^9$ . Then the received polynomial becomes  $r(x) = 1 + x + x^2 + x^3 + x^9 + x^{12}$ , or  $\mathbf{r} = (1\ 1\ 1\ 1\ 0\ 0\ 0\ 0\ 1\ 0\ 0\ 1\ 0\ 0)$ . When the received word has completely shifted into the syndrome and buffer registers, the syndrome register contains (0 1 0 1), which is the syndrome for the error at  $x^9$ . After the error is corrected, the buffer register must shift the entire received word back to its original position.

In Table 5.6, the contents in the syndrome generator and the contents in the buffer register are recorded after each shift. At shift 15 the buffer register contains the corrected code word to be delivered to the user.

Consider next the decoding circuit which is identical to the decoder shown in Fig. 5.3 except for the method of shifting the received polynomial  $r(x)$  into the syndrome register. Here  $r(x)$  is read into the syndrome register from the rightmost stage as shown in Fig. 5.4. This generates  $x^{n-k}r(x)$ , which is equivalent to preshifting  $n - k$  times cyclically to the right. If a single-error pattern  $e_i(x)$  has occurred at  $x^{n-1}$  in  $r(x)$ , then

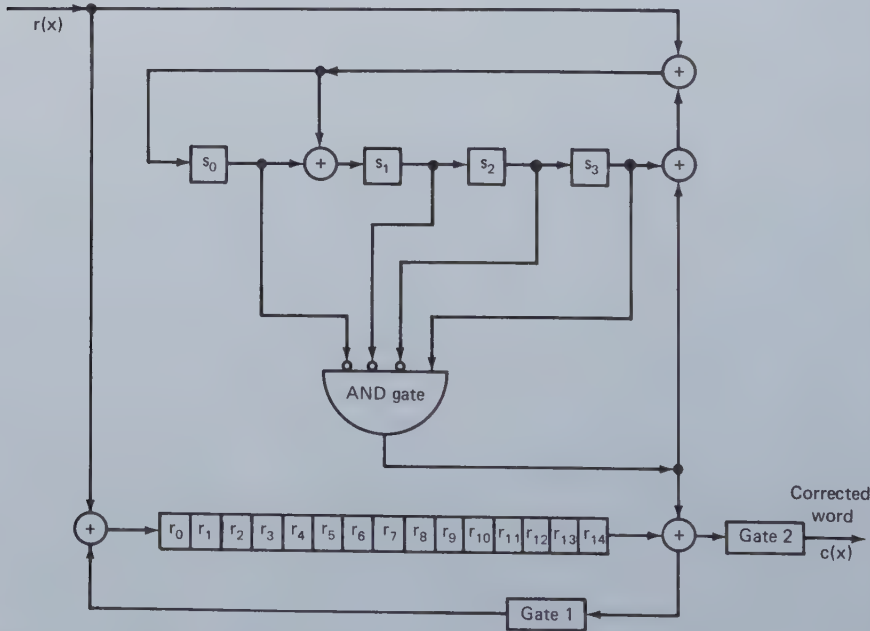
$$x^{n-k}e_{n-1}(x) = x^{n-k-1} \bmod (x^n + 1) \quad (5.10)$$

**TABLE 5.6** Error-Correction Table for the Circuit Shown in Fig. 5.3 (Input from the Left End)

| Shift no.<br>$i$ | Syndrome register contents |       |       |       | AND gate output | Buffer register contents        |
|------------------|----------------------------|-------|-------|-------|-----------------|---------------------------------|
|                  | $s_0$                      | $s_1$ | $s_2$ | $s_3$ |                 |                                 |
| 0                | 0                          | 1     | 0     | 1     | 0               | 1 1 1 1 0 0 0 0 0 1 0 0 1 0 0   |
| 1                | 1                          | 1     | 1     | 0     | 0               | 0 1 1 1 1 0 0 0 0 0 1 0 0 1 0   |
| 2                | 0                          | 1     | 1     | 1     | 0               | 0 0 1 1 1 1 0 0 0 0 0 1 0 0 1   |
| 3                | 1                          | 1     | 1     | 1     | 0               | 1 0 0 1 1 1 1 0 0 0 0 0 1 0 0   |
| 4                | 1                          | 0     | 1     | 1     | 0               | 0 1 0 0 1 1 1 1 0 0 0 0 0 1 0   |
| 5                | 1                          | 0     | 0     | 1     | 1               | 0 0 1 0 0 1 1 1 1 0 0 0 0 0 1   |
| 6                | 0                          | 0     | 0     | 0     | 0               | 0 0 0 1 0 0 1 1 1 1 1 0 0 0 0 0 |
| 7                | 0                          | 0     | 0     | 0     | 0               | 0 0 0 0 1 0 0 1 1 1 1 1 0 0 0 0 |
| 8                | 0                          | 0     | 0     | 0     | 0               | 0 0 0 0 0 1 0 0 1 1 1 1 1 0 0 0 |
| 9                | 0                          | 0     | 0     | 0     | 0               | 0 0 0 0 0 0 1 0 0 1 1 1 1 1 0 0 |
| 10               | 0                          | 0     | 0     | 0     | 0               | 0 0 0 0 0 0 0 1 0 0 1 1 1 1 1 0 |
| 11               | 0                          | 0     | 0     | 0     | 0               | 0 0 0 0 0 0 0 0 1 0 0 1 1 1 1 1 |
| 12               | 0                          | 0     | 0     | 0     | 0               | 1 0 0 0 0 0 0 0 0 1 0 0 1 1 1 1 |
| 13               | 0                          | 0     | 0     | 0     | 0               | 1 1 0 0 0 0 0 0 0 0 1 0 0 1 1 1 |
| 14               | 0                          | 0     | 0     | 0     | 0               | 1 1 1 0 0 0 0 0 0 0 0 1 0 0 1 1 |
| 15               | 0                          | 0     | 0     | 0     | 0               | 1 1 1 1 0 0 0 0 0 0 0 0 1 0 0*  |

\*Corrected code word.

Received word



**Figure 5.4** Decoder for the (15, 11) cyclic Hamming code generated by  $g(x) = 1 + x + x^4$  that used a different method for shifting  $r(x)$ .

Thus we see that the error is shifted to the location  $x^{n-k-1}$ . Therefore the syndrome register contains the syndrome corresponding to the error  $e_{n-k-1}(x) = x^{n-k-1}$ .

**Example 5.6** Consider again the decoding of the (15, 11) Hamming code generated by  $g(x) = 1 + x + x^4$ . Suppose that the received polynomial  $r(x)$  is shifted into the syndrome generator from the right end and that a single error occurs at location  $x^9$  as assumed in Example 5.5. For each shift, the syndromes corresponding to the error patterns and the buffer register contents are listed in Table 5.7. The syndrome register automatically resets its contents to 0 by adding the output "1" of the 4-input error detector and the syndrome bit "1" of the rightmost stage of the syndrome register at shift 6. The decoding circuits for the (15, 11) Hamming code discussed so far are obtained by fully employing the cyclic structure. But they may have a longer decoding delay.

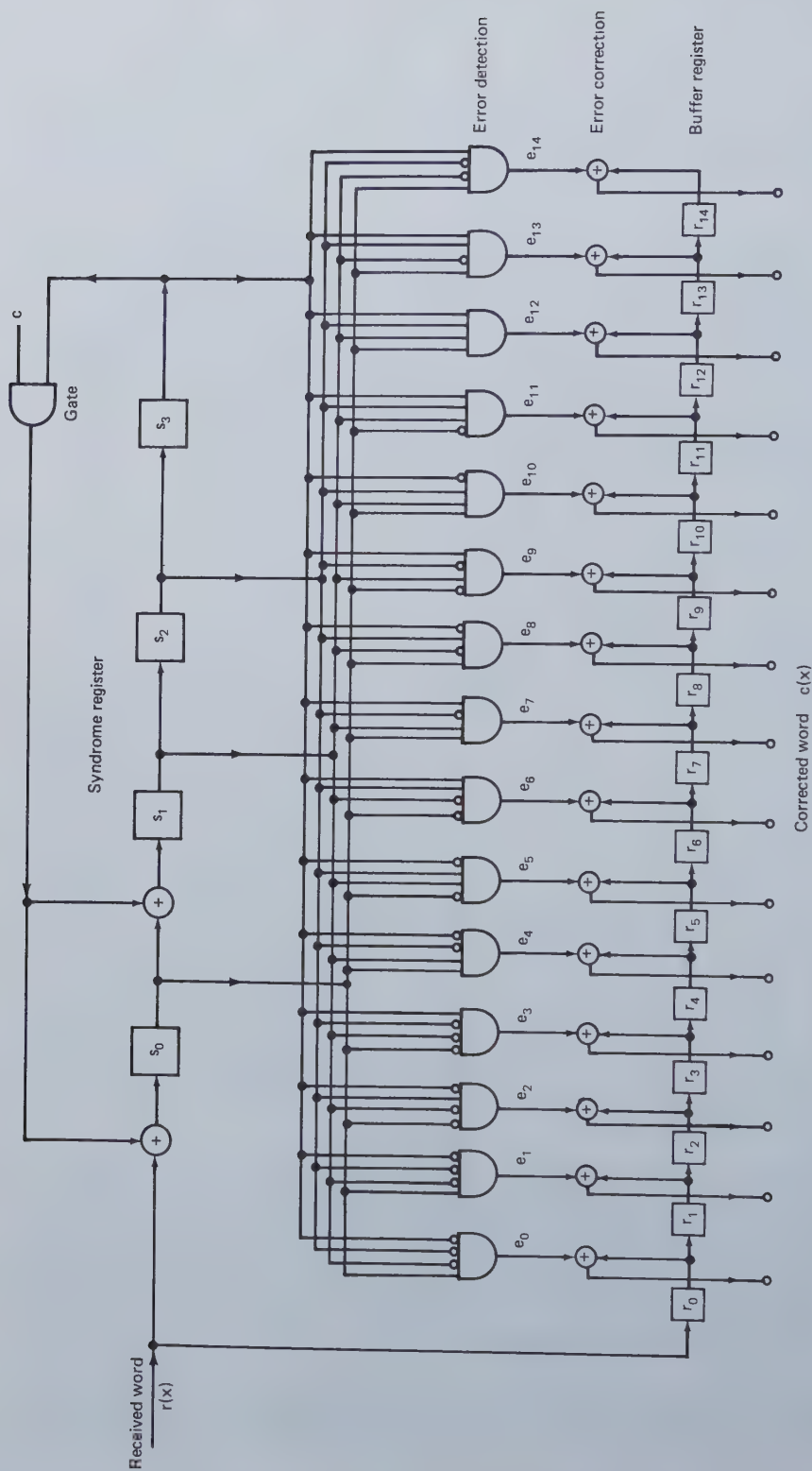
TABLE 5.7 Error-Correction Table for the Circuit Shown in Fig. 5.4 (Input from the Right End)

| Shift no.<br>$i$ | Syndrome register contents |       |       |       | AND gate output | Buffer register contents         |
|------------------|----------------------------|-------|-------|-------|-----------------|----------------------------------|
|                  | $s_0$                      | $s_1$ | $s_2$ | $s_3$ |                 |                                  |
| 0                | 1                          | 0     | 1     | 1     | 0               | 1 1 1 1 0 0 0 0 0 1 0 0 1 0 0    |
| 1                | 1                          | 0     | 0     | 1     | 0               | 0 1 1 1 1 0 0 0 0 0 0 1 0 0 1 0  |
| 2                | 1                          | 0     | 0     | 0     | 0               | 0 0 1 1 1 1 0 0 0 0 0 0 1 0 0 1  |
| 3                | 0                          | 1     | 0     | 0     | 0               | 1 0 0 1 1 1 1 0 0 0 0 0 0 1 0 0  |
| 4                | 0                          | 0     | 1     | 0     | 0               | 0 1 0 0 1 1 1 1 0 0 0 0 0 0 1 0  |
| 5                | 0                          | 0     | 0     | 1     | 1               | 0 0 1 0 0 1 1 1 1 0 0 0 0 0 0 1  |
| 6                | 0                          | 0     | 0     | 0     | 0               | 0 0 0 1 0 0 1 1 1 1 1 0 0 0 0 0  |
| 7                | 0                          | 0     | 0     | 0     | 0               | 0 0 0 0 1 0 0 1 1 1 1 1 0 0 0 0  |
| 8                | 0                          | 0     | 0     | 0     | 0               | 0 0 0 0 0 1 0 0 1 1 1 1 1 0 0 0  |
| 9                | 0                          | 0     | 0     | 0     | 0               | 0 0 0 0 0 0 1 0 0 1 1 1 1 1 0 0  |
| 10               | 0                          | 0     | 0     | 0     | 0               | 0 0 0 0 0 0 0 0 1 0 0 1 1 1 1 0  |
| 11               | 0                          | 0     | 0     | 0     | 0               | 0 0 0 0 0 0 0 0 0 1 0 0 1 1 1 1  |
| 12               | 0                          | 0     | 0     | 0     | 0               | 1 0 0 0 0 0 0 0 0 0 1 0 0 1 1 1  |
| 13               | 0                          | 0     | 0     | 0     | 0               | 1 1 0 0 0 0 0 0 0 0 0 0 1 0 0 1  |
| 14               | 0                          | 0     | 0     | 0     | 0               | 1 1 1 0 0 0 0 0 0 0 0 0 0 1 0 0  |
| 15               | 0                          | 0     | 0     | 0     | 0               | 1 1 1 1 0 0 0 0 0 0 0 0 0 0 1 0* |

\*Corrected code word.

Finally, let us introduce another kind of decoding circuit, one which uses parallel correction for the (15, 11) cyclic Hamming code. This decoder uses a combinational logic circuit that determines the error from the syndrome. While this decoding circuit is a little more complex than a Meggitt decoder with sequential correction, it brings about a considerable decrease in decoding time.

**Example 5.7** A complete decoder for the (15, 11) Hamming code with parallel correction is shown in Fig. 5.5. If a single error, as assumed before, occurs at the

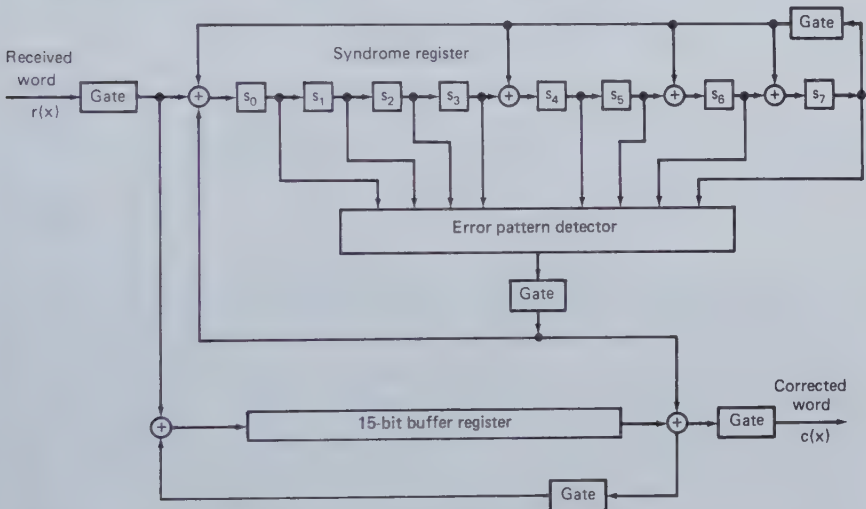


**Figure 5.5** Decoder for the (15, 11) Hamming cyclic code with parallel correction.

location  $x^9$ , then the received word is  $\mathbf{r} = (1\ 1\ 1\ 1\ 0\ 0\ 0\ 0\ 1\ 0\ 0\ 1\ 0\ 0)$ . Thus the syndrome register contains  $(0\ 1\ 0\ 1)$ , which is the syndrome corresponding to the error  $e_9 = r_9$  at location  $x^9$ . Therefore only the output of the 4-input AND gate at  $e_9$  is 1. The erroneous digit 1 at  $r_9$  in the buffer register is corrected by an exclusive-OR gate. Once this single error has been corrected, the information can be delivered to the user. Thus the syndrome register is reset to 0, and the decoder is ready to receive the next sequence. This decoder does minimize the decoding delay, but it is more complex than the circuits of Figs. 5.3 and 5.4.

**Example 5.8** As seen in Table 5.3, the polynomial  $g(x) = 1 + x^4 + x^6 + x^7 + x^8$  generates a  $(15, 7)$  BCH code. Since this code has the minimum distance of 5, it can correct all single and double errors. A Meggitt decoder for this  $(15, 7)$  BCH cyclic code is shown in Fig. 5.6. This code can be easily analyzed and implemented by the decoding algorithms that will be presented in Chap. 7 for BCH codes and in Chap. 8 for majority-logic-decodable codes (see Example 8.1).

Finally, it is recommended that the reader review Theorem 8.2 in order to understand the modification to the syndrome update procedure.



**Figure 5.6** Decoder for the  $(15, 7)$  BCH cyclic code generated by  $g(x) = 1 + x^4 + x^6 + x^7 + x^8$ .

### 5.3 Decoding of Modified Cyclic Codes

In coding applications for communication systems, the desired code must often be modified to fit the particular constraints on code length  $n$  and information digits  $k$ . In this section, the decoding of two code modifications is presented.

#### 5.3.1 Decoding of expurgated Hamming codes

A code can be expurgated by discarding the odd-weight code words of the original code. The expurgated code consists of the even-weight code



words. The expurgated Hamming code is generated by  $g(x) = (1 + x)p(x)$ , where  $p(x)$  is a primitive polynomial of degree  $m = n - k - 1$ . The result is a  $(2^m - 1, 2^m - m - 2)$  expurgated Hamming code which is capable of correcting a single error and detecting any combination of double errors.

Let  $r(x)$  be the received code word. In order to compute the syndrome of  $r(x)$ , divide  $x^m r(x)$  by  $p(x)$  and  $r(x)$  by  $1 + x$ , where

$$x^m r(x) = q_1(x)p(x) + s_\lambda(x) \quad (5.11)$$

$$r(x) = q_2(x)(1 + x) + s_\sigma \quad (5.12)$$

where  $s_\lambda(x)$  is of degree  $m - 1$  or less and  $s_\sigma$  is either 0 or 1. If  $r(x)$  is divisible by  $g(x) = (1 + x)p(x)$ , then it follows that  $s_\lambda(x) = 0$  and  $s_\sigma = 0$ . Thus  $r(x)$  is a legitimate code polynomial. If  $r(x)$  is not divisible by  $g(x)$ , then  $r(x)$  is not a code word and contains some errors in it. Since the degree of  $p(x)$  is  $m = n - k - 1$ , the degree of  $s_\lambda(x)$  must be  $m - 1 = n - k - 2$  or less. However, since the syndrome  $s(x)$  of  $r(x)$  is of degree  $m = n - k - 1$ , it must follow that

$$s(x) = xs_\lambda(x) + s_\sigma \quad (5.13)$$

Let  $e_i(x) = x^i$  be an error pattern representing a single error at the location  $x^i$ . Then we have

$$x^i = q_1(x)p(x) + s_\lambda(x) \quad 0 \leq i < n \quad (5.14)$$

For  $m \leq i$ , a single error occurs somewhere in the  $n$ -bit information section, which includes the rightmost position  $x^m$  in the  $m$ -bit parity-check section. Since the syndrome  $s_\lambda(x)$  is the remainder that results from dividing  $x^i$  by  $p(x)$ ,  $s_\lambda(x)$  is not 0. For  $i < m$ , a single error occurs somewhere in the  $m$ -bit parity-check section. In this case it is impossible to divide  $x^i$  by  $p(x)$ . Therefore  $q_1(x)$  must be 0. Next, dividing  $x^i$  by  $1 + x$  yields

$$x^i = q_2(x)(1 + x) + s_\sigma \quad (5.15)$$

from which we obtain  $s_\sigma = 1$  by substituting  $x = 1$ . Thus the syndrome for a single error will be represented by

$$s(x) = xs_\lambda(x) + 1 \quad (5.16)$$

For  $s_\sigma = 1$  and  $s_\lambda(x) \neq 0$ , the decoder assumes that a single error has occurred and initiates the error-correction process.

Finally, consider an error pattern of double errors as follows:

$$e(x) = x^i + x^j \quad i < j \quad (5.17)$$

If Eq. (5.17) is divided by  $p(x)$ , it follows that

$$x^{i+m}(1 + x^{j-i}) = q_1(x)p(x) + s_\lambda(x) \quad (5.18)$$

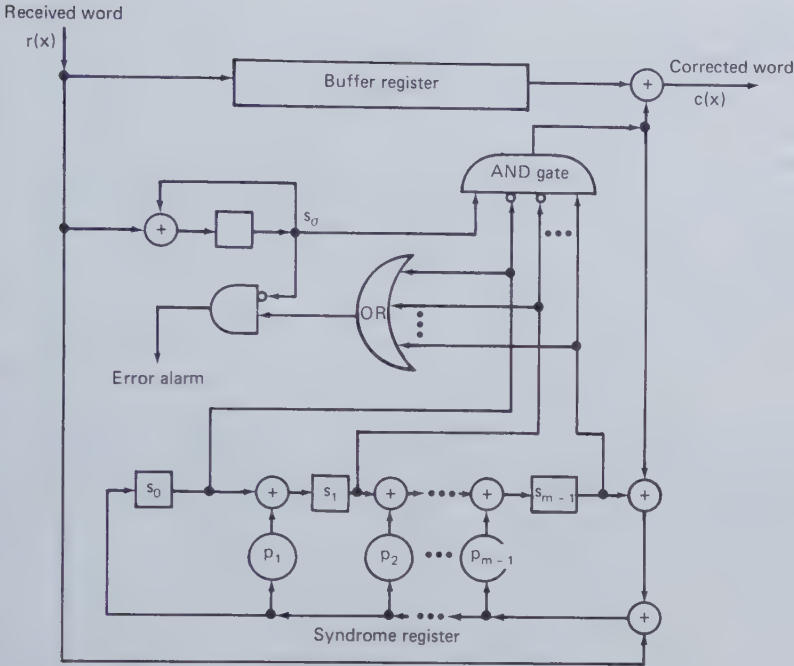


Figure 5.7 A decoder for the expurgated Hamming code with single-error correction and double-error detection.

Since  $j - i < n$ ,  $p(x)$  is not a divisor of  $1 + x^{j-i}$ . Hence  $s_\lambda(x)$  is the non-zero remainder, i.e.,  $s_\lambda(x) \neq 0$ . Next, consider the case of

$$x^i(1 + x^{j-i}) = q_2(x)(1 + x) + s_\sigma \quad (5.19)$$

Substituting  $x = 1$  into Eq. (5.19), we obtain  $s_\sigma = 0$ . Thus the syndrome for any combination of double errors has the form

$$s(x) = xs_\lambda(x) \quad (5.20)$$

Consequently, for  $s_\sigma = 0$  and  $s_\lambda(x) \neq 0$ , the decoder assumes that an error pattern of double errors has occurred. Figure 5.7 shows a decoder for the expurgated Hamming code with single-error correction and double-error detection.

### 5.3.2 Decoding of shortened cyclic codes

We covered the structural properties of shortened cyclic codes in Sec. 4.6.1. A shortened cyclic code of length  $n - i$  can be decoded by a Meggitt decoder designed for the original  $(n, k)$  code. However, a mismatch may occur in the timing of the original decoder since the incoming code word contains  $n - i$  digits. This mismatch in the timing

can be solved by proper circuit design based on  $n - i$  clock times. Fortunately, the deleted  $i$  digits do not affect the parity-check and syndrome computations. After the incoming code word of the shortened cyclic code has been received, the syndrome register must be shifted cyclically  $i$  times to generate the proper syndrome for decoding the rightmost bit  $r_{n-i-1}$  in the buffer register. However, elimination of the undesirable decoding delay caused by shifting  $i$  times must somehow be accommodated in the circuit design.

When  $i$  digits are dropped, the syndrome  $s^{(n-k+i)}(x)$  for decoding  $r_{n-i-1}$  is computed from

$$x^{n-k+i}r(x) = q_1(x)g(x) + s^{(n-k+i)}(x) \quad (5.21)$$

rather than computing it as the remainder of  $x^{n-k}r(x)$  which requires  $r(x)$  to be shifted into the syndrome register from the right end. Now, let us show how these extra  $i$  shifts can be eliminated by modifying the connections of the syndrome register. Let  $\gamma(x)$  be the remainder that results from dividing  $x^{n-k+i}$  by  $g(x)$ . Then we have

$$x^{n-k+i} = q_2(x)g(x) + \gamma(x)$$

$$\text{or} \quad \gamma(x) = x^{n-k+i} + q_2(x)g(x) \quad (5.22)$$

whose degree is  $n - k - 1$  or less. Coupling Eq. (5.21) with Eq. (5.22) gives

$$\gamma(x)r(x) = [q_1(x) + q_2(x)r(x)]g(x) + s^{(n-k+i)}(x) \quad (5.23)$$

Equation (5.23) indicates that the syndrome  $s^{(n-k+i)}(x)$  is the remainder obtained from dividing the product  $\gamma(x)r(x)$  by  $g(x)$ . Since this is accomplished by incorporating a circuit for multiplication and division, the extra  $i$  shifts of the syndrome register can be avoided. The following examples will show how this can be done. The shift-register circuits can be used for the multiplication and division of polynomials over  $GF(q)$ . They are frequently employed in the design and construction of encoders and decoders.

**Example 5.9** Consider a shift-register circuit for multiplying an input polynomial  $a(x) = 1 + x^2 + x^4$  by  $b(x) = 1 + x^2$  and then dividing by  $c(x) = 1 + x + x^2$ . The shift register shown in Fig. 5.8 contains part of the dividend, and the input connections add the product of  $a(x)$  and  $b(x)$  into the circuit. Hence the product  $a(x)b(x) = (1 + x^2 + x^4)(1 + x^2) = 1 + x^6$  is added into the circuit and the division  $a(x)b(x)/c(x) = (1 + x^6)/(1 + x + x^2) = 1 + x + x^3 + x^4$  becomes the output. The operational steps for the multiplication-division circuit shown in Fig. 5.8 are listed in Table 5.8.

**Example 5.10** Consider a (29, 24) shortened Hamming code; this code was originally a (31, 26) Hamming code generated by  $g(x) = 1 + x^2 + x^5$  and shortened by  $i = 2$  bits. In order to eliminate the extra two shifts of the syndrome register, we must modify the connection of the syndrome register. The remainder poly-

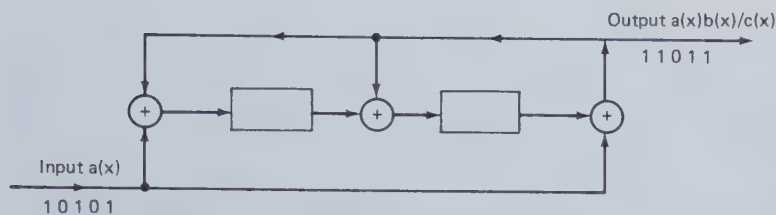


Figure 5.8 A shift-register circuit for multiplication and division.

TABLE 5.8 Operational Steps for the Multiplication-Division Circuit Shown In Fig. 5.8

| Shift no.<br>$i$ | After $i$ th shift |                   |          |        |
|------------------|--------------------|-------------------|----------|--------|
|                  | Input              | Register contents | Feedback | Output |
| 0                | 1                  | 0 0               | 1 1      | 1      |
| 1                | 0                  | 0 1               | 1 1      | 1      |
| 2                | 1                  | 1 1               | 0 0      | 0      |
| 3                | 0                  | 1 1               | 1 1      | 1      |
| 4                | 1                  | 1 0               | 1 1      | 1      |

nomial  $\gamma(x) = x^2 + x^4$  is determined by dividing  $x^{n-k+2}$  by  $g(x)$ . Using the principle discussed in Example 5.9, we find that the syndrome register must be modified in such a way that the product  $\gamma(x)r(x)$  is fed into the dividend circuit of  $g(x)$ . A decoder for the (29, 24) shortened cyclic code generated by  $g(x) = 1 + x^2 + x^5$  is shown in Fig. 5.9.

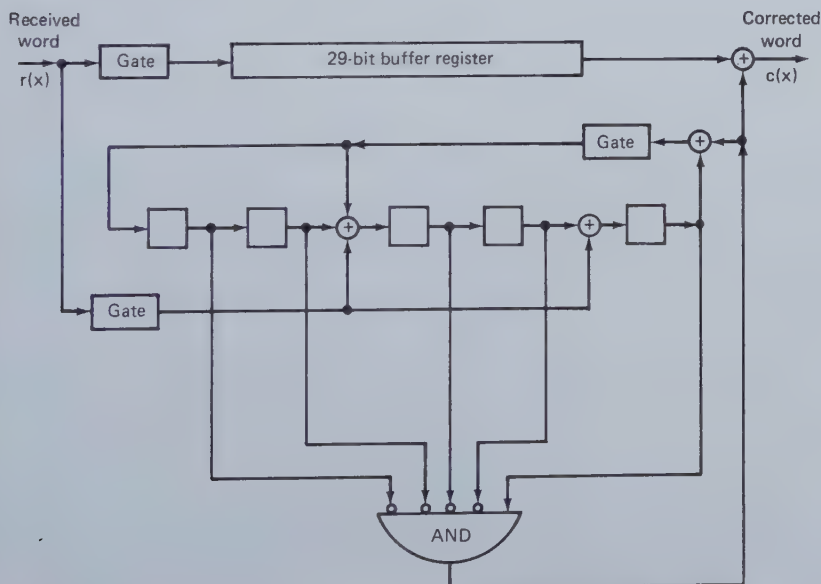
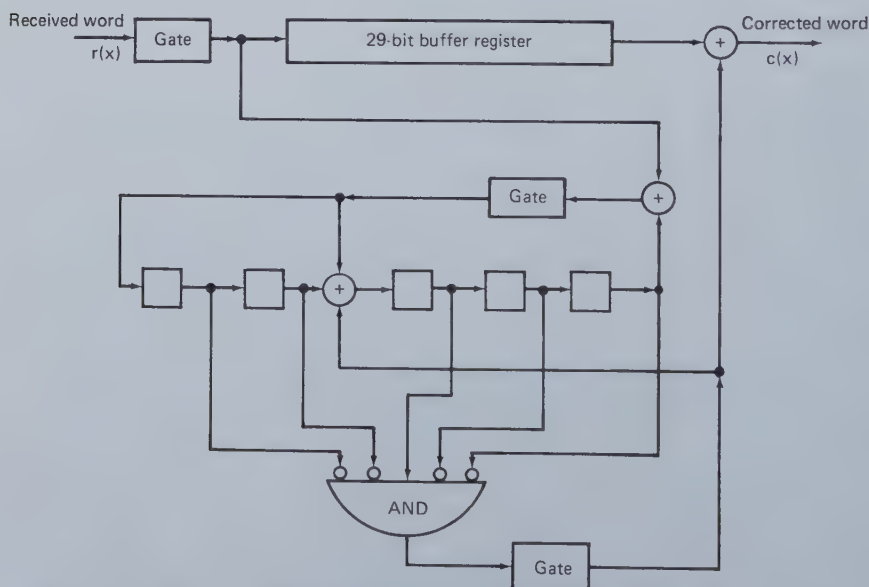


Figure 5.9 A decoder with a modified syndrome-register connection for the (29, 24) shortened Hamming code generated by  $g(x) = 1 + x^2 + x^5$ .

Another possible way of avoiding the extra  $i$  shifts of the syndrome register can be achieved by modifying the error-pattern detector of the original decoder. It is necessary to redesign the error-pattern detector in such a way that the syndrome in the syndrome generator corresponds to a correctable error at the location  $x^{n-i-1}$ . When the received bit  $r_{n-i-1}$  is corrected, the effect of the error bit  $e_{n-i-1}$  on the syndrome should be removed by syndrome resetting. For example, suppose that the received word  $r(x)$  is fed into the syndrome register from the right end. Let  $\gamma(x)$  be the remainder polynomial of degree  $n - k - 1$  that is obtained from dividing  $x^{n-i-1} \cdot x^{n-k} = x^{2n-k-i-1}$  by  $g(x)$ . By adding  $\gamma(x)$  to the syndrome, the effect of the error bit  $e_{n-i-1}$  on the syndrome will be removed.

**Example 5.11** Consider again the (29, 24) shortened Hamming code discussed in Example 5.10. This time assume that the received word  $r(x)$  is fed into the syndrome register from the right end. Suppose that a single error occurs at location  $x^{28}$ . Then the syndrome  $\gamma(x)$  corresponding to this error pattern is the remainder that results from dividing  $x^{28}e(x) = x^{33}$  by the generator polynomial  $g(x) = 1 + x^2 + x^5$ . Since  $\gamma(x) = x^2$ , the corresponding syndrome is (0 0 1 0 0). Thus, in decoding the (29, 24) shortened code, the error-pattern detector must be modified to check whether the syndrome in the syndrome register is (0 0 1 0 0). If it is, the extra two shifts of the syndrome register can be avoided. The decoder for the (29, 24) shortened Hamming code in this particular modification is shown in Fig. 5.10.



**Figure 5.10** A decoder with a modified error-pattern detector for the (29, 24) shortened Hamming code generated by  $g(x) = 1 + x^2 + x^5$ .



## 5.4 Basic Concept of Error Trapping

Meggitt decoders can be applied to any cyclic code, as discussed previously. Error-trapping decoding is an extended version of Meggitt decoding and is used for practical applications. It is particularly effective for decoding some double errors or single burst errors. The latter type of error is discussed in Chap. 6.

Let  $c(x)$  be the transmitted code word. If the error pattern  $e(x)$  is caused by the channel noise, the received word is simply expressed as  $r(x) = c(x) + e(x)$ , or

$$c(x) = e(x) + r(x) \quad (5.24)$$

If all errors are confined in the  $n - k$  redundancy section of  $r(x)$ , the errors are trapped in the syndrome register,  $e(x) = s(x)$ , as demonstrated in Theorem 5.1. Thus Eq. (5.24) becomes

$$\begin{aligned} c(x) &= s(x) + r(x) \\ &= (s_0 + r_0) + (s_1 + r_1)x + \cdots + (s_{n-k-1} + r_{n-k-1})x^{n-k-1} \\ &\quad + r_{n-k}x^{n-k} + r_{n-k+1}x^{n-k+1} + \cdots + r_{n-1}x^{n-1} \end{aligned}$$

or, in vector form,

$$\begin{aligned} \mathbf{c} &= (s_0 + r_0, s_1 + r_1, \dots, s_{n-k-1} + r_{n-k-1}, r_{n-k}, r_{n-k+1}, \dots, r_{n-1}) \\ &\quad | \leftarrow n - k \text{ parity-check section} \rightarrow | \leftarrow k \text{ information section} \rightarrow | \end{aligned} \quad (5.25)$$

Thus, if the errors in  $r(x)$  are all confined to the  $n - k$  parity-check section, correction is simply accomplished by adding the contents of the syndrome register to the received bits in the  $n - k$  parity-check section of  $r(x)$ .

**Theorem 5.2** Let  $s^{(n-k)}(x)$  be the syndrome of  $x^{n-k}r(x)$ . Then  $s^{(n-k)}(x)$  is the remainder after dividing  $x^{n-k}r(x)$  by  $g(x)$ . If all errors lie in the  $n - k$  high-order positions of  $r(x)$ , the error pattern  $e(x)$  is identical to  $x^k s^{(n-k)}(x)$ .

**Proof** If all errors lie in the  $n - k$  high-order positions of  $r(x)$ , then  $e(x) = e_k x^k + e_{k+1} x^{k+1} + \cdots + e_{n-1} x^{n-1}$ . If  $r(x)$  is cyclically shifted  $n - k$  times, we have  $x^{n-k}r(x)$ . Dividing  $x^{n-k}r(x)$  by  $g(x)$  yields the corresponding syndrome:

$$s^{(n-k)}(x) = x^{n-k}r(x) + q(x)g(x) \quad (5.26)$$

where  $q(x)$  is the quotient polynomial resulting from division. If we multiply  $s^{(n-k)}(x)$  by  $x^k$ , Eq. (5.26) becomes

$$x^k s^{(n-k)}(x) = x^n r(x) + x^k q(x)g(x)$$

Adding the syndrome  $x^k s^{(n-k)}(x)$  to the high-order part of  $r(x)$  yields

$$r(x) + x^k s^{(n-k)}(x) = r(x)(1 + x^n) + q_1(x)g(x) = q_1(x)g(x) \bmod (x^n + 1) \quad (5.27)$$

where  $q_1(x) = x^k q(x)$ . Thus, Eq. (5.27) represents a code polynomial, that is,

$$c(x) = r(x) + x^k s^{(n-k)}(x) \quad (5.28)$$

Since  $r(x)$  is the sum of  $c(x)$  and the error pattern  $e(x)$ ,  $x^k s^{(n-k)}(x)$  must be equivalent to the error pattern  $e(x)$ , that is,

$$e(x) = x^k s^{(n-k)}(x) \quad (5.29)$$

**Example 5.12** Consider the (15, 11) cyclic code generated by  $g(x) = 1 + x + x^4$ . Suppose that the code word  $\mathbf{c} = (1\ 1\ 1\ 1\ 0\ 0\ 0\ 0\ 0\ 0\ 0\ 1\ 0\ 0)$  is transmitted and  $\mathbf{r} = (1\ 1\ 1\ 1\ 0\ 0\ 0\ 0\ 0\ 0\ 1\ 1\ 0\ 1)$  is received. We see that double errors have occurred at locations  $x^{11}$  and  $x^{14}$ . These errors lie in the  $n - k = 4$  high-order part of  $\mathbf{r}$ . The error pattern is then  $e(x) = x^{11} + x^{14}$ . Now, if  $r(x)$  is cyclically shifted  $n - k = 4$  times, it follows that

$$x^4 r(x) = x^4 + x^5 + x^6 + x^7 + x^{15} + x^{16} + x^{18}$$

Thus the error locations of  $x^{11}$  and  $x^{14}$  move to  $x^0$  and  $x^3$  in the  $n - k$  parity-check section of  $r(x)$ . Hence the errors are trapped in the syndrome register. Dividing  $x^4 r(x)$  by  $g(x)$ , we obtain the remainder  $s^{(4)}(x) = 1 + x^3$  which is the syndrome of  $r^{(4)}(x)$ . According to Theorem 5.2, the error pattern  $e(x)$  is identical to  $x^{11} s^{(4)}(x)$ . Therefore the error pattern is  $e(x) = x^{11}(1 + x^3) = x^{11} + x^{14}$  as expected.

Suppose that this time the errors are not confined to the  $n - k$  high-order section of  $r(x)$  but are located in any  $n - k$  consecutive positions, i.e.,  $x^i, x^{i+1}, \dots, x^{i+(n-k-1)}$  of  $r(x)$ . The corresponding error pattern then has the form

$$e(x) = e_i x^i + e_{i+1} x^{i+1} + \dots + e_{i+(n-k-1)} x^{i+(n-k-1)} \quad (5.30)$$

If  $r(x)$  is cyclically shifted  $n - i$  times to the right, the error pattern becomes

$$\begin{aligned} x^{n-i} e(x) &= e_i x^n + e_{i+1} x^{n+1} + \dots + e_{i+(n-k-1)} x^{2n-k-1} \\ &= e_i + e_{i+1} x + \dots + e_{i+(n-k-1)} x^{n-k-1} \pmod{x^n + 1} \end{aligned} \quad (5.31)$$

Thus the errors will be confined to the  $n - k$  parity-check section of  $r^{(n-i)}(x)$ . The errors are thus trapped in the syndrome register, and the error pattern is equal to  $x^i s^{(n-i)}(x)$ . As a result, the errors can be corrected accordingly.

**Example 5.13** Consider again the (15, 11) cyclic code generated by  $g(x) = 1 + x + x^4$ . Assume that the code word  $\mathbf{c} = (1\ 1\ 1\ 1\ 0\ 0\ 0\ 0\ 0\ 0\ 0\ 1\ 0\ 0)$  is transmitted and  $\mathbf{r} = (1\ 1\ 1\ 1\ 0\ 0\ 0\ 1\ 0\ 0\ 1\ 0\ 0)$  is received. Therefore double errors have occurred at locations  $x^7$  and  $x^{10}$ . Apparently these errors are not con-

fined to the  $n - k = 4$  high-order portion of  $r(x)$ . Hence the error pattern has the form  $e(x) = x^7 + x^{10}$ . If  $r(x)$  is cyclically shifted  $n - i = 8$  times to the right, we have  $x^8 r(x) = x^8 + x^9 + x^{10} + x^{11} + x^{15} + x^{18} + x^{20}$ . Dividing  $r^{(8)}(x)$  by  $g(x)$  gives the corresponding syndrome  $s^{(8)}(x) = 1 + x^3$ . Thus the error pattern is  $e(x) = x^7 s^{(8)}(x) = x^7 + x^{10}$ , which will be trapped in the syndrome register. Error correction can be accomplished by simply adding the error pattern  $e(x) = x^7 s^{(8)}(x)$  to the received word  $r(x)$ .

Consider a  $t$ -error-correcting cyclic code. Assume that an error pattern  $e(x)$  with  $t$  or fewer errors is confined to the  $n - k$  consecutive positions. Now, let  $w[s(x)]$  denote the weight of the syndrome. If the weight of the syndrome is  $t$  or less, that is,  $w[s(x)] \leq t$ , then it is said that the errors are trapped in the syndrome register. Thus the syndrome bits in the syndrome register are identical to the error bits at the  $n - k$  high-order positions. After the received word  $r(x)$  has been completely fed into the syndrome register,  $w[s(x)]$  is tested by a threshold gate. If  $t$  or fewer of the inputs of the threshold gate are 1, the output of the threshold gate is 1; otherwise it is 0. Thus the received word is read out of the buffer register one bit at a time and is corrected by the error bits shifted from the syndrome register.

If  $w[s(x)] > t$ , the errors are not confined to the  $n - k$  high-order positions of  $r(x)$  and they cannot be trapped in the syndrome register. After shifting the syndrome register once, the weight of the new syndrome  $s^{(1)}(x)$  is tested. If  $w[s^{(1)}(x)] \leq t$ , then the errors are confined to the locations  $x^\lambda$  of  $r(x)$ ,  $k - 1 \leq \lambda \leq n - 2$ , and  $e^{(1)}(x) = s^{(1)}(x)$  because the errors are trapped. Thus the rightmost bit  $r_{n-1}$  of  $r(x)$  is error-free. If  $w[s^{(1)}(x)] > t$ , the syndrome register has to shift once more. Such a shifting operation continues until the value of  $w[s^{(i)}(x)]$  equals  $t$  or less at the end of the  $i$ th shift for  $1 \leq i \leq k$ . The first  $i$  received bits  $r_\lambda$ ,  $\lambda = n - i, n - i + 1, \dots, n - 1$  in the buffer register are error-free and consequently  $e^{(i)}(x) = s^{(i)}(x)$  at the locations of  $x^\lambda$ ,  $\lambda = k - i, k - i + 1, \dots, n - i - 1$ . After the  $i$  error-free bits of  $r(x)$  have been shifted out of the buffer register, the syndrome bits are read out and are used to correct the next  $n - k$  bits coming out of the buffer register. Thus it is possible to implement the error-trapping decoder shown in Fig. 5.11 using the error-trapping concepts already discussed.

## 5.5 Improved Version of Error-Trapping Decoding

The decoding scheme described here is appropriate for short codes with relatively high redundancy while it seems to be less useful for longer codes. Nevertheless, this improved version of error-trapping decoding is effective for many double- or triple-error-correcting codes.

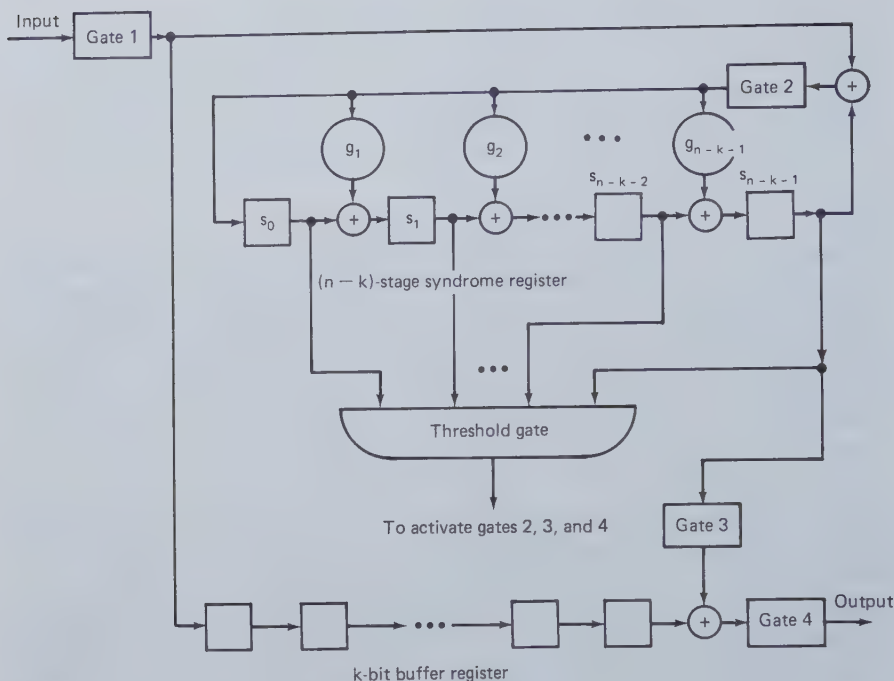


Figure 5.11 An error-trapping decoder.

This decoder can correct error patterns in which most of the errors are confined to the  $n - k$  parity-check section and some of the errors are in the  $k$  information section of the received word  $r(x)$ .

Assume that  $r(x)$  is shifted into the syndrome register from the right end. This is equivalent to preshifting  $r(x)$  cyclically  $n - k$  times to the right. It follows that  $x^{n-k}r(x) = r^{(n-k)}(x)$ . The  $i$  cyclic shifts of the error pattern  $e(x)$  are expressed as  $x^{n-k+i}e(x) = e^{(n-k+i)}(x)$ . This shifted error pattern can be segmented into the following two parts:

$$e^{(n-k+i)}(x) = e_p^{(n-k+i)}(x) + e_l^{(n-k+i)}(x) \quad i = 0, 1, \dots, n-1 \quad (5.32)$$

where  $e_p(x)$  denotes the errors contained in the parity-check section and  $e_l(x)$  designates the errors in the information section of  $r(x)$ . Dividing  $e_l^{(n-k+i)}(x)$  by the generator polynomial  $g(x)$  yields

$$e_l^{(n-k+i)}(x) = q_i(x)g(x) + \gamma_i(x) \quad (5.33)$$

where  $\gamma_i(x)$  is the remainder polynomial (or the syndrome) of degree  $n - k - 1$  or less. Since  $e_p^{(n-k+i)}(x)$  is the error pattern located in the  $n - k$  parity-check section, it is the trapped errors which are equal to the corresponding syndrome. Thus Eq. (5.32) becomes



$$e^{(n-k+i)}(x) = q_i(x)g(x) + e_p^{(n-k+i)}(x) + \gamma_i(x) \quad (5.34)$$

From Eq. (5.34), we see that the syndrome corresponding to  $e^{(n-k+i)}(x)$  is identical to the  $i$ th shifted syndrome  $s^{(i)}(x)$  of  $x^{n-k}r(x)$  such that

$$s^{(i)}(x) = e_p^{(n-k+i)}(x) + \gamma_i(x) \quad (5.35)$$

or 
$$e_p^{(n-k+i)}(x) = s^{(i)}(x) + \gamma_i(x) \quad i = 0, 1, \dots, n-1 \quad (5.36)$$

Therefore, if the  $i$ th shifted error pattern  $e_I^{(n-k+i)}(x)$  in the information section is known, then  $e_p^{(n-k+i)}(x)$  in the parity-check section can be found.

Let us consider the original error polynomial  $e_I(x)$  in the information section, that is,

$$\begin{aligned} e_I(x) &= e_{n-k}x^{n-k} + e_{n-k+1}x^{n-k+1} + \dots + e_{n-1}x^{n-1} \\ &= x^{n-k}(e_{n-k} + e_{n-k+1}x + \dots + e_{n-1}x^{k-1}) \\ &= x^{n-k}Q(x) \end{aligned} \quad (5.37)$$

where the polynomial  $Q(x)$  is of degree  $k-1$  or less. We call  $Q(x)$  the error sequence in the information section. Kasami (1964) proposed a method for finding a set of polynomials  $\{Q_j(x)\}$ ,  $j = 1, 2, \dots, N$ , of degree  $k-1$  or less. Let  $\gamma_j(x)$  be the remainder that results from dividing the error pattern  $x^{n-k}Q_j(x)$  by the generator polynomial  $g(x)$  of the code. We then have

$$x^{n-k}Q_j(x) = q_j(x)g(x) + \gamma_j(x) \quad (5.38)$$

If the shifted error pattern  $e_I^{(n-k+i)}(x)$  in the information section is identical to the error sequence  $x^{n-k}Q_j(x)$  in the information section, it follows from combining Eqs. (5.33) and (5.38) that

$$\gamma_i(x) = \gamma_j(x) \quad (5.39)$$

Substituting Eqs. (5.36), (5.38), and (5.39) into Eq. (5.34), we obtain

$$e^{(n-k+i)}(x) = s^{(i)}(x) + \gamma_j(x) + x^{n-k}Q_j(x) \quad (5.40)$$

Therefore the original error pattern can be expressed by

$$e(x) = x^{k-i}[s^{(i)}(x) + \gamma_j(x) + x^{n-k}Q_j(x)] \quad (5.41)$$

where  $s^{(i)}(x) + \gamma_j(x)$  represents the error pattern in the parity-check section if and only if  $x^{n-k}Q_j(x) = e_I^{(n-k+i)}(x)$ . For any correctable error pattern  $e(x)$  of weight  $t$  or less, there is one polynomial  $Q_j(x)$  such that  $x^{n-k}Q_j(x)$  locks  $e(x)$  or a cyclic shift of  $e(x)$  in the information section. This locking can be detected uniquely by the inequality

$$w[s^{(i)}(x) + \gamma_j(x)] \leq t - w[Q_j(x)] \quad (5.42)$$



where  $w[f(x)]$  denotes the weight of  $f(x)$ , i.e., the number of nonzero coefficients of a polynomial  $f(x)$ .

If any error pattern of weight  $t$  or less is covered by one polynomial  $Q_j(x)$  in a set  $\{Q_j(x)\}, j = 1, 2, \dots, N$ , we use the following decoding procedure.

1. Compute the syndrome  $s(x)$  from the received word  $r(x)$ .
2. Calculate  $w[s^{(i)}(x) + \gamma_j(x)], i = 0, 1, 2, \dots, n-1; j = 1, 2, \dots, N$ .
3. Determine if the inequality  $w[s^{(i)}(x) + \gamma_j(x)] \leq t - w[Q_j(x)]$  holds for some  $j$ . If so, the corrected word is

$$c(x) = r(x) + x^{k-i}[s^{(i)}(x) + \gamma_j(x) + x^{n-k}Q_j(x)] \quad (5.43)$$

The following is a detailed explanation of how the determination is made. First calculate  $w[e_P(x)] \leq t - w[e_I(x)]$ . If  $w[s(x) + \gamma_\rho(x)] \leq t - w[Q_\rho(x)]$  holds for some  $\rho$ , then  $x^{n-k}Q_\rho(x)$  locks the error pattern  $e_I(x)$  in the information section of  $e(x)$ , and  $s(x) + \gamma_\rho(x)$  locks the error pattern  $e_P(x)$  in the parity-check section of  $e(x)$ . Thus we have

$$e(x) = x^k[s(x) + \gamma_\rho(x) + x^{n-k}Q_\rho(x)] \quad (5.44)$$

The correction process is then completed by computing the modulo-2 sum  $r(x) + e(x)$ . This step requires  $(n-k)N$ -input threshold gates to test the weight of  $s(x) + \gamma_j(x)$  for  $j = 1, 2, \dots, N$ .

4. Shift the syndrome and buffer registers cyclically once if  $w[s(x) + \gamma_j(x)] > t - w[Q_j(x)]$  holds for  $j = 1, 2, \dots, N$ . The new syndrome  $s^{(1)}(x)$  that corresponds to  $e^{(1)}(x) = xe(x)$  is obtained by shifting  $e(x)$  once.

5. Calculate  $w[s^{(1)}(x) + \gamma_j(x)]$  for  $j = 1, 2, \dots, N$ . If  $w[s^{(1)}(x) + \gamma_\rho(x)] \leq t - w[Q_\rho(x)]$  holds for some  $\rho$ , then  $x^{n-k}Q_\rho(x)$  locks the errors  $e_I^{(1)}(x)$  in the information section of  $e^{(1)}(x)$ , and  $s^{(1)}(x) + \gamma_\rho(x)$  locks the errors  $e_P^{(1)}(x)$  in the parity-check section of  $e^{(1)}(x)$ . Thus, we have

$$e^{(1)}(x) = x^{k-1}[s^{(1)}(x) + \gamma_\rho(x) + x^{n-k}Q_\rho(x)] \quad (5.45)$$

The correction process is then completed by computing the modulo-2 sum  $r^{(1)}(x) + e^{(1)}(x)$ . If  $w[s^{(1)}(x) + \gamma_j(x)] > t - w[Q_j(x)]$  for  $j = 1, 2, \dots, N$ , then shift the syndrome and buffer registers cyclically once again.

6. Similarly, continuously shift the syndrome and buffer registers  $i$  times until  $s^{(i)}(x)$  is found such that  $w[s^{(i)}(x) + \gamma_\rho(x)] \leq t - w[Q_\rho(x)]$  for some  $\rho$ . Then the  $i$ th cyclic shift of  $e(x)$  can be obtained as follows:

$$e^{(i)}(x) = x^{k-i}[s^{(i)}(x) + \gamma_\rho(x) + x^{n-k}Q_\rho(x)] \quad (5.46)$$

Thus the corrected code word  $c(x)$  is given as

$$c^{(i)}(x) = r^{(i)}(x) + x^{k-i}[s^{(i)}(x) + \gamma_\rho(x) + x^{n-k}Q_\rho(x)] \quad (5.47)$$

An uncorrectable error pattern has been detected if the syndrome and buffer registers have been cyclically shifted  $n - 1$  times and  $w[s^{(i)}(x) + \gamma_j(x)] > t - w[Q_j(x)]$  for all  $j$ .

## 5.6 Error-Trapping Applications

Consider the (31, 16) BCH triple-error-correcting code. We will show by using the error-trapping technique proposed by Kasami that any error pattern of weight 3 or less is covered by either  $Q_1(x) = 0$ ,  $Q_2(x) = x^7$ , or  $Q_3(x) = x^8$ . In general, the fewer  $Q_j(x)$ , the simpler the decoding procedure. The generator polynomial of this code is one of the factors of  $x^{31} + 1$ . Since  $x^{31} + 1 = (1 + x)g_1(x)g_2(x)$ , the generator polynomial of this code can be either

$$g_1(x) = 1 + x + x^2 + x^3 + x^5 + x^7 + x^8 + x^9 + x^{10} + x^{11} + x^{15}$$

or 
$$g_2(x) = 1 + x^4 + x^5 + x^6 + x^7 + x^8 + x^{10} + x^{12} + x^{13} + x^{14} + x^{15}$$

If we choose  $g_1(x)$ , the encoding can be accomplished by a 15-stage shift register with a feedback connection.

Finding the set of covering polynomials  $Q_j(x)$  for a specific code is not so easy. It is applicable only for short, low rate codes because the decoding circuitry becomes impractical as the code length and error-correcting capability  $t$  become large. Therefore this type of decoder has historical value only and little practical application.

Before going into a fuller description of the error-trapping decoder, we need to clarify what is meant by the distance from  $x^{n-k}Q_j(x)$ , which is the error sequence in the information section. We need to divide the code length  $n$  into thirds as shown in Fig. 5.12 so that the dividing points measured from  $x^{n-k}Q_j(x)$  will lie in the  $n - k$  parity-check section.

Let the error pattern  $e(x)$  be

$$e(x) = x^{l_0} + x^{l_1} + \dots + x^{l_{\mu-1}} \quad 0 \leq l_0 < l_1 < \dots < l_{\mu-1} \quad \mu \leq t \quad (5.48)$$

If we define the distance  $\nu$  as

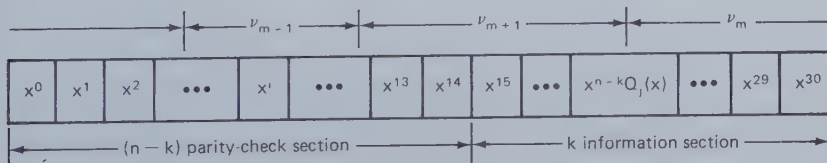


Figure 5.12 Distance division in code digit positions.

$$v_i = l_i - l_{i-1} \quad 1 \leq i \leq n$$

$$v_0 = n + l_0 - l_{\mu-1}$$

$$v_m = \max(v_i)$$

then it follows from Fig. 5.12 that we have three possible conditions, where  $v_m + v_{m+1} + v_{m-1} = n$ :

1.  $v_m > k$
2.  $k \geq n_m > n/3$
3.  $\max(v_{m-1}, v_{m+1}) > (n - k)/2$

For the (31, 16) BCH code we have  $16 \geq n_m > 10$  and  $\max(v_{m-1}, v_{m+1}) > 7$ . If  $v_m \geq 16$ , then  $Q_1(x) = 0$  covers the error pattern  $e(x)$ . When  $v_{m+1} > 8$  or  $v_{m-1} > 7$ , we have  $x^{n-k}Q_j(x) = x^{15}Q_j(x)$  for  $j = 2, 3$  which is the error pattern in the information section. From Fig. 5.13, we have  $x^{15}Q_2(x) = x^{22}$  for  $Q_2(x) = x^7$  and  $x^{15}Q_3(x) = x^{23}$  for  $Q_3(x) = x^8$ . Thus we see that  $Q_3(x)$  covers the error pattern  $e(x)$ . On the other hand, if  $v_{m+1} \leq 8$  or  $v_{m-1} \leq 7$ , then it follows from  $v_m + v_{m+1} + v_{m-1} = 31$  that  $v_m = 16$ . Thus  $Q_2(x) = x^7$  also covers the remaining error pattern.

Suppose that the received polynomial  $r(x)$  is shifted into the syndrome register from the rightmost stage. Once the entire received word has entered the syndrome register, the syndrome in the syndrome register is the one corresponding to  $r^{(15)}(x)$ , which is the fifteenth cyclic shift of  $r(x)$ . If the errors are all confined to the first 15 high-order positions of  $r(x)$ , the syndrome will lock the errors in these positions.

By dividing  $x^{15}Q_j(x)$  for  $j = 1, 2, 3$  by  $g_1(x)$ , we obtain the following remainders:

$$\gamma_1(x) = 0$$

$$\gamma_2(x) = 1 + x^2 + x^4 + x^5 + x^7 + x^{10}$$

$$\gamma_3(x) = x + x^3 + x^5 + x^6 + x^8 + x^{11}$$

Since the errors in the parity-check section of  $r(x)$  are expressed by  $e_{p_i}(x) = s(x) + \gamma_i(x)$ ,  $i = 1, 2, 3$ , we obtain

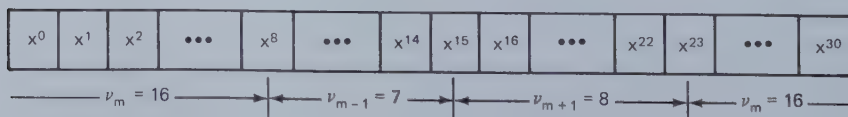


Figure 5.13

$$e_{P_1}(x) = s_0 + s_1x + s_2x^2 + \cdots + s_{14}x^{14}$$

$$e_{P_2}(x) = (s_0 + 1) + s_1x + (s_2 + 1)x^2 + \cdots + (s_{10} + 1)x^{10} + \cdots + s_{14}x^{14}$$

$$e_{P_3}(x) = s_0 + (s_1 + 1)x + s_2x^2 + \cdots + (s_8 + 1)x^8 + \cdots + (s_{11} + 1)x^{11} \\ + \cdots + s_{14}x^{14}$$

Applying the complement relation  $\bar{s}_i = s_i + 1$  (where  $s_i$  is either 1 or 0) to these error equations, we find that the weights of these errors are, respectively,

$$w[e_{P_1}(x)] = s_0 + s_1 + s_2 + \cdots + s_{14} = \sum_{i=0}^{14} s_i \leq 3$$

$$w[e_{P_2}(x)] = \bar{s}_0 + s_1 + \bar{s}_2 + s_3 + \bar{s}_4 + \bar{s}_5 + s_6 + \bar{s}_7 + s_8 \\ + s_9 + \bar{s}_{10} + s_{11} + s_{12} + s_{13} + s_{14} \leq 2$$

$$w[e_{P_3}(x)] = s_0 + \bar{s}_1 + s_2 + \bar{s}_3 + s_4 + \bar{s}_5 + \bar{s}_6 + s_7 + \bar{s}_8 \\ + s_9 + s_{10} + \bar{s}_{11} + s_{12} + s_{13} + s_{14} \leq 2$$

From the theoretical background presented above, the (31, 16) BCH code can be decoded by Kasami's error-trapping technique as shown in Fig. 5.14. The error-trapping procedure of this decoder is described below.

1. Assume that the syndrome and buffer registers are initially empty. If we turn gates 1, 3, and 5 On and gates 2 and 4 Off, then the syndrome of  $r(x)$  is  $s(x) = s_0 + s_1x + s_2x^2 + \cdots + s_{13}x^{13} + s_{14}x^{14}$ , and it can be read into three threshold gates.

2. If gates 1, 4, and 5 are Off and gates 2 and 3 are On, the syndrome is tested for correctable error patterns as follows. If  $w[s(x)] \leq 3$ , all the errors are confined to the 15 high-order positions of  $r(x)$ , and  $s(x)$  locks the errors. The output of the threshold gate  $T_1$  turns gate 4 On and gate 3 Off. Thus, the digits are read out from the error pattern generator  $G_1$  and the buffer register simultaneously, one bit at a time, and modulo-2-added for the error correction. If  $w[s(x)] > 3$ , the weight of  $e_{P_2}(x) = s(x) + \gamma_2(x)$  is tested. If  $w[e_{P_2}(x)] \leq 2$ , then  $e_{P_2}(x) = \bar{s}_0 + s_1x + \bar{s}_2x^2 + s_3x^3 + \cdots + \bar{s}_{10}x^{10} + \cdots + s_{13}x^{13} + s_{14}x^{14}$  is identical to the error pattern in the 15 high-order positions of  $r(x)$  and a single error occurs at the location  $x^7$ . The output of the threshold  $T_2$  turns gate 4 On, and the digits coming out of the buffer are then corrected by the digits coming out of the error pattern generator  $G_2$ . If  $w[s(x)] > 3$  and  $w[s(x) + \gamma_2(x)] > 2$ , the weight of  $e_{P_3}(x) = s(x) + \gamma_3(x)$  is tested. If  $w[e_{P_3}(x)] \leq 2$ , then  $e_{P_3}(x) = s_0 + \bar{s}_1x + s_2x^2 + \cdots + s_{10}x^{10} + \bar{s}_{11}x^{11} + \cdots + s_{14}x^{14}$  is identical to the error pattern in the 15 high-order posi-

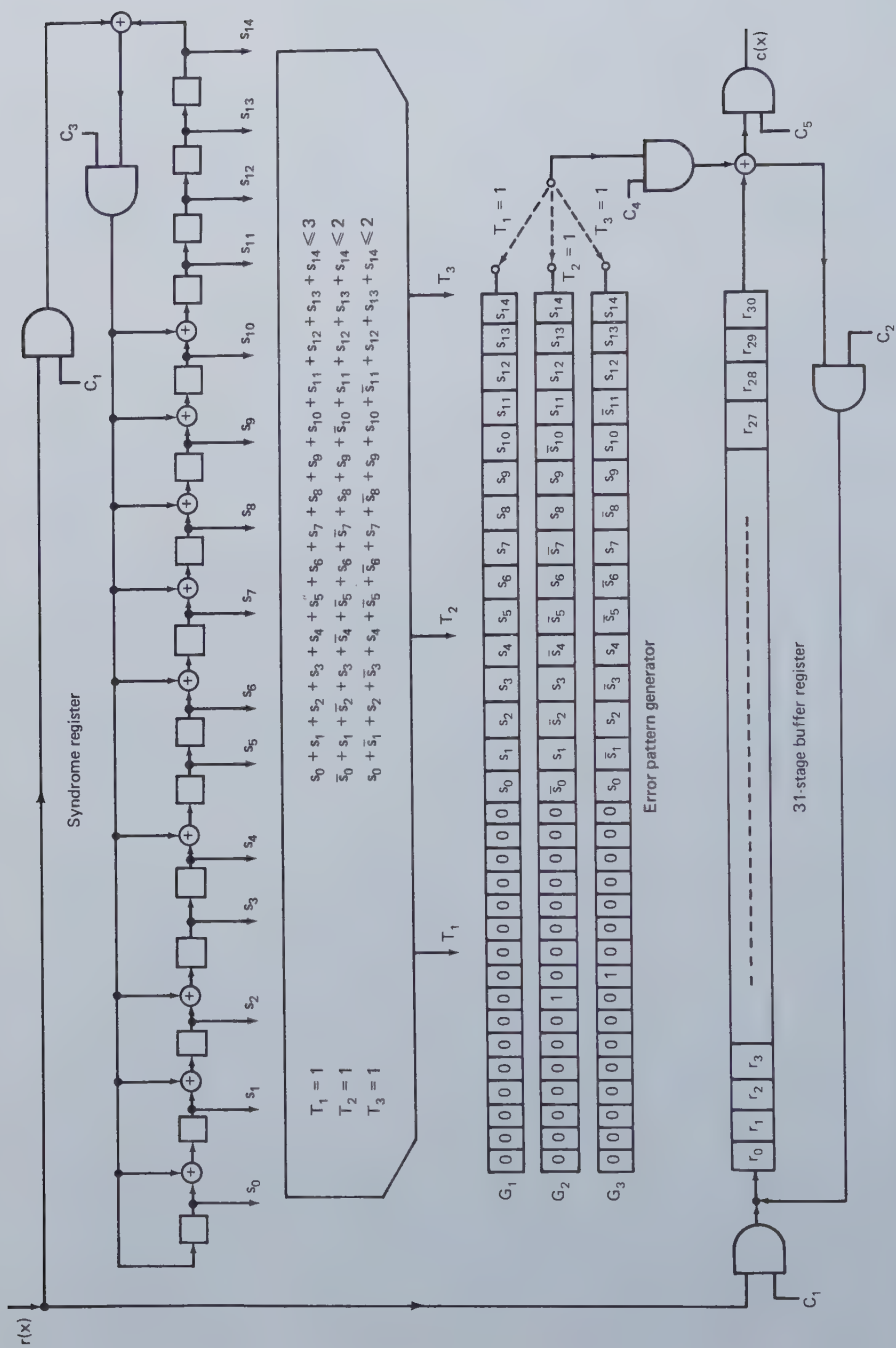


Figure 5.14 Error-trapping decoder for the (31, 16) BCH code.



tions of  $r(x)$  and a single error occurs at the position  $x^8$ . The output of the threshold gate  $T_3$  turns gate 4 On, and the digits coming out of the buffer are corrected by the digits coming out of the error-pattern generator  $G_3$ .

3. If  $w[s(x)] > 3$ ,  $w[e_{P_2}(x)] > 2$ , and  $w[e_{P_3}(x)] > 2$ , then the syndrome and buffer registers are cyclically shifted once with gates 1, 4, and 5 Off and gates 2 and 3 On. The new content of the syndrome register is  $s^{(1)}(x)$ . Step 2 is then repeated.

4. The decoding operation is completed after the buffer register has cyclically shifted 62 times. Gate 5 is then turned On, and the contents of the buffer are read out to the user. However, if there are more than three errors in  $r(x)$ , the digits in the buffer at the end of decoding may not be the transmitted code word  $c(x)$ .

## PROBLEMS

**5.1** Consider a  $(2^m - 1, m)$  cyclic code whose parity polynomial is  $p(x)$  of degree  $3 \leq m < 5$ .

- Find the generator polynomial  $g(x)$  of this code.
- Express the dual code of a  $(2^m - 1, m)$  cyclic code.
- Find the generator polynomial  $p^*(x)$  of this Hamming code.

**5.2** For  $m = 3, 4, 5$ , and 6, list the corresponding cyclic Hamming codes and their respective dual codes.

**5.3** For  $m = 3$ , the  $(7, 4)$  cyclic Hamming code is generated by  $g(x) = 1 + x + x^3$ . This code is capable of correcting any single error. Suppose that the received polynomial  $r(x)$  is shifted into the syndrome register from the left end.

- Determine the seven error patterns and the corresponding syndromes of  $e(x)$ .
- How many of the single-error patterns are identical to their syndromes?
- Devise the decoder for the  $(7, 4)$  cyclic code generated by  $g(x) = 1 + x + x^3$ .
- Suppose that the transmitted code word is  $\mathbf{c} = (1\ 1\ 0\ 0\ 1\ 0\ 1)$  and the received word is  $\mathbf{r} = (1\ 1\ 0\ 1\ 1\ 0\ 1)$ , which implies that a single error occurs at location  $x^3$ . What is the syndrome when the entire received word has been shifted into the syndrome register?

**5.4** Consider again the  $(7, 4)$  cyclic Hamming code generated by  $g(x) = 1 + x + x^3$ . This time suppose that  $\mathbf{r}(x)$  is fed into the syndrome register from the right end.

- Complete a table of all the error patterns versus the syndromes of  $e(x)$ .

- b* What is the syndrome when a single error occurs at the location  $e(x) = x^6$ ?
- c* Devise the decoding circuit for this code.

**5.5** For  $m = 5$ , there exists the (31, 26) cyclic Hamming code generated by  $g(x) = 1 + x^2 + x^5$ . Devise the decoder for this cyclic code assuming that the received word  $r(x)$  is fed into the syndrome register from the right end.

**5.6** Consider again the (31, 26) cyclic Hamming code generated by  $g(x) = 1 + x^2 + x^5$ . By deleting  $i = 3$  digits from this code, we obtain the (28, 23) shortened cyclic code.

- a* Determine the remainder polynomial  $\gamma(x)$  that results from dividing  $x^{n-k+3}$  by  $g(x)$ .
- b* Modify the syndrome register in the decoding circuit shown in Prob. 5.5 by feeding the product  $\gamma(x)r(x)$  into the dividend circuit of  $g(x)$ , and then devise a decoder for the (28, 23) shortened cyclic code generated by  $g(x) = 1 + x^2 + x^5$ .
- c* Devise the decoder for the (28, 23) shortened code when the received polynomial  $r(x)$  is fed into the syndrome register from the right end. Assume that a single error has occurred at location  $x^{27}$ .

## Burst-Error Correction for Block Codes

In this chapter we deal almost exclusively with block codes designed to correct burst errors. In memoryless channels, random noise affects each transmitted digit independently, whereas in certain memory channels, the errors tend to occur in bursts. When the burst noise is particularly severe over the unreliable digits, a burst of symbol erasures results. Therefore, it is desirable to design codes specifically for correcting burst errors.

In certain digital communications media, such as magnetic tape, magnetic disks, and magnetic memories in computer storage systems, we can observe that errors occur in the form of shorter or longer bursts. This same kind of phenomenon occurs as a result of telephone line surges, intentional jamming of spread-spectrum systems, and fading effects in microwave and radio links.

In a burst-error environment it is often desirable to use a random-error-correcting code with an interleaver-deinterleaver. An interleaver is a device that rearranges the ordering of a sequence of digits in a deterministic manner. A deinterleaver is a device that, through inverse permutation, restores the bit sequence to its original order. Thus, this approach involves scrambling the coded sequence prior to transmission, followed by unscrambling the sequence after reception in order to randomize the burst-error process. The two principal classes of interleavers are periodic and pseudorandom.

Cyclic codes, e.g., Fire codes, can be used to correct long error bursts and can be decoded using the error-trapping technique. Reed-Solomon codes, discussed in Chap. 7, can be used to correct multiple short bursts or they can be interleaved by digits to correct multiple long bursts. Threshold decoding of burst-error-correcting convolutional

codes, presented in Chap. 11, allows long bursts to be corrected through the use of guard space. Another type of error-burst problem occurs in gaussian noise channels that are corrupted by large bursts of interference, which can occur in intentional jamming environments. A related problem is coding for spread-spectrum systems.

## 6.1 Prerequisite Theorems

In this section we consider cyclic codes which are able to correct a single burst of errors and present simply implemented decoding algorithms for such codes. Unlike random-error-correcting codes, in which the most useful codes have been constructed by analytic techniques, the best burst-error-correcting codes have been found by computer-aided search procedures. However, before discussing these burst-error-correcting codes, it is worthwhile to consider the following theorems. Generally, a burst of length  $b$  is defined as nonzero components of a coded sequence which are in consecutive bit positions, the first and last of which are nonzero.

**Theorem 6.1** An  $(n, k)$  linear block code that has no burst of length  $b$  or less as a code word must have at least  $b$  parity-check bits.

**Proof** Consider the code words whose nonzero components are confined to the first  $b$  digits. Then the number of code words that are 0 everywhere except in their first  $b$  digits is  $2^b$ . No two such code words can be in the same coset of a standard array. Otherwise, their sum, which must be a burst of length  $b$  or less, would be a code word. Therefore, there must be at least  $2^b$  distinct cosets. However, since there are a total of  $2^{n-k}$  cosets for an  $(n, k)$  linear code,

$$2^{n-k} \geq 2^b \quad \text{or} \quad n - k \geq b \quad (6.1)$$

Thus the number of parity-check bits  $n - k$  must be at least equal to the burst length  $b$ .

**Theorem 6.2** Suppose that an  $(n, k)$  linear code is able to correct all burst errors of length  $b$  or less. Then there exists no such burst of length  $2b$  or less as a code word.

**Proof** Assume that there exists a burst  $\mathbf{u}$  of length  $2b$  or less as a code vector. If this code vector  $\mathbf{u}$  is expressed as a vector sum of two bursts  $\mathbf{v}$  and  $\mathbf{w}$  of length  $b$  or less, then  $\mathbf{v}$  and  $\mathbf{w}$  must be in the same coset of a standard array for this code. If  $\mathbf{v}$  is chosen as a coset leader,  $\mathbf{w}$  will be an uncorrectable error pattern. As a result, this code may not be able to correct all burst errors of length  $b$  or less. This contradicts our assumption. Therefore, in order to correct all bursts of length  $b$  or less, no burst of length  $2b$  or less can be a code vector.

**Theorem 6.3 (Reiger, 1960; Peterson, 1961)** The number of parity-check digits of a  $b$ -burst-error-correcting code must be at least  $2b$ . In order to correct all burst errors of length  $b$  or less and to simultaneously detect all bursts of length  $l$  or less, where  $l \geq b$ , the number of parity-check digits of this code must be at least  $b + l$ .

**Proof** Suppose that an  $(n, k)$  linear code corrects all bursts of length  $b$  or less. Then there is no code word that has a burst of length  $2b$ . Now if two vectors are in the same coset, their difference must be a code word. Let us choose any two vectors that each have a burst of length  $2b$ . If they are in the same coset of a standard array, then their difference must be a code word. However, since it is a burst of length  $2b$ , no such code word exists. Therefore two such vectors must be in different cosets, and the number of cosets is at least as large as the number of such vectors. Thus, in order to correct all burst errors of length  $b$  or less, an  $(n, k)$  linear code must have at least  $2b$  parity-check digits, that is,

$$n - k \geq 2b \quad \text{or} \quad b \leq (n - k)/2 \quad (6.2)$$

Equation (6.2) represents the upper bound on the burst-error-correcting capability  $b$  of an  $(n, k)$  code, which is at most  $\lfloor (n - k)/2 \rfloor$ .

Next, every burst of length  $b + l$  or less can be written as the vector sum of a burst of length  $b$  or less and a burst of length  $l$  or less. If the code is simultaneously to correct bursts of length  $b$  or less and to detect all bursts of length  $l$  or less, then the burst of length  $b$  and the burst of length  $l$  must be in different cosets, and their sum must not be a code word. Since the number of cosets is at least as large as the number of burst-error patterns of length  $b + l$  or less, it follows that

$$2^{n-k} \geq 2^{b+l} \quad \text{or} \quad n - k \geq b + l \quad (6.3)$$

Thus, the number of parity-check digits of this code must be at least  $b + l$ .

**Theorem 6.4** The number of parity-check digits in any linear  $(n, k)$  block code that corrects all bursts of length  $b$  or less is bounded by the following inequality:

$$n - k \geq b - 1 + \log_2(n - b + 2)$$

**Proof** It is clear from Table 6.1 that different burst-error patterns of length  $b$  can be expressed by  $2^{b-2}(n - b + 1)$ .

TABLE 6.1

| Length   | Number of different bursts |
|----------|----------------------------|
| 1        | $n$                        |
| 2        | $n - 1$                    |
| 3        | $2(n - 2)$                 |
| 4        | $2^2(n - 3)$               |
| $\vdots$ | $\vdots$                   |
| $i$      | $2^{i-2}(n - i + 1)$       |
| $\vdots$ | $\vdots$                   |
| $b$      | $2^{b-2}(n - b + 1)$       |

The total number  $L$  of different bursts, including the error pattern of all 0s, is

$$\begin{aligned}
 L &= 1 + n + (n - 1) + 2(n - 2) + \cdots + 2^{b-2}(n - b + 1) \\
 &= 1 + n + n(1 + 2 + 2^2 + \cdots + 2^{b-2}) \\
 &\quad - [1 + 2 \cdot 2 + 3 \cdot 2^2 + \cdots + (b - 1) \cdot 2^{b-2}]
 \end{aligned}$$



Since  $1 + t + t^2 + \cdots + t^{n-1} = (t^n - 1)/(t - 1) \mid t \mid > 1$

and  $1 + 2t + 3t^2 + \cdots + nt^{n-1} = [nt^{n+1} - (n+1)t^n + 1]/(t-1)^2$

$L$  becomes

$$\begin{aligned} L &= 1 + n + n(2^{b-1} - 1) - [(b-1)2^b - b2^{b-1} + 1] \\ &= 2^{b-1}(n - b + 2) \quad b \geq 2 \end{aligned} \quad (6.4)$$

Since the number of cosets in a standard array is at least equal to this total number  $L$ , we get the following bound:

$$2^{n-k} \geq 2^{b-1}(n - b + 2)$$

$$\text{or} \quad n - k \geq b - 1 + \log_2(n - b + 2) \quad (6.5)$$

Equation (6.5) provides a lower bound on the number of parity checks necessary for a code to have a desired error-correcting capability. In a shortened cyclic code, the error patterns that consist of a burst at each end of the  $n$ -tuple are not regarded as bursts.

A bound analogous to the Varshanov-Gilbert-Sacks bound can be derived for burst-error-correcting codes. This bound states that it is possible to construct a  $b$ -burst-error-correcting code for all  $b$  such that

$$n - k \geq 2(b - 1) + \log_2(n - 2b + 2) \quad (6.6)$$

**Theorem 6.5 (Melas and Gorog, 1963)** Let  $g'(x)$  denote the generator polynomial of an  $(n', k')$  cyclic code with burst-error-correcting capability  $b$ . Let  $p(x)$  be an irreducible polynomial of degree  $b$ , and the period  $e$  of  $p(x)$  be the smallest integer such that  $p(x)$  divides  $x^e + 1$ . Then the cyclic code generated by  $g(x) = p(x)g'(x)$  is capable of correcting all bursts of length  $b$  or less and the length of this code is the least common multiple of  $e$  and  $n'$ , that is,  $n = \text{LCM}(e, n')$ .

**Proof** In order for the code generated by  $g(x)$  to be capable of correcting all bursts of length  $b$  or less, no two such bursts are allowed to appear in the same coset and thus add to a code word. If  $x^i B_1(x)$  and  $B_2(x)$  denote two bursts of length not greater than  $b$  and they add to a code word, then we get

$$x^i B_1(x) + B_2(x) = q_1(x)g(x) = q_1(x)p(x)g'(x) \quad (6.7)$$

Since  $g'(x)$  generates a cyclic code of length  $n'$ , it is a factor of  $x^{n'} + 1$ . It is also true that  $x^{n'} + 1$  divides  $x^{vn'} + 1$  for all  $v$ . Hence it follows that

$$x^i B_1(x) = x^{i-vn'} B_1(x) x^{vn'} = x^{i-vn'} B_1(x) + q_2(x)g'(x) \quad (6.8)$$

By substituting Eq. (6.8) into Eq. (6.7), we get

$$\begin{aligned} x^{i-vn'} B_1(x) + B_2(x) &= [q_1(x)p(x) + q_2(x)]g'(x) \\ &= q_3(x)g'(x) \end{aligned} \quad (6.9)$$

In order to make the degree of  $x^{i-vn'}B_1(x)$  less than  $n'$ , we must choose the range of  $v$  so that  $0 \leq (i + b - n')/n' \leq v$ . This means that the sum of two bursts of length  $b$  or less produces a code word in the original  $(n', k')$  code generated by  $g'(x)$ . This contradicts our hypothesis that no two code words in the  $(n', k')$  code generated by  $g'(x)$  have the form of Eq. (6.9) unless  $i - vn' = 0$  and  $B_1(x) = B_2(x) = B(x)$ . Dropping the subscripts on the  $B$ 's and substituting this result into Eq. (6.7), we get

$$\begin{aligned}(x^{vn'} + 1)B(x) &= q_1(x)p(x)g'(x) \\ &= q_4(x)g'(x) \\ &= g_5(x)p(x)\end{aligned}\tag{6.10}$$

However, by hypothesis,  $p(x)$  is of degree  $b$ , which is at least one greater than the degree of  $B(x)$ . This implies that  $p(x)$  divides  $x^{vn'} + 1$ . But it is not possible that both  $p(x)$  and  $g'(x)$  can divide  $x^{vn'} + 1$  because  $vn'$  is necessarily less than  $n$  and  $x^n + 1$  is the smallest polynomial of this form divisible by  $g(x) = p(x)g'(x)$ . Thus, if  $g'(x)$  generates a code of length  $n'$  with the  $b$ -burst-correcting capability, then  $g(x)$  generates a code of length  $n = \text{LCM}(e, n')$  with the same burst-correcting capability. If  $g'(x)$  is chosen as  $x^{2b-1} + 1$ , the result is the Fire code because no code words in this code have the form  $x^iB_1(x) + B_2(x)$ ,  $1 \leq i \leq n' - 1$ .

**Theorem 6.6 (Fire, 1959)** The  $(n, n - 2b - m + 1)$  cyclic code generated by  $g(x) = p(x)(x^{2b-1} + 1)$  is capable of correcting all bursts of length  $b$  or less. Here the irreducible polynomial  $p(x)$  has degree  $m \geq b$  and the length of this code is the least common multiple of  $e$  and  $2b - 1$ , i.e.,  $n = \text{LCM}(e, 2b - 1)$ , in which  $e = 2^m - 1$  is an integer called the order of  $p(x)$ .

Given an  $(n, k)$  cyclic code, it is possible to construct an  $(ni, ki)$  cyclic code by interleaving. This can be done by arranging  $i$  code words in the original code into  $i$  rows of a rectangular array and then transmitting them column by column. The resulting code is called an interleaved code. If the original code corrects any single errors, the interleaved code corrects single bursts of length  $i$  or less. The following theorem shows that a long cyclic code of practically any length with maximum burst-correcting capability can be constructed by interleaving a short code.

**Theorem 6.7** If  $g(x)$  generates an  $(n, k)$  cyclic code capable of correcting all bursts of length  $b$  or less, then  $g(x^i)$  generates an  $(ni, ki)$  cyclic code with a burst-error-correcting capability of  $bi$ .

**Proof** Since  $g(x)$  is of degree  $n - k$  and divides  $x^n + 1$ ,  $g(x^i)$  clearly divides  $x^{ni} + 1$ , that is,  $x^{ni} + 1 = q(x)g(x^i)$ . Hence  $g(x^i)$  generates a cyclic code of length  $ni$ . Since the degree of  $g(x^i)$  is  $i$  times the degree of  $g(x)$ , i.e.,  $i(n - k)$ , the code has  $ik$  information digits. All that remains is to show that the code corrects all bursts of length  $bi$ . Let  $h(x^i)$  be the parity-check polynomial of this code. A set of  $n - k$  independent parity-check polynomials  $h(x^i)$ ,  $x^i h(x^i)$ , ...,  $x^{(n-k-1)i} h(x^i)$  check the information digits in positions  $x^{(n-k)i}$ ,  $x^{(n-k+1)i}$ , ...,  $x^{(n-1)i}$ . More specifically, the information digits in the positions corresponding to  $x^{(n-k)i+j}$ ,  $x^{(n-k+1)i+j}$ , ...,  $x^{(n-1)i+j}$  are checked by a set of  $n - k$  independent parity-check

polynomials  $x^j h(x^i)$ ,  $x^{i+j} h(x^i)$ , ...,  $x^{(n-k-1)i+j} h(x^i)$  for  $0 \leq j \leq i-1$ . It is thus clear that a parity check in one of these sets never checks an information digit in another.

In Table 6.2, we consider an  $i \times n$  rectangular array in which each row is an  $n$ -tuple code word in the original  $(n, k)$  code generated by  $g(x)$ .

TABLE 6.2 An  $i \times n$  Rectangular Array

|     |                                                 |            |     |                  |                                                  |     |                |                |
|-----|-------------------------------------------------|------------|-----|------------------|--------------------------------------------------|-----|----------------|----------------|
| —   | $x^0$                                           | $x^i$      | ... | $x^{(n-k-1)i}$   | $x^{(n-k)i}$                                     | ... | $x^{(n-2)i}$   | $x^{(n-1)i}$   |
| ↑   | $x^1$                                           | $x^{i+1}$  | ... | $x^{(n-k-1)i+1}$ | $x^{(n-k)i+1}$                                   | ... | $x^{(n-2)i+1}$ | $x^{(n-1)i+1}$ |
| $i$ | $\vdots$                                        |            |     | $\vdots$         | $\vdots$                                         |     | $\vdots$       | $\vdots$       |
| ↓   | $x^{i-1}$                                       | $x^{2i-1}$ | ... | $x^{(n-k)i-1}$   | $x^{(n-k+1)i-1}$                                 | ... | $x^{(n-1)i-1}$ | $x^{ni-1}$     |
| —   | $\longleftarrow (n-k)$ parity $\longrightarrow$ |            |     |                  | $\longleftarrow k$ information $\longrightarrow$ |     |                |                |

This array is simply constructed by arranging  $i$  code words in the original  $(n, k)$  code into  $i$  rows of an  $i \times n$  array and transmitting them column by column. The resulting code is called an interlaced code. The parameter  $i$  is referred to as the interlacing degree. As you can see, a burst of length  $i$  in any column will affect no more than one digit in each row. Thus, if the original code corrects single errors, the interlaced code corrects single bursts of length  $i$  or less. Similarly, if the original code will correct any combination of  $t$  or fewer errors, the interlaced code will correct any combination of  $t$  bursts of length  $i$  or less. As a result, if the original code corrects any single burst of length  $b$  or less, the interlaced code will correct any single burst of length  $bi$  or less.

**Theorem 6.8 (Burton, 1969)** Let  $p(x)$  be a polynomial of degree  $b$  and order  $e$ , where  $e$  is the smallest possible integer such that  $x^e + 1$  is divisible by  $p(x)$ . Then the  $(n, n-2b)$  code generated by  $g(x) = p(x)(x^b + 1)$  can correct any burst of length  $b$  or less which is confined to positions  $x^{ib}$ ,  $x^{ib+1}$ , ...,  $x^{ib+(b-1)}$  for  $0 \leq i \leq e-1$ . Here the code length is  $n = \text{LCM}(e, b) = eb$ .

**Proof** Let  $B_1(x)$  and  $B_2(x)$  denote two polynomials of degree  $b-1$  or less. If we assume that there exists a code word of the form

$$c(x) = x^{ib}B_1(x) + B_2(x) \quad i = 1, 2, \dots, e-1 \quad (6.11)$$

then we get

$$x^{ib}B_1(x) + B_2(x) = q_1(x)g(x) = q_1(x)p(x)(x^b + 1) \quad (6.12)$$

However, since  $(x^{ib} + 1)B_1(x)$  is divisible by  $x^b + 1$ , we then have

$$(x^{ib} + 1)B_1(x) = q_2(x)(x^b + 1)$$

or

$$x^{ib}B_1(x) = B_1(x) + q_2(x)(x^b + 1) \quad (6.13)$$

Substituting Eq. (6.13) into Eq. (6.12), we obtain

$$B_1(x) + B_2(x) = q_3(x)(x^b + 1) \quad (6.14)$$

where  $q_3(x) = q_2(x) + q_1(x)p(x)$ . Since the degrees of polynomials  $B_1(x)$  and  $B_2(x)$  are both less than  $b$ , from Eq. (6.14) we can see that

$$B_1(x) = B_2(x) \equiv B(x) \quad (6.15)$$

Substituting Eq. (6.15) into (6.12) gives

$$(x^{ib} + 1)B(x) = q_1(x)p(x)(x^b + 1) \quad (6.16)$$

where  $p(x)$  has degree  $b$  and is irreducible. Hence, in order for Eq. (6.16) to hold,  $p(x)$  must divide  $x^{ib} + 1$ ; and obviously  $x^b + 1$  also divides  $x^{ib} + 1$ . However, since  $i < e$ ,  $x^{eb} + 1$  is the polynomial of lowest degree of this form which is divisible by  $(x^b + 1)p(x)$ . Thus, no code word of the form  $x^{ib}B_1(x) + B_2(x)$  exists and every phased burst of length  $b$  or less is correctable.

Burton discovered a class of phased-burst-correcting cyclic codes. Polynomial  $p(x)$  is irreducible and of degree  $b$  and period (or order)  $e$ . The code length  $n$  is the least common multiple of  $b$  and  $e$ . For any positive integer  $b$ , there exists a  $b$ -phased burst-correcting Burton code of length  $n$  which is generated by  $g(x) = (x^b + 1)p(x)$ . The number of parity-check digits of this code is  $2b$ . A burst of length  $i$  or less is called a phased burst if and only if it is confined to  $i$  consecutive subblocks for  $i < e$ . The decoding of a Burton code can also be accomplished using the error-trapping technique.

## 6.2 Single-Burst Correction by Fire Codes

The first class of cyclic codes, which is constructed systematically for correcting a single burst of errors, is the Fire codes. Fire codes can be decoded by the error-trapping technique. It has been shown that a primitive polynomial  $p(x)$  of degree  $m$  is a factor of  $x^e + 1$ , where  $e = 2^m - 1$ . This positive integer  $e$  is called the period of  $p(x)$  because  $e$  is the order of the root  $\alpha$  of  $p(x)$  because  $\alpha^e = \alpha^{2^m - 1} = 1$ . Consider an  $(n, n - 2b - m + 1)$  Fire code that is generated by  $g(x) = p(x)(x^{2b-1} + 1)$  and that is capable of correcting any single error burst of length  $b$  or less. Here  $b \leq m$ , and  $2b - 1$  is not divisible by  $e$ . The code length  $n$  is the least common multiple of  $2b - 1$  and  $e$ , such that  $n = LCM(e, 2b - 1)$ .

**Example 6.1** If  $p(x) = 1 + x + x^3$  is chosen, then the period of  $p(x)$  is  $e = 2^m - 1 = 7$ . Let the burst length be assigned as  $b = 3$ . Thus the generator polynomial of this Fire code can be written as

$$\begin{aligned} g(x) &= (1 + x + x^3)(1 + x^5) \\ &= 1 + x + x^3 + x^5 + x^6 + x^8 \end{aligned}$$

The length of this code is  $n = LCM(2^m - 1, 2b - 1) = LCM(7, 5) = 35$ . Since the number of parity-check digits is  $n - k = m + 2b - 1 = 8$ , the  $(35, 27)$  cyclic Fire code, which is capable of correcting any single error burst of length 3 or less, is formed. Now, suppose that an error burst of length 3 or less has occurred start-

ing at location  $x^i$  and ending at location  $x^{i+\lambda}$  where  $\lambda \leq 2$ . Then the error burst of length  $b$  or less has the form:  $e(x) = x^i B(x) = x^i(1 + b_1x + b_2x^2)$ , where  $B(x)$  is the burst pattern. Once the burst pattern  $B(x)$  and the location exponent  $i$  are determined, the corrected code word  $c(x)$  is found by adding the error burst  $x^i B(x)$  to the received code polynomial  $r(x)$  such that

$$c(x) = r(x) + x^i B(x) \quad (6.17)$$

The error-trapping decoder for the (35, 27) Fire code is shown in Fig. 6.1.

Let  $r(x) = x^{16} + x^{18}$  be the received polynomial when the all-zero code word  $c(x) = 0$  or  $c = (0\ 0\ 0 \cdots 0\ 0)$  is transmitted. Then there exists only the transmission error whose form is  $e(x) = x^{16} + x^{18}$ . As seen from Fig. 6.1,  $r(x)$  is shifted into the syndrome register from the rightmost stage for implementation purposes. This is equivalent to preshifting the received word  $r(x) = e(x)$  by  $x^8$ . The content in the syndrome register is the syndrome of the error vector  $e^{(8)}(x)$  which is obtained by cyclically shifting  $e(x)$  8 times to the right. Thus, from the following relation:

$$x^{n-k}[x^i B(x)] = e^{(n-k)}(x) = q(x)g(x) + s(x) \quad (6.18)$$

we get  $e^{(8)}(x) = x^8[x^{16}(1 + x^2)] = q(x)(1 + x + x^3 + x^5 + x^6 + x^8) + s(x)$ , from which we obtain the syndrome  $s(x) = x + x^4$ . Therefore we see that the corresponding content in the syndrome register is  $s = (0\ 1\ 0\ 0\ 1\ 0\ 0\ 0)$ . As a result, the burst is not contained in the upper three stages of the syndrome register and

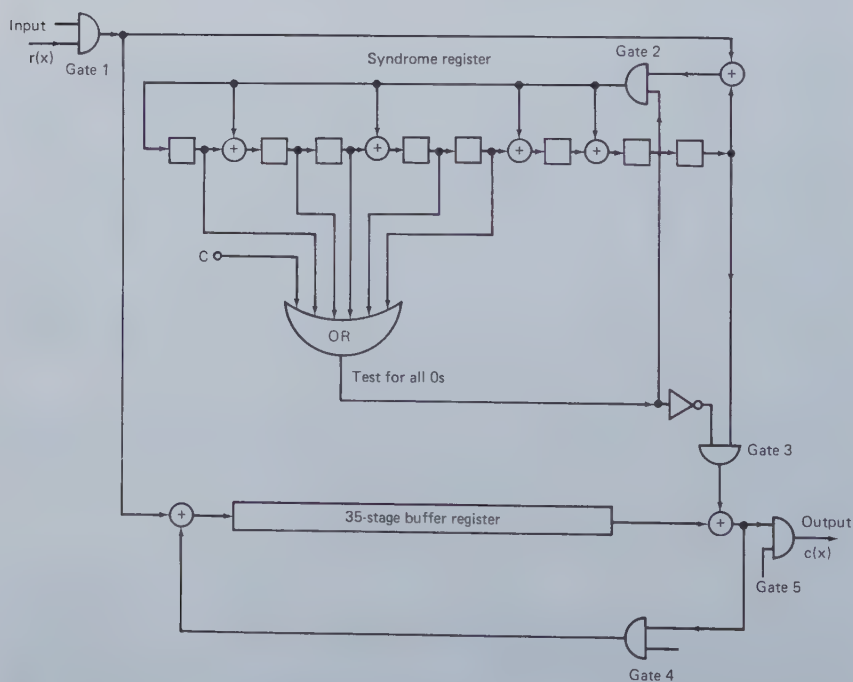


Figure 6.1 Error-trapping decoder for the (35, 27) Fire code.



the lower five stages do not contain all 0s. In order for the burst pattern  $B(x)$  to be trapped in the upper three stages,  $r(x)$  must be cyclically shifted simultaneously in both the buffer register and the syndrome register and gates 2 and 4 must be turned On and gates 1, 3, and 5 turned Off. Eventually, after a certain number of shifts, say  $i = 16$  in this case, the single burst of errors  $x^{16}$ ,  $x^{17}$ , and  $x^{18}$  of  $r(x)$  will be moved to the highest-order positions in the buffer register. As a result, the burst is trapped in the upper three stages of the syndrome register in the form (0 0 0 0 1 0 1). Table 6.3 shows that the contents of the syndrome register will change with each step after cyclic shifting. This event is detected when the lower five stages of the syndrome register contains only 0s.

TABLE 6.3 Syndrome Contents by Cyclic Shift

| Syndrome<br>$s^{(i)}(x)$ | Syndrome polynomial               | Syndrome vector |       |       |       |       |       |       |       |
|--------------------------|-----------------------------------|-----------------|-------|-------|-------|-------|-------|-------|-------|
|                          |                                   | $s_0$           | $s_1$ | $s_2$ | $s_3$ | $s_4$ | $s_5$ | $s_6$ | $s_7$ |
| $s(x)$                   | $x + x^4$                         | 0               | 1     | 0     | 0     | 1     | 0     | 0     | 0     |
| $s^{(1)}(x)$             | $x^2 + x^5$                       | 0               | 0     | 1     | 0     | 0     | 1     | 0     | 0     |
| $s^{(2)}(x)$             | $x^3 + x^6$                       | 0               | 0     | 0     | 1     | 0     | 0     | 1     | 0     |
| $s^{(3)}(x)$             | $x^4 + x^7$                       | 0               | 0     | 0     | 0     | 1     | 0     | 0     | 1     |
| $s^{(4)}(x)$             | $1 + x + x^3 + x^6 + x^7$         | 1               | 1     | 0     | 1     | 0     | 0     | 1     | 0     |
| $s^{(5)}(x)$             | $x + x^2 + x^4 + x^6 + x^7$       | 0               | 1     | 1     | 0     | 1     | 0     | 0     | 1     |
| $s^{(6)}(x)$             | $1 + x + x^2 + x^6$               | 1               | 1     | 1     | 0     | 0     | 0     | 1     | 0     |
| $s^{(7)}(x)$             | $x + x^2 + x^3 + x^7$             | 0               | 1     | 1     | 1     | 0     | 0     | 0     | 1     |
| $s^{(8)}(x)$             | $1 + x + x^2 + x^4 + x^5 + x^6$   | 1               | 1     | 1     | 0     | 1     | 1     | 1     | 0     |
| $s^{(9)}(x)$             | $x + x^2 + x^3 + x^5 + x^6 + x^7$ | 0               | 1     | 1     | 1     | 0     | 1     | 1     | 1     |
| $s^{(10)}(x)$            | $1 + x + x^2 + x^4 + x^5 + x^7$   | 1               | 1     | 1     | 0     | 1     | 1     | 0     | 1     |
| $s^{(11)}(x)$            | $1 + x^2$                         | 1               | 0     | 1     | 0     | 0     | 0     | 0     | 0     |
| $s^{(12)}(x)$            | $x + x^3$                         | 0               | 1     | 0     | 1     | 0     | 0     | 0     | 0     |
| $s^{(13)}(x)$            | $x^2 + x^4$                       | 0               | 0     | 1     | 0     | 1     | 0     | 0     | 0     |
| $s^{(14)}(x)$            | $x^3 + x^5$                       | 0               | 0     | 0     | 1     | 0     | 1     | 0     | 0     |
| $s^{(15)}(x)$            | $x^4 + x^6$                       | 0               | 0     | 0     | 0     | 1     | 0     | 1     | 0     |
| $s^{(16)}(x)$            | $x^5 + x^7$                       | 0               | 0     | 0     | 0     | 0     | 1     | 0     | 1     |

When this event occurs, gate 2 is automatically turned Off and gate 3 is automatically turned On. The burst is then shifted out of the syndrome register and is added to the received word coming out from the buffer register for error correction. Thus the decoding operation is completed as soon as the buffer register has been cyclically shifted 35 times. Gate 5 is then turned On, and the corrected code word is sent to the data sink.

**Example 6.2** Let  $p(x)$  be an irreducible polynomial of degree 4 whose order is  $e = 15$ . Consider a decoder with the burst-error-correcting capability  $b = 4$ . Then the polynomial

$$g(x) = (1 + x + x^4)(1 + x^7) = 1 + x + x^4 + x^7 + x^8 + x^{11}$$

generates a binary  $(n, n - 2b - m + 1)$  Fire code of length  $n = 7(2^4 - 1) = 105$  which corrects any single burst of length 4 or less. It has  $2b + m - 1 = 11$  parity-check digits and  $105 - 11 = 94$  information digits. The error-trapping decoder for this (105, 94) Fire code is shown in Fig. 6.2.

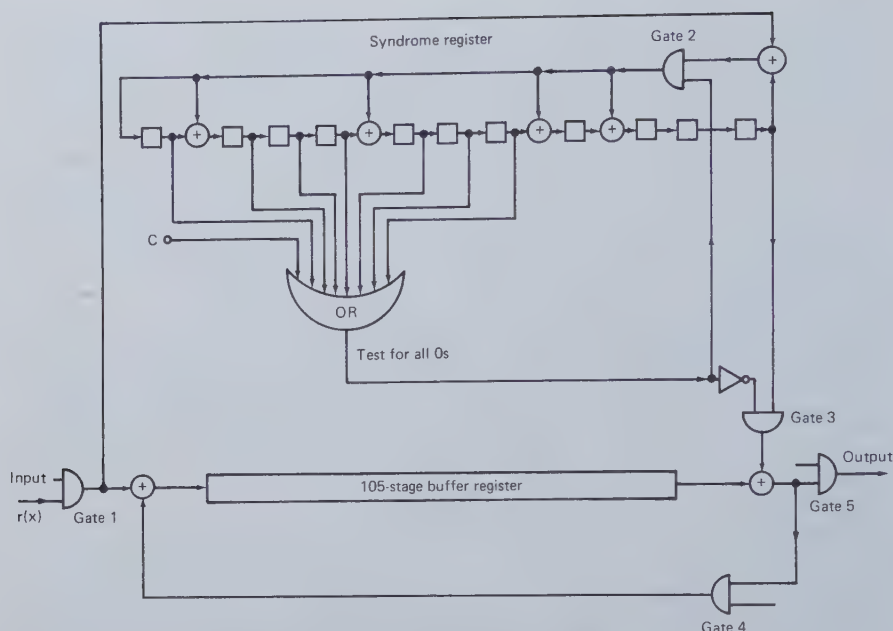


Figure 6.2 Error-trapping decoder for the (105, 94) Fire code.

When  $\lambda \leq 3$ , the error burst of length 4 or less has the form of  $e(x) = x^i B(x)$ , where  $B(x)$  is the burst pattern and  $x^i$  denotes the starting position of the error burst. The received polynomial  $r(x)$  is shifted into the syndrome register from the rightmost stage with gates 1 and 2 turned On and gates 4 and 5 turned Off. Then the content in the syndrome register corresponds to the syndrome of the vector  $r^{(11)}(x)$ , which is  $r(x)$  shifted 11 times. When the error burst occupies the upper four positions of  $r(x)$ , the error burst can be expressed as  $e(x) = x^{101} B(x)$ , where  $B(x) = 1 + b_1 x + b_2 x^2 + b_3 x^3$  is the burst-error pattern. If the syndrome and buffer registers are simultaneously shifted 11 times, then the burst positions  $x^{101}, x^{102}, x^{103}$ , and  $x^{104}$  in the buffer register will move to positions  $x^7, x^8, x^9$ , and  $x^{10}$  of  $r^{(11)}(x)$  or  $e^{(11)}(x) = x^7 B(x)$ . According to Theorem 5.1, the syndrome of  $r(x)$  is identical to the error pattern  $e(x)$  if and only if all the errors are confined to the parity-check section. Accordingly, it is clear that the error burst of length 4 is confined to the  $n - k = 11$  parity-check section. Therefore we have  $s(x) = e^{(11)}(x) = x^7 B(x) = x^7 + b_1 x^8 + b_2 x^9 + b_3 x^{10}$ . As a result, the burst pattern  $B(x)$  is trapped in the upper four stages of the syndrome register and the lower seven stages contain 0s only. Thus the OR gate detects all 0s and its output is 0. This output of a 0 bit simultaneously turns gate 3 On and gate 2 Off. This operating mechanism creates a path to the binary adder (error corrector) and

disconnects the feedback connection in the syndrome register so that no change can occur in the trapped error burst. When the syndrome and buffer registers are shifted simultaneously, the trapped error burst coming out of the syndrome register is added to the erroneous digits read out from the buffer register and thus the error correction is accomplished.

Had the error burst occurred elsewhere in the received word, it would have been corrected in exactly the same manner. Eventually, after a certain number of shifts, say  $i$ , the burst will occupy the  $b = 4$  high-order positions  $x^{101}, x^{102}, x^{103}$ , and  $x^{104}$  of  $r^{(i)}(x)$  in the buffer register. At this time the  $b = 4$  high-order digits in the syndrome register will be identical to the burst as before.

If a code of suitable natural length  $n$  cannot be found, then a code can be shortened by deleting some of the high-order information digits. Given an  $(n, k)$  cyclic code, it is always possible to form an  $(n - i, k - i)$  linear code by making all the  $i$  leading information digits zero and omitting them from all code words. The resulting code is not a cyclic code because it is no longer true in every case that the vector formed by shifting a code word is also a code vector. Such a code is called a shortened cyclic code, which was fully covered in Chaps. 4 and 5. An  $(n - i, k - i)$  shortened code can correct any burst-error pattern that the  $(n, k)$  original code could correct. Furthermore, since a pseudocyclic code shortened by  $i$  digits can be decoded by the same circuitry used to decode the original  $(n, k)$  code, it is necessary only to prefix the shortened words by  $i$  zeros. However, decoding can be accomplished more quickly if, instead of calculating the syndrome of  $x^{n-k}r(x)$  as would be done for a cyclic code, the syndrome of  $x^{n-k+i}r(x)$  is calculated.

Let  $s(x)$  be the syndrome of  $x^{n-k+i}r(x)$ . Then we have

$$s(x) = x^{n-k+i}r(x) + q(x)g(x) \quad (6.19)$$

If we choose  $\gamma(x)$  as the remainder resulting from dividing  $x^{n-k+i}$  by  $g(x)$ , it follows that

$$\gamma(x) \equiv x^{n-k+i} + q_1(x)g(x) \quad (6.20)$$

If  $\gamma(x)$  is multiplied by  $r(x)$ , Eq. (6.20) becomes

$$\gamma(x)r(x) = x^{n-k+i}r(x) + q_1(x)g(x)r(x) \quad (6.21)$$

Combining Eqs. (6.21) and (6.19), we get

$$\gamma(x)r(x) = s(x) + q_2(x)g(x) \quad (6.22)$$

where  $q_2(x) = q_1(x)r(x) + q(x)$  and the degree of the remainder  $\gamma(x)$  is less than that of  $g(x)$ . It is worthwhile to note here that quite a few of

the best-known random- or burst-error-correcting codes are shortened cyclic codes.

**Example 6.3** Consider a Fire code whose generator polynomial is  $g(x) = (1 + x^2 + x^5)(1 + x^9) = 1 + x^2 + x^5 + x^9 + x^{11} + x^{14}$ . Since the length of this code is  $n = LCM(9, 31) = 279$ , the number of information bits becomes  $k = 279 - 14 = 265$ . Thus we have the (279, 265) Fire code. Let us shorten this Fire code by 10 bits to give a (269, 255) shortened code. Suppose that this code is required to correct any burst of length 5 or less. Using Eq. (6.20), we see that the remainder  $\gamma(x)$  obtained by dividing  $x^{14+10}$  by  $g(x)$  is  $\gamma(x) = 1 + x^2 + x^4 + x^6 + x^9 + x^{11} + x^{13}$ . A decoding circuit, such as the one shown in Fig. 6.3, that multiplies the received 269-tuple word by  $\gamma(x)$  can compute the syndrome and correct an error burst of length 5 by using the error-trapping technique.

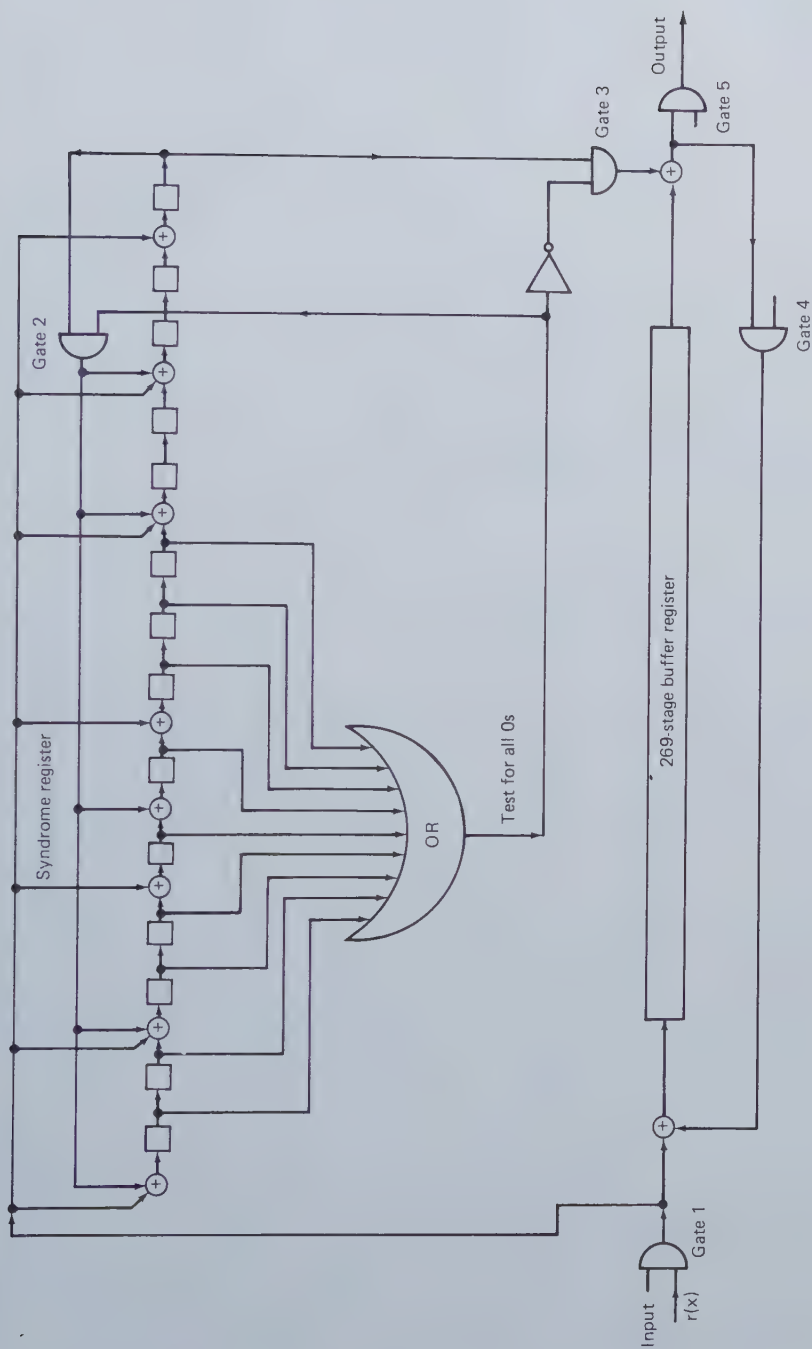
### 6.3 High-Speed Error-Trapping Decoder

Consider a Fire code over  $GF(2)$  which is generated by the following polynomial:

$$\begin{aligned} g(x) &= (1 + x + x^6)(1 + x^{11}) \\ &= 1 + x + x^6 + x^{11} + x^{12} + x^{17} \end{aligned}$$

This code is of length  $n = LCM(63, 11) = 693$  and is capable of correcting any single burst of length  $b \leq 6$ . A conventional decoder for this Fire code consists of (1) a 17-stage syndrome register with  $n - k = 17$  digits, which is the degree of  $g(x)$ , (2) a 693-bit buffer register, and (3) some simple feedback connections and decision logic, and is similar to the decoders in Figs. 6.1, 6.2, and 6.3. This conventional decoder is suitable for most systems where speed is not a prime consideration. However, for many digital systems that operate online with high-speed computers, any significant delay in decoding may become undesirable, and so speed becomes an important factor in coding design. Therefore, the object of this section is to investigate a technique for high-speed decoding of burst-error-correcting codes. Most decoders for burst-error correction are designed to use cyclic shifting and error-pattern detection. In view of the fast decoding required, it is desirable to develop decoding algorithms that would accomplish the decoding task at greater speed with the possible aid of some computation.

A fast error-trapping decoder for a class of Fire codes is now presented below. Let us consider a (693, 676) Fire code with the generator polynomial  $g(x) = (1 + x + x^6)(1 + x^{11})$ . An alternate decoding circuit for this Fire code consists of two feedback shift registers: (1) the error-pattern register based on the factor  $1 + x^{11}$  and (2) the error-tracking register based on  $p(x) = 1 + x + x^6$ . However, if no computation facility is provided for this alternate decoder, its speed will be the same as



**Figure 6.3** Error-trapping decoder for the (269, 255) shortened Fire code.



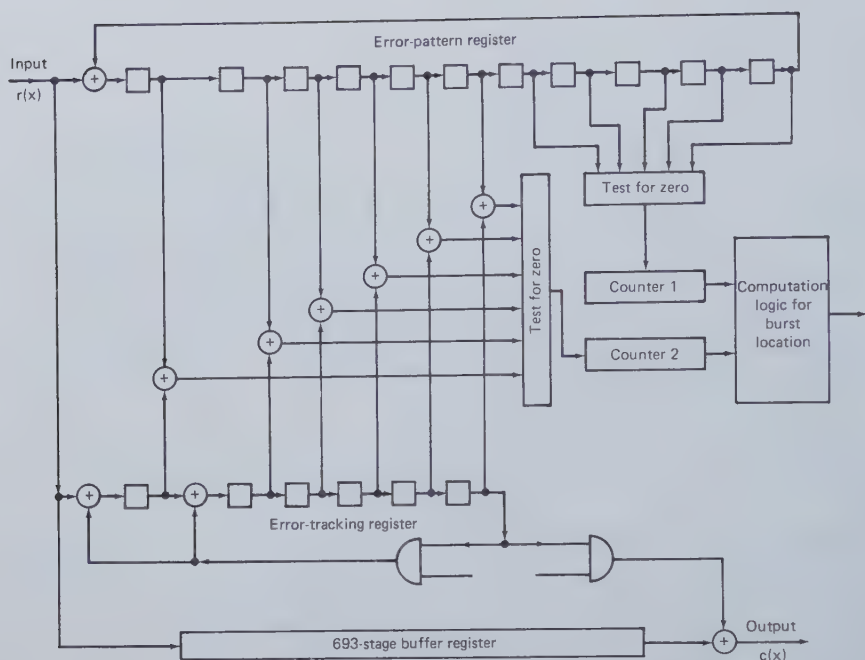


Figure 6.4 High-speed error-trapping decoder for the (693, 676) Fire code.

the conventional decoder discussed in the previous examples. The two shift registers are run in synchrony for syndrome computation and error detection. An alternate fast decoder for a (693, 676) Fire code is shown in Fig. 6.4. Let  $r(x)$  be the received polynomial. The syndromes are the remainders that result from dividing  $r(x)$  by the factors of  $g(x)$ , i.e.,  $1 + x + x^6$  and  $1 + x^{11}$ , respectively. The error pattern is identified when the contents of both registers are exactly the same. If an error burst of length  $b \leq 6$  has occurred, then the error pattern can be expressed as  $e(x) = x^i B(x)$ . In order for the error pattern to shift out from the buffer register, it must be cyclically shifted  $(n - 1) - (i + 5) = n - i - 6$  times. The syndromes calculated in the shift registers for the factors  $1 + x + x^6$  and  $1 + x^{11}$  are, respectively,

$$x^{i+6}B(x) = q_1(x)(1 + x^{11}) + s_c(x) \quad (6.23)$$

$$x^{i+6}B(x) = q_2(x)(1 + x + x^6) + s_p(x) \quad (6.24)$$

After  $n - i - 6$  cyclic shifts,  $e(x)$  becomes

$$e^{(n-i-6)}(x) = x^{i+6}B(x)x^{n-i-6} = B(x) \quad \text{modulo } (x^n + 1) \quad (6.25)$$

Thus the contents of the shift registers become, respectively,

$$B(x) = q_1(x)(1 + x^{11}) + s_c^{(n-i-6)}(x) \quad (6.26)$$

or 
$$B(x) = s_c^{(n-i-6)}(x) \quad \text{modulo } (1 + x^{11})$$

$$B(x) = q_2(x)(1 + x + x^6) + s_p^{(n-i-6)}(x) \quad (6.27)$$

or 
$$B(x) = s_p^{(n-i-6)}(x) \quad \text{modulo } (1 + x + x^6)$$

from which we obtain

$$s_c^{(n-i-6)}(x) = s_p^{(n-i-6)}(x) = B(x) \quad (6.28)$$

When the syndrome-error relationship is expressed in this form, it means that the burst-pattern  $B(x)$  is trapped in the  $b$  low-order stages of the error-pattern register and that the  $b - 1$  high-order stages of the error-pattern register contain all 0s by the time the register has been shifted  $n - i - 6$  times. Thus we see that the contents in the error-tracking register lock the burst pattern  $B(x)$  in the  $b$  low-order positions of the error-pattern register.

At this point, let us introduce the Chinese remainder theorem, which is useful for determining the error-starting number  $i$ . A positive integer  $p$  that is larger than 1 and that is only divisible by  $\pm p$  or  $\pm 1$  is called a prime integer. Two integers are said to be relatively prime if their greatest common divisor is 1. The greatest common divisor of two integers  $r$  and  $s$ , denoted by  $GCD(r, s)$ , is the largest positive integer that divides both of them.

**Theorem 6.9 (Chinese remainder theorem for integers)** Suppose that  $e$  and  $c$  are relatively prime and  $\gamma_c$  and  $\gamma_p$  are some integers. Then the simultaneous congruences

$$\begin{aligned} i &\equiv \gamma_c \quad \text{modulo } c \\ i &\equiv \gamma_p \quad \text{modulo } e \end{aligned} \quad (6.29)$$

have a unique solution for  $i$  such that

$$i = A_c e \gamma_c + A_p c \gamma_p \quad \text{modulo } n \quad (6.30)$$

where  $A_c$  and  $A_p$  are integers that satisfy the following  $GCD$  equation:

$$A_c e + A_p c = 1 \quad \text{modulo } n \quad (6.31)$$

**Proof** Since  $e$  and  $c$  are relatively prime, the greatest common divisor of these numbers is 1. Starting with the initial pair of numbers  $e$  and  $c$ , we just apply the euclidean division algorithm sequentially, and the last nonzero remainder is a greatest common divisor. That is,

$$\begin{aligned} e &= cq_1 + \gamma_1 \\ c &= \gamma_1 q_2 + \gamma_2 \end{aligned}$$

$$\begin{aligned}
\gamma_1 &= \gamma_2 q_3 + \gamma_3 \\
&\vdots \quad \quad \quad \vdots \\
\gamma_{n-3} &= \gamma_{n-2} q_{n-1} + \gamma_{n-1} \\
\gamma_{n-2} &= \gamma_{n-1} q_n \quad (\text{stop when } \gamma_n = 0)
\end{aligned}$$

where the remainders  $\gamma_i$ ,  $1 \leq i \leq n-1$  are in the range of  $0 \leq g_i \leq c$ . Since the set of remainders  $(\gamma_1, \gamma_2, \dots, \gamma_{n-1})$  forms a decreasing sequence of positive integers, the division algorithm must eventually terminate when  $\gamma_n = 0$  for some  $n$ .

From this division set, the residues  $\gamma_i$  can be written as

$$\begin{aligned}
\gamma_1 &= e + (-q_1)c \\
\gamma_2 &= (-q_2)e + (1 + q_1 q_2)c \\
\gamma_3 &= (1 + q_2 q_3)e + [(-q_1) - (1 + q_1 q_2)q_3]c \\
&\vdots \quad \quad \quad \vdots
\end{aligned}$$

If we continue this process repeatedly, then the greatest common divisor of  $e$  and  $c$ , i.e.,  $GCD(e, c) = 1$ , is derived by satisfying the following equation:

$$A_c e + A_p c = 1 \quad \text{modulo } n$$

where  $A_c$  and  $A_p$  are some integer coefficients that appear in  $\gamma_{n-1}$ . Thus it immediately follows that  $A_c e = 1$  and  $A_p c = 1$ . By putting  $i = A_c e \gamma_c + A_p c \gamma_p$ , we get

$$\begin{aligned}
i &\equiv A_c e \gamma_c \equiv \gamma_c \quad \text{modulo } c \\
i &\equiv A_p c \gamma_p \equiv \gamma_p \quad \text{modulo } e
\end{aligned}$$

which shows that Eq. (6.29) has been well proved.

Now consider the implementation of the fast error-trapping decoder for a (693, 676) Fire code with  $g(x) = (1 + x + x^6)(1 + x^{11})$  as shown in Fig. 6.4. The improvement in decoding speed is possible only when the decoder has some computation capability or facility that is available to the decoder. But this decoding circuit is clearly more complex than a conventional decoding circuit because a fast error-trapping decoder requires more logic. However, since speed and simplicity cannot be achieved at the same time, some trade-off must be made. The decoding operation of Fig. 6.4 proceeds as follows.

The received polynomial  $r(x)$  is first read into the two syndrome registers and the buffer register. As soon as all of  $r(x)$  has been shifted into the two registers, the remainders  $s_c(x)$  and  $s_p(x)$  are formed. The decoder starts to test  $s_c(x)$  and  $s_p(x)$ . If  $s_c(x) = s_p(x) = 0$ , the received  $r(x)$  is assumed to be free of error and is then delivered to the data sink. If  $s_c(x) = 0$  and  $s_p(x) \neq 0$  or  $s_c(x) \neq 0$  and  $s_p(x) = 0$ , then  $r(x)$  contains a detectable but uncorrectable error burst and is therefore discarded. If  $s_c(x) \neq 0$  and  $s_p(x) \neq 0$ , then  $r(x)$  is assumed to contain a cor-

rectable error burst and the decoder starts the error-correction process, which is carried out as follows. The error pattern is present in the error-pattern register that corresponds to  $1 + x^{11}$ . The error-pattern register is shifted cyclically until the burst moves to the lowest possible position so that the higher-order positions all contain 0s. The error burst is then trapped in the  $b$  low-order positions. The number of shifts performed in counter 1 is  $c - \gamma_c \bmod c = 11$ , from which we see that the error-starting number is identical to  $i = \gamma_c \bmod c$ . If the  $b - 1$  or  $c - b$  high-order positions of the error-pattern register never contain all 0s by the time the register has been shifted  $2(c - b)$  times, an uncorrectable error burst has been detected. In this situation, the decoder stops the error-correction process. Next, the error-tracking register which corresponds to  $1 + x + x^6$  is shifted until its contents lock the burst pattern in the  $b$  low-order position of the error-pattern register. Then we can easily count  $e - \gamma_p \bmod e = 63$  in counter 2 such that  $i = \gamma_p \bmod 63$ . Once  $\gamma_c$  and  $\gamma_p$  are determined from the Chinese remainder theorem, the two syndromes uniquely determine the error-starting number  $i$ . Hence, the precise location of the error pattern is given. Since the error burst begins at position  $x^i$  and ends at position  $x^{i+b-1}$  of  $r(x)$ , the burst begins at position  $x^0$  and ends at  $x^{b-1}$  of  $r(x)$  in the case of  $i = 0$ . Finally, adding  $x^i B(x)$  to  $r(x)$  in the buffer register terminates the error-correction process.

The error-trapping decoder for the Fire code described above requires at most  $2b + e - 3 = 72$  shifts of the two syndrome registers to carry out the error-correction process, while the conventional error-trapping decoder requires  $n = \text{LCM}(e, 2b - 1) = 693$  shifts. Since  $n$  is much greater than  $2b + e - 3$ , it can be easily seen that decoding speed is improved.

**Example 6.4** Consider the (693, 676) Fire code discussed above, which can correct any burst of length  $b \leq 6$ . Assume an error burst  $e(x) = x + x^6 = x(1 + x^5)$  which is of length 6, starts at position  $i = 1$ , and has the burst pattern  $B(x) = 1 + x^5$ . If  $e(x)$  is divided by  $1 + x^{11}$ , syndrome  $s_c(x)$  can then be obtained as  $s_c(x) = x + x^6$  from  $x + x^6 = q_c(x)(1 + x^{11}) + s_c(x)$ . If  $e(x)$  is divided by  $1 + x + x^6$ ,  $s_p(x) = 1$  from  $x + x^6 = q_p(x)(1 + x + x^6) + s_p(x)$ . Then  $s_c(x)$  and  $s_p(x)$  are the contents of the two syndrome registers when  $r(x)$  has been completely shifted into the error-pattern and error-tracking registers. Since  $x^6$  in  $e(x)$  is in the  $c - b$  higher section of the error-pattern register, the register must start to shift accordingly, because the error burst is not trapped in the  $b$  lower section of the error-pattern register. Since  $s_c^{(10)}(x) = 1 + x^5$ , or  $s_c = (1\ 0\ 0\ 0\ 0\ 1\ 0\ 0\ 0\ 0\ 0)$ , the contents in the five high-order stages are all 0s after the tenth shift and the error burst is trapped in the six low-order stages in the error-pattern register. Thus we get  $\gamma_c = 1$  from  $11 - \gamma_c = 10$ .

Now, the error-tracking register begins to shift. After 62 shifts, the contents in the error-tracking register lock the contents in the low-order section of the error-pattern register. Thus we have  $\gamma_p = 1$  from  $63 - \gamma_p = 62$ .

Next, let us compute the error-starting number  $i$ . Using  $A_c e + A_p c = 1$ , we get  $63A_c + 11A_p = 1$  from which we could determine  $A_c = -4$  and  $A_p = 23$ . From the Chinese remainder theorem, it follows that

$$i = A_c e \gamma_c + A_p c \gamma_p = (-4)(63)(1) + (23)(11)(1) = 1$$

which, as expected, is exactly equal to the error-starting number. Thus the error burst  $e(x) = x^i B(x) = x(1 + x^5)$  has been detected. Error correction is then achieved by adding the error burst  $e(x) = x + x^6$  to the received word  $r(x)$  in the buffer register.

**TABLE 6.4** Cyclic Shifts for the Error-Pattern Register of the (693, 676) Fire Decoder

| Shift no. $j$ | Syndrome polynomial<br>$s_c^{(j)}(x)$ | Register contents     |
|---------------|---------------------------------------|-----------------------|
| 0             | $x + x^6$                             | 0 1 0 0 0 0 1 0 0 0 0 |
| 1             | $x^2 + x^7$                           | 0 0 1 0 0 0 0 1 0 0 0 |
| 2             | $x^3 + x^8$                           | 0 0 0 1 0 0 0 0 1 0 0 |
| 3             | $x^4 + x^9$                           | 0 0 0 0 1 0 0 0 0 1 0 |
| 4             | $x^5 + x^{10}$                        | 0 0 0 0 0 1 0 0 0 0 1 |
| 5             | $1 + x^6$                             | 1 0 0 0 0 0 1 0 0 0 0 |
| 6             | $x + x^7$                             | 0 1 0 0 0 0 0 1 0 0 0 |
| 7             | $x^2 + x^8$                           | 0 0 1 0 0 0 0 0 1 0 0 |
| 8             | $x^3 + x^9$                           | 0 0 0 1 0 0 0 0 0 1 0 |
| 9             | $x^4 + x^{10}$                        | 0 0 0 0 1 0 0 0 0 0 1 |
| 10            | $1 + x^5$                             | 1 0 0 0 0 1 0 0 0 0 0 |

**TABLE 6.5** Cyclic Shifts for the Error-Tracking Register of the (693, 676) Fire Decoder

| Shift no. $j$ | Syndrome polynomial<br>$s_p^{(j)}(x)$ | Register contents |
|---------------|---------------------------------------|-------------------|
| 0             | 1                                     | 1 0 0 0 0 0       |
| 1             | $x$                                   | 0 1 0 0 0 0       |
| $\vdots$      | $\vdots$                              | $\vdots$          |
| 6             | $1 + x$                               | 1 1 0 0 0 0       |
| $\vdots$      | $\vdots$                              | $\vdots$          |
| 11            | $1 + x + x^5$                         | 1 1 0 0 0 1       |
| $\vdots$      | $\vdots$                              | $\vdots$          |
| 26            | $1 + x + x^2$                         | 1 1 1 0 0 0       |
| $\vdots$      | $\vdots$                              | $\vdots$          |
| 38            | $1 + x + x^3 + x^4$                   | 1 1 0 1 1 0       |
| $\vdots$      | $\vdots$                              | $\vdots$          |
| 53            | $x + x^3 + x^5$                       | 0 1 0 1 0 1       |
| $\vdots$      | $\vdots$                              | $\vdots$          |
| 61            | $1 + x^4 + x^5$                       | 1 0 0 0 1 1       |
| 62            | $1 + x^5$                             | 1 0 0 0 0 1       |

**Example 6.5** Consider a (1651, 1631) cyclic Fire code generated by  $g(x) = (1 + x^3 + x^7)(1 + x^{13})$ . This code is capable of correcting a single burst of length 7 or less. Polynomial  $p(x) = 1 + x^3 + x^7$  is irreducible and of degree 7, and the period of  $p(x)$  is a positive integer  $e$  such that  $x^e + 1$  is divisible by  $p(x)$ . Since  $e = 2^m - 1 = 127$  and  $c = 13$  are relatively prime, the length of this code



is  $n = (127)(13) = 1651$ . Also the number of parity-check digits is  $n - k = m + 2b - 1 = 7 + 14 - 1 = 20$ . Suppose that the error burst

$$\begin{aligned} e(x) &= x^3 + x^4 + x^5 + x^6 + x^7 \\ &= x^3(1 + x + x^2 + x^3 + x^4) \end{aligned}$$

has occurred. This is an error burst of length 5 starting at position  $i = 3$ , and the error pattern  $B(x) = 1 + x + x^2 + x^3 + x^4$ . The syndromes  $s_c(x)$  and  $s_p(x)$  are remainders that result from dividing  $e(x)$  by  $1 + x^{13}$  and  $1 + x^3 + x^7$ , respectively. They are

$$\begin{aligned} s_c(x) &= x^3 + x^4 + x^5 + x^6 + x^7 \\ s_p(x) &= 1 + x^4 + x^5 + x^6 \end{aligned}$$

The syndromes  $s_c(x)$  and  $s_p(x)$  are the contents in the error-pattern and error-tracking registers when the received word  $r(x)$  has been shifted into the two registers. Since the error burst  $e(x)$  is not trapped in the lower-order stages of the error-pattern register, the register must then be shifted cyclically. Table 6.6 shows the contents in the error-pattern register after each shift.

**TABLE 6.6** Cyclic Shifts for the Error-Pattern Register of the (1651, 1631) Fire Decoder

| Shift no. $j$ | Syndrome polynomial<br>$s_c^{(j)}(x)$  | Register contents $s_c^{(j)}$ |
|---------------|----------------------------------------|-------------------------------|
| 0             | $x^3 + x^4 + x^5 + x^6 + x^7$          | 0 0 0 1 1 1 1 1 0 0 0 0 0     |
| 1             | $x^4 + x^5 + x^6 + x^7 + x^8$          | 0 0 0 0 1 1 1 1 1 0 0 0 0     |
| 2             | $x^5 + x^6 + x^7 + x^8 + x^9$          | 0 0 0 0 0 1 1 1 1 1 0 0 0     |
| 3             | $x^6 + x^7 + x^8 + x^9 + x^{10}$       | 0 0 0 0 0 0 1 1 1 1 1 0 0     |
| 4             | $x^7 + x^8 + x^9 + x^{10} + x^{11}$    | 0 0 0 0 0 0 0 1 1 1 1 1 0     |
| 5             | $x^8 + x^9 + x^{10} + x^{11} + x^{12}$ | 0 0 0 0 0 0 0 0 1 1 1 1 1     |
| 6             | $1 + x^9 + x^{10} + x^{11} + x^{12}$   | 1 0 0 0 0 0 0 0 0 1 1 1 1     |
| 7             | $1 + x + x^{10} + x^{11} + x^{12}$     | 1 1 0 0 0 0 0 0 0 0 1 1 1     |
| 8             | $1 + x + x^2 + x^{11} + x^{12}$        | 1 1 1 0 0 0 0 0 0 0 0 1 1     |
| 9             | $1 + x + x^2 + x^3 + x^{12}$           | 1 1 1 1 0 0 0 0 0 0 0 0 1     |
| 10            | $1 + x + x^2 + x^3 + x^4$              | 1 1 1 1 1 0 0 0 0 0 0 0 0     |

Since  $s_c^{(10)} = (1 \ 1 \ 1 \ 1 \ 1 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0)$ , corresponding to  $s_c^{(10)}(x) = 1 + x + x^2 + x^3 + x^4$ , we see that the six high-order stages are all 0s after the tenth shift, so the error burst is trapped in the seven low-order stages in the error-pattern register. Thus the shift-count number is  $13 - \gamma_c = 10$ ,  $0 \leq \gamma_c < 13$ , from which we get  $\gamma_c = 3$ . Next, let us shift the error-tracking register in order to lock the contents in the seven lower stages of the error-pattern register.

As you see from Table 6.7, the contents in the error-tracking register lock the contents in the seven lower stages of the error-pattern register. Hence the counting number is  $127 - \gamma_p = 124$  from which we get  $\gamma_p = 3$ . Thus the error-starting number  $i$  can be computed by using the equation  $i = A_c e \gamma_c + A_p c \gamma_p$ . However, since  $c$  and  $e$  are relatively prime, there exist two integers  $A_c$  and  $A_p$

TABLE 6.7 Cyclic Shifts for the Error-Tracking Register of the (1651, 1631) Fire Decoder

| Shift no. $j$ | Syndrome polynomial $s_p^{(j)}(x)$ | Register contents $s_p^{(j)}$ |
|---------------|------------------------------------|-------------------------------|
| 0             | $1 + x^4 + x^5 + x^6$              | 1 0 0 0 1 1 1                 |
| 1             | $1 + x + x^5 + x^6$                | 1 1 0 0 0 1 1                 |
| 10            | $x^2 + x^4$                        | 0 0 1 0 1 0 0                 |
| 20            | $1 + x + x^4 + x^5 + x^6$          | 1 1 0 0 1 1 1                 |
| 30            | $1 + x^2 + x^3$                    | 1 0 1 1 0 0 0                 |
| 40            | $x + x^2 + x^3 + x^4$              | 0 1 1 1 1 0 0                 |
| 50            | $x + x^2 + x^3$                    | 0 1 1 1 0 0 0                 |
| 60            | $1 + x + x^2 + x^3 + x^6$          | 1 1 1 1 0 0 1                 |
| 70            | $1 + x^4$                          | 1 0 0 0 1 0 0                 |
| 80            | $1 + x^3$                          | 1 0 0 1 0 0 0                 |
| 90            | $x^2 + x^3 + x^5$                  | 0 0 1 1 0 1 0                 |
| 100           | $1 + x^2 + x^3 + x^4 + x^6$        | 1 0 1 1 1 0 1                 |
| 110           | $1 + x^3 + x^6$                    | 1 0 0 1 0 0 1                 |
| 120           | $x + x^3 + x^4 + x^5$              | 0 1 0 1 1 1 0                 |
| 124           | $1 + x + x^2 + x^3 + x^4$          | 1 1 1 1 1 0 0                 |

such that  $A_c e + A_p c = 1$  or  $127A_c + 13A_p = 1$ , from which  $A_c = 4$  and  $A_p = -39$  can be determined easily.

Once  $A_c$ ,  $A_p$ ,  $\gamma_c$ , and  $\gamma_p$  are determined, the error-starting number  $i$  can easily be determined as  $i = (4)(127)(3) + (-39)(13)(3) = 1524 - 1521 = 3$ . Thus the error burst  $e(x) = x^i B(x) = x^3(1 + x + x^2 + x^3 + x^4)$  has been detected. The  $x^3$  indicates the starting location of the error burst. Once  $B(x)$  and  $i$  are determined, the correction is accomplished by adding  $x^i B(x)$  to the received polynomial  $r(x)$ .

According to Theorem 6.3, the Reiger bound is expressed by  $b \leq (n - k)/2$ . The ratio  $z = 2b/(n - k)$ , which is called the burst-correcting efficiency, is unity and has an upper bound  $b = (n - k)/2$ . Codes which meet  $z = 1$  are said to be optimal. Since  $z$  is expressed by the ratio  $2b/(m + 2b - 1)$  for a Fire code, we obtain  $z = 0.62$  for  $b = 5$  and  $m = 7$ . If  $b$  is chosen to be equal to  $m$ , then we have  $z = 0.7$  for  $b = m = 7$ . As a result, Fire codes are not very efficient with respect to the Reiger bound.

## PROBLEMS

**6.1** The polynomial  $p(x) = 1 + x^2 + x^5$  over  $GF(2)$  is a primitive polynomial of degree  $m = 5$ .

- What is the period of  $p(x)$ ?
- Find the generator polynomial of a Fire code that is capable of correcting any single error burst of length  $b = 4$ .
- What is the length of this code?
- Determine the number of parity-check digits of this code.

**6.2** Construct an  $(n, k)$  Fire code based on Prob. 6.1, and design an error-trapping decoder for this code when the received polynomial  $r(x)$  is shifted into the syndrome register from the left end.

**6.3** Consider the (217, 205) cyclic Fire code constructed in Prob. 6.1. Suppose that a single burst of length  $b = 4$  starts at  $x^2$ .

- a* Determine the syndromes  $s_c(x)$  and  $s_p(x)$  that result from dividing  $e(x)$  by  $1 + x^c$ , where  $c = 2b - 1$ , and by  $p(x)$ , respectively.
- b* Design a high-speed error-trapping decoder for this code, and describe the decoding operation.

**6.4** Devise the conventional error-trapping decoder for the (693, 676) Fire code with the generator polynomial  $g(x) = (1 + x + x^6)(1 + x^{11})$ .

**6.5** Consider the (279, 265) cyclic Fire code generated by  $g(x) = 1 + x^2 + x^5 + x^9 + x^{11} + x^{14}$ . This Fire code corrects any single burst of length  $b = 5$  or less.

- a* Determine the primitive polynomial  $p(x)$  of degree 5.
- b* Find the polynomial  $1 + x^c$ , where  $c = 2b - 1$ , that results from dividing  $g(x)$  by  $p(x)$ .
- c* Devise a simple error-trapping decoder for this code.
- d* Design a high-speed error-trapping decoder for this code, and describe the decoding operation.

**6.6** Suppose that the Fire code constructed in Prob. 6.1 is shortened by deleting the 14 high-order information digits. Design a decoder for this shortened code.



## Bose-Chaudhuri-Hocquenghem Codes and Reed-Solomon Codes

A large class of the most powerful random-error-correcting codes was discovered by Bose and Ray-Chaudhuri (1960) and independently by Hocquenghem (1959). These codes are known as BCH codes, and they occupy a prominent place in the theory and practice of multiple-error correction. The original application for BCH codes was restricted to binary codes of length  $2^m - 1$  for some integer  $m$  and was later extended by Gorenstein and Zierler (1961) to the nonbinary codes with symbols from the Galois field  $GF(q)$ . Among the nonbinary BCH codes, the most important and popular subclass is the class of codes introduced by Reed and Solomon (1960).

The presentation of BCH codes and their implementation in the time domain is given in this chapter. However, BCH codes can also be defined and implemented in the frequency domain by using Fourier transforms over Galois fields.

The importance of BCH codes stems from the fact that they are capable of correcting all random patterns of  $t$  errors by a decoding algorithm that is simple and easily implemented with a reasonable amount of equipment. The first decoding algorithm for binary BCH codes was devised by Peterson (1960). Since then, Peterson's algorithm has been refined by Berlekamp, Massey, Chien, Forney, and others.

### 7.1 Definition of the Codes

The BCH codes are easily defined in terms of the roots of their generator polynomials. A primitive  $t$ -error-correcting BCH codes over  $GF(q)$  of block length  $n = q^m - 1$  has  $\alpha^{m_0}, \alpha^{m_0+1}, \dots, \alpha^{m_0+2t-1}$  as the roots of its generator polynomial  $g(x)$ , where  $\alpha^{m_0}$  is a primitive element of



$GF(q^m)$ . The codes with  $m_0 = 1$  and  $q = 2$  are called binary narrow-sense BCH codes. The BCH codes are a class of cyclic codes whose generator polynomial is the product of distinct minimal polynomials corresponding to  $\alpha, \alpha^2, \dots, \alpha^{2t}$ , where  $\alpha \in GF(2^m)$  is a root of the primitive polynomial  $p(x)$ .

Let  $m_i(x)$  be the minimal polynomial of  $\alpha^i$ . Let  $c(x) = c_0 + c_1x + c_2x^2 + \dots + c_{n-1}x^{n-1}$  be a code polynomial with coefficients from  $GF(2)$ . If  $c(x)$  has  $\alpha, \alpha^2, \dots, \alpha^{2t}$  as its roots,  $c(x)$  is then divisible by the minimal polynomials  $m_1(x), m_2(x), \dots, m_{2t}(x)$  of  $\alpha, \alpha^2, \dots, \alpha^{2t}$ . The generator polynomial  $g(x)$  of the  $t$ -error-correcting BCH code of block length  $n = 2^m - 1$  and rate  $k/n$  is the lowest degree polynomial over  $GF(2)$ . Thus, the generator polynomial of the code must be the least common multiple of these minimal polynomials. That is,

$$g(x) = LCM\{m_1(x), m_2(x), \dots, m_{2t}(x)\} \quad (7.1)$$

The nonprimitive codes are defined in the same manner, except that  $\alpha$  is replaced by a nonprimitive element  $\beta$  of  $GF(q^m)$ , and the code length becomes the order of  $\beta$ . Lists of code generator polynomials for both primitive and nonprimitive binary BCH codes are given in Appendix 7D.

**Example 7.1** Consider the triple-error-correcting (15, 5) BCH code. Let  $\alpha$  be a primitive element of the Galois field  $GF(2^4)$  such that  $1 + \alpha + \alpha^4 = 0$ . If  $m_i(x)$ ,  $i = 1, 2, 3, \dots, 14$ , denote the minimal polynomials of  $\alpha^i$ , which are the elements of  $GF(2^4)$ , we then have the list given by Table 7.1.

TABLE 7.1 Minimal Polynomials of the Elements in  $GF(2^4)$

| Elements      | Conjugates                                                                      | Minimal polynomials                   |
|---------------|---------------------------------------------------------------------------------|---------------------------------------|
| $\alpha$      | $\alpha^2, \alpha^4, \alpha^8$                                                  | $m_1(x) = 1 + x + x^4$                |
| $\alpha^2$    | $\alpha^4, \alpha^8, \alpha^{16} = \alpha$                                      | $m_2(x) = 1 + x + x^4$                |
| $\alpha^3$    | $\alpha^6, \alpha^{12}, \alpha^{24} = \alpha^9$                                 | $m_3(x) = 1 + x + x^2 + x^3 + x^4$    |
| $\alpha^4$    | $\alpha^8, \alpha^{16} = \alpha, \alpha^{32} = \alpha^2$                        | $m_4(x) = 1 + x + x^4$                |
| $\alpha^5$    | $\alpha^{10}$                                                                   | $m_5(x) = 1 + x + x^2$                |
| $\alpha^6$    | $\alpha^{12}, \alpha^{24} = \alpha^9, \alpha^{48} = \alpha^3$                   | $m_6(x) = 1 + x + x^2 + x^3 + x^4$    |
| $\alpha^7$    | $\alpha^{14}, \alpha^{28} = \alpha^{13}, \alpha^{56} = \alpha^{11}$             | $m_7(x) = 1 + x^3 + x^4$              |
| $\alpha^8$    | $\alpha^{16} = \alpha, \alpha^{32} = \alpha^2, \alpha^{64} = \alpha^4$          | $m_8(x) = 1 + x + x^4$                |
| $\alpha^9$    | $\alpha^{18} = \alpha^3, \alpha^{36} = \alpha^6, \alpha^{72} = \alpha^{12}$     | $m_9(x) = 1 + x + x^2 + x^3 + x^4$    |
| $\alpha^{10}$ | $\alpha^{20} = \alpha^5$                                                        | $m_{10}(x) = 1 + x + x^2$             |
| $\alpha^{11}$ | $\alpha^{22} = \alpha^7, \alpha^{44} = \alpha^{14}, \alpha^{88} = \alpha^{13}$  | $m_{11}(x) = 1 + x^3 + x^4$           |
| $\alpha^{12}$ | $\alpha^{24} = \alpha^9, \alpha^{48} = \alpha^3, \alpha^{96} = \alpha^6$        | $m_{12}(x) = 1 + x + x^2 + x^3 + x^4$ |
| $\alpha^{13}$ | $\alpha^{26} = \alpha^{11}, \alpha^{52} = \alpha^7, \alpha^{104} = \alpha^{14}$ | $m_{13}(x) = 1 + x^3 + x^4$           |
| $\alpha^{14}$ | $\alpha^{28} = \alpha^{13}, \alpha^{56} = \alpha^{11}, \alpha^{112} = \alpha^7$ | $m_{14}(x) = 1 + x^3 + x^4$           |

From the contents of Table 7.1, we easily see that

$$m_1(x) = m_2(x) = m_4(x) = m_8(x)$$

$$m_3(x) = m_6(x) = m_9(x) = m_{12}(x)$$

$$m_5(x) = m_{10}(x)$$

$$m_7(x) = m_{11}(x) = m_{13}(x) = m_{14}(x)$$

Hence,  $m_1(x)$ ,  $m_3(x)$ ,  $m_5(x)$ , and  $m_7(x)$  are only four distinct irreducible polynomials. Thus, the generator polynomial of the triple-error-correcting binary BCH code of length 15 can be given by

$$\begin{aligned} g(x) &= \text{LCM}\{m_1(x), m_2(x), m_3(x), m_4(x), m_5(x), m_6(x)\} \\ &= m_1(x)m_3(x)m_5(x) \end{aligned}$$

Consequently, we see that every even power of  $\alpha$  in the element sequence has the same minimal polynomial as some preceding odd power of  $\alpha$  in the sequence. As a result, the generator polynomial  $g(x)$  of the binary  $t$ -error-correcting BCH code of length  $2^m - 1$  given by Eq. (7.1) can be reduced to

$$g(x) = \text{LCM}\{m_1(x), m_3(x), \dots, m_{2t-1}(x)\} \quad (7.2)$$

where the degree of each  $m_i(x)$  is no greater than  $m$ . Therefore, Eq. (7.2) implies that  $c(x)$  is a code word if and only if  $\alpha, \alpha^3, \dots, \alpha^{2t-1}$  are roots of  $c(x)$ . Since the degree of each minimal polynomial is  $m$  or less, the largest possible value for the degree of  $g(x)$  is  $mt$ . Hence, the code has at most  $mt$  parity-check digits, i.e.,  $n - k \leq mt$ .

**Example 7.2** Consider the binary primitive BCH codes of length  $2^m - 1$  with  $m = 4$ . The parameters for these codes are shown in the following table.

| $n$          | $k$ | $t$ | $n - k$ | $mt$ |
|--------------|-----|-----|---------|------|
| $n - k = mt$ |     |     |         |      |
| 15           | 11  | 1   | 4       | 4    |
| 15           | 7   | 2   | 8       | 8    |
| $n - k < mt$ |     |     |         |      |
| 15           | 5   | 3   | 10      | 12   |
| 15           | 1   | 7   | 14      | 28   |

It is clear that the number of parity-check digits  $n - k$  is exactly equal to  $mt$  when  $t$  is small, but  $n - k < mt$  when  $t$  is large. Thus, the number of parity checks for any BCH code is generally expressed by  $n - k \leq mt$ , from which it can be calculated that the number of information digits is  $k \geq 2^m - mt - 1$ .

In general, for any positive integers  $m \geq 3$  and  $t < n/2$ , there exists a binary BCH code with parameters of block length  $n = 2^m - 1$ , number of parity-check bits  $n - k \leq mt$ , and minimum distance  $d_0 = 2t + 1 \leq d_{\min}$ . The designed distance of the code is  $d_0 = 2t + 1$ . The minimum distance  $d_{\min}$  may be larger than  $d_0$ . The following steps are used to determine the BCH codes.

1. Choose a primitive polynomial of degree  $m$ , and construct  $GF(2^m)$ .
2. Find the minimal polynomial  $m_i(x)$  of  $\alpha^i$  for  $i = 1, 2, \dots, 2t$ .
3. Obtain  $g(x) = LCM\{m_1(x), m_3(x), \dots, m_{2t-1}(x)\}$ .
4. Determine  $k$  from  $n - k$ , which is the degree of  $g(x)$ .
5. Find the minimum distance  $d_{\min} \geq 2t + 1$  by referring to the weight of  $g(x)$ .

**Example 7.3** Consider the  $t$ -error-correcting BCH codes of length  $n = 2^5 - 1 = 31$ . Let  $\alpha$  be a primitive element of  $GF(2^5)$  with  $1 + \alpha^2 + \alpha^5 = 0$ . Table 7.2 gives  $GF(2^5)$  as an extension field of  $GF(2)$  constructed with the primitive polynomial  $p(x) = 1 + x^2 + x^5$ . Let  $m_i(x)$  be the minimal polynomials of  $\alpha^i$ ,  $1 \leq i \leq 2t$ , which are roots of the code polynomial  $c(x)$ .

1.  $\alpha$  and its conjugates  $\alpha^2, \alpha^4, \alpha^8, \alpha^{16}$  are roots of  $m_1(x)$ , and  $m_1(x) = m_2(x) = m_4(x) = m_8(x) = m_{16}(x)$ . The minimal polynomial  $m_1(x)$  of  $\alpha$  is obtained with the aid of Table 7.2 such that

$$\begin{aligned} m_1(x) &= (x + \alpha)(x + \alpha^2)(x + \alpha^4)(x + \alpha^8)(x + \alpha^{16}) \\ &= 1 + x^2 + x^5 \end{aligned}$$

2.  $\alpha^3$  and its conjugates  $\alpha^6, \alpha^{12}, \alpha^{24}, \alpha^{17}$  are roots of  $m_3(x)$ , and  $m_3(x) = m_6(x) = m_{12}(x) = m_{24}(x) = m_{17}(x)$ . The minimal polynomial  $m_3(x)$  of  $\alpha^3$  is

$$\begin{aligned} m_3(x) &= (x + \alpha^3)(x + \alpha^6)(x + \alpha^{12})(x + \alpha^{24})(x + \alpha^{17}) \\ &= 1 + x^2 + x^3 + x^4 + x^5 \end{aligned}$$

3.  $\alpha^5$  and its conjugates  $\alpha^{10}, \alpha^{20}, \alpha^9, \alpha^{18}$  are roots of  $m_5(x)$ , and  $m_5(x) = m_{10}(x) = m_{20}(x) = m_9(x) = m_{18}(x)$ . The minimal polynomial  $m_5(x)$  of  $\alpha^5$  is

$$\begin{aligned} m_5(x) &= (x + \alpha^5)(x + \alpha^{10})(x + \alpha^{20})(x + \alpha^9)(x + \alpha^{18}) \\ &= 1 + x + x^2 + x^4 + x^5 \end{aligned}$$

4.  $\alpha^7$  and its conjugates  $\alpha^{14}, \alpha^{28}, \alpha^{25}, \alpha^{19}$  are roots of  $m_7(x)$ , and  $m_7(x) = m_{14}(x) = m_{28}(x) = m_{25}(x) = m_{19}(x)$ . The minimal polynomial  $m_7(x)$  of  $\alpha^7$  is

$$\begin{aligned} m_7(x) &= (x + \alpha^7)(x + \alpha^{14})(x + \alpha^{28})(x + \alpha^{25})(x + \alpha^{19}) \\ &= 1 + x + x^2 + x^3 + x^5 \end{aligned}$$

5.  $\alpha^{11}$  and its conjugates  $\alpha^{22}, \alpha^{13}, \alpha^{26}, \alpha^{21}$  are roots of  $m_{11}(x)$  and  $m_{11}(x) = m_{22}(x) = m_{13}(x) = m_{26}(x) = m_{21}(x)$ . The minimal polynomial  $m_{11}(x)$  of  $\alpha^{11}$  is

$$\begin{aligned} m_{11}(x) &= (x + \alpha^{11})(x + \alpha^{22})(x + \alpha^{13})(x + \alpha^{26})(x + \alpha^{21}) \\ &= 1 + x + x^3 + x^4 + x^5 \end{aligned}$$

TABLE 7.2 Galois Field  $GF(2^5)$  with  $p(\alpha) = 1 + \alpha^2 + \alpha^5 = 0$ 

| Field element (polynomial notation)                         | 5-tuple representation |
|-------------------------------------------------------------|------------------------|
| 0                                                           | 0 0 0 0 0              |
| 1                                                           | 1 0 0 0 0              |
| $\alpha^1$                                                  | 0 1 0 0 0              |
| $\alpha^2$                                                  | 0 0 1 0 0              |
| $\alpha^3$                                                  | 0 0 0 1 0              |
| $\alpha^4$                                                  | 0 0 0 0 1              |
| $\alpha^5 = 1 + \alpha^2$                                   | 1 0 1 0 0              |
| $\alpha^6 = \alpha + \alpha^3$                              | 0 1 0 1 0              |
| $\alpha^7 = \alpha^2 + \alpha^4$                            | 0 0 1 0 1              |
| $\alpha^8 = 1 + \alpha^2 + \alpha^3$                        | 1 0 1 1 0              |
| $\alpha^9 = \alpha + \alpha^3 + \alpha^4$                   | 0 1 0 1 1              |
| $\alpha^{10} = 1 + \alpha^2 + \alpha^4$                     | 1 0 0 0 1              |
| $\alpha^{11} = 1 + \alpha + \alpha^2$                       | 1 1 1 0 0              |
| $\alpha^{12} = \alpha + \alpha^2 + \alpha^3$                | 0 1 1 1 0              |
| $\alpha^{13} = \alpha^2 + \alpha^3 + \alpha^4$              | 0 0 1 1 1              |
| $\alpha^{14} = 1 + \alpha^2 + \alpha^3 + \alpha^4$          | 1 0 1 1 1              |
| $\alpha^{15} = 1 + \alpha + \alpha^2 + \alpha^3 + \alpha^4$ | 1 1 1 1 1              |
| $\alpha^{16} = 1 + \alpha + \alpha^3 + \alpha^4$            | 1 1 0 1 1              |
| $\alpha^{17} = 1 + \alpha + \alpha^4$                       | 1 1 0 0 1              |
| $\alpha^{18} = 1 + \alpha$                                  | 1 1 0 0 0              |
| $\alpha^{19} = \alpha + \alpha^2$                           | 0 1 1 0 0              |
| $\alpha^{20} = \alpha^2 + \alpha^3$                         | 0 0 1 1 0              |
| $\alpha^{21} = \alpha^3 + \alpha^4$                         | 0 0 0 1 1              |
| $\alpha^{22} = 1 + \alpha^2 + \alpha^4$                     | 1 0 1 0 1              |
| $\alpha^{23} = 1 + \alpha + \alpha^2 + \alpha^3$            | 1 1 1 1 0              |
| $\alpha^{24} = \alpha + \alpha^2 + \alpha^3 + \alpha^4$     | 0 1 1 1 1              |
| $\alpha^{25} = 1 + \alpha^2 + \alpha^3 + \alpha^4$          | 1 0 0 1 1              |
| $\alpha^{26} = 1 + \alpha + \alpha^2 + \alpha^4$            | 1 1 1 0 1              |
| $\alpha^{27} = 1 + \alpha + \alpha^3$                       | 1 1 0 1 0              |
| $\alpha^{28} = \alpha + \alpha^2 + \alpha^4$                | 0 1 1 0 1              |
| $\alpha^{29} = 1 + \alpha^3$                                | 1 0 0 1 0              |
| $\alpha^{30} = \alpha + \alpha^4$                           | 0 1 0 0 1              |

**Example 7.4** Consider again the  $t$ -error-correcting  $(31, k)$  BCH code. BCH codes are designed from the specification of  $n$  and  $t$ . The generator polynomials are constructed with the aid of the minimal polynomials found in Example 7.3. Let  $g_t(x)$  be the generator polynomials of  $(31, k)$  BCH codes for  $t = 1, 2, 3, 5, 7$ . The generator polynomial for the single-error-correcting BCH code of length  $n = 2^5 - 1 = 31$  is obtained as follows:

$$\begin{aligned}
 g_1(x) &= \text{LCM}\{m_1(x), m_2(x)\} \\
 &= m_1(x) \\
 &= 1 + x^2 + x^5
 \end{aligned}$$

because  $m_1(x) = m_2(x)$ . Since  $g_1(x)$  is of degree 5,  $n - k = 5$ . Therefore,  $k = 26$  and we have the  $(31, 26)$  BCH code with  $d_{\min} \geq 3$ , which is identical to a single-error-correcting Hamming code. Since  $g_1(x)$  is also a code polynomial of weight 3, the minimum distance of this code is exactly 3. Continuing in this way, we

can construct the generator polynomials for other primitive BCH codes of block length 31.

When  $t = 2$ ,  $g_2(x)$  for the double-error-correcting BCH code is

$$\begin{aligned}
 g_2(x) &= LCM\{m_1(x), m_2(x), m_3(x), m_4(x)\} \\
 &= g_1(x)m_3(x) \\
 &= m_1(x)m_3(x) \\
 &= (1 + x^2 + x^5)(1 + x^2 + x^3 + x^4 + x^5) \\
 &= 1 + x^3 + x^5 + x^6 + x^8 + x^9 + x^{10}
 \end{aligned}$$

This is the generator polynomial for a (31, 21) double-error-correcting BCH code. The number of information bits is  $k = 21$  because  $31 - k = 10$ . Since the weight of  $g_2(x)$  is 7, the minimum distance of this code is 7.

When  $t = 3$ , we have

$$\begin{aligned}
 g_3(x) &= g_2(x)m_5(x) \\
 &= (1 + x^3 + x^5 + x^6 + x^8 + x^9 + x^{10})(1 + x + x^2 + x^4 + x^5) \\
 &= 1 + x + x^2 + x^3 + x^5 + x^7 + x^8 + x^9 + x^{10} + x^{11} + x^{15}
 \end{aligned}$$

This is the generator polynomial for the triple-error-correcting (31, 16) BCH code with  $d_{\min} \geq 7$ . Since the weight of  $g_3(x)$  is 11, the minimum distance of this code is 11.

When  $t = 5$ , the generator polynomial is  $g_5(x) = LCM\{m_1(x), m_3(x), m_5(x), m_7(x), m_9(x)\}$ . Since  $m_5(x) = m_9(x)$ , we obtain

$$\begin{aligned}
 g_5(x) &= m_1(x)m_3(x)m_5(x)m_7(x) \\
 &= g_3(x)m_7(x) \\
 &= 1 + x^2 + x^4 + x^6 + x^7 + x^9 + x^{10} + x^{13} + x^{17} + x^{18} + x^{20}
 \end{aligned}$$

This five-error-correcting BCH code with  $d_{\min} \geq 11$  is a (31, 11) cyclic code with  $d_{\min} = 11$ , which is identical to the weight of  $g_5(x)$ . When  $t = 7$ ,  $g_7(x) = LCM\{m_1(x), m_3(x), m_5(x), m_7(x), m_9(x), m_{11}(x), m_{13}(x)\}$  becomes

$$\begin{aligned}
 g_7(x) &= m_1(x)m_3(x)m_5(x)m_7(x)m_{11}(x) \\
 &= g_5(x)m_{11}(x) \\
 &= 1 + x + x^2 + x^5 + x^9 + x^{11} + x^{13} + x^{14} + x^{15} + x^{16} + x^{18} + x^{19} + x^{21} \\
 &\quad + x^{24} + x^{25}
 \end{aligned}$$

because  $m_5(x) = m_9(x)$  and  $m_{11}(x) = m_{13}(x)$ . This is the generator polynomial for the seven-error-correcting (31, 6) BCH code with  $d_{\min} = 15$  because the weight of  $g_7(x)$  is exactly 15. Beyond  $t = 7$ , the BCH code is undefined because the number of nonzero field elements is 31. Thus, it should be noted that BCH codes are designed from the specification of  $n$  and  $t$ . The generator polynomials  $g_t(x)$  in octal for the  $t$ -error-correcting BCH codes of the same block length  $n = 31$  are listed in Table 7.3.



TABLE 7.3  $g_t(x)$  for the Primitive  $t$ -Error-Correcting  $(31, k)$  BCH Codes

| $n$ | $k$ | $t$ | Generator polynomial,<br>$g_t(x)$ (octal) |
|-----|-----|-----|-------------------------------------------|
| 31  | 26  | 1   | 45                                        |
| 31  | 21  | 2   | 3551                                      |
| 31  | 16  | 3   | 107657                                    |
| 31  | 11  | 5   | 5423325                                   |
| 31  | 6   | 7   | 313365047                                 |

Our analysis is based on the code length of the form  $n = 2^m - 1 = 31$  for  $m = 5$ . The first code is a Hamming code, so it has a primitive polynomial  $p(x)$  of degree  $m$  as its generator polynomial  $g_1(x)$ . The next generator polynomial  $g_2(x)$  of the same length  $n = 31$  is found by multiplying  $p(x) = g_1(x)$  by  $m_3(x)$ , the minimal polynomial of  $\alpha^3$ . If we continue in this fashion, the previous generator polynomial is multiplied by a new minimal polynomial at each step. This process produces a sequence of BCH code generator polynomials that increase in degree and error-correcting capability, as shown in Table 7.3.

## 7.2 Minimum Distance and BCH Bounds

Let  $c(x) = c_0 + c_1x + c_2x^2 + \cdots + c_{n-1}x^{n-1}$  be a code polynomial in a primitive  $t$ -error-correcting BCH code of length  $n = 2^m - 1$ . If  $c(x)$  has  $\alpha, \alpha^2, \dots, \alpha^{2t}$  as its roots,  $c(x)$  is divisible by the generator polynomial which is the least common multiple of  $m_i(x)$ ,  $1 \leq i \leq 2t$ .

Since  $\alpha^i$ ,  $1 \leq i \leq 2t$ , are roots of  $c(x)$ , it follows that

$$c(\alpha^i) = c_0 + c_1(\alpha^i)^1 + c_2(\alpha^i)^2 + \cdots + c_{n-1}(\alpha^i)^{n-1} = 0 \quad (7.3)$$

Equation (7.3) is also expressed by the following matrix product form:

$$(c_0, c_1, c_2, \dots, c_{n-1}) \cdot \begin{bmatrix} (\alpha^i)^0 \\ (\alpha^i)^1 \\ (\alpha^i)^2 \\ \vdots \\ (\alpha^i)^{n-1} \end{bmatrix} = 0 \quad 1 \leq i \leq 2t \quad (7.4)$$

or  $\mathbf{c} \cdot \mathbf{H}^T = \mathbf{0}$  because  $\mathbf{c} = (c_0, c_1, c_2, \dots, c_{n-1})$  is a code word in the  $t$ -error-correcting BCH code. Hence, the code is the null space of  $\mathbf{H}$ . Thus, the parity-check matrix of the code is

$$\mathbf{H} = [(\alpha^i)^0 (\alpha^i)^1 (\alpha^i)^2 \cdots (\alpha^i)^{n-1}] \quad 1 \leq i \leq 2t \quad (7.5)$$

where the entries of  $\mathbf{H}$  are elements in  $GF(2^m)$ .

Recall that the  $t$ -error-correcting BCH code is a remarkable generation of the single-error-correcting Hamming codes. Therefore, the powers of  $\alpha$  in Eq. (7.5) represent  $2t$ -tuple column vectors, and every nonzero column of length  $2t$ , or  $d_0 - 1$ , appears as a column of  $\mathbf{H}$  as shown below:

$$\mathbf{H} = \begin{bmatrix} \alpha^0 & \alpha^1 & \alpha^2 & \cdots & \alpha^{n-1} \\ (\alpha^2)^0 & (\alpha^2)^1 & (\alpha^2)^2 & \cdots & (\alpha^2)^{n-1} \\ (\alpha^3)^0 & (\alpha^3)^1 & (\alpha^3)^2 & \cdots & (\alpha^3)^{n-1} \\ \vdots & \vdots & \vdots & & \vdots \\ (\alpha^{2t})^0 & (\alpha^{2t})^1 & (\alpha^{2t})^2 & \cdots & (\alpha^{2t})^{n-1} \end{bmatrix} \quad (7.6)$$

If each entry of  $\mathbf{H}$  is replaced by its corresponding  $m$ -tuple over  $GF(2)$  arranged in column form, a binary parity-check matrix for the code is produced.

Since the  $k$ th row vector of  $\mathbf{H}$  is denoted by  $\mathbf{h}_k = [(\alpha^k)^0 (\alpha^k)^1 (\alpha^k)^2 \cdots (\alpha^k)^{n-1}]$ ,  $1 \leq k \leq 2t$ , the inner product of  $\mathbf{c}$  and  $\mathbf{h}_k^T$  is then

$$\mathbf{c} \cdot \mathbf{h}_k^T = \sum_{i=0}^{n-1} c_i (\alpha^k)^i = 0 \quad 1 \leq k \leq 2t \quad (7.7)$$

if and only if  $\mathbf{c}$  and  $\mathbf{h}_k^T$  are orthogonal to each other, i.e.,  $\mathbf{c} \cdot \mathbf{H}^T = \mathbf{0}$ . Now we must show that any set of  $d_0 - 1$  or  $2t$  columns of  $\mathbf{H}$  cannot be linearly dependent, which implies that the  $t$ -error-correcting BCH code has the minimum distance of at least  $d_0$  or  $2t + 1$ . To prove this, we need to show that no  $2t$  or fewer columns of  $\mathbf{H}$  given by Eq. (7.6) sum to zero. Let  $\alpha^j$ ,  $j = 2i, 4i, \dots$ , be the conjugates of  $\alpha^i$  for some  $i$ . Each  $c(\alpha^i) = 0$  because  $\alpha^i$  are the roots of  $c(x)$ , and so each  $c(\alpha^j) = 0$ . Thus, the even-numbered rows of  $\mathbf{H}$  given in Eq. (7.6) can be omitted.

Consider a code word whose components consist of only nonzero digits  $c_{j_\lambda} = 1$ ,  $1 \leq \lambda \leq 2t$ , i.e., the zero components are omitted. Then, it follows from  $\mathbf{c} \cdot \mathbf{H}^T = \mathbf{0}$  that

$$(c_{j_1}, c_{j_2}, \dots, c_{j_{2t}}) \cdot \begin{bmatrix} \alpha^{j_1} & (\alpha^{j_1})^2 & \cdots & (\alpha^{j_1})^{2t} \\ \alpha^{j_2} & (\alpha^{j_2})^2 & \cdots & (\alpha^{j_2})^{2t} \\ \vdots & \vdots & & \vdots \\ \alpha^{j_{2t}} & (\alpha^{j_{2t}})^2 & \cdots & (\alpha^{j_{2t}})^{2t} \end{bmatrix} = \mathbf{0} \quad (7.8)$$

where  $c_{j_1} = c_{j_2} = \dots = c_{j_{2t}} = 1$ .

Recall that the minimum weight of a code word  $\mathbf{c}$  is equal to the smallest number of columns of  $\mathbf{H}$  that sum to zero. If no  $2t$  (i.e.,  $d_0 - 1$ ) or fewer columns of  $\mathbf{H}$  add to zero, the code has the minimum weight  $d_0$ . Now, the second matrix on the left in Eq. (7.8) is a  $2t \times 2t$ , or  $(d_0 - 1) \times (d_0 - 1)$  square matrix. To satisfy the matrix system of Eq. (7.8), the determinant of the  $(d_0 - 1) \times (d_0 - 1)$  matrix must be 0, namely,

$$D = \begin{vmatrix} \alpha^{j_1} & (\alpha^{j_1})^2 & \dots & (\alpha^{j_1})^{2t} \\ \alpha^{j_2} & (\alpha^{j_2})^2 & \dots & (\alpha^{j_2})^{2t} \\ \vdots & \vdots & & \vdots \\ \alpha^{j_{2t}} & (\alpha^{j_{2t}})^2 & \dots & (\alpha^{j_{2t}})^{2t} \end{vmatrix} = 0 \quad 0 \leq j_1 < j_2 < \dots < j_{2t} \leq n - 1 \quad (7.9)$$

However, the determinant  $D$  of Eq. (7.9) is called a Van der Monde determinant, which is actually nonzero. This clearly contradicts our assumption of Eq. (7.8).

We can factor out  $\alpha^{j_\lambda}$ ,  $\lambda = 1, 2, \dots, 2t$ , to obtain

$$D = \alpha^{(j_1 + j_2 + \dots + j_{2t})} \begin{vmatrix} 1 & \alpha^{j_1} & \dots & (\alpha^{j_1})^{2t-1} \\ 1 & \alpha^{j_2} & \dots & (\alpha^{j_2})^{2t-1} \\ \vdots & \vdots & & \vdots \\ 1 & \alpha^{j_{2t}} & \dots & (\alpha^{j_{2t}})^{2t-1} \end{vmatrix} = \alpha^{(j_1 + j_2 + \dots + j_{2t})} \prod_{k < l} (\alpha^{j_l} - \alpha^{j_k}) \neq 0 \quad (7.10)$$

Therefore, the product on the left in Eq. (7.10) cannot be zero; hence any set of  $d_0 - 1$  columns is linearly independent. Thus, our assumption that there exists a nonzero code word  $\mathbf{c}$  of weight  $d_0 - 1 \leq 2t$  is invalid. This implies that the minimum distance of the  $t$ -error-correcting BCH code is at least the designed distance  $d_0 = 2t + 1$ . The true minimum distance of a BCH code may be equal to or greater than the designed distance  $d_0$ . The designed distance  $d_0$  is only a lower bound on the true distance of the code. Thus, the minimum distance of a BCH code with designed distance  $d_0$  is at least  $d_0$ . This is usually called the BCH bound.

**Example 7.5** Consider the triple-error-correcting (31, 16) BCH code with minimum distance  $d_{\min} = 7$ . Let  $\alpha$  be a primitive element of  $GF(2^5)$  with  $\alpha^5 = 1 + \alpha^2$ . Then, by using Eq. (7.6), the parity-check matrix  $\mathbf{H}$  of this code is expressed by

$$\mathbf{H} = \begin{bmatrix} 1 & \alpha^1 & \alpha^2 & \alpha^3 & \alpha^4 & \alpha^5 & \alpha^6 & \alpha^7 & \alpha^8 & \alpha^9 & \alpha^{10} & \alpha^{11} & \alpha^{12} & \alpha^{13} & \alpha^{14} & \alpha^{15} \\ 1 & \alpha^3 & \alpha^6 & \alpha^9 & \alpha^{12} & \alpha^{15} & \alpha^{18} & \alpha^{21} & \alpha^{24} & \alpha^{27} & \alpha^{30} & \alpha^2 & \alpha^5 & \alpha^8 & \alpha^{11} & \alpha^{14} \\ 1 & \alpha^5 & \alpha^{10} & \alpha^{15} & \alpha^{20} & \alpha^{25} & \alpha^{30} & \alpha^4 & \alpha^9 & \alpha^{14} & \alpha^{19} & \alpha^{24} & \alpha^{29} & \alpha^3 & \alpha^8 & \alpha^{13} \\ \alpha^{16} & \alpha^{17} & \alpha^{18} & \alpha^{19} & \alpha^{20} & \alpha^{21} & \alpha^{22} & \alpha^{23} & \alpha^{24} & \alpha^{25} & \alpha^{26} & \alpha^{27} & \alpha^{28} & \alpha^{29} & \alpha^{30} \\ \alpha^{17} & \alpha^{20} & \alpha^{23} & \alpha^{26} & \alpha^{29} & \alpha & \alpha^4 & \alpha^7 & \alpha^{10} & \alpha^{13} & \alpha^{16} & \alpha^{19} & \alpha^{22} & \alpha^{25} & \alpha^{28} \\ \alpha^{18} & \alpha^{23} & \alpha^{28} & \alpha^2 & \alpha^7 & \alpha^{12} & \alpha^{17} & \alpha^{22} & \alpha^{27} & \alpha & \alpha^6 & \alpha^{11} & \alpha^{16} & \alpha^{21} & \alpha^{26} \end{bmatrix} \quad (7.11)$$

Notice that  $\alpha^{31} = 1$  is used in  $\mathbf{H}$ . All the entries of  $\mathbf{H}$  are elements of  $GF(2^5)$ . With the aid of Table 7.2, each entry of  $\mathbf{H}$  is replaced by its corresponding 5-tuple arranged in column form. Thus, we obtain the following  $15 \times 31$  parity-check matrix  $\mathbf{H}$  for the (31, 16) BCH code of length  $n = 2^5 - 1 = 31$ .

$$\mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 1 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 1 & 0 & 1 & 0 & 1 \\ \hline 1 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 1 & 1 \\ \hline 1 & 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 1 \end{bmatrix} \quad (7.12)$$

**Example 7.6** Let  $\alpha$  be a primitive element of  $GF(2^7)$  with  $p(\alpha) = 1 + \alpha^3 + \alpha^7 = 0$ . Construct the Galois field  $GF(2^7)$  and find the generator polynomials of all the BCH codes of block length  $n = 2^7 - 1 = 127$ . Let  $m_i(x)$  denote the minimal polynomials of  $\alpha^i$ . Then  $\alpha, \alpha^2, \alpha^4, \alpha^8, \alpha^{16}, \alpha^{32}$ , and  $\alpha^{64}$  are roots of  $m_1(x)$ , and  $m_1(x) = m_2(x) = m_4(x) = m_8(x) = m_{16}(x) = m_{32}(x) = m_{64}(x)$ . Thus, the minimal polynomial of  $\alpha$  is

$$\begin{aligned} m_1(x) &= (x + \alpha)(x + \alpha^2)(x + \alpha^4)(x + \alpha^8)(x + \alpha^{16})(x + \alpha^{32})(x + \alpha^{64}) \\ &= 1 + x^3 + x^7 \end{aligned}$$

which is identical to the primitive polynomial  $p(x)$  of degree  $m$  over  $GF(2)$ . If we continue the analysis in this way, the minimal polynomial  $m_i(x)$  can be obtained by listing only the exponents of  $\alpha$  as shown in Table 7.4.

Using  $g(x) = LCM\{m_1(x), m_3(x), \dots, m_{2^{t-1}}(x)\}$ , the generator polynomials  $g_t(x)$  with respect to the error-correcting capability  $t$  can be constructed as shown in Table 7.5.

TABLE 7.4

| Exponents of $\alpha$ |                                                                                                | Minimal polynomial                             |
|-----------------------|------------------------------------------------------------------------------------------------|------------------------------------------------|
| 1 2 4 8 16 32 64      | $m_1(x) = m_2(x) = m_4(x)$<br>$= m_8(x) = m_{16}(x) = m_{32}(x)$<br>$= m_{64}(x)$              | $m_1(x) = 1 + x^3 + x^7$                       |
| 3 6 12 24 48 96 65    | $m_3(x) = m_6(x) = m_{12}(x)$<br>$= m_{24}(x) = m_{48}(x)$<br>$= m_{96}(x) = m_{65}(x)$        | $m_3(x) = 1 + x + x^2 + x^3 + x^7$             |
| 5 10 20 40 80 33 66   | $m_5(x) = m_{10}(x) = m_{20}(x)$<br>$= m_{40}(x) = m_{80}(x)$<br>$= m_{33}(x) = m_{66}(x)$     | $m_5(x) = 1 + x^2 + x^3 + x^4 + x^7$           |
| 7 14 28 56 112 97 67  | $m_7(x) = m_{14}(x) = m_{28}(x)$<br>$= m_{56}(x) = m_{112}(x)$<br>$= m_{97}(x) = m_{67}(x)$    | $m_7(x) = 1 + x + x^2 + x^4 + x^5 + x^6 + x^7$ |
| 9 18 36 72 17 34 68   | $m_9(x) = m_{18}(x) = m_{36}(x)$<br>$= m_{72}(x) = m_{17}(x)$<br>$= m_{34}(x) = m_{68}(x)$     | $m_9(x) = 1 + x + x^2 + x^3 + x^4 + x^5 + x^7$ |
| 11 22 44 88 49 98 69  | $m_{11}(x) = m_{22}(x) = m_{44}(x)$<br>$= m_{88}(x) = m_{49}(x)$<br>$= m_{98}(x) = m_{69}(x)$  | $m_{11}(x) = 1 + x^2 + x^4 + x^6 + x^7$        |
| 13 26 52 104 81 35 70 | $m_{13}(x) = m_{26}(x) = m_{52}(x)$<br>$= m_{104}(x) = m_{81}(x)$<br>$= m_{35}(x) = m_{70}(x)$ | $m_{13}(x) = 1 + x + x^7$                      |



TABLE 7.5

| $t$ | Generator polynomial                                                                                                                                                                                                                         | Type of code                                                     |
|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| 1   | $g_1(x) = m_1(x)$<br>$= 1 + x^3 + x^7$                                                                                                                                                                                                       | Single-error-correcting<br>(127, 120) BCH with<br>$d_{\min} = 3$ |
| 2   | $g_2(x) = g_1(x)m_3(x)$<br>$= 1 + x + x^2 + x^4 + x^5 + x^6 + x^8$<br>$+ x^9 + x^{14}$                                                                                                                                                       | Double-error-correcting<br>(127, 113) BCH code                   |
| 3   | $g_3(x) = g_2(x)m_5(x)$<br>$= 1 + x + x^5 + x^6 + x^7 + x^8 + x^{11}$<br>$+ x^{12} + x^{14} + x^{15} + x^{17} + x^{18}$<br>$+ x^{21}$                                                                                                        | Triple-error-correcting<br>(127, 106) BCH code                   |
| 4   | $g_4(x) = g_3(x)m_7(x)$<br>$= 1 + x + x^3 + x^4 + x^5 + x^7 + x^9 + x^{10}$<br>$+ x^{13} + x^{18} + x^{19} + x^{20} + x^{23}$<br>$+ x^{26} + x^{27} + x^{28}$                                                                                | Four-error-correcting<br>(127, 99) BCH code                      |
| 5   | $g_5(x) = g_4(x)m_9(x)$<br>$= 1 + x + x^2 + x^4 + x^6 + x^7 + x^{10}$<br>$+ x^{13} + x^{21} + x^{22} + x^{24} + x^{25}$<br>$+ x^{26} + x^{29} + x^{31} + x^{34} + x^{35}$                                                                    | Five-error-correcting<br>(127, 92) BCH code                      |
| 6   | $g_6(x) = g_5(x)m_{11}(x)$<br>$= 1 + x + x^3 + x^4 + x^5 + x^7 + x^{10}$<br>$+ x^{13} + x^{15} + x^{16} + x^{19} + x^{20}$<br>$+ x^{21} + x^{22} + x^{23} + x^{26} + x^{29}$<br>$+ x^{33} + x^{34} + x^{35} + x^{39} + x^{40}$<br>$+ x^{42}$ | Six-error-correcting (127,<br>85) BCH code                       |
| 7   | $g_7(x) = g_6(x)m_{13}(x)$<br>$= 1 + x^2 + x^3 + x^6 + x^{12} + x^{13} + x^{15}$<br>$+ x^{19} + x^{20} + x^{22} + x^{23} + x^{24}$<br>$+ x^{28} + x^{39} + x^{40} + x^{43} + x^{46}$<br>$+ x^{47} + x^{49}$                                  | Seven-error-correcting<br>(127, 78) BCH code                     |

### 7.3 Decoding Procedure for BCH Codes

There are several algorithms that have been specifically developed for decoding BCH codes. The algorithm studied in this chapter was first developed by Peterson and Berlekamp for binary BCH codes.

#### 7.3.1 Syndrome computation

Suppose that a code word  $c(x)$  is transmitted and that because of the channel error  $e(x)$ , the received word  $r(x)$  is

$$r(x) = c(x) + e(x) \quad (7.13)$$

where  $e(x) = e_0 + e_1x + \dots + e_{n-1}x^{n-1}$  and is called the error pattern. No more than  $t$  coefficients of  $e(x)$  are nonzero. Suppose that  $v$ ,  $1 \leq v \leq t$ , errors actually occur and that they occur in unknown locations  $j_1, j_2, \dots, j_v$ , that is,

$$e(x) = \sum_{\lambda=1}^v x^{j_\lambda} \quad 0 \leq j_\lambda \leq n-1 \quad (7.14)$$

Since  $\alpha, \alpha^2, \dots, \alpha^{2t}$  are roots of each code polynomial,  $c(\alpha^i) = 0$  for  $1 \leq i \leq 2t$ . Therefore, from Eq. (7.13), it follows that

$$r(\alpha^i) = e(\alpha^i) \quad i = 1, 2, \dots, 2t \quad (7.15)$$

As usual, the first step in decoding a code sequence is to compute the syndrome  $s(x)$  from the received polynomial  $r(x)$ . For decoding a  $t$ -error-correcting BCH code, the syndrome of  $2t$ -tuple is related by

$$\mathbf{s} = (s_1, s_2, \dots, s_{2t}) = \mathbf{r} \cdot \mathbf{H}^T \quad (7.16)$$

where  $\mathbf{H}$  is given by Eq. (7.6). Thus, the  $k$ th component of the syndrome is

$$\begin{aligned} s_k &= r(\alpha^k) \\ &= r_0 + r_1\alpha^k + r_2\alpha^{2k} + \dots + r_{n-1}\alpha^{(n-1)k} \quad 1 \leq k \leq 2t \end{aligned} \quad (7.17)$$

Note that the syndrome components are a function of the field elements of  $GF(2^m)$ . Thus, each syndrome component is computed by dividing  $r(x)$  by the minimal polynomial  $m_k(x)$ ,  $1 \leq k \leq 2t$ , of  $\alpha^k$  such that

$$r(x) = q_k(x)m_k(x) + \gamma_k(x) \quad (7.18)$$

The remainder  $\gamma_k(x)$ , where  $x = \alpha^k$ , is the syndrome component  $s_k$  since  $m_k(\alpha^k) = 0$ . Thus, in general, computing  $r(\alpha^i)$  is equivalent to computing  $\gamma_k(\alpha^i)$ . Hence, if it is combined with Eq. (7.15), the syndrome component is expressed as

$$s_k = \gamma_k(\alpha^k) = r(\alpha^k) = e(\alpha^k) \quad 1 \leq k \leq 2t \quad (7.19)$$

from which we see that the syndrome  $\mathbf{s}$  depends only on the error pattern  $\mathbf{e}$ .

All we need to know is how to find the error locations. The error pattern is completely described by the error-location numbers. Substituting  $s_k = e(\alpha^k)$  from Eq. (7.19) into Eq. (7.14), we have

$$s_k = \sum_{\lambda=1}^v (\alpha^{j_\lambda})^k \quad 1 \leq k \leq 2t \quad (7.20)$$

It thus follows that we have a set of equations that relate the syndrome components and the unknown parameters  $\alpha^{j_\lambda}$ ,  $1 \leq \lambda \leq \nu$ .

$$\begin{aligned}
 s_1 &= \alpha^{j_1} + \alpha^{j_2} + \cdots + \alpha^{j_\nu} \\
 s_2 &= (\alpha^{j_1})^2 + (\alpha^{j_2})^2 + \cdots + (\alpha^{j_\nu})^2 \\
 s_3 &= (\alpha^{j_1})^3 + (\alpha^{j_2})^3 + \cdots + (\alpha^{j_\nu})^3 \\
 &\vdots \quad \quad \quad \vdots \quad \quad \quad \vdots \quad \quad \quad \vdots \\
 s_{2t} &= (\alpha^{j_1})^{2t} + (\alpha^{j_2})^{2t} + \cdots + (\alpha^{j_\nu})^{2t}
 \end{aligned} \tag{7.21}$$

where the unknown parameters  $\alpha^{j_\lambda}$ ,  $\lambda = 1, 2, \dots, \nu$ , are called the error-location numbers. Once the  $\alpha^{j_\lambda}$ ,  $1 \leq \lambda \leq \nu$ , have been found, the powers  $j_\lambda$ ,  $\lambda = 1, 2, \dots, \nu$ , give us the error locations in  $e(x)$ . These  $2t$  equations of Eq. (7.21) are known as power-sum symmetric functions. Consequently, the decoding algorithm of the BCH codes is the way to solve these power-sum symmetric functions and to find the unknown numbers  $\alpha^{j_\lambda}$ ,  $\lambda = 1, 2, \dots, \nu$ , from the syndrome components  $s_k$ . In fact, any method of solving Eq. (7.21) is the basis for an error-correction procedure.

**Example 7.7** Consider the single-error-correcting (7, 4) BCH code with minimum distance  $d_{\min} = 3$ . Let  $r(x)$  be the received polynomial. The syndrome consists of a binary 2-tuple because  $2t = 2$ . That is,

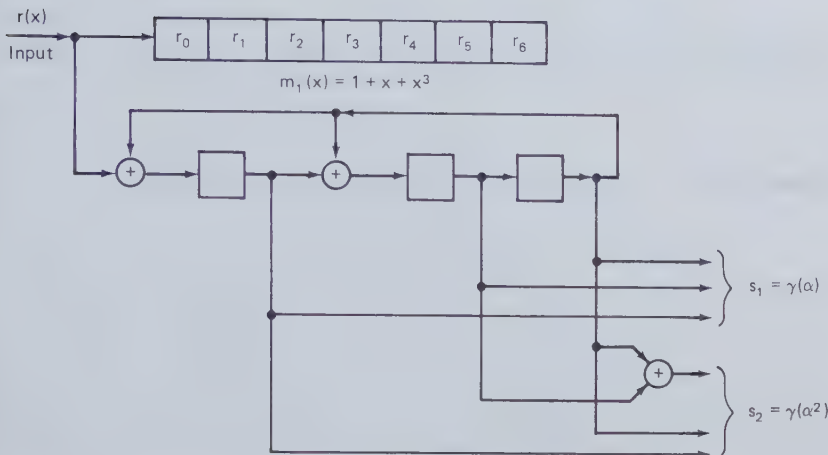
$$\mathbf{s} = (s_1, s_2)$$

Let  $\alpha$  be the primitive element of  $GF(2^3)$ . Since the minimal polynomials for  $\alpha$ ,  $\alpha^2$ , and  $\alpha^4$  are equivalent, we have

$$\begin{aligned}
 m_1(x) &= m_2(x) = m_4(x) = (x + \alpha)(x + \alpha^2)(x + \alpha^4) \\
 &= 1 + x + x^3
 \end{aligned}$$

To compute the syndrome digits, we divide  $r(x)$  by  $m_1(x)$ , and the remainder  $\gamma(x)$  is assumed to be  $\gamma(x) = \gamma_0 + \gamma_1 x + \gamma_2 x^2$ . Substituting  $\alpha$  and  $\alpha^2$  into  $\gamma(x)$ , we obtain

$$\begin{aligned}
 s_1 &= \gamma_0 + \gamma_1 \alpha + \gamma_2 \alpha^2 \\
 s_2 &= \gamma_0 + \gamma_1 \alpha^2 + \gamma_2 \alpha^4 \\
 &= \gamma_0 + \gamma_1 \alpha^2 + \gamma_2 (\alpha + \alpha^2) \\
 &= \gamma_0 + \gamma_2 \alpha + (\gamma_1 + \gamma_2) \alpha^2
 \end{aligned}$$



**Figure 7.1** Syndrome computation circuit for the single-error-correcting (7, 4) BCH code.

respectively. Thus, the syndrome computation circuit for the (7, 4) BCH code can be devised as shown in Fig. 7.1.

Now, assume that the all-zero code word  $\mathbf{c} = (0\ 0\ 0\ 0\ 0\ 0\ 0)$  is transmitted and  $\mathbf{r} = (0\ 0\ 0\ 0\ 0\ 0\ 1)$  is received. The received polynomial is then

$$r(x) = x^6$$

After  $r(x) = x^6$  is divided by  $m_1(x) = 1 + x + x^3$ , the remainder is

$$\gamma(x) = 1 + x^2$$

Substituting  $\alpha$  and  $\alpha^2$  into  $\gamma(x)$  and using the field elements of  $GF(2^3)$ , we obtain

$$s_1 = 1 + \alpha^2 = \alpha^6 = (1\ 0\ 1)$$

$$s_2 = 1 + \alpha + \alpha^2 = \alpha^5 = (1\ 1\ 1)$$

respectively. Thus, the syndrome is

$$\mathbf{s} = (\alpha^6, \alpha^5) = (1\ 0\ 1, 1\ 1\ 1)$$

**Example 7.8** Consider the double-error-correcting (31, 21) BCH code with the designed distance  $d_0 = 5$ . Suppose that the all-zero code word  $\mathbf{c} = (0\ 0\ 0 \cdots 0)$  is transmitted and  $r(x) = x^3 + x^{18}$  is received. Clearly, there are two errors at locations  $x^3$  and  $x^{18}$ . Since  $2t = 4$ , the syndrome is a 4-tuple and each tuple consists of five digits.

$$\mathbf{s} = (s_1, s_2, s_3, s_4)$$

The minimal polynomial of  $\alpha$  is  $m_1(x) = 1 + x^2 + x^5$ . The minimal polynomial of  $\alpha^3$  is  $m_3(x) = 1 + x^2 + x^3 + x^4 + x^5$ . Dividing  $r(x)$  by  $m_1(x)$  gives the remainder

$$\gamma(x) = \gamma_0 + \gamma_1 x + \gamma_2 x^2 + \gamma_3 x^3 + \gamma_4 x^4$$

Using Table 7.2 for  $GF(2^5)$ , the syndrome digits can be obtained as

$$s_1 = \gamma(\alpha) = \gamma_0 + \gamma_1 \alpha + \gamma_2 \alpha^2 + \gamma_3 \alpha^3 + \gamma_4 \alpha^4$$

$$s_2 = \gamma(\alpha^2) = (\gamma_0 + \gamma_4) + \gamma_3 \alpha + (\gamma_1 + \gamma_4) \alpha^2 + (\gamma_3 + \gamma_4) \alpha^3 + \gamma_2 \alpha^4$$

$$s_4 = \gamma(\alpha^4) = (\gamma_0 + \gamma_2 + \gamma_4) + (\gamma_3 + \gamma_4) \alpha + (\gamma_2 + \gamma_3) \alpha^2 + (\gamma_2 + \gamma_3 + \gamma_4) \alpha^3 + (\gamma_1 + \gamma_4) \alpha^4$$

and  $s_3 = \gamma(\alpha^3) = \gamma_0 + (\gamma_2 + \gamma_3 + \gamma_4) \alpha$

$$+ \gamma_4 \alpha^2 + (\gamma_1 + \gamma_2 + \gamma_3 + \gamma_4) \alpha^3 + \gamma_3 \alpha^4$$

Thus, the syndrome generator for the (31, 21) BCH code can be devised as shown in Fig. 7.2.

Since the received word is  $r(x) = x^3 + x^{18}$  when the all-zero code word  $c = 0$  is transmitted, dividing  $r(x) = x^3 + x^{18}$  by  $m_1(x) = 1 + x^2 + x^5$  yields the remainder

$$\gamma_1(x) = 1 + x + x^3$$

The syndrome digits are obtained by substituting  $x = \alpha, \alpha^2$ , and  $\alpha^4$ , respectively, into  $\gamma_1(x)$ . Hence, we have

$$s_1 = \gamma_1(\alpha) = 1 + \alpha + \alpha^3 = \alpha^{27}$$

$$s_2 = \gamma_1(\alpha^2) = 1 + \alpha + \alpha^2 + \alpha^3 = \alpha^{23}$$

$$s_4 = \gamma_1(\alpha^4) = 1 + \alpha + \alpha^2 + \alpha^3 + \alpha^4 = \alpha^{15}$$

Similarly, dividing  $r(x) = x^3 + x^{18}$  by  $m_3(x) = 1 + x^2 + x^3 + x^4 + x^5$  yields the remainder

$$\gamma_3(x) = 1 + x^3 + x^4$$

The syndrome digit for  $x = \alpha^3$  becomes

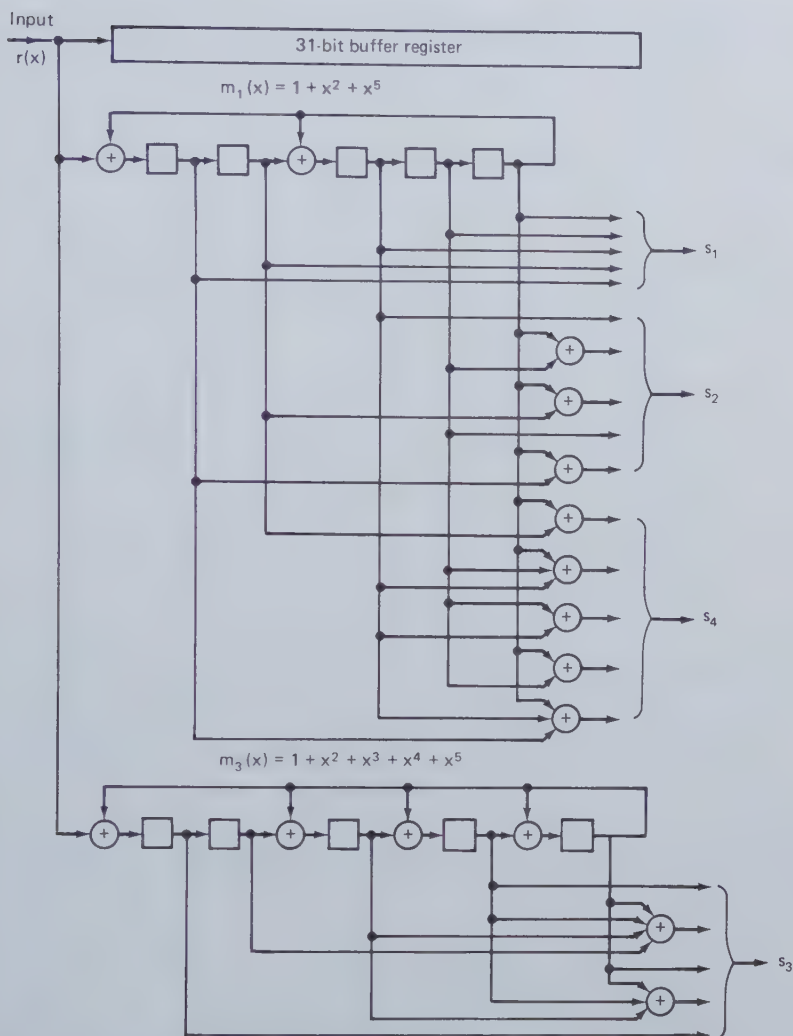
$$s_3 = \gamma_3(\alpha^3) = 1 + \alpha^2 + \alpha^4 = \alpha^{22}$$

Thus, the syndrome is expressed as

$$\begin{aligned} \mathbf{s} &= (\alpha^{27}, \alpha^{23}, \alpha^{22}, \alpha^{15}) \\ &= (1 \ 1 \ 0 \ 1 \ 0, \ 1 \ 1 \ 1 \ 1 \ 0, \ 1 \ 0 \ 1 \ 0 \ 1, \ 1 \ 1 \ 1 \ 1 \ 1) \end{aligned}$$

**Example 7.9** Consider the triple-error-correcting (15, 5) BCH code with the minimum distance  $d_{\min} = 7$ . Construct the field elements of  $GF(2^4)$  first, and ob-





**Figure 7.2** Syndrome computation circuit for the double-error-correcting (31, 21) BCH code.

tain the minimal polynomials  $m_i(x)$  for  $\alpha^i$ ,  $1 \leq i \leq 2t - 1$ . The minimal polynomials  $m_i(x)$  of  $\alpha^i$ ,  $i = 1, 3, 5$ , are

$$m_1(x) = 1 + x + x^4$$

$$m_3(x) = 1 + x + x^2 + x^3 + x^4$$

and

$$m_5(x) = 1 + x + x^2$$

Let  $r(x)$  be the received polynomial. If  $r(x)$  is divided by  $m_i(x)$ , the remainder  $\gamma(x)$  is as follows:

$$\gamma(x) = \gamma_0 + \gamma_1 x + \gamma_2 x^2 + \gamma_3 x^3$$

Since  $2t = 6$ , the syndrome consists of a 6-tuple:

$$\mathbf{s} = (s_1, s_2, s_3, s_4, s_5, s_6)$$

Let us compute the syndrome components  $s_i$  for  $1 \leq i \leq 6$  as follows:

$$s_1 = \gamma_1(\alpha) = \gamma_0 + \gamma_1 \alpha + \gamma_2 \alpha^2 + \gamma_3 \alpha^3$$

$$s_2 = \gamma_1(\alpha^2) = (\gamma_0 + \gamma_2) + \gamma_2 \alpha + (\gamma_1 + \gamma_3) \alpha^2 + \gamma_3 \alpha^3$$

$$s_4 = \gamma_1(\alpha^4) = (\gamma_0 + \gamma_1 + \gamma_2 + \gamma_3) + (\gamma_1 + \gamma_3) \alpha + (\gamma_2 + \gamma_3) \alpha^2 + \gamma_3 \alpha^3$$

$$s_3 = \gamma_3(\alpha^3) = \gamma_0 + \gamma_3 \alpha + \gamma_2 \alpha^2 + (\gamma_1 + \gamma_2 + \gamma_3) \alpha^3$$

$$s_6 = \gamma_3(\alpha^6) = (\gamma_0 + \gamma_2) + \gamma_2 \alpha + (\gamma_1 + \gamma_2) \alpha^2 + (\gamma_1 + \gamma_2 + \gamma_3) \alpha^3$$

$$s_5 = \gamma_5(\alpha^5) = \gamma_0 + \gamma_1 \alpha + \gamma_1 \alpha^2$$

Thus, the syndrome generator for the (15, 5) BCH code can be devised as shown in Fig. 7.3.

Suppose that the all-zero code word  $\mathbf{c} = (0\ 0\ 0\ \dots\ 0)$  is transmitted and  $\mathbf{r} = (0\ 1\ 0\ 0\ 0\ 0\ 1\ 0\ 0\ 0\ 0\ 0\ 1\ 0\ 0)$  is received. Then, the received polynomial is

$$r(x) = x + x^6 + x^{12}$$

Hence, there are three errors at locations  $x$ ,  $x^6$ , and  $x^{12}$ . If  $r(x)$  is divided by  $m_1(x)$ , the remainder is

$$\gamma_1(x) = 1$$

and if  $\alpha$ ,  $\alpha^2$ , and  $\alpha^4$  are substituted into  $\gamma_1(x)$ , we obtain

$$s_1 = s_2 = s_4 = 1$$

If  $r(x)$  is divided by  $m_3(x)$ , we obtain the remainder

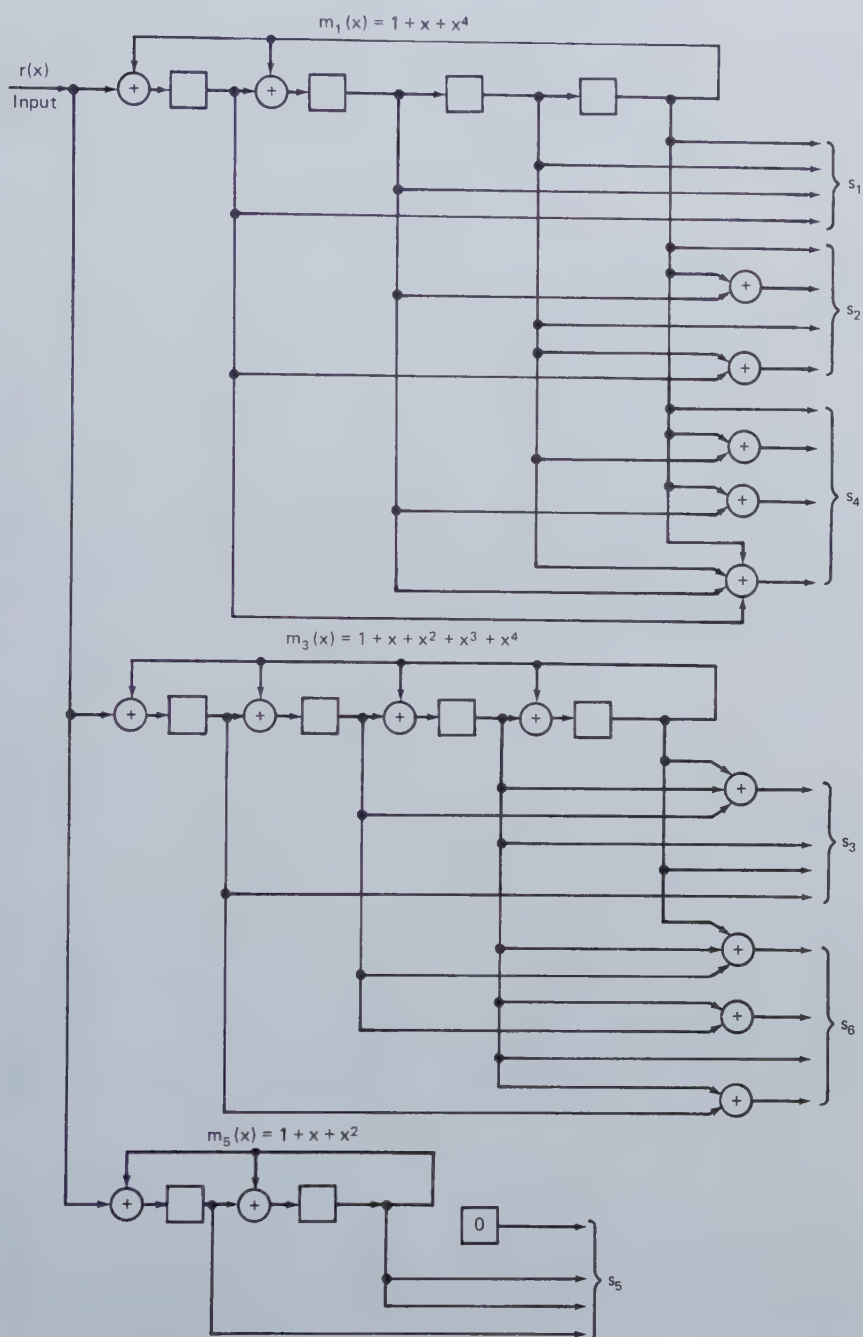
$$\gamma_3(x) = x^2$$

and if  $\alpha^3$  and  $\alpha^6$  are substituted into  $\gamma_3(x)$ , we obtain

$$s_3 = \alpha^6 \quad s_6 = \alpha^{12}$$

Finally, if  $r(x)$  is divided by  $m_5(x)$ , the remainder is

$$\gamma_5(x) = x$$



**Figure 7.3** Syndrome computation circuit for the triple-error-correcting (15, 5) BCH code.

If  $\alpha^5$  is substituted into  $\gamma_5(x)$ , we obtain the syndrome digit

$$s_5 = \alpha^5$$

Thus, the 6-tuple syndrome is obtained as

$$\mathbf{s} = (1, 1, \alpha^6, 1, \alpha^5, \alpha^{12})$$

### 7.3.2 The error-locator polynomial

Suppose that  $\nu \leq t$  errors actually occur. Let the polynomial  $\sigma(x)$  be

$$\begin{aligned}\sigma(x) &= \sigma_0 + \sigma_1 x + \sigma_2 x^2 + \cdots + \sigma_\nu x^\nu \\ &= (1 + \beta_1 x)(1 + \beta_2 x) \cdots (1 + \beta_\nu x)\end{aligned}\quad (7.22)$$

by letting  $\beta_\lambda = \alpha^{j_\lambda}$  for simplicity. The roots of  $\sigma(x)$  are then  $\beta_1^{-1}, \beta_2^{-1}, \dots, \beta_\nu^{-1}$ , which are the inverses of the error-location numbers  $\alpha^{-j_\lambda}$ ,  $\lambda = 1, 2, \dots, \nu$ . This unknown polynomial  $\sigma(x)$  is called the error-locator polynomial; its coefficients and the error-location numbers are related by the following set of equations:

$$\begin{aligned}\sigma_0 &= 1 \\ \sigma_1 &= \beta_1 + \beta_2 + \cdots + \beta_\nu \\ \sigma_2 &= \beta_1\beta_2 + \beta_2\beta_3 + \cdots + \beta_{\nu-1}\beta_\nu \\ &\vdots \\ \sigma_\nu &= \beta_1\beta_2 \cdots \beta_\nu\end{aligned}\quad (7.23)$$

The  $\sigma_i$ ,  $0 \leq i \leq \nu$ , are known as the elementary symmetric functions of  $\beta_i$ 's.

Since  $\alpha^k$  is a root of the generator polynomial, the decoder may immediately compute the sum of the  $k$ th powers of the error-location numbers by evaluating the received polynomial  $r(x)$  at  $x = \alpha^k$ . This process results in

$$r(\alpha^k) = e(\alpha^k) = s_k = \sum_{\lambda=1}^{\nu} (\beta_\lambda)^k$$

Given the syndrome components  $s_k$ 's, the decoder must then attempt to solve for the error-locator polynomial  $\sigma(x)$  whose reciprocal roots give the error locations. The relation between  $\sigma(x)$  and the  $s_j$ 's is given by Newton's identities, which may be readily derived by relating the  $\sigma_i$ 's,  $0 \leq i \leq \nu$ , in Eq. (7.23) and the syndrome components  $s_j$ 's,  $1 \leq j \leq 2t$ , in Eq. (7.21) as demonstrated in the following discussion. If it is

possible to determine the coefficients of  $\sigma(x)$ , i.e.,  $\sigma_1, \sigma_2, \dots, \sigma_v$  from Newton's identities, the error-location numbers  $\beta_\lambda$ 's,  $1 \leq \lambda \leq v$ , can be found by determining the roots of the error-locator polynomial  $\sigma(x)$ .

Differentiating  $\sigma(x)$  with respect to  $x$  once, we get

$$\begin{aligned}\sigma'(x) &= \sigma_1 + 2\sigma_2x + 3\sigma_3x^2 + \dots + v\sigma_vx^{v-1} \\ &= \beta_1 \frac{\sigma(x)}{1 + \beta_1x} + \beta_2 \frac{\sigma(x)}{1 + \beta_2x} + \dots + \beta_v \frac{\sigma(x)}{1 + \beta_vx}\end{aligned}\quad (7.24)$$

Since  $1 + \beta_1x$  is a factor of  $\sigma(x)$ , Eq. (7.22) can be written as

$$\begin{aligned}\sigma(x) &= (1 + \beta_1x)[1 + (\beta_1 + \sigma_1)x + (\beta_1^2 + \sigma_1\beta_1 + \sigma_2)x^2 + \dots \\ &\quad + (\beta_1^{v-1} + \sigma_1\beta_1^{v-2} + \sigma_2\beta_1^{v-3} + \dots + \sigma_{v-2}\beta_1 + \sigma_{v-1})x^{v-1}]\end{aligned}\quad (7.25)$$

Therefore, the first term of Eq. (7.24) becomes

$$\begin{aligned}\beta_1 \frac{\sigma(x)}{1 + \beta_1x} &= \beta_1 + (\beta_1^2 + \sigma_1\beta_1)x + (\beta_1^3 + \sigma_1\beta_1^2 + \sigma_2\beta_1)x^2 + \dots \\ &\quad + (\beta_1^v + \sigma_1\beta_1^{v-1} + \sigma_2\beta_1^{v-2} + \dots + \sigma_{v-2}\beta_1^2 + \sigma_{v-1}\beta_1)x^{v-1}\end{aligned}$$

The second term of Eq. (7.24) is

$$\begin{aligned}\beta_2 \frac{\sigma(x)}{1 + \beta_2x} &= \beta_2 + (\beta_2^2 + \sigma_1\beta_2)x + (\beta_2^3 + \sigma_1\beta_2^2 + \sigma_2\beta_2)x^2 + \dots \\ &\quad + (\beta_2^v + \sigma_1\beta_2^{v-1} + \sigma_2\beta_2^{v-2} + \dots + \sigma_{v-2}\beta_2^2 + \sigma_{v-1}\beta_2)x^{v-1}\end{aligned}$$

Similarly, the  $v$ th term of Eq. (7.24) is expressed as

$$\begin{aligned}\beta_v \frac{\sigma(x)}{1 + \beta_vx} &= \beta_v + (\beta_v^2 + \sigma_1\beta_v)x + (\beta_v^3 + \sigma_1\beta_v^2 + \sigma_2\beta_v)x^2 + \dots \\ &\quad + (\beta_v^v + \sigma_1\beta_v^{v-1} + \sigma_2\beta_v^{v-2} + \dots + \sigma_{v-2}\beta_v^2 + \sigma_{v-1}\beta_v)x^{v-1}\end{aligned}$$

Adding these  $v$  equations into a single form, we obtain

$$\begin{aligned}\beta_1 \frac{\sigma(x)}{1 + \beta_1x} + \beta_2 \frac{\sigma(x)}{1 + \beta_2x} + \dots + \beta_v \frac{\sigma(x)}{1 + \beta_vx} &= (\beta_1 + \beta_2 + \dots + \beta_v) \\ &\quad + [(\beta_1^2 + \beta_2^2 + \dots + \beta_v^2) + \sigma_1(\beta_1 + \beta_2 + \dots + \beta_v)]x \\ &\quad + [(\beta_1^3 + \beta_2^3 + \dots + \beta_v^3) + \sigma_1(\beta_1^2 + \beta_2^2 + \dots + \beta_v^2) \\ &\quad + \sigma_2(\beta_1 + \beta_2 + \dots + \beta_v)]x^2 + \dots + [(\beta_1^v + \beta_2^v + \dots + \beta_v^v) \\ &\quad + \sigma_1(\beta_1^{v-1} + \beta_2^{v-1} + \dots + \beta_v^{v-1}) + \sigma_2(\beta_1^{v-2} + \beta_2^{v-2} \\ &\quad + \dots + \beta_v^{v-2}) + \dots + \sigma_{v-1}(\beta_1 + \beta_2 + \dots + \beta_v)]x^{v-1}\end{aligned}\quad (7.26)$$



Equation (7.26) can also be written into the following form with the aid of Eqs. (7.21), (7.23), and (7.24):

$$\sigma'(x) = s_1 + (s_2 + \sigma_1 s_1)x + (s_3 + \sigma_1 s_2 + \sigma_2 s_1)x^2 + \cdots + (s_\nu + \sigma_1 s_{\nu-1} + \cdots + \sigma_{\nu-2} s_2 + \sigma_{\nu-1} s_1)x^{\nu-1} \quad (7.27)$$

Comparing Eq. (7.24), term by term, with Eq. (7.26), the following identities are obtained:

$$\begin{aligned} s_1 + \sigma_1 &= 0 \\ s_2 + \sigma_1 s_1 + 2\sigma_2 &= 0 \\ s_3 + \sigma_1 s_2 + \sigma_2 s_1 + 3\sigma_3 &= 0 \\ &\vdots \\ s_\nu + \sigma_1 s_{\nu-1} + \sigma_2 s_{\nu-2} + \cdots + \sigma_{\nu-2} s_2 + \sigma_{\nu-1} s_1 + \nu\sigma_\nu &= 0 \end{aligned} \quad (7.28)$$

where  $k\sigma_k = \sigma_k$  for the binary case only when  $k$  is odd, and it is zero otherwise. The equations of Eq. (7.28) are called Newton's identities. If the  $\sigma_i$ ,  $1 \leq i \leq \nu$ , are determined from Eq. (7.28),  $\beta_\lambda$ ,  $1 \leq \lambda \leq \nu$ , can be found by evaluating the roots of  $\sigma(x)$ . We can generally express the identities of Eq. (7.28) into the following single form:

$$s_k + \sigma_1 s_{k-1} + \cdots + \sigma_{k-1} s_1 + k\sigma_k = 0 \quad k = 1, 2, \dots, \nu \quad (7.29)$$

Consider the case for  $k > \nu$ . Multiplying  $\sigma(x)$  from Eq. (7.22) through by  $x^{-k}$ , we have

$$x^{-k}\sigma(x) = x^{-k} + \sigma_1 x^{1-k} + \sigma_2 x^{2-k} + \cdots + \sigma_{\nu-1} x^{\nu-1-k} + \sigma_\nu x^{\nu-k} \quad (7.30)$$

Substituting the roots of  $\sigma(x)$  (i.e.,  $\beta_1^{-1}, \beta_2^{-1}, \dots, \beta_\nu^{-1}$ ) into Eq. (7.30), it follows, respectively:

$$\begin{aligned} \beta_1^k + \sigma_1 \beta_1^{k-1} + \sigma_2 \beta_1^{k-2} + \cdots + \sigma_{\nu-1} \beta_1^{k-\nu+1} + \sigma_\nu \beta_1^{k-\nu} &= 0 \\ \beta_2^k + \sigma_1 \beta_2^{k-1} + \sigma_2 \beta_2^{k-2} + \cdots + \sigma_{\nu-1} \beta_2^{k-\nu+1} + \sigma_\nu \beta_2^{k-\nu} &= 0 \\ &\vdots \\ \beta_\nu^k + \sigma_1 \beta_\nu^{k-1} + \sigma_2 \beta_\nu^{k-2} + \cdots + \sigma_{\nu-1} \beta_\nu^{k-\nu+1} + \sigma_\nu \beta_\nu^{k-\nu} &= 0 \end{aligned} \quad (7.31)$$

Adding these  $\nu$  equations in Eq. (7.31) term by term yields

$$\begin{aligned} (\beta_1^k + \beta_2^k + \cdots + \beta_\nu^k) + \sigma_1(\beta_1^{k-1} + \beta_2^{k-1} + \cdots + \beta_\nu^{k-1}) + \cdots \\ + \sigma_\nu(\beta_1^{k-\nu} + \beta_2^{k-\nu} + \cdots + \beta_\nu^{k-\nu}) = 0 \end{aligned} \quad (7.32)$$

If we express Eq. (7.32) in terms of the syndrome components, then

$$s_k + \sigma_1 s_{k-1} + \cdots + \sigma_{v-1} s_{k-v+1} + \sigma_v s_{k-v} = 0 \quad k > v \quad (7.33)$$

In particular, for  $k = v + 1$ , we get

$$s_{v+1} + \sigma_1 s_v + \sigma_2 s_{v-1} + \cdots + \sigma_{v-1} s_2 + \sigma_v s_1 = 0 \quad (7.34)$$

In such a manner, Newton's identities can be extended. As a result, we have seen that the  $\sigma_i$ ,  $0 \leq i \leq v$ , are closely related to the syndrome components  $s_j$ ,  $1 \leq j \leq v + 1$ . Therefore, if it is possible to determine  $\sigma_i$ , then the error-location numbers  $\beta_\lambda = \alpha^{j_\lambda}$ ,  $1 \leq \lambda \leq v$ , can be found by determining the roots of  $\sigma(x)$  of minimal degree. This  $\sigma(x)$  will produce an error pattern  $e(x)$  with a minimum number of errors. Hence, if  $v \leq t$  errors occurred,  $\sigma(x)$  will give the actual error pattern  $e(x)$ .

Finally, the error-correcting procedure for the BCH codes can be outlined as follows:

1. Compute the syndrome components  $s_j$ ,  $1 \leq j \leq 2t$ , from the received polynomial  $r(x)$ .
2. Determine the error-locator polynomial  $\sigma(x)$  from the syndrome components  $s_j$ ,  $0 \leq j \leq 2t$ .
3. Set each  $\sigma_i = 0$ ,  $v + 1 \leq i \leq t$ , and solve the first  $v$  equations from Eq. (7.28) for  $\sigma_i$ ,  $1 \leq i \leq v$ .
4. Determine the error-location numbers  $\beta_1, \beta_2, \dots, \beta_v$  by finding the roots of  $\sigma(x)$ , and correct the errors in  $r(x)$ .

## 7.4 Decoding Algorithm for $t = 2$ or Fewer Errors

Consider a double-error-correcting BCH code with block length  $n$  and rate  $k/n$ . Suppose that the all-zero code word is transmitted and that the two errors occur in the received word  $r(x)$  at positions  $x^{j_1}$  and  $x^{j_2}$ . Then  $r(x)$  is clearly equal to  $e(x)$  such that

$$r(x) = e(x) = x^{j_1} + x^{j_2} \quad (7.35)$$

Thus, the two syndrome components can be computed from  $r(x)$  as follows:

$$\begin{aligned} s_1 &= r(\alpha) = \alpha^{j_1} + \alpha^{j_2} \\ s_3 &= r(\alpha^3) = (\alpha^{j_1})^3 + (\alpha^{j_2})^3 \end{aligned} \quad (7.36)$$

If we replace  $\alpha^{j_1}$  and  $\alpha^{j_2}$  by  $\beta_1$  and  $\beta_2$ , respectively, Eq. (7.36) becomes

$$s_1 = \beta_1 + \beta_2 \quad s_3 = \beta_1^3 + \beta_2^3 \quad (7.37)$$

These two power sums are called the decoder's equations.

Since the error-locator polynomial  $\sigma(x)$  is defined as

$$\begin{aligned} \sigma(x) &= (1 + \beta_1 x)(1 + \beta_2 x) \\ &= 1 + (\beta_1 + \beta_2)x + \beta_1\beta_2x^2 \end{aligned} \quad (7.38)$$

the coefficient of  $x$  in  $\sigma(x)$  is

$$\beta_1 + \beta_2 = s_1$$

and the coefficient of  $x^2$  is obtained from the following manipulation:

$$\begin{aligned} \beta_1^3 + \beta_2^3 &= (\beta_1 + \beta_2)(\beta_1^2 + \beta_1\beta_2 + \beta_2^2) \\ &= s_1(\beta_1\beta_2 + s_1^2) \\ &= s_3 \end{aligned}$$

from which we get

$$\beta_1\beta_2 = s_1^2 + \frac{s_3}{s_1} \quad \text{for } s_1 \neq 0 \quad s_3 \neq s_1^3 \quad (7.39)$$

Thus, once the coefficients of  $\sigma(x)$  are determined from the syndrome components, the error-locator polynomial for the double-error-correcting BCH code is expressed by

$$\sigma(x) = 1 + s_1x + \left(s_1^2 + \frac{s_3}{s_1}\right)x^2 \quad (7.40)$$

where  $s_1 \neq 0$  and  $s_3 \neq s_1^3$ . Thus, the errors may be located by finding the roots  $\alpha^i$ ,  $i = 1, 2, \dots, 2^m - 1$ , of  $\sigma(x)$  to satisfy  $\sigma(\alpha^i) = 0$ .

Consider the case where only one error occurs at  $r(x) = x^{j_1}$  when  $c(x) = 0$  is transmitted. The syndrome components are computed as

$$\begin{aligned} s_1 &= \gamma(\alpha) = \alpha^{j_1} = \beta_1 \\ s_3 &= \gamma(\alpha^3) = (\alpha^{j_1})^3 = \beta_1^3 \end{aligned}$$

from which we have

$$s_3 = s_1^3 \quad (7.41)$$

Hence, the error-locator polynomial for the lone error is

$$\sigma(x) = 1 + \beta_1 x = 1 + s_1 x \quad (7.42)$$

only when  $s_1 \neq 0$  and  $s_3 = s_1^3$ .

Finally, if no errors in  $r(x)$  exist, the decoder generates no syndromes, i.e.,  $s_1 = 0$  and  $s_3 = 0$ . Therefore, the error-locator polynomial simply becomes

$$\sigma(x) = 1 \quad (7.43)$$

As a result, the algorithm for finding  $\sigma(x)$  for the error correction of  $t = 2$  or fewer random errors is summarized as follows.

1.  $\sigma(x) = 1$ ,  $s_1 = s_3 = 0$  for no errors.
2.  $\sigma(x) = 1 + s_1 x$ ,  $s_1 \neq 0$ ,  $s_3 = s_1^3$ , for a lone error.
3.  $\sigma(x) = 1 + s_1 x + [s_1^2 + (s_3/s_1)]x^2$ ,  $s_1 \neq 0$ ,  $s_3 \neq s_1^3$ , for double errors.

Thus far, it is clear that the decoder can determine the error-locator polynomial  $\sigma(x)$  if there exist no more than two errors. For three or more errors, this decoding algorithm will decode incorrectly or fail to decode. If  $s_1 = 0$  but  $s_3 \neq 0$ , the decoding algorithm for a double-error correction fails. This deficiency can easily be remedied with Berlekamp's iterative algorithm, which is discussed below. The flow diagram for BCH correction of  $t = 2$  or fewer errors is shown in Fig. 7.4.

**Example 7.10** Consider the single-error-correcting (31, 26) BCH code that was partly discussed in Example 7.4. Suppose that a lone error has occurred at  $r(x) = x^{21}$  when  $c(x) = 0$  is transmitted. Then, we have  $r(x) = e(x) = x^{21}$ . The minimal polynomial of  $\alpha$  is  $m_1(x) = 1 + x^2 + x^5$ . Dividing  $r(x) = x^{21}$  by  $m_1(x) = 1 + x^2 + x^5$ , we obtain the remainder as  $\gamma_1(x) = x^3 + x^4$ . Thus the syndrome component is

$$s_1 = \gamma_1(\alpha) = \alpha^3 + \alpha^4 = \alpha^{21}$$

According to the decoding algorithm for a lone error, the error-locator polynomial becomes

$$\sigma(x) = 1 + s_1 x = 1 + \alpha^{21} x$$

Since  $s_1 = \beta_1 = \alpha^{21} \neq 0$ , the error pattern can be immediately written as

$$e(x) = x^{21}$$

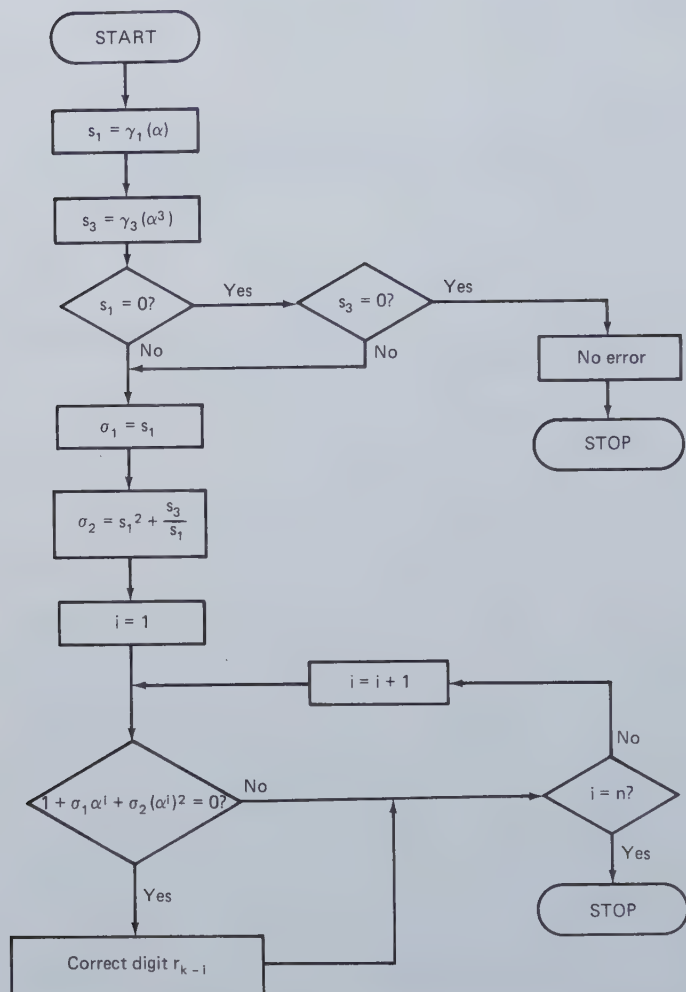


Figure 7.4 Flow diagram for the double-error-correcting BCH code.

Next, let us check the condition for a lone error,  $s_3 = s_1^3$ . Dividing  $r(x) = x^{21}$  by  $m_3(x) = 1 + x^2 + x^3 + x^4 + x^5$  for computing  $s_3$ , we have the remainder  $\gamma_3(x) = x + x^2$ . Then, the syndrome component is

$$s_3 = \gamma_3(\alpha^3) = \alpha^3 + \alpha^6 = \alpha$$

Therefore, we have proved that  $s_3 = s_1^3 = (\alpha^{21})^3 = \alpha$ .

**Example 7.11** Consider the double-error-correcting (31, 21) BCH code given in Example 7.8. Suppose that  $r(x) = x^9 + x^{16}$  is received when the all-zero code



word is transmitted. If  $r(x) = x^9 + x^{15}$  is divided by  $m_1(x) = 1 + x^2 + x^5$ , the remainder is  $\gamma_1(x) = 1 + x^2$ . The syndrome component  $s_1$  is computed as

$$s_1 = \gamma_1(\alpha) = 1 + \alpha^2 = \alpha^5$$

If  $r(x) = x^9 + x^{15}$  is divided by  $m_3(x) = 1 + x^2 + x^3 + x^4 + x^5$ , the remainder is obtained as  $\gamma_3(x) = x + x^2 + x^3 + x^4$ . Thus, the syndrome component  $s_3$  is then

$$s_3 = \gamma_3(\alpha^3) = \alpha^6 + \alpha^9 + \alpha^{12} = \alpha^{28}$$

Therefore, the error-locator polynomial becomes

$$\begin{aligned} \sigma(x) &= 1 + \alpha^5 x + \left( \alpha^{10} + \frac{\alpha^{28}}{\alpha^5} \right) x^2 \\ &= 1 + \alpha^5 x + \alpha^{24} x^2 \end{aligned}$$

Since  $\alpha^{16}$  and  $\alpha^{22}$  are the only roots for  $\sigma(x) = 0$ , the error-locator polynomial is also expressed as

$$\begin{aligned} \sigma(x) &= (x + \alpha^{16})(x + \alpha^{22}) \\ &= (1 + \beta_1 x)(1 + \beta_2 x) \end{aligned}$$

Hence, the error-location numbers are

$$\beta_1 = \frac{1}{\alpha^{16}} = \alpha^{15} \quad \beta_2 = \frac{1}{\alpha^{22}} = \alpha^9$$

Thus, the error pattern becomes

$$e(x) = x^9 + x^{15}$$

as expected.

**Example 7.12** Consider the double-error-correcting (127, 113) BCH code given in Example 7.6. Under the assumption that the received polynomial  $r(x) = x^{32} + x^{78}$  is given, the syndrome consists of four components,

$$\begin{aligned} \mathbf{s} &= (s_1, s_2, s_3, s_4) \\ &= (\alpha^{10}, \alpha^{20}, \alpha^3, \alpha^{40}) \end{aligned}$$

Since  $s_1 = \alpha^{10} \neq 0$  and  $s_1^3 = (\alpha^{10})^3 = \alpha^{30} \neq s_3$ , the error-locator polynomial is

$$\begin{aligned} \sigma(x) &= 1 + \alpha^{10} x + \left( \alpha^{20} + \frac{\alpha^3}{\alpha^{10}} \right) x^2 \\ &= 1 + \alpha^{10} x + \alpha^{110} x^2 \end{aligned}$$

Since  $\sigma(x) = 0$  only for  $x = \alpha^{49}$  and  $x = \alpha^{95}$ , they are the roots of  $\sigma(x)$ . Therefore, the error-locator polynomial becomes

$$\begin{aligned}\sigma(x) &= (x + \alpha^{49})(x + \alpha^{95}) \\ &= (1 + \beta_1 x)(1 + \beta_2 x)\end{aligned}$$

from which the error-location numbers can be obtained as

$$\beta_1 = \frac{1}{\alpha^{49}} = \alpha^{78} \quad \text{and} \quad \beta_2 = \frac{1}{\alpha^{95}} = \alpha^{32}$$

Thus, the error pattern is determined as

$$e(x) = x^{32} + x^{78}$$

Let a  $(2^m - 1, 2m)$  code be the dual of the double-error-correcting BCH code of length  $2^m - 1$ . If  $B_i$  denotes the weight distribution of this dual code, then  $B_i$  stands for the number of code words of weight  $i$ . The weight distribution of the dual of the double-error-correcting BCH code of length  $2^m - 1$ , where  $m$  is odd, is listed below.

| $i$                     | $B_i$                              |
|-------------------------|------------------------------------|
| 0                       | 1                                  |
| $2^{m-1} - 2^{(m-1)/2}$ | $(2^m - 1)(2^{m-2} + 2^{(m-3)/2})$ |
| $2^{m-1}$               | $(2^m - 1)(2^{m-1} + 1)$           |
| $2^{m-1} + 2^{(m-1)/2}$ | $(2^m - 1)(2^{m-2} - 2^{(m-3)/2})$ |

**Example 7.13** Consider the  $(31, 21)$  double-error-correcting BCH code given in Examples 7.8 and 7.11. The  $(31, 10)$  dual code has the following weight distribution:

| $i$   | 0 | 12  | 16  | 20  |
|-------|---|-----|-----|-----|
| $B_i$ | 1 | 310 | 527 | 186 |

Thus, the weight enumerator of this dual code is

$$B(z) = 1 + 310z^{12} + 527z^{16} + 186z^{20}$$

According to the MacWilliams identity, the weight enumerator of the  $(31, 21)$  code is given by

$$\begin{aligned}A(z) &= 2^{-10}(1+z)^{31}B\left(\frac{1-z}{1+z}\right) \\ &= 2^{-10}[(1+z)^{31} + 310(1-z)^{12}(1+z)^{19} + 527(1-z)^{16}(1+z)^{15} \\ &\quad + 186(1-z)^{20}(1+z)^{11}]\end{aligned}$$

Therefore, the weight distribution of the (31, 21) code is

| $i$   | 0, 27  | 5, 26  | 6, 25   | 7, 24   | 8, 23   | 9, 22   |
|-------|--------|--------|---------|---------|---------|---------|
| $A_i$ | 1      | 186    | 806     | 2635    | 7905    | 18,910  |
| $i$   | 10, 21 | 11, 20 | 12, 19  | 13, 18  | 14, 17  | 15, 16  |
| $A_i$ | 41,602 | 85,560 | 142,600 | 195,300 | 251,100 | 301,971 |

## 7.5 Decoding Algorithm for Multiple ( $t > 2$ ) Errors

The iterative algorithm for finding the error-locator polynomial  $\sigma(x)$  for ( $t > 2$ )-error correction is considered in this section. Let

$$\sigma^{(n)}(x) = \sigma_0^{(n)} + \sigma_1^{(n)}x + \sigma_2^{(n)}x^2 + \cdots + \sigma_{l_n}^{(n)}x^{l_n} \quad l_n \leq n \quad (7.44)$$

be a minimal-degree polynomial determined at the  $n$ th step of the iteration whose coefficients satisfy the first  $n$  of Newton's identities in Eq. (7.28). At the  $n$ th step of the iteration, the decoder considers only the first  $n$  power sums (Newton's identities):

$$\sum_{i=0}^{l_n} s_{j-i} \sigma_i^{(n)} = 0 \quad l_n + 1 \leq j \leq n \quad (7.45)$$

where  $\sigma_0^{(n)} = 1$ .

Now, suppose that at the  $n$ th step the decoder has determined  $\sigma^{(n)}(x)$  with minimal degree  $l_n$  so that Eq. (7.45) is satisfied. At the  $(n + 1)$ th step, the decoder seeks to find the next minimal solution  $\sigma^{(n+1)}(x)$  of lowest degree so that the following equations are satisfied:

$$\sum_{i=0}^{l_{n+1}} s_{j-i} \sigma_i^{(n+1)} = 0 \quad l_{n+1} + 1 \leq j \leq n + 1 \quad (7.46)$$

Upon reaching step  $\nu$ , if the minimal-degree polynomial of degree  $\nu$  is the only possible solution, then this solution polynomial remains unchanged for  $n$  greater than  $\nu$  and less than or equal to  $t$ . Let

$$d_n \triangleq s_{n+1} + s_n \sigma_1^{(n)} + s_{n-1} \sigma_2^{(n)} + \cdots + s_{n+1-l_n} \sigma_{l_n}^{(n)} \quad (7.47)$$

be the  $n$ th discrepancy.

If  $d_n = 0$ , then it follows that  $\sigma^{(n+1)}(x) = \sigma^{(n)}(x)$ , which means that the coefficients of  $\sigma^{(n)}(x)$  satisfy the  $(n + 1)$ th Newton's identity of Eq. (7.28). Hence, since  $\sigma^{(n)}(x)$  is a minimal solution at the  $n$ th step, it will also be a minimal solution at the  $(n + 1)$ th step of the iteration. On the other hand, if  $d_n \neq 0$ , the coefficients of  $\sigma^{(n)}(x)$  do not satisfy the  $(n + 1)$ th Newton's identity, and the determination of  $\sigma^{(n+1)}(x)$  from

$\sigma^{(n)}(x)$  is not so straightforward. Therefore, a correction term must be added to  $\sigma^{(n)}(x)$  to form  $\sigma^{(n+1)}(x)$ . To accomplish this correction, the following three theorems (after Peterson and Weldon, 1972) will be helpful for the determination of  $\sigma^{(n+1)}(x)$ .

**Theorem 7.1** Let  $\sigma^{(n)}(x)$  be a minimal-degree polynomial of degree  $l_n \leq n$  at the  $n$ th step of the iteration, and it has the  $n$ th discrepancy  $d_n \neq 0$ . Let

$$\sigma^{(m)}(x) = 1 + \sigma_1^{(m)}x + \sigma_2^{(m)}x^2 + \cdots + \sigma_{l_m}^{(m)}x^{l_m} \quad (7.48)$$

be any solution to the first  $m$  power sums,  $1 \leq m < n$ , where the  $m$ th discrepancy  $d_m \neq 0$ . Then, the polynomial

$$\sigma^{(n+1)}(x) = \sigma^{(n)}(x) - d_n d_m^{-1} x^{n-m} \sigma^{(m)}(x) \quad (7.49)$$

is a solution for the first  $n + 1$  power sums.

Furthermore, the degree of  $\sigma^{(n+1)}(x)$  is

$$l_{n+1} = \max[l_n, l_m + n - m] \quad (7.50)$$

**Theorem 7.2** Let  $\sigma^{(n)}(x)$  be a minimal solution polynomial of degree  $l_n$  for the first  $n$  power sums:

$$\sum_{i=0}^{l_n} s_{j-i} \sigma_i^{(n)} = \begin{cases} 0 & l_n + 1 \leq j \leq n \\ d_n \neq 0 & j = n + 1 \end{cases} \quad (7.51)$$

Suppose that  $\sigma^{(n+1)}(x)$  is any solution polynomial of

$$\sum_{i=0}^{l_{n+1}} s_{j-i} \sigma_i^{(n+1)} = 0 \quad l_{n+1} + 1 \leq j \leq n + 1 \quad (7.52)$$

satisfying  $n + 1 - l_{n+1}$  equations. Let's put

$$\sigma^{(n+1)}(x) - \sigma^{(n)}(x) = \alpha x^{n-m} \sigma^{(m)}(x) \quad (7.53)$$

where  $\alpha \neq 0$  and  $\sigma_0^{(m)} = 1$ .

Then, the polynomial  $\sigma^{(m)}(x)$  is a solution polynomial for the first  $m - l_m$  equations and its discrepancy  $d_m \neq 0$ , and its degree is

$$l_m = l_{n+1} - (n - m) \quad (7.54)$$

**Theorem 7.3** Let  $\sigma^{(n)}(x)$  be a minimal polynomial at the  $n$ th step. Let  $\sigma^{(m)}(x)$  be one of the prior minimal solutions for the first  $m$  power sums,  $1 \leq m \leq n$ , with  $d_m \neq 0$ , such that  $m - l_m$  has the largest value. Then, a minimal solution at the  $(n + 1)$ th step is  $\sigma^{(n+1)}(x)$ , where

$$\sigma^{(n+1)}(x) = \begin{cases} \sigma^{(n)}(x) & l_{n+1} = l_n \quad d_n = 0 \\ \sigma^{(n)}(x) - d_n d_m^{-1} x^{n-m} \sigma^{(m)}(x) & d_n \neq 0 \end{cases} \quad (7.55)$$

$$(7.56)$$

and the degree of  $\sigma^{(n+1)}(x)$  is

$$l_{n+1} = \max[l_n, l_m + n - m] \quad d_n \neq 0 \quad (7.57)$$

These three theorems are proved in Appendixes 7A, 7B, and 7C, respectively.

### 7.5.1 Initial conditions for finding $\sigma(x)$

In order to proceed with the iterative algorithm for finding  $\sigma(x)$ , we begin with the initial conditions. Therefore, let us define the initial conditions as follows. Consider the minimal solution polynomial  $\sigma^{(n)}(x) = 1 + \sigma_1^{(n)}x + \sigma_2^{(n)}x^2 + \dots + \sigma_{l_n}^{(n)}x^{l_n}$  which is determined at the  $n$ th step of iteration.

If no errors occurred,  $\sigma^{(0)}(x) = 1$  is the solution at every step. But if  $t$  errors occurred, where  $1 < t < (d_{\min} - 1)/2$ , then not all the syndrome components  $s_j$ ,  $1 \leq j \leq 2t$ , can be zero. Hence, the resulting vector is certainly not a code word.

Let  $\nu$  be the lowest value of  $j$  for which  $s_j \neq 0$ . Then, the  $(\nu - 1)$ th discrepancy is defined as

$$d_{\nu-1} = s_\nu + s_{\nu-1}\sigma_1^{(\nu-1)} + s_{\nu-2}\sigma_2^{(\nu-1)} + \dots + s_{\nu-l_{\nu-1}}\sigma_{l_{\nu-1}}^{(\nu-1)} \quad (7.58)$$

from which we get  $s_{\nu-1} = s_{\nu-2} = \dots = s_{\nu-l_{\nu-1}} = 0$ . Accordingly, we have  $d_{\nu-1} = s_\nu \neq 0$ . Thus,  $d_{\nu-2} = 0$ ,  $d_{\nu-3} = 0$ ,  $\dots$ ,  $d_2 = 0$ ,  $d_1 = 0$ . When  $d_n = 0$ , by Theorem 7.3 we have  $\sigma^{(n+1)}(x) = \sigma^{(n)}(x)$  and  $l_{n+1} = l_n$ . Consequently, the following relations must hold:

$$\begin{aligned} d_0 &= 0 & \sigma^{(1)}(x) &= \sigma^{(0)}(x) & l_1 &= l_0 = 0 \\ d_1 &= 0 & \sigma^{(2)}(x) &= \sigma^{(1)}(x) & l_2 &= l_1 \\ &\vdots & & & & \\ d_{\nu-2} &= 0 & \sigma^{(\nu-1)}(x) &= \sigma^{(\nu-2)}(x) & l_{\nu-1} &= l_{\nu-2} \end{aligned}$$

Thus, in general, when  $d_i = 0$ ,  $0 \leq i \leq \nu - 2$ , it follows that

$$\sigma^{(0)}(x) = \sigma^{(1)}(x) = \sigma^{(2)}(x) = \dots = \sigma^{(\nu-1)}(x) = 1 \quad (7.59)$$

$$\text{and} \quad l_0 = l_1 = l_2 = \dots = l_{\nu-1} = 0 \quad (7.60)$$



Let us look at the  $\nu$ th Newton's identity

$$s_\nu + \sigma_1^{(\nu)} s_{\nu-1} + \sigma_2^{(\nu)} s_{\nu-2} + \cdots + \sigma_{l_\nu}^{(\nu)} s_{\nu-l_\nu} = 0 \quad (7.61)$$

Since  $s_\nu \neq 0$ , but  $s_{\nu-1} = s_{\nu-2} = \cdots = s_{\nu-l_\nu} = 0$ , there is no polynomial  $\sigma^{(\nu)}(x)$  that satisfies Eq. (7.61). Therefore, at the  $\nu$ th step of the iteration, the decoder considers only the first  $\nu$  power sums and attempts to determine a set of  $\sigma_i^{(\nu)}$ ,  $1 \leq i \leq l_\nu$ , such that  $\nu - l_\nu$  equations are satisfied, while keeping  $l_\nu$  as small as possible. However, it is impossible to have a  $\sigma^{(\nu)}(x)$  that satisfies Eq. (7.61) at step  $\nu$ . In this case,  $\nu - l_\nu = 0$ , i.e.,  $\nu = l_\nu$  and  $l_\nu$  must be taken to be zero. Hence,  $\sigma^{(0)}(x)$  is an arbitrary polynomial of degree  $l_\nu = \nu$  or less. We shall choose  $\sigma^{(\nu)}(x) = 1$ , which is a satisfactory choice. As a result, if we set

$$\begin{aligned} \sigma^{(\nu-1)}(x) &= 1 & l_{\nu-1} &= 0 & d_{\nu-1} &= s_\nu \neq 0 \\ \sigma^{(\nu)}(x) &= 1 & l_\nu &= \nu & d_\nu &= s_{\nu+1} \end{aligned} \quad (7.62)$$

it is possible to apply Theorem 7.3.

By letting  $\nu = 0$  in Eq. (7.62), we can define the initial conditions as follows:

$$\begin{aligned} \sigma^{(-1)}(x) &= 1 & l_{-1} &= 0 & d_{-1} &= 1 \\ \sigma^{(0)}(x) &= 1 & l_0 &= 0 & d_0 &= s_1 \end{aligned} \quad (7.63)$$

By applying Theorem 7.3, we get a minimal polynomial  $\sigma^{(\nu)}(x)$ :

$$\begin{aligned} \sigma^{(\nu)}(x) &= \sigma^{(\nu-1)}(x) - d_{\nu-1} d_{-1}^{-1} x^\nu \sigma^{(-1)}(x) \\ &= 1 - s_\nu x^\nu \end{aligned} \quad (7.64)$$

For  $\nu = 0$ , we see that  $\sigma^{(0)}(x) = 1$  because  $s_0 = 0$ . Therefore, Eq. (7.64) is an acceptable choice for  $\sigma^{(\nu)}(x)$ . The degree of  $\sigma^{(\nu)}(x)$  is then

$$\begin{aligned} l_\nu &= \max[l_{\nu-1}, l_\nu + (\nu - 1) - (-1)] \\ &= \max[l_{\nu-1}, l_{-1} + \nu] = \max[0, \nu] = \nu \end{aligned} \quad (7.65)$$

which is the correct value for  $l_\nu$ .

Once the initial conditions have been set, the overall iterative operation for finding  $\sigma(x)$  can be fully discussed. This iterative algorithm of Peterson and Berlekamp-Massey can be applied not only to binary BCH codes but to nonbinary BCH codes as well.

### 7.5.2 Computation of error-location numbers

The next important decoding step is to find the actual error-location numbers  $\alpha^{n-j_\lambda}$ ,  $0 \leq j_\lambda \leq n-1$ . These numbers are the reciprocals of roots,  $\alpha^{j_\lambda}$  of  $\sigma(x)$ . According to Peterson, this step requires substituting the field elements  $\alpha^i$ ,  $0 \leq i \leq n-1$  ( $n = 2^m - 1$ ), of  $GF(2^m)$  into an error-location polynomial  $\sigma(x)$  of degree  $t$  to determine its roots. But, an easily implemented procedure for finding these roots, and hence the error locations, was given by Chien. Observe that an error occurs at the position  $j_\lambda = l$  if and only if  $\sigma(\alpha^l) = 0$  or  $\sum_{j=0}^t \sigma_j \alpha^{jl} = 0$  where  $\sigma_0 = 1$ . Then, we can see that this fact suggests the implementation of Chien's searching circuit, and this circuit can also be implemented in hardware as shown in Fig. 7.5. Chien's searching circuit requires  $t$  multipliers for multiplying by  $\alpha$ ,  $\alpha^2$ , ...,  $\alpha^t$ , respectively. Initially,  $\sigma_1$ ,  $\sigma_2$ , ...,  $\sigma_t$  are loaded into the registers of the  $t$  multipliers. Then, these multipliers are shifted  $n$  times. At the end of the  $l$ th shift, the  $t$  registers contain  $\sigma_1 \alpha^l$ ,  $\sigma_2 \alpha^{2l}$ , ...,  $\sigma_t \alpha^{tl}$ . Thus, the sum  $1 + \sigma_1 \alpha^l + \sigma_2 \alpha^{2l} + \dots + \sigma_t \alpha^{tl}$  is tested. If the sum is zero,  $\alpha^{n-l}$  is an error-location num-

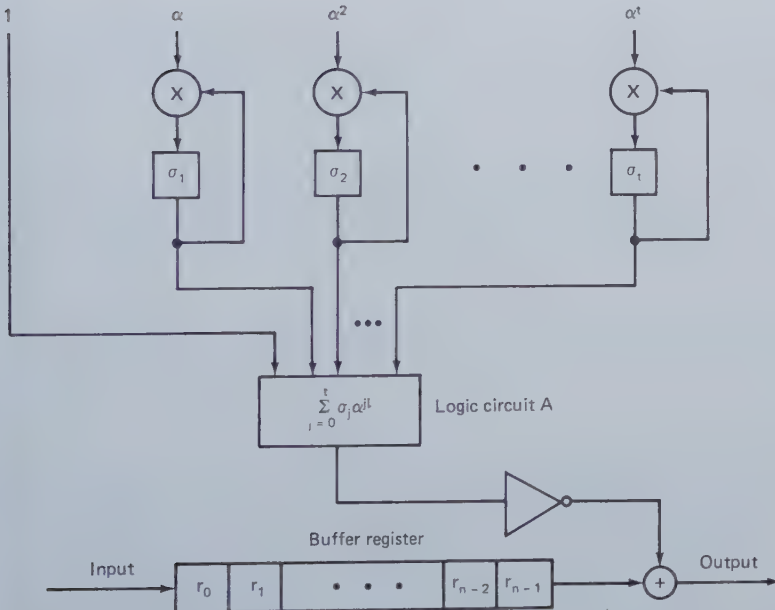


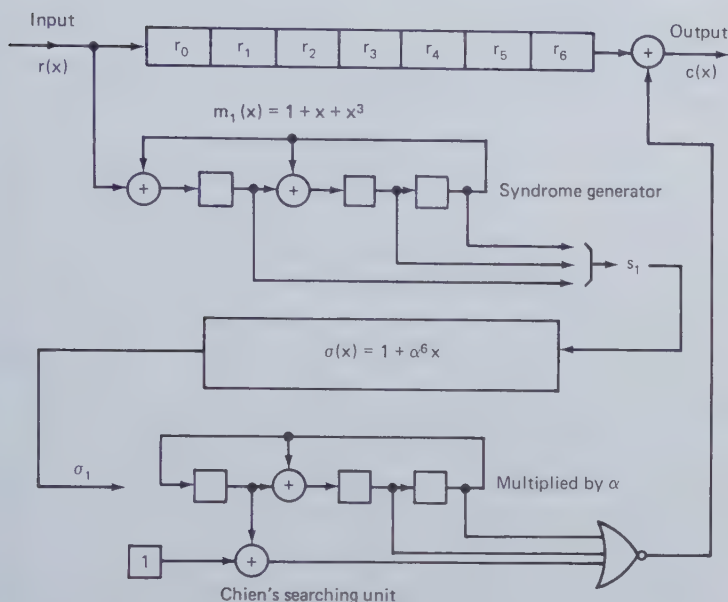
Figure 7.5 Chien's error-location searching unit.

ber; otherwise, it is not. This sum can be formed by using  $t$   $m$ -input modulo-2 adders. An  $m$ -input OR gate is used to test whether the sum is zero.

Chien's searching procedure is outlined as follows. The circuit indicated by  $\otimes$  in Fig. 7.5 consists of a few modulo-2 adders and a number of feedback connections. The logic circuit  $A$  consists of  $m$  modulo-2 adders with  $t$  inputs. But the output of each adder in logic circuit  $A$  is one of  $m$  inputs to the OR gate.

1. The  $t$   $\sigma$  registers initially contain  $\sigma_1, \sigma_2, \dots, \sigma_t$ . The received word  $r(x)$  is stored in the buffer register.
2. The  $\alpha, \alpha^2, \dots, \alpha^t$  are fed into the  $t$  multipliers. The decoder forms  $\sigma_1\alpha, \sigma_2\alpha^2, \dots, \sigma_t\alpha^t$ .
3. The sum  $1 + \sigma_1\alpha + \sigma_2\alpha^2 + \dots + \sigma_t\alpha^t$  is formed in the logic circuit  $A$  when the  $t$  multipliers are pulsed once.
4. The highest-order bit  $r_{n-1}$  is decoded as follows: If the sum is zero, then  $\alpha$  is a root of  $\sigma(x)$  and  $r_{n-1}$  is an erroneous digit. The output of the logic circuit  $A$  is 0. Otherwise,  $r_{n-1}$  is a correct digit.
5. The output of the inverter then becomes 1 and corrects the digit  $r_{n-1}$  coming out of the buffer.
6. After  $r_{n-1}$  is decoded, the  $t$  multipliers are pulsed again. Now, the sum  $1 + \sigma_1\alpha^2 + \sigma_2(\alpha^2)^2 + \dots + \sigma_t(\alpha^2)^t$  is tested. If the sum is zero,  $r_{n-2}$  is read out of the buffer and corrected by the output of the inverter.
7. Then  $r_{n-l}$  is decoded as follows: Test the sum  $1 + \sigma_1\alpha^l + \sigma_2(\alpha^l)^2 + \dots + \sigma_t(\alpha^l)^t$ . If this sum is zero, then  $\alpha^l$  is a root of  $\sigma(x)$  and  $r_{n-l}$  is an erroneous digit; otherwise, it is a correct digit. The digit  $r_{n-l}$  is read out of the buffer and corrected in the same manner as  $r_{n-1}$ .
8. The searching circuit decodes  $r(x)$  on a bit-by-bit basis in the manner outlined above until the entire received word is read out of the buffer.

**Example 7.14** Consider the single-error-correcting (7, 4) BCH code given in Example 7.7. Suppose that a lone error has occurred at  $r(x) = x^6$  when  $c(x) = 0$  is transmitted. Reread the algorithm in Sec. 7.4 for  $t = 2$  or fewer errors. Since the syndrome component is  $s_1 = \sigma_1 = \alpha^6$ , the error-location polynomial of this code is  $\sigma(x) = 1 + \alpha^6x$ . The element  $\alpha^6$  from  $GF(2^3)$  is (1 0 1). Since  $\sigma(\alpha) = 1 + \alpha^7 = 0$ , the error-location number is  $\alpha^{n-1} = \alpha^6$ . Thus,  $e(x) = x^6$  as expected. See Fig. 7.6 for the decoder for the (7, 4) BCH code.



**Figure 7.6** Decoder for the single-error-correcting (7, 4) BCH code.

Now, consider the Chien's searching unit for error correction. A  $\sigma$ -register is initially filled with  $\sigma_1 = \alpha^6 = (1 \ 0 \ 1)$ . The received word  $r(x)$  is stored in the buffer register. Next the Chien's circuit forms  $\sigma_1 \alpha = \alpha^7 = (1 \ 0 \ 0)$  when  $\alpha$  is fed into the multiplier. When the multiplier is pulsed once, the sum  $1 + \sigma_1 \alpha = (1 \ 0 \ 0) + (1 \ 0 \ 0) = (0 \ 0 \ 0)$  is formed. Since the sum  $1 + \sigma_1 \alpha$  is 0,  $\alpha$  is a root of  $\sigma(x)$  and the highest-order bit  $r_6$  of  $r(x)$  is an erroneous digit as assumed. Thus, the output of the inverter becomes 1 and corrects the erroneous bit  $r_6$  coming out of the buffer.

### 7.5.3 Solution of $\sigma(x)$ by Peterson-Berlekamp algorithm

As pointed out earlier, the problem of finding the minimal-degree solution to the error-locator polynomial is best understood by following the iterative operation. The application of the Peterson-Berlekamp algorithm to solve  $\sigma(x)$  is essential when approached from this recursive viewpoint. At each step of the iteration, the algorithm must retain both a minimal-degree polynomial whose coefficients satisfy Newton's identities and a possible correction term associated with the discrepancy.

The Peterson-Berlekamp algorithm may now be stated formally.

## Peterson-Berlekamp Algorithm

1. Set the following initial conditions before taking the iterative step.

$$\begin{array}{lll} \sigma^{(-1)}(x) = 1 & l_{-1} = 0 & d_{-1} = 1 \\ \sigma^{(0)}(x) = 1 & l_0 = 0 & d_0 = s_1 \end{array}$$

2. If  $d_n = 0$ , then  $\sigma^{(n+1)}(x) = \sigma^{(n)}(x)$  and  $l_{n+1} = l_n$ .
3. If  $d_n \neq 0$ , find  $\sigma^{(m)}(x)$  prior to  $\sigma^{(n)}(x)$  such that  $d_m \neq 0$ ,  $1 \leq m \leq n$ , and the number  $m - l_m$  has the largest value. Then

$$\begin{aligned} \sigma^{(n+1)}(x) &= \sigma^{(n)}(x) - d_n d_m^{-1} x^{n-m} \sigma^{(m)}(x) \\ l_{n+1} &= \max[l_n, l_m + n - m] \end{aligned}$$

4. For either  $d_n = 0$  or  $d_n \neq 0$ , the next discrepancy is

$$d_{n+1} = s_{n+2} + s_{n+1} \sigma_1^{(n+1)} + \cdots + s_{n+2-l_{n+1}} \sigma_{l_{n+1}}^{(n+1)}$$

where  $\sigma_i^{(n+1)}$ ,  $1 \leq i \leq l_{n+1}$ , are the coefficients of  $\sigma^{(n+1)}(x)$ .

The details are illustrated by the following example.

**Example 7.15** Consider the triple-error-correcting (31, 16) BCH code given in Example 7.5. If the received polynomial is  $r(x) = x + x^{12} + x^{25}$  when  $c(x) = 0$  is transmitted, then  $r(x)$  represents nothing but the error pattern  $e(x)$ . Now, solve the error-locator polynomial  $\sigma(x)$  by applying the Peterson-Berlekamp algorithm described above. The generator polynomial of this code is

$$\begin{aligned} g(x) &= m_1(x)m_3(x)m_5(x) \\ &= 1 + x + x^2 + x^3 + x^5 + x^7 + x^8 + x^9 + x^{10} + x^{11} + x^{15} \end{aligned}$$

where  $m_1(x) = 1 + x^2 + x^5$ ,  $m_3(x) = 1 + x^2 + x^3 + x^4 + x^5$ , and  $m_5(x) = 1 + x + x^2 + x^4 + x^5$  are the minimal polynomials corresponding to  $\alpha$ ,  $\alpha^3$ , and  $\alpha^5$ , respectively.

Thus, the syndrome is obtained from the remainders  $\gamma_k(x)$  resulting from division of  $r(x)$  by  $m_k(x)$ ,  $k = 1, 3, 5$ . The remainders are  $\gamma_1(x) = 1 + x^2 + x^4$ ,  $\gamma_3(x) = 1 + x^2 + x^3$ , and  $\gamma_5(x) = x^2 + x^3$ . Therefore, the syndrome digits are:

$$s_1 = \gamma_1(\alpha) = 1 + \alpha^2 + \alpha^4 = \alpha^{22}$$

$$s_2 = \gamma_1(\alpha^2) = 1 + \alpha^4 + \alpha^8 = \alpha^2 + \alpha^3 + \alpha^4 = \alpha^{13}$$

$$s_4 = \gamma_1(\alpha^4) = 1 + \alpha^8 + \alpha^{16} = 1 + \alpha + \alpha^2 + \alpha^4 = \alpha^{26}$$

$$s_3 = \gamma_3(\alpha^3) = 1 + \alpha^6 + \alpha^9 = 1 + \alpha^4 = \alpha^{10}$$

$$s_6 = \gamma_3(\alpha^6) = 1 + \alpha^{12} + \alpha^{18} = \alpha^2 + \alpha^3 = \alpha^{20}$$

$$s_5 = \gamma_5(\alpha^5) = \alpha^{10} + \alpha^{15} = \alpha + \alpha^2 + \alpha^3 = \alpha^{12}$$



Thus, the 6-tuple syndrome is

$$\begin{aligned}\mathbf{s} &= (s_1, s_2, s_3, s_4, s_5, s_6) \\ &= (\alpha^{22}, \alpha^{13}, \alpha^{10}, \alpha^{26}, \alpha^{12}, \alpha^{20})\end{aligned}$$

Starting with the initial conditions,

$$\begin{aligned}\sigma^{(-1)}(x) &= 1 & l_{-1} &= 0 & d_{-1} &= 1 \\ \sigma^{(0)}(x) &= 1 & l_0 &= 0 & d_0 &= s_1 = \alpha^{22} \neq 0\end{aligned}$$

the recursive operation can proceed as follows:

1. Let  $n = 0$  and  $m = -1$ , and use the initial conditions. We then have

$$\begin{aligned}\sigma^{(1)}(x) &= \sigma^{(0)}(x) - d_0 d_{-1}^{-1} x \sigma^{(-1)}(x) \\ &= 1 + \alpha^{22} x \\ d_1 &= s_2 + s_1 \sigma_1^{(1)} \\ &= \alpha^{13} + \alpha^{22} \cdot \alpha^{22} = \alpha^{13} + \alpha^{13} = 0 \\ l_1 &= \max[l_0, l_{-1} + n - m] \\ &= \max[0, 0 + 0 + 1] = 1 \\ 1 - l_1 &= 1 - 1 = 0\end{aligned}$$

2. Let  $n = 1$ .

$$\begin{aligned}\sigma^{(2)}(x) &= \sigma^{(1)}(x) = 1 + \alpha^{22} x \\ d_2 &= s_3 + s_2 \sigma_1^{(2)} \\ &= \alpha^{10} + \alpha^4 = 1 \\ l_2 &= l_1 = 1 \\ 2 - l_2 &= 2 - 1 = 1\end{aligned}$$

3. Let  $n = 2$  and  $m = 0$ . Since  $d_2 = 1 \neq 0$ ,  $m$  must be chosen such that  $d_m \neq 0$  and  $m - l_m$  has the largest value. In this case, we take  $m = 0$ .

$$\begin{aligned}\sigma^{(3)}(x) &= \sigma^{(2)}(x) - d_2 d_0^{-1} x^2 \sigma^{(0)}(x) \\ &= 1 + \alpha^{22} x + \alpha^9 x^2 \\ d_3 &= s_4 + s_3 \sigma_1^{(3)} + s_2 \sigma_2^{(3)} \\ &= \alpha^{26} + \alpha^{10} \cdot \alpha^{22} + \alpha^{13} \cdot \alpha^9 \\ &= \alpha^{26} + \alpha + \alpha^{22} = 0 \\ l_3 &= \max[l_2, l_0 + 2 - 0] \\ &= \max[1, 2] = 2 \\ 3 - l_3 &= 3 - 2 = 1\end{aligned}$$

4. Let  $n = 3$ .

$$\begin{aligned}
 \sigma^{(4)}(x) &= \sigma^{(3)}(x) = 1 + \alpha^{22}x + \alpha^9x^2 \\
 d_4 &= s_5 + s_4\sigma_1^{(4)} + s_3\sigma_2^{(4)} \\
 &= \alpha^{12} + \alpha^{26} \cdot \alpha^{22} + \alpha^{10} \cdot \alpha^9 \\
 &= 1 + \alpha + \alpha^3 + \alpha^4 = \alpha^{16} \\
 l_4 &= l_3 = 2 \\
 4 - l_4 &= 4 - 2 = 2
 \end{aligned}$$

5. Let  $n = 4$  and  $m = 2$ .

$$\begin{aligned}
 \sigma^{(5)}(x) &= \sigma^{(4)}(x) - d_4d_2^{-1}x^2\sigma^{(2)}(x) \\
 &= 1 + \alpha^{22}x + x^2 + \alpha^7x^3 \\
 d_5 &= s_6 + s_5\sigma_1^{(5)} + s_4\sigma_2^{(5)} + s_3\sigma_3^{(5)} \\
 &= \alpha^{20} + \alpha^3 + \alpha^{26} + \alpha^{17} = 0 \\
 l_5 &= \max[l_4, l_2 + 4 - 2] = \max[2, 3] = 3 \\
 5 - l_5 &= 5 - 3 = 2
 \end{aligned}$$

6. Let  $n = 5$ .

$$\sigma^{(6)}(x) = \sigma^{(5)}(x) = 1 + \alpha^{22}x + x^2 + \alpha^7x^3$$

This is the legitimate error-locator polynomial  $\sigma(x)$ , that is,

$$\sigma(x) = \sigma^{(6)}(x) = 1 + \alpha^{22}x + x^2 + \alpha^7x^3$$

By substituting the field elements of  $GF(2^5)$ ,  $\alpha^k$ ,  $0 \leq k \leq 31$ , into  $\sigma(x)$ , the roots of  $\sigma(x)$  that satisfy  $\sigma(\alpha^k) = 0$  are  $\alpha^6$ ,  $\alpha^{19}$ , and  $\alpha^{30}$ . Therefore, the error-location numbers are easily found by taking the inverse of the roots, which are  $\alpha^{25}$ ,  $\alpha^{12}$ , and  $\alpha$ , respectively. Thus the error pattern is

$$e(x) = x + x^{12} + x^{25}$$

Adding this  $e(x)$  to the received polynomial  $r(x)$  yields the all-zero transmitted code word. Since the iterative steps for finding  $\sigma(x)$  have been carried out, the results can be summarized in Table 7.6 in the following compact form.

#### 7.5.4 Solution of $\sigma(x)$ by the Berlekamp-Massey algorithm

An efficient algorithm for finding  $\sigma(x)$  is the iterative procedure introduced by Berlekamp in 1967.

Let  $c(x) = c_0 + c_1x + \cdots + c_{n-1}x^{n-1}$  be the transmitted code polynomial,  $r(x) = r_0 + r_1x + \cdots + r_{n-1}x^{n-1}$  the received code polynomial,

TABLE 7.6 Solution of  $\sigma(x)$  by the Peterson-Berlekamp Algorithm

| $n$ | $\sigma^{(n)}(x)$                      | $d_n$         | $l_n$ | $n - l_n$ |
|-----|----------------------------------------|---------------|-------|-----------|
| -1  | 1                                      | 1             | 0     | -1        |
| 0   | 1                                      | $\alpha^{22}$ | 0     | 0         |
| 1   | $1 + \alpha^{22}x$                     | 0             | 1     | 0         |
| 2   | $1 + \alpha^{22}x$                     | 1             | 1     | 1         |
| 3   | $1 + \alpha^{22}x + \alpha^9x^2$       | 0             | 2     | 1         |
| 4   | $1 + \alpha^{22}x + \alpha^9x^2$       | $\alpha^{16}$ | 2     | 2         |
| 5   | $1 + \alpha^{22}x + x^2 + \alpha^7x^3$ | 0             | 3     | 2         |
| 6   | $1 + \alpha^{22}x + x^2 + \alpha^7x^3$ | —             | —     | —         |

and  $e(x) = r(x) + c(x)$  the error pattern. Now, define the syndrome  $\mathbf{s}$  as a vector with components

$$s_i = r(\alpha^{1+i}) \quad 0 \leq i \leq 2t - 1 \quad (7.66)$$

Since  $\alpha^{1+i}$ ,  $0 \leq i \leq 2t - 1$ , is a root of  $c(x)$ , Eq. (7.66) becomes

$$s_i = c(\alpha^{1+i}) + e(\alpha^{1+i}) = e(\alpha^{1+i}) \quad 0 \leq i \leq 2t - 1 \quad (7.67)$$

Suppose that some given number,  $\nu \leq \lfloor (d_{\min} - 1)/2 \rfloor$ , of errors has occurred in transmission. Then, the error digits are  $e_j = 0$  except for  $j = j_1, j_2, \dots, j_\nu$ . Hence, the error pattern is actually

$$e(x) = x^{j_1} + x^{j_2} + \dots + x^{j_\nu} \quad (7.68)$$

Thus, the syndrome digits become

$$\begin{aligned} s_i &= (\alpha^{1+i})^{j_1} + (\alpha^{1+i})^{j_2} + \dots + (\alpha^{1+i})^{j_\nu} \\ &= \sum_{\lambda=1}^{\nu} (\alpha^{1+i})^{j_\lambda} \quad 0 \leq i \leq 2t - 1 \end{aligned} \quad (7.69)$$

Let  $\sigma^{(n)}(x) = 1 + \sigma_1^{(n)}x + \sigma_2^{(n)}x^2 + \dots + \sigma_{l_n}^{(n)}x^{l_n}$  be the minimum-degree polynomial determined at the  $n$ th step of the iteration whose coefficients satisfy the first  $n$  of Newton's identities. Also, let the discrepancy at the  $n$ th step be

$$d_n = s_n + \sum_{i=1}^n \sigma_i^{(n)} s_{n-i} \quad (7.70)$$

where  $\sigma_i^{(n)}$ ,  $1 \leq i \leq l_n$ , are the coefficients of  $\sigma^{(n)}(x)$ . If  $d_n = 0$ , the algorithm increases  $n$  by one, but keeps the same register. If  $d_n \neq 0$ , a correction term is added to  $\sigma^{(n)}(x)$  to make it generate  $s_n$  correctly.

Let us introduce the detailed rules for the algorithm devised by Berlekamp (1967) and discussed by Massey (1968).

### The Berlekamp-Massey algorithm

1. Start at  $n = 0$  with the initial conditions:

$$\begin{aligned}\sigma^{(-1)}(x) &= 1 & l_{-1} &= 0 & d_{-1} &= 1 \\ \sigma^{(0)}(x) &= 1 & l_0 &= 0 & k_0 &= -1\end{aligned}$$

2. If  $d_n = 0$ , then  $\sigma^{(n+1)}(x) = \sigma^{(n)}(x)$ ,  $l_{n+1} = l_n$ , and  $k_{n+1} = k_n$ .
3. If  $d_n \neq 0$ , we have

$$\sigma^{(n+1)}(x) = \sigma^{(n)}(x) - d_n d_{k_n}^{-1} x^{n-k_n} \sigma^{(k_n)}(x)$$

$$l_{n+1} = \max[l_n, n - (k_n - l_{k_n})]$$

$$\text{and } k_{n+1} = \begin{cases} k_n & \text{if } l_{n+1} = l_n \\ n & \text{if } l_{n+1} > l_n \end{cases}$$

4. For either  $d_n = 0$  or  $d_n \neq 0$ , the next discrepancies are

$$\begin{aligned}d_{k_{n+1}} &= s_{k_{n+1}} + \sum_{i=1}^{k_{n+1}} \sigma_i^{(k_{n+1})} s_{k_{n+1}-i} \\ d_{n+1} &= s_{n+1} + \sum_{i=1}^{n+1} \sigma_i^{(n+1)} s_{n+1-i}\end{aligned}$$

**Example 7.16** Consider again the triple-error-correcting (31, 16) BCH code given in Example 7.15. Find the error-locator polynomial  $\sigma(x)$  by means of the Berlekamp-Massey algorithm and obtain the error pattern under the assumption that  $r(x) = x + x^{12} + x^{25}$  provided that  $c(x) = 0$ . As in Example 7.15, the syndrome components are  $s_0 = \alpha^{22}$ ,  $s_1 = \alpha^{13}$ ,  $s_2 = \alpha^{10}$ ,  $s_3 = \alpha^{26}$ ,  $s_4 = \alpha^{12}$ , and  $s_5 = \alpha^{20}$ . Starting with the initial conditions, the recursive operation proceeds as follows:

1. Let  $n = 0$ . Then

$$\sigma^{(1)}(x) = \sigma^{(0)}(x) - d_0 d_{k_0}^{-1} x^{-k_0} \sigma^{(k_0)}(x)$$

where

$$d_0 = s_0 + \sum_{i=1}^0 \sigma_i^{(k_0)} s_{k_0-i} = \alpha^{22}$$

$$d_{k_0} = d_{-1} = 1$$

Thus,

$$\begin{aligned}\sigma^{(1)}(x) &= \sigma^{(0)}(x) + \alpha^{22} \cdot 1 \cdot x^{(-1)} \cdot \sigma^{(-1)}(x) \\ &= 1 + \alpha^{22}x\end{aligned}$$

$$\begin{aligned}l_1 &= \max[l_0, 0 - (-1 - l_{-1})] \\ &= \max[0, 0 + 1 + 0] = 1\end{aligned}$$

Since  $l_1 > l_0$ ,

$$k_1 = 0$$

$$d_{k_1} = s_{k_1} + \sum_{i=1}^0 \sigma_i^{(0)} s_{k_1-i} = s_0 = \alpha^{22}$$

$$\begin{aligned}d_1 &= s_1 + \sum_{i=1}^1 \sigma_i^{(1)} s_{1-i} = s_1 + \sigma_1^{(1)} s_0 \\ &= \alpha^{13} + \alpha^{22} \cdot \alpha^{22} = \alpha^{13} + \alpha^{13} = 0\end{aligned}$$

2. Let  $n = 1$ . Since  $d_1 = 0$ , it follows that

$$\begin{aligned}\sigma^{(2)}(x) &= \sigma^{(1)}(x) = 1 + \alpha^{22}x \\ l_2 &= l_1 = 1 \\ k_2 &= k_1 = 0\end{aligned}$$

and

$$\begin{aligned}d_{k_2} &= s_{k_2} + \sum_{i=1}^{k_2} \sigma_i^{(k_2)} s_{k_2-i} \\ &= s_0 + \sum_{i=1}^0 \sigma_i^{(0)} s_{0-i} = s_0 = \alpha^{22}\end{aligned}$$

$$\begin{aligned}d_2 &= s_2 + \sum_{i=1}^2 \sigma_i^{(2)} s_{2-i} = s_2 + \sigma_1^{(2)} s_1 + \sigma_2^{(2)} s_0 \\ &= \alpha^{10} + \alpha^{22} \cdot \alpha^{13} = \alpha^{10} + \alpha^4 = 1\end{aligned}$$

3. Let  $n = 2$ . Since  $d_2 \neq 0$ ,

$$\begin{aligned}\sigma^{(3)}(x) &= \sigma^{(2)}(x) - d_2 d_{k_2}^{-1} x^{2-k_2} \sigma^{(k_2)}(x) \\ &= 1 + \alpha^{22}x + 1 \cdot (\alpha^{22})^{-1} \cdot x^2 \cdot 1 \\ &= 1 + \alpha^{22}x + \alpha^9 x^2\end{aligned}$$

$$\begin{aligned}l_3 &= \max[l_2, 2 - (k_2 - l_0)] \\ &= \max[1, 2] = 2\end{aligned}$$



Since  $l_3 > l_2$ ,

$$k_3 = 2$$

$$\begin{aligned} d_{k_3} &= s_{k_2} + \sum_{i=1}^{k_3} \sigma_i^{(k_3)} s_{k_3-i} = s_2 + \sum_{i=1}^2 \sigma_i^{(2)} s_{2-i} \\ &= s_2 + \sigma_1^{(2)} s_1 = \alpha^{10} + \alpha^4 = 1 \end{aligned}$$

$$\begin{aligned} d_3 &= s_3 + \sum_{i=1}^3 \sigma_i^{(3)} s_{3-i} \\ &= s_3 + \sigma_1^{(3)} s_2 + \sigma_2^{(3)} s_1 \\ &= \alpha^{26} + \alpha + \alpha^{22} = 0 \end{aligned}$$

4. Let  $n = 3$ . Since  $d_3 = 0$ ,

$$\begin{aligned} \sigma^{(4)}(x) &= \sigma^{(3)}(x) = 1 + \alpha^{22}x + \alpha^9x^2 \\ l_4 &= l_3 = 2 \\ k_4 &= k_3 = 2 \end{aligned}$$

and

$$\begin{aligned} d_{k_4} &= s_{k_4} + \sum_{i=1}^{k_4} \sigma_i^{(k_4)} s_{k_4-i} \\ &= s_2 + \sigma_1^{(2)} s_1 = \alpha^{10} + \alpha^4 = 1 \\ d_4 &= s_4 + \sum_{i=1}^4 \sigma_i^{(4)} s_{4-i} \\ &= s_4 + \sigma_1^{(4)} s_3 + \sigma_2^{(4)} s_2 = \alpha^{12} + \alpha^{17} + \alpha^{18} \\ &= 1 + \alpha + \alpha^3 + \alpha^4 = \alpha^{16} \end{aligned}$$

5. Let  $n = 4$ . Since  $d_4 \neq 0$ ,

$$\begin{aligned} \sigma^{(5)}(x) &= \sigma^{(4)}(x) - d_4 d_{k_4}^{-1} x^{4-k_4} \sigma^{(k_4)}(x) \\ &= 1 + \alpha^{22}x + \alpha^9x^2 + \alpha^{16}x^2(1 + \alpha^{22}x) \\ &= 1 + \alpha^{22}x + x^2 + \alpha^7x^3 \end{aligned}$$

$$l_5 = \max[l_4, 4 - (2 - l_2)] = \max[2, 3] = 3$$

Because  $l_5 > l_4$ ,

$$k_5 = 4$$

Accordingly, it follows that

$$\begin{aligned} d_{k_5} &= s_{k_5} + \sum_{i=1}^{k_5} \sigma_i^{(k_5)} s_{k_5-i} \\ &= s_4 + \sum_{i=1}^4 \sigma_i^{(4)} s_{4-i} \\ &= \alpha^{12} + \sigma_1^{(4)} s_3 + \sigma_2^{(4)} s_2 = \alpha^{16} \end{aligned}$$

$$\begin{aligned}
 d_5 &= s_5 + \sum_{i=1}^5 \sigma_1^{(5)} s_{5-i} \\
 &= s_5 + \sigma_1^{(5)} s_4 + \sigma_2^{(5)} s_3 + \sigma_3^{(5)} s_2 \\
 &= \alpha^{20} + \alpha^3 + \alpha^{26} + \alpha^{17} = 0
 \end{aligned}$$

6. Let  $n = 5$ . Since  $d_5 = 0$ , the minimum-degree polynomial determined at the sixth step of the iteration is given by

$$\sigma^{(6)}(x) = \sigma^{(5)}(x) = 1 + \alpha^{22}x + x^2 + \alpha^7x^3$$

This is the desired error-locator polynomial  $\sigma(x)$  which is exactly identical to the solution found in Example 7.15. Since the roots of  $\sigma(x) = \sigma^{(6)}(x)$  are  $\alpha^6$ ,  $\alpha^{19}$ , and  $\alpha^{30}$ , the error-location numbers are  $\alpha^{25}$ ,  $\alpha^{12}$ , and  $\alpha$ , respectively. Thus the error pattern is

$$e(x) = x + x^{12} + x^{25}$$

and the resulting decoder output is simply  $c(x) = r(x) + e(x) = 0$ . The detailed solution for finding  $\sigma(x)$  of this code is given in Table 7.7.

**Example 7.17** Consider the design problem for the (31, 16) BCH decoder, placing particular emphasis on the searching circuit for the error-location numbers. Review Example 7.15 for the computation of the syndrome  $\mathbf{s} = (\alpha^{22}, \alpha^{13}, \alpha^{10}, \alpha^{26}, \alpha^{12}, \alpha^{20})$  and the error-locator polynomial  $\sigma(x) = 1 + \alpha^{22}x + x^2 + \alpha^7x^3$ . The last step in decoding this code is to find the error-location numbers. With a hardware implementation, this can be done by devising a cyclic-error-location search unit as shown in Fig. 7.7. The  $t = 3$  wired multipliers for multiplying  $\alpha$ ,  $\alpha^2$ , and  $\alpha^3$  in one clock cycle are required for the Chien's search unit. The result of the computation is as follows:

$$\begin{aligned}
 \sigma_1\alpha &= \sigma_{1,4} + \sigma_{1,0}\alpha + (\sigma_{1,1} + \sigma_{1,4})\alpha^2 + \sigma_{1,2}\alpha^3 + \sigma_{1,3}\alpha^4 \\
 \sigma_2\alpha^2 &= \sigma_{2,3} + \sigma_{2,4}\alpha + (\sigma_{2,0} + \sigma_{2,3})\alpha^2 + (\sigma_{2,1} + \sigma_{2,4})\alpha^3 + \sigma_{2,2}\alpha^4
 \end{aligned}$$

TABLE 7.7 Solution of  $\sigma(x)$  by the Berlekamp-Massey Algorithm

| $n$ | $\sigma^{(n)}(x)$                      | $l_n$ | $d_n$         | $k_n$ | $d_{k_n}$     |
|-----|----------------------------------------|-------|---------------|-------|---------------|
| -1  | 1                                      | 0     | 1             |       |               |
| 0   | 1                                      | 0     | $\alpha^{22}$ | -1    | 1             |
| 1   | $1 + \alpha^{22}x$                     | 1     | 0             | 0     | $\alpha^{22}$ |
| 2   | $1 + \alpha^{22}x$                     | 1     | 1             | 0     | $\alpha^{22}$ |
| 3   | $1 + \alpha^{22}x + \alpha^9x^2$       | 2     | 0             | 2     | 1             |
| 4   | $1 + \alpha^{22}x + \alpha^9x^2$       | 2     | $\alpha^{16}$ | 2     | 1             |
| 5   | $1 + \alpha^{22}x + x^2 + \alpha^7x^3$ | 3     | 0             | 4     | $\alpha^{16}$ |
| 6   | $1 + \alpha^{22}x + x^2 + \alpha^7x^3$ | —     | —             | —     | —             |

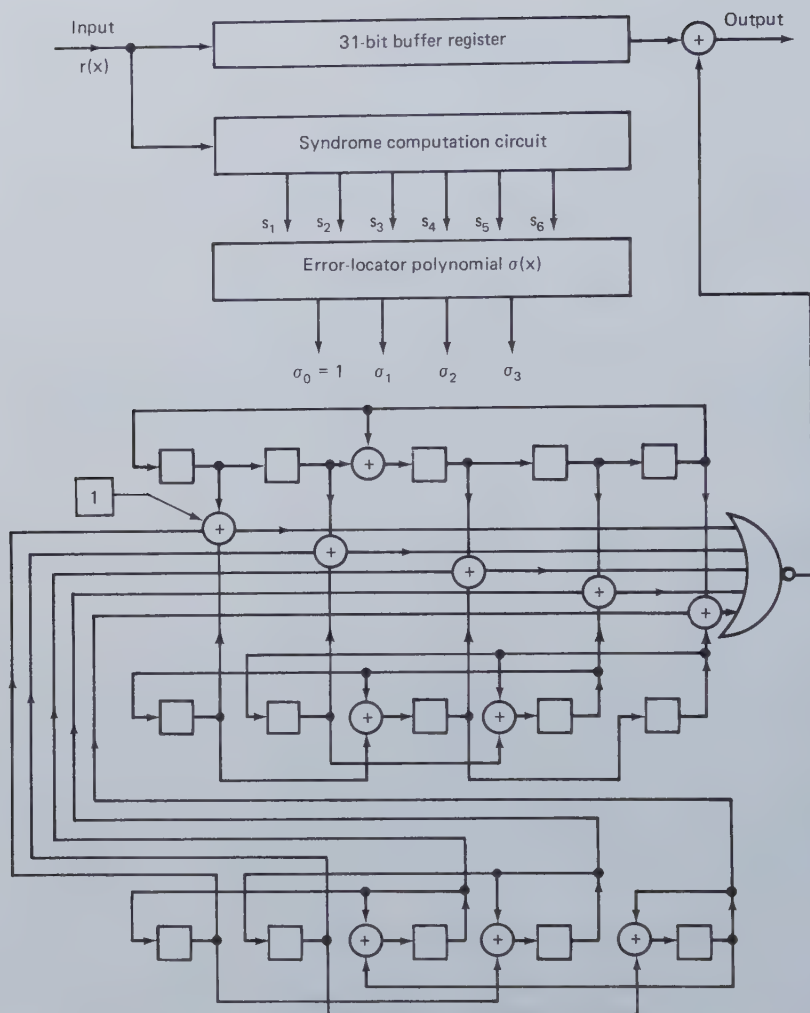


Figure 7.7 Decoding circuit for the triple-error-correcting (31, 16) BCH code.

$$\text{and} \quad \sigma_3 \alpha^3 = \sigma_{3,2} + \sigma_{3,3} \alpha + (\sigma_{3,2} + \sigma_{3,4}) \alpha^2 + (\sigma_{3,0} + \sigma_{3,3}) \alpha^3 \\ + (\sigma_{3,1} + \sigma_{3,4}) \alpha^4$$

### 7.5.5 Fast iterative algorithm for $\sigma(x)$

The fast iterative algorithm applies only to binary BCH codes. For binary BCH codes, the recursive operation for finding  $\sigma(x)$  has to carry

out  $t$  steps rather than  $2t$  steps. The first step in decoding is to calculate the syndrome components (power-sum symmetric functions) from the received polynomial  $r(x)$  such that  $s_j = r(\alpha^j)$ . The elementary symmetric functions  $\sigma_i$  are related to the power-sum symmetric functions  $s_j$  by the following Newton identities:

$$\begin{aligned}
 s_1 + \sigma_1 &= 0 \\
 s_3 + s_2\sigma_1 + s_1\sigma_2 + \sigma_3 &= 0 \\
 &\vdots \\
 s_{2t-3} + s_{2t-4}\sigma_1 + \cdots + s_{2t-3-\nu}\sigma_\nu &= 0 \\
 s_{2t-1} + s_{2t-1}\sigma_1 + \cdots + s_{2t-1-\nu}\sigma_\nu &= 0
 \end{aligned} \tag{7.71}$$

Equation (7.71) may have many solutions if  $\nu$  is allowed to be arbitrarily large. However, if it is restricted so that  $\nu \leq t$ , there is only one solution.

The decoder's problem is to find the minimum-degree polynomial  $\sigma^{(n)}(x)$  so that  $\sigma_0$  of Eq. (7.71) equals 1. This can be done recursively. Let  $\sigma(x)$  be the error-locator polynomial

$$\sigma(x) = \sigma_0 + \sigma_1 x + \sigma_2 x^2 + \cdots + \sigma_\nu x^\nu$$

which is an unknown polynomial whose coefficients must be determined. At the  $n$ th step, the decoder considers only the first  $n$  of Newton's identities and determines a lowest-degree polynomial

$$\sigma^{(n)}(x) = \sigma_0 + \sigma_1^{(n)}x + \sigma_2^{(n)}x^2 + \cdots + \sigma_{l_n}^{(n)}x^{l_n}$$

whose degree  $l_n$  is not greater than  $n$ . At step  $n + 1$ , this result and previous minimal solutions are used to find the next minimal solution  $\sigma^{(n+1)}(x)$ . Upon reaching step  $\nu$ , the only solution of degree  $\nu$  is obtained as the minimal-solution polynomial. This solution remains unchanged for  $n$  greater than  $\nu$  and less than or equal to  $t$ ,  $\nu < n \leq t$ . Thus, if the number of errors in  $r(x)$  is less than  $t$ , it is not necessary to carry out the  $2t$  iteration steps to determine  $\sigma(x)$  for a  $t$ -error-correcting binary BCH code. Hence, a fast algorithm can be obtained that requires filling out a table that only has  $t$  empty rows. Such an algorithm is presented below.

## Fast iterative algorithm

1. Use the initial conditions

$$\begin{aligned}\sigma^{(-1/2)}(x) &= 1 & l_{-1/2} &= 0 & d_{-1/2} &= 1 \\ \sigma^{(0)}(x) &= 1 & l_0 &= 0 & d_0 &= s_1 \neq 0\end{aligned}$$

2. If
- $d_n = 0$
- , then

$$\sigma^{(n+1)} = \sigma^{(n)}(x)$$

3. If
- $d_n \neq 0$
- , find a
- $m$
- th row preceding the
- $n$
- th row such that
- $2m - l_m$
- is as large as possible and
- $d_m \neq 0$
- . Then

$$\sigma^{(n+1)}(x) = \sigma^{(n)}(x) + d_n d_m^{-1} x^{2(n-m)} \sigma^{(m)}(x)$$

4. In either
- $d_n = 0$
- or
- $d_n \neq 0$
- , the degree of
- $\sigma^{(n+1)}(x)$
- is

$$l_{n+1} = \max[l_n, l_m + 2(n - m)]$$

and the discrepancy at the  $(n + 1)$ th step is

$$d_{n+1} = s_{2n+3} + s_{2n+2} \sigma_1^{(n+1)} + s_{2n+1} \sigma_2^{(n+1)} + \dots + s_{2n+3-l_{n+1}} \sigma_{l_{n+1}}^{(n+1)}$$

The polynomial  $\sigma^{(l)}(x)$  must be the required error-locator polynomial  $\sigma(x)$ .

**Example 7.18** Consider again the decoding of the triple-error-correcting (31, 16) BCH code discussed in previous examples. The syndrome components of  $r(x) = 1 + x^{12} + x^{25}$  are  $s_1 = \alpha^{22}$ ,  $s_2 = \alpha^{13}$ ,  $s_3 = \alpha^{10}$ ,  $s_4 = \alpha^{26}$ ,  $s_5 = \alpha^{12}$ , and  $s_6 = \alpha^{20}$ . Use  $d_0 = \alpha^{22}$ , and apply the fast algorithm for finding  $\sigma(x)$ .

1. Let
- $n = 0$
- and
- $m = -1/2$
- .

$$\sigma^{(1)}(x) = \sigma^{(0)}(x) + d_0 d_{-1/2}^{-1} x \sigma^{(-1/2)}(x)$$

$$= 1 + \alpha^{22} x$$

$$l_1 = \max[l_0, l_{-1/2} + 2(0 + 1/2)]$$

$$= \max[0, 0 + 1] = 1$$

$$d_1 = s_3 + s_2 \sigma_1^{(1)} = \alpha^{10} + \alpha^4 = 1$$

2. Let
- $n = 1$
- and
- $m = 0$
- .

$$\sigma^{(2)}(x) = \sigma^{(1)}(x) + d_1 d_0^{-1} x^2 \sigma^{(0)}(x)$$

$$= 1 + \alpha^{22} x + s_1^{-1} x^2 = 1 + \alpha^{22} x + \alpha^9 x^2$$

$$l_2 = \max[l_1, l_0 + 2(1 - 0)]$$

$$= \max[1, 0 + 2] = 2$$

$$d_2 = s_5 + s_4 \sigma_1^{(2)} + s_3 \sigma_2^{(2)}$$

$$= \alpha^{12} + \alpha^{17} + \alpha^{19} = \alpha^{16}$$



TABLE 7.8 Solution of  $\sigma(x)$  by the Fast Algorithm

| $n$            | $\sigma^{(n)}(x)$                      | $l_n$ | $d_k$         | $2n - l_n$ |
|----------------|----------------------------------------|-------|---------------|------------|
| $-\frac{1}{2}$ | 1                                      | 0     | 1             | -1         |
| 0              | 1                                      | 0     | $\alpha^{22}$ | 0          |
| 1              | $1 + \alpha^{22}x$                     | 1     | 1             | 1          |
| 2              | $1 + \alpha^{22}x + \alpha^9x^2$       | 2     | $\alpha^{16}$ | 2          |
| 3              | $1 + \alpha^{22}x + x^2 + \alpha^7x^3$ | 3     | —             | 3          |

3. Let  $n = 2$  and  $m = 1$ .

$$\begin{aligned}
 \sigma^{(3)}(x) &= \sigma^{(2)}(x) + d_2 d_1^{-1} x^2 \sigma^{(1)}(x) \\
 &= 1 + \alpha^{22}x + \alpha^9x^2 + \alpha^{16}x^2(1 + \alpha^{22}x) \\
 &= 1 + \alpha^{22}x + x^2 + \alpha^7x^3
 \end{aligned}$$

which leads to the unique solution for  $\sigma^{(t)}(x) = \sigma^{(3)}(x) = \sigma(x)$ .

The simplified table for finding  $\sigma(x)$  for this code is summarized in Table 7.8.

### 7.5.6 Checking of error pattern

In general, the error-locator polynomial for  $t$ -error correction is represented by

$$\begin{aligned}
 \sigma(x) &= \sigma_0 + \sigma_1x + \sigma_2x^2 + \cdots + \sigma_tx^t \\
 &= \sum_{j=0}^t \sigma_j x^j \quad \sigma_0 = 1
 \end{aligned} \tag{7.72}$$

When an error occurs in the  $i$ th position of the received word  $r(x)$ , it must result in

$$\sigma(\alpha^{-i}) = \sum_{j=0}^t \sigma_j \alpha^{-ji} = 0 \tag{7.73}$$

Setting  $\sigma_{j,i} = \sigma_j \alpha^{-ji}$ ,  $j = 1, 2, \dots, t$ , yields

$$\begin{aligned}
 \sigma_{1,i} &= \sigma_1 \alpha^{-i} \\
 \sigma_{2,i} &= \sigma_2 \alpha^{-2i} \\
 &\vdots \\
 \sigma_{t,i} &= \sigma_t \alpha^{-ti}
 \end{aligned}$$

Thus, we can choose  $i$  from  $GF(2^m)$  so that  $\sigma(x)$  could be the legitimate polynomial that satisfies the error-location number  $\alpha^{n-i}$ . Accordingly,

the error pattern  $e(x)$  can easily be determined. This is one method of substitution, first introduced by Peterson, for finding the roots of  $\sigma(x)$ .

**Example 7.19** Consider the error-locator polynomial

$$\sigma(x) = 1 + \alpha^{22}x + x^2 + \alpha^7x^3$$

which is a result obtained from the previous examples. Since the coefficients of  $\sigma(x)$  are  $\sigma_0 = 1$ ,  $\sigma_1 = \alpha^{22}$ ,  $\sigma_2 = 1$ , and  $\sigma_3 = \alpha^7$ ,  $\sigma_{j,i}$  become as follows:

$$\sigma_{1,i} = \alpha^{-i+22}$$

$$\sigma_{2,i} = \alpha^{-2i}$$

$$\sigma_{3,i} = \alpha^{-3i+7}$$

For  $i = 1$ , we have  $\sigma(\alpha^{-1}) = 1 + \alpha^{21} + \alpha^{29} + \alpha^4 = 0$  because the parameters are  $\sigma_{1,1} = \alpha^{21}$ ,  $\sigma_{1,2} = \alpha^{29}$ , and  $\sigma_{3,1} = \alpha^4$ .

For  $i = 12$ , we have  $\sigma_{1,12} = \alpha^{10}$ ,  $\sigma_{2,12} = \alpha^{-24} = \alpha^7$ , and  $\sigma_{3,12} = \alpha^{-29} = \alpha^2$ . Hence, we have  $\sigma(\alpha^{-12}) = 1 + \alpha^{10} + \alpha^7 + \alpha^2 = 0$ .

For  $i = 25$ , we have  $\sigma(\alpha^{-25}) = 1 + \alpha^{28} + \alpha^{12} + \alpha^{25} = 0$  because the parameters are  $\sigma_{1,25} = \alpha^{-3} = \alpha^{28}$ ,  $\sigma_{2,25} = \alpha^{12}$ , and  $\sigma_{3,25} = \alpha^{25}$ .

As a result, the error-position  $i$ 's are at 1, 12, and 25, respectively. Therefore, the error pattern is immediately written as

$$e(x) = x + x^{12} + x^{25}$$

### 7.5.7 Decoder design architecture

Before ending this section, a complete implementation for a BCH decoder is presented. These decoders are typically implemented using time-domain techniques. The decoding of a received BCH word requires that three successive computational processes performed over  $GF(2^m)$  be executed. These processes are the syndrome calculation, error-locator polynomial determination, and the Chien search (with error-value computation for nonbinary codes). However, the details of implementing these functions may be modified somewhat depending on the code and the required data rate. To achieve a very efficient design, the designer must carefully match the architecture of the central processing unit (CPU) to the hardware requirement of the computational subroutines.

In a low-data-rate implementation, the decoder architecture is most efficiently organized as a special-purpose processor having (1) a CPU which may require one or more  $GF(2^m)$  multipliers depending on the required data rate, (2) random-access memory (RAM) for buffering and temporary storage of intermediate outcomes, and (3) the read-only memory (ROM) for storage of computational subroutines. When

the syndrome computation is 0, no error correction is necessary. Accordingly, all of the operations associated with the Berlekamp or Peterson algorithms and Chien's search may be skipped. The single-error case is easily accommodated through special calculations discussed in Sec. 7.4. Thus, in this case, the average computational requirement is substantially reduced.

BCH decoders for implementing the syndrome computation and the Chien search are basically shift registers with  $GF(2^m)$  multipliers and adders attached. The iterative algorithm for solving the  $\sigma(x)$  equation provides a technique that can be implemented in the central processor with an associated  $GF(2^m)$  arithmetic unit and memory at data rates in the several megabits per second range. If the multiplication is done using a general  $GF(2^m)$  multiplier, then a custom integrated circuit performing this function could be used in many places in the decoder. Several such circuits could be used in the syndrome computation, the Chien search, and the CPU. Such a circuit would substantially reduce the implementation complexity of very-high-speed decoders.

The following example is not implemented with a special-purpose processor, but for the demonstration purpose it is presented with the actual circuit layout rather than a block diagram presentation.

**Example 7.20** Consider the decoder architecture of the double-error-correcting (255, 239) BCH code. Let  $\alpha$  be a root of the primitive polynomial  $p(x) = 1 + x^2 + x^3 + x^4 + x^8$  over  $GF(2)$ . Then,  $\alpha$  is also a primitive element of  $GF(2^m)$ . The minimal polynomial  $m_1(x)$  of  $\alpha$  is  $m_1(x) = 1 + x^2 + x^3 + x^4 + x^8$  and the minimal polynomial  $m_3(x)$  of  $\alpha^3$  is  $m_3(x) = 1 + x + x^2 + x^4 + x^5 + x^6 + x^8$ . Therefore, the generator polynomial of this code is

$$g(x) = 1 + x + x^5 + x^6 + x^8 + x^9 + x^{10} + x^{11} + x^{13} + x^{14} + x^{16}$$

Based on this generator polynomial, the encoding circuit for the (255, 239) BCH code is easily implemented as shown in Fig. 7.8.

Suppose that the received polynomial  $r(x) = x^{45} + x^{213}$  is received when the all-zero code word  $c(x) = 0$  is transmitted. This means that two errors occur at the positions  $x^{45}$  and  $x^{213}$ . The double-error-correcting decoder for the (255, 239) BCH code, which can implement all three component units, is shown in Fig. 7.9. In this figure, the first unit is the syndrome generator, the second unit represents ROM for producing  $\sigma(x)$ , and the third one is Chien's searching unit. The pertinent layout for all these units will be illustrated subsequently.

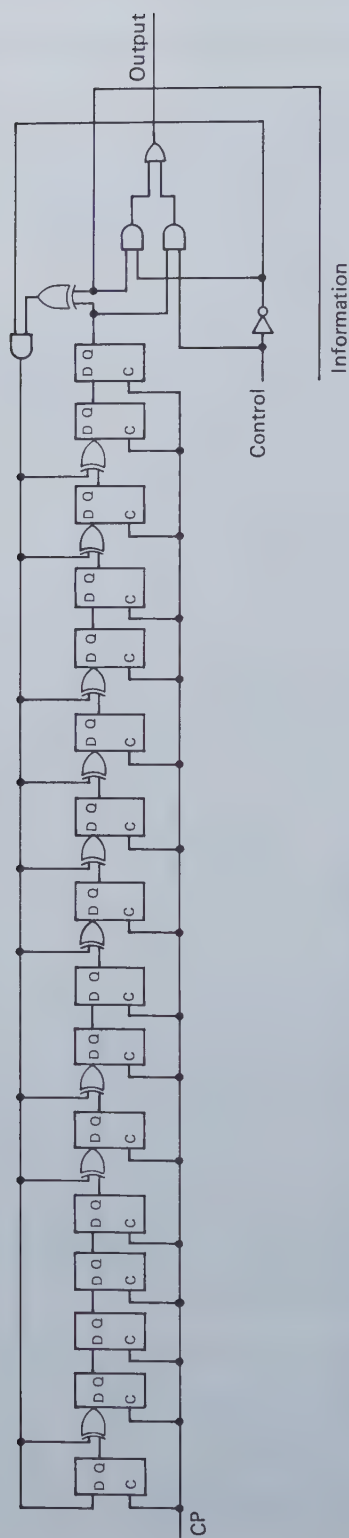
If  $r(x)$  is divided by  $m_k(x)$ ,  $k = 1, 3$ , the syndrome components are easily computed from the remainders  $\gamma_k(x)$  as usual.

$$s_1 = \gamma_1(\alpha) = \alpha^{125} = (1\ 1\ 0\ 0\ 1\ 1\ 0\ 0)$$

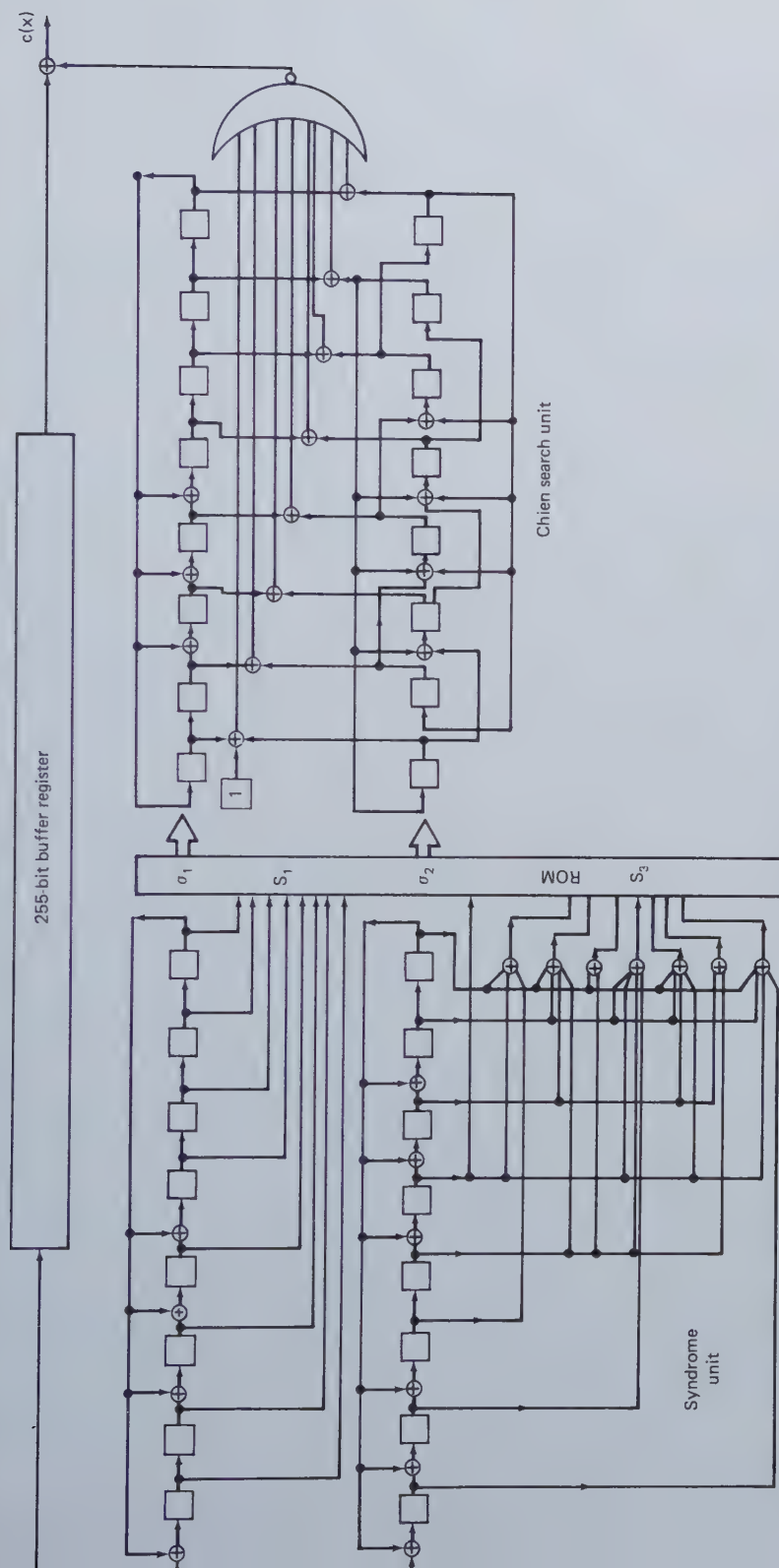
$$s_2 = \gamma_1(\alpha^2) = \alpha^{250} = (0\ 0\ 1\ 1\ 0\ 1\ 1\ 0)$$

$$s_3 = \gamma_3(\alpha^3) = \alpha^{65} = (0\ 1\ 1\ 1\ 1\ 1\ 0\ 1)$$

The syndrome generator is shown in Fig. 7.10.

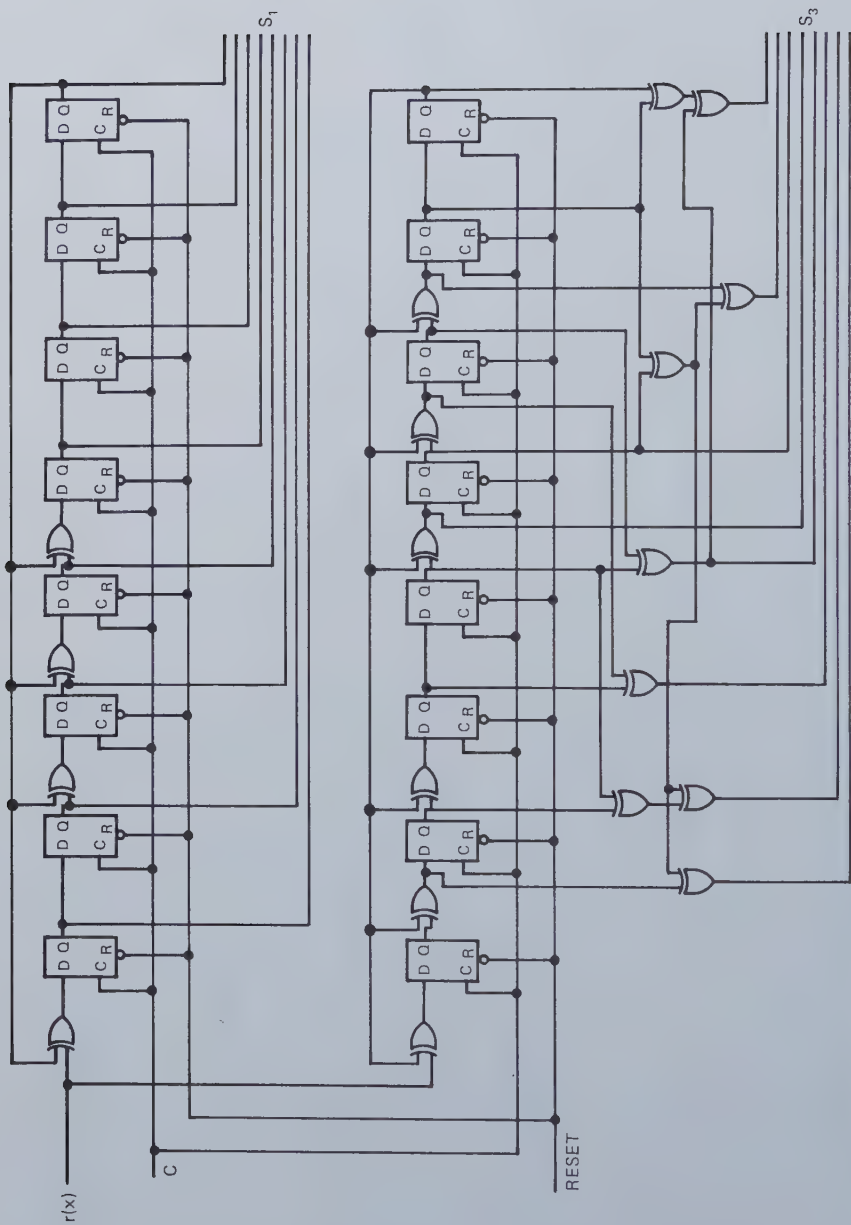


**Figure 7.8** Encoder for the double-error-correcting (255, 239) BCH code.



7.9 Decoding circuit for the double-error-correcting (255, 239) BCH code.





**Figure 7.10** Syndrome computation circuit for the (255, 239) BCH code.

When substituting  $s_1 = \alpha^{125}$  and  $s_3 = \alpha^{65}$  into Eq. (7.40), we obtain the error locator polynomial  $\sigma(x) = 1 + \alpha^{125}x + \alpha^3x^2$ . This  $\sigma(x)$  is also evaluated by using the iterative algorithms of Peterson-Berlekamp or Berlekamp-Massey as shown in Tables 7.9 and 7.10. Thus, the ROM device for  $\sigma(x)$  implementation is as shown in Fig. 7.11.

Finally, Chien's searching unit for seeking the error-location numbers requires two multipliers as shown in Fig. 7.12. The  $\sigma_1$  and  $\sigma_2$  from  $\sigma(x)$  are initially loaded into each register of the two multipliers. They are then multiplied by  $\alpha$  and  $\alpha^2$  and shifted next.

Thus, the decoding circuit for the (255, 239) BCH code is accomplished by integrating these three units as shown in Fig. 7.9.

## 7.6 Reed-Solomon Codes

In this section, we discuss an extremely important and practical class of codes, the Reed-Solomon (RS) codes. A RS code is a cyclic symbol error-correcting code. These codes are coming into widespread use in many communications and computer storage systems. In particular, a concatenated Reed-Solomon convolutional encoding system has been adopted for the deep-space downlink. A convolutional code could be used as the inner code and achieve excellent performance with relatively high overall code rates. Since the convolutional code would operate at a moderately high error rate, the use of short-constraint-length codes with Viterbi decoding is the most promising approach. Also, secure communications systems commonly use a RS code as one method for protection against jamming. RS codes are also used for error control in the data storage systems, such as magnetic drums and photodigital storage systems. The IBM Digital Cypress uses a (61, 50) shortened RS code with symbols from  $GF(2^6)$  for error control. The code used in the IBM 3370 disk storage system is a shortened RS code with symbols from the field  $GF(2^8)$ . Each data block consists of 512 bytes. When a data block is recorded, nine parity-check bytes are appended to it for error control during the read operation.

The importance of RS codes is partly due to their superior error-correcting capabilities, but it is equally due to the availability of an efficient decoding algorithm. The Berlekamp iterative algorithm can be used for efficient decoding of RS codes if a slight modification is made that requires the calculation of the error values at the error locations.

An RS code is a block sequence of finite field  $GF(2^m)$  of  $2^m$  binary symbols, where  $m$  is the number of bits per symbol. This sequence of symbols can be viewed as the coefficients of a code polynomial  $c(x) = c_0 + c_1x + c_2x^2 + \cdots + c_{n-1}x^{n-1}$  where the field elements  $c_i \in GF(2^m)$ .

TABLE 7.9 Solution of  $\sigma(x)$  by the Peterson-Berlekamp Algorithm

| $n$ | $\sigma^{(n)}(x)$                 | $d_n$          | $l_n$ | $n - l_n$ |
|-----|-----------------------------------|----------------|-------|-----------|
| -1  | 1                                 | 1              | 0     | -1        |
| 0   | 1                                 | $\alpha^{125}$ | 0     | 0         |
| 1   | $1 + \alpha^{125}x$               | 0              | 1     | 0         |
| 2   | $1 + \alpha^{125}x$               | $\alpha^{128}$ | 1     | 1         |
| 3   | $1 + \alpha^{125}x + \alpha^3x^2$ | —              | —     | —         |

TABLE 7.10 Solution of  $\sigma(x)$  by the Berlekamp-Massey Algorithm

| $n$ | $\sigma^{(n)}(x)$                 | $l_n$ | $d_n$          | $k_n$ | $d_{k_n}$      |
|-----|-----------------------------------|-------|----------------|-------|----------------|
| -1  | 1                                 | 0     | 1              |       |                |
| 0   | 1                                 | 0     | $\alpha^{125}$ | -1    | 1              |
| 1   | $1 + \alpha^{125}x$               | 1     | 0              | 0     | $\alpha^{125}$ |
| 2   | $1 + \alpha^{125}x$               | 1     | $\alpha^{128}$ | 0     | $\alpha^{125}$ |
| 3   | $1 + \alpha^{125}x + \alpha^3x^2$ | —     | —              | —     | —              |

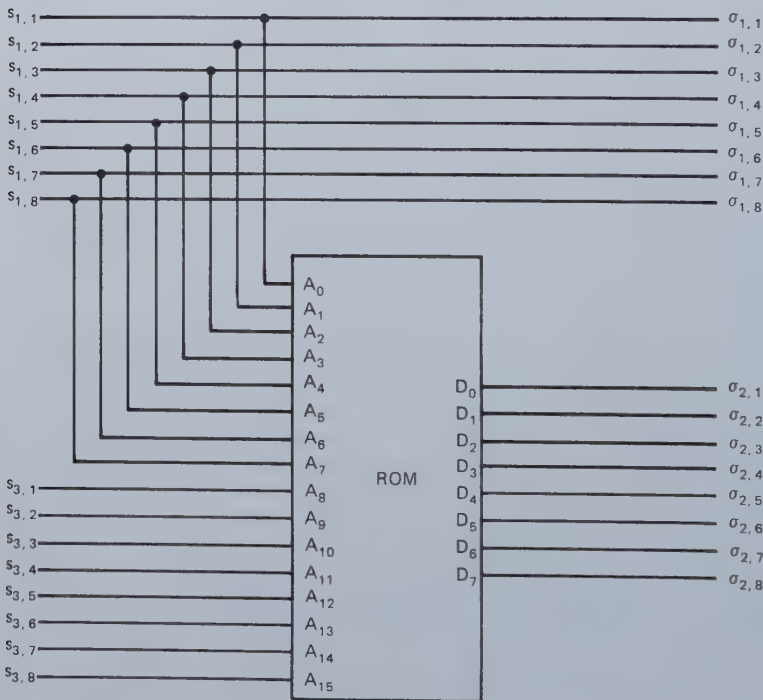


Figure 7.11 ROM circuit for  $\sigma(x)$  of the (255, 239) BCH code.

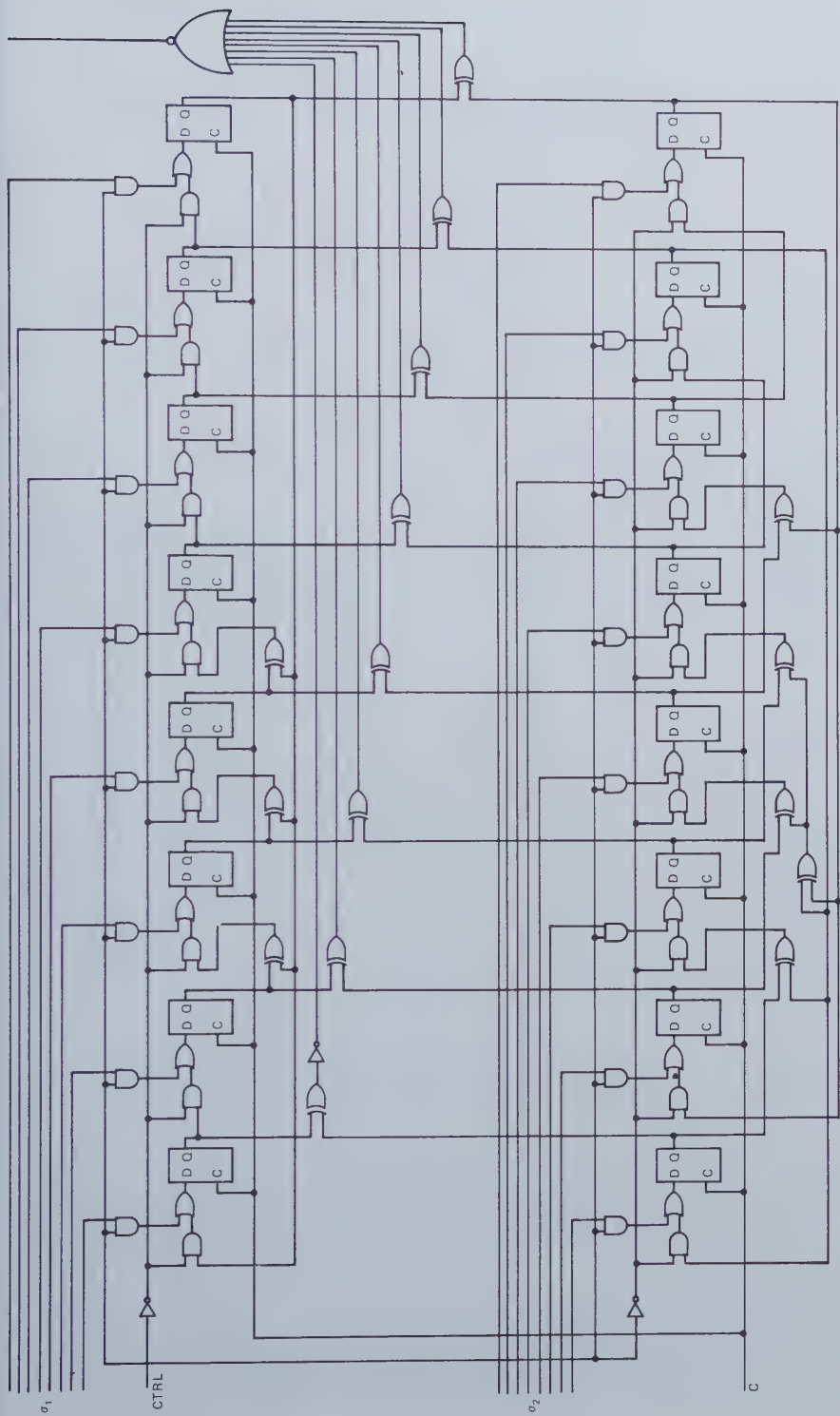


Figure 7.12 Chien's searching circuit for the (255, 239) BCH code.

An  $(n, t)$  RS code with symbols from  $GF(2^m)$  has the following parameters:

|                           |                                      |
|---------------------------|--------------------------------------|
| $n = 2^m - 1$             | code length in symbols               |
| $k = n - 2t$              | number of information symbols        |
| $n - k = 2t$              | number of check symbols              |
| $d_0 = 2t + 1 = d_{\min}$ | designed distance = minimum distance |

where  $t$  is the number of error symbols that can be corrected.

### 7.6.1 Encoding of RS codes

Consider RS codes with code symbols from  $GF(2^m)$ , where  $m$  is the number of bits per symbol.

Let  $d(x) = c_{n-k}x^{n-k} + c_{n-k+1}x^{n-k+1} + \dots + c_{n-1}x^{n-1}$  be the information polynomial and  $p(x) = c_0 + c_1x + \dots + c_{n-k-1}x^{n-k-1}$  be the check polynomial. Then the encoded RS code polynomial is expressed by

$$c(x) = p(x) + d(x) = \sum_{i=0}^{n-1} c_i x^i \quad (7.74)$$

where  $c_i$ ,  $0 \leq i \leq n-1$ , are field elements in  $GF(2^m)$ . Thus, a vector of  $n$  symbols,  $(c_0, c_1, \dots, c_{n-1})$  is a code word if and only if its corresponding polynomial  $c(x)$  is a multiple of the generator polynomial  $g(x)$ . The common method of encoding a cyclic code is to find  $p(x)$  from  $d(x)$  and  $g(x)$ , which is accomplished by dividing  $d(x)$  by  $g(x)$ , which results in an irrelevant quotient  $q(x)$  and an important remainder  $\gamma(x)$ . That is,

$$d(x) = q(x)g(x) + \gamma(x) \quad (7.75)$$

Substituting Eq. (7.75) into (7.74) gives

$$c(x) = p(x) + q(x)g(x) + \gamma(x) \quad (7.76)$$

If we define the check digits as the negatives of the coefficients of  $\gamma(x)$ , i.e.,  $p(x) = -\gamma(x)$ , it follows that

$$c(x) = q(x)g(x) \quad (7.77)$$

This ensures that the code polynomial  $c(x)$  is a multiple of  $g(x)$ . Thus, the RS encoder will perform the above division process to obtain the check polynomial  $p(x)$ . The straightforward method of getting the remainder from a division by a monic polynomial  $g(x)$  is to connect a shift register according to  $g(x)$ .

Let  $\alpha$  be a primitive element of  $GF(2^m)$ . The generator polynomial of a  $t$ -error-correcting RS code of length  $2^m - 1$  is defined by



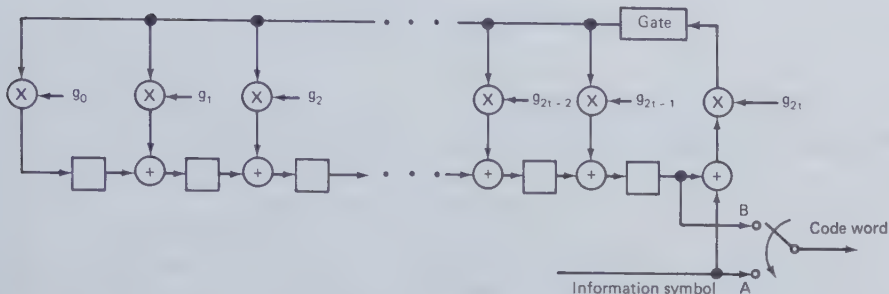


Figure 7.13 An  $(n, t)$  Reed-Solomon encoder over  $GF(2^m)$ .

$$g(x) = (x + \alpha)(x + \alpha^2) \cdots (x + \alpha^{2t}) = \sum_{i=0}^{2t} g_i x^i \quad (7.78)$$

where the coefficients  $g_i$ ,  $0 \leq i \leq 2t$ , are from  $GF(2^m)$ . An  $(n, t)$  RS code generated by  $g(x)$  is an  $(n, n - 2t)$  cyclic code whose code vectors are multiples of  $g(x)$ . Thus, the structure of a  $t$ -error-correcting RS encoder over  $GF(2^m)$  is shown in Fig. 7.13. All lines shown in this figure represent  $m$ -bit symbols. Each  $\oplus$  represents an exclusive-OR of two  $m$ -bit words, and each  $\otimes$  represents a multiplication in the  $GF(2^m)$ .

As a matter of convenience, let the information symbols be  $(c_{2t}, c_{2t+1}, \dots, c_{n-1})$  and the check symbols be  $(c_0, c_1, \dots, c_{2t-1})$ . Initially, all registers are set to 0, and the switch turns on position A. Then, the information symbols are fed into the division circuit of the encoder and are simultaneously transmitted symbol by symbol from the encoder. Thus, the remainder coefficients are computed sequentially by the division circuit. As soon as the information symbol  $c_{2t}$  enters the circuit, the switch is turned to position B. At the very same moment, the first check symbol  $c_{2t-1}$  is formed and transmitted. The  $c_i$  are successively computed and loaded into the registers for  $0 \leq i \leq 2t - 2$ . The gate is then turned off. The rest of the check symbols  $(c_0, c_1, \dots, c_{2t-2})$  are read out of the encoder symbol by symbol.

The complexity of the  $(n, t)$  RS encoder design results from the computation of the check symbols  $c_i$  for  $0 \leq i \leq 2t - 1$ . This computation can be performed by any conventional RS encoder, but according to a recent report, Berlekamp has developed a bit-serial multiplier algorithm applicable to the design for an RS encoder, which can be implemented on a simple VLSI chip.

**Example 7.21** Consider the double-error-correcting  $(15, 11)$  RS code with symbols from  $GF(2^4)$ . Since  $m = 4$  and  $t = 2$ , we have  $k = 15 - 2t = 11$  information symbols. Let  $\alpha$  be a root of the primitive polynomial  $p(x) = 1 + x + x^4$  over  $GF(2)$ . The generator polynomial of this code is

$$\begin{aligned}
 g(x) &= (x + \alpha)(x + \alpha^2)(x + \alpha^3)(x + \alpha^4) \\
 &= \alpha^{10} + \alpha^3x + \alpha^6x^2 + \alpha^{13}x^3 + x^4
 \end{aligned}$$

Let one information symbol be  $d_s = (a_0, a_1, a_2, a_3)$ , where  $a_i \in GF(2)$ ,  $0 \leq i \leq 3$ . It is convenient to represent an information symbol as the following polynomial:

$$d_s(\alpha) = a_0 + a_1\alpha + a_2\alpha^2 + a_3\alpha^3$$

The results of multiplying an information symbol  $d_s(\alpha)$  by the coefficients of  $g(x)$ , i.e.,  $\alpha^{10}$ ,  $\alpha^3$ ,  $\alpha^6$ , and  $\alpha^{13}$ , are as follows:

$$\begin{aligned}
 \alpha^{10}d_s(\alpha) &= (a_0 + a_2 + a_3) + (a_0 + a_1 + a_2)\alpha \\
 &\quad + (a_0 + a_1 + a_2 + a_3)\alpha^2 + (a_1 + a_2 + a_3)\alpha^3 \\
 \alpha^3d_s(\alpha) &= a_1 + (a_1 + a_2)\alpha + (a_2 + a_3)\alpha^2 + (a_0 + a_3)\alpha^3 \\
 \alpha^6d_s(\alpha) &= (a_1 + a_2) + (a_1 + a_3)\alpha + (a_0 + a_2)\alpha^2 + (a_0 + a_1 + a_3)\alpha^3 \\
 \text{and} \quad \alpha^{13}d_s(\alpha) &= (a_0 + a_1 + a_2) + a_3 + a_0\alpha^2 + (a_0 + a_1)\alpha^3
 \end{aligned}$$

Thus, as you see, we could convert the  $GF(2^4)$  multipliers into the symbol adders so as to make the implementation straightforward. The layout of this (15, 11) RS encoder is shown in Fig. 7.14.

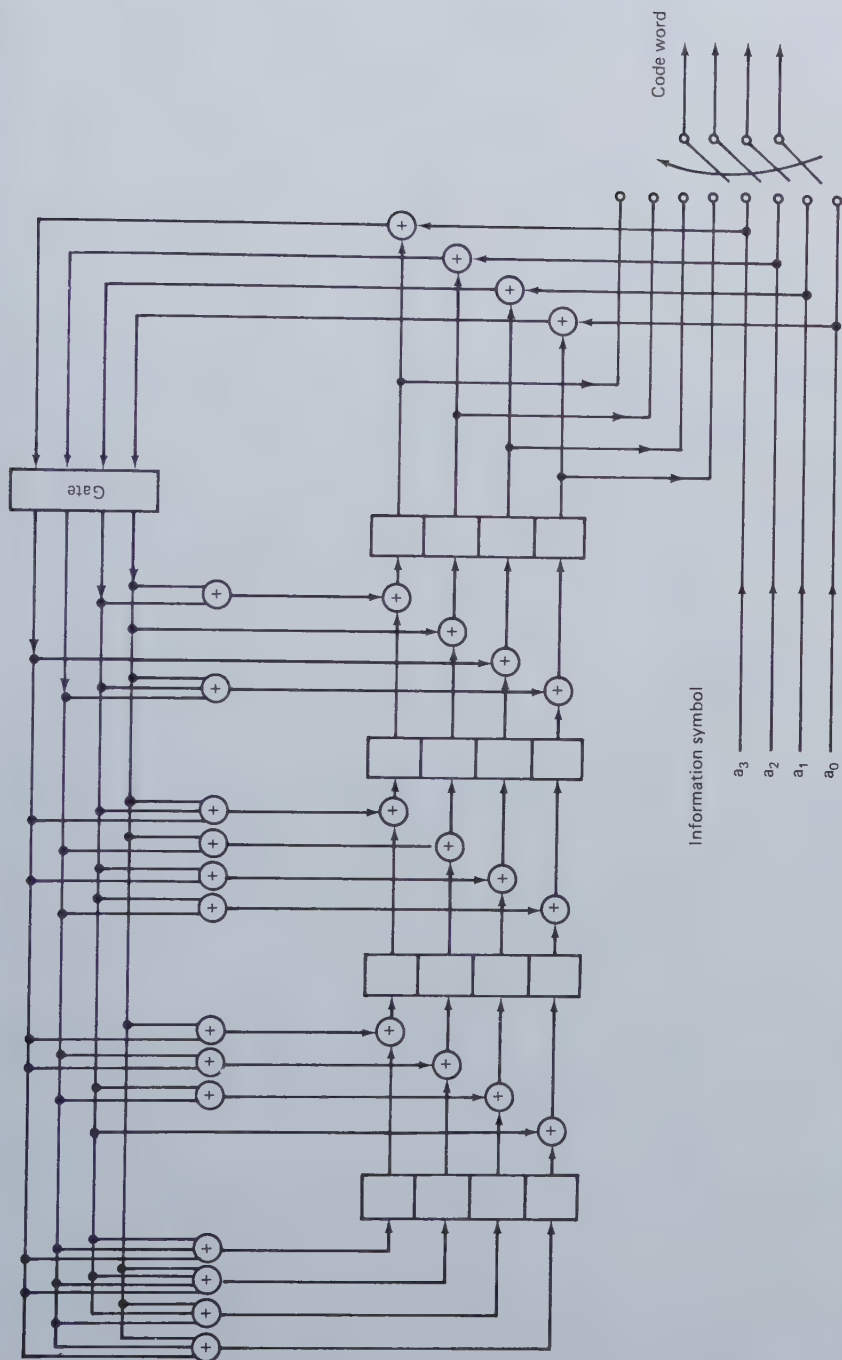
## 7.6.2 Expurgated RS code for a one-symbol-error correction

By slightly changing a  $(2^m - 1, 2^m - 1 - 2t)$  RS code, it is possible to obtain a new expurgated  $(2^m - 1, 2^m - 2 - 2t)$  RS code for  $t = 1$ . This can be done by removing one information symbol but leaving the code length unchanged. This expurgated RS code can be used to remodel a known  $(n, t)$  RS code to fit a specific application. It might also be used to devise a new class of RS code, as shown in Table 7.11. However, it may not be a simple matter to find such modifications that are particularly worthwhile. Specifically, an  $(2^m - 1, 2^m - 3)$  RS code can be expurgated into a  $(2^m - 1, 2^m - 4)$  RS code, which is a single-error-correcting, double-error-detecting RS code.

Let us define the generator polynomial of expurgated RS codes as

$$\begin{aligned}
 g(x) &= \prod_i (x + \alpha^i) \quad i = -1, 0, 1 \\
 &= x^3 + (1 + \alpha + \alpha^{-1})x^2 + (1 + \alpha + \alpha^{-1})x + 1
 \end{aligned} \tag{7.79}$$

$g(x)$  can be increased by one extra factor  $1 + \alpha^{-1}$  without any loss of generality, so that the result is a symmetrical pattern of coefficients



**Figure 7.14** The double-error-correcting (15, 11) RS encoder.

**TABLE 7.11 A New Class of Expurgated ( $2^m - 1$ ,  $2^m - 4$ ) RS Codes for One-Symbol-Error Correction**

| $m$ | $n$ | $k$ |
|-----|-----|-----|
| 3   | 7   | 4   |
| 4   | 15  | 12  |
| 5   | 31  | 28  |
| 6   | 63  | 60  |
| 7   | 127 | 124 |
| 8   | 255 | 252 |

and  $d_{\min} = 4$ . An encoder for expurgated RS codes that uses Eq. (7.79) is depicted in Fig. 7.15. All lines shown in Fig. 7.15 represent  $m$ -bit symbols. Each  $\oplus$  denotes an exclusive-OR of two  $m$ -bit words, and each  $\otimes$  represents multiplication by the factor  $1 + \alpha + \alpha^{-1}$  in  $GF(2^m)$ .

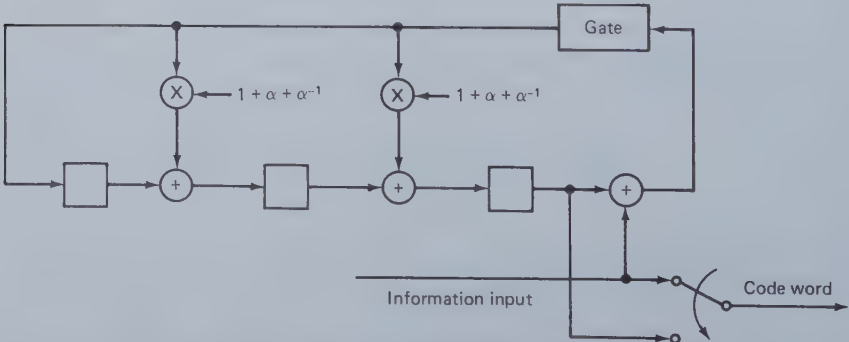
**Example 7.22** Consider the (15, 12) expurgated RS code over  $GF(2^4)$ . A field element  $\beta$  in  $GF(2^4)$  is represented by 0 or  $\alpha^k$  for some  $k$ ,  $0 \leq k \leq 14$ . Using Eq. (7.79), the generator polynomial  $g(x)$  of this code is given by

$$g(x) = x^3 + \alpha^9 x^2 + \alpha^9 x + 1$$

Based on the generator polynomial  $g(x)$ , the encoding circuit for the (15, 12) expurgated RS code over  $GF(2^4)$  is shown in Fig. 7.16.

Let  $r(x)$  be the received polynomial when the code polynomial  $c(x) = 0$  is transmitted. The error-pattern polynomial  $e(x)$  is then equal to  $r(x)$ , i.e.,  $e(x) = r(x)$ . The syndrome components  $s_i$ ,  $i = -1, 0, 1$ , are computed from  $e(x) = r(x) = q(x)(x + \alpha^i) + \gamma_i$  by substituting  $\alpha^i$  for  $x$  in this equation. They are

$$s_i = r(\alpha^i) = e(\alpha^i) = \gamma_i \quad i = -1, 0, 1 \tag{7.80}$$



**Figure 7.15** An expurgated RS encoder for one-symbol-error correction.

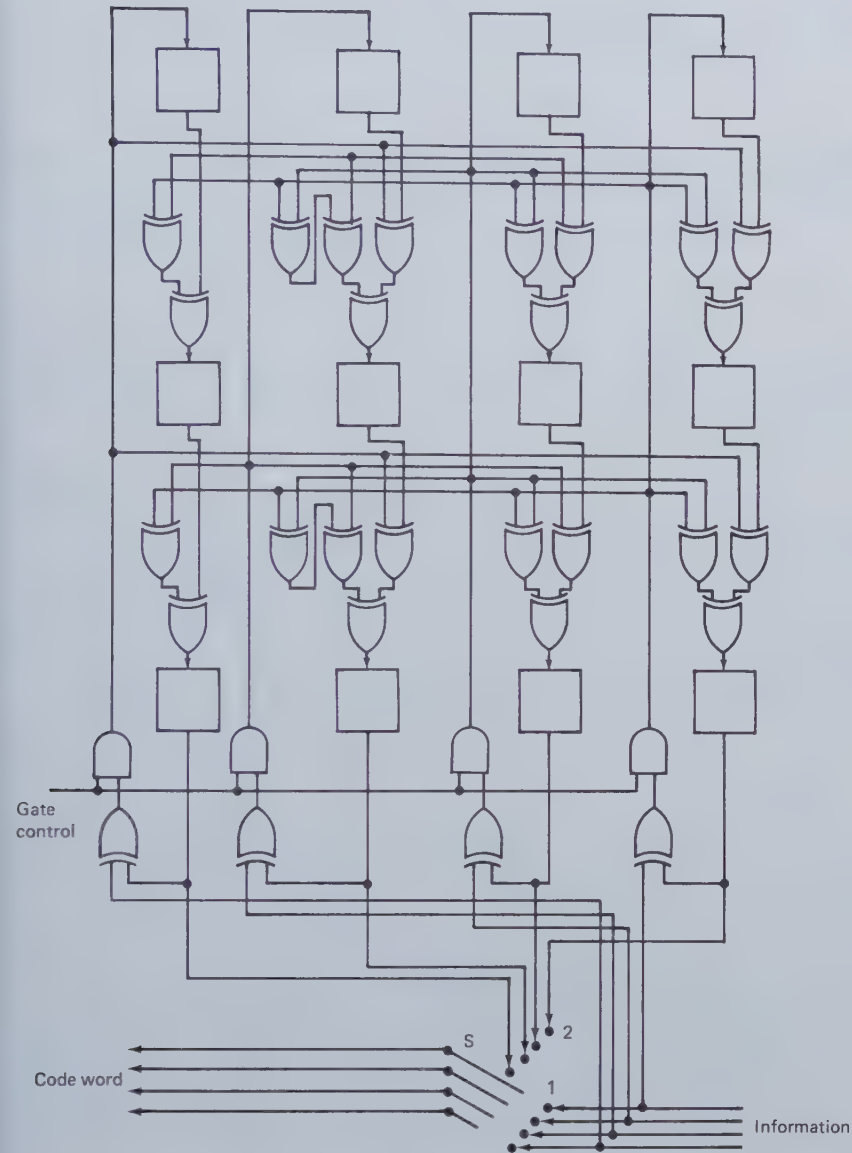


Figure 7.16 Actual design for (15, 12) expurgated RS encoder over  $GF(2^4)$ .



Since an  $(n, k)$  expurgated RS code is a single-error-correcting code, the error-pattern polynomial can be put in the following form:

$$e(x) = e_l x^l \quad 0 \leq l \leq n-1 \quad (7.81)$$

where  $e_l$  is the error value at error position  $x^l$ . Thus the syndrome components are, respectively,

$$s_{-1} = e_l \alpha^{-l} \quad s_0 = e_l \quad s_1 = e_l \alpha^l \quad (7.82)$$

From Eq. (7.82),

$$s_0^2 = s_{-1} s_1 \quad (7.83)$$

Hence, the error value and error position are related to the syndrome components as follows:

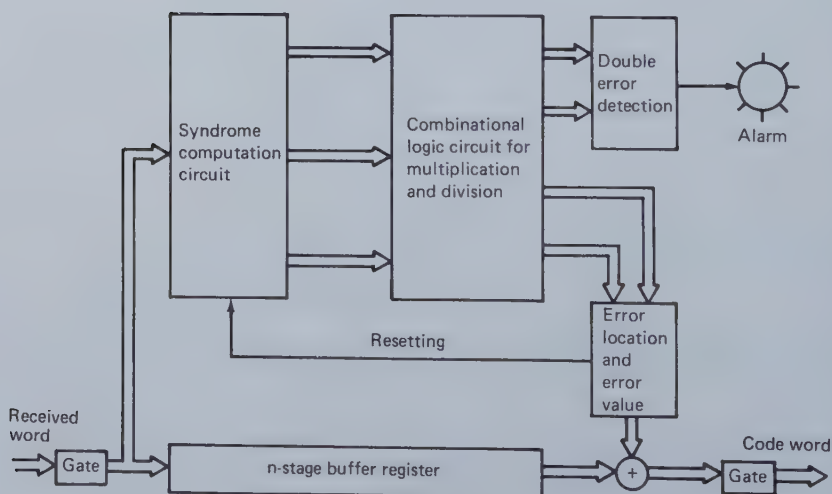
$$e_l = s_0 \quad (7.84)$$

$$\alpha^l = \frac{s_1}{s_0}$$

With the material described above, we could lay out the decoder block diagram for the  $(2^m - 1, 2^m - 4)$  expurgated RS code over  $GF(2^m)$  as shown in Fig. 7.17.

Now, consider the occurrence of double errors

$$e(x) = e_{l_1} x^{l_1} + e_{l_2} x^{l_2} \quad 0 \leq l_1 < l_2 \leq n-1$$



**Figure 7.17** Block diagram for  $(2^m - 1, 2^m - 4)$  expurgated RS decoder over  $GF(2^m)$ .

during data transmission or storage. The syndrome components for this code are

$$s_{-1} = e_{l_1}\alpha^{-l_1} + e_{l_2}\alpha^{-l_2} \quad s_0 = e_{l_1} + e_{l_2} \quad s_1 = e_{l_1}\alpha^{l_1} + e_{l_2}\alpha^{l_2} \quad (7.85)$$

We can see from Eq. (7.85) that the relation  $s_0^2 = s_{-1}s_1$  does not hold. Accordingly, it is impossible to correct double errors, but they can be detected. However, in the case where the double errors tend to occur side by side in bursts, the error pattern will be

$$e(x) = e_k x^{l-1} + e_l x^l \quad (7.86)$$

Thus, the syndrome components like Eq. (7.85) are expressible as shown below:

$$s_{-1} = \alpha^{-l}(e_k\alpha + e_l) \quad s_0 = e_k + e_l \quad s_1 = \alpha^l(e_k\alpha^{-1} + e_l) \quad (7.87)$$

From Eq. (7.87), we could easily determine the error values,  $e_k$  and  $e_l$ , and the error position  $\alpha^l$ . Therefore, the burst type of double symbol errors could be corrected, and that is an interesting subject for the reader to study further.

**Example 7.23** Consider again the (15, 12) expurgated RS code for its decoder design. Using Eqs. (7.81) through (7.84), we can devise the decoding circuit shown in Fig. 7.18. The flowchart that corresponds to Fig. 7.18 for the logic test by computer is shown in Fig. 7.19.

### 7.6.3 Syndrome computation for RS codes

Using the same argument as for the binary BCH code, we define

$$\beta_l = \alpha^{jl} \quad 1 \leq l \leq n$$

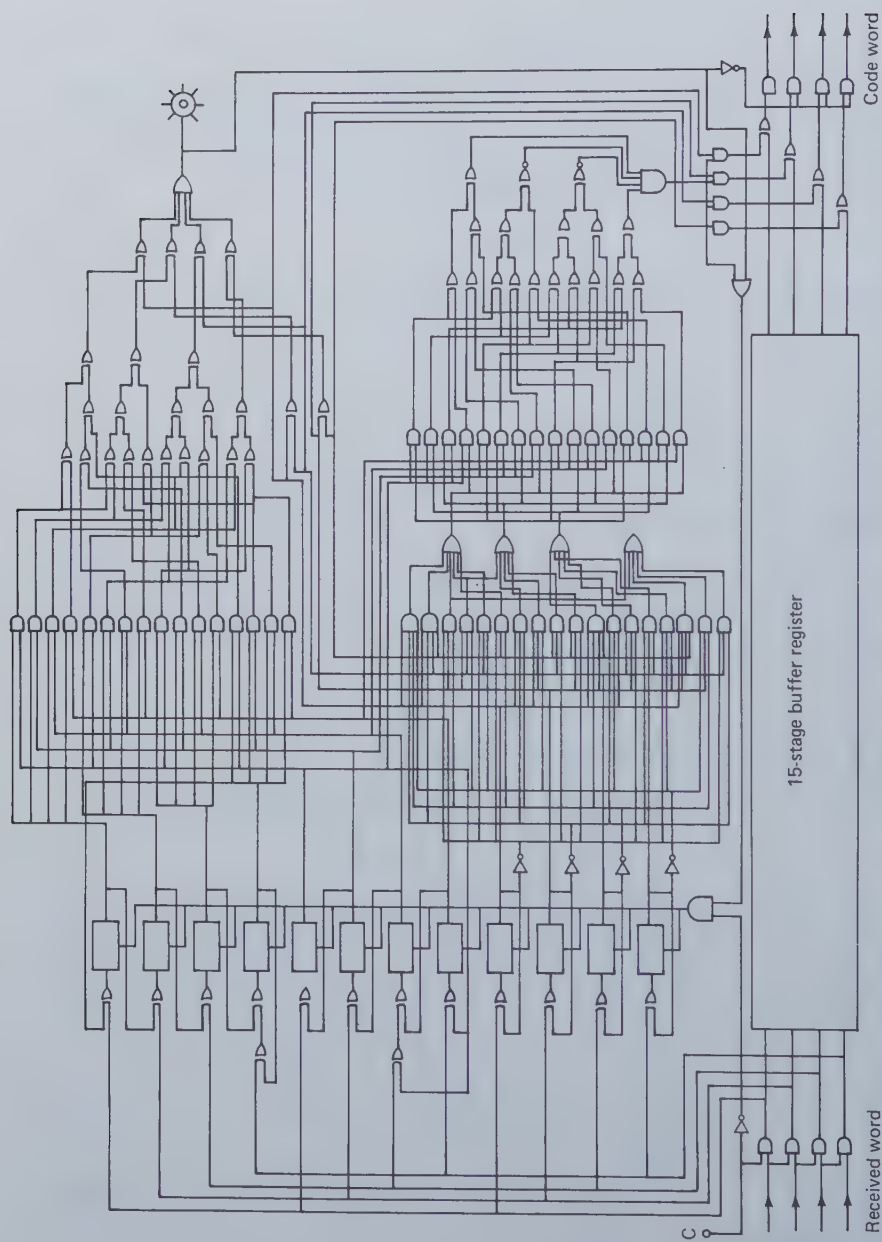
as the error-location numbers when the error pattern  $e(x)$  contains  $\nu$  errors. The  $2t$  syndrome components  $s_i$  are not only obtained by substituting  $\alpha^i$ ,  $1 \leq i \leq 2t$ , into the received polynomial  $r(x)$ ; they can also be computed by dividing  $r(x)$  by  $x + \alpha^i$  as shown below. The components are either

$$s_i = r(\alpha^i) = \sum_{j=1}^{\nu} e_j \alpha^{(ji)^i} \quad 1 \leq i \leq 2t \quad (7.88)$$

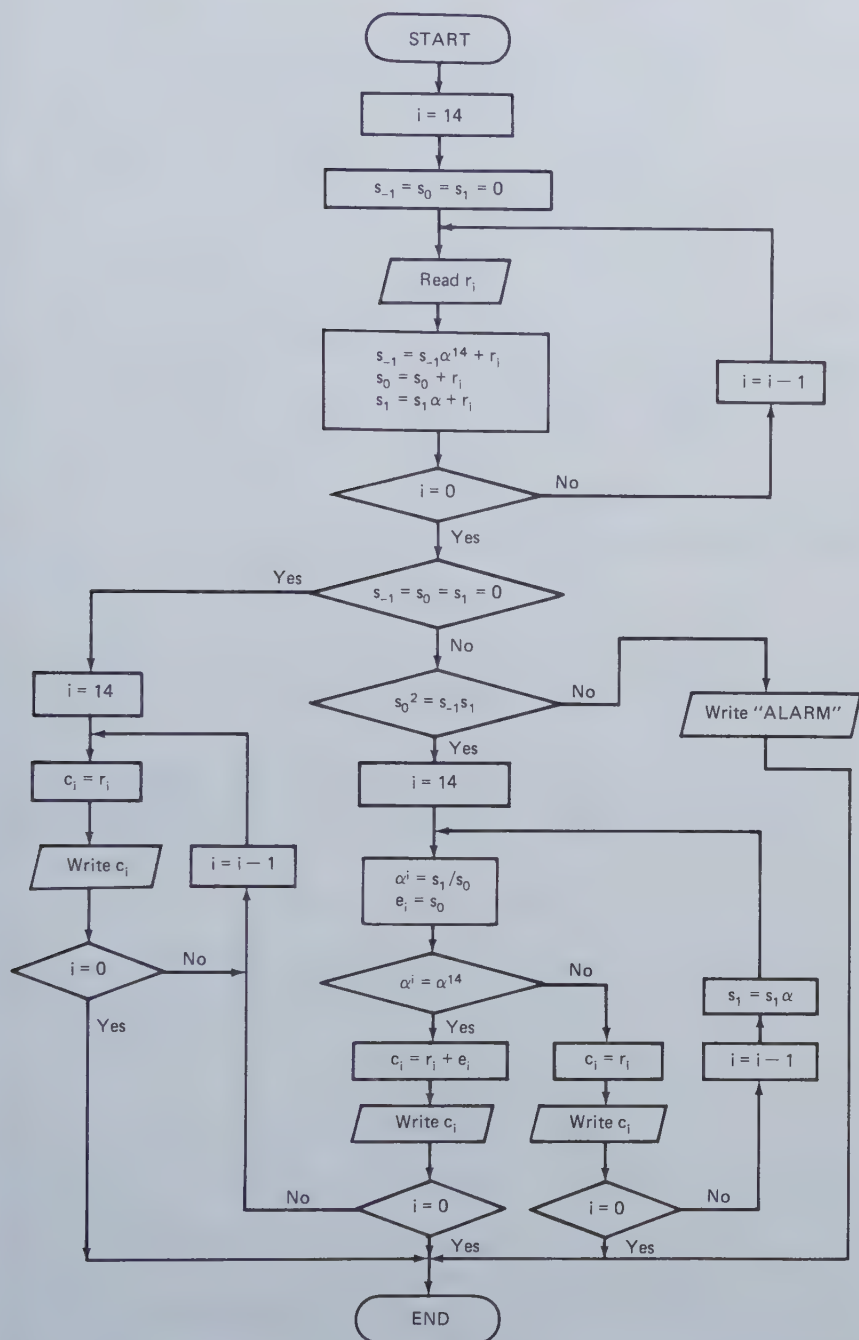
$$\text{or} \quad r(x) = q_i(x)(x + \alpha^i) + \gamma_i \quad (7.89)$$

The result of this division is

$$s_i = r(\alpha^i) = \gamma_i \quad (7.90)$$



**Figure 7.18** Decoder for the (15, 12) expurgated RS code over  $GF(2^4)$ .



**Figure 7.19** Flowchart for computer logic test of the (15, 12) expurgated RS code over  $GF(2^4)$ .

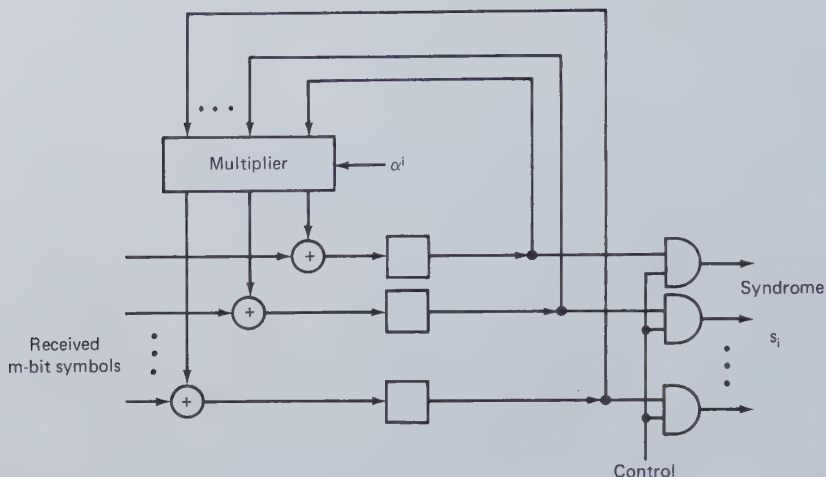


Figure 7.20 Syndrome generator for RS codes with symbols from  $GF(2^m)$ .

if  $\alpha^i$  is substituted in Eq. (7.89). Thus, the syndrome computation can be accomplished by the division circuit illustrated in Fig. 7.20.

**Example 7.24** Consider the triple-error-correcting (31, 25) RS code over  $GF(2^5)$ . Let the received polynomial be  $r(x) = \alpha^8 x^2 + \alpha^2 x^5 + \alpha x^{10}$  when the all-zero code word  $c(x) = 0$  is transmitted. The  $2t = 6$  syndrome components from  $r(x)$  are computed as follows:

$$s_1 = r(\alpha) = \alpha^{10} + \alpha^7 + \alpha^{11} = \alpha$$

$$s_2 = r(\alpha^2) = \alpha^{12} + \alpha^{12} + \alpha^{21} = \alpha^{21}$$

$$s_3 = r(\alpha^3) = \alpha^{14} + \alpha^{17} + \alpha^{31} = \alpha^{23}$$

$$s_4 = r(\alpha^4) = \alpha^{16} + \alpha^{22} + \alpha^{20} = \alpha^{15}$$

$$s_5 = r(\alpha^5) = \alpha^{18} + \alpha^{27} + \alpha^{20} = \alpha^2$$

$$s_6 = r(\alpha^6) = \alpha^{20} + \alpha + \alpha^{30} = \alpha^{13}$$

#### 7.6.4 Error-evaluator polynomial and error values

Consider the RS codes with code symbols from  $GF(2^m)$ , where  $m$  is the number of bits per symbol. Let  $\alpha$  be a primitive element in  $GF(2^m)$ . The generator polynomial of a primitive  $t$ -error-correcting RS code of length  $2^m - 1$  is expressed by



$$g(x) = (x + \alpha)(x + \alpha^2) \cdots (x + \alpha^{2t}) = \sum_{i=0}^{2t} g_i x^i$$

where the coefficients  $g_i$ ,  $0 \leq i \leq 2t$ , are from  $GF(2^m)$ .

If  $c(x)$  is the transmitted code word and  $r(x)$  is the corresponding received word, then the error pattern caused by the channel noise becomes

$$e(x) = r(x) + c(x) = \sum_{i=0}^{n-1} (r_i + c_i)x^i = \sum_{i=0}^{n-1} e_i x^i \quad (7.91)$$

where  $e_i = r_i + c_i$ ,  $0 \leq i \leq n-1$ , is a symbol from  $GF(2^m)$ . If the error pattern  $e(x)$  contains  $\nu$  errors at the locations  $x^{j_k}$ ,  $1 \leq k \leq \nu$ , then we have

$$e(x) = e_{j_1} x^{j_1} + e_{j_2} x^{j_2} + \cdots + e_{j_\nu} x^{j_\nu} \quad (7.92)$$

In order to determine  $e(x)$ , we need to know the error locations  $x^{j_k}$  and the error values  $e_{j_k}$  for  $1 \leq k \leq \nu$ .

As with binary BCH codes, the error-locator polynomial  $\sigma(x)$  for a  $\nu$ -error-correcting RS code

$$\sigma(x) = \prod_{i=1}^{\nu} (1 + \alpha^{j_i} x) = 1 + \sigma_1 x + \sigma_2 x^2 + \cdots + \sigma_{\nu-1} x^{\nu-1} + \sigma_\nu x^\nu \quad (7.93)$$

can also be obtained from Berlekamp's algorithm. Let the syndrome polynomial be

$$s(x) = s_1 x + s_2 x^2 + \cdots + s_\nu x^\nu = \sum_{\lambda=1}^{\nu} s_\lambda x^\lambda \quad \nu \leq 2t \quad (7.94)$$

Using Eqs. (7.93) and (7.94), the error-evaluator polynomial  $\Omega(x)$  is defined as the product of  $\sigma(x)$  and  $s(x)$  such that

$$\begin{aligned} \Omega(x) = \sigma(x)s(x) = 1 + (s_1 + \sigma_1)x + (s_2 + \sigma_1 s_1 + \sigma_2)x^2 + \cdots \\ + (s_\nu + \sigma_1 s_{\nu-1} + \cdots + \sigma_\nu)x^\nu \end{aligned} \quad (7.95)$$

For nonbinary RS codes the error-evaluator polynomial  $\Omega(x)$  is used to find the error value (or error magnitude) at each error location as described below.

Suppose that  $\nu$  errors have occurred in locations corresponding to the indices  $j_1 < j_2 < \cdots < j_\nu \leq n-1$ . Then, the syndrome components can be expressed as

$$s_\lambda = \sum_{k=1}^{\nu} e_{j_k} (\alpha^{j_k})^\lambda \quad 1 \leq \lambda \leq 2t \quad (7.96)$$

where  $\alpha^{j_k}$  for  $k = 1, 2, \dots, v$  are defined as the error-location numbers at position indices  $j_k$ . For convenience sake, let us consider the syndrome polynomial of infinite degree such that

$$s(x) = \sum_{\lambda=0}^{\infty} s_{\lambda} x^{\lambda} \quad (7.97)$$

Substituting Eq. (7.96) into Eq. (7.97) gives

$$s(x) = \sum_{k=1}^v e_{j_k} \sum_{\lambda=0}^{\infty} (\alpha^{j_k})^{\lambda} x^{\lambda} \quad (7.98)$$

If the following identity is utilized,

$$\sum_{\lambda=0}^{\infty} \alpha^{j_k \lambda} x^{\lambda} = 1 + e^{j_k x} + e^{2j_k x^2} + \dots = \frac{1}{1 + e^{j_k x}}$$

Eq. (7.98) becomes

$$s(x) = \sum_{k=1}^v \frac{e_{j_k}}{1 + e^{j_k x}} \quad (7.99)$$

Using Eqs. (7.93) and (7.99), the error-evaluator polynomial  $\Omega(x)$  of degree less than  $v$  can be written as

$$\Omega(x) = \sum_{k=1}^v e_{j_k} \prod_{\substack{l=1 \\ l \neq k}}^v (1 + \alpha^{j_l} x) \quad (7.100)$$

Thus, the error value at location  $x = \alpha^{-j_m}$  is easily obtained as

$$e_{j_m} = \frac{\Omega(\alpha^{-j_m})}{\prod_{\substack{l=1 \\ l \neq m}}^v (1 + \alpha^{j_l} \alpha^{-j_m})} \quad (7.101)$$

This error magnitude is important because the error-pattern polynomial  $e(x)$  is composed from it. The complete decoding process for  $t$ -random-error-correcting  $(n, k)$  RS codes is best explained by the following example.

**Example 7.25** Consider again the triple-error-correcting RS (31, 25) code with symbols from  $GF(2^5)$ . The generator polynomial of this code is

$$\begin{aligned} g(x) &= (x + \alpha)(x + \alpha^2)(x + \alpha^3)(x + \alpha^4)(x + \alpha^5)(x + \alpha^6) \\ &= \alpha^{21} + \alpha^{24}x + \alpha^{16}x^2 + \alpha^{24}x^3 + \alpha^9x^4 + \alpha^{10}x^5 + x^6 \end{aligned}$$

As assumed in Example 7.24, the received vector is

$$\mathbf{r} = (0\ 0\ \alpha^8\ 0\ 0\ \alpha^2\ 0\ 0\ 0\ 0\ \alpha\ 0\ 0\ 0\ 0\ 0\ 0\ 0\ 0\ 0\ 0\ 0\ 0\ 0)$$

The syndrome vector of this code is computed from the received vector, as obtained in the previous example. That is,

$$\begin{aligned} \mathbf{s} &= (s_1, s_2, s_3, s_4, s_5, s_6) \\ &= (\alpha, \alpha^{21}, \alpha^{23}, \alpha^{15}, \alpha^2, \alpha^{13}) \end{aligned}$$

The error-locator polynomial  $\sigma(x)$  can be found by applying the iterative algorithm as follows:

1.  $n = 0, m = -1$

$$\begin{aligned}\sigma^{(1)}(x) &= \sigma^{(0)}(x) - d_0 d_{-1}^{-1} x \sigma^{(-1)}(x) \\ &= 1 + \alpha x \\ d_1 &= s_2 + s_1 \sigma_1^{(1)} = \alpha^{21} + \alpha^2 = \alpha^{13}\end{aligned}$$

2.  $n = 1, m = 0$

$$\begin{aligned}\sigma^{(2)}(x) &= \sigma^{(1)}(x) - d_1 d_0^{-1} x \sigma^{(0)}(x) \\ &= 1 + \alpha x + \alpha^{13} \alpha^{-1} x \\ &= 1 + \alpha^{20} x \\ d_2 &= s_3 + s_2 \sigma_1^{(2)} = \alpha^{23} + \alpha^{10} = \alpha^{24}\end{aligned}$$

3.  $n = 2, m = 0$

$$\begin{aligned}\sigma^{(3)}(x) &= \sigma^{(2)}(x) - d_2 d_0^{-1} x^2 \sigma^{(0)}(x) \\ &= 1 + \alpha^{20} x + \alpha^{24} \alpha^{-1} x^2 \\ &= 1 + \alpha^{20} x + \alpha^{23} x^2 \\ d_3 &= s_4 + s_3 \sigma_1^{(3)} + s_2 \sigma_2^{(3)} \\ &= \alpha^{15} + \alpha^{12} + \alpha^{13} = \alpha^8\end{aligned}$$

4.  $n = 3, m = 2$

$$\begin{aligned}\sigma^{(4)}(x) &= \sigma^{(3)}(x) - d_3 d_2^{-1} x \sigma^{(2)}(x) \\ &= 1 + \alpha^{20} x + \alpha^{23} x^2 + \alpha^{15} x + \alpha^4 x^2 \\ &= 1 + \alpha^{17} x + \alpha^{15} x^2 \\ d_4 &= s_5 + s_4 \sigma_1^{(4)} + s_3 \sigma_2^{(4)} \\ &= \alpha^2 + \alpha + \alpha^7 = \alpha^{30}\end{aligned}$$

5.  $n = 4, m = 2$

$$\begin{aligned}\sigma^{(5)}(x) &= \sigma^{(4)}(x) - d_4 d_2^{-1} x^2 \sigma^{(2)}(x) \\ &= 1 + \alpha^{17} x + \alpha^{22} x^2 + \alpha^{26} x^3 \\ d_5 &= s_6 + s_5 \sigma_1^{(5)} + s_4 \sigma_2^{(5)} + s_3 \sigma_3^{(5)} \\ &= \alpha^{13} + \alpha^{19} + \alpha^6 + \alpha^{18} = \alpha^{17}\end{aligned}$$

6.  $n = 5, m = 4$

$$\begin{aligned}\sigma^{(6)}(x) &= \sigma^{(5)}(x) - d_5 d_4^{-1} x \sigma^{(4)}(x) \\ &= 1 + \alpha^4 x + \alpha^5 x^2 + \alpha^{17} x^3\end{aligned}$$

Since  $\sigma(x) = \sigma^{(6)}(x)$ , the error-locator polynomial is

$$\sigma(x) = 1 + \alpha^4 x + \alpha^5 x^2 + \alpha^{17} x^3$$

where  $\sigma_0 = 1$ ,  $\sigma_1 = \alpha^4$ ,  $\sigma_2 = \alpha^5$ , and  $\sigma_3 = \alpha^{17}$ .

By Peterson's substitution rule, we can easily find that  $\alpha^{21}$ ,  $\alpha^{26}$ , and  $\alpha^{29}$  are roots of  $\sigma(x)$ . The reciprocals of these roots are to be the error-location numbers of  $e(x)$ . These numbers are calculated as  $\alpha^{10}$ ,  $\alpha^5$ , and  $\alpha^2$ . Thus, the triple errors occur at positions  $x^2$ ,  $x^5$ , and  $x^{10}$ . As the final step, let us find the error values at  $x^2$ ,  $x^5$ , and  $x^{10}$  by first calculating the error-evaluator polynomial  $\Omega(x)$  and applying it to Eq. (7.101). From Eq. (7.95), the error-evaluator polynomial is

$$\begin{aligned}\Omega(x) &= 1 + (\alpha + \alpha^4)x + (\alpha^{21} + \alpha^4\alpha + \alpha^5)x^2 + (\alpha^{23} + \alpha^4\alpha^{21} + \alpha^5\alpha + \alpha^{17})x^3 \\ &= 1 + \alpha^{30}x + \alpha^{21}x^2 + \alpha^{23}x^3\end{aligned}$$

As with binary BCH codes, we define  $\beta_k = \alpha^{j_k}$ ,  $k = 1, 2, \dots, v$ , as the error-location numbers. Hence, we have  $\beta_1 = \alpha^2$ ,  $\beta_2 = \alpha^5$ , and  $\beta_3 = \alpha^{10}$ . Using Eq. (7.101), the error values at these error locations are finally computed as follows:

$$\begin{aligned}e_2 &= \frac{\Omega(\alpha^{-2})}{(1 + \alpha^5\alpha^{-2})(1 + \alpha^{10}\alpha^{-2})} = \frac{\alpha^{26}}{\alpha^{18}} = \alpha^8 \\ e_5 &= \frac{\Omega(\alpha^{-5})}{(1 + \alpha^2\alpha^{-5})(1 + \alpha^{10}\alpha^{-5})} = \frac{\alpha^{30}}{\alpha^{28}} = \alpha^2 \\ e_{10} &= \frac{\Omega(\alpha^{-10})}{(1 + \alpha^2\alpha^{-10})(1 + \alpha^5\alpha^{-10})} = \frac{\alpha^{10}}{\alpha^9} = \alpha\end{aligned}$$

Thus, the error-pattern polynomial is easily found to be

$$\begin{aligned}e(x) &= e_2 x^2 + e_5 x^5 + e_{10} x^{10} \\ &= \alpha^8 x^2 + \alpha^2 x^5 + \alpha x^{10}\end{aligned}$$

The decoding process is completed by taking  $c(x) = r(x) + e(x)$  which should be the all-zero transmitted code word as we assumed.

So far, we have dealt primarily with the random-symbol-error-correcting RS codes. The burst-error-correcting RS codes will be discussed in the following subsection.

### 7.6.5 Burst-error correction

Consider a  $t$ -error-correcting RS code with code symbols from  $GF(2^m)$ . If each symbol is represented by its corresponding  $m$ -bit byte, we ob-

tain a binary  $(n, k)$  linear code with the parameters of  $n = m(2^m - 1)$  and  $k = m(2^m - 1 - 2t)$ . This code is capable of correcting any error pattern that affects  $t$  or fewer  $m$ -bit bytes. It is immaterial whether there exists one bit error in a byte or there exist several bit errors in a byte. The error is still counted as one byte error. The received word is divided into  $2^m - 1$  bytes, and then each byte is transformed back into a symbol from  $GF(2^m)$ . Thus, if an error pattern affects  $t$  or fewer bytes, it will affect  $t$  or fewer symbols in a RS code. We shall call this code a multiple-phased-burst-error-correcting code. In order to present a burst-error correction application, the following example will be used.

**Example 7.26** Consider the double-error-correcting  $(7, 3)$  RS code with symbols from  $GF(2^3)$ . Since  $t = 2$ , the  $(7, 3)$  RS code can correct two-symbol errors. The generator polynomial of this code is then

$$\begin{aligned} g(x) &= (x + \alpha)(x + \alpha^2)(x + \alpha^3)(x + \alpha^4) \\ &= \alpha^3 + \alpha x + x^2 + \alpha^3 x^3 + x^4 \end{aligned}$$

Let the transmitted code vector be

$$\mathbf{c} = (\alpha^3, \alpha, 1, \alpha^3, 1, 0, 0)$$

The Galois field  $GF(2^3)$  is represented as 0, and the powers of a primitive root  $\alpha$  are represented by using  $\alpha^3 = 1 + \alpha$ , as shown in Table 7.12. If the code vector  $\mathbf{c} = (\alpha^3, \alpha, 1, \alpha^3, 1, 0, 0)$  from the  $(7, 3)$  RS code is converted into the binary form, it becomes

$$\mathbf{c} = (1 \ 1 \ 0, \ 0 \ 1 \ 0, \ 1 \ 0 \ 0, \ 1 \ 1 \ 0, \ 1 \ 0 \ 0, \ 0 \ 0 \ 0, \ 0 \ 0 \ 0)$$

with the aid of Table 7.12. If the error pattern caused by binary channel noise is assumed to be  $\mathbf{e} = (0, 0, 0, 1, \alpha^6, 0, 0)$ , then its binary form is

$$\mathbf{e} = (0 \ 0 \ 0, \ 0 \ 0 \ 0, \ 0 \ 0 \ 0, \ 1 \ 0 \ 0, \ 1 \ 0 \ 1, \ 0 \ 0 \ 0, \ 0 \ 0 \ 0)$$

**TABLE 7.12**  $GF(2^3)$  Represented as Powers of  $\alpha$

| $i$ | $\alpha^i$ |
|-----|------------|
| 0   | 1 0 0      |
| 1   | 0 1 0      |
| 2   | 0 0 1      |
| 3   | 1 1 0      |
| 4   | 0 1 1      |
| 5   | 1 1 1      |
| 6   | 1 0 1      |



Therefore, from  $\mathbf{r} = \mathbf{c} + \mathbf{e}$ , the received vector would be

$$\mathbf{r} = (1 \ 1 \ 0, \ 0 \ 1 \ 0, \ 1 \ 0 \ 0, \ 0 \ 1 \ 0, \ 0 \ 0 \ 1, \ 0 \ 0 \ 0, \ 0 \ 0 \ 0)$$

which differs from  $\mathbf{c}$  in three positions in two 3-bit bytes. Thus, this received word  $\mathbf{r}$  can be mapped into a vector form over  $GF(2^3)$  as follows:

$$\mathbf{r} = (\alpha^3, \alpha, 1, \alpha, \alpha^2, 0, 0)$$

Since each symbol of the  $(7, 3)$  RS code over  $GF(2^3)$  is represented by a 3-bit byte, the corresponding  $(n, k)$  linear code over  $GF(2)$  has the code length  $n = 3(2^3 - 1) = 21$  and the information bits  $k = 21 - 3 \cdot 2 \cdot 2 = 9$ . This converted  $(21, 9)$  binary code is capable of correcting any error pattern that affects  $t$  or fewer 3-bit bytes. However, it would be somewhat difficult to correct triple errors in a code word of length 21, but when the error pattern  $e(x)$  is converted into a vector form of  $\mathbf{e} = (0, 0, 0, 1, \alpha^6, 0, 0)$ , it only has a Hamming weight of 2. Therefore, we take advantage of the fact that a triple random error over  $GF(2)$  has occurred in a short burst over  $GF(2^3)$ . In this way, the  $(7, 3)$  RS code over  $GF(2^3)$  has become the  $(21, 9)$  linear cyclic code over  $GF(2)$  whose generator polynomial is  $g(x) = 1 + x^2 + x^3 + x^6 + x^8 + x^{11} + x^{12}$ . In fact, the burst-error-correcting  $(21, 9)$  code is capable of correcting any pattern of a short single burst of length  $b = 6$ . This problem can be easily solved, either analytically or with the aid of a computer. This problem will be left to the reader as an exercise.

In general, it is obvious that a  $(n, t)$  RS code over  $GF(2^m)$  can be implemented as a linear  $(m(2^m - 1), m(2^m - 1 - 2t))$  cyclic code over  $GF(2)$ . But this transformed binary code is capable of correcting any burst-error pattern that does not affect more than  $t$  symbols in an RS code word over  $GF(2^m)$ . Like Fire codes, some burst-correcting cyclic codes derived from the  $(n, t)$  RS codes can be implemented by the error-trapping technique discussed in Chap. 6.

### 7.6.6 Application of RS codes to concatenated coding

In this subsection, we consider the application of RS codes to concatenated coding. Concatenated coding was introduced by Forney in 1966 as a practical technique for implementing a code with a very long block length and large error-correcting capability. A practical technique for implementing such a code is that of concatenating two or more codes. Since this can be accomplished by utilizing multiple levels of coding, numerous configurations are possible. Such a typical application will provide the reader with much of the insight required to effectively utilize coding in communication systems. The most common approach uses two levels of coding, i.e., an inner code over  $GF(2)$  and an outer code with symbols from  $GF(2^m)$ . Reed-Solomon codes are commonly used as the outer code, whereas many different codes (short-block code, orthogonal code, convolutional code, etc.) have been sug-

gested for use as the inner code. Concatenated codes are effective against a mixture of random errors and bursts. Scattered random errors in a received word are correctable by the inner code. But bursts through a few bytes may not be corrected by the inner code. These bursts of a few bytes can then be corrected by the outer code. A simple concatenated code is formed from an  $(n_1, k_1)$  binary inner code and an  $(n_2, k_2)$  nonbinary outer code with symbols from  $GF(2^{k_1})$ . Thus, the resultant code is equivalent to an  $(n_1 n_2, k_1 k_2)$  binary linear code.

The basic concept of concatenated codes for two levels of coding is illustrated in Fig. 7.21. Encoding consists of two steps, as shown in the figure. The outer code is a nonbinary code which utilizes  $k_1$ -bit symbols. The  $k_2$  symbols, namely, the  $k_1 k_2$  binary information digits are divided into  $k_2$   $k_1$ -bit bytes. Here we say that the  $k_2$   $k_1$ -bit bytes interpret  $k_2$  bytes of  $k_1$  information bits. These  $k_2$   $k_1$ -bit bytes (or symbols) enter the outer encoder from the data source to form the code vectors of  $n_2$   $k_1$ -bit bytes. The most natural choice for outer codes are the RS codes. The RS codes have been most widely used because they are maximum-distance-separable codes ( $d_{\min} = 2t + 1$ ). The outer code is assumed to be a linear  $(k_1 n_2, k_1 k_2)$  block code converted from an  $(n_2, t)$  RS code that has  $n_2$   $k_1$ -bit symbols, where  $k_2$  of these symbols are information symbols. The  $n_2$   $k_1$ -bit symbols from the outer encoder are further encoded into  $n_2 n_1$ -bit bytes (or  $n_1 n_2$  bits) by the inner encoder. As mentioned above, the short-block, orthogonal or biorthogonal, and convolutional codes are commonly used for the inner codes. Here  $n_1 - k_1$  parity-check bits are appended to give an inner code of block length  $n_1$  bits. Thus, the  $n_1$ -bit symbol enters the channel, but a parallel-to-serial conversion is typically used to allow binary transmission. Thus the resulting concatenated code has the overall length  $n = n_1 n_2$  bits with  $k = k_1 k_2$  information bits per overall code word and with code rate  $R = R_1 R_2 = (k_1/n_1)(k_2/n_2)$ . If the minimum distances of the inner and outer codes are  $d_1$  and  $d_2$ , respectively, then the overall minimum distance of their concatenation is at least  $d_1 d_2$ .

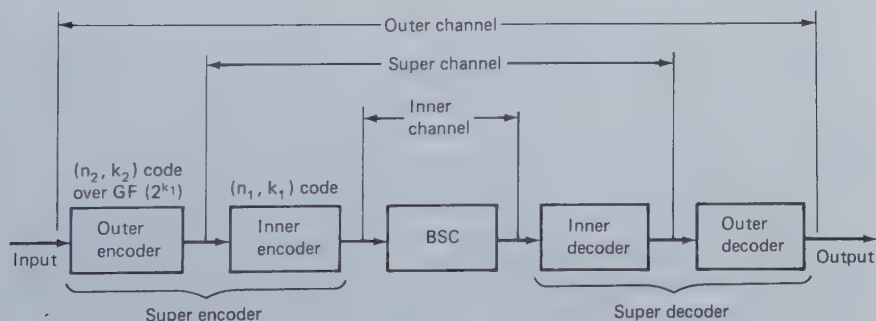


Figure 7.21 Concatenated coding system.

Although the overall length of the code is  $n_1 n_2$ , the structure imposed by the concatenation concept allows the decoding operation to be performed by two decoders for codes of  $n_1$  and  $n_2$ , respectively. Decoding of the concatenated system can also be done by the reverse-order process. When  $n_2$   $n_1$ -bit bytes arrive at the inner decoder, the check bits are removed and they are decoded into a sequence of  $n_2$   $k_1$ -bit bytes. These  $n_2$   $k_1$ -bit bytes are then decoded again by the outer decoder to leave the final corrected data in  $k_2$   $k_1$ -bit bytes. As we shall see, this technique significantly reduces the complexity that would be required with a single level of coding to achieve the same error rate.

The combination of inner encoder, binary symmetric channel (BSC), and inner decoder is sometimes referred to as the super channel. Likewise, the combination of outer and inner encoders is referred to as the super encoder, whereas the combination of inner and outer decoders is called the super decoder. We shall illustrate this coding idea with the following numerical example.

**Example 7.27** Suppose that the (7, 4) Hamming code is used as the inner code on a BSC with the transition probability  $p = .033$ . Consider the concatenation of the (15, 11) RS code with symbols from  $GF(2^4)$  and the binary (7, 4) Hamming code. Each code symbol of the RS code is represented by a 4-bit byte. Each 4-bit byte is then encoded into a 7-bit code word in the (7, 4) Hamming code. This 7-bit code word is transmitted and received by the inner decoder where the 3-bit parity check is removed and then decoded into a 4-bit byte after single-error correction. These fifteen 4-bit bytes are decoded once again by the outer decoder into the eleven 4-bit bytes, which is the corrected data sequence to be sent to the user. The resultant concatenated code is the (105, 44) binary code with a code rate of 0.42. Since the minimum distance of the (7, 4) Hamming code is 3 and the minimum distance of the (15, 11) RS code is 5, the concatenated code has a minimum distance of at least 15.

In the super channel, i.e., inner encoder–BSC–inner decoder part of Fig. 7.21, the probability of an inner-decoder error is

$$\epsilon = P[c_1 \neq r_1] = \sum_{i=2}^7 \binom{7}{i} p^i (1-p)^{7-i} = .0205$$

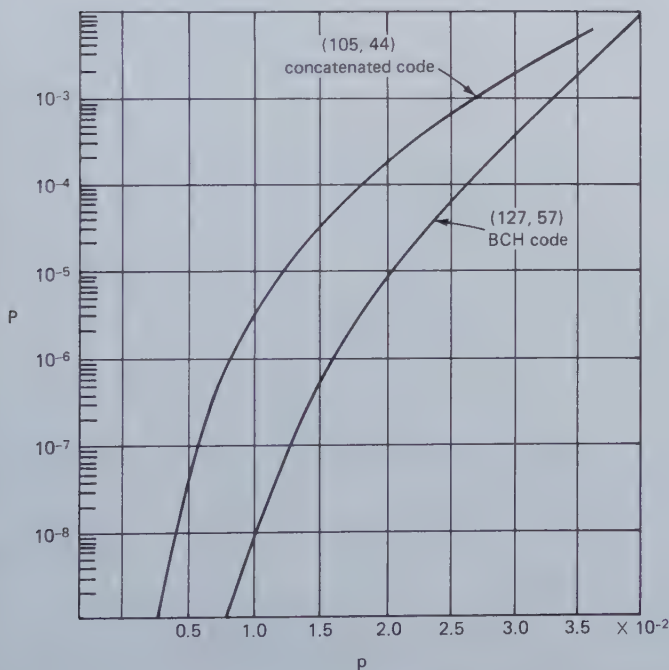
for  $p = .033$ . This super channel is a discrete memoryless channel (DMC) with 16 inputs and outputs from  $GF(2^4)$ . Now, consider the case where the burst noise in the super channel is to be reduced by using the (15, 11) RS code. The RS outer encoder in Fig. 7.21 takes 11 information symbols  $\alpha = (\alpha_0, \alpha_1, \alpha_2, \dots, \alpha_{10})$  from  $GF(2^4)$ , which are actually 44 bits (eleven 4-bit bytes) from the original source, and produces a code word  $\mathbf{c} = (c_0, c_1, c_2, \dots, c_{14})$  from the (15, 11) RS code. The super channel deforms  $\mathbf{c}$  as a result of the burst noise, and the received word is then  $\mathbf{r} = (r_0, r_1, r_2, \dots, r_{14})$ . The RS decoder then produces an estimate  $\hat{\alpha} = (\hat{\alpha}_0, \hat{\alpha}_1, \hat{\alpha}_2, \dots, \hat{\alpha}_{10})$  of  $\alpha$  that will be equal to  $\alpha$  if the super channel has caused no more than two symbol errors. Thus, the probability that the outer decoder in Fig. 7.21 causes a decoding error is not more than

$$P\{\alpha \neq \hat{\alpha}\} = \sum_{i=3}^{15} \binom{15}{i} \epsilon^i (1 - \epsilon)^{15-i} = 0.0032 \quad \epsilon = 0.0205$$

The overall rate of this concatenated coding system is  $(11/15)(4/7) = 0.42$ . Indeed, this concatenation can be interpreted as the system just being a  $(105, 44)$  binary linear code which has been factored into the super encoder and super decoder as shown in Fig. 7.21. It is, therefore, interesting to compare our concatenated system to an approximately comparable unfactored system which is an 11-error-correcting  $(127, 57)$  BCH code. The code rate 0.448 of a  $(127, 57)$  BCH code is slightly higher than that of a  $(105, 44)$  linear code, and the probability of a BCH decoder error

$$P = \sum_{i=12}^{127} \binom{127}{i} p^i (1 - p)^{127-i} = 0.00106$$

is slightly lower, but its decoding complexity is considerably greater. A performance comparison between a Reed-Solomon and Hamming  $(105, 44)$  concatenated coding system and a  $(127, 57)$  BCH code is shown in Fig. 7.22. In general, the moderate code rates in the range  $\frac{1}{4} \leq R \leq \frac{3}{4}$  are easily achieved by using short-block codes as inner codes.



**Figure 7.22** Probability of decoding error  $P$  against channel transition probability  $p$  pertaining to the RS and Hamming concatenated coding systems and BCH code.



The use of orthogonal or biorthogonal codes for inner codes offers excellent performance. But there is the disadvantage that the overall code rate is very low. A convolutional code can also be used as the inner code, and it achieves excellent performance with relatively high overall code rates. However, sequential decoding is preferred at very low error rates, while Viterbi decoding is preferred at high error rates.

## PROBLEMS

**7.1** Consider a  $t$ -error-correcting  $(n, k)$  BCH code of length  $n = 2^m - 1$  for  $m = 5$ .

- a Find all values of  $k$  and  $t$  for satisfying  $n - k = mt$ .
- b Repeat part a for  $m = 6$ .

**7.2** Let  $\alpha$  be a primitive element of the Galois field  $GF(2^6)$  with the primitive polynomial  $p(x) = 1 + x + x^6$ .

- a Is  $\alpha$  a root of  $p(x)$ ? If so, construct a table for  $GF(2^6)$  based on  $\alpha^6 = 1 + \alpha$ .
- b Find the minimal polynomials of  $\alpha$ ,  $\alpha^3$ ,  $\alpha^5$ , and  $\alpha^7$ .
- c What is the code length  $n$ ?
- d Find the generator polynomials for  $t = 2$  and  $3$ .
- e Determine the  $t$ -error-correcting  $(n, k)$  BCH code for  $t = 2$  and  $t = 3$ .
- f Determine the BCH bounds for the codes obtained in part e.

**7.3** Let  $\alpha$  be a root of the primitive polynomial  $p(x) = 1 + x^3 + x^7$ .

- a Construct a table for the Galois field  $GF(2^7)$ .
- b Find the conjugates of  $\alpha$ ,  $\alpha^3$ , and  $\alpha^5$ .
- c Determine the minimal polynomials of  $\alpha$ ,  $\alpha^3$ , and  $\alpha^5$ .
- d Find the generator polynomial for the triple-error-correcting BCH code of length  $n = 2^m - 1$ .
- e What is the designed distance for the code obtained in part d? Find the BCH bound for the minimum distance.

**7.4** Let  $\alpha$  be a primitive element of  $GF(2^4)$  with the primitive polynomial  $p(x) = 1 + x + x^4$ .

- a Construct a table for  $GF(2^4)$  that shows the exponential element, the polynomial form, and the vector representation.
- b Find the minimal polynomials of  $\alpha$ ,  $\alpha^3$ , and  $\alpha^5$ .
- c Determine the generator polynomial of the double-error-correcting BCH code of length  $n = 2^4 - 1 = 15$ .
- d Find the value of  $k$  and the minimum distance  $d_{\min}$ . Is the minimum distance  $d_{\min}$  equal to the designed distance  $d_0 = 2t + 1$ ?
- e Using Eq. (7.6), find the parity-check matrix  $\mathbf{H}$  of this  $(15, 7)$  BCH code.



**7.5** Consider the double-error-correcting (15, 7) BCH code. Suppose that the all-zero code word  $c(x) = 0$  is transmitted and  $r(x) = x^2 + x^7$  is received.

- Form the error pattern  $e(x)$ .
- Compute the syndrome from the remainders that result from dividing  $e(x)$  by the minimal polynomials.
- Devise the syndrome computation circuit for this code.

**7.6** Let  $\sigma(x)$  be the error-locator polynomial for the (15, 7) BCH code. Suppose that  $r(x) = x^6 + x^{13}$  is received when  $c(x) = 0$  is transmitted.

- Find  $\sigma(x)$  with the Peterson-Berlekamp algorithm.
- Find  $\sigma(x)$  with the Berlekamp-Massey algorithm.
- Determine the error-location numbers.

**7.7** Repeat Prob. 7.6 but find  $\sigma(x)$  for the error correction of  $t = 2$  or fewer random errors with the methods discussed in Sec. 7.4.

**7.8** Devise a Chien's searching circuit for the double-error-correcting (15, 7) BCH code.

**7.9** The (15, 8) code is the dual of the (15, 7) BCH code.

- Find the weight distribution of the (15, 8) dual code.
- Form the weight enumerator of this dual code.
- Find the weight enumerator of the (15, 7) BCH code.
- Determine the weight distribution of the (15, 7) BCH code.

**7.10** Suppose that  $\alpha$  is the primitive element and  $\beta$  is an arbitrary element from the same Galois field  $GF(2^4)$ .

- Design a circuit that is capable of multiplying a polynomial  $\beta = a_0 + a_1\alpha + a_2\alpha^2 + a_3\alpha^3$  by  $\alpha$ .
- Devise a circuit to multiply  $\beta$  by  $\alpha^3$ .

**7.11** Let  $r(x_4) = r_0 + r_1x + \cdots + r_4x^{14}$  be a polynomial over  $GF(2)$ . Devise a circuit for computing  $r(\alpha^4)$ , where  $\alpha^4$  is a field element in  $GF(2^4)$ .

**7.12** Consider the triple-error-correcting (127, 106) BCH code. The Galois field  $GF(2^7)$  is generated by  $p(x) = 1 + x^3 + x^7$ .

- Show that the degree of  $p(x)$  is  $m = 7$ .
- Prove that the (127, 106) BCH code is generated by

$$g(x) = 1 + x + x^5 + x^6 + x^7 + x^8 + x^{11} + x^{12} + x^{14} \\ + x^{15} + x^{17} + x^{18} + x^{21}$$

- Find the parity-check matrix for this code.
- Suppose that  $r(x) = x^{22} + x^{51} + x^{101}$  is received when  $c(x) = 0$  is transmitted. Show that the error-locator polynomial is

$$\sigma(x) = 1 + \alpha^{42}x + \alpha^2x^2 + \alpha^{47}x^3$$

- e Find the roots of  $\sigma(x)$  and determine the error-location numbers.
- f Devise an encoding circuit for this code.
- g Design a decoder of this code.

**7.13** Consider the triple-error-correcting (15, 9) RS code with symbols from  $GF(2^4)$ .

- a Find the generator polynomial of this code.
- b Suppose that  $\mathbf{r} = (0\ 0\ 0\ \alpha^5\ 0\ 0\ 0\ \alpha^2\ 0\ 0\ 0\ 0\ 0\ \alpha^4\ 0\ 0)$  is received when the all-zero code word was transmitted. Compute the syndrome components.
- c Find the error-locator polynomial  $\sigma(x)$ .
- d Find the error-location numbers of the error pattern  $e(x)$ .
- e Determine the error-evaluator polynomial  $\Omega(x)$ .
- f Find the error pattern by computing the error values.

## APPENDIX 7A

### Proof of Theorem 7.1

Since  $\sigma^{(n)}(x)$  is a solution polynomial for the first  $n$  power sums, Eq. (7.45) holds. That is,

$$\sum_{i=0}^{l_n} s_{j-i} \sigma_i^{(n)} = \begin{cases} 0 & l_n + 1 \leq j \leq n \\ d_n \neq 0 & j = n + 1 \end{cases} \quad (7A.1)$$

Similarly, since  $\sigma^{(m)}(x)$  is a solution polynomial for the first  $m$  power sums, there exists

$$\sum_{i=0}^{l_n} s_{j-i} \sigma_i^{(m)} = \begin{cases} 0 & l_m + 1 \leq j \leq m < n \\ d_m \neq 0 & j = m + 1 \end{cases} \quad (7A.2)$$

If 
$$\sigma^{(n+1)}(x) = \sigma^{(n)}(x) - d_n d_m^{-1} x^{n-m} \sigma^{(m)}(x) \quad (7A.3)$$

is to be a polynomial solution for the first  $n + 1$  power sums, then

$$\sum_{i=0}^{l_{n+1}} s_{j-i} \sigma_i^{(n+1)} = 0 \quad l_{n+1} + 1 \leq j \leq n + 1 \quad (7A.4)$$

must hold. Using Eq. (7.44), Eq. (7A.3) becomes

$$\begin{aligned} \sigma^{(n+1)}(x) = & (\sigma_0^{(n)} + \sigma_1^{(n)}x + \cdots + \sigma_{l_n}^{(n)}x^{l_n}) - d_n d_m^{-1} (\sigma_0^{(m)}x^{n-m} + \sigma_1^{(m)}x^{n-m+1} \\ & + \cdots + \sigma_{l_m}^{(m)}x^{n-m+l_m}) \end{aligned}$$

Thus, the sum Eq. (7A.4) has the following form, but is not yet identical:

$$\sum_{i=0}^{l_{n+1}} s_{j-i} (\sigma_i^{(n)} - d_n d_m^{-1} \sigma_{i-(n-m)}^{(m)}) \quad (7A.5)$$

Since  $\sigma_i^{(n)} = 0$  for  $i < 0$  and  $l_n < i$ , and since  $\sigma_i^{(m)} = 0$  for  $i < 0$  and  $l_m < i$ , the sum Eq. (7A.5) can be written

$$\sum_{i=0}^{l_n} s_{j-i} \sigma_i^{(n)} - d_n d_m^{-1} \sum_{i=n-m}^{l_m+n-m} s_{j-i} \sigma_{i-(n-m)}^{(m)} = 0 \quad (7A.6)$$

At this moment, we have two cases to consider.

*Case I:* For  $j = n + 1$ , the first sum in Eq. (7A.6) has the value  $d_n$  such that

$$\sum_{i=0}^{l_n} s_{j-i} \sigma_i^{(n)} = s_{n+1} + s_n \sigma_1^{(n)} + \cdots + s_{n+1-l_n} \sigma_{l_n}^{(n)} = d_n$$

and the second sum in Eq. (7A.6) has the value  $d_m$  such that

$$\begin{aligned} \sum_{i=n-m}^{l_m+n-m} s_{j-i} \sigma_{i-(n-m)}^{(m)} &= \sum_{i'=0}^{l_m} s_{j-i'-(n-m)} \sigma_{i'}^{(m)} = \sum_{i'=0}^{l_m} s_{m+1-i'} \sigma_{i'}^{(m)} \\ &= s_{m+1} + s_m \sigma_1^{(m)} + \cdots + s_{m+1-l_m} \sigma_{l_m}^{(m)} = d_m \end{aligned}$$

Thus, Eq. (7A.6) reduces to

$$d_n - d_n d_m^{-1} d_m = 0 \quad \text{for } j = n + 1$$

*Case II:* Referring to Eq. (7A.1), the first sum in Eq. (7A.6) becomes

$$\sum_{i=0}^{l_n} s_{j-i} \sigma_i^{(n)} = 0 \quad \text{provided } l_n + 1 \leq j \leq n$$

and from Eq. (7A.2), the second sum in Eq. (7A.6) is

$$\sum_{i=n-m}^{l_m+n-m} s_{j-i} \sigma_{i-(n-m)}^{(m)} = \sum_{i'=0}^{l_m} s_{j-i'-(n-m)} \sigma_{i'}^{(m)} = 0$$

if  $l_m + 1 \leq j - (n - m) \leq m$  or  $l_m + (n - m) + 1 \leq j \leq n$ . Thus Eq. (7A.6) is satisfied provided that

$$\max [l_n, l_m + n - m] + 1 \leq j \leq n + 1$$

Therefore, Eq. (7.45) is satisfied by  $\sigma^{(n+1)}(x)$ , the degree of which is formally given by

$$l_{n+1} = \max [l_n, l_m + n - m] \quad (7A.7)$$

However, we still have not proved that  $\sigma^{(n+1)}(x)$  is minimal, but the proof of Theorem 7.2 in Appendix 7B will show that  $\sigma^{(n+1)}(x)$  is a minimal solution at the  $(n + 1)$ th step of iteration.

## APPENDIX 7B

### Proof of Theorem 7.2

From Eq. (7.51), the first  $n$  power sums consist of the following  $n - l_n$  equations:

$$\begin{aligned} s_{l_n+1}\sigma_0^{(n)} + s_{l_n}\sigma_1^{(n)} + \cdots + s_1\sigma_{l_n}^{(n)} &= 0 \\ s_{l_n+2}\sigma_0^{(n)} + s_{l_n+1}\sigma_1^{(n)} + \cdots + s_2\sigma_{l_n}^{(n)} &= 0 \\ &\vdots \\ s_n\sigma_0^{(n)} + s_{n-1}\sigma_1^{(n)} + \cdots + s_{n-l_n}\sigma_{l_n}^{(n)} &= 0 \end{aligned} \quad (7B.1)$$

$$s_{n+1}\sigma_0^{(n)} + s_n\sigma_1^{(n)} + \cdots + s_{n-l_n+1}\sigma_{l_n}^{(n)} = d_n \neq 0 \quad (7B.2)$$

And from Eq. (7.52), the first  $n + 1$  power sums consist of the following  $n + 1 - l_{n+1}$  equations:

$$\begin{aligned} s_{l_{n+1}+1}\sigma_0^{(n+1)} + s_{l_{n+1}}\sigma_1^{(n+1)} + \cdots + s_1\sigma_{l_{n+1}}^{(n+1)} &= 0 \\ &\vdots \\ s_n\sigma_0^{(n+1)} + s_{n-1}\sigma_1^{(n+1)} + \cdots + s_{n-l_{n+1}}\sigma_{l_{n+1}}^{(n+1)} &= 0 \end{aligned} \quad (7B.3)$$

$$s_{n+1}\sigma_0^{(n+1)} + s_n\sigma_1^{(n+1)} + \cdots + s_{n+1-l_{n+1}}\sigma_{l_{n+1}}^{(n+1)} = 0 \quad (7B.4)$$

Since  $\sigma^{(n)}(x)$  is a minimal solution, it follows that  $l_{n+1} \geq l_n$ . Subtracting Eq. (7B.4) from Eq. (7B.2) gives

$$\begin{aligned} s_{n+1}(\sigma_0^{(n+1)} - \sigma_0^{(n)}) + s_n(\sigma_1^{(n+1)} - \sigma_1^{(n)}) + \cdots + s_{n+1-l_n}(\sigma_{l_n}^{(n+1)} - \sigma_{l_n}^{(n)}) \\ + \cdots + s_{n+1-l_{n+1}}(\sigma_{l_{n+1}}^{(n+1)} - \sigma_{l_{n+1}}^{(n)}) = -d_n \end{aligned} \quad (7B.5)$$

where  $\sigma_i^{(n)} = 0$  for  $l_n < i$ . Also, subtracting Eq. (7B.3) from Eq. (7B.1), yields

$$\begin{aligned} s_n(\sigma_0^{(n+1)} - \sigma_0^{(n)}) + s_{n-1}(\sigma_1^{(n+1)} - \sigma_1^{(n)}) + \cdots + s_{n-l_n}(\sigma_{l_n}^{(n+1)} - \sigma_{l_n}^{(n)}) \\ + \cdots + s_{n-l_{n+1}}(\sigma_{l_{n+1}}^{(n+1)} - \sigma_{l_{n+1}}^{(n)}) = 0 \end{aligned} \quad (7B.6)$$

where  $\sigma_i^{(n)} = 0$  for  $i > l_n$ .

Finally, for  $j = l_{n+1} + 1$  we have

$$s_{l_{n+1}+1}(\sigma_0^{(n+1)} - \sigma_0^{(n)}) + s_{l_{n+1}}(\sigma_1^{(n+1)} - \sigma_1^{(n)}) + \cdots + s_{l_{n+1}+1-l_n}(\sigma_{l_n}^{(n+1)} - \sigma_{l_n}^{(n)}) \\ + \cdots + s_1(\sigma_{l_{n+1}}^{(n+1)} - \sigma_{l_{n+1}}^{(n)}) = 0 \quad (7B.7)$$

Thus, Eqs. (7B.5), (7B.6), and (7B.7) are together expressible by the following single form:

$$\sum_{i=0}^{l_{n+1}} s_{j-i}(\sigma_i^{(n+1)} - \sigma_i^{(n)}) = \begin{cases} 0 & l_{n+1}+1 \leq j \leq n \\ -d_n & j = n+1 \end{cases} \quad (7B.8)$$

Referring to Eq. (7.53),  $\sigma_{n-m-1}^{(n+1)} - \sigma_{n-m-1}^{(n)} = 0$  because the coefficient of  $x^{n-m-1}$  is zero. Hence, it follows that

$$\sigma_i^{(n+1)} = \sigma_i^{(n)} \quad \text{for } i \leq n - m - 1$$

Thus, Eq. (7B.8) reduces to

$$\sum_{i=n-m}^{l_{n+1}} s_{j-i}(\sigma_i^{(n+1)} - \sigma_i^{(n)}) = \begin{cases} 0 & l_{n+1} + 1 \leq j \leq n \\ -d_n & j = n+1 \end{cases} \quad (7B.9)$$

Letting  $j' = j - (n - m)$ ,  $i' = i - (n - m)$ , and  $l_m = l_{n+1} - (n - m)$ , we see that Eq. (7B.9) becomes

$$\sum_{i'=0}^{l_m} s_{j'-i'}(\sigma_{i'+(n-m)}^{(n+1)} - \sigma_{i'+(n-m)}^{(n)}) = \begin{cases} 0 & l_m + 1 \leq j' \leq m \\ -d_n \neq 0 & j' = m+1 \end{cases} \quad (7B.10)$$

where  $l_m + 1 \leq j' \leq m$  is obtained from  $l_m + (n - m) + 1 \leq j' + (n - m) \leq n$ . Let the coefficients of  $\sigma^{(m)}(x) = \sigma_0^{(m)} + \sigma_1^{(m)}x + \cdots + \sigma_{l_m}^{(m)}x^{l_m}$  be  $\sigma_{i'}^{(m)}$  for  $0 \leq i' \leq l_m$ . If we put  $\sigma_{i'}^{(m)} = (\sigma_{i'+n-m}^{(n+1)} - \sigma_{i'+n-m}^{(n)}) a^{-1}$ , then Eq. (7B.10) becomes

$$\sum_{i'=0}^{l_m} s_{j'-i'}\sigma_{i'}^{(m)} = \begin{cases} 0 & l_m + 1 \leq j' \leq m \\ -d_n a^{-1} = d_m \neq 0 & j' = m+1 \end{cases} \quad (7B.11)$$

where  $a = -d_n d_m^{-1}$ .

By Eq. (7B.11),  $\sigma^{(m)}(x)$  is a solution polynomial to the first  $m - l_m$  equations and has the discrepancy  $d_m \neq 0$ . Thus, the degree of  $\sigma^{(m)}(x)$  is formally  $l_m = l_{n+1} - (n - m)$ .

## APPENDIX 7C

### Proof of Theorem 7.3

If  $d_n = 0$ , it implies that  $\sigma^{(n+1)}(x) = \sigma^{(n)}(x)$ . Since  $\sigma^{(n)}(x)$  is a minimal solution,  $\sigma^{(n+1)}(x)$  is also a minimal solution such that  $l_{n+1} = l_n$ . Next



consider the case where  $d_n \neq 0$ . Since  $\sigma^{(m)}(x)$  and  $\sigma^{(n)}(x)$  are known to the decoder, so is  $\sigma^{(n+1)}(x)$ , which is given by Eq. (7.56). By Theorem 7.2,  $\sigma^{(n+1)}(x)$  is a solution polynomial of degree  $l_{n+1} = \max[l_n, l_m + n - m]$ . It remains now to show that this is a minimal solution.

1. If  $m - l_m \geq n - l_n$ , then it follows that  $l_m + (n - m) \leq l_n$ . Since  $l_m + (n - m) = n - (m - l_m) \leq n - (n - l_n) = l_n$ , it follows that  $l_{n+1} = \max[l_n, l_m + n - m] = l_n$ , that is,  $l_{n+1} = l_n$ , which implies that  $\sigma^{(n+1)}(x)$  is a minimal solution at the step  $n + 1$ .
2. If  $m - l_m < n - l_n$ , then  $l_n < l_m + (n - m)$ . We then have  $l_{n+1} = \max[l_n, l_m + n - m] = l_m + n - m$ . Now, assume that there exists a polynomial  $Q^{(n+1)}(x)$  of degree  $q$  where  $l_n \leq q \leq l_m + (n - m)$ . There are two cases to be considered.
  - a. If  $q = l_n$ ,  $q$  is equivalent to  $l_{n+1}$ . Therefore, we have  $q = \max[l_n, l_{m'} + n - m']$  from which  $l_{m'} + n - m' \leq l_n$ , or  $n - l_n \leq m' - l_{m'}$ . Thus, by Theorem 7.2 there is a polynomial solution  $\sigma^{(m')}(x)$  with  $n - l_n \leq m' - l_{m'}$ . But the actual condition is  $m - l_m < n - l_n$ , and  $m - l_m$  was chosen to be the largest of the  $m' - l_{m'}$  values for the previous solutions. Thus, we can see that  $n - l_n \leq m' - l_{m'}$  contradicts  $m - l_m < n - l_n$ .
  - b. If  $l_n < q < l_m + n - m$ , by Theorem 7.2  $q = l_{m'} + n - m'$ . However, since  $m' - l_{m'} \leq m - l_m$ , then  $q < l_{n+1} = n - (m - l_m) \leq n - (m' - l_{m'}) = q$ , which is also a contradiction. Thus,  $\sigma^{(n+1)}(x)$  must be a minimal solution.

## APPENDIX 7D

Generator Polynomials of BCH Codes of Length up to  $2^9 - 1$ 

| $n$ | $k$ | $t$ | Generator polynomial (octal)                 |
|-----|-----|-----|----------------------------------------------|
| 7   | 4   | 1   | 13                                           |
| 15  | 11  | 1   | 23                                           |
| 15  | 7   | 2   | 721                                          |
| 15  | 5   | 3   | 2467                                         |
| 31  | 26  | 1   | 45                                           |
| 31  | 21  | 2   | 3551                                         |
| 31  | 16  | 3   | 107657                                       |
| 31  | 11  | 5   | 5423325                                      |
| 31  | 6   | 7   | 313365047                                    |
| 63  | 57  | 1   | 103                                          |
| 63  | 51  | 2   | 12471                                        |
| 63  | 45  | 3   | 1701317                                      |
| 63  | 39  | 4   | 166623567                                    |
| 63  | 36  | 5   | 1033500423                                   |
| 63  | 30  | 6   | 157464165547                                 |
| 63  | 24  | 7   | 17323260404441                               |
| 63  | 18  | 10  | 1363026512351725                             |
| 63  | 16  | 11  | 6331141367235453                             |
| 63  | 10  | 13  | 472622305527250155                           |
| 63  | 7   | 15  | 5231045543503271737                          |
| 127 | 120 | 1   | 211                                          |
| 127 | 113 | 2   | 41567                                        |
| 127 | 106 | 3   | 11554743                                     |
| 127 | 99  | 4   | 3447023271                                   |
| 127 | 92  | 5   | 624730022327                                 |
| 127 | 85  | 6   | 130704476322273                              |
| 127 | 78  | 7   | 26230002166130115                            |
| 127 | 71  | 9   | 6255010713253127753                          |
| 127 | 64  | 10  | 1206534025570773100045                       |
| 127 | 57  | 11  | 335265252505705053517721                     |
| 127 | 50  | 13  | 54446512523314012421501421                   |
| 127 | 43  | 14  | 17721772213651227521220574343                |
| 127 | 36  | 15  | 31460746665220750447645<br>74721735          |
| 127 | 29  | 21  | 403114461367670603667530141176155            |
| 127 | 22  | 23  | 1233760704047225224354456266376<br>47043     |
| 127 | 15  | 27  | 22057042445604554770523013762217<br>604353   |
| 127 | 8   | 31  | 7047264052751030651476224271567<br>733130217 |
| 255 | 247 | 1   | 435                                          |
| 255 | 239 | 2   | 267543                                       |
| 255 | 231 | 3   | 156720665                                    |

| $n$ | $k$ | $t$ | Generator polynomial (octal)                                                         |
|-----|-----|-----|--------------------------------------------------------------------------------------|
| 255 | 223 | 4   | 75626641375                                                                          |
| 255 | 215 | 5   | 23157564726421                                                                       |
| 255 | 207 | 6   | 16176560567636227                                                                    |
| 255 | 199 | 7   | 7633031270420722341                                                                  |
| 255 | 191 | 8   | 2663470176115333714567                                                               |
| 255 | 187 | 9   | 52755313540001322236351                                                              |
| 255 | 179 | 10  | 22624710717340432416300455                                                           |
| 255 | 171 | 11  | 15416214212342356077061630637                                                        |
| 255 | 163 | 12  | 7500415510075602551574724514601                                                      |
| 255 | 155 | 13  | 3757513005407665015722506464677<br>633                                               |
| 255 | 147 | 14  | 1642130173537165525304165305441<br>011711                                            |
| 255 | 139 | 15  | 4614017320601755615707227302474<br>53567445                                          |
| 255 | 131 | 18  | 2157133314715101512612502774421<br>42024165471                                       |
| 255 | 123 | 19  | 1206140522420660037172103265161<br>41226272506267                                    |
| 255 | 115 | 21  | 6052666557210024726363640460027<br>6352556313472737                                  |
| 255 | 107 | 22  | 2220577232206625631241730023534<br>7420176574750154441                               |
| 255 | 99  | 23  | 1065666725347317422274141620157<br>4332252411076432303431                            |
| 255 | 91  | 25  | 6750265030327444172723631724732<br>511075550762720724344561                          |
| 255 | 87  | 26  | 1101367634147432364352316343071<br>72046206722545273311721317                        |
| 255 | 79  | 27  | 6670003563765750002027034420736<br>6174621015326711766541342355                      |
| 255 | 71  | 29  | 2402471052064432151555417211233<br>1163205444250362557643221706035                   |
| 255 | 63  | 30  | 1075447505516354432531521735770<br>700366611172645526761365670254<br>3301            |
| 255 | 55  | 31  | 7315425203501100133015275306032<br>0543254143267550105570444260354<br>73617          |
| 255 | 47  | 42  | 2533542017062646563033041377406<br>2331751233341454460450050660245<br>52543173       |
| 255 | 45  | 43  | 1520205605523416113110134637642<br>3701563670024470762373033202157<br>025051541      |
| 255 | 37  | 45  | 5136330255067007414177447245437<br>5304207357061743234323476443547<br>37403044003    |
| 255 | 29  | 47  | 3025715536673071465527064012361<br>3771153422423242011741140602546<br>57410403565037 |

| $n$ | $k$ | $t$ | Generator polynomial (octal)                                                                |
|-----|-----|-----|---------------------------------------------------------------------------------------------|
| 255 | 21  | 55  | 1256215257060332656001773153607<br>6121032273414056530745425211531<br>21614466513473725     |
| 255 | 13  | 59  | 4641732005052564544426573714250<br>0660043306774454765614031746772<br>1357026134460500547   |
| 255 | 9   | 63  | 1572602521747246320103104325535<br>5134614162367212044074545112766<br>115547705561677516057 |
| 511 | 502 | 1   | 1021                                                                                        |
| 511 | 493 | 2   | 1112711                                                                                     |
| 511 | 484 | 3   | 1530225571                                                                                  |
| 511 | 475 | 4   | 1630256304641                                                                               |
| 511 | 466 | 5   | 1112724662161763                                                                            |
| 511 | 457 | 6   | 1142677410335765707                                                                         |
| 511 | 448 | 7   | 1034122337164372224005                                                                      |
| 511 | 439 | 8   | 1561350064670543777423345                                                                   |
| 511 | 430 | 9   | 1727400306127620173461431627                                                                |
| 511 | 421 | 10  | 1317711625267264610360644707513                                                             |
| 511 | 412 | 11  | 1337530164410305712316173767147<br>101                                                      |
| 511 | 403 | 12  | 1573436303657311762726657724651<br>203651                                                   |
| 511 | 394 | 13  | 110251034413033354270407474305<br>341234033                                                 |
| 511 | 385 | 14  | 177554602577712372455452107300<br>530331444031                                              |
| 511 | 376 | 15  | 1116744706521725532227162607146<br>216210106733203                                          |
| 511 | 367 | 17  | 1126657202505666323017001652245<br>562614435511600655                                       |
| 511 | 358 | 18  | 1574456154545450414733416176535<br>156070037760411373255                                    |
| 511 | 349 | 19  | 1455012234675753242074501555657<br>377006165521557376050313                                 |
| 511 | 340 | 20  | 1036075427062474623220662047122<br>611677363511364110105517777                              |
| 511 | 331 | 21  | 1540644721106050570342772405117<br>177453215136663677461641457321                           |
| 511 | 322 | 22  | 1153614675506111211374366675624<br>670755236523645062677061770735<br>073                    |
| 511 | 313 | 23  | 1037043005346411102774516447047<br>0707356023272246374215367362515<br>17437                 |
| 511 | 304 | 25  | 1510736212272133353502352536320<br>7034101472252730643370771600352<br>54047351              |
| 511 | 295 | 26  | 1112630477530033170044524747727<br>7675327520466126030774720522476<br>71744467035           |

| $n$ | $k$ | $t$ | Generator polynomial (octal)                                                                                                     |
|-----|-----|-----|----------------------------------------------------------------------------------------------------------------------------------|
| 511 | 286 | 27  | 1317463403265645042064775326044<br>7757374167140717560167145236500<br>22734505401471                                             |
| 511 | 277 | 28  | 1111170752122547100341422773660<br>3022562303177512454137173036077<br>37426401526326045                                          |
| 511 | 268 | 29  | 1241160247151367165615372023170<br>221264427226437653 163043503436<br>310631425301735205601                                      |
| 511 | 259 | 30  | 1121314111162101532370722243711<br>0144633334772560250516566143547<br>13760662350433214646117                                    |
| 511 | 250 | 31  | 1007276607431444342624513277572<br>7522374652773105376325757760735<br>235335336157163637347245                                   |
| 511 | 241 | 36  | 1142630652622415305026460545443<br>2304067012432776321666056566417<br>10317303600602611251467341721                              |
| 511 | 238 | 37  | 1266115765104357454721732420217<br>7555474111377076177455240605755<br>033452532207221321266342055373                             |
| 511 | 229 | 38  | 1441474125377630723466544074104<br>3635272625110602507742556745735<br>3756537372452015247217677400031<br>05                      |
| 511 | 220 | 39  | 1330350650041246137404371541263<br>7635707201431575333227603262602<br>7520300774064360307604546443426<br>65055                   |
| 511 | 211 | 41  | 1537262671145756636471271211466<br>0230574345664477051253665031617<br>1240071400321447134000077557120<br>30376003                |
| 511 | 202 | 42  | 1355074061631664750650654005233<br>2241263735501617417747326444652<br>0771004707676072273354041742531<br>26577344477             |
| 511 | 193 | 43  | 1151502313317462115717012400332<br>0541507162357615552060114767351<br>2045732572600454734256756736735<br>05075470425055          |
| 511 | 184 | 45  | 1320551776766374340772233460740<br>7275505522041306675275253552627<br>1014443703451367651642536441722<br>65100216633505671       |
| 511 | 175 | 46  | 1202260564674312433743724246412<br>3131006733703766567307723446714<br>6116166500624231372142007622705<br>77576135715734456503    |
| 511 | 166 | 47  | 1105764456740764631742144442303<br>7231346624415551173770646017645<br>0445776705220527361070224470173<br>56700151227221443336113 |



| $n$ | $k$ | $t$ | Generator polynomial (octal)                                                                                                                                          |
|-----|-----|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 511 | 157 | 51  | 17101711711400604264434731140711<br>5213707536440012114236665475350<br>7011024155156243101741614000045<br>17373603747715514145565503                                  |
| 511 | 148 | 53  | 1414131546120371255541603030602<br>6011246427727350174267055345645<br>5606707075471070705355774504056<br>62075046364112446577412302461                                |
| 511 | 139 | 54  | 1775336143053070047771371572323<br>0761476550747705134625235605330<br>7651341430002336341267612204553<br>5364751255470074263427064055176<br>3                         |
| 511 | 130 | 55  | 1126210420631670733262507445577<br>0252664000744756745104751542455<br>5465415664131357140502035376242<br>3212303052053173456612477213206<br>321                       |
| 511 | 121 | 58  | 1541271357655772525604106044116<br>5521701354434750270764625323125<br>3243276777423251212771735535703<br>7343174053743117567643575017423<br>3577257                   |
| 511 | 112 | 59  | 1264236440177126545461720120275<br>7566143212522620050735005241425<br>7454504160572312612131152145444<br>6351075657542421261177533722452<br>2460653365                |
| 511 | 103 | 61  | 1267502203437327636237202040115<br>0506771057315147230644142256367<br>3334061674753504325307201333612<br>3603406751622266602654733076535<br>7307372010437             |
| 511 | 94  | 62  | 1034624554522076136543376246276<br>5014127513662235012007006252201<br>6513051706647407075134565753153<br>5730115603353544742113753260263<br>6772420754543335          |
| 511 | 85  | 63  | 1311423305612705436042312400305<br>3553606135536677204454060166477<br>6524234770321371403157621106144<br>2326015202070001615020030224474<br>4512244665460344315       |
| 511 | 76  | 85  | 1266305771250774445612456156722<br>7606073244460173371073371716611<br>5440360443775323546736213462531<br>1060452035156371713426212525270<br>5066142632374105237105    |
| 511 | 67  | 87  | 1572123711765346653773251761251<br>6350024126410254565007065546756<br>0150435723020575734010547010044<br>5773034567736504540164416765264<br>5603000655246474332003041 |

| $n$ | $k$ | $t$ | Generator polynomial (octal)                                                                                                                                                                 |
|-----|-----|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 511 | 58  | 91  | 1057521760004167050454306543214<br>4446762371110323251120106217732<br>4460255156456633005122223411465<br>4542017032114275132464456243035<br>6677543412533230060230330601                     |
| 511 | 49  | 93  | 1400373125226245616434770021451<br>4347747150430652515175422234333<br>7653446262505146642652723355123<br>6436336157163433456557233735677<br>3023742177503076702013524172673                  |
| 511 | 40  | 95  | 1065643260601413014144161647626<br>3131666646125563104544564702042<br>2675726040023677222071247202762<br>2367026402375544751636502305426<br>1717244400242403714177455740276<br>623           |
| 511 | 31  | 109 | 1004032013504641301465031022773<br>7333772312030141207564005615007<br>2774563771162315116355123767005<br>7451236201623017225001737551261<br>3477752065655234212557646767220<br>145745        |
| 511 | 28  | 111 | 1506424217016105771311126531671<br>3155370066227135624440407472104<br>0675344372405062040406060362307<br>5530602521050711015101113460677<br>7107742251416201736565601262012<br>1335531       |
| 511 | 19  | 119 | 1323204574570661164212403733264<br>1140234046117057401637365534154<br>1431377444730207044764741324776<br>5407662453355110160322611407052<br>4201045221103033616173547503631<br>4555046713    |
| 511 | 10  | 127 | 1702172311027431733521206643077<br>0426053277252024574422247642034<br>1454507135002647263477463246601<br>1333100645365661150236253067473<br>6776734670522343552711406072215<br>3373142701037 |

# Majority-Logic Decodable Codes

In this chapter, majority-logic decoding is presented for a subclass of cyclic codes that is much smaller than the class of BCH codes. The attractiveness of majority-logic decoding lies in its simple implementation. These codes, which are cyclic random-error-correcting, can be decoded rather simply in most cases. However, they seem to be somewhat less powerful than the BCH codes in terms of error-correcting ability for most interesting values of code length and efficiency. Hence majority-logic decoding is of minor importance in practice. Nevertheless, these two types of codes are often competitive in many practical situations. Most known codes that can be decoded with majority logic are cyclic codes or extended cyclic codes. For these codes, the implementation of majority-logic decoders is similar to that of Meggitt decoders and characterized by a simple majority gate for examining the syndrome. The first majority-logic decoding algorithm was devised by Reed in 1954 for a class of multiple-error-correcting codes discovered by Muller. But the first unified treatment of majority-logic decoding was done by Massey.

## 8.1 Orthogonal Parity-Check Sums

Before describing the error-control scheme of majority-logic decodable codes extensively, we introduce the concept of a set of orthogonal parity-check sums.

Recall that any linear  $(n, k)$  cyclic code over  $GF(2)$  has a parity-check matrix  $\mathbf{H}$ , and the code words satisfy  $\mathbf{c} \cdot \mathbf{H}^T = \mathbf{0}$ . Let  $C$  be an  $(n, k)$  cyclic code with parity-check matrix  $\mathbf{H}$ . The row space of  $\mathbf{H}$  is an  $(n, n - k)$  dual code of  $C$ , denoted by  $C_D$ . For any code vector  $\mathbf{c} = (c_0, c_1, \dots, c_{n-2}, c_{n-1})$  in  $C$  and any row-space vector  $\mathbf{h} = (h_0, h_1, \dots, h_{n-2}, h_{n-1})$  in  $C_D$ , the inner product of  $\mathbf{c}$  and  $\mathbf{h}$  must be zero, that is,

$$\mathbf{h} \cdot \mathbf{c} = h_0 c_0 + h_1 c_1 + \cdots + h_{n-2} c_{n-2} + h_{n-1} c_{n-1} = 0 \quad (8.1)$$

This is called the parity-check equation. Of course, we can also use any linear combination of row spaces of  $\mathbf{H}$  to form another parity-check equation. A total of  $2^{n-k}$  parity-check equations can be formed in this way. Consider an  $n$ -tuple vector  $\mathbf{c}'$  which is not a code word. Some modulo-2 sums of the following form may not be zero because this vector is not a code word:

$$\mathbf{h} \cdot \mathbf{c}' = h_0 c'_0 + h_1 c'_1 + \cdots + h_{n-1} c'_{n-1} \neq 0 \quad (8.2)$$

In general, this type of sum is referred to as a parity-check sum or simply a check sum. Since  $h_i$ ,  $0 \leq i \leq n-1$ , is either 1 or 0 over  $GF(2)$ , such a sum is simply the modulo-2 sum of the digits of the  $n$ -tuple vector  $\mathbf{c}'$  in positions where 1s in  $\mathbf{h}$  correspond.

Suppose that  $\mathbf{r} = (r_0, r_1, \dots, r_{n-1})$  is the received word when  $\mathbf{c} = (c_0, c_1, \dots, c_{n-1})$  was transmitted. If  $\mathbf{r} \neq \mathbf{c}$ ,  $\mathbf{r} = \mathbf{c} + \mathbf{e}$  because  $\mathbf{r}$  contains the error pattern  $\mathbf{e} = (e_0, e_1, \dots, e_{n-1})$  in it. Since from Eq. (8.1)  $\mathbf{h} \cdot \mathbf{c} = 0$ , it follows that  $\mathbf{h} \cdot \mathbf{r} = \mathbf{h} \cdot (\mathbf{c} + \mathbf{e}) = \mathbf{h} \cdot \mathbf{e}$ . Therefore, the parity-check sum  $\xi$  is defined and representable as follows:

$$\xi = \mathbf{h} \cdot \mathbf{e} = h_0 e_0 + h_1 e_1 + \cdots + h_{n-1} e_{n-1} \quad (8.3)$$

An error digit  $e_{n-1}$  may be checked by the check sum  $\xi$  if the coefficient  $h_{n-1} = 1$ . A set of parity-check sums is said to be orthogonal on a particular digit position if that digit is involved in every check sum in the set  $\{\xi_1, \xi_2, \dots, \xi_J\}$  and no other digit is involved in more than one check sum. Therefore, if the set of check sums is orthogonal on the digit, say,  $e_{n-1}$ , it is possible to construct a set of  $J$  parity-check sums orthogonal on  $e_{n-1}$  in the error pattern  $\mathbf{e}$ . However, such a set can be constructed on every error digit in the error pattern as a result of the cyclic symmetry of the code.

Consider the case where  $\lfloor J/2 \rfloor$  or fewer digits of  $\mathbf{e}$  are 1s. If  $e_{n-1} = 1$ , the other nonzero error bits can be distributed among at most  $\lfloor J/2 \rfloor - 1$  check sums orthogonal on  $e_{n-1}$ . Hence, the majority vote  $J - \lfloor J/2 \rfloor + 1$  of the check sums orthogonal on  $e_{n-1}$  is equal to  $e_{n-1} = 1$ . Thus, the error bit  $e_{n-1}$  is decoded as 1 if a majority vote of the parity-check sums orthogonal on  $e_{n-1}$  is 1; otherwise,  $e_{n-1}$  will be decoded as 0. Because of this, the parity-check sums orthogonal on  $e_{n-1}$  can be used to decode the received bit  $r_{n-1}$ .

**Theorem 8.1** If it is possible to form a set of  $J = d_{\min} - 1$  parity-check sums orthogonal on any digit in a cyclic code word, the code has a minimum distance  $d_{\min}$  of at least  $J + 1$ .

**Proof** If it is possible to construct a set of  $d_{\min} - 1$  parity-check sums orthogonal on a particular digit in a cyclic code word, then such a set of check sums  $\{\xi_1, \xi_2,$



$\dots, \xi_{d_{\min} - 1}$  can be constructed on every digit in a code word because of the cyclic symmetry of the code. In order to show that every nonzero code word has a Hamming weight of at least  $d_{\min}$ , consider a set of check sums orthogonal on one of the nonzero digits in a code word. All these check sums  $\xi_i$ ,  $i = 1, 2, \dots, d_{\min} - 1$ , orthogonal on that nonzero digit are 0 because each sum  $\xi_i$  has a nonzero entry in addition to (at least) one other nonzero digit in each check sum.

Consider, on the other hand, an  $n$ -tuple vector of Hamming weight  $d_{\min} - 1$  or less and the set of check sums on one of the nonzero digits. There are  $d_{\min} - 1$  check sums and at most  $d_{\min} - 2$  1s in these check sums in addition to the 1 column of every sum in the set. Hence, at least one check sum will be nonzero. It thus implies that a  $n$ -tuple vector of weight  $d_{\min} - 1$  or less cannot be a code word. Therefore, the code must have a minimum distance of at least  $d_{\min} = J + 1$ , which is sometimes called the majority-decodable distance.

**Example 8.1** Consider the (15, 7) cyclic code with the generator polynomial  $g(x) = 1 + x^4 + x^6 + x^7 + x^8$ . Since this code has the minimum distance  $d_{\min} = 5$  and is completely orthogonalized on a digit  $c_{12}$  in one step, the number of check sums is  $J = d_{\min} - 1 = 4$ . Therefore, the four parity-check sums orthogonal on  $c_{12}$  can be constructed as shown below:

$$\begin{aligned}
 \xi_1 &= c_1 && + c_9 + c_{10} && + c_{12} \\
 \xi_2 &= c_3 && && + c_{11} + c_{12} && + c_{14} \\
 \xi_3 &= c_0 && + c_4 && && + c_{12} + c_{13} \\
 \xi_4 &= && c_5 + c_6 + c_8 && && + c_{12}
 \end{aligned} \tag{8.4}$$

We see that  $c_{12}$  is checked by all four check sums and no other digit is checked by more than one check sum. For the information digit  $d(x) = x^4$ , the corresponding code polynomial is  $c(x) = x^4 g(x) = x^4 + x^8 + x^{10} + x^{11} + x^{12}$ , or the code word  $\mathbf{c} = (0\ 0\ 0\ 0\ 1\ 0\ 0\ 0\ 1\ 0\ 1\ 1\ 1\ 0\ 0)$ , which has a weight of 5. From Eq. (8.4), we see that all parity-check sums are 0s, i.e.,  $\xi_1 = 0$ ,  $\xi_2 = 0$ ,  $\xi_3 = 0$ , and  $\xi_4 = 0$ , because  $c_4 = c_8 = c_{10} = c_{11} = c_{12} = 1$  and the other digits are all 0s. Consider now a 15-tuple word of weight  $d_{\min} - 1 = 4$ , say,  $\mathbf{c} = (0\ 0\ 0\ 0\ 1\ 0\ 0\ 0\ 1\ 0\ 0\ 1\ 1\ 0\ 0)$ . Since the nonzero digits are  $c_4 = c_8 = c_{11} = c_{12} = 1$ , these four check sums orthogonal on the digit  $c_{12}$  are not all 0s. Thus, we have  $\xi_1 = 1$ ,  $\xi_2 = 0$ ,  $\xi_3 = 0$ , and  $\xi_4 = 0$ . In fact, one sum  $\xi_1$  is not 0. Hence, a 15-tuple word of weight 4,  $\mathbf{c} = (0\ 0\ 0\ 0\ 1\ 0\ 0\ 0\ 1\ 0\ 0\ 1\ 1\ 0\ 0)$  is not a code vector as expected.

## 8.2 One-Step Majority-Logic Decoding

As stated in the previous section, an estimate of an error digit  $e_{n-1}$  and the decoding of the received error digit  $r_{n-1}$  can be guaranteed if it is possible to form  $J$  parity-check sums orthogonal on  $e_{n-1}$  and there exist  $\lfloor J/2 \rfloor$  or fewer errors in the error pattern  $\mathbf{e}$ . The decoding algorithm described above is called one-step majority-logic decoding. In other words, if  $J$  is the maximum number of parity-check sums orthogonal on  $e_{n-1}$ , any error pattern of  $\lfloor J/2 \rfloor$  or fewer errors can be corrected by one-step majority-logic decoding. Thus, a code with minimum distance  $d_{\min}$  is said to be completely orthogonalizable in one



step if it is possible to construct  $J = d_{\min} - 1$  check sums orthogonal on every error digit.

The parity-check sums orthogonal on an error digit can be formed from the syndrome digits. The syndrome  $\mathbf{s}$  associated with the  $n$ -tuple  $\mathbf{r}$  is just the following  $(n - k)$ -tuple:

$$\mathbf{s} = (s_0, s_1, \dots, s_{n-k-1}) = \mathbf{e} \cdot \mathbf{H}^T \quad (8.5)$$

because  $\mathbf{c} \cdot \mathbf{H}^T = \mathbf{0}$ . Then, the  $i$ th syndrome digit corresponds to  $s_i = \mathbf{e} \cdot \mathbf{h}_i$ ,  $0 \leq i \leq n - k - 1$ .

Recall that the parity-check matrix  $\mathbf{H}$  for an  $(n, k)$  cyclic code has the following systematic form:

$$\mathbf{H} = [\mathbf{I}_{n-k} : \mathbf{P}_{(n-k) \times k}]$$

$$= \begin{bmatrix} \mathbf{h}_0 \\ \mathbf{h}_1 \\ \vdots \\ \mathbf{h}_{n-k-1} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & \cdots & 0 & h_{0,0} & h_{0,1} & \cdots & h_{0,k-1} \\ 0 & 1 & 0 & \cdots & 0 & h_{1,0} & h_{1,1} & \cdots & h_{1,k-1} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & 1 & h_{n-k-1,0} & h_{n-k-1,1} & \cdots & h_{n-k-1,k-1} \end{bmatrix}$$

From Eq. (8.5), we could form the syndrome vector

$$\mathbf{s} = (e_0, e_1, \dots, e_{n-1}) \cdot \begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ 0 & 1 & 0 & & 0 \\ 0 & 0 & 1 & & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \\ h_{0,0} & h_{1,0} & h_{2,0} & \cdots & h_{n-k-1,0} \\ h_{0,1} & h_{1,1} & h_{2,1} & & h_{n-k-1,1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ h_{0,k-1} & h_{1,k-1} & h_{2,k-1} & & h_{n-k-1,k-1} \end{bmatrix}$$

from which the syndrome digits corresponding to  $\mathbf{e}$  are obtained as the following set of equations:

$$\begin{aligned} s_0 &= e_0 & + & h_{0,0}e_{n-k} & + & h_{0,1}e_{n-k+1} & + \cdots & + & h_{0,k-1}e_{n-1} \\ s_1 &= e_1 & + & h_{1,0}e_{n-k} & + & h_{1,1}e_{n-k+1} & + \cdots & + & h_{1,k-1}e_{n-1} \\ &\vdots & & & & & & & \\ s_{n-k-1} &= & e_{n-k-1} & + & h_{n-k-1,0}e_{n-k} & + & h_{n-k-1,1}e_{n-k+1} & + \cdots & + & h_{n-k-1,k-1}e_{n-1} \end{aligned} \quad (8.6)$$

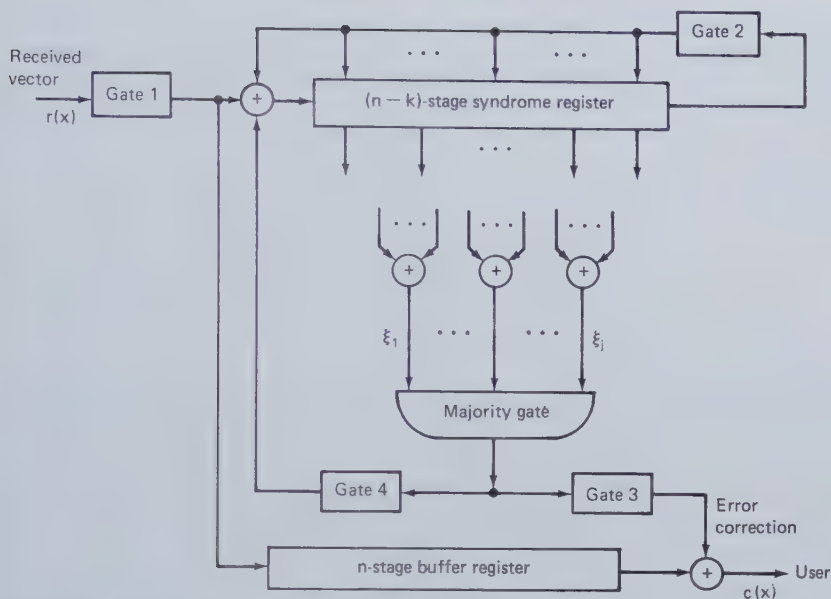


Figure 8.1 General type I one-step majority-logic decoder.

Since  $h_{i,j}$  in Eq. (8.6) are either 0 or 1, we see that each syndrome digit is a modulo-2 sum of certain error digits. Consider now the parity-check sum defined as

$$\xi_j = a_{j,0}s_0 + a_{j,1}s_1 + \cdots + a_{j,n-k-1}s_{n-k-1} \quad (8.7)$$

This check sum is simply a linear combination of the syndrome digits with the coefficients  $a_{j,i}$ , which are either 0 or 1. We obtain an implementation of the type I decoder, which is based on Eq. (8.7) for one-step majority-logic decoding as shown in Fig. 8.1. In order to implement a general type I decoder for an  $(n, k)$  cyclic code, the reader must first understand the following theorem.

**Theorem 8.2** Let  $s(x)$  be the syndrome of the received polynomial  $r(x)$ . Then, the syndrome corresponding to  $xr(x) + x^n$  is  $s^{(1)}(x) + 1$ .

**Proof** Consider the event that the received vector in the buffer register and the syndrome generator are simultaneously cyclically shifted once. Then we have

$$xr(x) = r^{(1)}(x) = r_{n-1} + r_0x + r_1x^2 + \cdots + r_{n-2}x^{n-1}$$

for the polynomial modulo  $x^n + 1$ , and the syndrome contents form the shifted syndrome  $s^{(1)}(x)$  of  $r^{(1)}(x)$ . Therefore, it follows that

$$xr(x) = q(x)g(x) + s^{(1)}(x) \quad (8.8)$$

Since  $g(x)$  is a factor of  $x^n + 1$ , we have

$$x^n + 1 = h(x)g(x)$$

or

$$x(x^{n-1}) = h(x)g(x) + 1 \quad (8.9)$$

where 1 is the remainder (syndrome) that results from dividing  $x^n$  by  $g(x)$ . By adding the terms of both sides of Eqs. (8.8) and (8.9), we get

$$x[r(x) + x^{n-1}] = [q(x) + h(x)]g(x) + [s^{(1)}(x) + 1] \quad (8.10)$$

Thus, we see that  $s^{(1)}(x) + 1$  is the remainder (syndrome) that results from dividing  $xr(x) + x^n$  by  $g(x)$ . This completes the proof.

Theoretically, the first term  $xr(x)$  of  $xr(x) + x^n$  in Eq. (8.10) designates  $r^{(1)}(x)$  cyclically shifted in the buffer register one place to the right. Hence, the first received digit  $r_{n-1}$  is read out of the buffer register. The  $J$  parity-check sums orthogonal on  $e_{n-1}$  are formed by taking the proper sum of the syndrome digits. These  $J$  check sums are fed into the majority gate. If the syndrome  $s(x)$  of  $r(x)$  corresponds to a correctable error pattern  $e(x) = e_0 + e_1x + \cdots + e_{n-1}x^{n-1}$  with an error at the rightmost position  $e_{n-1} = 1$ , the first received digit  $r_{n-1}$  is erroneous and must be corrected by taking the sum  $r_{n-1} \oplus e_{n-1}$ . Thus, the interpretation of  $s^{(1)}(x) + 1$  is that the shifted syndrome contents corresponding to  $r^{(1)}(x) = xr(x)$  have been formed in the syndrome register; at the same time a 1 is added to the left end of the syndrome register, which removes the effect on the syndrome of the previous error digit. Thus, the new contents in the syndrome register will be the altered received digit obtained from one shift to the right. As a result, we see that the output of a majority-logic gate,  $e_{n-1} = 1$ , not only corrects an erroneous digit from the buffer register but also resets the syndrome.

Let  $t = \lfloor J/2 \rfloor$  be the error-correcting capability for majority-logic decoding. Here  $J$  is the number of parity-check sums. In one-step majority-logic decoding, it is possible to correct any error pattern  $e(x)$  with  $\lfloor J/2 \rfloor$  or fewer errors if and only if  $J$  is the maximum number of check sums orthogonal on any error digit in  $e(x)$ . Let  $d_{\min}$  designate the minimum distance of the majority-logic decodable code. By coupling the error-correcting capability  $t = \lfloor (d_{\min} - 1)/2 \rfloor$  with  $t = \lfloor J/2 \rfloor$ , we have  $J \doteq d_{\min} - 1$  where the symbol  $\doteq$  denotes "equal to or close to." Thus, a majority-logic decodable code with  $d_{\min}$  is said to be completely orthogonalizable in one step if we can form  $J = d_{\min} - 1$  check sums orthogonal on an error digit in  $e(x)$ .

**Example 8.2** Consider the (7, 3) cyclic code, dual to the (7, 4) Hamming code, which is generated by the polynomial  $g(x) = 1 + x^2 + x^3 + x^4$ . This code, known as a simplex code, has minimum distance 4 and is capable of correcting a single

error ( $t = 1$ ). This cyclic code is categorically classified as either the maximum-length code or the difference-set code. One way to get a parity-check matrix  $\mathbf{H}$  for this code is to form the generator matrix  $\mathbf{G}$  as shown below. There are three code words for three messages of the form  $d_i(x) = x^i$  for  $i = 0, 1, 2$ . Using the formula  $x^{4+i} = q_i g(x) + \gamma_i(x)$  for  $i = 0, 1, 2$ , we can form these code words as follows:

| $i$ | $\gamma_i + x^{4+i}$  | Code word     |
|-----|-----------------------|---------------|
| 0   | $1 + x^2 + x^3 + x^4$ | 1 0 1 1 1 0 0 |
| 1   | $1 + x + x^2 + x^5$   | 1 1 1 0 0 1 0 |
| 2   | $x + x^2 + x^3 + x^6$ | 0 1 1 1 0 0 1 |

Thus, the generator matrix  $\mathbf{G}$  is obtained by using these three code words for its rows:

$$\mathbf{G} = \begin{bmatrix} 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 1 \end{bmatrix}$$

The parity-check matrix  $\mathbf{H}$  is easily obtained as follows:

$$\mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 \end{bmatrix}$$

Let  $\mathbf{e} = (e_0, e_1, \dots, e_6)$  be an error vector. The syndrome corresponding to  $\mathbf{e}$  is  $\mathbf{s} = (s_0, s_1, s_2, s_3) = \mathbf{e} \cdot \mathbf{H}^T$ , from which we obtain the following four syndrome digits:

$$\begin{aligned} s_0 &= e_0 && + e_4 + e_5 \\ s_1 &= e_1 && + e_5 + e_6 \\ s_2 &= e_2 && + e_4 + e_5 + e_6 \\ s_3 &= e_3 + e_4 && + e_6 \end{aligned}$$

Therefore, three parity-check sums orthogonal on  $e_6$  can be formed from the four syndrome digits:

$$\begin{aligned} \xi_1 &= s_0 + s_2 = e_0 && + e_2 && + e_6 \\ \xi_2 &= s_1 &= e_1 && + e_5 + e_6 \\ \xi_3 &= s_3 &= && + e_3 + e_4 && + e_6 \end{aligned} \quad (8.11)$$

Thus, each check sum  $\xi_k$ ,  $k = 1, 2, 3$ , is simply a sum of certain error digits. An error digit  $e_6$  is checked by all the check sums of Eq. (8.11), and no other error

digit is checked by more than one check sum. These three check sums are fed into the majority-logic gate. Therefore, the check sums orthogonal on  $e_6$  will be used to decode  $e_6$  using a majority-decision scheme. If, for the sake of simplicity, we assume that the all-zero code word  $\mathbf{c} = (0\ 0\ 0\ 0\ 0\ 0)$  is transmitted and  $\mathbf{r} = (0\ 0\ 0\ 0\ 0\ 1)$  is received, then clearly there is one error at the location  $x^6$ . The type I one-step majority-logic decoder for the  $(7, 3)$  simplex code is shown in Fig. 8.2.

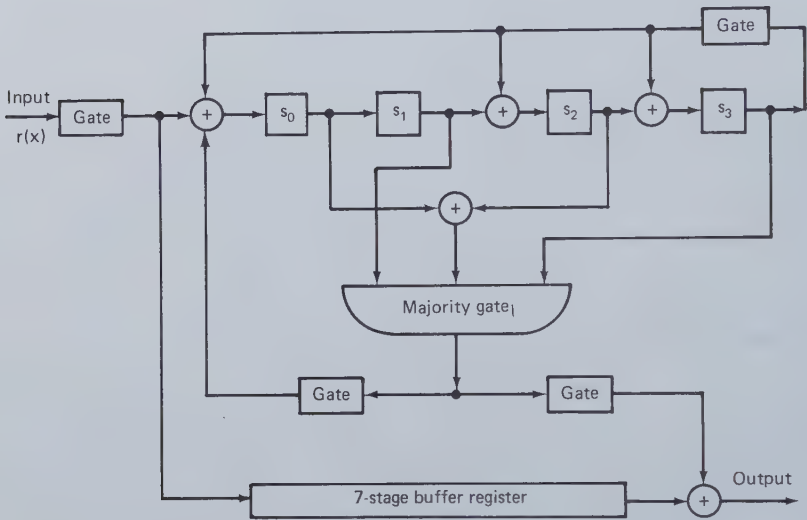


Figure 8.2 The type I one-step majority-logic decoder for the  $(7, 3)$  simplex code.

The error-correction procedure is described as follows. After the received word  $r(x)$  has entered the syndrome register, the syndrome contents become  $\mathbf{s} = (0\ 1\ 1\ 1)$ . Thus, the three check sums orthogonal on  $e_6$  will be:  $\xi_1 = 1$ ,  $\xi_2 = 1$ ,  $\xi_3 = 1$ . Since the majority vote of these check sums is 1, the output of the majority-logic gate is 1, which is the value of  $e_6$ . Next, the buffer and syndrome registers are shifted once; then the error digit  $e_6 = 1$  corrects the highest-order received digit  $r_6 = 1$  and also resets the syndrome. The new contents of the syndrome register are  $(0\ 0\ 0\ 0)$ . Thus, the new parity-check sums are all 0s. At this point, the error correction for the  $(7, 3)$  cyclic code is completed, and the next six received digits are error-free.

The following theorem gives the necessary condition for one-step majority-logic decoding of an  $(n, k)$  cyclic code. However, for many cyclic codes this theorem severely limits the number of errors which can be corrected with one-step majority-logic decoding.

**Theorem 8.3** Let  $d'_{\min}$  be the minimum distance of the  $(n, n - k)$  dual code of an  $(n, k)$  cyclic code. Then the upper bound on the error-correcting ability  $t$  for one-step majority-logic decoding is



$$t \leq \left\lfloor \frac{n-1}{2(d'_{\min}-1)} \right\rfloor$$

**Proof** Suppose that there exist  $J$  vectors made up of linear combinations of the rows of  $\mathbf{H}$ , which are orthogonal on a digit, in the highest-order position. Since the minimum weight in the null space is  $d_{\min}$ , it must have at least  $d'_{\min}$  digits in each of these  $J$  vectors. Hence, the total number of 1s in these  $J$  vectors is at least  $Jd'_{\min}$ . Since there are  $n-1$  error digits in addition to the ones on which the  $J$  vectors are orthogonal, the total number of 1s in these  $J$  vectors cannot exceed  $J + (n-1)$ . Thus, it follows that  $Jd'_{\min} \leq J + (n-1)$ . Therefore, it is possible to construct  $J \leq \lfloor (n-1)/(d'_{\min}-1) \rfloor$  because  $J$  is an integer. However, since  $t$  is at most half of the number  $J$ , we have  $t \leq \lfloor (n-1)/[2(d'_{\min}-1)] \rfloor$ , and the theorem is proved.

As explained before, if the  $J$  parity-check sums orthogonal on an error digit can be formed for a given  $(n, k)$  cyclic code, the one-step majority-logic decoding can easily be implemented. Let  $\mathbf{c}$  be a code word and  $\mathbf{r}$  be a received word. For any vector  $\mathbf{c}$  in an  $(n, k)$  cyclic code and any vector  $\boldsymbol{\phi}$  in its  $(n, n-k)$  dual code, the inner product of  $\boldsymbol{\phi}$  and  $\mathbf{r}$  will produce the following check sum  $\xi$ , which is the linear sum of the received digits:

$$\xi = \boldsymbol{\phi} \cdot \mathbf{r} = \phi_0 r_0 + \phi_1 r_1 + \phi_2 r_2 + \cdots + \phi_{n-1} r_{n-1} \quad (8.12)$$

If  $\mathbf{r} = \mathbf{c}$ ,  $\xi$  becomes zero; but if  $\mathbf{r} \neq \mathbf{c}$ ,  $\xi$  is certainly not zero. Since  $\boldsymbol{\phi} \cdot \mathbf{c} = 0$ , Eq. (8.12) becomes  $\xi = \boldsymbol{\phi} \cdot (\mathbf{c} + \mathbf{e}) = \boldsymbol{\phi} \cdot \mathbf{e}$ . Hence, we obtain the following relationship between the check sum  $\xi$  and the error digits in  $\mathbf{e}$ :

$$\xi = \phi_0 e_0 + \phi_1 e_1 + \cdots + \phi_{n-2} e_{n-2} + \phi_{n-1} e_{n-1} \quad (8.13)$$

Let  $\boldsymbol{\phi}_k$  be either a row vector  $\mathbf{h}_k$  in the row space of  $\mathbf{H}$  or a linear combination thereof. Then, finding the  $J$  check sums orthogonal on  $e_{n-1}$  is equivalent to finding the  $J$  vectors  $\boldsymbol{\phi}_k$  for  $k = 1, 2, \dots, J$  that are orthogonal on the  $(n-1)$ th digit position. The  $J$  parity-check sums  $\xi_k$ ,  $1 \leq k \leq J$ , orthogonal on the error digit  $e_{n-1}$  are formed from these  $J$  orthogonal vectors  $\boldsymbol{\phi}_k$ ,  $k = 1, 2, \dots, J$ , and the error pattern  $\mathbf{e}$ :

$$\xi_k = \boldsymbol{\phi}_k \cdot \mathbf{e} \quad 1 \leq k \leq J$$

That is,

$$\begin{aligned} \xi_1 &= \boldsymbol{\phi}_1 \cdot \mathbf{e} = \phi_{1,0} e_0 + \phi_{1,1} e_1 + \cdots + \phi_{1,n-1} e_{n-1} \\ \xi_2 &= \boldsymbol{\phi}_2 \cdot \mathbf{e} = \phi_{2,0} e_0 + \phi_{2,1} e_1 + \cdots + \phi_{2,n-1} e_{n-1} \\ &\vdots \\ \xi_J &= \boldsymbol{\phi}_J \cdot \mathbf{e} = \phi_{J,0} e_0 + \phi_{J,1} e_1 + \cdots + \phi_{J,n-1} e_{n-1} \end{aligned} \quad (8.14)$$

From Eq. (8.14), we see that the vector  $\boldsymbol{\phi}_k$  tells which error digits should be summed up to form the check sum  $\xi_k$ . In fact, the  $J$  check

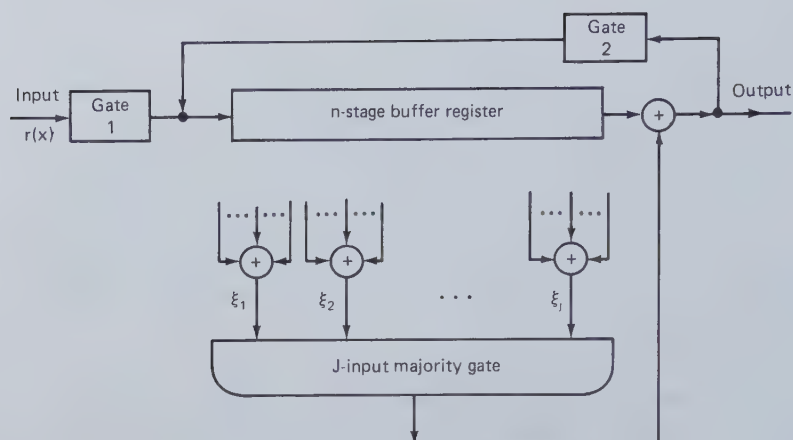


Figure 8.3 General type II one-step majority-logic decoder.

sums can be formed by using  $J$  multi-input modulo-2 adders. Once these  $J$  check sums are formed, they are used as inputs to the majority gate to estimate  $e_{n-1}$ . If the  $(n-1)$ th component of each vector  $\phi_k$ ,  $k = 1, 2, \dots, J$ , in the null space of an  $(n, k)$  cyclic code is 1, the error digit  $e_{n-1}$  is of course checked by all check sums, and it is used for correcting the first (rightmost) received digit  $r_{n-1}$ . Therefore, when a majority vote of check sums orthogonal on  $e_{n-1}$  is 1, the output of the  $J$ -input majority gate is 1 and it is also used to decode the first received digit  $r_{n-1}$ . Since the code is cyclic, the other error digits are corrected in the same way that  $e_{n-1}$  is decoded. Thus, the received vector  $\mathbf{r} = (r_0, r_1, \dots, r_{n-1})$  is decoded bit by bit in this manner until a total of  $n$  digits have been decoded. A general type II decoder for one-step majority-logic decoding is shown in Fig. 8.3.

The decoding procedure is as follows (see Fig. 8.3): The received word  $\mathbf{r}$  is put into the buffer register with gate 1 On and gate 2 Off. The  $J$  check sums orthogonal on  $e_{n-1}$  are formed by summing the proper received digits. These  $J$  check sums are fed into the majority gate. The first received digit  $r_{n-1}$ , which is read out of the buffer register, is corrected by the output of the majority gate. Soon after the first received digit  $r_{n-1}$  is corrected, the second received digit  $r_{n-2}$ , which has already been moved into the rightmost stage of the buffer, will be corrected in exactly the same manner as  $r_{n-1}$ , if  $r_{n-2}$  is an erroneous digit of  $r(x)$ . Thus, the received word  $\mathbf{r} = (r_0, r_1, \dots, r_{n-1})$  is decoded bit by bit for a total of  $n$  shifts.

**Example 8.3** Consider again the  $(7, 3)$  simplex code given in Example 8.2. The parity-check matrix of this code in systematic form is as follows:

$$\mathbf{H} = \begin{bmatrix} \mathbf{h}_0 \\ \mathbf{h}_1 \\ \mathbf{h}_2 \\ \mathbf{h}_3 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 \end{bmatrix}$$

Since the minimum distance of this code is 4,  $J = d_{\min} - 1 = 3$  orthogonal vectors  $\phi_k$ ,  $k = 1, 2, 3$ , can be formed by taking linear combinations of rows of  $\mathbf{H}$ :

$$\begin{aligned} \mathbf{f}_1 &= \mathbf{h}_0 + \mathbf{h}_2 = (1 \ 0 \ 1 \ 0 \ 0 \ 0 \ 1) \\ \phi_2 &= \mathbf{h}_1 = (0 \ 1 \ 0 \ 0 \ 0 \ 1 \ 1) \\ \phi_3 &= \mathbf{h}_3 = (0 \ 0 \ 0 \ 1 \ 1 \ 0 \ 1) \end{aligned} \quad (8.15)$$

From Eq. (8.15), we see that the rightmost component of each vector is 1 and no two vectors have a 1 at any other location. Therefore, these three vectors are orthogonal on the digit position 7. Let  $\mathbf{e} = (e_0, e_1, \dots, e_6)$  be the error pattern. The received vector represents  $\mathbf{r} = \mathbf{c} + \mathbf{e}$ . The inner product of  $\mathbf{r}$  and  $\phi$  produces the parity-check sums of  $\xi = \phi \cdot \mathbf{r} = \phi \cdot \mathbf{e}$  because  $\phi \cdot \mathbf{c} = 0$ . Thus the three check sums are related to the error digits as follows:

$$\begin{aligned} \xi_1 &= \phi_1 \cdot \mathbf{e} = e_0 && + e_2 && + e_6 \\ \xi_2 &= \phi_2 \cdot \mathbf{e} = && e_1 && + e_5 + e_6 \\ \xi_3 &= \phi_3 \cdot \mathbf{e} = && e_3 + e_4 && + e_6 \end{aligned} \quad (8.16)$$

As observed from Eq. (8.16),  $e_6$  is checked by all three check sums and no other digit is checked by more than one check sum. Now, consider an error pattern  $\mathbf{e} = (0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1)$  with a lone error at location  $x^6$  when an all-zero code word is transmitted. From the three check sums orthogonal on  $e_6 = 1$ , we have  $\xi_1 = \xi_2 = \xi_3 = 1$ . Since all three check sums are 1s,  $e_6$  is decoded as 1. As a consequence,  $e_6 \oplus r_6 = 1 \oplus 1 = 0$ . Thus the received vector  $\mathbf{r} = (0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1)$  is corrected as  $(0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0)$ . This results in a correct decoding, and the type II decoder for this (7, 3) simplex code is shown in Fig. 8.4.

As pointed out at the beginning of this chapter, there exist very few majority-logic decodable codes that are completely orthogonalizable. A list of some one-step majority-logic decodable codes is given in Table 8.1.

There are two more small classes of one-step majority-logic decodable codes: the maximum-length code and the difference-set code.

### 8.2.1 Maximum-length codes

There is a  $(2^m - 1, m)$  maximum-length code,  $m \geq 3$ , with minimum distance  $d_{\min} = 2^{m-1}$ . These codes are the duals of Hamming codes of length  $2^m - 1$ , which are obtained by taking a primitive polynomial

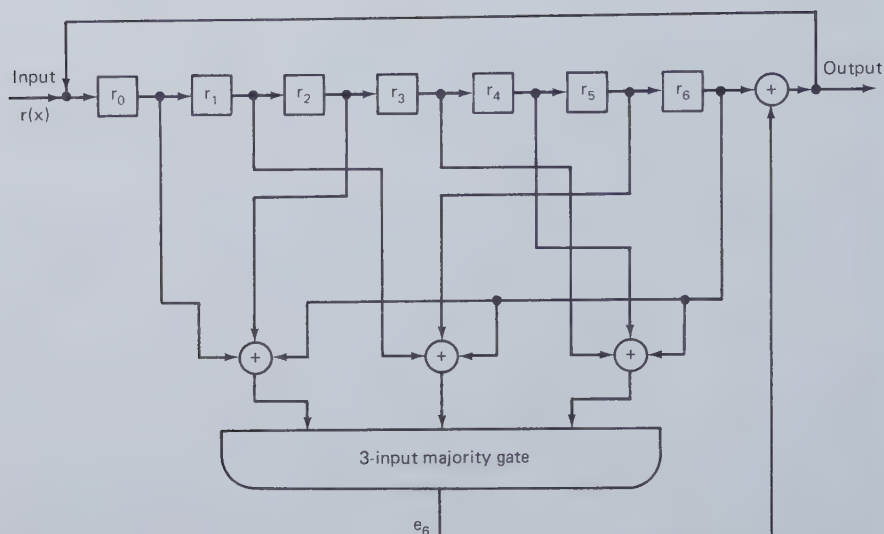


Figure 8.4 The type II one-step majority-logic decoder for the (7, 3) simplex code.

$p(x)$  of degree  $m$  as their generator polynomials. The dual code of the same length is obtained by letting the parity-check polynomial  $h(x)$  equal  $p(x)$ . Therefore, the generator polynomial of the maximum-length code is defined as

$$g(x) = \frac{x^n + 1}{p(x)} \quad (8.17)$$

where  $p(x)$  is a primitive polynomial of degree  $m$ . Hence, the dual code of the maximum-length code is nothing but a  $(2^m - 1, 2^m - m - 1)$  Hamming code. This dual code of a  $(2^m - 1, m)$  code is generated by the reciprocal polynomial  $p^*(x) = x^m p(x^{-1})$ , which is also a primitive polynomial of degree  $m$ .

Consider the set  $S$  consisting of distinct code polynomials

$$S = \{\varphi(x) = x^i + x^k + x^{n-1}, 0 \leq i < k \leq n-1\} \quad (8.18)$$

in the null space of the maximum-length code generated by the reciprocal polynomial  $p^*(x)$ . The necessary condition for this set is that no two polynomials in  $S$  have any common terms except the term  $x^{n-1}$ . Once this requirement is met, the set  $S$  contains polynomials orthogonal on the highest-order digit position  $x^{n-1}$ . To find each distinct code polynomial  $\varphi(x)$ , we must determine  $x^i$  so that  $x^{n-1} + x^k + x^i$  is divisible by  $p^*(x)$ . Let us divide  $x^{n-1} + x^k$  by  $p^*(x)$  step by step with long division until a single term  $x^i$  appears at the end of some step. But the following two conditions must be carefully observed: (1) the exponent

TABLE 8.1 List of Some Majority-Logic Decodable Codes

| $s$ | $m$ | $n$ | $k$ | $d$ | $t$ | Generator polynomial, $g(x)^*$                                                                                                                                                                                                                                 | Check polynomial, $h(x)$                                                                  | Codes† |
|-----|-----|-----|-----|-----|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|--------|
| 1   | 3   | 7   | 3   | 4   | 1   | 0, 2, 3, 4                                                                                                                                                                                                                                                     | 0, 2, 3                                                                                   | ML     |
|     |     | 7   | 3   | 4   | 1   | 0, 2, 3, 4                                                                                                                                                                                                                                                     | 0, 2, 3                                                                                   | DS     |
|     |     | 15  | 7   | 5   | 2   | 0, 4, 6, 7, 8                                                                                                                                                                                                                                                  | 0, 4, 6, 7                                                                                | BCH    |
|     | 4   | 15  | 7   | 5   | 2   | 0, 1, 2, 4, 8                                                                                                                                                                                                                                                  | 0, 1, 3, 7                                                                                | BCH    |
|     |     | 15  | 4   | 8   | 3   | 0, 1, 2, 3, 5, 7, 8, 11                                                                                                                                                                                                                                        | 0, 1, 4                                                                                   | ML     |
| 2   | 21  | 11  | 11  | 6   | 2   | 0, 2, 4, 6, 7, 10                                                                                                                                                                                                                                              | 0, 2, 7, 8, 11                                                                            | DS     |
|     |     | 31  | 5   | 16  | 7   | 0, 2, 4, 5, 6, 8, 9, 13, 14, 15, 16, 17, 20, 21, 23, 26                                                                                                                                                                                                        | 0, 2, 5                                                                                   | ML     |
|     | 63  | 37  | 9   | 4   | 4   | 0, 2, 10, 11, 12, 13, 14, 16, 20, 24, 26                                                                                                                                                                                                                       | 0, 2, 4, 6, 8, 11, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 27, 29, 33, 35, 37 | EG     |
|     |     |     |     |     |     | 0, 1, 3, 5, 6, 9, 13, 14, 15, 18, 19, 20, 23, 25, 26, 31, 32, 33, 37, 38, 47, 49, 50                                                                                                                                                                           | 0, 1, 2, 4, 5, 6, 9, 11, 12, 13                                                           | EG     |
|     | 6   | 63  | 6   | 32  | 15  | 0, 1, 2, 3, 4, 5, 7, 9, 11, 12, 15, 16, 18, 19, 20, 22, 23, 25, 28, 31, 32, 33, 37, 39, 40, 41, 42, 45, 47, 51, 52, 57                                                                                                                                         | 0, 1, 6                                                                                   | ML     |
| 3   | 73  | 45  | 10  | 10  | 4   | 0, 2, 4, 6, 8, 12, 16, 22, 25, 28                                                                                                                                                                                                                              | 0, 2, 10, 24, 25, 29, 36, 42, 45                                                          | DS     |
|     |     | 85  |     | 22  | 10  | 0, 1, 3, 4, 5, 6, 8, 9, 11, 13, 14, 17, 19, 20, 21, 25, 27, 28, 31, 32, 33, 35, 36, 39, 40, 41, 47, 49, 50, 58, 60, 61                                                                                                                                         | 0, 1, 2, 5, 8, 10, 11, 12, 15, 16, 17, 20, 22, 23, 24                                     | PG     |
|     | 7   | 127 | 7   | 64  | 31  | 0, 1, 2, 3, 4, 5, 6, 8, 10, 12, 15, 16, 19, 20, 21, 23, 24, 25, 27, 30, 32, 33, 37, 38, 40, 41, 42, 43, 45, 46, 48, 50, 51, 53, 54, 57, 60, 64, 65, 66, 71, 73, 74, 75, 76, 77, 80, 82, 84, 85, 86, 89, 90, 92, 96, 99, 100, 101, 102, 106, 108, 113, 114, 120 | 0, 1, 7                                                                                   | ML     |
|     |     |     |     |     |     |                                                                                                                                                                                                                                                                |                                                                                           |        |
|     |     |     |     |     |     |                                                                                                                                                                                                                                                                |                                                                                           |        |



TABLE 8.1 List of Some Majority-Logic Decodable Codes (Continued)

| $s$ | $m$ | $n$ | $k$ | $d$ | $t$ | Generator polynomial, $g(x)^*$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | Check polynomial, $h(x)$ | Codes† |
|-----|-----|-----|-----|-----|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------|--------|
| 8   |     | 255 | 8   | 128 | 63  | 0, 1, 2, 3, 4, 8, 11, 13, 17, 21, 23, 24, 26, 28, 30, 31, 32, 35, 39, 40, 43, 46, 49, 50, 51, 58, 63, 66, 67, 69, 70, 71, 72, 75, 76, 79, 80, 81, 83, 84, 85, 86, 87, 88, 91, 93, 94, 97, 99, 101, 102, 104, 105, 107, 110, 116, 117, 121, 122, 124, 126, 127, 131, 133, 135, 137, 140, 141, 147, 149, 150, 151, 153, 155, 156, 157, 158, 159, 161, 162, 164, 165, 166, 168, 169, 174, 176, 179, 181, 182, 183, 184, 186, 189, 190, 191, 192, 193, 194, 195, 196, 198, 200, 205, 206, 207, 210, 211, 212, 213, 218, 219, 221, 222, 225, 226, 228, 232, 233, 234, 235, 237, 238, 239, 243, 244, 245, 247 | 0, 1, 5, 6, 8            | ML     |

\*For example, the generator polynomial  $g(x) = 1 + x^2 + x^3 + x^4$  represents  $\{0, 2, 3, 4\}$ .

†Codes: ML, maximum-length code; DS, difference-set code; EG, euclidean geometry code; PG, projective geometry code.

$i$  of the step remainder  $x^i$  must be less than  $k$  and (2) the step remainder must be deleted if it consists of more than one term. Then  $\varphi(x) = x^{n-1} + x^k + x^i$  is a polynomial orthogonal on digit position  $x^{n-1}$  and is divisible by  $p^*(x)$ . Since the error-correction capability  $t$  of a Hamming code is 1 (see Theorem 8.3), the number of polynomials orthogonal on digit position  $x^{n-1}$  must be  $(n-1)/2 = 2^{m-1} - 1$ . Thus the total of the check sums orthogonal on  $e_{n-1}$  is given as  $J = d_{\min} - 1 = 2^{m-1} - 1$ . As a result, the maximum-length code with minimum distance  $d_{\min} = 2^{m-1}$  is completely orthogonalizable in one step because it is possible to form  $J = 2^{m-1} - 1$  check sums orthogonal on an error digit  $e_{n-1}$ .

**Example 8.4** Consider the (7, 3) maximum-length code with  $m = 3$  and the check polynomial  $p(x) = 1 + x^2 + x^3$ . This code has block length  $n = 2^m - 1 = 7$ , minimum distance  $d_{\min} = 2^{m-1} = 4$ , and number of parity-check sums  $J = 2^{m-1} - 1 = 3$ . Since the generator polynomial of this code is

$$g(x) = \frac{x^7 + 1}{p(x)} = 1 + x^2 + x^3 + x^4$$

it is the same code described in Examples 8.2 and 8.3. The dual code of the (7, 3) maximum-length code is generated by

$$p^*(x) = x^3 p(x^{-1}) = x^3 + x + 1$$

Divide  $x^6 + x^5$  by  $p^*(x) = x^3 + x + 1$  in order to obtain a polynomial  $\varphi_1(x)$  which is orthogonal on  $x^6$ . Then we have  $\varphi_1(x) = x^6 + x^5 + x$  because the exponent of a single term  $x$  is less than the exponent of  $x^5$ . Next, if we divide  $x^6 + x^4$  by  $p^*(x)$ , we obtain  $\varphi_2(x) = x^6 + x^4 + x^3$ , which is the second polynomial orthogonal on  $x^6$ . Dividing  $x^6 + x^3$  by  $p^*(x)$ , we obtain a single term  $x^4$  which appears at the end of the first step of long division. Thus a polynomial orthogonal on  $x^6$  does not form because the exponent of the step remainder  $x^4$  is greater than that of  $x^3$ . Finally, dividing  $x^6 + x^2$  by  $p^*(x)$ , we obtain a remainder of 1 and  $\varphi_3(x) = x^6 + x^2 + 1$ , which is the third polynomial orthogonal on  $x^6$ . From the set  $S = \{\varphi_1(x), \varphi_2(x), \varphi_3(x)\}$ , we can form the following three parity-check sums orthogonal on  $x^6$ :

$$\begin{aligned}\xi_1 &= e_1 && + e_5 + e_6 \\ \xi_2 &= && e_3 + e_4 && + e_6 \\ \xi_3 &= e_0 && + e_2 && + e_6\end{aligned}\tag{8.19}$$

These three parity-check sums are exactly identical to Eq. (8.11) in Example 8.2 and Eq. (8.16) in Example 8.3. Thus this code is capable of correcting a single error by one-step majority-logic decoding. The type I and type II one-step majority decoders for this code are shown in Figs. 8.2 and 8.4, respectively.

The weight structure of these  $(2^m - 1, m)$  codes is easily determined by examining the encoder structure. The weight of each nonzero code word is  $2^m - 1$ , and the weight enumerator is

$$B(z) = 1 + (2^m - 1)z^{2^{m-1}} \quad (8.20)$$

which is related to that of the  $(2^m - 1, 2^m - m - 1)$  Hamming codes through the MacWilliams identity. When  $m$  is large, the encoder for these maximum-length codes produces a sequence with excellent randomness properties. It has been used in applications requiring random-number generation and also in those requiring pseudorandom binary sequences such as spread-spectrum systems.

**Example 8.5** Consider the  $(31, 5)$  maximum-length code with  $m = 5$  and generator polynomial  $g(x) = 1 + x^2 + x^4 + x^6 + x^8 + x^{13} + x^{14} + x^{15} + x^{16} + x^{17} + x^{20} + x^{21} + x^{23} + x^{26}$ . This code is capable of correcting seven errors or less by one-step majority-logic decoding. Since the parity polynomial is  $p(x) = 1 + x^2 + x^5$ , the null space of this code is generated by  $p^*(x) = x^5 p(x^{-1}) = 1 + x^3 + x^5$ . Divide  $x^{30} + x^i$  by  $p^*(x)$  step by step with long division until a single term  $x^j$  appears at the end of some step. But delete the dividends  $x^{30} + x^i$  for  $i = 20, 16, 14, 13, 12, 11, 10, 8, 7, 6, 5, 3, 2, 1, 0$ . Thus, the polynomial  $\varphi(x) = x^{30} + x^i + x^j$ ,  $0 \leq j < i < 30$ , orthogonal on digit position  $x^{30}$  is obtained as follows. The number of  $\varphi(x)$ -polynomials is  $2^{m-1} - 1 = 15$ .

$$\begin{aligned} \varphi_1(x) &= x^{12} + x^{29} + x^{30} & \varphi_2(x) &= x^{25} + x^{28} + x^{30} \\ \varphi_3(x) &= x + x^{27} + x^{30} & \varphi_4(x) &= x^{20} + x^{26} + x^{30} \\ \varphi_5(x) &= x^3 + x^{24} + x^{30} & \varphi_6(x) &= x^8 + x^{23} + x^{30} \\ \varphi_7(x) &= x^{10} + x^{22} + x^{30} & \varphi_8(x) &= x^{14} + x^{21} + x^{30} \\ \varphi_9(x) &= x^{11} + x^{19} + x^{30} & \varphi_{10}(x) &= x^7 + x^{18} + x^{30} \\ \varphi_{11}(x) &= x^{16} + x^{17} + x^{30} & \varphi_{12}(x) &= x^6 + x^{15} + x^{30} \\ \varphi_{13}(x) &= 1 + x^{13} + x^{30} & \varphi_{14}(x) &= x^5 + x^9 + x^{30} \\ \varphi_{15}(x) &= x^2 + x^4 + x^{30} \end{aligned}$$

From the set of these 15 polynomials orthogonal on  $x^{30}$ , the following 15 parity-check sums orthogonal on  $e_{30}$  can be obtained as:

$$\begin{aligned} \xi_1 &= e_{12} + e_{29} + e_{30} & \xi_2 &= e_{25} + e_{28} + e_{30} \\ \xi_3 &= e_1 + e_{27} + e_{30} & \xi_4 &= e_{20} + e_{26} + e_{30} \\ \xi_5 &= e_3 + e_{24} + e_{30} & \xi_6 &= e_8 + e_{23} + e_{30} \\ \xi_7 &= e_{10} + e_{22} + e_{30} & \xi_8 &= e_{14} + e_{21} + e_{30} \\ \xi_9 &= e_{11} + e_{19} + e_{30} & \xi_{10} &= e_7 + e_{18} + e_{30} \\ \xi_{11} &= e_{16} + e_{17} + e_{30} & \xi_{12} &= e_6 + e_{15} + e_{30} \end{aligned}$$

$$\xi_{13} = e_0 + e_{13} + e_{30} \quad \xi_{14} = e_5 + e_9 + e_{30}$$

$$\xi_{15} = e_2 + e_4 + e_{30}$$

The type I and type II one-step majority decoders for this code based on these check sums are shown in Figs. 8.5 and 8.6, respectively.

### 8.2.2 Difference-set codes

In this section, a new class of random-error-correcting codes, the difference-set codes, is presented. The difference-set codes can be more simply implemented than the BCH codes and are approximately as powerful, even though the class is small. Therefore, in this class there are relatively few codes with useful parameters. Binary difference-set cyclic codes have been investigated and refined by Rudolph (1964), Weldon (1966), Graham and MacWilliams (1966), and others. Since the difference-set code is based on the theory of perfect difference sets, it is recommended that the reader refer to works by Singer (1938) or Evans and Mann (1955).

Let

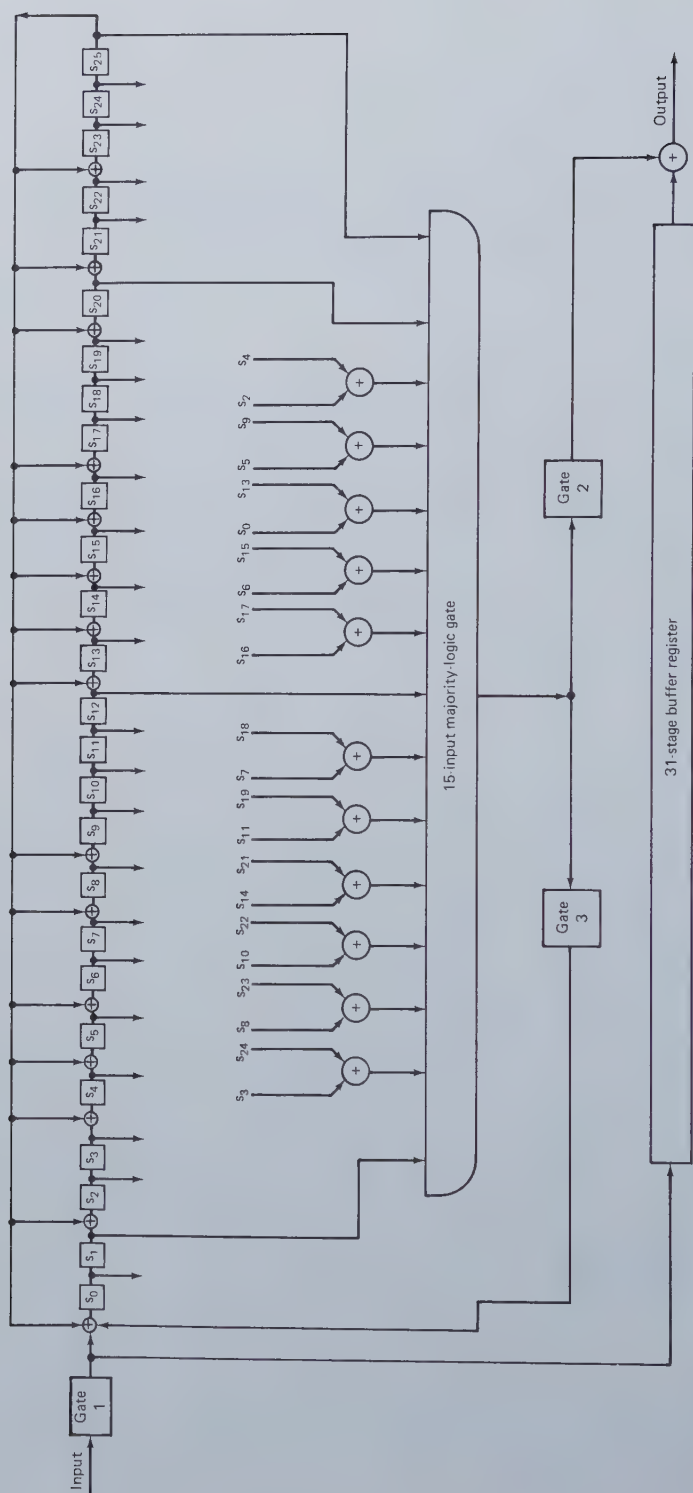
$$Q = \{d_0 = 0, d_1, d_2, \dots, d_q\} \quad 0 \leq d_0 < d_1 < \dots < d_q \quad (8.21)$$

be a perfect difference set whose elements are a collection of  $q + 1$  integers. If some  $d_i$  and  $d_j$  are two different elements in a set  $Q$ , it is possible to form  $q(q + 1)$  ordered differences such that every integer  $a (\neq 0)$  can be uniquely expressed in the form  $d_j - d_i = a$ ,  $i \neq j$ . In a simple perfect difference set for order  $q$ , all the positive or negative differences are distinct and their respective numbers are identically equal. The only known simple difference sets for order  $q$  are obtained from a construction developed by Singer who has shown how to construct such sets when  $q = p^s$ , where  $p$  is a prime and  $s$  is any positive integer. For this construction, the code length  $n$  must be of the form  $p^{2s} + p^s + 1 = q(q + 1) + 1$  for  $q = p^s$ , which determines the finite field  $GF(p)$ . In what follows, we only consider  $q = 2^s$  for radix-2 over  $GF(2)$ . For  $p = 2$  and  $1 \leq s \leq 5$ , Weldon found the number of parity-check digits  $n - k$  to be  $3^s + 1$ . The difference-set codes have a minimum distance of  $2^s + 1$ , and their dual codes have a minimum distance of at least  $2^s + 2$ . Let

$$\lambda(x) = 1 + x^{d_1} + x^{d_2} + \dots + x^{d_{2^s}} \quad (8.22)$$

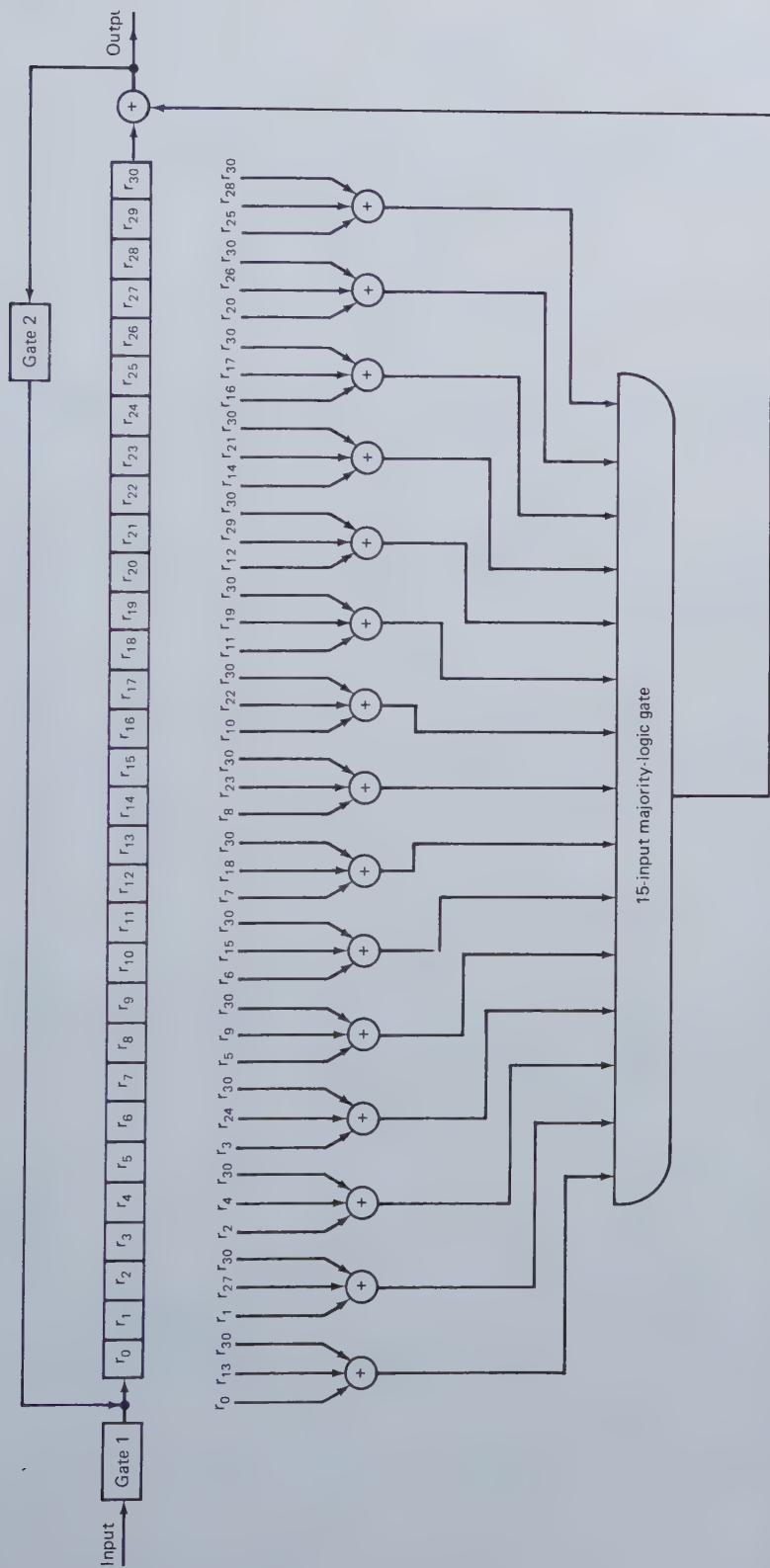
be the difference-set polynomial in the algebra of polynomials modulo  $(x^n + 1)$ .

Consider the product



**Figure 8.5** The type I one-step majority-logic decoder for the (31, 5) maximum-length code.





**Figure 8.6** The type II one-step majority-logic decoder for the (31, 5) maximum-length code.

$$\begin{aligned}
\lambda(x)\lambda(x^{-1}) &= (x^{d_0} + x^{d_1} + \dots + x^{d_{2^s}})(x^{-d_0} + x^{-d_1} + \dots + x^{-d_{2^s}}) \\
&= x^{d_0 - d_0} + x^{d_0 - d_1} + \dots + x^{d_0 - d_{2^s}} \\
&\quad + x^{d_1 - d_0} + x^{d_1 - d_1} + \dots + x^{d_1 - d_{2^s}} \\
&\quad \vdots \qquad \qquad \qquad \vdots \qquad \qquad \qquad \vdots \\
&\quad + x^{d_{2^s} - d_0} + x^{d_{2^s} - d_1} + \dots + x^{d_{2^s} - d_{2^s}} \qquad (8.23) \\
&= 2^s + 1 + x + x^2 + \dots + x^{n-1} \pmod{x^n + 1}
\end{aligned}$$

because each integer  $d_j - d_i = a(\neq 0)$ ,  $i \neq j$ , appears as an exponent in this polynomial exactly once. Since  $2^s = 0 \pmod{2}$ , Eq. (8.23) is reduced to

$$\begin{aligned}
\lambda(x)\lambda^{-1}(x) &= 1 + x + x^2 + \dots + x^{n-1} \\
&= \frac{x^n + 1}{x + 1} \pmod{x^n + 1} \qquad (8.24)
\end{aligned}$$

It can also be written as

$$\lambda(x)\lambda^{-1}(x)(x + 1) = x^n + 1 = 0 \pmod{x^n + 1} \quad (8.25)$$

This means that  $\lambda(x)$  has a nontrivial highest common factor  $h(x)$  in common with  $x^n + 1$  over  $GF(2)$ . The degree of  $h(x)$  is the number of zeros of  $x^n + 1$  which are also zeros of  $\lambda(x)$ . Let  $\lambda(x) = f(x)h(x)$  where  $h(x)$  of degree  $k$  is the greatest common divisor of  $\lambda(x)$  and  $x^n + 1$ ; that is,

$$\begin{aligned}
h(x) &= \text{GCD}\{\lambda(x), x^n + 1\} \\
&= 1 + h_1x + h_2x^2 + \dots + h_{k-1}x^{k-1} + x^k \qquad (8.26)
\end{aligned}$$

The cyclic codes (or ideals) in the algebra of polynomials modulo  $(x^n + 1)$  generated by  $\lambda(x)$  and  $h(x)$  are identical. Thus, the rank of the matrix  $L$  is simply  $n$  minus the degree of  $h(x)$ ; that is, the number of parity-check digits in the code generated by

$$\begin{aligned}
g(x) &= \frac{x^n + 1}{h(x)} \\
&= 1 + g_1x + g_2x^2 + \dots + g_{n-k-1}x^{n-k-1} + x^{n-k} \qquad (8.27)
\end{aligned}$$

The reciprocal polynomial of  $h(x)$ , which is denoted by  $h^*(x)$ , is equal to  $h^*(x) = x^k h(x^{-1})$ . Then the  $(n, n - k)$  cyclic code generated by  $h^*(x)$  is the null space of the difference-set code generated by  $g(x)$ .

Let  $\lambda^*(x)$  be the reciprocal polynomial of  $\lambda(x)$ . Then  $\lambda^*(x)$  is defined to be  $\lambda^*(x) = x^{\text{degree of } \lambda(x)} \lambda(x^{-1})$ . Thus

$$\begin{aligned}
\lambda^*(x) &= x^{d_{2^s}} \lambda(x^{-1}) \\
&= 1 + x^{d_{2^s} - d_{2^s} - 1} + \dots + x^{d_{2^s} - d_2} + x^{d_{2^s} - d_1} + x^{d_{2^s}} \qquad (8.28)
\end{aligned}$$

Since  $\lambda^*(x)$  is in the null space of the difference-set code generated by  $g(x)$ ,  $\lambda^*(x)$  is divisible by  $h^*(x)$  and  $\lambda(x)$  is divisible by  $h(x)$ . Since  $\lambda(x) = f(x)h(x)$ , Eq. (8.28) yields

$$\begin{aligned}\lambda^*(x) &= x^{\text{degree of } \lambda(x)} \lambda(x^{-1}) \\ &= x^{\text{degree of } f(x)} f(x^{-1}) x^{\text{degree of } h(x)} h(x^{-1}) \\ &= f^*(x)h^*(x)\end{aligned}\quad (8.29)$$

Now, if we define  $\varphi_0(x)$  as follows:

$$\begin{aligned}\varphi_0(x) &= x^{n-1-d_2^s} \lambda^*(x) \\ &= x^{n-1-d_2^s} + x^{n-1-d_2^s-1} + \dots + x^{n-1-d_2} + x^{n-1-d_1} + x^{n-1}\end{aligned}\quad (8.30)$$

$\varphi_0(x)$  is also in the null space of the difference-set code generated by  $g(x)$  because  $h^*(x)$  is a factor of  $\varphi_0(x)$ . Consider the polynomials obtained by shifting  $\varphi_0(x)$  cyclically to the right  $d_i$  times in sequence. We have then

$$\begin{aligned}\varphi_1(x) &= \varphi_0^{(d_1)}(x) = x^{d_1} \varphi_0(x) \\ &= x^{d_1-1} + x^{n-1-d_2^s+d_1} + x^{n-1-d_2^s-1+d_1} + \dots + x^{n-1-d_2+d_1} + x^{n-1} \\ \varphi_2(x) &= \varphi_0^{(d_2)}(x) = x^{d_2} \varphi_0(x) \\ &= x^{d_2-d_1-1} + x^{d_2-1} + x^{n-1-d_2^s+d_2} \\ &\quad + x^{n-1-d_2^s-1+d_2} + \dots + x^{n-1-d_3+d_2} + x^{n-1} \\ &\vdots \\ \varphi_i(x) &= \varphi_0^{(d_i)}(x) = x^{d_i} \varphi_0(x) \\ &= x^{d_i-d_{i-1}-1} + x^{d_i-d_{i-2}-1} + \dots + x^{d_i-d_1-1} + x^{d_i-1} \\ &\quad + x^{n-1-d_2^s+d_i} + x^{n-1-d_2^s-1+d_i} + \dots + x^{n-1-d_{i+1}+d_i} + x^{n-1}\end{aligned}\quad (8.31)$$

Since  $Q = \{d_0 = 0, d_1, d_2, \dots, d_{2^s}\}$  is a perfect difference set, no two polynomials  $\varphi_i(x)$  and  $\varphi_j(x)$ , where  $i \neq j$ , can have any common term except for  $x^{n-1}$ . Thus,  $\varphi_0(x), \varphi_1(x), \dots, \varphi_{2^s}(x)$  form a set of  $J = 2^s + 1$  polynomials orthogonal on the digit at position  $x^{n-1}$ . Since the dual code generated by  $h(x)$  is proved to have the minimum distance  $2^s + 2$ , the number of parity-check sums is  $J = 2^s + 1$ , and it is capable of correcting  $t = [(2^s + 1)/2] = 2^{s-1}$  or fewer errors.

Difference-set codes of most practical interest that are one-step orthogonalizable are the (21, 11), (73, 45), and (273, 191) codes. These codes were introduced by Weldon in 1966 and are constructed using perfect difference sets. A list of the first few binary difference-set

TABLE 8.2 List of Binary Difference-Set Codes

| $s$ | $n$  | $k$ | $d$ | $t$ | Generator polynomial,* $g(x)$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | Difference-set polynomial, $\lambda(x)$                                                                                                                  |
|-----|------|-----|-----|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1   | 7    | 3   | 4   | 1   | 0, 2, 3, 4                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | 0, 2, 3                                                                                                                                                  |
| 2   | 21   | 11  | 6   | 2   | 0, 2, 4, 6, 7, 10                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | 0, 2, 7, 8, 11                                                                                                                                           |
| 3   | 73   | 45  | 10  | 4   | 0, 2, 4, 6, 8, 12, 16, 22, 25, 28                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | 0, 2, 10, 24, 25, 29, 36, 42, 45                                                                                                                         |
| 4   | 273  | 191 | 18  | 8   | 0, 4, 10, 18, 22, 24, 34, 36, 40, 48, 52, 56, 66, 67, 71, 77, 82                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | 0, 18, 24, 46, 50, 67, 103, 112, 115, 126, 128, 159, 166, 167, 186, 196, 201                                                                             |
| 5   | 1057 | 813 | 34  | 16  | 0, 1, 3, 4, 5, 11, 14, 17, 18, 22, 23, 26, 27, 28, 32, 33, 35, 37, 39, 41, 43, 45, 47, 48, 51, 52, 55, 59, 62, 68, 70, 71, 72, 74, 75, 76, 79, 81, 83, 88, 95, 96, 98, 101, 103, 105, 106, 108, 111, 114, 115, 116, 120, 121, 122, 123, 124, 126, 129, 131, 132, 135, 137, 138, 141, 142, 146, 147, 149, 150, 151, 153, 154, 155, 158, 160, 161, 164, 165, 166, 167, 169, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 186, 188, 189, 191, 193, 194, 195, 198, 199, 200, 201, 202, 203, 208, 209, 210, 211, 212, 214, 216, 222, 224, 226, 228, 232, 234, 236, 242, 244 | 0, 1, 3, 7, 15, 31, 54, 63, 109, 127, 138, 219, 255, 277, 298, 338, 348, 439, 452, 511, 528, 555, 597, 677, 697, 702, 754, 792, 897, 905, 924, 990, 1023 |

\*For example, a generator polynomial represented by the exponents  $\{0, 2, 4, 6, 7, 10\}$  designates  $g(x) = 1 + x^2 + x^4 + x^6 + x^7 + x^{10}$ .

codes with their generator polynomials and their corresponding perfect simple difference sets is given in Table 8.2.

**Example 8.6** Consider the  $(7, 3)$  difference-set code which is specified by a Singer difference set  $Q = \{0, 2, 3\}$  of order 2. There exist the  $2 \times 3 = 6$  ordered differences  $\{2, 3, 1, -2, -3, -1\}$ . Referring to Table 8.2, we have the difference-set polynomial for this code:  $\lambda(x) = 1 + x^2 + x^3$ . Then, we have

$$\begin{aligned} h(x) &= \text{GCD}\{1 + x^2 + x^3, 1 + x^7\} \\ &= 1 + x^2 + x^3 \end{aligned}$$

The generator polynomial of this difference-set code of length  $n = 7$  is

$$g(x) = \frac{x^7 + 1}{h(x)} = 1 + x^2 + x^3 + x^4$$

Since  $h^*(x) = x^k h(x^{-1})$  is the reciprocal polynomial of  $h(x)$ ,  $\lambda^*(x)$  is divisible by  $h^*(x)$ , but in this case  $\lambda^*(x)$  is actually equal to  $h^*(x)$ . Therefore, we have

$$\lambda^*(x) = x^3(1 + x^{-2} + x^{-3}) = 1 + x + x^3$$

Hence,

$$\varphi_0(x) = x^3 \lambda^*(x) = x^3 + x^4 + x^6$$

By cyclically shifting  $\varphi_0(x)$  to the right in sequence, we obtain

$$\varphi_1(x) = \varphi_0^{(2)}(x) = x + x^5 + x^6$$

$$\varphi_2(x) = \varphi_0^{(3)}(x) = 1 + x^2 + x^6$$

Thus, we see that these three polynomials  $\varphi_0(x)$ ,  $\varphi_1(x)$ , and  $\varphi_2(x)$  are orthogonal on  $x^6$ . From these orthogonal polynomials, the three parity-check sums orthogonal on  $e_6$  can be obtained as follows:

$$\xi_1 = e_3 + e_4 + e_6$$

$$\xi_2 = e_1 + e_5 + e_6$$

$$\xi_3 = e_0 + e_2 + e_6$$

Now, we can see that these three check sums are identical to the results discussed in Examples 8.2, 8.3, and 8.4.

**Example 8.7** Consider the  $(73, 45)$  difference-set code with a perfect difference set  $Q = \{0, 2, 10, 24, 25, 29, 36, 42, 45\}$  of order  $2^3$ . There exist  $8 \times 9 = 72$  ordered differences from the set  $Q$ . Let  $\lambda(x) = 1 + x^2 + x^{10} + x^{24} + x^{25} + x^{29} + x^{36} + x^{42} + x^{45}$  be the difference-set polynomial in the algebra of polynomials modulo  $(x^{73} + 1)$ . Then the check polynomial becomes

$$\begin{aligned} h(x) &= \text{GCD}\{\lambda(x), x^{73} + 1\} \\ &= 1 + x^2 + x^{10} + x^{24} + x^{25} + x^{29} + x^{36} + x^{42} + x^{45} \end{aligned}$$

The generator polynomial of the difference-set code of length  $n = 73$  is obtained as

$$\begin{aligned} g(x) &= \frac{x^{73} + 1}{h(x)} \\ &= 1 + x^2 + x^4 + x^6 + x^8 + x^{12} + x^{16} + x^{22} + x^{25} + x^{28} \end{aligned}$$



Since  $\lambda^*(x)$  is the reciprocal polynomial of  $\lambda(x)$ ,

$$\begin{aligned}\lambda^*(x) &= x^{45}\lambda(x^{-1}) \\ &= 1 + x^3 + x^9 + x^{16} + x^{20} + x^{21} + x^{35} + x^{43} + x^{45}\end{aligned}$$

First, compute

$$\begin{aligned}\varphi_0(x) &= x^{27}\lambda^*(x) \\ &= x^{27} + x^{30} + x^{36} + x^{43} + x^{47} + x^{48} + x^{62} + x^{70} + x^{72}\end{aligned}$$

Second, cyclically shift  $\varphi_0(x)$  to the right 2, 10, ..., 42, 45 times. Then, we get the following orthogonal polynomials:

$$\begin{aligned}\varphi_1(x) &= x + x^{29} + x^{32} + x^{38} + x^{45} + x^{49} + x^{50} + x^{64} + x^{72} \\ \varphi_2(x) &= x^7 + x^9 + x^{37} + x^{40} + x^{46} + x^{53} + x^{57} + x^{58} + x^{72} \\ \varphi_3(x) &= x^{13} + x^{21} + x^{23} + x^{51} + x^{54} + x^{60} + x^{67} + x^{71} + x^{72} \\ \varphi_4(x) &= 1 + x^{14} + x^{22} + x^{24} + x^{52} + x^{55} + x^{61} + x^{68} + x^{72} \\ \varphi_5(x) &= x^3 + x^4 + x^{18} + x^{26} + x^{28} + x^{56} + x^{59} + x^{65} + x^{72} \\ \varphi_6(x) &= x^6 + x^{10} + x^{11} + x^{25} + x^{33} + x^{35} + x^{63} + x^{66} + x^{72} \\ \varphi_7(x) &= x^5 + x^{12} + x^{16} + x^{17} + x^{31} + x^{39} + x^{41} + x^{69} + x^{72} \\ \varphi_8(x) &= x^2 + x^8 + x^{15} + x^{19} + x^{20} + x^{34} + x^{42} + x^{44} + x^{72}\end{aligned}$$

These nine polynomials are orthogonal on  $x^{72}$ . Hence, from these nine orthogonal polynomials, the following nine parity-check sums orthogonal on  $e_{72}$  are formed:

$$\begin{aligned}\xi_1 &= s_{27} &= e_{27} + e_{30} + e_{36} + e_{43} + e_{47} + e_{48} + e_{62} + e_{70} + e_{72} \\ \xi_2 &= s_1 &= e_1 + e_{29} + e_{32} + e_{38} + e_{45} + e_{49} + e_{50} + e_{64} + e_{72} \\ \xi_3 &= s_7 + s_9 &= e_7 + e_9 + e_{37} + e_{40} + e_{46} + e_{53} + e_{57} + e_{58} + e_{72} \\ \xi_4 &= s_{13} + s_{21} + s_{23} &= e_{13} + e_{21} + e_{23} + e_{51} + e_{54} + e_{60} + e_{67} + e_{71} + e_{72} \\ \xi_5 &= s_0 + s_{14} + s_{22} + s_{24} &= e_0 + e_{14} + e_{22} + e_{24} + e_{52} + e_{55} + e_{61} + e_{68} + e_{72} \\ \xi_6 &= s_3 + s_4 + s_{18} + s_{26} &= e_3 + e_4 + e_{18} + e_{26} + e_{28} + e_{56} + e_{59} + e_{65} + e_{72}\end{aligned}$$

$$\xi_7 = s_6 + s_{10} + s_{11} + s_{25} = e_6 + e_{10} + e_{11} + e_{25} + e_{33} + e_{35} + e_{63} + e_{66} + e_{72}$$

$$\xi_8 = s_5 + s_{12} + s_{16} + s_{17} = e_5 + e_{12} + e_{16} + e_{17} + e_{31} + e_{39} + e_{41} + e_{69} + e_{72}$$

$$\xi_9 = s_2 + s_8 + s_{15} + s_{19} + s_{20} = e_2 + e_8 + e_{15} + e_{19} + e_{20} + e_{34} + e_{42} + e_{44} + e_{72}$$

### 8.3 Multiple-Step Majority-Logic Decoding

In the preceding sections, we have shown that one-step majority-logic decoding is effective for some cyclic codes if a clear majority of the parity-check sums orthogonal on an erroneous digit can be formed. However, the concept of parity-check sums orthogonal on a single error digit can be extended in such a way that many cyclic codes could be decoded by employing many levels of majority-logic gates, as explained below.

Let  $\mathbf{e} = (e_0, e_1, e_2, \dots, e_{n-1})$  be the error pattern which developed during the transmission of messages. Consider the set  $E_{\alpha_i}$ ,  $i = 1, 2, \dots, (1 \leq i < n)$ , whose members are some properly selected error digits in the error pattern  $\mathbf{e}$ . That is,

$$\begin{aligned} E_{\alpha_1} &= \{e_{\alpha_{1,1}}, e_{\alpha_{1,2}}, \dots, e_{\alpha_{1,M}}\} \\ E_{\alpha_2} &= \{e_{\alpha_{2,1}}, e_{\alpha_{2,2}}, \dots, e_{\alpha_{2,M}}\} \\ E_{\alpha_3} &= \{e_{\alpha_{3,1}}, e_{\alpha_{3,2}}, \dots, e_{\alpha_{3,M}}\} \\ &\vdots \end{aligned} \tag{8.32}$$

where  $M$  is the size of the set.

Let  $\Sigma E_{\alpha_i}$  denote the modulo-2 sum of the error digits in  $E_{\alpha_i}$ , namely,  $\Sigma E_{\alpha_i} = e_{\alpha_{i,1}} + e_{\alpha_{i,2}} + \dots + e_{\alpha_{i,M}}$  in which  $M < n$ . If it is possible to form at least  $J$ , but not more than  $(J)^L$ , parity-check sums orthogonal on each set  $E_{\alpha_i}$ , then  $\Sigma E_{\alpha_i}$  can be estimated from these  $J$  orthogonal check sums. Note that  $L$  denotes the number of steps or gate levels. In general, the parity-check sums orthogonal on certain properly selected sets of error digits can be formed by summing appropriate sets of received digits. The inputs to a majority-logic gate are these  $J$  orthogonal check sums. Thus, these check sums are fed into at most  $(J)^{L-1}$  of the first-level majority-logic gates. An  $L$ -step majority-logic decoder requires  $L$  levels of majority-logic gates. At the  $i$ th level, no more than  $(J)^{L-i}$  gates are required. Thus, the total number of majority-logic gates needed is upper bounded by  $\sum_{k=0}^{L-1} (J)^k$ . The estimated value of  $\Sigma E_{\alpha_i}$  is the output of a  $J$ -input majority-logic gate, which is 1 only when more than one-half of the

inputs are 1; otherwise it is 0. The estimate is then said to be proper provided that there exist  $\lfloor J/2 \rfloor$  or fewer errors in  $\mathbf{e}$ . Next, the outputs of the first-level majority-logic gates,  $\Sigma E_{\alpha_i}$ ,  $i = 1, 2, \dots$ , are used to form inputs to the second-level majority-logic gates, of which there are at most  $(J)^{L-2}$ . Thus, the  $\Sigma E_{\alpha_i}$ ,  $i = 1, 2, \dots$ , are needed to estimate the sums of error digits in the second selected set,  $E_{\beta_i}$ ,  $1 \leq i < n$ ,  $i = 1, 2, \dots$ , which is listed below:

$$\begin{aligned} E_{\beta_1} &= \{e_{\beta_{1,1}}, e_{\beta_{1,2}}, \dots, e_{\beta_{1,N}}\} \\ E_{\beta_2} &= \{e_{\beta_{2,1}}, e_{\beta_{2,2}}, \dots, e_{\beta_{2,N}}\} \\ &\vdots \end{aligned} \quad (8.33)$$

where the size of the second set  $N$  is smaller than the size  $M$  of the first selected set  $M$ . Assume that it is possible to form  $J$  or more check sums orthogonal on each set  $E_{\beta_i}$ . Then, the sum  $\Sigma E_{\beta_i}$  can be estimated correctly from the check sums orthogonal on  $E_{\beta_i}$  under the condition that there are no more than  $\lfloor J/2 \rfloor$  errors in  $\mathbf{e}$ . The outputs of the second-level majority-logic gates are then used to form inputs to the third-level majority-logic gates, of which there are at least  $(J)^{L-3}$ . Thus, once the second sums  $\Sigma E_{\beta_i}$  are determined, they are used to estimate the sums of error digits in the third selected set  $E_{\gamma_i}$ ,  $i = 1, 2, \dots$ , which has a size that is smaller than that of the second selected set  $N$ . Such an orthogonalization process continues until the last level is reached. There is only one gate at the last level. The  $J$  inputs to this gate are parity-check sums orthogonal on the highest-order error digit  $e_{n-1}$ . The output of this gate is used to correct the received digit  $r_{n-1}$ . Because of the cyclic structure of the code, other error digits can be estimated in the same manner and by the same decoding circuitry. Since majority-logic gates are used to estimate selected sums of error digits at each step of orthogonalization, a total of  $L$  levels of majority-logic gates are required for decoding. Thus, since  $L$  steps of orthogonalization are required to make a decoding decision on an error digit, we say that a code is  $L$ -step majority-logic decodable if  $J$  is one less than the minimum distance of the code. General type I and type II  $L$ -step majority-logic decoders are shown in Figs. 8.7 and 8.8, respectively.

**Example 8.8** Consider the  $(7, 3)$  cyclic code generated by  $g(x) = 1 + x^2 + x^3 + x^4$ . As discussed before, this code is classified as either a maximum-length code or a difference-set code and is a one-step majority decodable code. However, it can also be shown that the  $(7, 3)$  cyclic code is two-step majority-logic decodable. Since this code can be two-step and is completely orthogonalizable, we can decode it with two steps of orthogonalization. Since the minimum distance is

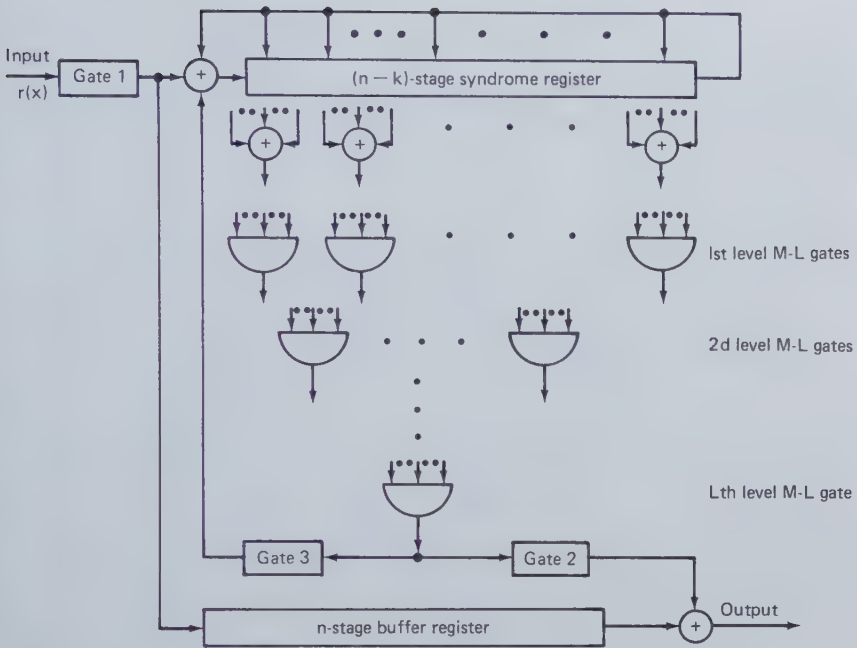


Figure 8.7 General type I  $L$ -step majority-logic decoder.

$d_{\min} = 4$ , the number of check sums is  $J = 3$ . The row vectors of the parity-check matrix  $\mathbf{H}$  are

$$\begin{aligned} \mathbf{h}_0 &= (1 \ 0 \ 0 \ 0 \ 1 \ 1 \ 0) \\ \mathbf{h}_1 &= (0 \ 1 \ 0 \ 0 \ 0 \ 1 \ 1) \\ \mathbf{h}_2 &= (0 \ 0 \ 1 \ 0 \ 1 \ 1 \ 1) \\ \mathbf{h}_3 &= (0 \ 0 \ 0 \ 1 \ 1 \ 0 \ 1) \end{aligned}$$

We see that the vectors  $\mathbf{h}_1$ ,  $\mathbf{h}_2$ , and  $\mathbf{h}_0 + \mathbf{h}_3$  are orthogonal on digit positions 5 and 6; and that the vectors  $\mathbf{h}_0 + \mathbf{h}_1$ ,  $\mathbf{h}_2$ , and  $\mathbf{h}_3$  are orthogonal on digit positions 4 and 6. Let  $E_{\alpha_1} = \{e_5, e_6\}$  and  $E_{\alpha_2} = \{e_4, e_6\}$  be two selected sets. Then, the parity-check sums formed from the orthogonal vectors  $\mathbf{h}_1$ ,  $\mathbf{h}_2$ , and  $\mathbf{h}_0 + \mathbf{h}_3$  and the received vector  $\mathbf{r}$  are

$$\begin{aligned} \xi_{1,1} &= \mathbf{r} \cdot \mathbf{h}_1 = e_1 + e_5 + e_6 \\ \xi_{1,2} &= \mathbf{r} \cdot \mathbf{h}_2 = e_2 + e_4 + e_5 + e_6 \\ \xi_{1,3} &= \mathbf{r} \cdot (\mathbf{h}_0 + \mathbf{h}_3) = e_0 + e_3 + e_5 + e_6 \end{aligned}$$

These check sums,  $\xi_{1,1}$ ,  $\xi_{1,2}$ , and  $\xi_{1,3}$  are orthogonal on the set  $E_{\alpha_1} = \{e_5, e_6\}$ . Next, the parity-check sums formed from  $\mathbf{h}_0 + \mathbf{h}_1$ ,  $\mathbf{h}_2$ , and  $\mathbf{h}_3$  and  $\mathbf{r}$  are

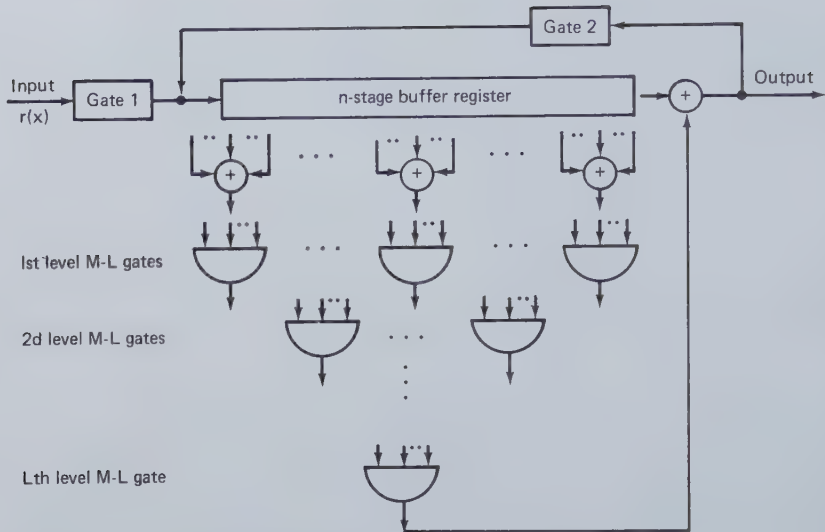


Figure 8.8 General type II  $L$ -step majority-logic decoder.

$$\begin{aligned}\xi_{2,1} &= \mathbf{r} \cdot (\mathbf{h}_0 + \mathbf{h}_1) = e_0 + e_1 && + e_4 && + e_6 \\ \xi_{2,2} &= \mathbf{r} \cdot \mathbf{h}_2 && = e_2 && + e_4 + e_5 + e_6 \\ \xi_{2,3} &= \mathbf{r} \cdot \mathbf{h}_3 && = e_3 + e_4 && + e_6\end{aligned}$$

As you see, the parity-check sums  $\xi_{2,1}$ ,  $\xi_{2,2}$ , and  $\xi_{2,3}$  are orthogonal on the set  $E_{\alpha_2} = \{e_4, e_6\}$ . Thus, the sum  $\Sigma E_{\alpha_1} = e_5 + e_6$  is estimated from the check sums  $\xi_{1,1}$ ,  $\xi_{1,2}$ , and  $\xi_{1,3}$ ; and the sum  $\Sigma E_{\alpha_2} = e_4 + e_6$  can be estimated from the parity-check sums  $\xi_{2,1}$ ,  $\xi_{2,2}$ , and  $\xi_{2,3}$ . If we let  $E_{\beta_1} = \{e_6\}$ , the sums  $\Sigma E_{\alpha_1}$  and  $\Sigma E_{\alpha_2}$  are orthogonal on  $e_6$ . Therefore, the error digit  $e_6$  (if it is assumed to be erroneous) can be estimated from  $\Sigma E_{\alpha_1}$  and  $\Sigma E_{\alpha_2}$ . Thus, the value of  $e_6$  will be correctly estimated provided that no more than one error exists in  $\mathbf{e}$ . A type II two-step majority-logic decoder for this code is shown in Fig. 8.9.

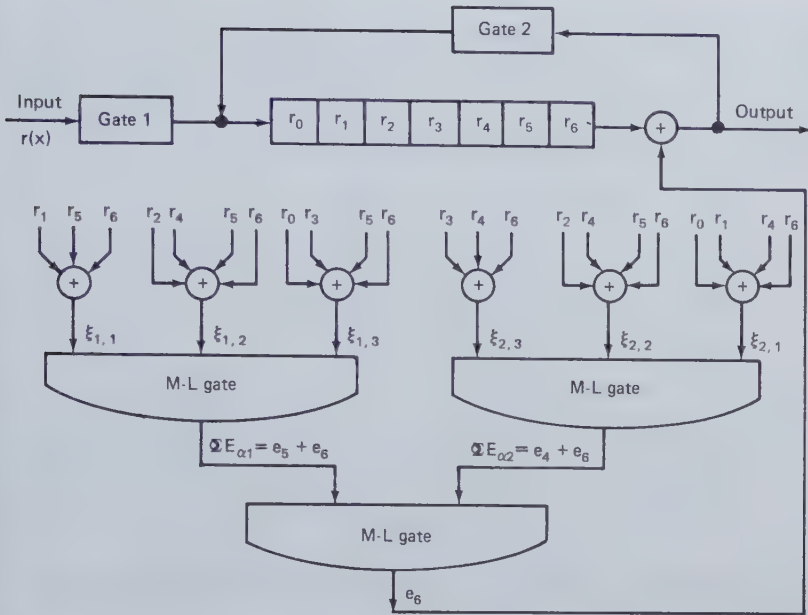
Since the syndrome  $\mathbf{s} = (s_0, s_1, s_2, s_3)$  is the inner product of  $\mathbf{r}$  and  $\mathbf{H}^T$ , the parity-check sums  $\xi_{i,j}$  for  $i = 1, 2$  and  $j = 1, 2, 3$  can be formed as follows:

$$\begin{aligned}\xi_{1,1} &= s_1 && \xi_{1,2} = s_2 && \xi_{1,3} = s_0 + s_3 \\ \xi_{2,1} &= s_0 + s_1 && \xi_{2,2} = s_2 && \xi_{2,3} = s_3\end{aligned}$$

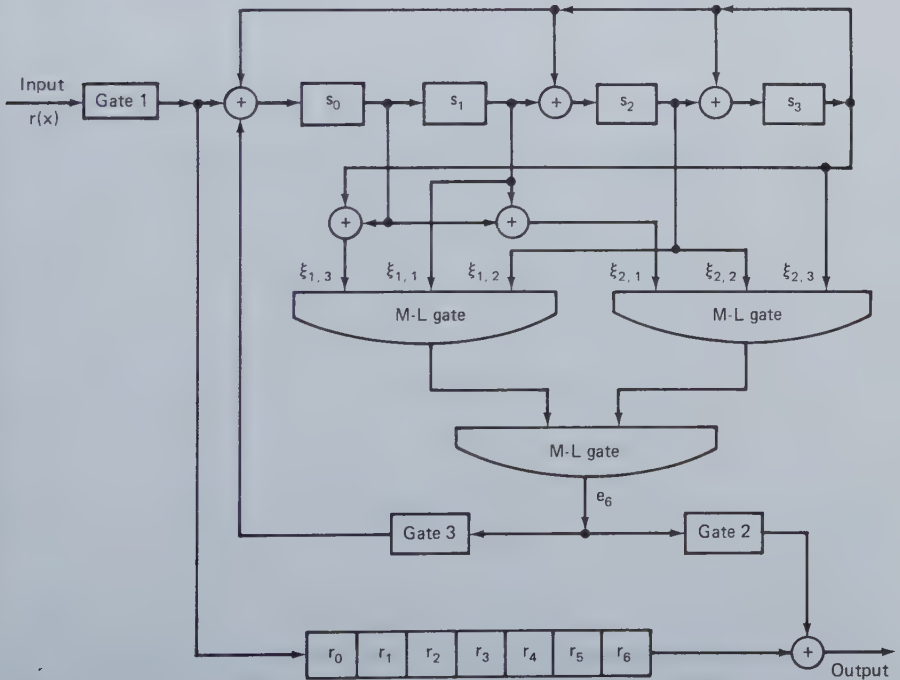
Based on these parity-check sums, we can construct a type I majority-logic decoder for the  $(7, 3)$  cyclic code as shown in Fig. 8.10.

Thus, referring to the number of levels of majority gates, a two-step majority-logic decoder for this  $(7, 3)$  code has the structure shown in Figs. 8.9 and 8.10. Of course, a simpler decoder for this particular code does exist, as shown in Figs. 8.2 and 8.4, but this example simply serves to illustrate the concept of  $L$ -step majority decoding.





**Figure 8.9** A type II two-step majority-logic decoder for the (7, 3) cyclic code.



**Figure 8.10** A type I two-step majority-logic decoder for the (7, 3) cyclic code.

**Example 8.9** Consider the (15, 7) BCH code whose generator polynomial is  $g(x) = 1 + x^4 + x^6 + x^7 + x^8$ . Since the minimum distance of this code is  $d_{\min} = 5$ , there exist four check sums ( $J = 4$ ). Hence, this code is capable of correcting  $\lfloor J/2 \rfloor = 2$  or fewer errors with two-step majority-logic decoding. The  $\mathbf{h}$  row-vectors from the parity-check matrix  $\mathbf{H}$  are expressible as

$$\mathbf{h}_0 = (1 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 1 \ 0 \ 1 \ 0 \ 0 \ 0)$$

$$\mathbf{h}_1 = (0 \ 1 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 1 \ 0 \ 1 \ 0 \ 0)$$

$$\mathbf{h}_2 = (0 \ 0 \ 1 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 1 \ 0 \ 1 \ 0)$$

$$\mathbf{h}_3 = (0 \ 0 \ 0 \ 1 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 1 \ 0 \ 1)$$

$$\mathbf{h}_4 = (0 \ 0 \ 0 \ 0 \ 1 \ 0 \ 0 \ 0 \ 1 \ 1 \ 0 \ 1 \ 1 \ 1 \ 0)$$

$$\mathbf{h}_5 = (0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 0 \ 0 \ 0 \ 1 \ 1 \ 0 \ 1 \ 1 \ 1)$$

$$\mathbf{h}_6 = (0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 0 \ 1 \ 1 \ 1 \ 0 \ 0 \ 1 \ 1)$$

$$\mathbf{h}_7 = (0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 1 \ 0 \ 1 \ 0 \ 0 \ 0 \ 1)$$

By choosing the row vectors of  $\mathbf{H}$  properly and making linear combinations of them, we have

$$\Phi_1 = \mathbf{h}_3 = (0 \ 0 \ 0 \ 1 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 1 \ 0 \ 1)$$

$$\Phi_2 = \mathbf{h}_1 + \mathbf{h}_5 = (0 \ 1 \ 0 \ 0 \ 0 \ 1 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 1)$$

$$\Phi_3 = \mathbf{h}_0 + \mathbf{h}_2 + \mathbf{h}_6 = (1 \ 0 \ 1 \ 0 \ 0 \ 0 \ 1 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1)$$

$$\Phi_4 = \mathbf{h}_7 = (0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 1 \ 0 \ 1 \ 0 \ 0 \ 0 \ 1)$$

Let  $\mathbf{r} = (r_0, r_1, r_2, \dots, r_{14}, r_{15})$  be the received vector. Therefore, the inner product of  $\Phi_i$  and  $\mathbf{r}$  produce the parity-check sums on the error digit  $e_{14}$  as shown below.

$$\xi_1 = \mathbf{r} \cdot \Phi_1 = e_3 + e_{11} + e_{12} + e_{14}$$

$$\xi_2 = \mathbf{r} \cdot \Phi_2 = e_1 + e_5 + e_{13} + e_{14}$$

$$\xi_3 = \mathbf{r} \cdot \Phi_3 = e_0 + e_2 + e_6 + e_{14}$$

$$\xi_4 = \mathbf{r} \cdot \Phi_4 = e_7 + e_8 + e_{10} + e_{14}$$

Let

$$E_{\alpha_1} = \{e_{11}, e_{12}, e_{14}\}$$

$$E_{\alpha_2} = \{e_8, e_{10}, e_{14}\}$$

$$E_{\alpha_3} = \{e_4, e_9, e_{14}\}$$

$$E_{\alpha_4} = \{e_1, e_{13}, e_{14}\}$$

be four selected sets of error digits. Now we must find the four parity-check sums orthogonal on each of the  $E_{\alpha_i}$  sets listed above. After making use of the following relations:

$$\mathbf{r} \cdot \mathbf{h}_0 = e_0 + e_8 + e_9 + e_{11}$$

$$\mathbf{r} \cdot \mathbf{h}_1 = e_1 + e_9 + e_{10} + e_{12}$$

$$\mathbf{r} \cdot \mathbf{h}_2 = e_2 + e_{10} + e_{11} + e_{13}$$

$$\mathbf{r} \cdot \mathbf{h}_3 = e_3 + e_{11} + e_{12} + e_{14}$$

$$\mathbf{r} \cdot \mathbf{h}_4 = e_4 + e_8 + e_9 + e_{11} + e_{12} + e_{13}$$

$$\mathbf{r} \cdot \mathbf{h}_5 = e_5 + e_9 + e_{10} + e_{12} + e_{13} + e_{14}$$

$$\mathbf{r} \cdot \mathbf{h}_6 = e_6 + e_8 + e_9 + e_{10} + e_{13} + e_{14}$$

$$\mathbf{r} \cdot \mathbf{h}_7 = e_7 + e_8 + e_{10} + e_{14}$$

we still have to find the parity-check sums orthogonal on the four selected sets of error digits, namely,  $E_{\alpha_1}$ ,  $E_{\alpha_2}$ ,  $E_{\alpha_3}$ , and  $E_{\alpha_4}$ .

1. The parity-check sums orthogonal on  $E_{\alpha_1} = \{e_{11}, e_{12}, e_{14}\}$  are

$$\xi_{1,1} = \mathbf{r} \cdot \mathbf{h}_3 = e_3 + e_{11} + e_{12} + e_{14}$$

$$\xi_{1,2} = \mathbf{r} \cdot (\mathbf{h}_2 + \mathbf{h}_5) = e_2 + e_5 + e_9 + e_{11} + e_{12} + e_{14}$$

$$\xi_{1,3} = \mathbf{r} \cdot (\mathbf{h}_4 + \mathbf{h}_6) = e_4 + e_6 + e_{10} + e_{11} + e_{12} + e_{14}$$

$$\xi_{1,4} = \mathbf{r} \cdot (\mathbf{h}_0 + \mathbf{h}_1 + \mathbf{h}_7) = e_0 + e_1 + e_7 + e_{11} + e_{12} + e_{14}$$

2. The parity-check sums orthogonal on  $E_{\alpha_2} = \{e_8, e_{10}, e_{14}\}$  are

$$\xi_{2,1} = \mathbf{r} \cdot \mathbf{h}_0 = e_6 + e_8 + e_9 + e_{10} + e_{13} + e_{14}$$

$$\xi_{2,2} = \mathbf{r} \cdot \mathbf{h}_7 = e_7 + e_8 + e_{10} + e_{14}$$

$$\xi_{2,3} = \mathbf{r} \cdot (\mathbf{h}_0 + \mathbf{h}_5) = e_4 + e_5 + e_8 + e_{10} + e_{11} + e_{14}$$

$$\xi_{2,4} = \mathbf{r} \cdot (\mathbf{h}_0 + \mathbf{h}_1 + \mathbf{h}_3) = e_0 + e_1 + e_3 + e_8 + e_{10} + e_{14}$$

3. The parity-check sums orthogonal on  $E_{\alpha_3} = \{e_4, e_9, e_{14}\}$  are

$$\xi_{3,1} = \mathbf{r} \cdot (\mathbf{h}_3 + \mathbf{h}_4) = e_3 + e_4 + e_8 + e_9 + e_{13} + e_{14}$$

$$\xi_{3,2} = \mathbf{r} \cdot (\mathbf{h}_1 + \mathbf{h}_4 + \mathbf{h}_6) = e_1 + e_4 + e_6 + e_9 + e_{11} + e_{14}$$

$$\xi_{3,3} = \mathbf{r} \cdot (\mathbf{h}_0 + \mathbf{h}_4 + \mathbf{h}_5) = e_0 + e_4 + e_5 + e_9 + e_{10} + e_{14}$$

$$\xi_{3,4} = \mathbf{r} \cdot (\mathbf{h}_2 + \mathbf{h}_4 + \mathbf{h}_7) = e_2 + e_4 + e_7 + e_9 + e_{12} + e_{14}$$

4. The parity-check sums orthogonal on  $E_{\alpha_4} = \{e_1, e_{13}, e_{14}\}$  are

$$\xi_{4,1} = \mathbf{r} \cdot (\mathbf{h}_1 + \mathbf{h}_5) = e_1 + e_5 + e_{13} + e_{14}$$

$$\xi_{4,2} = \mathbf{r} \cdot (\mathbf{h}_1 + \mathbf{h}_6) = e_1 + e_6 + e_8 + e_{12} + e_{13} + e_{14}$$

$$\xi_{4,3} = \mathbf{r} \cdot (\mathbf{h}_1 + \mathbf{h}_2 + \mathbf{h}_3) = e_1 + e_2 + e_3 + e_9 + e_{13} + e_{14}$$

$$\xi_{4,4} = \mathbf{r} \cdot (\mathbf{h}_1 + \mathbf{h}_4 + \mathbf{h}_7) = e_1 + e_4 + e_7 + e_{11} + e_{13} + e_{14}$$

Observing the error sums  $\Sigma E_{\alpha_1} = e_{11} + e_{12} + e_{14}$ ,  $\Sigma E_{\alpha_2} = e_8 + e_{10} + e_{14}$ ,  $\Sigma E_{\alpha_3} = e_4 + e_9 + e_{14}$ , and  $\Sigma E_{\alpha_4} = e_1 + e_{13} + e_{14}$ , we can easily see that these sums are orthogonal on  $e_{14}$  and that no other error digit is checked by more than one check sum. Therefore,  $e_{14}$  can be estimated from these sums only when there are no more than two errors in the error vector  $\mathbf{e}$ .

Thus, the (15, 7) BCH code is a two-step majority-logic decodable code. Since  $J = 4$  and  $L = 2$ , a total of  $1 + J = 5$  majority-logic gates are required. A type II two-step majority-logic decoder for this code is shown in Fig. 8.11. Analysis and construction of a type I majority-logic decoder for the (15, 7) BCH code is not a difficult job at all. Hence, it is left to the reader as an exercise.

Majority-logic decoding is often useful in very-high-speed applications where a moderate amount of coding gain is desired. As discussed in Chap. 11, some convolutional codes can also be decoded by a majority-logic decoder. Like cyclic block codes, these convolutional decoders are simple and extremely fast, but a code with a higher rate is usually preferred. In the following sections, we discuss two more classes of cyclic codes that can be  $L$ -step decoded and that are well known. They are the euclidean geometry codes (EG codes) and projective geometry codes (PG codes).

## 8.4 Euclidean Geometry Codes

A finite geometry consists of a set containing an infinite number of points together with subsets known as lines and planes, or flats. The most meaningful finite geometries are euclidean geometries and projective geometries. Using the theory of finite geometries, we can invent two classes of codes, i.e., EG codes and PG codes, that are majority-logic decodable. This section discusses EG codes, and the next section discusses PG codes.

A euclidean geometry, denoted by  $EG(m, 2^s)$ , of dimension  $m$  over  $GF(2^s)$  consists of  $2^{ms}$  points together with certain subsets called flats. We usually call the 0-flats the points, the 1-flats the lines, and the 2-flats the planes of  $EG(m, 2^s)$ . The Reed-Muller codes presented in Sec. 3.7 have an interesting geometric interpretation. The  $2^m$  digits can be uniquely associated with  $2^m$  points in  $EG(m, 2)$ , which designates an  $m$ -dimensional euclidean geometry over  $GF(2)$ . The number of  $r$ -flats in  $EG(m, 2^s)$  is quite large.  $EG(m, 2)$  for  $s = 1$  contains

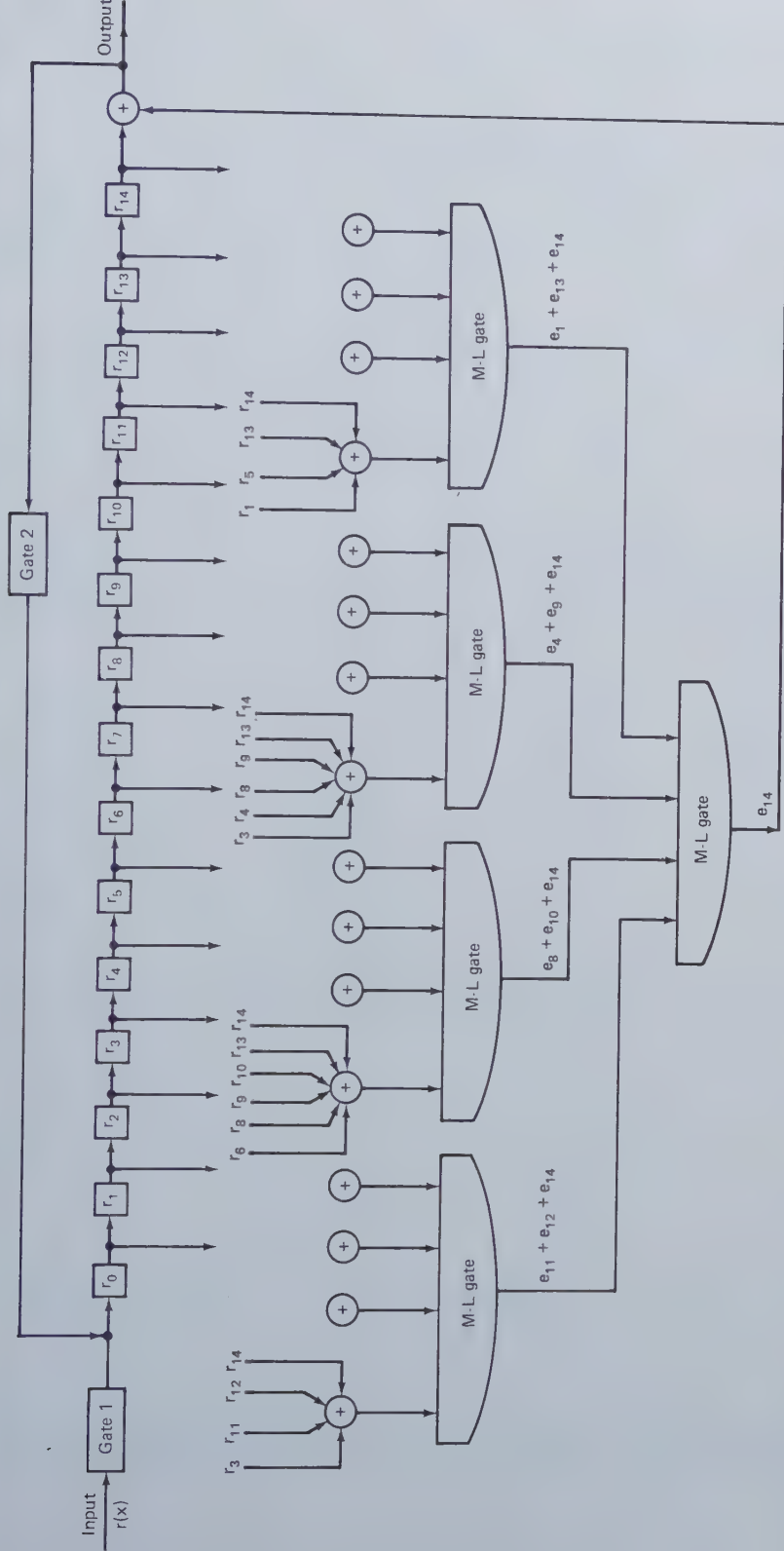
$$2^{m-r} \begin{bmatrix} m \\ r \end{bmatrix}$$

distinct  $r$ -flats

where

$$\begin{bmatrix} m \\ r \end{bmatrix} = \prod_{j=0}^{r-1} \left( \frac{2^m - 2^j}{2^r - 2^j} \right) \quad \text{for } r = 0, 1, \dots, m$$

**Example 8.10** Consider the euclidean geometry  $EG(3, 2)$  for  $m = 3$  and  $s = 1$ . The number of 0-flats is



**Figure 8.11** A type II two-step majority-logic decoder for the (15, 7) BCH code.



$$2^3 \begin{bmatrix} 3 \\ 0 \end{bmatrix} = (8)(1) = 8$$

The number of 1-flats is

$$2^2 \begin{bmatrix} 3 \\ 1 \end{bmatrix} = (4)(7) = 28$$

And the number of 2-flats is

$$2 \begin{bmatrix} 2 \\ 3 \end{bmatrix} = (2) \left( \frac{7}{3} \right) \left( \frac{6}{2} \right) = 14$$

Thus, there are eight points, 28 lines, and 14 planes as shown in Table 8.3.

**TABLE 8.3** *r*-Flats, *r*=0, 1, 2, in the Euclidean Geometry *EG*(3, 2) over *GF*(2)

| 0-flats | 1-flats  | 2-flats            |
|---------|----------|--------------------|
| 000     | 000, 001 | 000, 001, 010, 011 |
| 001     | 000, 010 | 000, 001, 100, 101 |
| 010     | 000, 011 | 000, 001, 110, 111 |
| 011     | 000, 100 | 000, 010, 100, 110 |
| 100     | 000, 101 | 000, 010, 101, 111 |
| 101     | 000, 110 | 000, 011, 100, 111 |
| 110     | 000, 111 | 000, 011, 101, 110 |
| 111     | 001, 010 | 001, 011, 101, 111 |
|         | 001, 011 | 001, 011, 100, 110 |
|         | 001, 100 | 001, 010, 101, 110 |
|         | 001, 101 | 001, 010, 100, 111 |
|         | 001, 110 | 010, 011, 110, 111 |
|         | 001, 111 | 010, 011, 100, 101 |
|         | 010, 011 | 100, 101, 110, 111 |
|         | 010, 100 |                    |
|         | 010, 101 |                    |
|         | 010, 110 |                    |
|         | 010, 111 |                    |
|         | 011, 100 |                    |
|         | 011, 101 |                    |
|         | 011, 110 |                    |
|         | 011, 111 |                    |
|         | 100, 101 |                    |
|         | 100, 110 |                    |
|         | 100, 111 |                    |
|         | 101, 110 |                    |
|         | 101, 111 |                    |
|         | 110, 111 |                    |

In general, the number of *r*-flats in a euclidean geometry *EG*(*m*, 2<sup>*s*</sup>) is expressible as

$$2^{(m-r)s} \begin{bmatrix} m \\ r \end{bmatrix} = 2^{(m-r)s} \prod_{j=0}^{r-1} \left( \frac{2^{ms} - 2^{js}}{2^{rs} - 2^{js}} \right) \quad (8.34)$$

Let  $a_0, a_1, \dots, a_r$  be  $r+1$  0-flats in  $EG(m, 2^s)$ , where  $r < m$ . The  $2^{rs}$  points of the form  $a_0 + \sum_{j=1}^r \mu_j a_j$  with  $\mu_j \in GF(2^s)$  constitute  $r$ -flats passing through the point  $a_0$  in  $EG(m, 2^s)$ .

**Example 8.11** Consider again the euclidean geometry  $EG(3, 2)$  over  $GF(2)$  given in Example 8.10. Since  $m = 3$ , all 3-tuples, such as  $a_i = (a_{i,0}, a_{i,1}, a_{i,2})$  for  $0 \leq i \leq 7$ , are the points in  $EG(3, 2)$ , which are shown in the first column (0-flats) of Table 8.3. They are

$$\begin{aligned} a_0 &= (0 \ 0 \ 0) & a_1 &= (0 \ 0 \ 1) & a_2 &= (0 \ 1 \ 0) & a_3 &= (0 \ 1 \ 1) \\ a_4 &= (1 \ 0 \ 0) & a_5 &= (1 \ 0 \ 1) & a_6 &= (1 \ 1 \ 0) & a_7 &= (1 \ 1 \ 1) \end{aligned}$$

If we use these points or 0-flats, the 2-flats (or planes) expressed in the third column are easily computed from these 3-tuple linearly independent points and have the form  $a_i + \mu_1 a_j + \mu_2 a_k$ , where  $\mu_1, \mu_2 \in GF(2)$ .

| $\mu_1$ | $\mu_2$ | $a_0 + \mu_1 a_1 + \mu_2 a_2$ |         |
|---------|---------|-------------------------------|---------|
| 0       | 0       | $a_0$                         | (0 0 0) |
| 0       | 1       | $a_0 + a_2 = a_2$             | (0 1 0) |
| 1       | 0       | $a_0 + a_1 = a_1$             | (0 0 1) |
| 1       | 1       | $a_0 + a_1 + a_2 = a_3$       | (0 1 1) |

Thus, the vector set of this 2-flat is

$$\{a_0, a_1, a_2, a_3\} = \{0 \ 0 \ 0, 0 \ 0 \ 1, 0 \ 1 \ 0, 0 \ 1 \ 1\}$$

which is the first position of 2-flats in the third column.

If we choose linearly independent 3-tuple points  $a_1, a_2$ , and  $a_4$ , the form for another 2-flat is  $a_1 + \mu_1 a_2 + \mu_2 a_4$ , where  $\mu_1, \mu_2 \in GF(2)$ .

| $\mu_1$ | $\mu_2$ | $a_1 + \mu_1 a_2 + \mu_2 a_4$ |         |
|---------|---------|-------------------------------|---------|
| 0       | 0       | $a_1$                         | (0 0 1) |
| 0       | 1       | $a_1 + a_4 = a_5$             | (1 0 1) |
| 1       | 0       | $a_1 + a_2 = a_3$             | (0 1 1) |
| 1       | 1       | $a_1 + a_2 + a_4 = a_7$       | (1 1 1) |

Thus, this is another 2-flat,  $\{a_1, a_3, a_5, a_7\} = \{0 \ 0 \ 1, 0 \ 1 \ 1, 1 \ 0 \ 1, 1 \ 1 \ 1\}$ , which is the eighth position in the third column. Likewise, by doing so we can obtain all the 2-flats shown in the third column of Table 8.3.

Let  $\alpha$  be a primitive element of the Galois field  $GF(2^{ms})$ . Then the  $2^{ms}$  elements in  $GF(2^{ms})$  are expressed as powers of  $\alpha$  such that  $\alpha^\infty = 0$ ,  $\alpha^0 = 1$ ,  $\alpha^1, \alpha^2, \dots, \alpha^{2^{ms}-2}$ . These  $2^{ms}$  elements in  $GF(2^{ms})$  are regarded as the  $2^{ms}$  points in  $EG(m, 2^s)$ . Thus, we treat  $GF(2^{ms})$  as  $EG(m, 2^s)$ ,

which is the  $m$ -dimensional euclidean geometry over  $GF(2^s)$ . If  $GF(2^s)$  is a subfield of  $GF(2^{ms})$ , every element of  $GF(2^{ms})$  is then formed as follows:

$$\alpha^i = \sum_{j=0}^{m-1} \alpha_{i,j} \alpha^j \quad \alpha_{i,j} \in GF(2^s) \quad (8.35)$$

Also, an  $r$ -flat passing through the point  $\alpha^{\lambda_0}$  consists of  $2^{rs}$  points and is expressed as

$$\alpha^{\lambda_0} + \sum_{i=1}^r \mu_i \alpha^{\lambda_i} \quad \mu_i \in GF(2^s) \quad (8.36)$$

where  $\alpha^{\lambda_i}$ ,  $0 \leq i \leq r$ , are  $r+1$  linearly independent elements in  $GF(2^{ms})$ .

Let  $F^{(r)}$  denote an  $r$ -flat in  $EG(m, 2^s)$  that does not pass through the origin  $\alpha^\infty = 0$ . One can then form a vector of  $2^{ms} - 1$  tuples as shown below:

$$\mathbf{v}_{F^{(r)}} = (v_0, v_1, v_2, \dots, v_{2^{ms}-2})$$

If the components  $v_i$  of  $\mathbf{v}_{F^{(r)}}$  are numbered with the nonzero elements  $\alpha^i$ ,  $0 \leq i \leq 2^{ms} - 2$ , of  $GF(2^{ms})$ , the  $\alpha^i$  represent the location numbers of  $v_i$ . Thus, the location numbers for  $v_i$ ,  $0 \leq i \leq 2^{ms} - 2$ , of  $\mathbf{v}_{F^{(r)}}$  will form the points of the  $r$ -flat  $F^{(r)}$ . Thus,  $\mathbf{v}_{F^{(r)}}$  is usually called the incidence vector of  $F^{(r)}$ .

**Example 8.12** Consider the euclidean geometry  $EG(2, 2^2)$  over  $GF(2^2)$ . The Galois field  $GF(2^4)$  may be regarded as  $EG(2, 2^2)$  over  $GF(2^2)$ . Therefore,  $GF(2^2)$  is a subfield of  $GF(2^4)$ . Setting  $\beta = \alpha^5$ , we see that the elements of subfield  $GF(2^2)$  are 0, 1,  $\beta$ ,  $\beta^2$  because the order of  $\beta$  is 3. A 1-flat (or line) passing through the point  $\alpha^{13}$  is formed from the points  $\alpha^{\lambda_0} + \mu_s \alpha^{\lambda_s}$ , where  $\lambda_0 = 13$ ,  $1 \leq \lambda_s \leq 5$ , and  $\mu_s$  represents each element of  $GF(2^2)$ . Thus, the points for  $\lambda_s = 1$  are

$$\begin{aligned} \alpha^{13} + 0 \cdot \alpha &= \alpha^{13} & \alpha^{13} + 1 \cdot \alpha &= \alpha^{12} \\ \alpha^{13} + \beta \cdot \alpha &= 1 & \alpha^{13} + \beta^2 \cdot \alpha &= \alpha^4 \end{aligned}$$

Hence, the line passing through  $\alpha^{13}$  is formed as follows:

$$\{\alpha^{13}, \alpha^{12}, 1, \alpha^4\}$$

Similarly, for  $\lambda_s = 2, 3, 4$ , and 5 the other four lines in  $EG(2, 2^2)$  passing through  $\alpha^{13}$  are

$$\begin{aligned} \{\alpha^{13} + \mu_s \alpha^2\} &= \{\alpha^{13}, \alpha^{14}, \alpha^5, \alpha\}, & \{\alpha^{13} + \mu_s \alpha^3\} &= \{\alpha^{13}, \alpha^8, \alpha^3, 0\} \\ \{\alpha^{13} + \mu_s \alpha^4\} &= \{\alpha^{13}, \alpha^{11}, \alpha^{10}, \alpha^2\}, & \{\alpha^{13} + \mu_s \alpha^5\} &= \{\alpha^{13}, \alpha^7, \alpha^9, \alpha^6\} \end{aligned}$$

Thus, the four 1-flats (lines) passing through the point  $\alpha^{13}$  but not the origin are

$$L_1 = \{\alpha^{13}, \alpha^{12}, 1, \alpha^4\} \quad L_2 = \{\alpha^{13}, \alpha^{14}, \alpha^5, \alpha\}$$

$$L_3 = \{\alpha^{13}, \alpha^{11}, \alpha^{10}, \alpha^2\} \quad L_4 = \{\alpha^{13}, \alpha^7, \alpha^9, \alpha^6\}$$

Hence, the incidence vectors for these four lines are

| $\alpha^i$       | 0  | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 |
|------------------|----|---|---|---|---|---|---|---|---|---|----|----|----|----|----|
| $\mathbf{v}_1 =$ | (1 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0  | 0  | 1  | 1  | 0) |
| $\mathbf{v}_2 =$ | (0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0  | 0  | 0  | 1  | 1) |
| $\mathbf{v}_3 =$ | (0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1  | 1  | 0  | 1  | 0) |
| $\mathbf{v}_4 =$ | (0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 0 | 1 | 0  | 0  | 0  | 1  | 0) |

Let  $\theta$  be a nonnegative integer that is less than  $2^{ms}$  with the radix- $2^s$  representation:

$$\theta = \sum_{i=0}^{m-1} \rho_i 2^{is} = \rho_0 + \rho_1 2^s + \rho_2 2^{2s} + \cdots + \rho_{m-1} 2^{(m-1)s} \quad (8.37)$$

where  $0 \leq \rho_i < 2^s$  for  $0 \leq i < m$ . Then the radix- $2^s$  weight of  $\theta$  is given with addition as shown below:

$$W_{2^s}(\theta) \triangleq \sum_{i=0}^{m-1} \rho_i = \rho_0 + \rho_1 + \rho_2 + \cdots + \rho_{m-1} \quad (8.38)$$

If  $\theta_l$  is the remainder that results from dividing  $2^l \theta$  ( $0 \leq l$ ) by  $2^{ms} - 1$ , we have

$$2^l \theta = q(2^{ms} - 1) + \theta_l \quad 0 \leq l \quad (8.39)$$

where  $0 \leq \theta_l < 2^{ms} - 1$ . From Eq. (8.39), we see that  $\theta_l$  is clearly divisible by  $2^s - 1$  if and only if  $\theta$  is divisible by  $2^s - 1$ . A euclidean geometry code over  $GF(2)$  with parameters  $r$  and  $s$  and block length  $2^{ms} - 1$  is a generalization of the Reed-Muller code of order  $r$  over  $GF(2^s)$ .

Let  $\alpha$  be a primitive element of  $GF(2^{ms})$ . The generator polynomial  $g(x)$  of the  $(r, s)$ th-order EG code of length  $2^{ms} - 1$  has roots at all  $\alpha^\theta$ ,  $\theta = 1, 2, \dots, 2^{ms} - 2$ , such that

$$0 < \max_{0 \leq l < s} W_{2^s}(2^l \theta) \leq (m - r - 1)(2^s - 1) \quad \text{for } 0 \leq 2^l \theta < 2^{ms} - 1 \quad (8.40)$$

$$\text{and } 0 < \max_{1 \leq l < s} W_{2^s}(\theta_l) \leq (m - r - 1)(2^s - 1) \quad \text{for } 2^{ms} - 1 \leq 2^l \theta \quad (8.41)$$

This is a rule that characterizes the roots of the generator polynomial  $g(x)$  of a  $(r, s)$ th-order EG code.

**Example 8.13** Consider the  $(0, 2)$ th-order EG code of length  $n = 2^{ms} - 1 = 15$ . Then we have  $r = 0$ ,  $m = 2$ , and  $s = 2$ . Thus,  $GF(2^{ms}) = GF(2^4)$  could be regarded as the two-dimensional euclidean geometry  $EG(2, 2^2)$  over  $GF(2^2)$ . Let  $\alpha$  be a primitive element in  $GF(2^4)$  and  $\theta$  be a nonnegative integer that is less than  $(2^{ms} - 1) = 15$ . Thus, from the rules of Eqs. (8.40) and (8.41), the generator polynomial  $g(x)$  of this EG code has  $\alpha^\theta$  as a root if and only if

$$0 < \max_{0 \leq l < 2} W_4(2^l \theta) \leq 3 \quad 0 \leq 2^l \theta < 15$$

$$\text{and} \quad 0 < \max_{1 \leq l < 2} W_4(\theta_l) \leq 3 \quad 15 \leq 2^l \theta$$

Accordingly, we obtain the following results.

1. When  $l = 0$ :

| $2^l \theta$ | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 |
|--------------|---|---|---|---|---|---|---|---|---|---|----|----|----|----|----|
| $W_4$        | 0 | 1 | 2 | 3 | 1 | 2 | 3 | 4 | 2 | 3 | 4  | 5  | 3  | 4  | 5  |

2. When  $l = 1$ :

| $2^l \theta$ | 0 | 2 | 4 | 6 | 8 | 10 | 12 | 14 |   |   |   |   |   |    |    |
|--------------|---|---|---|---|---|----|----|----|---|---|---|---|---|----|----|
| $\theta_l$   |   |   |   |   |   |    |    |    | 1 | 3 | 5 | 7 | 9 | 11 | 13 |
| $W_4$        | 0 | 2 | 1 | 3 | 2 | 4  | 3  | 5  | 1 | 3 | 2 | 4 | 3 | 5  | 4  |

| $\theta$   | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 |
|------------|---|---|---|---|---|---|---|---|---|---|----|----|----|----|----|
| $\max W_4$ | 0 | 2 | 2 | 3 | 2 | 4 | 3 | 5 | 2 | 3 | 4  | 5  | 3  | 5  | 5  |

Thus, the  $\theta$  that satisfy the rule above are 1, 2, 3, 4, 6, 8, 9, and 12. Consequently, the roots of  $g(x)$  are  $\alpha, \alpha^2, \alpha^3, \alpha^4, \alpha^6, \alpha^8, \alpha^9$ , and  $\alpha^{12}$ . Therefore, the generator polynomial of the  $(0, 2)$ th-order EG code of length 15 is

$$\begin{aligned} g(x) &= (x + \alpha)(x + \alpha^2)(x + \alpha^3)(x + \alpha^4)(x + \alpha^6)(x + \alpha^8)(x + \alpha^9)(x + \alpha^{12}) \\ &= m_1(x)m_3(x) \\ &= 1 + x^4 + x^6 + x^7 + x^8 \end{aligned}$$

This is equal to the generator polynomial of the  $(15, 7)$  BCH code. Therefore, the  $(0, 2)$ th-order EG code is one-step majority decodable.

**Example 8.14** Let  $m = 4$ ,  $s = 1$ , and  $r = 1$ . Then we regard  $GF(2^4)$  as the four-dimensional euclidean geometry  $EG(4, 2)$  over  $GF(2)$ . Therefore, this is the  $(1, 1)$ th-order EG code of length  $2^4 - 1 = 15$ . Let  $\alpha$  be a primitive element of  $GF(2^4)$ . For  $\theta < 15$ , the generator polynomial  $g(x)$  of this code has  $\alpha^\theta$  as a root if and only if

$$0 < \max_{0 \leq l < 1} W_2(2^l \theta) \leq (4 - 1 - 1)(2 - 1)$$



or

$$0 < W_2(\theta) \leq 2$$

where  $\theta = \rho_0 + 2\rho_1 + 4\rho_2 + 8\rho_3$ . For  $\alpha^0$  to be a root of  $g(x)$  when written in the radix-2 expansion,  $\theta$  must be the sum of at least  $r + 1 = 2$  multiples of  $2^s - 1 = 1$ . Thus, we can easily compute the  $\theta$  and  $W_2$ , which are shown in the tabulation below.

| $\theta$ | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 |
|----------|---|---|---|---|---|---|---|---|---|---|----|----|----|----|----|
| $W_2$    | 0 | 1 | 1 | 2 | 1 | 2 | 2 | 3 | 1 | 2 | 2  | 3  | 2  | 3  | 3  |

The nonnegative integers  $\theta$ s of less than 16 that satisfy the rule above are

$$1, 2, 3, 4, 5, 6, 8, 9, 10, 12$$

Therefore,  $g(x)$  has the following roots:

$$\alpha, \alpha^2, \alpha^3, \alpha^4, \alpha^5, \alpha^6, \alpha^8, \alpha^9, \alpha^{10}, \alpha^{12}$$

From the roots  $\alpha^0$  of  $g(x)$ , the minimal polynomials  $m_i(x)$ ,  $i = 1, 3, 5$ , can be computed as

$$m_1(x) = 1 + x + x^4 \quad \text{for } \alpha \text{ and its conjugates } \alpha^2, \alpha^4, \text{ and } \alpha^8$$

$$m_3(x) = 1 + x + x^2 + x^3 + x^4 \quad \text{for } \alpha^3 \text{ and its conjugates } \alpha^6, \alpha^9, \text{ and } \alpha^{12}$$

$$m_5(x) = 1 + x + x^2 \quad \text{for } \alpha^5 \text{ and its conjugate } \alpha^{10}$$

Thus, the generator polynomial is found to be

$$\begin{aligned} g(x) &= m_1(x)m_3(x)m_5(x) \\ &= (1 + x + x^4)(1 + x + x^2 + x^3 + x^4)(1 + x + x^2) \\ &= 1 + x + x^2 + x^4 + x^5 + x^8 + x^{10} \end{aligned}$$

Hence the  $(1, 1)$ th-order EG code of length 15 is actually the  $(15, 5)$  BCH code and is two-step majority decodable.

The euclidean geometry  $EG(m, 2^s)$  plays an important role in decoding the  $(r, s)$ th-order EG code of length  $n = 2^{ms} - 1$ . The null space of the EG code contains the incidence vectors of all the  $(r + 1)$ -flats of  $EG(m, 2^s)$  that do not pass through the origin. From a set of parity-check sums based on the incidence vectors of  $(r + 1)$ -flats, one can determine the check sums based on the incidence vectors of  $r$ -flats. The key point for orthogonalization is that the check sums corresponding to the  $(r + 1)$ -flats intersecting on a given  $r$ -flat are orthogonal on the check sums corresponding to this  $r$ -flat. Since the check sums corresponding to all  $(r + 1)$ -flats are known to the decoder, it can determine all check sums corresponding to  $r$ -flats with one level of majority logic.

As will be seen shortly, these flats are useful for correcting random errors.

A lower bound on the minimum distance (without proof here) of EG codes is given by the BCH bound as shown below:

$$d_{\text{EG}} \geq d_{\text{BCH}} = 2^{(m-r-1)s} + 2^{(m-r-2)s+1} - 1 \quad (8.42)$$

Hence it is possible to correct nearly  $\lfloor J/2 \rfloor = \lfloor (d_{\text{EG}} - 1)/2 \rfloor$  random errors with EG codes using an  $(r + 1)$ -step majority-logic decoder. As you can see from the following discussion, the decoding of each error digit requires  $r + 1$  steps of orthogonalization. Therefore, the  $(r, s)$ th-order EG code of length  $2^{ms} - 1$  is  $(r + 1)$ -step majority decodable. The error-correcting capability of EG codes is given by

$$t_{\text{EG}} = \left\lfloor \frac{J}{2} \right\rfloor = \left\lfloor \frac{2^{(m-r)s} - 1}{2(2^s - 1)} - \frac{1}{2} \right\rfloor \quad (8.43)$$

The error-correcting process can use the following recursive argument. Given an  $r$ -flat  $F^{(r)}$ , the number of  $(r + 1)$ -flats in  $EG(m, 2^s)$  that intersect on  $F^{(r)}$  but do not pass through the origin is given by

$$J = \frac{2^{(m-r)s} - 1}{2^s - 1} - 1 \quad (8.44)$$

These  $J$  incidence vectors of  $(r + 1)$ -flats are orthogonal on the digits that correspond to the points in  $F^{(r)}$ . Hence, the parity-check sums formed from these  $J$  incidence vectors are orthogonal on the error digits corresponding to the points in  $F^{(r)}$ . If there are  $\lfloor J/2 \rfloor$  or fewer errors in the received word, the error sums at the locations corresponding to the points in  $F^{(r)}$  can be determined correctly. Thus, for every  $r$ -flat passing through the point  $\alpha^{2^{ms}-2}$ , the error sum can be formed as the first step of orthogonalization. These error sums are then used for the second step of orthogonalization. Let  $F^{(r-1)}$  be an  $(r - 1)$ -flat. There are  $J_1$   $r$ -flats which intersect on  $F^{(r-1)}$ , where

$$J_1 = \frac{2^{(m-r+1)s} - 1}{2^s - 1} - 1 \quad J_1 > J \quad (8.45)$$

The error sums corresponding to these  $J_1$   $r$ -flats are orthogonal on the error sums corresponding to the points in  $F^{(r-1)}$ . Then the error sums for  $F^{(r-1)}$  can be determined from the  $J_1$  error sums for  $F^{(r)}$ . For the third step of orthogonalization, there are

$$J_2 = \frac{2^{(m-r+2)s} - 1}{2^s - 1} - 1 \quad J_2 > J_1 > J \quad (8.46)$$

error sums for  $F^{(r-1)}$  that are orthogonal on the error sums for  $F^{(r-2)}$ . These error sums for  $F^{(r-2)}$  are used for the next step of orthogonalization. Using this recursive process for all the 1-flats, we can determine the check sums (which are formed from  $J_\mu$ ,  $\mu = 3, 4, \dots, r$ ) orthogonal on the error sums as follows.

$$\begin{aligned} J_3 &= \frac{2^{(m-r+3)s} - 1}{2^s - 1} - 1 \\ &\vdots \\ J_r &= \frac{2^{ms} - 1}{2^s - 1} - 1 \quad J_r > J_{r-1} > \dots > J_1 > J \end{aligned} \quad (8.47)$$

where  $J_r$  represents the error sums orthogonal on the error digit  $e_{2^{ms-2}}$  at the location  $\alpha^{2^{ms-2}}$ . Thus,  $e_{2^{ms-2}}$  can be determined from these  $J_r$  if and only if there are  $\lfloor J/2 \rfloor$  or fewer errors in the received word. This shows that the check sums based on the incidence vectors of the  $(r+1)$ -flats can be pushed down in  $r+1$  steps to the check sums orthogonal on the individual error digit.

In summary, an  $(r, s)$ th-order EG code of length  $2^{ms} - 1$ , where the null space contains all  $(r+1)$ -flats of  $EG(m, 2^s)$ , can be majority-decoded in  $r+1$  steps in the presence of up to  $\lfloor (d_{EG} - 1)/2 \rfloor$  errors, where  $d_{EG} = \{(2^{(m-r)s} - 1)/(2^s - 1)\}$ .

For a small length  $n$ , the majority-logic decoding for EG codes is much simpler than the decoding for comparable BCH codes in terms of implementation. In fact, EG codes of code length  $n$ ,  $7 \leq n \leq 31$ , can be decoded in no more than three steps, as illustrated in the subsequent examples. However, for moderate lengths or even longer ones, i.e.,  $n > 63$ , the three-step-decoding rule does not always apply, but the fewest possible decoding steps must be used. For large  $n$ , BCH codes are definitely superior to the EG codes in error-correcting capability and decoding complexity. A list of some EG codes along with their parameters is given in Table 8.4.

**Example 8.15** Consider the  $(2, 1)$ th-order EG code of length 15. Since  $k = 11$ , the parameters of this code are  $m = 4$ ,  $s = 1$ , and  $r = 2$ . This EG code is actually the  $(15, 11)$  BCH code for single-error correction. Since  $GF(2^4)$  is the euclidean geometry  $EG(4, 2)$  over  $GF(2)$ , the null space of this code contains the incidence vectors of all the 3-flats of the  $EG(4, 2)$  code that do not pass through the origin. Next we show how to form the orthogonal check sums based on the structure of  $EG(4, 2)$ . Let  $\alpha$  be a primitive element of  $GF(2^4)$  and  $\theta$  be a nonnegative integer less than  $2^{ms} - 1 = 15$ . Then we can express  $\theta$  in the radix- $2^s$  form ( $s = 1$ ) as  $\theta = \rho_0 + 2\rho_1 + 4\rho_2 + 8\rho_3$ , where  $0 \leq \rho_i < 2$ . The real sum of the coefficients in the  $2^s$ -weight expansion of  $\theta$  is  $W_2(\theta) = \rho_0 + \rho_1 + \rho_2 + \rho_3$ . Thus, the generator polynomial  $g(x)$  of the  $(2, 1)$ th-order EG code of length 15 has  $\alpha^\theta$  as a root under the condition  $0 < W_2(\theta) \leq 1$  for  $l = 0$  and  $0 \leq u < 15$ .

TABLE 8.4 Parameters of Some Euclidean Geometry Codes

| $n$  | $k$  | $m$ | $s$ | $r$ | $J$ | $t_{EG}$ |
|------|------|-----|-----|-----|-----|----------|
| 7    | 4    | 3   | 1   | 1   | 2   | 1        |
|      | 1    | 3   | 1   | 0   | 6   | 3        |
| 15   | 11   | 4   | 1   | 2   | 2   | 1        |
|      | 7    | 2   | 2   | 0   | 4   | 2        |
|      | 5    | 4   | 1   | 1   | 2   | 3        |
|      | 1    | 4   | 1   | 0   | 14  | 7        |
| 31   | 26   | 5   | 1   | 3   | 2   | 1        |
|      | 16   | 5   | 1   | 2   | 6   | 3        |
|      | 6    | 5   | 1   | 1   | 14  | 7        |
|      | 1    | 5   | 1   | 0   | 30  | 15       |
| 63   | 57   | 6   | 1   | 4   | 2   | 1        |
|      | 48   | 3   | 2   | 1   | 4   | 2        |
|      | 42   | 6   | 1   | 3   | 6   | 3        |
|      | 37   | 2   | 3   | 0   | 8   | 4        |
|      | 22   | 6   | 1   | 2   | 14  | 7        |
|      | 13   | 3   | 2   | 0   | 20  | 10       |
|      | 7    | 6   | 1   | 1   | 30  | 15       |
|      | 1    | 6   | 1   | 0   | 62  | 31       |
|      | 127  | 7   | 1   | 5   | 2   | 1        |
| 255  | 64   | 7   | 1   | 3   | 14  | 7        |
|      | 247  | 8   | 1   | 6   | 2   | 1        |
| 511  | 93   | 8   | 1   | 3   | 30  | 15       |
|      | 502  | 9   | 1   | 7   | 2   | 1        |
|      | 256  | 9   | 1   | 4   | 30  | 15       |
| 1023 | 130  | 9   | 1   | 3   | 62  | 31       |
|      | 1013 | 10  | 1   | 8   | 2   | 1        |
|      | 386  | 10  | 1   | 4   | 62  | 31       |

| $\theta$      | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 |
|---------------|---|---|---|---|---|---|---|---|---|----|----|----|----|----|
| $W_2(\theta)$ | 1 | 1 | 2 | 1 | 2 | 2 | 3 | 1 | 2 | 2  | 3  | 2  | 3  | 3  |

From the tabulation above, the nonnegative integers  $\theta$  that satisfy the condition  $0 < W_2(\theta) \leq 1$  are 1, 2, 4, and 8. Therefore, all the roots of  $g(x)$  are  $\alpha$ ,  $\alpha^2$ ,  $\alpha^4$ , and  $\alpha^8$ . Thus, the generator polynomial of the (2, 1)th-order EG code of length 15 becomes

$$\begin{aligned} g(x) &= (x + \alpha)(x + \alpha^2)(x + \alpha^4)(x + \alpha^8) \\ &= 1 + x + x^4 \end{aligned}$$

Since the error-correcting capability of this code is

$$t_{EG} = \left\lfloor \frac{2^2 - 1}{2(2 - 1)} - \frac{1}{2} \right\rfloor = 1$$

it is a single-error-correcting EG code.

A 2-flat passing through the point  $\alpha^{14}$  but not the origin consists of the points that have the form:  $\{\alpha^{14} + \mu_1\alpha^i + \mu_2\alpha^j; \mu_1, \mu_2 \in GF(2)\}$ . Since  $J = 2$ , there are two 2-flats as shown below:

$$\{\alpha^{14} + \mu_1\alpha^2 + \mu_2\alpha^3; \mu_1, \mu_2 \in GF(2)\} = \{\alpha^0, \alpha^8, \alpha^{13}, \alpha^{14}\}$$

$$\{\alpha^{14} + \mu_1\alpha^4 + \mu_2\alpha^5; \mu_1, \mu_2 \in GF(2)\} = \{\alpha^6, \alpha^9, \alpha^{12}, \alpha^{14}\}$$

in which, for example, the first point of these 2-flats can be computed with the aid of the elements of  $GF(2^4)$  generated by  $p(x) = 1 + x + x^4$  as follows:

| $\mu_1$ | $\mu_2$ | $\alpha^{14} + \mu_1\alpha^2 + \mu_2\alpha^3$  |                                                                                                                                                                                                                                              |
|---------|---------|------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0       | 0       | $\alpha^{14}$                                  | $\left. \begin{array}{l} \alpha^{14} \\ \alpha^{14} + \alpha^3 = \alpha^0 = 1 \\ \alpha^{14} + \alpha^2 = \alpha^{13} \\ \alpha^{14} + \alpha^2 + \alpha^3 = \alpha^8 \end{array} \right\} \{\alpha^0, \alpha^8, \alpha^{13}, \alpha^{14}\}$ |
| 0       | 1       | $\alpha^{14} + \alpha^3 = \alpha^0 = 1$        |                                                                                                                                                                                                                                              |
| 1       | 0       | $\alpha^{14} + \alpha^2 = \alpha^{13}$         |                                                                                                                                                                                                                                              |
| 1       | 1       | $\alpha^{14} + \alpha^2 + \alpha^3 = \alpha^8$ |                                                                                                                                                                                                                                              |

For each of these 2-flats, there are two 3-flats which consist of the points that have the form  $\{\alpha^{14} + \mu_1\alpha^2 + \mu_2\alpha^3 + \mu_3\alpha^k; \mu_1, \mu_2, \mu_3 \in GF(2)\}$ . The 3-flats that intersect on  $\{\alpha^0, \alpha^8, \alpha^{13}, \alpha^{14}\}$  have the same point  $\{\alpha^0, \alpha^4, \alpha^7, \alpha^8, \alpha^{10}, \alpha^{12}, \alpha^{13}, \alpha^{14}\}$  for  $k = 1, 5, 9, 11$ ; while the 3-flats that intersect on  $\{\alpha^0, \alpha^8, \alpha^{13}, \alpha^{14}\}$  have the same point  $\{\alpha^0, \alpha^1, \alpha^5, \alpha^8, \alpha^9, \alpha^{11}, \alpha^{13}, \alpha^{14}\}$  for  $k = 4, 7, 10, 12$ . Similarly, two 3-flats which intersect on  $\{\alpha^6, \alpha^9, \alpha^{12}, \alpha^{14}\}$  are

$$\{\alpha^3, \alpha^6, \alpha^7, \alpha^9, \alpha^{11}, \alpha^{12}, \alpha^{13}, \alpha^{14}\}$$

$$\{\alpha^0, \alpha^1, \alpha^2, \alpha^6, \alpha^9, \alpha^{10}, \alpha^{12}, \alpha^{14}\}$$

respectively. The incidence vector for these four 3-flats are

|                      | $\alpha^0$ | $\alpha^1$ | $\alpha^2$ | $\alpha^3$ | $\alpha^4$ | $\alpha^5$ | $\alpha^6$ | $\alpha^7$ | $\alpha^8$ | $\alpha^9$ | $\alpha^{10}$ | $\alpha^{11}$ | $\alpha^{12}$ | $\alpha^{13}$ | $\alpha^{14}$ |
|----------------------|------------|------------|------------|------------|------------|------------|------------|------------|------------|------------|---------------|---------------|---------------|---------------|---------------|
| $\mathbf{v}_{1,1} =$ | 1          | 0          | 0          | 0          | 1          | 0          | 0          | 1          | 1          | 0          | 1             | 0             | 1             | 1             | 1             |
| $\mathbf{v}_{1,2} =$ | 1          | 1          | 0          | 0          | 0          | 1          | 0          | 0          | 1          | 1          | 0             | 1             | 0             | 1             | 1             |
| $\mathbf{v}_{2,1} =$ | 0          | 0          | 0          | 1          | 0          | 0          | 1          | 1          | 0          | 1          | 0             | 1             | 1             | 1             | 1             |
| $\mathbf{v}_{2,2} =$ | 1          | 1          | 1          | 0          | 0          | 0          | 1          | 0          | 0          | 1          | 1             | 0             | 1             | 0             | 1             |

Let  $\mathbf{r}$  be the received word. The parity-check sums formed from these four incidence vectors are

$$\xi_{1,1} = \mathbf{v}_{1,1} \cdot \mathbf{r} = e_0 + e_4 + e_7 + e_8 + e_{10} + e_{12} + e_{13} + e_{14}$$

$$\xi_{1,2} = \mathbf{v}_{1,2} \cdot \mathbf{r} = e_0 + e_1 + e_5 + e_8 + e_9 + e_{11} + e_{13} + e_{14}$$

$$\xi_{2,1} = \mathbf{v}_{2,1} \cdot \mathbf{r} = e_3 + e_6 + e_7 + e_9 + e_{11} + e_{12} + e_{13} + e_{14}$$

$$\xi_{2,2} = \mathbf{v}_{2,2} \cdot \mathbf{r} = e_0 + e_1 + e_2 + e_6 + e_9 + e_{10} + e_{12} + e_{14}$$

Thus, we see that the check sums  $\xi_{1,1}$  and  $\xi_{1,2}$  are orthogonal on the error sum  $e_0 + e_8 + e_{13} + e_{14}$  corresponding to the 2-flat  $\{\alpha^0, \alpha^8, \alpha^{13}, \alpha^{14}\}$  and that the check sums  $\xi_{2,1}$  and  $\xi_{2,2}$  are orthogonal on the error sum  $e_6 + e_9 + e_{12} + e_{14}$  corresponding to the 2-flat  $\{\alpha^6, \alpha^9, \alpha^{12}, \alpha^{14}\}$ . Therefore, we can determine the error sums corresponding to these two 2-flats which pass through the point  $\alpha^{14}$ . Since



$J = 2$ , we need only determine the two error sums that are used to determine  $e_{14}$ . Therefore, the (2, 1)th-order EG code of length 15 is a two-step majority-logic decodable code as shown in Fig. 8.12.

For checking purposes, consider the (15, 11) BCH code that is equivalent to the (2, 1)th-order EG code of length 15. Since this EG code is generated by  $g(x) = 1 + x + x^4$ , the parity-check matrix  $\mathbf{H}$  of the (15, 11) BCH code is

$$\mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} \mathbf{h}_0 \\ \mathbf{h}_1 \\ \mathbf{h}_2 \\ \mathbf{h}_3 \end{bmatrix}$$

We see that the vectors  $\mathbf{h}_0$  and  $\mathbf{h}_0 + \mathbf{h}_1$  are orthogonal on digit positions 0, 8, 13, and 14; and that the vectors  $\mathbf{h}_3$  and  $\mathbf{h}_0 + \mathbf{h}_1 + \mathbf{h}_2$  are orthogonal on digit positions 6, 9, 12, and 14. Let  $E_{\alpha_1} = \{e_0, e_8, e_{13}, e_{14}\}$  and  $E_{\alpha_2} = \{e_6, e_9, e_{12}, e_{14}\}$  be two selected error sets. Then, the parity-check sums formed from the orthogonal vectors  $\mathbf{h}_0$ ,  $\mathbf{h}_0 + \mathbf{h}_1$ ,  $\mathbf{h}_3$ , and  $\mathbf{h}_0 + \mathbf{h}_1 + \mathbf{h}_2$  with the received word  $\mathbf{r}$  are

$$\xi_{1,1} = \mathbf{r} \cdot \mathbf{h}_0 = e_0 + e_4 + e_7 + e_8 + e_{10} + e_{12} + e_{13} + e_{14}$$

$$\xi_{1,2} = \mathbf{r} \cdot (\mathbf{h}_0 + \mathbf{h}_1) = e_0 + e_1 + e_5 + e_8 + e_9 + e_{11} + e_{13} + e_{14}$$

$$\xi_{2,1} = \mathbf{r} \cdot \mathbf{h}_3 = e_3 + e_6 + e_7 + e_9 + e_{11} + e_{12} + e_{13} + e_{14}$$

$$\xi_{2,2} = \mathbf{r} \cdot (\mathbf{h}_0 + \mathbf{h}_1 + \mathbf{h}_2) = e_0 + e_1 + e_2 + e_6 + e_9 + e_{10} + e_{12} + e_{14}$$

These check sums  $\xi_{1,1}$ ,  $\xi_{1,2}$ ,  $\xi_{2,1}$ , and  $\xi_{2,2}$  are orthogonal only on the error digit  $e_{14}$ . Thus, the value of  $e_{14}$  can be correctly estimated provided that there is only one error in  $\mathbf{e}$ .

## 8.5 Projective Geometry Codes

A projective geometry is closely related to a euclidean geometry augmented by some new flats involving some extra points. Therefore, the PG codes constitute a class of codes that is similar to the class of EG codes. Let  $PG(m, 2^s)$  denote an  $m$ -dimensional projective geometry over  $GF(2^s)$ .  $PG(m, 2^s)$  has  $(2^{(m+1)s} - 1)/(2^s - 1)$  points and may be constructed from the nonzero elements of the Galois field  $GF(2^{(m+1)s})$ . There are  $2^{(m+1)s} - 1$  such nonzero points, and they are divided into  $(2^{(m+1)s} - 1)/(2^s - 1)$  sets, each set constituting one point of  $PG(m, 2^s)$ . Each point of  $PG(m, 2^s)$  has  $2^s - 1$  points of  $GF(2^{(m+1)s})$  mapped onto

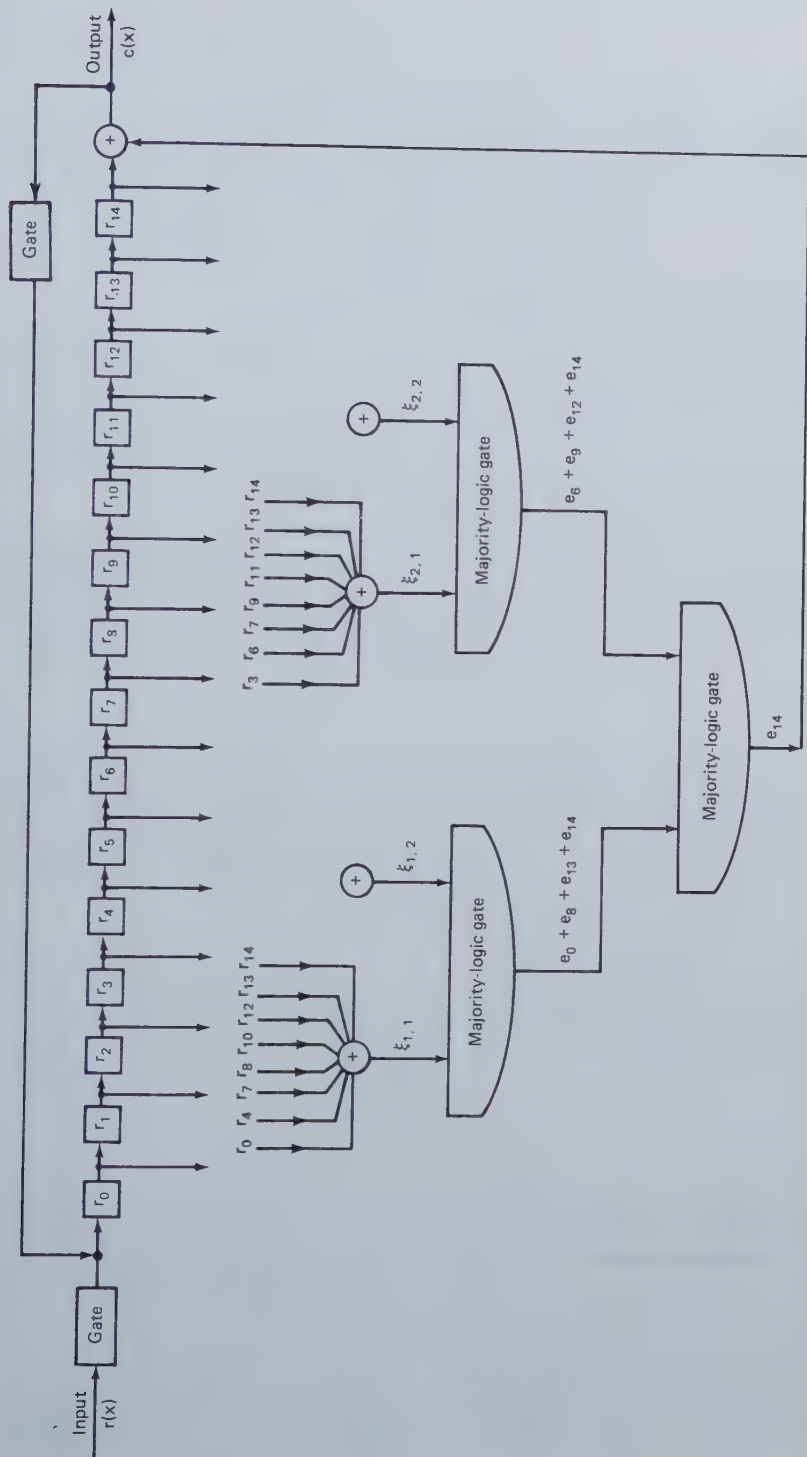


Figure 8.12 The majority-logic decoder for the (2, 1)th-order (15, 11) EG code.

it. Thus, it is well known that  $PG(m, 2^s)$  can be constructed from  $GF(2^{(m+1)s})$ .

Consider  $GF(2^{(m+1)s})$  which contains  $GF(2^s)$  as a subfield. Let  $\alpha$  be a primitive element of  $GF(2^{(m+1)s})$  and  $\beta$  be a primitive element of  $GF(2^s)$ . Then, all the nonzero elements of  $GF(2^{(m+1)s})$  consist of  $\alpha^0, \alpha^1, \dots, \alpha^{2^{(m+1)s}-2}$  and the nonzero elements of a subfield  $GF(2^s)$  are formed as  $1, \beta = \alpha^n, \beta^2, \dots, \beta^{2^s-2}$ .

Let us define the  $(r, s)$ th-order PG code of length

$$n = \frac{2^{(m+1)s} - 1}{2^s - 1} = 2^{ms} + 2^{(m-1)s} + \dots + 2^s + 1 \quad (8.48)$$

with symbols from  $GF(2)$  as the null space of the space generated by all  $(r+1)$ -flats of  $PG(m, 2^s)$ . Thus, the decoding procedure described for EG codes also applies to the decoding of PG codes.

Consider the set  $U = \{\alpha^0, \alpha^1, \alpha^2, \dots, \alpha^{n-1}\}$ , which consists of the first  $n$  powers of  $\alpha$ . No element  $\alpha^i$  in  $U$  can be a product of an element in a subfield  $GF(2^s)$  and another element  $\alpha^j$  in  $U$ , e.g.,

$$\alpha^i \neq \mu \cdot \alpha^j; \mu \in GF(2^s) \quad 0 \leq i, j \leq n-1, i \neq j \quad (8.49)$$

If the set

$$(\alpha^i) \triangleq \{\alpha^i, \beta\alpha^i, \beta^2\alpha^i, \dots, \beta^{2^s-2}\alpha^i\} \quad 0 \leq i \leq n-1 \quad (8.50)$$

is defined such that it consists of  $2^s - 1$  elements and each element is a multiple of the first element in the set, the nonzero elements of  $GF(2^{(m+1)s})$  can be partitioned into the following  $n$  disjoint subsets:

$$\begin{aligned} (\alpha^0) &= \{\alpha^0, \beta\alpha^0, \beta^2\alpha^0, \dots, \beta^{2^s-2}\alpha^0\} \\ (\alpha^1) &= \{\alpha^1, \beta\alpha^1, \beta^2\alpha^1, \dots, \beta^{2^s-2}\alpha^1\} \\ (\alpha^2) &= \{\alpha^2, \beta\alpha^2, \beta^2\alpha^2, \dots, \beta^{2^s-2}\alpha^2\} \\ &\vdots \\ (\alpha^{n-1}) &= \{\alpha^{n-1}, \beta\alpha^{n-1}, \beta^2\alpha^{n-1}, \dots, \beta^{2^s-2}\alpha^{n-1}\} \end{aligned} \quad (8.51)$$

where  $\beta = \alpha^n$  is a primitive element in  $GF(2^s)$ . From Eq. (8.51), we see that no element in one set can be a product of an element of  $GF(2^s)$  and an element from a different set. These  $n$  elements  $(\alpha^0), (\alpha^1), (\alpha^2), \dots, (\alpha^{n-1})$  actually form  $PG(m, 2^s)$  over  $GF(2^s)$ . Therefore, they are considered to be points of  $PG(m, 2^s)$ .

A 1-flat passing through any two distinct points  $(\alpha^i)$  and  $(\alpha^j)$  in  $PG(m, 2^s)$  constitutes the points of

$$\{(\mu_1\alpha^i + \mu_2\alpha^j); \mu_1, \mu_2 \in GF(2^s)\} \quad (8.52)$$

where  $\mu_1$  and  $\mu_2$  are not both equal to zero. There are  $(2^s)^2 - 1$  possible combinations of  $\mu_1$  and  $\mu_2$  from  $GF(2^s)$  except for  $\mu_1 = \mu_2 = 0$ . Also, there are  $2^s - 1$  choices of  $\mu_1$  and  $\mu_2$  that result in the same point, such as  $\mu_1\alpha^i + \mu_2\alpha^j$ ,  $\beta\mu_1\alpha^i + \beta\mu_2\alpha^j$ , ...,  $\beta^{2^s-2}\mu_1\alpha^i + \beta^{2^s-2}\mu_2\alpha^j$  in  $PG(m, 2^s)$ . Thus, a 1-flat in  $PG(m, 2^s)$  consists of

$$\frac{(2^2)^s - 1}{2^s - 1} = 2^s + 1 \quad (8.53)$$

distinct points. If  $(\alpha^k)$  is not a point on the 1-flat  $\{(\mu_1\alpha^i + \mu_2\alpha^j); \mu_1, \mu_2 \in GF(2^s)\}$ , then the 1-flat  $\{(\mu_1\alpha^i + \mu_2\alpha^j); \mu_1, \mu_2 \in GF(2^s)\}$  and the 1-flat  $\{(\mu_1\alpha^k + \mu_2\alpha^j); \mu_1, \mu_2 \in GF(2^s)\}$  will have only one common point  $(\alpha^j)$ . Therefore, they intersect at  $(\alpha^j)$ . Thus, the number of 1-flats in  $PG(m, 2^s)$  that intersect at a given point is

$$\frac{2^{ms} - 1}{2^s - 1} = 2^{(m-1)s} + \dots + 2^s + 1 \quad (8.54)$$

Generally, if  $(\alpha^{\eta_1}), (\alpha^{\eta_2}), \dots, (\alpha^{\eta_{r+1}})$  are assumed to be linearly independent points in  $PG(m, 2^s)$ , the points

$$\{(\mu_1\alpha^{\eta_1} + \mu_2\alpha^{\eta_2} + \dots + \mu_{r+1}\alpha^{\eta_{r+1}}); \mu_i \in GF(2^s), 1 \leq i \leq r+1\} \quad (8.55)$$

will constitute an  $r$ -flat in  $PG(m, 2^s)$ . There are  $2^{(r+1)s} - 1$  combinations for  $\mu_1, \mu_2, \dots, \mu_{r+1}$ , except when  $\mu_1 = \mu_2 = \dots = \mu_{r+1} = 0$ . Hence, in order for the same point to exist in  $PG(m, 2^s)$ , there must be

$$\frac{2^{(r+1)s} - 1}{2^s - 1} = 2^{rs} + 2^{(r-1)s} + \dots + 2^s + 1 \quad (8.56)$$

points in an  $r$ -flat in  $PG(m, 2^s)$  because there are always  $2^s - 1$  choices of  $\mu_1$  to  $\mu_{r+1}$ .

If  $(\alpha^{\eta_{r+1}})$  is not a point in the  $r$ -flat expressed by Eq. (8.55), then the  $r$ -flat of Eq. (8.55) and the  $r$ -flat

$$\{(\mu_1\alpha^{\eta_1} + \mu_2\alpha^{\eta_2} + \dots + \mu_{r+1}\alpha^{\eta_{r+1}}); \mu_i \in GF(2^s), 1 \leq i \leq r+1\} \quad (8.57)$$

intersect on the  $(r-1)$ -flat

$$\{(\mu_1\alpha^{\eta_1} + \mu_2\alpha^{\eta_2} + \dots + \mu_r\alpha^{\eta_r}); \mu_i \in GF(2^s), 1 \leq i \leq r\} \quad (8.58)$$

Thus, the number of  $r$ -flats in  $PG(m, 2^s)$  that intersect on a given  $(r-1)$ -flat in  $PG(m, 2^s)$  is

$$J = \frac{2^{(m-r+1)s} - 1}{2^s - 1} = 2^{(m-r)s} + 2^{(m-r-1)s} + \dots + 2^s + 1 \quad (8.59)$$

Let an  $r$ -flat in  $PG(m, 2^s)$  denote  $F^{(r)}$ . The incidence vector of the  $r$ -flat  $F^{(r)}$  is

$$\mathbf{u}_{F^{(r)}} = (u_0, u_1, u_2, \dots, u_{n-1}) \quad (8.60)$$

which is represented as an  $n$ -tuple over  $GF(2)$ . Any  $i$ th component  $u_i$  in  $\mathbf{u}_{F^{(r)}}$  is 1 if  $(\alpha^i)$  is a point in  $F^{(r)}$  and is 0 otherwise. Next, let us consider the generator polynomial of an  $(r, s)$  th-order PG code. Let  $\alpha$  be a primitive element in  $GF(2^{(m+1)s})$  and  $\theta$  be a nonnegative integer less than  $2^{(m+1)s} - 1$ . Then  $\theta$  can be expanded into the radix- $2^s$  form as in the case of the EG codes.

$$\theta = \sum_{i=0}^m \rho_i 2^{is} = \rho_0 + \rho_1 2^s + \rho_2 2^{2s} + \dots + \rho_m 2^{ms} \quad (8.61)$$

where  $0 \leq \rho_i < 2^s$  for  $0 \leq i < m + 1$ . Thus the real sum of the coefficients in the radix- $2^s$  expansion of  $\theta$  is

$$W_{2^s}(\theta) \triangleq \sum_{i=0}^m \rho_i = \rho_0 + \rho_1 + \rho_2 + \dots + \rho_m \quad (8.62)$$

If  $\theta_l$  is the remainder that results from dividing  $2^l \theta$  ( $0 \leq l$ ) by  $2^{(m+1)s} - 1$ , then we have

$$2^l \theta = q(2^{(m+1)s} - 1) + \theta_l \quad (8.63)$$

with  $0 \leq \theta_l < 2^{(m+1)s} - 1$ . Thus, the generator polynomial  $g(x)$  of the  $(r, s)$  th-order PG code of length  $n = (2^{(m+1)s} - 1)/(2^s - 1)$  has  $\alpha^\theta$  as a root under the following condition:

$$0 < \max_{1 \leq l < s} W_{2^s}(\theta_l) = j(2^s - 1) \quad 0 \leq j \leq m - r \quad (8.64)$$

For  $r = 1$ , we obtain a class of one-step majority-logic decodable PG codes. When  $m = 2$  and  $r = 1$ , difference-set codes result. A  $(1, 1)$ th-order PG code for  $s = 1$  yields a maximum-length code. A list of some PG codes is given in Table 8.5.

**Example 8.16** Consider a  $(1, 1)$ th-order PG code with  $m = 2$ . Since  $r = 1$  and  $s = 1$  by inspection, the projective geometry  $PG(2, 2)$  can be constructed from  $GF(2^3)$ , which contains  $GF(2)$  as its subfield. Hence, the code length of this PG code is

$$n = \frac{2^{(2+1)} - 1}{2 - 1} = 7$$

If  $\alpha$  denotes a primitive element of  $GF(2^3)$ , then the geometry  $PG(2, 2)$  consists of the following seven points:

$$(\alpha^0), (\alpha^1), (\alpha^2), (\alpha^3), (\alpha^4), (\alpha^5), (\alpha^6)$$

Consider the 1-flat passing through the point  $(\alpha^6)$ , which consists of three distinct points:



TABLE 8.5 Parameters of Some Projective Geometry Codes

| $n$  | $k$  | $m$ | $s$ | $r$ | $J$ | $t_{PG}$ |
|------|------|-----|-----|-----|-----|----------|
| 7    | 3    | 2   | 1   | 1   | 3   | 1        |
| 15   | 10   | 3   | 1   | 2   | 3   | 1        |
|      | 4    | 3   | 1   | 1   | 7   | 3        |
| 21   | 11   | 2   | 2   | 1   | 5   | 2        |
| 31   | 25   | 4   | 1   | 3   | 3   | 1        |
|      | 15   | 4   | 1   | 2   | 7   | 3        |
|      | 5    | 4   | 1   | 1   | 15  | 7        |
| 63   | 56   | 5   | 1   | 4   | 3   | 1        |
|      | 41   | 5   | 1   | 3   | 7   | 3        |
|      | 21   | 5   | 1   | 2   | 15  | 7        |
|      | 6    | 5   | 1   | 1   | 31  | 15       |
| 73   | 45   | 2   | 3   | 1   | 9   | 4        |
| 85   | 68   | 3   | 2   | 2   | 5   | 2        |
|      | 24   | 3   | 2   | 1   | 21  | 10       |
| 273  | 191  | 2   | 4   | 1   | 17  | 8        |
| 341  | 195  | 4   | 2   | 2   | 21  | 10       |
| 585  | 520  | 3   | 3   | 2   | 9   | 4        |
| 1057 | 813  | 2   | 5   | 1   | 33  | 16       |
| 1365 | 1063 | 5   | 2   | 3   | 21  | 10       |

$$\{(\mu_1\alpha^0 + \mu_2\alpha^6); \mu_1, \mu_2 \in GF(2)\} = \{(\alpha^0), (\alpha^2), (\alpha^6)\}$$

$$\{(\mu_1\alpha^1 + \mu_2\alpha^6); \mu_1, \mu_2 \in GF(2)\} = \{(\alpha^1), (\alpha^5), (\alpha^6)\}$$

$$\{(\mu_1\alpha^3 + \mu_2\alpha^6); \mu_1, \mu_2 \in GF(2)\} = \{(\alpha^3), (\alpha^4), (\alpha^6)\}$$

Using Eq. (8.64), we have  $0 \leq \max_{\theta} W(\theta) = j$ , where  $j = 0$  or  $1$  and  $\theta_0 = \theta$ . Since  $\theta$  is a nonnegative integer less than  $2^{(m+1)s} - 1 = 7$ , the  $\theta$ 's satisfying the condition above are  $0, 1, 2$ , and  $4$ . Therefore, the roots of the generator polynomial  $g(x)$  are  $\alpha^0, \alpha^1, \alpha^2$ , and  $\alpha^4$ . Thus, the generator polynomial  $g(x)$  of the  $(1, 1)$ th-order PG code of length  $7$  is

$$\begin{aligned} g(x) &= (x + \alpha^0)(x + \alpha^1)(x + \alpha^2)(x + \alpha^4) \\ &= 1 + x^2 + x^3 + x^4 \end{aligned}$$

Hence, this PG code is the  $(7, 3)$  difference-set cyclic code.

Consider the decoding of PG codes. As discussed before, the null space of a PG code contains the incidence vectors of all the  $r$ -flats of  $PG(m, 2^s)$ . Let  $F^{(r-1)}$  be an  $(r-1)$ -flat in  $PG(m, 2^s)$  that contains the point  $(\alpha^{n-1})$ . We see that the number of  $r$ -flats that intersect on the  $(r-1)$ -flat  $F^{(r-1)}$  is  $J = (2^{(m-r+1)s} - 1)/(2^s - 1)$  as seen in Eq. (8.59). The parity-check sums formed from these  $J$  incidence vectors are orthogonal on the error sum at the locations corresponding to the points in  $F^{(r-1)}$ . Thus, this error sum is determined correctly from the  $J$  check sums provided that there are no more than

$$\left\lfloor \frac{J}{2} \right\rfloor = \left\lfloor \frac{2^{(m-r+1)s} - 1}{2(2^s - 1)} \right\rfloor \quad (8.65)$$

errors in the received word  $\mathbf{r}$ . With such a recursive process, we can determine all the error sums corresponding to all the  $(r-1)$ -flats,  $(r-2)$ -flats, ..., 1-flats that contain the point  $(\alpha^{n-1})$ . These last error sums, which correspond to all the 1-flats, will be orthogonal on the error digit  $e_{n-1}$  at the location  $\alpha^{n-1}$ . Thus, the value of  $e_{n-1}$  can be determined. A total of  $r$  steps of orthogonalization are required to decode  $e_{n-1}$ . Hence, in this manner we say that a PG code is  $r$ -step decodable. Moreover, notice that at each step of orthogonalization, the error sums needed for the next step can always be determined correctly if and only if no more than  $\lfloor J/2 \rfloor$  errors have occurred in the received word  $\mathbf{r}$ . Because of the cyclic property of this code, other error digits in  $\mathbf{r}$  can be decoded in the same manner as described above. Thus, the  $(r, s)$ th-order PG code of length  $n = (2^{(m+1)s} - 1)/(2^s - 1)$  has the error-correcting capability

$$t_{PG} = \left\lfloor \frac{J}{2} \right\rfloor = \left\lfloor \frac{2^{(m-r+1)s} - 1}{2(2^s - 1)} \right\rfloor \quad (8.66)$$

with majority-logic decoding.

**Example 8.17** Consider the decoding of the  $(2, 1)$ th-order PG code with  $m = 3$ ,  $r = 2$ , and  $s = 1$ . The code length of this code is

$$n = \frac{2^{(3+1)} - 1}{2 - 1} = 15$$

First compute the generator polynomial  $g(x)$ , and then derive the parity-check sums orthogonal on the error digit  $e_{15}$ . Let  $\alpha$  be a primitive element of  $GF(2^4)$ . Since  $\theta$  is a nonnegative integer less than 15, the values of  $\theta$  that satisfy the condition

$$0 \leq \max W(\theta) = j \quad j = 0 \text{ or } 1$$

are 0, 1, 2, 4, and 8. Thus, the generator polynomial of this code is

$$\begin{aligned} g(x) &= (x + \alpha^0)(x + \alpha^1)(x + \alpha^2)(x + \alpha^4)(x + \alpha^8) \\ &= 1 + x^2 + x^4 + x^5 \end{aligned}$$

Since  $n = 15$ , this code is a  $(15, 10)$  cyclic code. The binary projective geometry  $PG(3, 2)$  consists of 15 points, i.e.,  $(\alpha^0), (\alpha^1), (\alpha^2), \dots, (\alpha^{14})$ . Since  $\beta = \alpha^{15} = 1$ ,  $\{0, 1\}$  forms the subfield  $GF(2)$ . A 1-flat in  $PG(3, 2)$  consists of  $2^s + 1 = 3$  points. Therefore, all the 1-flats passing through the point  $(\alpha^{14})$ , which consists of seven distinct points, are

$$\{(\mu_1 \alpha^0 + \mu_2 \alpha^{14}); \mu_1, \mu_2 \in GF(2)\} = \{(\alpha^0), (\alpha^3), (\alpha^{14})\}$$

$$\{(\mu_1 \alpha^1 + \mu_2 \alpha^{14}); \mu_1, \mu_2 \in GF(2)\} = \{(\alpha^1), (\alpha^7), (\alpha^{14})\}$$

$$\{(\mu_1\alpha^2 + \mu_2\alpha^{14}); \mu_1, \mu_2 \in GF(2)\} = \{(\alpha^2), (\alpha^{13}), (\alpha^{14})\}$$

$$\{(\mu_1\alpha^4 + \mu_2\alpha^{14}); \mu_1, \mu_2 \in GF(2)\} = \{(\alpha^4), (\alpha^9), (\alpha^{14})\}$$

$$\{(\mu_1\alpha^5 + \mu_2\alpha^{14}); \mu_1, \mu_2 \in GF(2)\} = \{(\alpha^5), (\alpha^{12}), (\alpha^{14})\}$$

$$\{(\mu_1\alpha^6 + \mu_2\alpha^{14}); \mu_1, \mu_2 \in GF(2)\} = \{(\alpha^6), (\alpha^8), (\alpha^{14})\}$$

$$\{(\mu_1\alpha^{10} + \mu_2\alpha^{14}); \mu_1, \mu_2 \in GF(2)\} = \{(\alpha^{10}), (\alpha^{11}), (\alpha^{14})\}$$

The null space of this code contains the incidence vectors of all the 2-flats of  $PG(3, 2)$ . The number of 2-flats in  $PG(3, 2)$  that intersect on a 1-flat in  $PG(3, 2)$  is 3. Therefore, all the 2-flats intersecting on each 1-flat are computed as follows:

1. The 2-flats that intersect on the 1-flat  $\{(\alpha^0), (\alpha^3), (\alpha^{14})\}$  are

$$\begin{aligned} \{(\mu_1\alpha^0 + \mu_2\alpha^{14} + \mu_3\alpha^1); \mu_1, \mu_2, \mu_3 \in GF(2)\} \\ = \{(\alpha^0), (\alpha^1), (\alpha^3), (\alpha^4), (\alpha^7), (\alpha^9), (\alpha^{14})\} \end{aligned}$$

$$\begin{aligned} \{(\mu_1\alpha^0 + \mu_2\alpha^{14} + \mu_3\alpha^2); \mu_1, \mu_2, \mu_3 \in GF(2)\} \\ = \{(\alpha^0), (\alpha^2), (\alpha^3), (\alpha^6), (\alpha^8), (\alpha^{13}), (\alpha^{14})\} \end{aligned}$$

$$\begin{aligned} \{(\mu_1\alpha^0 + \mu_2\alpha^{14} + \mu_3\alpha^5); \mu_1, \mu_2, \mu_3 \in GF(2)\} \\ = \{(\alpha^0), (\alpha^3), (\alpha^5), (\alpha^{10}), (\alpha^{11}), (\alpha^{12}), (\alpha^{14})\} \end{aligned}$$

2. The 2-flats that intersect on the 1-flat  $\{(\alpha^1), (\alpha^7), (\alpha^{14})\}$  are

$$\begin{aligned} \{(\mu_1\alpha^1 + \mu_2\alpha^{14} + \mu_3\alpha^0); \mu_1, \mu_2, \mu_3 \in GF(2)\} \\ = \{(\alpha^0), (\alpha^1), (\alpha^3), (\alpha^4), (\alpha^7), (\alpha^9), (\alpha^{14})\} \end{aligned}$$

$$\begin{aligned} \{(\mu_1\alpha^1 + \mu_2\alpha^{14} + \mu_3\alpha^2); \mu_1, \mu_2, \mu_3 \in GF(2)\} \\ = \{(\alpha^1), (\alpha^2), (\alpha^5), (\alpha^7), (\alpha^{12}), (\alpha^{13}), (\alpha^{14})\} \end{aligned}$$

$$\begin{aligned} \{(\mu_1\alpha^1 + \mu_2\alpha^{14} + \mu_3\alpha^6); \mu_1, \mu_2, \mu_3 \in GF(2)\} \\ = \{(\alpha^1), (\alpha^6), (\alpha^7), (\alpha^8), (\alpha^{10}), (\alpha^{11}), (\alpha^{14})\} \end{aligned}$$

3. The 2-flats that intersect on the 1-flat  $\{(\alpha^2), (\alpha^{13}), (\alpha^{14})\}$  are

$$\begin{aligned} \{(\mu_1\alpha^2 + \mu_2\alpha^{14} + \mu_3\alpha^0); \mu_1, \mu_2, \mu_3 \in GF(2)\} \\ = \{(\alpha^0), (\alpha^2), (\alpha^3), (\alpha^6), (\alpha^8), (\alpha^{13}), (\alpha^{14})\} \end{aligned}$$

$$\begin{aligned} \{(\mu_1\alpha^2 + \mu_2\alpha^{14} + \mu_3\alpha^1); \mu_1, \mu_2, \mu_3 \in GF(2)\} \\ = \{(\alpha^1), (\alpha^2), (\alpha^5), (\alpha^7), (\alpha^{12}), (\alpha^{13}), (\alpha^{14})\} \end{aligned}$$

$$\begin{aligned} \{(\mu_1\alpha^2 + \mu_2\alpha^{14} + \mu_3\alpha^4); \mu_1, \mu_2, \mu_3 \in GF(2)\} \\ = \{(\alpha^2), (\alpha^4), (\alpha^9), (\alpha^{10}), (\alpha^{11}), (\alpha^{13}), (\alpha^{14})\} \end{aligned}$$

4. The 2-flats that intersect on the 1-flat  $\{(\alpha^4), (\alpha^9), (\alpha^{14})\}$  are

$$\begin{aligned} &\{(\mu_1\alpha^4 + \mu_2\alpha^{14} + \mu_3\alpha^0); \mu_1, \mu_2, \mu_3 \in GF(2)\} \\ &= \{(\alpha^0), (\alpha^1), (\alpha^3), (\alpha^4), (\alpha^7), (\alpha^9), (\alpha^{14})\} \end{aligned}$$

$$\begin{aligned} &\{(\mu_1\alpha^4 + \mu_2\alpha^{14} + \mu_3\alpha^2); \mu_1, \mu_2, \mu_3 \in GF(2)\} \\ &= \{(\alpha^2), (\alpha^4), (\alpha^9), (\alpha^{10}), (\alpha^{11}), (\alpha^{13}), (\alpha^{14})\} \end{aligned}$$

$$\begin{aligned} &\{(\mu_1\alpha^4 + \mu_2\alpha^{14} + \mu_3\alpha^5); \mu_1, \mu_2, \mu_3 \in GF(2)\} \\ &= \{(\alpha^4), (\alpha^5), (\alpha^6), (\alpha^8), (\alpha^9), (\alpha^{12}), (\alpha^{14})\} \end{aligned}$$

5. The 2-flats that intersect on the 1-flat  $\{(\alpha^5), (\alpha^{12}), (\alpha^{14})\}$  are

$$\begin{aligned} &\{(\mu_1\alpha^5 + \mu_2\alpha^{14} + \mu_3\alpha^0); \mu_1, \mu_2, \mu_3 \in GF(2)\} \\ &= \{(\alpha^0), (\alpha^3), (\alpha^5), (\alpha^{10}), (\alpha^{11}), (\alpha^{12}), (\alpha^{14})\} \end{aligned}$$

$$\begin{aligned} &\{(\mu_1\alpha^5 + \mu_2\alpha^{14} + \mu_3\alpha^1); \mu_1, \mu_2, \mu_3 \in GF(2)\} \\ &= \{(\alpha^1), (\alpha^2), (\alpha^5), (\alpha^7), (\alpha^{12}), (\alpha^{13}), (\alpha^{14})\} \end{aligned}$$

$$\begin{aligned} &\{(\mu_1\alpha^5 + \mu_2\alpha^{14} + \mu_3\alpha^4); \mu_1, \mu_2, \mu_3 \in GF(2)\} \\ &= \{(\alpha^4), (\alpha^5), (\alpha^6), (\alpha^8), (\alpha^9), (\alpha^{12}), (\alpha^{14})\} \end{aligned}$$

6. The 2-flats that intersect on the 1-flat  $\{(\alpha^6), (\alpha^8), (\alpha^{14})\}$  are

$$\begin{aligned} &\{(\mu_1\alpha^6 + \mu_2\alpha^{14} + \mu_3\alpha^0); \mu_1, \mu_2, \mu_3 \in GF(2)\} \\ &= \{(\alpha^0), (\alpha^2), (\alpha^3), (\alpha^6), (\alpha^8), (\alpha^{13}), (\alpha^{14})\} \end{aligned}$$

$$\begin{aligned} &\{(\mu_1\alpha^6 + \mu_2\alpha^{14} + \mu_3\alpha^1); \mu_1, \mu_2, \mu_3 \in GF(2)\} \\ &= \{(\alpha^1), (\alpha^6), (\alpha^7), (\alpha^8), (\alpha^{10}), (\alpha^{11}), (\alpha^{14})\} \end{aligned}$$

$$\begin{aligned} &\{(\mu_1\alpha^6 + \mu_2\alpha^{14} + \mu_3\alpha^4); \mu_1, \mu_2, \mu_3 \in GF(2)\} \\ &= \{(\alpha^4), (\alpha^5), (\alpha^6), (\alpha^8), (\alpha^9), (\alpha^{12}), (\alpha^{14})\} \end{aligned}$$

7. The 2-flats that intersect on the 1-flat  $\{(\alpha^{10}), (\alpha^{11}), (\alpha^{14})\}$  are

$$\begin{aligned} &\{(\mu_1\alpha^{10} + \mu_2\alpha^{14} + \mu_3\alpha^0); \mu_1, \mu_2, \mu_3 \in GF(2)\} \\ &= \{(\alpha^0), (\alpha^3), (\alpha^5), (\alpha^{10}), (\alpha^{11}), (\alpha^{12}), (\alpha^{14})\} \end{aligned}$$

$$\begin{aligned} &\{(\mu_1\alpha^{10} + \mu_2\alpha^{14} + \mu_3\alpha^1); \mu_1, \mu_2, \mu_3 \in GF(2)\} \\ &= \{(\alpha^1), (\alpha^6), (\alpha^7), (\alpha^8), (\alpha^{10}), (\alpha^{11}), (\alpha^{14})\} \end{aligned}$$

$$\begin{aligned} &\{(\mu_1\alpha^{10} + \mu_2\alpha^{14} + \mu_3\alpha^2); \mu_1, \mu_2, \mu_3 \in GF(2)\} \\ &= \{(\alpha^2), (\alpha^4), (\alpha^9), (\alpha^{10}), (\alpha^{11}), (\alpha^{13}), (\alpha^{14})\} \end{aligned}$$

Now, the incidence vectors  $\mathbf{v}$  for these twenty-one 2-flats must be determined. Let  $\mathbf{r}$  be the received word. Then the parity-check sums  $\xi$  are formed from the inner product of these incidence vectors  $\mathbf{v}$  and the received word  $\mathbf{r}$  such that  $\xi = \mathbf{v} \cdot \mathbf{r}$ .

1. The check sums orthogonal on the error sum  $e_0 + e_3 + e_{14}$  are

$$\xi_{1,1} = e_0 + e_1 + e_3 + e_4 + e_7 + e_9 + e_{14}$$

$$\xi_{1,2} = e_0 + e_2 + e_3 + e_6 + e_8 + e_{13} + e_{14}$$

$$\xi_{1,3} = e_0 + e_3 + e_5 + e_{10} + e_{11} + e_{12} + e_{14}$$

2. The check sums orthogonal on the error sum  $e_1 + e_7 + e_{14}$  are

$$\xi_{2,1} = e_0 + e_1 + e_3 + e_4 + e_7 + e_9 + e_{14}$$

$$\xi_{2,2} = e_1 + e_2 + e_5 + e_7 + e_{12} + e_{13} + e_{14}$$

$$\xi_{2,3} = e_1 + e_6 + e_7 + e_8 + e_{10} + e_{11} + e_{14}$$

3. The check sums orthogonal on the error sum  $e_2 + e_{13} + e_{14}$  are

$$\xi_{3,1} = e_0 + e_2 + e_3 + e_6 + e_8 + e_{13} + e_{14}$$

$$\xi_{3,2} = e_1 + e_2 + e_5 + e_7 + e_{12} + e_{13} + e_{14}$$

$$\xi_{3,3} = e_2 + e_4 + e_9 + e_{10} + e_{11} + e_{13} + e_{14}$$

4. The check sums orthogonal on the error sum  $e_4 + e_9 + e_{14}$  are

$$\xi_{4,1} = e_0 + e_1 + e_3 + e_4 + e_7 + e_9 + e_{14}$$

$$\xi_{4,2} = e_2 + e_4 + e_9 + e_{10} + e_{11} + e_{13} + e_{14}$$

$$\xi_{4,3} = e_4 + e_5 + e_6 + e_8 + e_9 + e_{12} + e_{14}$$

5. The check sums orthogonal on the error sum  $e_5 + e_{12} + e_{14}$  are

$$\xi_{5,1} = e_0 + e_3 + e_5 + e_{10} + e_{11} + e_{12} + e_{14}$$

$$\xi_{5,2} = e_1 + e_2 + e_5 + e_7 + e_{12} + e_{13} + e_{14}$$

$$\xi_{5,3} = e_4 + e_5 + e_6 + e_8 + e_9 + e_{12} + e_{14}$$

6. The check sums orthogonal on the error sum  $e_6 + e_8 + e_{14}$  are

$$\xi_{6,1} = e_0 + e_2 + e_3 + e_6 + e_8 + e_{13} + e_{14}$$

$$\xi_{6,2} = e_1 + e_6 + e_7 + e_8 + e_{10} + e_{11} + e_{14}$$

$$\xi_{6,3} = e_4 + e_5 + e_6 + e_8 + e_9 + e_{12} + e_{14}$$

7. The check sums orthogonal on the error sum  $e_{10} + e_{11} + e_{14}$  are

$$\xi_{7,1} = e_0 + e_3 + e_5 + e_{10} + e_{11} + e_{12} + e_{14}$$

$$\xi_{7,2} = e_1 + e_6 + e_7 + e_8 + e_{10} + e_{11} + e_{14}$$

$$\xi_{7,3} = e_2 + e_4 + e_9 + e_{10} + e_{11} + e_{13} + e_{14}$$

Thus, we see that these 21 check sums are orthogonal on the error digit  $e_{14}$ . However, the error-correcting capability of this PG code is

$$t = \left\lfloor \frac{J}{2} \right\rfloor = \left\lfloor \frac{2^{(3-2+1)} - 1}{2(2-1)} \right\rfloor = \left\lfloor \frac{3}{2} \right\rfloor = 1$$

Since  $J = 3$ , we need only determine three check sums, which correspond to any three error sums relating to any three 2-flats passing through  $\alpha^{14}$ . These error sums are then used to determine  $e_{14}$ . Thus, the (2, 1)th-order PG code of length 15 is a two-step majority-logic decodable code.

Rudolph (1967) was the first person to apply finite geometries to the construction of majority-logic decodable codes. Since then, several the-



oretical techniques for analyzing finite geometry codes were developed and generalized by Peterson, Kasami, Lin, Weldon, Graham and MacWilliams, Goethals and Delsarte, and others.

## PROBLEMS

- 8.1** For the (15, 7) BCH code generated by  $g(x) = 1 + x^4 + x^6 + x^7 + x^8$ :
- Show that the minimum distance of this code is exactly 5 by giving its generator matrix  $\mathbf{G}$ .
  - Determine the parity-check matrix  $\mathbf{H}$  of this code in systematic form.
  - Let  $\mathbf{e} = (e_0, e_1, \dots, e_{14})$  be an error pattern. Find the syndrome bits in terms of error bits.
  - Find the number of parity-check sums and the majority-logic error-correcting capability.
  - Determine the parity-check sums orthogonal on the error bit  $e_{14}$ .
  - Design both type I and type II majority-logic decoders for this code.
- 8.2** For the (21, 11) difference-set code generated by  $g(x) = 1 + x^2 + x^4 + x^6 + x^7 + x^{10}$ :
- Find the parity-check matrix for this code.
  - Find the syndrome bits in terms of error bits by using  $\mathbf{s} = \mathbf{e} \cdot \mathbf{H}^T$ .
  - Determine the parity-check sums orthogonal on the error bit  $e_{20}$ .
  - What is the minimum distance and error-correcting capability?
  - Devise the type I decoder for this code.
- 8.3** For the (21, 11) difference-set code in Prob. 8.2:
- Determine the null-space vectors  $\phi_k$ ,  $k = 1, 2, \dots, J$ , orthogonal on the received digit  $r_{20}$ .
  - Devise the type II decoder for this code.
- 8.4** A maximum-length code with  $m \geq 3$  is the dual of the Hamming code of length  $2^m - 1$ . For the (15, 4) maximum-length code with the parity polynomial  $h(x) = p(x) = 1 + x + x^4$ :
- Find the generator polynomial of this code.
  - What is the minimum distance of this code?
  - Find the generator polynomial of this code. Find the generator polynomial  $p^*(x)$  of the (15, 11) dual code.
  - Determine the polynomials orthogonal on  $x^{14}$ .
  - Find the parity-check sums orthogonal on  $e_{14}$ .
  - Design the type I and type II one-step majority-logic decoders for this code.
- 8.5** For the  $(n, k)$  maximum-length code with  $m = 6$ :
- Find  $n$  and  $k$ .
  - What is the minimum distance of this code?
  - What is the dual code of the same length?
  - Suppose that the dual code is generated by  $p(x) = 1 + x + x^6$ . Find the generator polynomial of this maximum-length code.

- 8.6** For the (15, 8) cyclic code generated by  $g(x) = 1 + x + x^3 + x^7$ :
- Find  $G$  and  $H$  for this code.
  - Is this code one-step majority-logic decodable? If so, find all the syndrome bits in terms of the error digits.
  - Find the parity-check sums orthogonal on  $e_{14}$ .
- 8.7** For the code which is specified by the perfect difference set  $Q = \{0, 2, 7, 8, 11\}$  with order  $q = 4$ :
- Obtain all the positive and negative ordered differences.
  - Determine the difference-set polynomial  $\lambda(x)$ .
  - Find  $h(x) = \text{GCD} \{\lambda(x), x^n + 1\}$  and  $g(x) = (x^n + 1)/h(x)$ . Is this code the (21, 11) difference-set code specified in Prob. 8.2?
  - Determine  $\varphi_0(x)$  by first using Eq. (8.30), and find four polynomials orthogonal on  $x^{20}$  by shifting  $\varphi_0(x)$ .
  - Find the parity-check sums orthogonal on  $e_{20}$ .
  - Devise the type II decoder for this code.
- 8.8** For the (15, 5) BCH code whose generator polynomial is  $g(x) = 1 + x + x^2 + x^4 + x^5 + x^8 + x^{10}$ , let  $\mathbf{r} = (r_0, r_1, r_2, \dots, r_{14})$  be the received word.
- Find the parity-check matrix in systematic form.
  - For six selected sets of error digits
 
$$\begin{array}{lll} E_{\alpha_1} = \{e_{13}, e_{14}\} & E_{\alpha_2} = \{e_{12}, e_{14}\} & E_{\alpha_3} = \{e_{11}, e_{14}\} \\ E_{\alpha_4} = \{e_{10}, e_{14}\} & E_{\alpha_5} = \{e_5, e_{14}\} & E_{\alpha_6} = \{e_2, e_{14}\} \end{array}$$
 form the six parity-check sums orthogonal on each of them.
  - Estimate  $E_{\beta_1} = \{e_{14}\}$  from the six error sums.
  - Design the type II two-step majority-logic decoder for this code.
- 8.9** For the euclidean geometry  $EG(4, 2)$  of dimension  $m = 4$  and  $s = 1$  over  $GF(2)$ :
- Find the number of 1-flats for  $r = 1$ .
  - Determine all the lines (2-flats) passing through the point  $\alpha^{14}$  but not the origin.
  - Find all the 2-flats (excluding those through the origin) that intersect on the line  $\{\alpha^{13}, \alpha^{14}\}$ .
  - Compute the incidence vectors for these 2-flats.
  - Form the parity-check sums orthogonal on  $E_{\alpha_1} = \{e_{13}, e_{14}\}$ .
- 8.10** Let  $\alpha$  be a primitive element of  $GF(2^6)$ . If we let  $\beta = \alpha^9$ , the set  $\{0, 1, \beta, \beta^2, \beta^3, \beta^4, \beta^5, \beta^6\}$  forms a subfield  $GF(2^3)$  of the Galois field  $GF(2^6)$ . Find all the 1-flats  $\{\alpha^{62} + \mu_i \alpha^{\lambda_i}, 1 \leq i \leq 8; \mu_i \in GF(2^3)\}$  in  $EG(2, 2^3)$  that pass through the point  $\alpha^{62}$ .
- 8.11** For the (1, 2)th-order EG code of length  $n = 63$ ,  $r = 1$ ,  $m = 3$ , and  $s = 2$ .  $GF(2^6)$  can be regarded as the two-dimensional  $EG(3, 2^2)$  over  $GF(2^2)$ .
- Find all the 1-flats passing through the point  $\alpha^{62}$ .
  - Determine all the 2-flats not passing through the origin that intersect on each of the 1-flats  $\{\alpha^{62} + \mu_i \alpha^0; \mu_i \in GF(2^3)\}$  obtained in part *a* and find the incidence vectors for these 2-flats.
  - Form the check sums orthogonal on  $\{e^5, e^{38}, e^{49}, e^{62}\}$ .

**8.12** Let  $m = 2$ ,  $s = 2$ , and  $r = 1$ . Then the  $(1, 2)$ th-order  $(21, 11)$  PG code can be one-step majority-logic decodable. A PG code for  $m = 2$  and  $r = 1$  also becomes a difference-set code. Let  $\alpha$  be a primitive element in  $GF(2^6)$ . The two-dimensional projective geometry over  $GF(2^2)$  is  $PG(2, 2^2)$ , which has 21 points.

- a* Find the generator polynomial of this PG code.
- b* If we put  $\beta = \alpha^{21}$ , then the set  $\{0, 1, \beta, \beta^2\}$  forms the field  $GF(2^2)$ . Determine all the five lines passing through the point  $(\alpha^{20})$  and their corresponding incidence vectors  $\mathbf{v}$  in polynomial form.
- c* Let  $\mathbf{r}$  be the received word. Form the parity-check sums  $\xi$  using  $\xi = \mathbf{v} \cdot \mathbf{r}$ .
- d* Express these parity-check sums in terms of the syndrome components.
- e* Devise the type I decoder for this PG code.

**8.13** For the  $(2, 1)$ th-order PG code where  $m = 4$ :

- a* Find the code length  $n$  of this code.
- b* What is the error-correcting capability  $t_{PG}$  of this code?
- c* Obtain all the 1-flats passing through the point  $(\alpha^{n-1})$ .
- d* Find the 2-flats that intersect on the 1-flats  $\{(\alpha^0), (\alpha^{17}), (\alpha^{30})\}$ ,  $\{(\alpha^3), (\alpha^9), (\alpha^{30})\}$ , and  $\{(\alpha^8), (\alpha^{15}), (\alpha^{39})\}$ , respectively.

## Convolutional Codes and Their Structures

In previous chapters, we presented codes and decoding techniques that apply only to block codes. With block codes, the information sequences are grouped into  $k$ -bit blocks which are independently encoded into  $n$ -bit code words. Thus, the coded sequence becomes a sequence of fixed-length independent code words. However, convolutional codes, first introduced by Elias in 1955, differ from block codes because the convolutional encoder contains memory and the  $n$  encoder outputs depend not only on the  $k$  inputs but also on  $m$  previous input blocks. Therefore, convolutional codes have a more complex structure than block codes. As a result, convolutional codes are somewhat harder to analyze but are a little easier to decode.

With block codes, algebraic properties have been very important in constructing good classes of codes and in developing decoding algorithms. This is not the case with convolutional codes. There is no convolutional decoding algorithm similar to the algebraic techniques for multiple-error correction presented in the previous chapters. However, there are three efficient decoding schemes available for convolutional codes: (1) the suboptimum tree search of sequential decoding proposed by Wozencraft in 1961 and developed by Fano in 1963; (2) threshold decoding, which was proposed by Massey in 1963 and which is less efficient but simpler to implement, and (3) the optimal trellis search utilizing the Viterbi algorithm which was developed in 1967 and which is a maximum-likelihood decoding technique. The central idea of sequential decoding is to decode the received message one digit at a time without searching the entire tree. The Viterbi algorithm is a

simple decoding procedure which determines the path having the largest accumulated metric of all possible distinct paths.

Just as for block codes, there are systematic and nonsystematic convolutional codes, the classification depending on how the information bits and parity bits are interleaved. Convolutional codes can be studied from many different points of view. For encoding, there are four different approaches: (1) the register-stage and generator-sequence approach, (2) the discrete convolution approach, (3) the scalar matrix approach, and (4) the polynomial approach. For decoding, there are three approaches: (1) the trellis approach, (2) the tree approach, which makes use of the state diagram, and (3) the majority-logic approach.

## 9.1 Convolutional Encoding

A convolutional encoder with constraint length  $n_A$  consists of an  $m$ -stage shift register (or delay line implementation) together with  $n$  modulo-2 adders and a parallel-series multiplexer for serializing encoder outputs into a single code sequence. Thus, the data input to the encoder, which is called the information sequence, is shifted into and along the shift register  $k$  bits at a time, and the encoder output sequences are obtained by taking the convolution of the information sequence with the generator sequences (the impulse response of the encoder) of the code. Thus, a convolutional encoder continuously processes information bits in a serial fashion for transmission over the channel. The generator matrix  $\mathbf{G}$  of the code is a semi-infinite matrix and has an infinite number of rows and columns, which allows the information and code-word sequences to be arbitrarily very large. However, for any practical treatment, there is a maximum allowable length  $L$  for which we often define the  $L$ th truncation of a convolutional code. Thus, the information sequence consists of  $kL$  bits, and the code sequence is represented by  $n(m + L)$  bits. Therefore, the  $L$ th truncation of an  $(n, k)$  convolutional code with memory order  $m$  can be viewed as an  $(n(m + L), kL)$  linear block code. Hence, convolutional codes in this sense may be thought of as a special class of linear block codes with superior properties which both facilitate decoding and improve performance. Generally,  $k$  and  $n$  for  $k < n$  are small integers. The constraint length is defined as  $n_A = (m + 1)n$  because it is the maximum number of encoder outputs that can be affected by a single information bit. Since a convolutional encoder generates  $n$  encoded bits for every  $k$  information bits, the code rate is  $R = k/n$ . The rate of truncated code is given by

$$R_T = \frac{kL}{n(m + L)} = R \left( 1 - \frac{m}{m + L} \right)$$



Since  $L \gg m$  in most practical situations, the rate  $R_T$  will be very close to  $R$ . This is one reason why  $R$  is called the rate of the convolutional code.

A convolutional code of rate  $R = k/n$  with constraint length  $n_A = (m + 1)n$  is described by a set of  $n$  generator sequences

$$\mathbf{g}^{(j)} = (g_{i,0}^{(j)}, g_{i,1}^{(j)}, \dots, g_{i,m-1}^{(j)}, g_{i,m}^{(j)}) \quad (9.1)$$

for the input script  $i = 1, 2, \dots, k$  and the output script  $j = 1, 2, \dots, n$ . The generator sequences can be interpreted as the  $n$  impulse responses of the encoder when the information sequence is  $\mathbf{d} = (1 \ 0 \ 0 \dots)$ . These sequences can also be represented as polynomials of degree  $m$ :

$$g_i^{(j)}(D) = g_{i,0}^{(j)} + g_{i,1}^{(j)}D + \dots + g_{i,m-1}^{(j)}D^{m-1} + g_{i,m}^{(j)}D^m \quad (9.2)$$

The connections between the shift register stages and the modulo-2 adders are conveniently described by generator sequences. Each generator sequence can be determined directly from the sequence of connections from the shift register stages to the corresponding modulo-2 adder, 1 representing a connection and 0 no connection. The components of each generator sequence consist of  $m + 1$  binary digits. If the information sequence  $\mathbf{d}^{(i)} = (d_0^{(i)}, d_1^{(i)}, d_2^{(i)}, \dots)$  enters the encoder one bit at a time, then the encoder output sequence  $\mathbf{c}^{(j)} = (c_0^{(j)}, c_1^{(j)}, c_2^{(j)}, \dots)$  can be obtained by combining the discrete convolution of the information sequence  $\mathbf{d}^{(i)}$  with the generator sequence  $\mathbf{g}_i^{(j)}$  such that

$$\mathbf{c}^{(j)} = \sum_{i=1}^k \mathbf{d}^{(i)} * \mathbf{g}_i^{(j)} \quad j = 1, 2, \dots, n \quad (9.3)$$

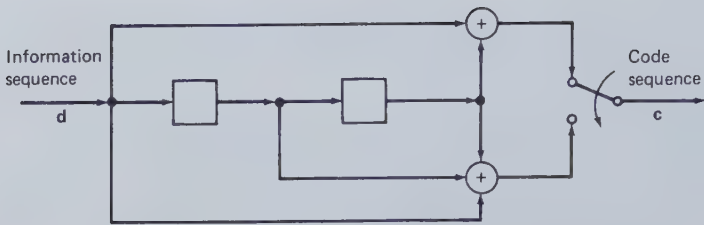
This is generally called the encoding equation. Equation (9.3) can also be written as

$$c_\lambda^{(j)} = \sum_{l=0}^m \left[ \sum_{i=1}^k d_{\lambda-l}^{(i)} g_{i,l}^{(j)} \right] \quad (9.4)$$

for all  $0 \leq l \leq \lambda$  and  $1 \leq j \leq n$ .

As a simple example, the  $R = 1/2$  convolutional encoder for a binary (2, 1) code with  $m = 2$  is shown in Fig. 9.1. Since  $m = 2$  and  $n = 2$ , this encoder consists of a 2-stage shift register together with two modulo-2 adders and a multiplexer (or commutator) for serializing the encoder outputs.

**Example 9.1 (Register-Stage and Generator-Sequence Approach)** For the encoder shown in Fig. 9.1, the generator sequences are  $\mathbf{g}_1^{(1)} = (g_{1,0}^{(1)}, g_{1,1}^{(1)}, g_{1,2}^{(1)}) = (1 \ 0 \ 1)$  and  $\mathbf{g}_1^{(2)} = (g_{1,0}^{(2)}, g_{1,1}^{(2)}, g_{1,2}^{(2)}) = (1 \ 1 \ 1)$ , respectively. Using Eq. (9.4) for  $i = 1$  and  $j = 1, 2$ , we have



**Figure 9.1** Encoder for the (2, 1) convolutional code with  $m = 2$ .

$$c_{\lambda}^{(1)} = d_{\lambda}^{(1)} g_{1,0}^{(1)} + d_{\lambda-1}^{(1)} g_{1,1}^{(1)} + d_{\lambda-2}^{(1)} g_{1,2}^{(1)}$$

$$c_{\lambda}^{(2)} = d_{\lambda}^{(1)} g_{1,0}^{(2)} + d_{\lambda-1}^{(1)} g_{1,1}^{(2)} + d_{\lambda-2}^{(1)} g_{1,2}^{(2)}$$

Substituting  $\mathbf{g}_1^{(1)} = (1 \ 0 \ 1)$  and  $\mathbf{g}_1^{(2)} = (1 \ 1 \ 1)$  into these two equations above, we get

$$c_{\lambda}^{(1)} = d_{\lambda}^{(1)} + d_{\lambda-2}^{(1)}$$

$$c_{\lambda}^{(2)} = d_{\lambda}^{(1)} + d_{\lambda-1}^{(1)} + d_{\lambda-2}^{(1)}$$

Let the information sequence of truncated length  $L = 5$  be  $\mathbf{d}^{(1)} = (1 \ 0 \ 1 \ 0 \ 1)$ . Then the corresponding code sequence for  $0 \leq \lambda \leq 6$  can be obtained as follows:

| $\lambda$ | $c_{\lambda}^{(j)}, j = 1, 2$                   |                                                                 | $(c_{\lambda}^{(1)}, c_{\lambda}^{(2)})$ |
|-----------|-------------------------------------------------|-----------------------------------------------------------------|------------------------------------------|
| 0         | $c_0^{(1)} = d_0^{(1)} = 1$                     | $c_0^{(2)} = d_0^{(1)} = 1$                                     | (1, 1)                                   |
| 1         | $c_1^{(1)} = d_1^{(1)} = 0$                     | $c_1^{(2)} = d_1^{(1)} + d_0^{(1)} = 0 + 1 = 1$                 | (0, 1)                                   |
| 2         | $c_2^{(1)} = d_2^{(1)} + d_0^{(1)} = 1 + 1 = 0$ | $c_2^{(2)} = d_2^{(1)} + d_1^{(1)} + d_0^{(1)} = 1 + 0 + 1 = 0$ | (0, 0)                                   |
| 3         | $c_3^{(1)} = d_3^{(1)} + d_1^{(1)} = 0 + 0 = 0$ | $c_3^{(2)} = d_3^{(1)} + d_2^{(1)} + d_1^{(1)} = 0 + 1 + 0 = 1$ | (0, 1)                                   |
| 4         | $c_4^{(1)} = d_4^{(1)} + d_2^{(1)} = 1 + 1 = 0$ | $c_4^{(2)} = d_4^{(1)} + d_3^{(1)} + d_2^{(1)} = 1 + 0 + 1 = 0$ | (0, 0)                                   |
| 5         | $c_5^{(1)} = d_3^{(1)} = 0$                     | $c_5^{(2)} = d_4^{(1)} + d_3^{(1)} = 1 + 0 = 1$                 | (0, 1)                                   |
| 6         | $c_6^{(1)} = d_4^{(1)} = 1$                     | $c_6^{(2)} = d_4^{(1)} = 1$                                     | (1, 1)                                   |

Thus, the two output sequences  $(c_{\lambda}^{(1)}, c_{\lambda}^{(2)})$  are multiplexed into a single sequence that is the code word to be transmitted over the channel, which is then given by

$$\begin{aligned} \mathbf{c} &= (c_0^{(1)} c_0^{(2)}, c_1^{(1)} c_1^{(2)}, \dots, c_6^{(1)} c_6^{(2)}) \\ &= (1 \ 1, \ 0 \ 1, \ 0 \ 0, \ 0 \ 1, \ 0 \ 0, \ 0 \ 1, \ 1 \ 1) \end{aligned}$$

**Example 9.2 (Discrete Convolution Approach)** Consider again the encoder shown in Fig. 9.1. The generator sequences  $\mathbf{g}_1^{(1)} = (1 \ 0 \ 1)$  and  $\mathbf{g}_1^{(2)} = (1 \ 1 \ 1)$  and the information sequence  $\mathbf{d}^{(1)} = (1 \ 0 \ 1 \ 0 \ 1)$  are the same as those in Example 9.1. Thus, the output sequences can be obtained from the following encoding equations:

$$\mathbf{c}^{(1)} = \mathbf{d}^{(1)} * \mathbf{g}_1^{(1)} = (1 \ 0 \ 1 \ 0 \ 1) * (1 \ 0 \ 1) = (1 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1)$$

$$\mathbf{c}^{(2)} = \mathbf{d}^{(1)} * \mathbf{g}_1^{(2)} = (1 \ 0 \ 1 \ 0 \ 1) * (1 \ 1 \ 1) = (1 \ 1 \ 0 \ 1 \ 0 \ 1 \ 1)$$

where  $*$  denotes convolution. Hence, the code word is obtained by multiplexing  $\mathbf{c}^{(1)}$  and  $\mathbf{c}^{(2)}$  into a single sequence as follows:

$$\mathbf{c} = (1 \ 1, \ 0 \ 1, \ 0 \ 0, \ 0 \ 1, \ 0 \ 0, \ 0 \ 1, \ 1 \ 1)$$

This is the same result as found in the previous example where the register-stage and generator-sequence approach was used.

Consider an  $(n, k)$  convolutional code with memory order  $m$ . If any arbitrary information sequence  $\mathbf{d} = (\mathbf{d}_0, \mathbf{d}_1, \mathbf{d}_2, \dots)$ , where  $\mathbf{d}_\lambda = (d_\lambda^{(1)}, d_\lambda^{(2)}, \dots, d_\lambda^{(k)})$  for  $\lambda = 0, 1, 2, \dots$ , enters the encoder, then the code-word sequence  $\mathbf{c} = (\mathbf{c}_0, \mathbf{c}_1, \mathbf{c}_2, \dots)$  in which  $\mathbf{c}_\lambda = (c_\lambda^{(1)}, c_\lambda^{(2)}, \dots, c_\lambda^{(n)})$  for  $\lambda = 0, 1, 2, \dots$ , is given by the following equation in the matrix form:

$$\mathbf{c} = \mathbf{d} \cdot \mathbf{G} \quad (9.5)$$

where  $\mathbf{G}$  is the semi-infinite generator matrix whose structure is generally given by

$$\mathbf{G} = \begin{bmatrix} \mathbf{G}_0 & \mathbf{G}_1 & \mathbf{G}_2 & \cdots & \mathbf{G}_m & & \\ & \mathbf{G}_0 & \mathbf{G}_1 & \cdots & \mathbf{G}_{m-1} & \mathbf{G}_m & 0 \\ 0 & & \mathbf{G}_0 & \cdots & \mathbf{G}_{m-2} & \mathbf{G}_{m-1} & \mathbf{G}_m \\ & & \ddots & & & & \ddots \end{bmatrix} \quad (9.6)$$

Here the other entries 0 in  $\mathbf{G}$  are all 0s. Each subgenerator matrix  $\mathbf{G}_\lambda$ ,  $\lambda = 0, 1, 2, \dots, m$  in  $\mathbf{G}$  contains  $k$  rows and  $n$  columns. However, each set of  $k$  rows of  $\mathbf{G}$  is identical to the previous set of  $k$  rows but is shifted  $n$  columns to the right. Each  $k \times n$  subgenerator matrix  $\mathbf{G}_\lambda$  in  $\mathbf{G}$  is expressed by

$$\mathbf{G}_\lambda = \begin{bmatrix} \mathbf{g}_{1,\lambda}^{(1)} & \mathbf{g}_{1,\lambda}^{(2)} & \cdots & \mathbf{g}_{1,\lambda}^{(n)} \\ \mathbf{g}_{2,\lambda}^{(1)} & \mathbf{g}_{2,\lambda}^{(2)} & \cdots & \mathbf{g}_{2,\lambda}^{(n)} \\ \vdots & \vdots & \cdots & \vdots \\ \mathbf{g}_{k,\lambda}^{(1)} & \mathbf{g}_{k,\lambda}^{(2)} & \cdots & \mathbf{g}_{k,\lambda}^{(n)} \end{bmatrix} \quad (9.7)$$

**Example 9.3 (Scalar Matrix Approach)** Consider again the encoder shown in Fig. 9.1. The first row  $[\mathbf{G}_0 \ \mathbf{G}_1 \ \mathbf{G}_2] = [1 \ 1 \ 0 \ 1 \ 1 \ 1]$  of  $\mathbf{G}$  is obtained by either interleaving  $\mathbf{g}_1^{(1)} = (1 \ 0 \ 1)$  and  $\mathbf{g}_1^{(2)} = (1 \ 1 \ 1)$ , or substituting the components of  $\mathbf{g}_1^{(1)}$  and  $\mathbf{g}_1^{(2)}$  into Eq. (9.7). If the information sequence  $\mathbf{d} = (1 \ 0 \ 1 \ 0 \ 1)$  is given, then we have

$$\mathbf{c} = \mathbf{d} \cdot \mathbf{G}$$

$$= (1 \ 0 \ 1 \ 0 \ 1) \cdot \begin{bmatrix} 1 & 1 & 0 & 1 & 1 & 1 \\ & 1 & 1 & 0 & 1 & 1 & 1 \\ & & 1 & 1 & 0 & 1 & 1 & 1 \\ & & & 1 & 1 & 0 & 1 & 1 & 1 \\ & & & & 1 & 1 & 0 & 1 & 1 & 1 \end{bmatrix}$$

$$= (1 \ 1, 0 \ 1, 0 \ 0, 0 \ 1, 0 \ 0, 0 \ 1, 1 \ 1)$$

which agrees with the results of the two previous examples. We also notice that each row ( $k = 1$ ) of  $\mathbf{G}$  is identical to the preceding row but shifted  $n = 2$  places to the right.

The polynomial approach for convolutional encoding will be considered in the following way. Since a convolutional encoder is a linear system, each sequence in the encoding equations can be replaced by a corresponding polynomial, and the convolution operation replaced by polynomial multiplication. The information sequence  $\mathbf{d}^{(i)} = (d_0^{(i)}, d_1^{(i)}, d_2^{(i)}, \dots)$  for  $i = 1, 2, \dots, k$ , is therefore represented by the polynomial representation

$$d^{(i)}(D) = d_0^{(i)} + d_1^{(i)}D + d_2^{(i)}D^2 + \dots$$

and the encoded sequence  $\mathbf{c}^{(j)} = (c_0^{(j)}, c_1^{(j)}, c_2^{(j)}, \dots)$  for  $j = 1, 2, \dots, n$ , by the polynomial

$$c^{(j)}(D) = c_0^{(j)} + c_1^{(j)}D + c_2^{(j)}D^2 + \dots$$

Thus, the generator polynomial of the code is

$$g_i^{(j)}(D) = g_{i,0}^{(j)} + g_{i,1}^{(j)}D + \dots + g_{i,m}^{(j)}D^m \quad (9.8)$$

for  $i = 1, 2, \dots, k$ , and  $j = 1, 2, \dots, n$ , which is interpreted as the encoder transfer function relating the input script  $i$  and output script  $j$ . Notice here that  $D$  denotes a delay operator. Consider any  $k$ -input,  $n$ -output encoding circuit. There is the  $k \times n$  generator matrix in polynomial form whose entries are a total of  $kn$  subgenerator polynomials, i.e.,

$$\mathbf{G}(D) = \begin{bmatrix} g_1^{(1)}(D) & g_1^{(2)}(D) & \dots & g_1^{(n)}(D) \\ g_2^{(1)}(D) & g_2^{(2)}(D) & \dots & g_2^{(n)}(D) \\ \vdots & & & \vdots \\ g_k^{(1)}(D) & g_k^{(2)}(D) & \dots & g_k^{(n)}(D) \end{bmatrix} \quad (9.9)$$

The encoding equations for the  $R = k/n$  convolutional code with memory order  $m$  can be written as

$$c^{(j)}(D) = \sum_{i=1}^k d^{(i)}(D)g_i^{(j)}(D) \quad j = 1, 2, \dots, n \quad (9.10)$$

because the convolution operation is replaced by polynomial multiplication. Thus, the encoder equations generally give

$$\mathbf{c}'(D) = \mathbf{d}(D)\mathbf{G}(D) \quad (9.11)$$

in which  $\mathbf{d}(D) = [d^{(1)}(D), d^{(2)}(D), \dots, d^{(k)}(D)]$  for the information sequence and  $\mathbf{c}'(D) = [c^{(1)}(D), c^{(2)}(D), \dots, c^{(n)}(D)]$  for the encoded sequences. After multiplexing, the code word to be transmitted becomes

$$c(D) = c^{(1)}(D^n) + Dc^{(2)}(D^n) + \dots + D^{n-1}c^{(n)}(D^n) \quad (9.12)$$

**Example 9.4 (Polynomial Matrix Approach)** Consider the  $(2, 1)$  convolutional code with  $m = 2$  whose encoder is shown in Fig. 9.1. The generator polynomials are  $g_1^{(1)}(D) = 1 + D^2$  and  $g_1^{(2)}(D) = 1 + D + D^2$ . Thus, this code is then characterized by a  $1 \times 2$  generator polynomial matrix

$$\mathbf{G}(D) = [1 + D^2, 1 + D + D^2]$$

For the information sequence  $d(D) = 1 + D^2 + D^4$ , the encoding equation is

$$\begin{aligned} \mathbf{c}'(D) &= \mathbf{d}(D)\mathbf{G}(D) \\ &= (1 + D^2 + D^4)[1 + D^2, 1 + D + D^2] \\ &= [1 + D^6, 1 + D + D^3 + D^5 + D^6] \\ &= [c^{(1)}(D), c^{(2)}(D)] \end{aligned}$$

Thus, the code sequence becomes

$$\begin{aligned} c(D) &= c^{(1)}(D^2) + Dc^{(2)}(D^2) \\ &= 1 + D^{12} + D(1 + D^2 + D^6 + D^{10} + D^{12}) \\ &= 1 + D + D^3 + D^7 + D^{11} + D^{12} + D^{13} \end{aligned}$$

Hence, we have the code word

$$\mathbf{c} = (1 \ 1, 0 \ 1, 0 \ 0, 0 \ 1, 0 \ 0, 0 \ 1, 1 \ 1)$$

corresponding to  $c(D)$  above. This result is also the same as for our previous calculations which used the register-stage and generator-sequence, discrete convolution, and scalar matrix approaches.

Most code construction for convolutional codes has been done by computer search. But most computer searches for noncatastrophic codes are limited to relatively short constraint lengths. We shall tabulate the generator polynomials for some binary, short-constraint-



length convolutional codes for several code rates. Convolutional codes for a given rate and constraint length are said to be optimal when they have the largest possible minimum free distance. A list of the good codes for code rates  $R = 1/2, 1/3, 2/3, 1/4$ , and  $3/4$  are given in Table 9.1.

TABLE 9.1 The  $R = k/n$  Convolutional Codes with Generator Sequences  $g_{(i)}$

| 1. $R = \frac{1}{2}$ |                                              |  |                                              |                      |                                              |  |                      |  |                    |                      |  |  |  |
|----------------------|----------------------------------------------|--|----------------------------------------------|----------------------|----------------------------------------------|--|----------------------|--|--------------------|----------------------|--|--|--|
| $m$                  | $\mathbf{g}^{(1)}$                           |  |                                              |                      |                                              |  | $\mathbf{g}^{(2)}$   |  |                    |                      |  |  |  |
| 2                    | 1 0 1                                        |  |                                              |                      |                                              |  | 1 1 1                |  |                    |                      |  |  |  |
| 3                    | 1 1 0 1                                      |  |                                              |                      |                                              |  | 1 1 1 1              |  |                    |                      |  |  |  |
| 4                    | 1 0 0 1 1                                    |  |                                              |                      |                                              |  | 1 1 1 0 1            |  |                    |                      |  |  |  |
| 5                    | 1 1 0 1 0 1                                  |  |                                              |                      |                                              |  | 1 0 1 1 1 1          |  |                    |                      |  |  |  |
| 6                    | 1 0 1 1 0 1 1                                |  |                                              |                      |                                              |  | 1 1 1 1 0 0 1        |  |                    |                      |  |  |  |
| 7                    | 1 1 1 0 0 1 0 1                              |  |                                              |                      |                                              |  | 1 0 0 1 1 1 1 1      |  |                    |                      |  |  |  |
| 2. $R = \frac{1}{3}$ |                                              |  |                                              |                      |                                              |  |                      |  |                    |                      |  |  |  |
| $m$                  | $\mathbf{g}^{(1)}$                           |  |                                              |                      | $\mathbf{g}^{(2)}$                           |  |                      |  | $\mathbf{g}^{(3)}$ |                      |  |  |  |
| 2                    | 1 0 1                                        |  |                                              |                      | 1 1 1                                        |  |                      |  | 1 1 1              |                      |  |  |  |
| 3                    | 1 0 1 1                                      |  |                                              |                      | 1 1 0 1                                      |  |                      |  | 1 1 1 1            |                      |  |  |  |
| 4                    | 1 0 1 0 1                                    |  |                                              |                      | 1 1 0 1 1                                    |  |                      |  | 1 1 1 1 1          |                      |  |  |  |
| 5                    | 1 0 0 1 1 1                                  |  |                                              |                      | 1 0 1 0 1 1                                  |  |                      |  | 1 1 1 1 0 1        |                      |  |  |  |
| 6                    | 1 0 1 1 0 1 1                                |  |                                              |                      | 1 1 0 0 1 0 1                                |  |                      |  | 1 1 1 1 1 0 1      |                      |  |  |  |
| 7                    | 1 0 0 1 0 1 0 1                              |  |                                              |                      | 1 1 0 1 1 0 0 1                              |  |                      |  | 1 1 1 1 0 1 1 1    |                      |  |  |  |
| 3. $R = \frac{2}{3}$ |                                              |  |                                              |                      |                                              |  |                      |  |                    |                      |  |  |  |
| $m$                  | $\mathbf{g}_1^{(1)}$<br>$\mathbf{g}_2^{(1)}$ |  | $\mathbf{g}_1^{(2)}$<br>$\mathbf{g}_2^{(2)}$ |                      | $\mathbf{g}_1^{(3)}$<br>$\mathbf{g}_2^{(3)}$ |  |                      |  |                    |                      |  |  |  |
| 1                    | 1 1                                          |  | 0 1                                          |                      | 1 1                                          |  |                      |  |                    |                      |  |  |  |
|                      | 0 1                                          |  | 1 0                                          |                      | 1 0                                          |  |                      |  |                    |                      |  |  |  |
| 2                    | 1 0 0                                        |  | 0 1 0                                        |                      | 1 1 0                                        |  |                      |  |                    |                      |  |  |  |
|                      | 0 0 1                                        |  | 1 0 0                                        |                      | 1 1 1                                        |  |                      |  |                    |                      |  |  |  |
| 2                    | 1 1 1                                        |  | 0 0 1                                        |                      | 1 0 0                                        |  |                      |  |                    |                      |  |  |  |
|                      | 0 1 0                                        |  | 1 0 1                                        |                      | 1 1 1                                        |  |                      |  |                    |                      |  |  |  |
| 3                    | 1 1 0 0                                      |  | 0 1 1 0                                      |                      | 1 1 1 0                                      |  |                      |  |                    |                      |  |  |  |
|                      | 0 0 1 1                                      |  | 1 0 0 0                                      |                      | 1 1 1 1                                      |  |                      |  |                    |                      |  |  |  |
| 3                    | 1 1 0 1                                      |  | 0 1 1 0                                      |                      | 1 1 0 1                                      |  |                      |  |                    |                      |  |  |  |
|                      | 0 1 1 0                                      |  | 1 1 0 1                                      |                      | 1 1 1 1                                      |  |                      |  |                    |                      |  |  |  |
| 4. $R = \frac{1}{4}$ |                                              |  |                                              |                      |                                              |  |                      |  |                    |                      |  |  |  |
| $m$                  | $\mathbf{g}_1^{(1)}$                         |  |                                              | $\mathbf{g}_1^{(2)}$ |                                              |  | $\mathbf{g}_1^{(3)}$ |  |                    | $\mathbf{g}_1^{(4)}$ |  |  |  |
| 2                    | 1 0 1                                        |  |                                              | 1 1 1                |                                              |  | 1 1 1                |  |                    | 1 1 1                |  |  |  |
| 3                    | 1 0 1 1                                      |  |                                              | 1 1 0 1              |                                              |  | 1 1 0 1              |  |                    | 1 1 1 1              |  |  |  |
| 4                    | 1 0 1 0 1                                    |  |                                              | 1 0 1 1 1            |                                              |  | 1 1 0 1 1            |  |                    | 1 1 1 1 1            |  |  |  |
| 5                    | 1 0 1 0 1 1                                  |  |                                              | 1 1 0 1 1 1          |                                              |  | 1 1 1 0 0 1          |  |                    | 1 1 1 1 0 1          |  |  |  |
| 6                    | 1 0 1 1 1 0 1                                |  |                                              | 1 0 1 1 1 0 1        |                                              |  | 1 1 0 0 1 1 1        |  |                    | 1 1 1 0 0 1 1        |  |  |  |
| 7                    | 1 0 0 1 1 1 0 1                              |  |                                              | 1 0 1 1 1 1 0 1      |                                              |  | 1 1 0 0 1 0 1 1      |  |                    | 1 1 1 0 1 1 1 1      |  |  |  |

| 5. $R = 3/4$ |                                                                      |                                                                      |                                                                      |                                                                      |  |  |  |  |  |  |  |  |
|--------------|----------------------------------------------------------------------|----------------------------------------------------------------------|----------------------------------------------------------------------|----------------------------------------------------------------------|--|--|--|--|--|--|--|--|
| $m$          | $\mathbf{g}_1^{(1)}$<br>$\mathbf{g}_2^{(1)}$<br>$\mathbf{g}_3^{(1)}$ | $\mathbf{g}_1^{(2)}$<br>$\mathbf{g}_2^{(2)}$<br>$\mathbf{g}_3^{(2)}$ | $\mathbf{g}_1^{(3)}$<br>$\mathbf{g}_2^{(3)}$<br>$\mathbf{g}_3^{(3)}$ | $\mathbf{g}_1^{(4)}$<br>$\mathbf{g}_2^{(4)}$<br>$\mathbf{g}_3^{(4)}$ |  |  |  |  |  |  |  |  |
| 1            | 1 0<br>0 0<br>0 0                                                    | 1 0<br>1 1<br>0 1                                                    | 1 0 0<br>0 1 0<br>1 0 1                                              | 1 0 0<br>1 0 0<br>1 0 1                                              |  |  |  |  |  |  |  |  |
| 2            | 1 1 0<br>0 0 1<br>0 0 0                                              | 0 1 0<br>1 1 0<br>0 1 0                                              | 0 1 0<br>0 0 0<br>1 0 1                                              | 1 1 0<br>1 1 1<br>1 0 1                                              |  |  |  |  |  |  |  |  |
| 2            | 1 1 0<br>0 1 1<br>0 1 0                                              | 0 0 1<br>1 0 0<br>0 1 1                                              | 0 0 0<br>0 0 1<br>1 1 1                                              | 1 1 1<br>1 1 0<br>1 0 0                                              |  |  |  |  |  |  |  |  |
| 3            | 1 1 1 0<br>0 0 1 1<br>0 0 0 1                                        | 0 1 1 0<br>1 0 1 0<br>0 0 1 0                                        | 0 1 0 0<br>0 0 0 0<br>1 1 1 1                                        | 1 0 0 0<br>1 0 1 1<br>1 0 0 0                                        |  |  |  |  |  |  |  |  |
| 3            | 1 0 0 0<br>0 0 0 1<br>0 1 1 1                                        | 0 0 1 1<br>1 1 0 1<br>0 0 0 0                                        | 0 1 1 1<br>0 1 0 0<br>1 1 0 0                                        | 1 1 0 0<br>1 1 1 0<br>1 1 0 1                                        |  |  |  |  |  |  |  |  |

## 9.2 Systematic Convolutional Codes

We have so far discussed the encoding properties of nonsystematic convolutional codes. The  $R = 1/2$  convolutional code with  $m = 2$  used in the previous several examples is a nonsystematic code because neither  $\mathbf{c}^{(1)}$  nor  $\mathbf{c}^{(2)}$  is identically equal to the information sequence  $\mathbf{d}$ . For reasons which will subsequently become clear, nonsystematic codes are usually preferred in systems which utilize Viterbi decoding. However, in order for the convolutional encoder shown in Fig. 9.1 to be systematic, this encoder should have either  $\mathbf{g}^{(1)}(D) = 1$  or  $\mathbf{g}^{(2)}(D) = 1$  so that the information sequence would appear directly in the encoder output sequence. Thus, for a convolutional code in systematic form, the first  $k$  output sequences are exactly identical to the  $k$  information sequences. One advantage of systematic codes is that encoding is somewhat simpler than for nonsystematic codes because less hardware is required. Also there is no need to provide the inverting circuit for recovering the information sequence from the code word. To begin with, consider the generator matrix

$$\mathbf{G} = \begin{bmatrix} \mathbf{G}_0 & \mathbf{G}_1 & \cdots & \mathbf{G}_{m-1} & \mathbf{G}_m \\ & \mathbf{G}_0 & \mathbf{G}_1 & \cdots & \mathbf{G}_{m-1} & \mathbf{G}_m \\ & & \mathbf{G}_0 & \mathbf{G}_1 & \cdots & \mathbf{G}_{m-1} & \mathbf{G}_m \\ & & & \ddots & & \ddots & \\ & & & & \mathbf{G}_0 & \mathbf{G}_1 & \cdots & \mathbf{G}_{m-1} & \mathbf{G}_m \end{bmatrix} \quad (9.13)$$

where each  $\mathbf{G}_\lambda$  is a  $k \times n$  submatrix whose entries are

$$\mathbf{G}_\lambda = \begin{bmatrix} \delta_{1,\lambda} & 0 & \cdots & 0 & g_{1,\lambda}^{(k+1)} & g_{1,\lambda}^{(k+2)} & \cdots & g_{1,\lambda}^{(n)} \\ 0 & \delta_{2,\lambda} & \cdots & 0 & g_{2,\lambda}^{(k+1)} & g_{2,\lambda}^{(k+2)} & \cdots & g_{2,\lambda}^{(n)} \\ \vdots & & & \vdots & & \vdots & & \vdots \\ 0 & 0 & \cdots & \delta_{k,\lambda} & g_{k,\lambda}^{(k+1)} & g_{k,\lambda}^{(k+2)} & \cdots & g_{k,\lambda}^{(n)} \end{bmatrix} \quad (9.14)$$

in which

$$\delta_{i,\lambda} = \begin{cases} 1, \lambda = 0 \\ 0, \lambda \neq 0 \end{cases} \quad 1 \leq i \leq k$$

Now, substituting Eq. (9.14) into Eq. (9.13), the generator matrix can also be written as

$$\mathbf{G} = \begin{bmatrix} \mathbf{I}\mathbf{P}_0 & \mathbf{O}\mathbf{P}_1 & \mathbf{O}\mathbf{P}_2 & \cdots & \mathbf{O}\mathbf{P}_m \\ & \mathbf{I}\mathbf{P}_0 & \mathbf{O}\mathbf{P}_1 & \cdots & \mathbf{O}\mathbf{P}_{m-1} & \mathbf{O}\mathbf{P}_m \\ & & \mathbf{I}\mathbf{P}_0 & \cdots & \mathbf{O}\mathbf{P}_{m-2} & \mathbf{O}\mathbf{P}_{m-1} & \mathbf{O}\mathbf{P}_m \\ & & \vdots & & & \vdots & \end{bmatrix} \quad (9.15)$$

where  $\mathbf{I}$  is the  $k \times k$  identity matrix,  $\mathbf{O}$  is the  $k \times k$  all-zero matrix, and  $\mathbf{P}_\lambda$  is the  $k \times (n - k)$  matrix.

Using the encoding equations  $\mathbf{c} = \mathbf{d} \cdot \mathbf{G}$ , we see that

$$(c_0^{(1)}c_0^{(2)} \cdots c_0^{(n)}, c_1^{(1)}c_1^{(2)} \cdots c_1^{(n)}, \dots) \\ = (d_0^{(1)}d_0^{(2)} \cdots d_0^{(k)},$$

$$d_1^{(1)}d_1^{(2)} \cdots d_1^{(k)}, \dots) \cdot \begin{bmatrix} \mathbf{I}\mathbf{P}_0 & \mathbf{O}\mathbf{P}_1 & \cdots & \mathbf{O}\mathbf{P}_m \\ & \mathbf{I}\mathbf{P}_0 & \mathbf{O}\mathbf{P}_1 & \cdots & \mathbf{O}\mathbf{P}_m \\ & & \mathbf{I}\mathbf{P}_0 & \mathbf{O}\mathbf{P}_1 & \cdots & \mathbf{O}\mathbf{P}_m \\ & & \vdots & & & \vdots \end{bmatrix}$$

which we can separate into the following two parts:

$$c_\lambda^{(j)} = d_\lambda^{(j)} \quad j = 1, 2, \dots, k \quad (9.16)$$

$$\text{and} \quad c_\lambda^{(j)} = \sum_{i=1}^k d_\lambda^{(i)} g_{i,0}^{(j)} + \sum_{i=1}^k d_{\lambda-1}^{(i)} g_{i,1}^{(j)} + \cdots + \sum_{i=1}^k d_{\lambda-m}^{(i)} g_{i,m}^{(j)} \quad (9.17)$$

for  $j = k + 1, k + 2, \dots, n$ . Equation (9.17) can also be reduced to a compact form:

$$c_{\lambda}^{(j)} = \sum_{\mu=0}^m \sum_{i=1}^k d_{\lambda-\mu}^{(i)} g_{i,\mu}^{(j)} \quad j = k+1, k+2, \dots, n \quad (9.18)$$

or

$$c_{\lambda}^{(j)} = \sum_{i=1}^k \sum_{\mu=0}^m d_{\lambda-\mu}^{(i)} g_{i,\mu}^{(j)} \quad j = k+1, k+2, \dots, n \quad (9.19)$$

Encoding of an  $(n, k)$  systematic convolutional code can be easily implemented by using shift registers in connection with the set of subgenerators which specifies the code. We obtain a type I encoder based on Eqs. (9.16) and (9.18) as shown in Fig. 9.2. A type II encoder based on Eqs. (9.16) and (9.19) is shown in Fig. 9.3. These two encoding circuits were developed and devised by Massey. Since both types of encoders accept the information sequence in parallel form, a serial-to-parallel multiplexer is required at the input of either circuit and a parallel-to-serial multiplexer is also required at the output of the encoder to serialize the output sequence into the code sequence for transmission.

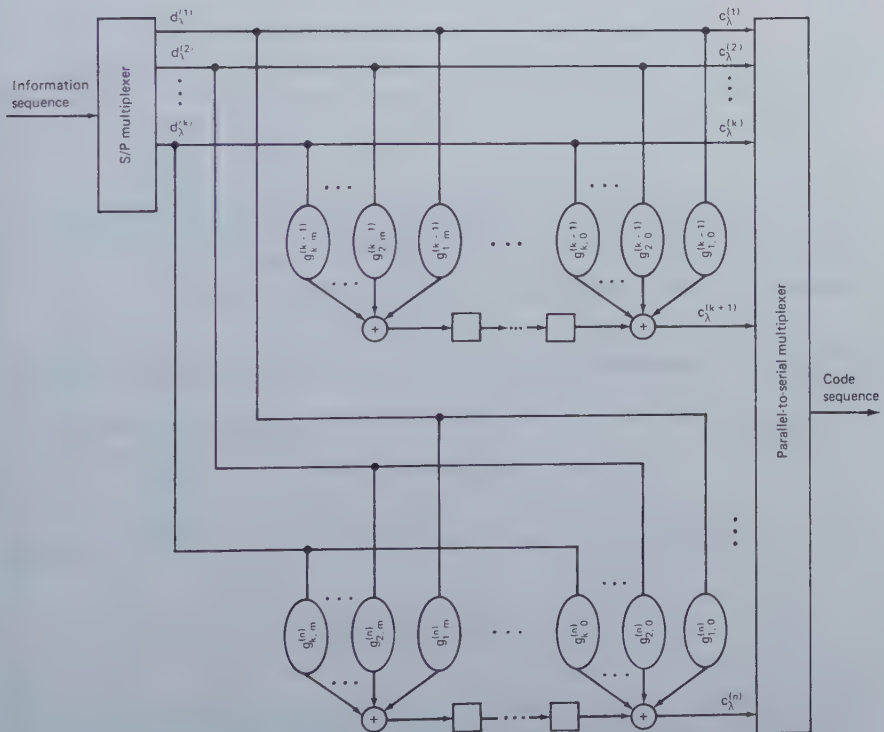
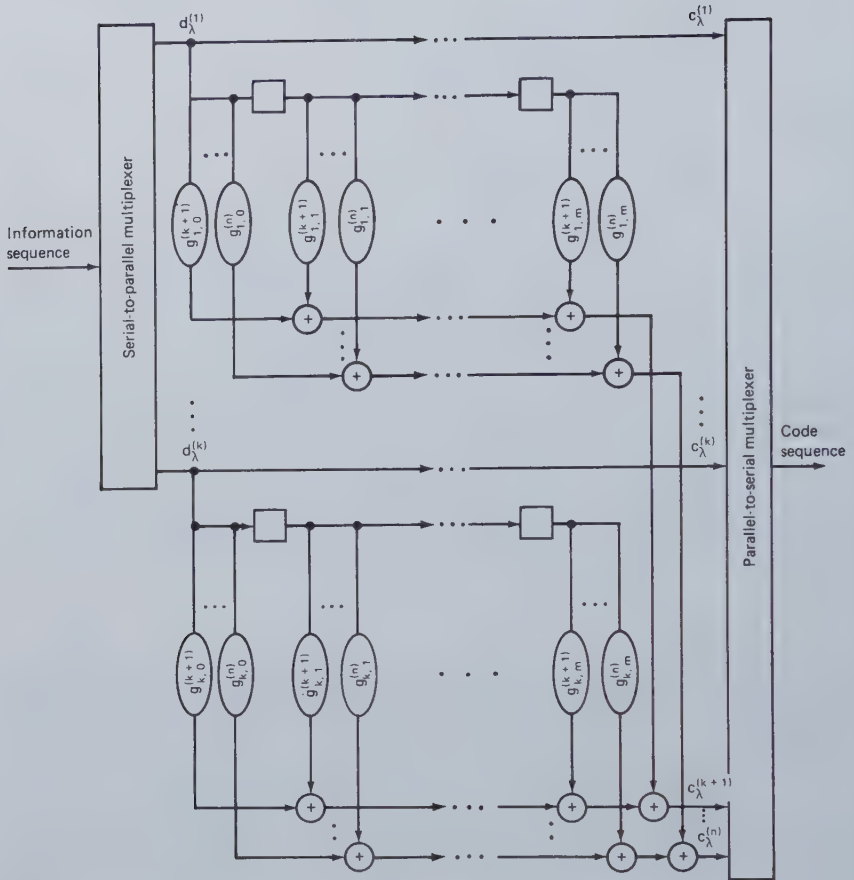


Figure 9.2 A type I encoder for a  $(n, k)$  systematic convolutional code.



**Figure 9.3** A type II encoder for a  $(n, k)$  systematic convolutional code.

The  $k \times n$  generator matrix in polynomial form is specified as follows:

$$\mathbf{G}(D) = \begin{bmatrix} 1 & 0 & \cdots & 0 & g_1^{(k+1)}(D) & g_1^{(k+2)}(D) & \cdots & g_1^{(n)}(D) \\ 0 & 1 & \cdots & 0 & g_2^{(k+1)}(D) & g_2^{(k+2)}(D) & \cdots & g_2^{(n)}(D) \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & \cdots & 1 & g_k^{(k+1)}(D) & g_k^{(k+2)}(D) & \cdots & g_k^{(n)}(D) \end{bmatrix} \quad (9.20)$$

Since  $\mathbf{G}(D)$  is in systematic form, the corresponding  $(n - k) \times n$  parity-check matrix  $\mathbf{H}(D)$  may take the following form:



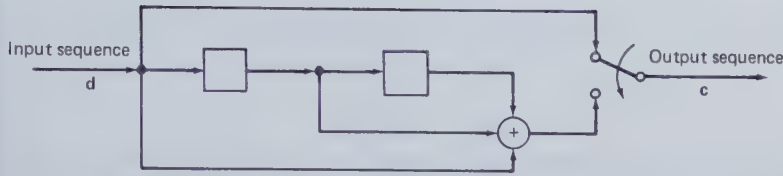


Figure 9.4 The (2, 1) systematic convolutional encoder with  $m = 2$ .

$$\mathbf{H}(D) = \begin{bmatrix} g_1^{(k+1)}(D) & \cdots & g_k^{(k+1)}(D) & 1 & 0 & \cdots & 0 \\ g_1^{(k+2)}(D) & \cdots & g_k^{(k+2)}(D) & 0 & 1 & \cdots & 0 \\ \vdots & & \vdots & \vdots & \vdots & \vdots & \vdots \\ g_1^{(n)}(D) & \cdots & g_k^{(n)}(D) & 0 & 0 & \cdots & 1 \end{bmatrix} \quad (9.21)$$

If any sequence in the row space of  $\mathbf{G}(D)$  is orthogonal to the rows of  $\mathbf{H}(D)$ , i.e.,  $\mathbf{G}(D) \cdot \mathbf{H}^T(D) = \mathbf{0}$ , we can say that  $\mathbf{c}(D) \cdot \mathbf{H}^T(D) = \mathbf{0}$  if and only if  $\mathbf{c}(D)$  is a code polynomial. Note that only  $k(n - k)$  generator sequences need be specified to define a systematic code. Therefore, systematic codes normally require fewer shift register stages and less hardware than nonsystematic codes.

Consider the (2, 1) systematic convolutional encoder with  $m = 2$  shown in Fig. 9.4. With this systematic code, we demonstrate several methods for encoding the information sequence in what follows.

**Example 9.5 (Register-Stage and Generator-Sequence Approach)** From the encoder of Fig. 9.4, we obtain the generator sequences  $\mathbf{g}_1^{(1)} = (1 \ 0 \ 0)$  and  $\mathbf{g}_1^{(2)} = (1 \ 1 \ 1)$ . Using Eqs. (9.16) and (9.17) for  $i = 1$  and  $j = 1, 2$ , we get

$$c_\lambda^{(1)} = d_\lambda^{(1)}$$

$$c_\lambda^{(2)} = d_\lambda^{(1)} + d_{\lambda-1}^{(1)} + d_{\lambda-2}^{(1)}$$

For the information sequence  $\mathbf{d}^{(1)} = (1 \ 0 \ 1 \ 0 \ 1)$ , the corresponding code sequence can be obtained as follows:

| $\lambda$ | $c_\lambda^{(j)}, j = 1, 2$ |                             | $c_\lambda^{(1)}$ | $c_\lambda^{(2)}$ |
|-----------|-----------------------------|-----------------------------|-------------------|-------------------|
| 0         | $c_0^{(1)} = 1$             | $c_0^{(2)} = 1$             | 1                 | 1                 |
| 1         | $c_1^{(1)} = 0$             | $c_1^{(2)} = 0 + 1 = 1$     | 0                 | 1                 |
| 2         | $c_2^{(1)} = 1$             | $c_2^{(2)} = 1 + 0 + 1 = 0$ | 1                 | 0                 |
| 3         | $c_3^{(1)} = 0$             | $c_3^{(2)} = 0 + 1 + 0 = 1$ | 0                 | 1                 |
| 4         | $c_4^{(1)} = 1$             | $c_4^{(2)} = 1 + 0 + 1 = 0$ | 1                 | 0                 |
| 5         | $c_5^{(1)} = 0$             | $c_5^{(2)} = 1 + 0 = 1$     | 0                 | 1                 |
| 6         | $c_6^{(1)} = 0$             | $c_6^{(2)} = 1$             | 0                 | 1                 |

Thus, the serialized sequence of code word to be transmitted is as follows:

$$\begin{array}{ccccccc} \mathbf{c} = (1 & 1, & 0 & 1, & 1 & 0, & 0 & 1, & 1 & 0, & 0 & 1, & 0 & 1) \\ & \uparrow & & \uparrow & & \uparrow & & \uparrow & & \uparrow & & & & \end{array}$$

We see after multiplexing that the information sequence is (1 0 1 0 1) as indicated by the arrows, and the parity sequence is (1 1 0 1 0 1 1).

**Example 9.6 (Discrete Convolution Approach)** Again, use the systematic encoder shown in Fig. 9.4, and use the generator sequences  $\mathbf{g}_1^{(1)} = (1 \ 0 \ 0)$  and  $\mathbf{g}_1^{(2)} = (1 \ 1 \ 1)$ , and the information sequence  $\mathbf{d}^{(1)} = (1 \ 0 \ 1 \ 0 \ 1)$ . The discrete convolution of  $\mathbf{d}^{(i)}$  by  $\mathbf{g}^{(j)}$  yields

$$\mathbf{c}^{(1)} = \mathbf{d}^{(1)} * \mathbf{g}^{(1)} = (1 \ 0 \ 1 \ 0 \ 1) * (1 \ 0 \ 0) = (1 \ 0 \ 1 \ 0 \ 1 \ 0 \ 0)$$

which is the replica of the information sequence, and

$$\mathbf{c}^{(2)} = \mathbf{d}^{(1)} * \mathbf{g}_1^{(2)} = (1 \ 0 \ 1 \ 0 \ 1) * (1 \ 1 \ 1) = (1 \ 1 \ 0 \ 1 \ 0 \ 1 \ 1)$$

which is the parity sequence. Therefore, the code sequence after multiplexing through the commutator is

$$\begin{array}{ccccccc} \mathbf{c} = (1 & 1, & 0 & 1, & 1 & 0, & 0 & 1, & 1 & 0, & 0 & 1, & 0 & 1) \\ & \uparrow & & \uparrow & & \uparrow & & \uparrow & & \uparrow & & & & \end{array}$$

This is exactly the same result as in the previous example.

**Example 9.7 (Scalar Matrix Approach)** Again use the systematic encoder shown in Fig. 9.4. The first row of  $\mathbf{G}$ ,  $(\mathbf{G}_0 \ \mathbf{G}_1 \ \mathbf{G}_2) = (1 \ 1 \ 0 \ 1 \ 0 \ 1)$ , is obtained by interleaving  $\mathbf{g}_1^{(1)} = (1 \ 0 \ 0)$  and  $\mathbf{g}_1^{(2)} = (1 \ 1 \ 1)$ . When  $\mathbf{d} = (1 \ 0 \ 1 \ 0 \ 1)$  is given, we have

$$\begin{aligned} \mathbf{c} &= \mathbf{d} \cdot \mathbf{G} \\ &= (1 \ 0 \ 1 \ 0 \ 1) \cdot \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 1 & & & \\ & 1 & 1 & 0 & 1 & 0 & 1 & & \\ & & 1 & 1 & 0 & 1 & 0 & 1 & \\ & & & 1 & 1 & 0 & 1 & 0 & 1 \\ & & & & 1 & 1 & 0 & 1 & 0 & 1 \end{bmatrix} \\ &= (1 \ 1, \ 0 \ 1, \ 1 \ 0, \ 0 \ 1, \ 1 \ 0, \ 0 \ 1, \ 0 \ 1, \ 0 \ 1) \\ &\quad \uparrow \quad \uparrow \quad \uparrow \quad \uparrow \quad \uparrow \end{aligned}$$

which is identical to the result obtained in our previous two examples.

**Example 9.8 (Polynomial Matrix Approach)** Consider again the  $R = \frac{1}{2}$  systematic code with  $m = 2$ . The systematic encoder of Fig. 9.4 requires only one modulo-2 adder with three inputs, but one branch is replaced by a direct line from the input to the multiplexer (or commutator) through which the informa-

tion sequence will be carried. Hence the generator polynomials are  $g_1^{(1)}(D) = 1$  and  $g_1^{(2)}(D) = 1 + D + D^2$ . This systematic code is then characterized by the following generator matrix in polynomial form:

$$\mathbf{G}(D) = [1, 1 + D + D^2]$$

For the information sequence  $\mathbf{d}(D) = 1 + D^2 + D^4$ , the encoding equation is

$$\begin{aligned}\mathbf{c}'(D) &= \mathbf{d}(D)\mathbf{G}(D) \\ &= (1 + D^2 + D^4)[1, 1 + D + D^2] \\ &= [1 + D^2 + D^4, 1 + D + D^3 + D^5 + D^6]\end{aligned}$$

where  $c^{(1)}(D) = 1 + D^2 + D^4$  and  $c^{(2)}(D) = 1 + D + D^3 + D^5 + D^6$ . Therefore, the code polynomial can be expressed by

$$\begin{aligned}c(D) &= c^{(1)}(D^2) + Dc^{(2)}(D^2) \\ &= 1 + D^4 + D^8 + D(1 + D^2 + D^6 + D^{10} + D^{12}) \\ &= 1 + D + D^3 + D^4 + D^7 + D^8 + D^{11} + D^{13}\end{aligned}$$

and its code sequence is

$$\begin{array}{cccccccc} \mathbf{c} = (1 & 1, & 0 & 1, & 1 & 0, & 0 & 1, & 1 & 0, & 0 & 1, & 0 & 1) \\ & \uparrow & & \uparrow & & \uparrow & & \uparrow & & \uparrow & & & & \end{array}$$

This result also agrees with the previous calculations of Examples 9.5, 9.6, and 9.7.

### 9.3 Nonsystematic Convolutional Codes

Good convolutional codes have as large a free distance as possible; at high signal-to-noise ratios these codes are optimum. Some nonsystematic convolutional codes have a superior distance structure. Therefore, nonsystematic codes are sometimes preferred over systematic codes because a larger free distance can be achieved for a given value of  $m$ . This leads to a performance advantage which increases with code rate. Since code sequences of nonsystematic codes do not display the information directly, an inverter is required to recover the information sequence in the absence of errors. Nonsystematic convolutional codes are commonly used in Viterbi decoders.

A convolutional code is linear and can be described by a semi-infinite generator matrix. A generator matrix for a convolutional code is more cumbersome than a generator matrix for a block code. The generator matrix characterizing the convolutional code has the form

$$\mathbf{G} = \begin{bmatrix} \mathbf{G}_0 & \mathbf{G}_1 & \mathbf{G}_2 & \cdots & \cdots & \mathbf{G}_m \\ & \mathbf{G}_0 & \mathbf{G}_1 & \mathbf{G}_2 & \cdots & \mathbf{G}_{m-1} & \mathbf{G}_m \\ & & \mathbf{G}_0 & \mathbf{G}_1 & \cdots & \mathbf{G}_{m-2} & \mathbf{G}_{m-1} & \mathbf{G}_m \\ & & & \ddots & & & & \ddots \\ & & & & \mathbf{G}_0 & \mathbf{G}_1 & \mathbf{G}_2 & \cdots & \mathbf{G}_{m-1} & \mathbf{G}_m \\ & & & & & \mathbf{G}_0 & \mathbf{G}_1 & \cdots & \mathbf{G}_{m-2} & \mathbf{G}_{m-1} & \mathbf{G}_m \\ & & & & & & \ddots & & & & \ddots \end{bmatrix} \quad (9.22)$$

where the matrix continues indefinitely down and to the right. Except for the diagonal band of  $m$  nonzero submatrices, all entries in blank areas in  $\mathbf{G}$  are 0. The submatrix  $\mathbf{G}_\lambda$ ,  $\lambda = 1, 2, \dots, m$ , in the diagonal zone is expressed as

$$\mathbf{G}_\lambda = \begin{bmatrix} \mathbf{g}_{1,\lambda}^{(1)} & \mathbf{g}_{1,\lambda}^{(2)} & \cdots & \mathbf{g}_{1,\lambda}^{(n)} \\ \mathbf{g}_{2,\lambda}^{(1)} & \mathbf{g}_{2,\lambda}^{(2)} & \cdots & \mathbf{g}_{2,\lambda}^{(n)} \\ \vdots & & & \vdots \\ \mathbf{g}_{k,\lambda}^{(1)} & \mathbf{g}_{k,\lambda}^{(2)} & \cdots & \mathbf{g}_{k,\lambda}^{(n)} \end{bmatrix} \quad \mathbf{g}_{i,\lambda}^{(j)} \in GF(2) \text{ for } 1 \leq \lambda \leq m \quad (9.23)$$

Let the information sequence be  $\mathbf{d} = (d_0^{(1)} d_0^{(2)} \cdots d_0^{(k)}, d_1^{(1)} d_1^{(2)} \cdots d_1^{(k)}, \dots, d_\lambda^{(1)} d_\lambda^{(2)} \cdots d_\lambda^{(k)}, \dots)$ . Since the code is linear, the code word  $\mathbf{c}$  corresponding to any information sequence  $\mathbf{d}$  can be found from the following encoding equation:

$$\begin{aligned} \mathbf{c} &= \mathbf{d} \cdot \mathbf{G} \\ &= (d_0^{(1)} d_0^{(2)} \cdots d_0^{(k)}, \dots, d_\lambda^{(1)} d_\lambda^{(2)} \cdots d_\lambda^{(k)}, \dots) \cdot \begin{bmatrix} \mathbf{G}_0 & \mathbf{G}_1 & \cdots & \mathbf{G}_m \\ & \mathbf{G}_0 & \cdots & \mathbf{G}_{m-1} & \mathbf{G}_m \\ & & \cdots & \mathbf{G}_{m-2} & \mathbf{G}_{m-1} & \mathbf{G}_m \end{bmatrix} \\ &= (c_0^{(1)} c_0^{(2)} \cdots c_0^{(n)}, c_1^{(1)} c_1^{(2)} \cdots c_1^{(n)}, \dots, c_\lambda^{(1)} c_\lambda^{(2)} \cdots c_\lambda^{(n)}, \dots) \end{aligned}$$

from which code digits in the code sequence  $\mathbf{c}$  can be obtained as

$$c_\lambda^{(j)} = \sum_{i=1}^k d_\lambda^{(i)} g_{i,0}^{(j)} + \sum_{i=1}^k d_{\lambda-1}^{(i)} g_{i,1}^{(j)} + \cdots + \sum_{i=1}^k d_{\lambda-m}^{(i)} g_{i,m}^{(j)} \quad (9.24)$$

Thus,  $c_\lambda^{(j)}$  for  $j = 1, 2, \dots, n$  can be expressed by either

$$c_{\lambda}^{(j)} = \sum_{l=0}^m \sum_{i=1}^k d_{\lambda-l}^{(i)} g_{i,l}^{(j)} \quad j = 1, 2, \dots, n \quad (9.25)$$

or

$$c_{\lambda}^{(j)} = \sum_{i=1}^k \sum_{l=0}^m d_{\lambda-l}^{(i)} g_{i,l}^{(j)} \quad j = 1, 2, \dots, n \quad (9.26)$$

We obtain a type I nonsystematic convolutional encoder based on Eq. (9.25), which is shown in Fig. 9.5, and a type II nonsystematic convolutional encoder based on Eq. (9.26), which is shown in Fig. 9.6.

Consider a (3, 2) convolutional encoder with  $m = 2$ , as shown in Fig. 9.7, for encoding  $\mathbf{d}$  to  $\mathbf{c}$  via the various methods that follow. From Fig. 9.7, we have the generator sequences:  $\mathbf{g}_1^{(1)} = (1 \ 0 \ 0)$ ,  $\mathbf{g}_1^{(2)} = (0 \ 1 \ 0)$ ,  $\mathbf{g}_1^{(3)} = (1 \ 1 \ 0)$ ,  $\mathbf{g}_2^{(1)} = (0 \ 0 \ 1)$ ,  $\mathbf{g}_2^{(2)} = (1 \ 0 \ 0)$ , and  $\mathbf{g}_2^{(3)} = (1 \ 1 \ 1)$ . The information sequences are  $\mathbf{d}^{(1)} = (1 \ 0 \ 1)$  and  $\mathbf{d}^{(2)} = (0 \ 1 \ 0)$ .

**Example 9.9** Since  $k = 2$ , the encoder of Fig. 9.7 consists of a S/P multiplexer with two levels of shift registers that are coupled with  $n = 3$  modulo-2 adders and a P/S multiplexer with three output terminals. From Eq. (9.25), the outputs of the modulo-2 adders are expressed as

$$\begin{aligned} c_{\lambda}^{(j)} &= \sum_{l=0}^2 \left( \sum_{i=1}^2 d_{\lambda-l}^{(i)} g_{i,l}^{(j)} \right) \quad j = 1, 2, 3 \\ &= d_{\lambda}^{(1)} g_{1,0}^{(j)} + d_{\lambda}^{(2)} g_{2,0}^{(j)} + d_{\lambda-1}^{(1)} g_{1,1}^{(j)} + d_{\lambda-1}^{(2)} g_{2,1}^{(j)} + d_{\lambda-2}^{(1)} g_{1,2}^{(j)} + d_{\lambda-2}^{(2)} g_{2,2}^{(j)} \end{aligned}$$

from which we have

$$\begin{aligned} c_{\lambda}^{(1)} &= d_{\lambda}^{(1)} && + d_{\lambda-2}^{(2)} \\ c_{\lambda}^{(2)} &= && d_{\lambda}^{(2)} + d_{\lambda-1}^{(1)} \\ c_{\lambda}^{(3)} &= d_{\lambda}^{(1)} + d_{\lambda}^{(2)} + d_{\lambda-1}^{(1)} + d_{\lambda-1}^{(2)} + d_{\lambda-2}^{(2)} \end{aligned}$$

Consequently,

$$\begin{array}{lllll} \lambda = 0 & c_0^{(1)} = 1 & c_0^{(2)} = 0 & c_0^{(3)} = 1 & (1 \ 0 \ 1) \\ \lambda = 1 & c_1^{(1)} = 0 & c_1^{(2)} = 0 & c_1^{(3)} = 0 & (0 \ 0 \ 0) \\ \lambda = 2 & c_2^{(1)} = 1 & c_2^{(2)} = 0 & c_2^{(3)} = 0 & (1 \ 0 \ 0) \\ \lambda = 3 & c_3^{(1)} = 1 & c_3^{(2)} = 1 & c_3^{(3)} = 0 & (1 \ 1 \ 0) \\ \lambda = 4 & c_4^{(1)} = 0 & c_4^{(2)} = 0 & c_4^{(3)} = 0 & (0 \ 0 \ 0) \end{array}$$

Thus, after the output sequences are interlaced by the P/S multiplexer, the code sequence to be sent to the channel is

$$\mathbf{c} = (1 \ 0 \ 1, 0 \ 0 \ 0, 1 \ 0 \ 0, 1 \ 1 \ 0, 0 \ 0 \ 0)$$



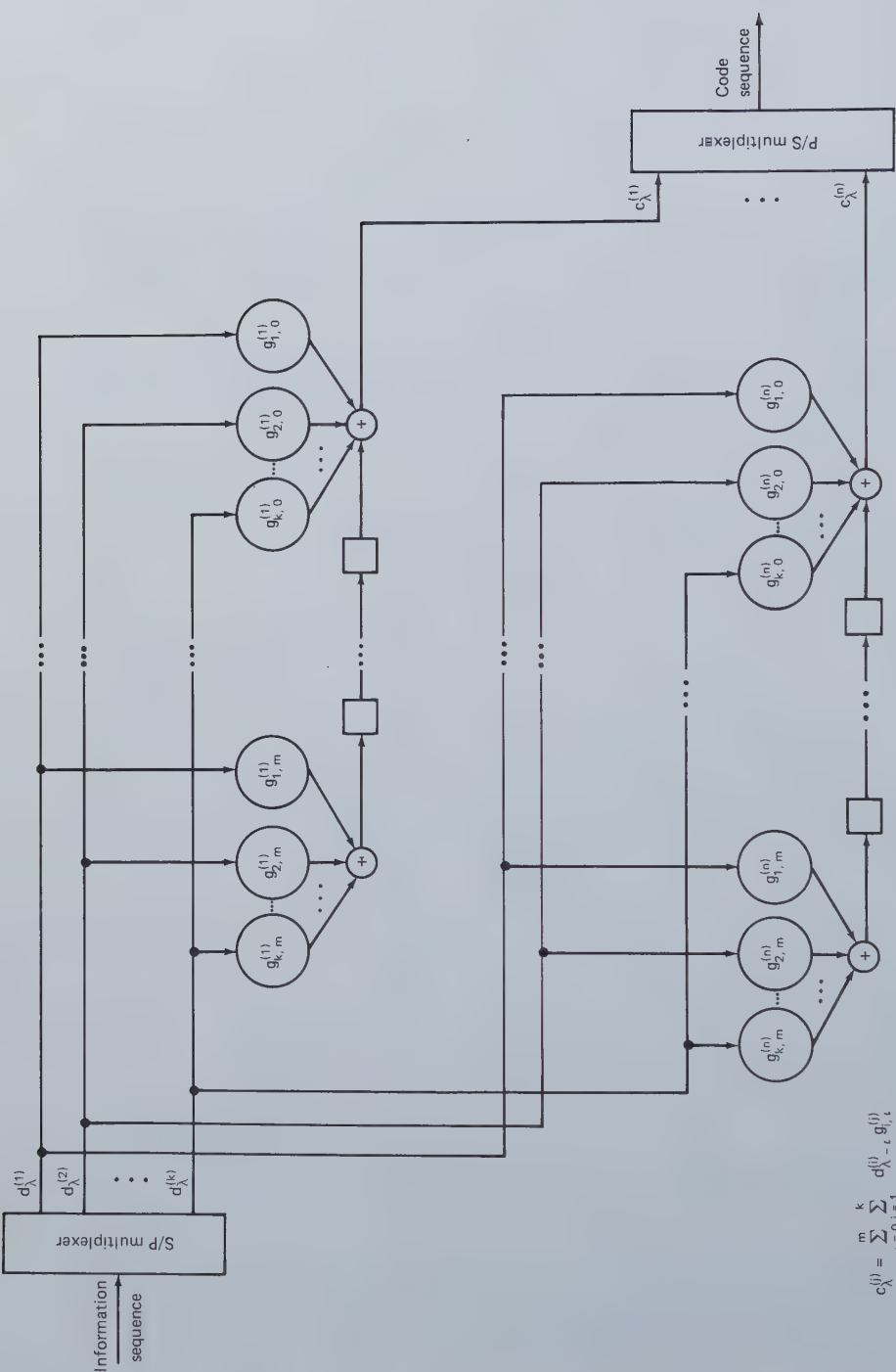
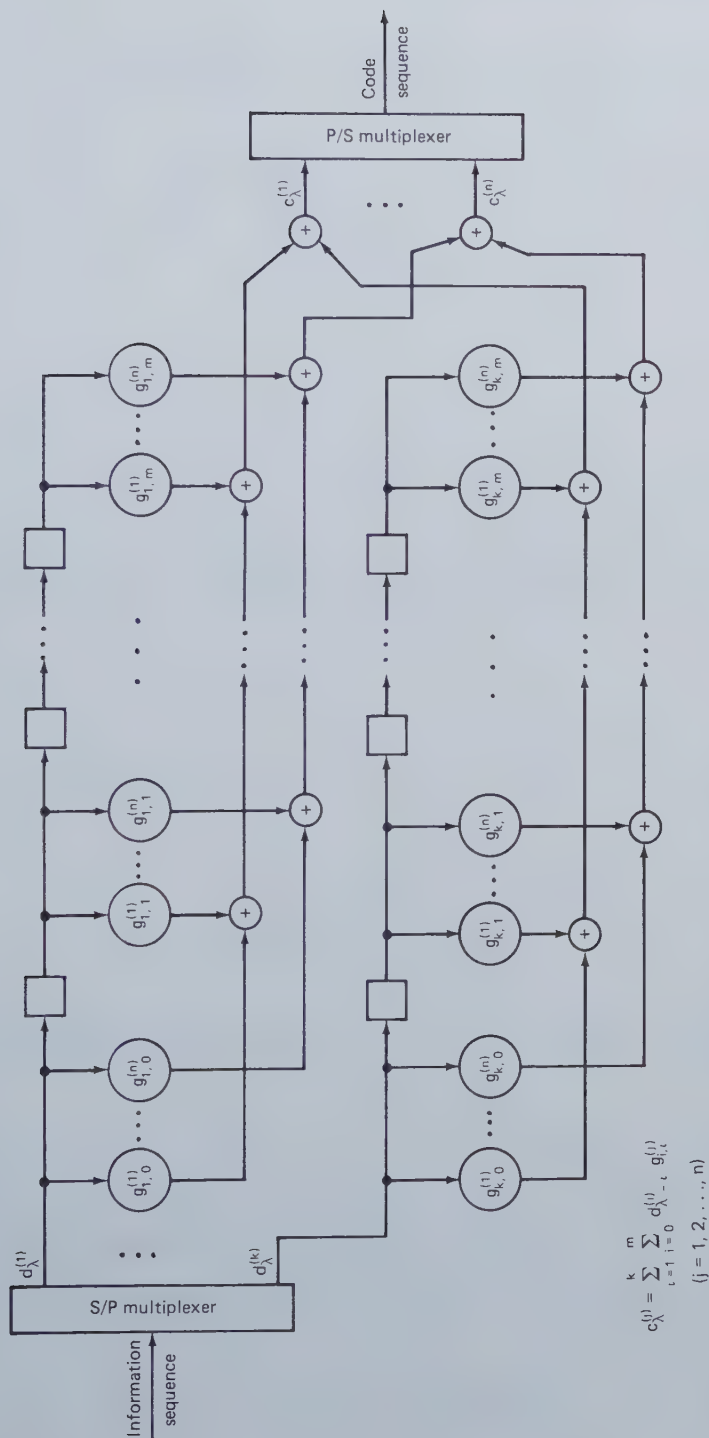


Figure 9.5 A type I encoder for a  $(n, k)$  nonsystematic convolutional code.



**Figure 9.6** A type II encoder for a  $(n, k)$  nonsystematic convolutional code.

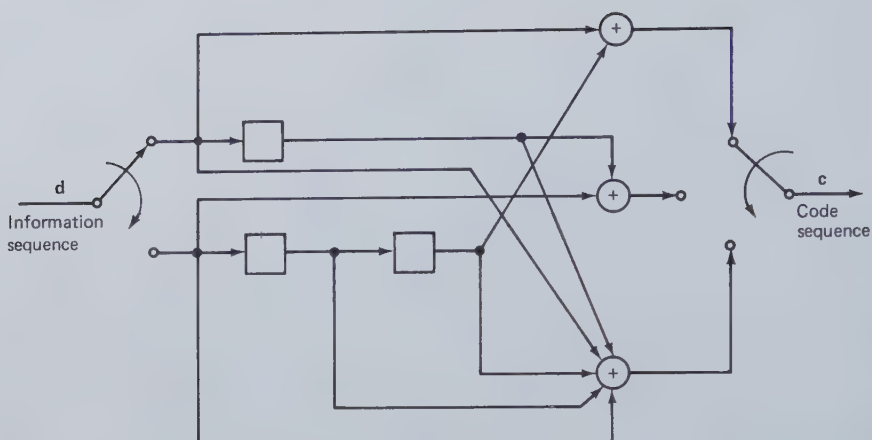


Figure 9.7 A (3, 2) nonsystematic convolutional encoder with  $m = 2$ .

**Example 9.10 (Discrete Convolution Approach)** Use Fig. 9.7 again. Taking the respective discrete convolution  $\mathbf{d}^{(i)}$  by  $\mathbf{g}_i^{(j)}$ , we have

$$\begin{aligned}\mathbf{c}^{(1)} &= \mathbf{d}^{(1)} * \mathbf{g}_1^{(1)} + \mathbf{d}^{(2)} * \mathbf{g}_2^{(1)} \\ &= (1 \ 0 \ 1) * (1 \ 0 \ 0) + (0 \ 1 \ 0) * (0 \ 0 \ 1) \\ &= (1 \ 0 \ 1 \ 1 \ 0)\end{aligned}$$

$$\begin{aligned}\mathbf{c}^{(2)} &= \mathbf{d}^{(1)} * \mathbf{g}_1^{(2)} + \mathbf{d}^{(2)} * \mathbf{g}_2^{(2)} \\ &= (1 \ 0 \ 1) * (0 \ 1 \ 0) + (0 \ 1 \ 0) * (1 \ 0 \ 0) \\ &= (0 \ 0 \ 0 \ 1 \ 0)\end{aligned}$$

$$\begin{aligned}\mathbf{c}^{(3)} &= \mathbf{d}^{(1)} * \mathbf{g}_1^{(3)} + \mathbf{d}^{(2)} * \mathbf{g}_2^{(3)} \\ &= (1 \ 0 \ 1) * (1 \ 1 \ 0) + (0 \ 1 \ 0) * (1 \ 1 \ 1) \\ &= (1 \ 0 \ 0 \ 0 \ 0)\end{aligned}$$

After  $\mathbf{c}^{(1)}$ ,  $\mathbf{c}^{(2)}$ , and  $\mathbf{c}^{(3)}$  are interlaced in that order through the multiplexer, the code word is

$$\mathbf{c} = (1 \ 0 \ 1, 0 \ 0 \ 0, 1 \ 0 \ 0, 1 \ 1 \ 0, 0 \ 0 \ 0)$$

**Example 9.11 (Scalar Matrix Approach)** Since the two input sequences are  $\mathbf{d}^{(1)} = (1 \ 0 \ 1)$  and  $\mathbf{d}^{(2)} = (0 \ 1 \ 0)$ , the information sequence  $\mathbf{d} = (1 \ 0 \ 0 \ 1 \ 1 \ 0)$  enters the encoder  $k = 2$  bits at a time. There are three generator sequences  $\mathbf{g}_i^{(j)}$  for  $i = 1, 2$  and  $j = 1, 2, 3$ , which correspond to the two input sequences. Each set of  $k = 2$  rows of  $\mathbf{G}$  is formed by interlacing  $\mathbf{g}_i^{(j)}$  one after another. Hence, the first row  $(1 \ 0 \ 1 \ 0 \ 1 \ 1 \ 0 \ 0 \ 0)$  is formed by interlacing  $\mathbf{g}_1^{(1)}$ ,  $\mathbf{g}_1^{(2)}$ , and  $\mathbf{g}_1^{(3)}$ , whereas the second row  $(0 \ 1 \ 1 \ 0 \ 0 \ 1 \ 1 \ 0 \ 1)$  is formed from  $\mathbf{g}_2^{(1)}$ ,  $\mathbf{g}_2^{(2)}$ , and  $\mathbf{g}_2^{(3)}$ . Thus, the set which constitutes the first two rows of  $\mathbf{G}$  is

$$\begin{bmatrix} 1 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 1 \end{bmatrix}$$

The scalar generator matrix is therefore obtained by sequentially shifting the set of two rows to the right by  $n = 3$  digits. Finally, the code word is given by

$$\mathbf{c} = \mathbf{d} \cdot \mathbf{G}$$

$$= (1 \ 0 \ 0 \ 1 \ 1 \ 0) \cdot \begin{bmatrix} 1 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 1 \\ & & 1 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ & & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 1 \\ & & & 1 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ & & & & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 1 \end{bmatrix}$$

$$= (1 \ 0 \ 1, \ 0 \ 0 \ 0, \ 1 \ 0 \ 0, \ 1 \ 1 \ 0, \ 0 \ 0 \ 0)$$

which is the same result as obtained from our two previous calculations.

**Example 9.12 (Polynomial Matrix Approach)** Again use the  $(3, 2)$  convolutional encoder with  $m = 2$  found in Fig. 9.7. From Eq. (9.9), the  $2 \times 3$  generator polynomial matrix (or transfer function matrix) is

$$\mathbf{G}(D) = \begin{bmatrix} 1 & D & 1 + D \\ D^2 & 1 & 1 + D + D^2 \end{bmatrix}$$

After utilizing the encoding equation  $\mathbf{c}'(D) = \mathbf{d}(D) \cdot \mathbf{G}(D)$  for  $d^{(1)}(D) = 1 + D^2$  and  $d^{(2)}(D) = D$ , we have

$$\begin{aligned} \mathbf{c}'(D) &= [1 + D^2, D] \begin{bmatrix} 1 & D & 1 + D \\ D^2 & 1 & 1 + D + D^2 \end{bmatrix} \\ &= [1 + D^2 + D^3, D^3, 1] \\ &= [c^{(1)}(D), c^{(2)}(D), c^{(3)}(D)] \end{aligned}$$

Hence we have  $c^{(1)}(D) = 1 + D^2 + D^3$ ,  $c^{(2)}(D) = D^3$ , and  $c^{(3)}(D) = 1$ . Thus, after multiplexing, the code word becomes

$$\begin{aligned} c(D) &= c^{(1)}(D^3) + Dc^{(2)}(D^3) + D^2c^{(3)}(D^3) \\ &= 1 + D^2 + D^6 + D^9 + D^{10} \end{aligned}$$

or  $\mathbf{c} = (1 \ 0 \ 1, \ 0 \ 0 \ 0, \ 1 \ 0 \ 0, \ 1 \ 1 \ 0, \ 0 \ 0 \ 0)$

Again, this is the same result as in the previous examples.

Lastly, the code sequence  $c(D)$  can also be expressed in terms of the input sequence by the following formula:

$$c(D) = \sum_{i=1}^k d^{(i)}(D^n) g_i(D) \quad (9.27)$$

where

$$g_i(D) = g_i^{(1)}(D^n) + Dg_i^{(2)}(D^n) + \cdots + D^{n-1}g_i^{(n)}(D^n) \quad (9.28)$$

We consider the following example, which uses Eq. (9.27).

**Example 9.13 (Delay Approach by Generator Polynomials)** Again use the convolutional encoder for a (3, 2) code with  $m = 2$  as shown in Fig. 9.7. Since  $k = 2$ , the code word of Eq. (9.27) becomes

$$\begin{aligned} c(D) &= \sum_{i=1}^2 d^{(i)}(D^3)g_i(D) \\ &= d^{(1)}(D^3)g_1(D) + d^{(2)}(D^3)g_2(D) \end{aligned}$$

where

$$g_1(D) = g_1^{(1)}(D^3) + Dg_1^{(2)}(D^3) + D^2g_1^{(3)}(D^3)$$

$$g_2(D) = g_2^{(1)}(D^3) + Dg_2^{(2)}(D^3) + D^2g_2^{(3)}(D^3)$$

Using

$$g_1^{(1)}(D) = 1 \quad g_1^{(2)}(D) = D \quad g_1^{(3)}(D) = 1 + D$$

$$g_2^{(1)}(D) = D^2 \quad g_2^{(2)}(D) = 1 \quad g_2^{(3)}(D) = 1 + D + D^2$$

and Eq. (9.28), we have

$$\begin{aligned} g_1(D) &= 1 + D(D^3) + D^2(1 + D^3) \\ &= 1 + D^2 + D^4 + D^5 \end{aligned}$$

$$\begin{aligned} g_2(D) &= D^6 + D + D^2(1 + D^3 + D^6) \\ &= D + D^2 + D^5 + D^6 + D^8 \end{aligned}$$

Thus the code polynomial for  $d^{(1)}(D) = 1 + D^2$  and  $d^{(2)}(D) = D$  is expressed as

$$\begin{aligned} c(D) &= (1 + D^6)(1 + D^2 + D^4 + D^5) + D^3(D + D^2 + D^5 + D^6 + D^8) \\ &= 1 + D^2 + D^6 + D^9 + D^{10} \end{aligned}$$

Therefore the corresponding code sequence is

$$\mathbf{c} = (1 \ 0 \ 1, \ 0 \ 0 \ 0, \ 1 \ 0 \ 0, \ 1 \ 1 \ 0, \ 0 \ 0 \ 0)$$

which is the same result as found in the previous examples.

## 9.4 Code Structure

Convolutional codes have a more complex structure than block codes. The code structure is not algebraic, but topological. Traditionally, convolutional codes have been described in terms of a tree structure. However, convolutional codes are also characterized by a trellis struc-



ture as we shall explain below. The tree diagram is used for sequential decoding. But if the tree diagram is used for maximum-likelihood decoding, certain important structural information which is crucial to effective decoding is lost. Hence, the trellis diagram has been devised for maximum-likelihood decoding. Maximum-likelihood decoding is characterized as finding the shortest path through the code trellis. However, maximum-likelihood decoding and sequential decoding are intimately related, because maximum-likelihood decoding is probabilistic and emerged primarily from the sequential-decoding-oriented discipline.

To begin with, let us consider the convolutional encoder which consists of  $k$ -level shift registers. If the  $i$ th level shift register among  $k$  levels contains  $\gamma_i$  stages, these stages will then store  $\gamma_i$  previous information bits. In general, the  $(n, k)$  encoder can be thought of as containing  $k$ -level shift registers. Therefore the total number of stages contained in the encoder will be  $\gamma = \gamma_1 + \gamma_2 + \cdots + \gamma_k$ , where  $\gamma_i$  designates the number of stages in the  $i$ th level only. When  $k = 1$ , the memory order is simply  $\gamma = m$ . When the encoder inputs are  $d_{\lambda}^{(1)} d_{\lambda}^{(2)} \cdots d_{\lambda}^{(k)}$  at time  $t = \lambda$ , the encoder state at  $t = \lambda$  will be the binary  $\gamma$ -tuple of inputs

$$\xi = (d_{\lambda-1}^{(1)} d_{\lambda-2}^{(1)} \cdots d_{\lambda-\gamma_1}^{(1)}, d_{\lambda-1}^{(2)} d_{\lambda-2}^{(2)} \cdots d_{\lambda-\gamma_2}^{(2)}, \dots, d_{\lambda-1}^{(k)} d_{\lambda-2}^{(k)} \cdots d_{\lambda-\gamma_k}^{(k)})$$

and there are a total of  $2^\gamma$  different possible states. Let us label the states as  $s_0, s_1, \dots, s_{2^\gamma-1}$ . Then a particular state  $s_\sigma$  represents a binary  $\gamma$ -tuple  $(a_0 a_1 \cdots a_{\gamma-1})$  and the subscript  $\sigma$  is determined from  $\sigma = a_0 + a_1 2^1 + \cdots + a_{\gamma-1} 2^{\gamma-1}$ . For example, for an  $(n, 1)$  code with  $m = 2$ ,  $\gamma = \gamma_1 = 2$  and the encoder state at  $t = \lambda$  becomes, simply,  $(d_{\lambda-1} d_{\lambda-2})$ . Therefore the states  $\{00, 01, 10, 11\}$  correspond to  $\{s_0, s_2, s_1, s_3\}$  in which the subscripts  $\sigma = 0, 2, 1$ , and  $3$  are obtained from the relation  $\sigma = a_0 + 2a_1$ .

Consider an  $(n, 1)$  encoder which has a single shift register with  $m$  stages and a sole input tap. The state  $s_\lambda$  at time  $t = \lambda$  is one of  $2^m$  finite states. Then the set of finite states is simply expressed by

$$S = \{s_0, s_1, s_2, \dots, s_{2^m-1}\}$$

The state transition may be characterized as a Markov process. Initially, assume that the convolutional process only runs over the time period  $0 \leq t \leq L + m$  and that the initial and final stages  $s_0$  and  $s_{L+m}$  are known, where  $L$  denotes the finite length of the input data sequence. Thus the state sequence is represented by a state vector  $\mathbf{s} = (s_0, s_1, \dots, s_{L+m})$ . Since the process is Markov, the transition probability  $P(s_{\lambda+1})$  in a state  $s_{\lambda+1}$  at time  $t = \lambda + 1$ , given that all states up to  $t = \lambda$  are known, depends only on the state  $s_\lambda$  at  $t = \lambda$  such that



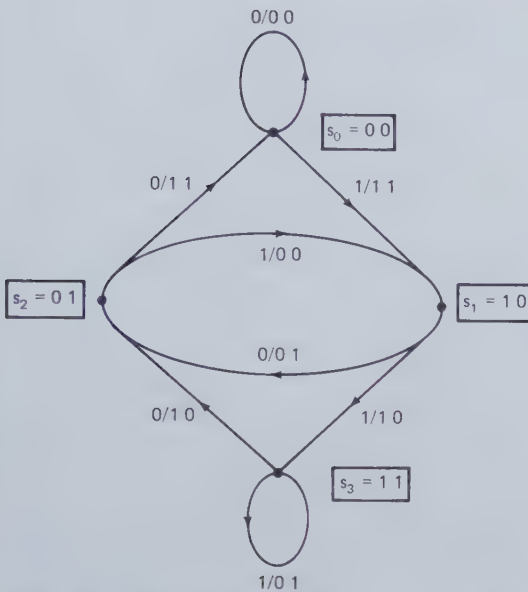
Figure 9.8 Delay line implementation with  $d_\lambda$  the input data.

$$P(s_{\lambda+1}|s_0, s_1, \dots, s_\lambda) = P(s_{\lambda+1}|s_\lambda) \quad (9.29)$$

If we define the transition  $x_\lambda$  at  $t = \lambda$  as the pair of states  $(s_{\lambda+1}, s_\lambda)$ , then the number of all possible transitions is  $2^{2m}$  or less. If the transition sequence is denoted as  $\mathbf{x} = (x_0, x_1, \dots, x_{L+m-1})$ , we can easily see that  $\mathbf{s}$  and  $\mathbf{x}$  are in a one-to-one correspondence by observing the trellis diagram.

Consider an input data sequence  $\mathbf{d} = (d_0, d_1, \dots, d_\lambda, \dots)$ , where  $d_\lambda$  is generated independently according to some probability distribution  $P(d_\lambda)$ . For an  $(n, 1)$  code with memory order  $m$ , the sequence can be modeled by a shift register of length  $m$  with the input symbol  $d_\lambda$  as shown in Fig. 9.8. Thus the state  $s_\lambda$  at  $t = \lambda$  can be expressed by  $s_\lambda = (d_{\lambda-1}, d_{\lambda-2}, \dots, d_{\lambda-m})$  and the number of states is  $2^m$ . Hence the transition  $x_\lambda$  at  $t = \lambda$  can be written as  $x_\lambda = (d_\lambda, d_{\lambda-1}, d_{\lambda-2}, \dots, d_{\lambda-m})$  and the number of transitions is  $2^{m+1}$ . As a result, the idea discussed above can be extended into the multiple  $k$ -level case of an  $(n, k, m)$  convolutional code as follows. The number of states is  $2^\gamma$ , where  $\gamma = \gamma_1 + \gamma_2 + \dots + \gamma_k$ ; and the number of transitions is  $2^{\gamma+k}$ , where  $k$  is the number of taps in the S/P input multiplexer.

Since a convolutional encoder is a finite-state machine, its operation can be described by a state diagram. For expository purposes, we again use the example of the  $m = 2, R = 1/2$  code of Fig. 9.1. For the state diagram depicted in Fig. 9.9, there is a total of  $2^m = 4$  states. Each input causes a transition to a new state. There are two branches leaving each state, one corresponding to the input "0" and the other to "1." Each branch is labeled with the encoder input 0 or 1 and the output of two digits corresponding to that state transition. For example, two branches leaving the state  $s_0 = (0 \ 0)$  have the designations 0/00 and 1/11. The time sequence of input, output, and transition states can be enumerated in a state table. The state table for the  $(2, 1, 2)$  encoder of Fig. 9.1 is shown in Table 9.2. It consists of three sections, labeled present state, transition state, and output. The derivation of the state table starts from an initial state  $s_0 = (0 \ 0)$ . The information available from Table 9.2 may be represented graphically by the state diagram shown in Fig. 9.9. If the all-zero state is assumed to be the initial state  $s_0 = (0 \ 0)$ , the code sequence corresponding to any given information sequence can be determined by tracing the indicated path through the state diagram, and then returning to the all-zero state  $s_0 = (0 \ 0)$ . For example, from the state diagram of Fig. 9.9, if the information se-



**Figure 9.9** Encoder state diagram for a (2, 1) convolutional code with  $m = 2$ .

**TABLE 9.2** State Table for the (2, 1, 2) Encoder of Fig. 9.1

| Present state |   | Input                   |                         |
|---------------|---|-------------------------|-------------------------|
|               |   | 0                       | 1                       |
|               |   | Output/transition state | Output/transition state |
| 0             | 0 | 0 0/0 0                 | 1 1/1 0                 |
| 0             | 1 | 1 1/0 0                 | 0 0/1 0                 |
| 1             | 1 | 1 0/0 1                 | 0 1/1 1                 |
| 1             | 0 | 0 1/0 1                 | 1 0/1 1                 |

quence is  $\mathbf{d} = (1\ 0\ 1\ 0\ 1)$ , then the code sequence is, accordingly,  $\mathbf{c} = (1\ 1, 0\ 1, 0\ 0, 0\ 1, 0\ 0, 0\ 1, 1\ 1)$ .

Since a convolutional encoder is regarded as a linear finite-state machine, its structure can be exhibited with the aid of state, tree, and trellis diagrams. A code tree and trellis structure, as shown in Figs. 9.10 and 9.11, are convenient methods of describing the relationship between input and output sequences. It is traditional and instructive to begin with the code tree. Both input and output sequences of the encoder can be displayed on the tree diagram which can be drawn directly from the state diagram of Fig. 9.9. The leftmost node in the code

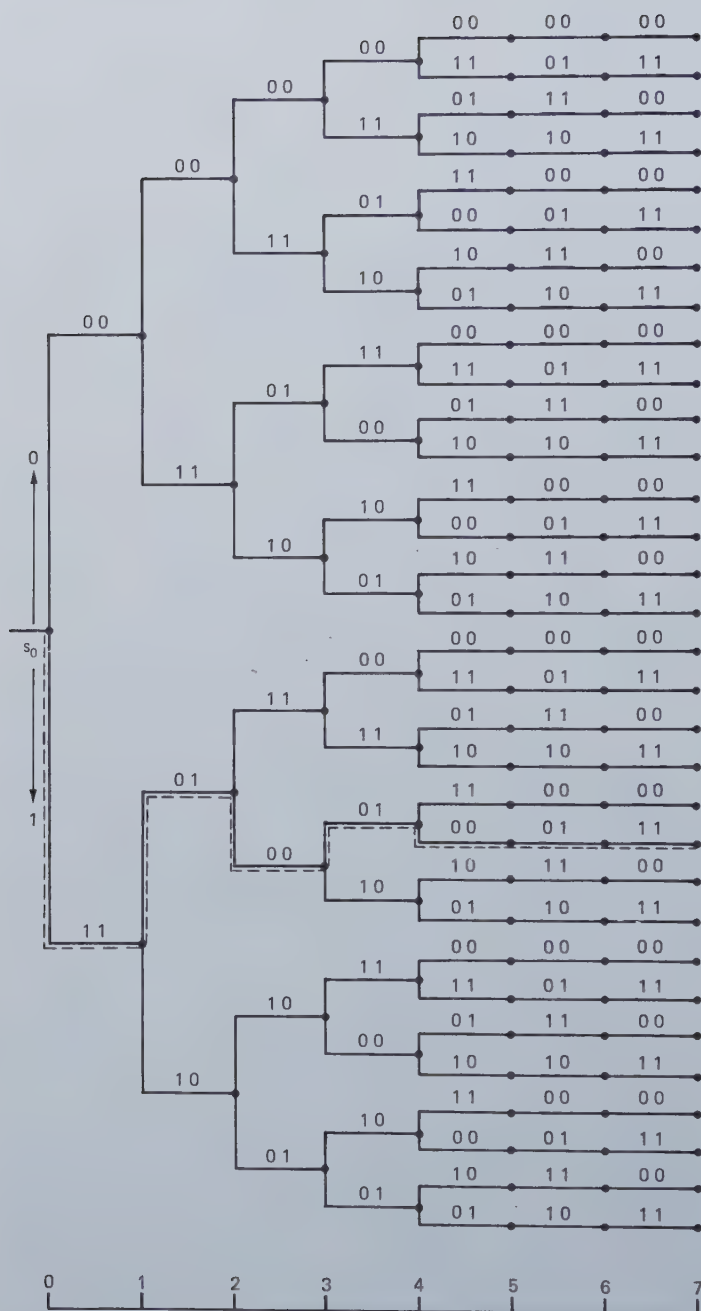
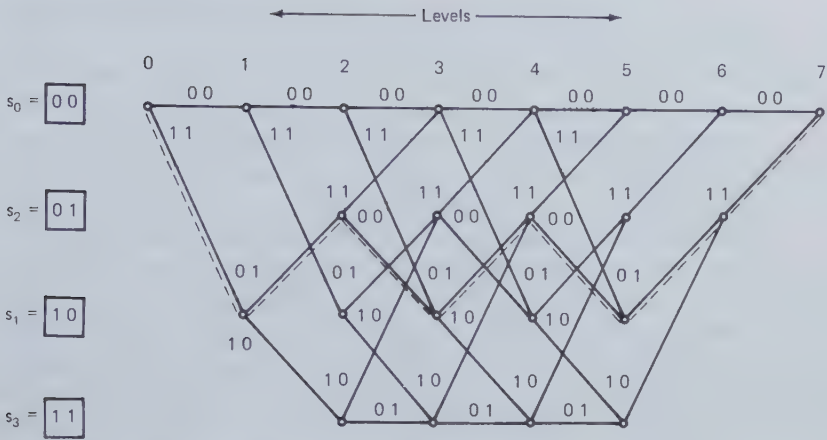


Figure 9.10 Tree diagram for a (2, 1) convolutional code with  $m = 2$  and  $L = 5$ .



**Figure 9.11** Trellis diagram for a  $(2, 1)$  convolutional code with  $m = 2$  and  $L = 5$ .

tree is called the origin node, and it represents the starting node of the encoder. In general, there are  $2^k$  branches leaving each node in the first  $L$  levels for an  $(n, k)$  code with memory order  $m$ . If  $kL$  designates the length of an information sequence, the  $L + m + 1$  tree levels are labeled from 0 to  $L + m$  in the code tree. Thus, for the  $m = 2$ ,  $R = 1/2$  code with  $L = 5$ , there are two branches leaving each node in the first five levels. The convention commonly used is that an input 0 corresponds to the upper branch in a tree and an input 1 corresponds to the lower branch. Thus, inputs are indicated by the jump up for 0 and down for 1 in the code tree, whereas outputs are indicated by digits along the tree branches. Thus, any input sequence traces out a particular path through the tree. With the code tree of Fig. 9.10, the input information sequence  $\mathbf{d} = (1\ 0\ 1\ 0\ 1)$  is indicated by moving down at the first branching level, up at the second, down at the third, up at the fourth, and down at the fifth to produce the output code sequence  $\mathbf{c} = (1\ 1, 0\ 1, 0\ 0, 0\ 1, 0\ 0, 0\ 1, 1\ 1)$  as indicated by the dotted line in Fig. 9.10. Hence it can be shown that all outputs correspond to  $2^{kL} = 32$  possible sequences for the first  $L = 5$  input digits. It becomes clear after the first three branches, that the structure becomes repetitive. The reason becomes obvious on examination of the encoder. When the fourth input digit enters the encoder, the first information bit falls off at the right end and no longer influences the output code sequence. Consequently, the information sequence generates the same code digits after the third branch, as shown in the code tree. This leads to redrawing the tree into a treelike structure that is called a trellis diagram and that is shown in Fig. 9.11. This repetitive structure of the trellis diagram suggests a further reduction into the state dia-



gram of Fig. 9.9. Therefore we have seen that these three diagrams are, on the whole, alike. The trellis diagram also contains  $L + m + 1$  levels. Hence, levels in Fig. 9.11 are labeled from 0 to 7 when  $L = 5$ . From Fig. 9.11, we see that the encoder always starts from state  $s_0 = (0\ 0)$  and returns to state  $s_0 = (0\ 0)$ . The first  $m = 2$  levels correspond to the encoder's departure from state  $s_0$  and the last  $m = 2$  levels correspond to the encoder's return to state  $s_0$ . In the center portion of the trellis diagram, each level contains a replica of the state diagram. There are two branches leaving and one branch entering each state in the first  $m$  levels. But in the center portion of the trellis, each state has two incoming branches and two outgoing branches. In the last  $m$  levels, each state has two incoming branches and one outgoing branch. The upper branch leaving each state at level  $i$  represents the input  $d_i = 0$ , while the lower branch represents  $d_i = 1$ . Each branch is labeled with  $n = 2$  corresponding outputs  $c_i$ , and each of the  $2^L = 32$  code words of length  $N = n(L + m) = 14$  is represented by a unique path through the trellis. Thus the possible encoder outputs can be found by tracing the appropriate path through the trellis. The trellis contains  $2^{kL}$  distinct paths corresponding to the  $2^{kL}$  code words. For example, the input data sequence  $\mathbf{d} = (1\ 0\ 1\ 0\ 1)$  generates the output code sequence  $\mathbf{c} = (1\ 1, 0\ 1, 0\ 0, 0\ 1, 0\ 0, 0\ 1, 1\ 1)$  as indicated by the dotted line in Fig. 9.11.

**Example 9.14** Consider the  $R = 1/2$ ,  $m = 3$  convolutional encoder illustrated in Fig. 9.12. In this case, the input is shifted 1 bit at a time into the decoder, and 2-bit outputs are generated. The generator polynomials of this code are  $g^{(1)}(D) = 1 + D + D^3$  and  $g^{(2)}(D) = 1 + D + D^2 + D^3$ . After computing the state table, we shall draw the state diagram and trellis diagram for this convolutional code. The contents of the state table are tabulated in Table 9.3. A complete description of the encoder as far as the input and the resulting output are concerned can be obtained from a state diagram. The state of this convolutional encoder is assigned a number from 0 to  $2^m - 1 = 7$ . The present state and output of the encoder are always uniquely determined by the previous state and cur-

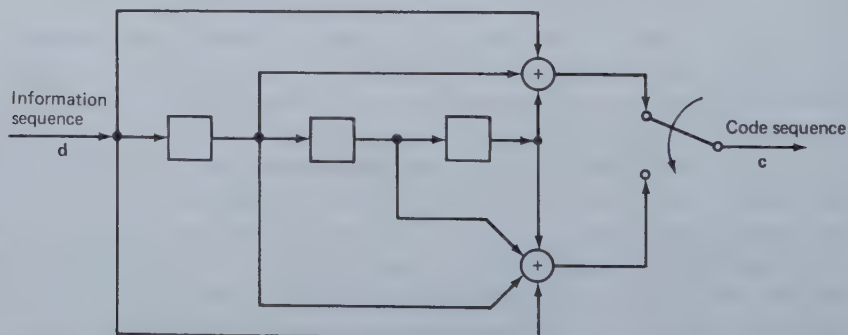


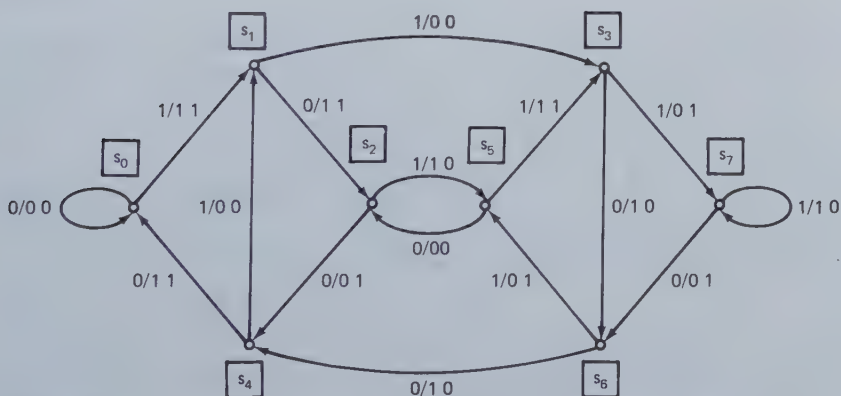
Figure 9.12 A (2, 1) convolutional encoder with memory order  $m = 3$ .

TABLE 9.3 State Table for the (2, 1, 3) Encoder of Fig. 9.12

| Current state | Input                   |                         |
|---------------|-------------------------|-------------------------|
|               | 0                       | 1                       |
|               | Output/transition state | Output/transition state |
| $s_0$ 0 0 0   | 0 0/0 0 0               | 1 1/1 0 0               |
| $s_1$ 1 0 0   | 1 1/0 1 0               | 0 0/1 1 0               |
| $s_2$ 0 1 0   | 0 1/0 0 1               | 1 0/1 0 1               |
| $s_3$ 1 1 0   | 1 0/0 1 1               | 0 1/1 1 1               |
| $s_4$ 0 0 1   | 1 1/0 0 0               | 0 0/1 0 0               |
| $s_5$ 1 0 1   | 0 0/0 1 0               | 1 1/1 1 0               |
| $s_6$ 0 1 1   | 1 0/0 0 1               | 0 1/1 0 1               |
| $s_7$ 1 1 1   | 0 1/0 1 1               | 1 0/1 1 1               |

rent input. The state diagram is simply a graph of the possible states of the encoder and the possible transitions from one state to another. Therefore, the state diagram is even more compact than the trellis because the encoder output is determined by the input and the state of the encoder. The state diagram for this encoder is illustrated in Fig. 9.13. Thus, the information sequence dictates the path through the state diagram, and the output sequence encountered along this path constitutes the resulting encoded sequence.

In order to understand Viterbi decoding, it is convenient to expand the state diagram of the encoder to the trellis diagram. Assuming an information sequence of length  $L = 6$ , the trellis diagram contains  $L + m + 1 = 10$  levels. Levels are labeled from 0 to  $L + m = 9$  as shown in Fig. 9.14. For an information sequence of  $kL = 6$  bits, there are two branches leaving and entering each state in the center portion of the trellis, and 64 distinct paths through the trellis corresponding to the  $2^{kL} = 64$  code words. Each branch is labeled with the 2-bit output, and each of the 64 code words of length  $N = n(L + m) = 18$  is represented by a unique path through the trellis. The trellis diagram of this encoder is shown in Fig. 9.14.

Figure 9.13 Encoder state diagram for a  $m = 3$  (2, 1) convolutional code.

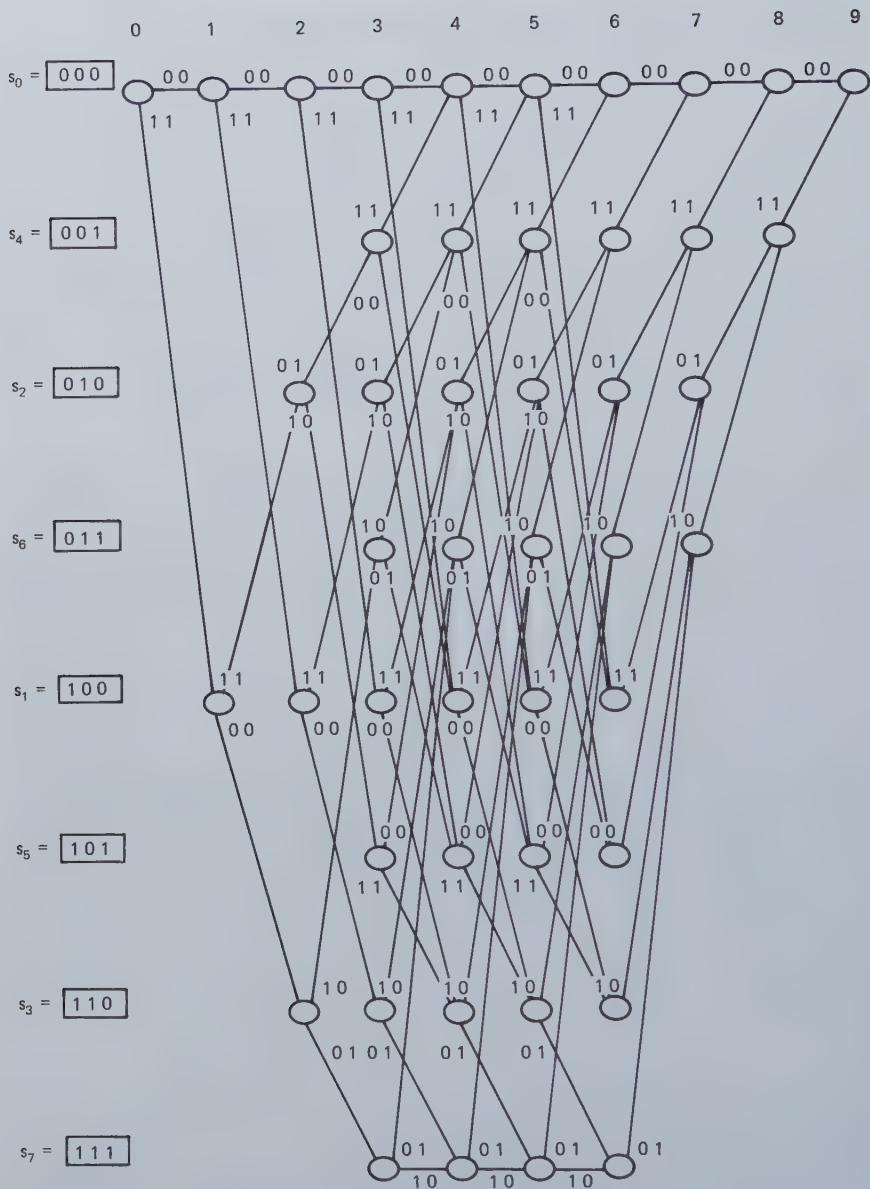


Figure 9.14 Trellis diagram for a  $m = 3$  (2, 1) convolutional code.

**Example 9.15** Consider the  $R = \frac{2}{3}$ ,  $m = 2$  nonsystematic convolutional encoder illustrated in Fig. 9.7, with a transfer matrix having the following generator polynomial

$$G(D) = \begin{bmatrix} 1 & D & 1 + D \\ D^2 & 1 & 1 + D + D^2 \end{bmatrix}$$

Hence, the elements of  $G(D)$  are expressed as  $\mathbf{g}_1^{(1)} = (1 \ 0 \ 0)$ ,  $\mathbf{g}_1^{(2)} = (0 \ 1 \ 0)$ ,  $\mathbf{g}_1^{(3)} = (1 \ 1 \ 0)$ ,  $\mathbf{g}_2^{(1)} = (0 \ 0 \ 1)$ ,  $\mathbf{g}_2^{(2)} = (1 \ 0 \ 0)$ , and  $\mathbf{g}_2^{(3)} = (1 \ 1 \ 1)$ . The number of states of this encoder is  $2^\gamma = 2^{\gamma_1 + \gamma_2} = 2^3 = 8$ , and the number of transitions is  $2^{\gamma+k} = 2^{3+2} = 32$ . From these data, we can complete the state table shown in Table 9.4. Using Table 9.4, we can easily draw the state diagram and the trellis diagram as shown in Figs. 9.15 and 9.16, respectively.

**TABLE 9.4** State Table for the (3, 2, 2) Encoder Illustrated in Fig. 9.7

| Current state | Input                          |                                |                                |                                |
|---------------|--------------------------------|--------------------------------|--------------------------------|--------------------------------|
|               | 0 0                            | 0 1                            | 1 0                            | 1 1                            |
|               | Output/<br>transition<br>state | Output/<br>transition<br>state | Output/<br>transition<br>state | Output/<br>transition<br>state |
| $s_0$ 0 0 0   | 0 0 0 / 0 0 0                  | 0 1 1 / 0 1 0                  | 1 0 1 / 1 0 0                  | 1 1 0 / 1 1 0                  |
| $s_1$ 0 0 1   | 1 0 1 / 0 0 0                  | 1 1 0 / 0 1 0                  | 0 0 0 / 1 0 0                  | 0 1 1 / 1 1 0                  |
| $s_2$ 0 1 0   | 0 0 1 / 0 0 1                  | 0 1 0 / 0 1 1                  | 1 0 0 / 1 0 1                  | 1 1 1 / 1 1 1                  |
| $s_3$ 0 1 1   | 1 0 0 / 0 0 1                  | 1 1 1 / 0 1 1                  | 0 0 1 / 1 0 1                  | 0 1 0 / 1 1 1                  |
| $s_4$ 1 0 0   | 0 1 1 / 0 0 0                  | 0 0 0 / 0 1 0                  | 1 1 0 / 1 0 0                  | 1 0 1 / 1 1 0                  |
| $s_5$ 1 0 1   | 1 1 0 / 0 0 0                  | 1 0 1 / 0 1 0                  | 0 1 1 / 1 0 0                  | 0 0 0 / 1 1 0                  |
| $s_6$ 1 1 0   | 0 1 0 / 0 0 1                  | 0 0 1 / 0 1 1                  | 1 1 1 / 1 0 1                  | 1 0 0 / 1 1 1                  |
| $s_7$ 1 1 1   | 1 1 1 / 0 0 1                  | 1 0 0 / 0 1 1                  | 0 1 0 / 1 0 1                  | 0 0 1 / 1 1 1                  |

## 9.5 Generating Function and Distance Properties

The performance of a convolutional code depends on the decoding algorithm employed and the distance properties of the code. The distance properties and performance bounds for maximum-likelihood decoding can be obtained from the state diagram. The generating function (or transfer function) approach to finding the weight distribution becomes impractical when the total encoder memory  $\gamma$  exceeds 4 or 5. Since the number of states  $2^\gamma$  grows very rapidly as  $\gamma$  increases, the state and trellis diagrams become too complicated to analyze the weight distribution function for the code.

In order to introduce distance measures for convolutional codes, the state diagram for the  $m = 2$ ,  $R = \frac{1}{2}$  convolutional encoder shown in Fig. 9.9 will be used to demonstrate the method used for obtaining the distance properties of this code. In order to provide a complete descrip-

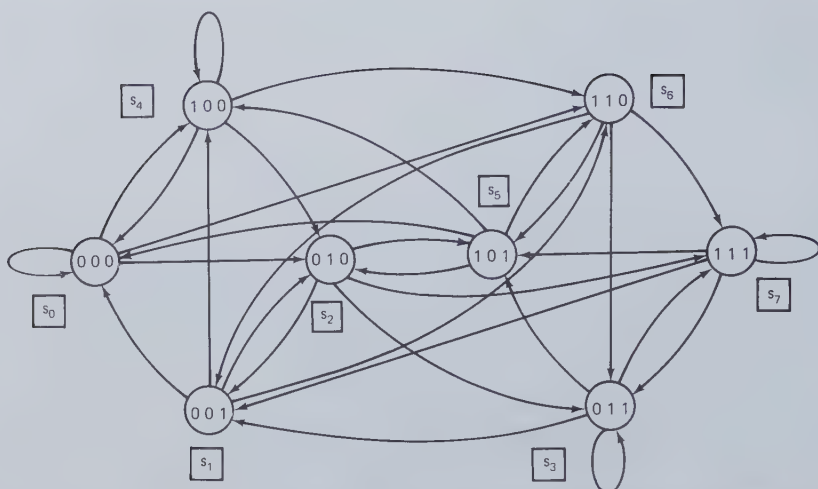


Figure 9.15 State diagram for the (3, 2, 2) convolutional encoder shown in Fig. 9.7.

tion of the Hamming weight of all nonzero code words, we must modify the state diagram of Fig. 9.9 as explained below. The state  $s_0 = (0\ 0)$  is split into two states, one which represents an initial state and one which represents a final state of the state diagram. The self-loop around state  $s_0$  can be eliminated since it contributes nothing to the distance properties of a nonzero code sequence relative to the all-zero code sequence. Furthermore, each branch is labeled with a branch gain  $x^j$  in which the exponent  $j$  denotes the Hamming weight of the  $n$  coded bits on that branch. Thus we have the resulting modified state diagram illustrated in Fig. 9.17. Each path connecting the initial state to the final state represents a nonzero code sequence that diverges from and converges on the state  $s_0$  exactly once. The path gain is the product of the branch gain along a path, and the weight of the associated code word is the power of  $x$  in the path gain.

The weight distribution function of a code can be determined by considering the modified state diagram as a signal flow graph and applying Mason's gain formula to compute its generating function. An interesting property of state topology based on Mason's theorem permits us to formulate the generating function, which is given by

$$T(x) = \frac{\sum_{\lambda} G_{\lambda} \Delta_{\lambda}}{\Delta} \quad (9.30)$$

where  $\Delta = 1 - \Sigma L_1 + \Sigma L_2 - \Sigma L_3 + \cdots + (-1)^m \Sigma L_m$  in which  $L_1$  is the gain of each closed loop,  $L_2$  is the product of loop gains of any two nontouching loops, and  $L_m$  is the product of loop gains of any  $m$  nontouching loops. Also,  $G_{\lambda}$  represents gain of the  $\lambda$ th forward path,





Figure 9.16 Trellis diagram for the (3, 2, 2) convolutional encoder shown in Fig. 9.7.

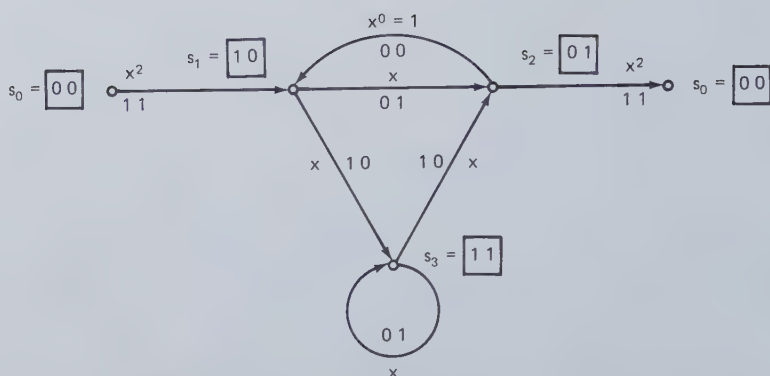


Figure 9.17 Modified state diagram for the  $m = 2$ ,  $R = \frac{1}{2}$  convolutional encoder.

and  $\Delta_\lambda$  is the value of  $\Delta$  for a corresponding portion of the graph not touching the  $\lambda$ th forward path. By observing the graph shown in Fig. 9.17, we see that there are two forward paths, i.e.,  $s_0 s_1 s_2 s_0$  and  $s_0 s_1 s_3 s_2 s_0$ . Thus, their path gains are  $G_1 = x^5$  and  $G_2 = x^6$ , respectively. Since we can compute

$$\Delta_1 = 1 - x$$

$$\Delta_2 = 1$$

$$\Delta = 1 - (x + x^2 + x) + x^2 = 1 - 2x$$

the generating function for this graph is then given by

$$\begin{aligned} T(x) &= \frac{x^5(1-x) + x^6}{1-2x} \\ &= x^5 + 2x^6 + 4x^7 + \dots \end{aligned} \quad (9.31)$$

Therefore,  $T(x)$  provides a complete description of the weight distributions of nonzero code words that enter and leave state  $s_0$  exactly once. In this case there is one such code word of weight 5, two of weight 6, four of weight 7, and so forth. Thus, we see that the generating function provides the distance properties of the convolutional code. The most important distance measure for convolutional codes is the minimum free distance, denoted as  $d_{\text{free}}$ . As stated earlier, it is not practical to compute  $T(x)$  if  $\gamma$  exceeds 5. Consequently, for larger values of  $\gamma$ , the number of states  $2^\gamma$  becomes unacceptably large. Hence, other means must be employed to find  $d_{\text{free}}$  for most codes. But no general solution for finding  $d_{\text{free}}$  for large values of  $\gamma$ , i.e.,  $> 20$ , has yet been discovered. Since  $d_{\text{free}}$  is the minimum distance between any two code words,  $d_{\text{free}} = 5$  in this example. For a given rate and encoder

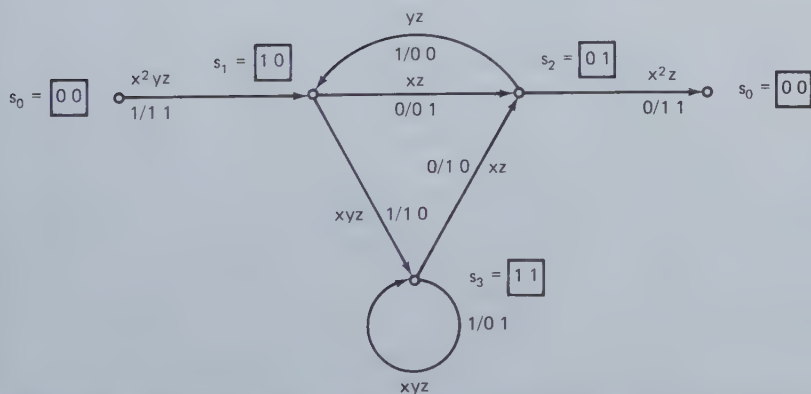
memory, the achievable free distance  $d_{\text{free}}$  is more available with nonsystematic codes than with systematic codes. Therefore, non-systematic codes with large  $d_{\text{free}}$  are used for Viterbi decoding.

The generating function can also be used to provide more detailed information than just the distance of the various paths. Of course, additional information about the structure of a code can be obtained by using another similar procedure. Suppose that a factor  $y$  is introduced to all branch transitions that result from an input bit that is equal to 1. As each branch is traversed, the cumulative exponent on  $y$  increases by 1 if that branch transition is a result of an input bit that is equal to 1. Furthermore, we introduce a factor  $z$  into each branch of the state diagram so that the exponent of  $z$  will serve as a counting variable to indicate the number of branches in any given path from and to the state  $s_0$ . For the rate  $R = 1/2$ ,  $m = 2$  convolutional code of this example, the state diagram that incorporates the additional factors  $y$  and  $z$  is shown in Fig. 9.18. As indicated in the augmented state diagram of Fig. 9.18, the only paths of interest are those paths that start and end in the state  $s_0$  and do not go through  $s_0$  in between. The self-loop on the state  $s_0$  is therefore removed, and the zero state  $s_0$  is split into an input and an output. The gain of this state graph is thus computed by using Mason's rule. Of course, there is an alternate technique for evaluating the generating function, which involves writing the state equations, but it is not difficult to obtain. From the augmented graph of Fig. 9.18, we have

$$G_1\Delta_1 = x^5yz^3(1 - xyz)$$

$$G_2\Delta_2 = x^6y^2z^4$$

$$\Delta = 1 - xyz(1 + z)$$



**Figure 9.18** Augmented state diagram for the  $m = 2$ ,  $R = 1/2$  convolutional encoder.

Hence the generating function is obtained as follows:

$$\begin{aligned}
 T(x, y, z) &= \frac{x^5 y z^3}{1 - xyz(1 + z)} \\
 &= x^5 y z^3 + x^6 y^2 z^4(1 + z) + x^7 y^3 z^5(1 + z)^2 + \dots \\
 &\quad + x^{5+k} y^{1+k} z^{3+k}(1 + z)^k + \dots
 \end{aligned} \tag{9.32}$$

when expanded by long division. This generating function implies that the code sequence of weight 5 has branches of length 3 and an information sequence of weight 1; a code sequence of weight 6 has branches of length 4 and an information sequence of weight 2; another has branches of length 5 and an information sequence of weight 2; and so on. Thus the minimum free distance  $d_{\text{free}}$ , which is the weight of the minimum-weight path beginning and ending in the state  $s_0$ , is  $d_{\text{free}} = 5$ .

Obviously  $d_{\text{free}} \geq d_{\text{min}}$  must hold, but it is also true that for many codes,  $d_{\text{free}} = d_{\text{min}}$ .

**Example 9.16** Consider the  $R = 1/3$ ,  $m = 2$  convolutional encoder shown in Fig. 9.19, with  $g^{(1)}(D) = 1 + D$ ,  $g^{(2)}(D) = 1 + D^2$ , and  $g^{(3)}(D) = 1 + D + D^2$ . The number of states is  $2^m = 4$  and the number of transitions is  $2^{m+k} = 8$ ; the resulting state table is shown in Table 9.5. The state diagram derived from Table 9.5 is shown in Fig. 9.20, and the augmented state diagram for this code is shown in Fig. 9.21. The generating function is then calculated as follows. Let  $T(x, y, z)$  be the generating function for all paths connecting the all-zero states  $s_0$ . From Mason's rule, we have

$$G_1 = x^7 y z^3 \quad G_2 = x^8 y^2 z^4$$

$$\Delta_1 = 1 - xyz \quad \Delta_2 = 1$$

and

$$\begin{aligned}
 \Delta &= 1 - (x^3 y z^2 + xyz + x^4 y^2 z^3) + x^4 y^2 z^3 \\
 &= 1 - xyz(1 + x^2 z)
 \end{aligned}$$

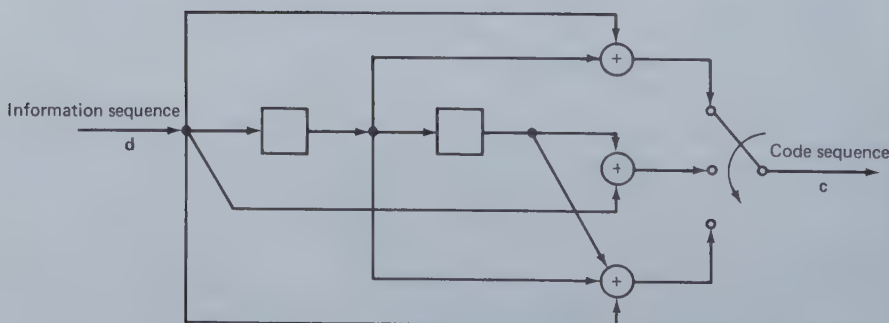
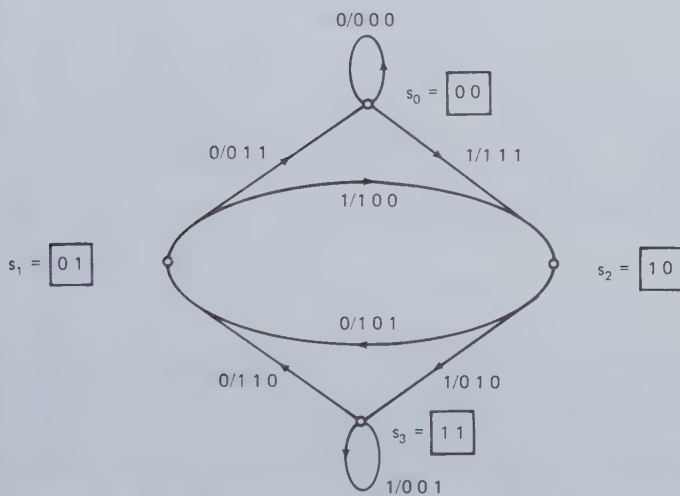


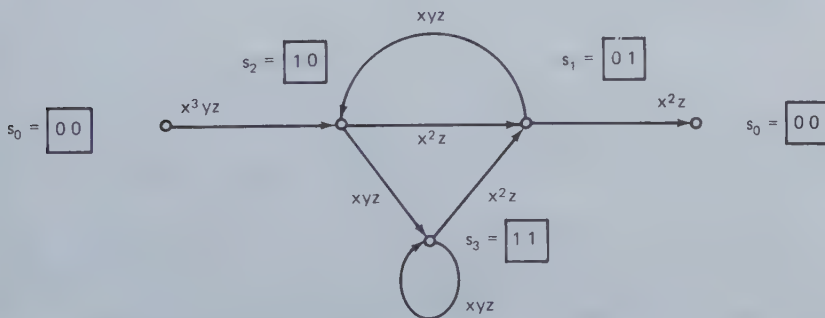
Figure 9.19 The  $R = 1/3$ ,  $m = 2$  convolutional encoder.

**TABLE 9.5** State Table for the (3, 1, 2) Encoder Illustrated in Fig. 9.19

| Present state |     | Input                   |                         |
|---------------|-----|-------------------------|-------------------------|
|               |     | 0                       | 1                       |
|               |     | Output/transition state | Output/transition state |
| $s_0$         | 0 0 | 0 0 0/0 0               | 1 1 1/1 0               |
| $s_1$         | 0 1 | 0 1 1/0 0               | 1 0 0/1 0               |
| $s_2$         | 1 0 | 1 0 1/0 1               | 0 1 0/1 1               |
| $s_3$         | 1 1 | 1 1 0/0 1               | 0 0 1/1 1               |



**Figure 9.20** State diagram for the (3, 1, 2) convolutional encoder shown in Fig. 9.19.



**Figure 9.21** Augmented state diagram for the (3, 1, 2) convolutional encoder shown in Fig. 9.19.



Thus the generating function is given by

$$T(x, y, z) = \frac{x^7 y z^3}{1 - x y z (1 + x^2 z)} \\ = x^7 y z^3 + x^8 y^2 z^4 + x^9 y^3 z^5 + x^{10} y^2 z^5 + \dots$$

Therefore, the generating function  $T(x, y, z)$  implies the contents of the following table.

| Weight of code sequence | Weight of information sequence | Number of branches |
|-------------------------|--------------------------------|--------------------|
| 7                       | 1                              | 3                  |
| 8                       | 2                              | 4                  |
| 9                       | 3                              | 5                  |
| 10                      | 2                              | 5                  |
| $\vdots$                | $\vdots$                       | $\vdots$           |

Since we have fully studied the structure and encoding problems of convolutional codes, convolutional decoding techniques will be presented in the following two chapters.

## PROBLEMS

**9.1** For the (2, 1) convolutional code with  $m = 3$ :

- Draw the encoder diagram with  $\mathbf{g}^{(1)} = (1 \ 1 \ 0 \ 1)$  and  $\mathbf{g}^{(2)} = (1 \ 1 \ 1 \ 1)$ .
- Find the code sequence  $\mathbf{c}$  corresponding to the information sequence  $\mathbf{d} = (1 \ 1 \ 0 \ 1 \ 1)$  using  $c_\lambda^{(j)} = \sum_{i=0}^3 d_{\lambda-i}^{(1)} g_{i,\lambda}^{(j)}$ ,  $0 \leq \lambda \leq 3$ ,  $1 \leq j \leq 2$ .
- Repeat part *b* using discrete convolution  $\mathbf{c}^{(j)} = \mathbf{d} * \mathbf{g}^{(j)}$ ,  $j = 1, 2$ , and compare the result with part *b*.
- Is this a systematic encoder?

**9.2** For the (4, 3) nonsystematic convolutional code with  $m = 2$ , suppose that the generator sequences are given as

$$\mathbf{g}_1^{(1)} = (1 \ 0 \ 0) \quad \mathbf{g}_1^{(2)} = (1 \ 0 \ 0) \quad \mathbf{g}_1^{(3)} = (1 \ 0 \ 0) \quad \mathbf{g}_1^{(4)} = (1 \ 0 \ 0)$$

$$\mathbf{g}_2^{(1)} = (0 \ 0 \ 0) \quad \mathbf{g}_2^{(2)} = (1 \ 1 \ 0) \quad \mathbf{g}_2^{(3)} = (0 \ 1 \ 0) \quad \mathbf{g}_2^{(4)} = (1 \ 0 \ 0)$$

$$\mathbf{g}_3^{(1)} = (0 \ 0 \ 0) \quad \mathbf{g}_3^{(2)} = (0 \ 1 \ 0) \quad \mathbf{g}_3^{(3)} = (1 \ 0 \ 1) \quad \mathbf{g}_3^{(4)} = (1 \ 0 \ 1)$$

- What is the constraint length of this code?
- Draw the encoder diagram using the generator sequences given above.
- Find the generator matrix  $\mathbf{G}$ .
- Find the code word  $\mathbf{c}$  corresponding to the information sequence  $\mathbf{d} = (1 \ 1 \ 0 \ 0 \ 1 \ 1 \ 1 \ 0 \ 1)$  by using  $\mathbf{c} = \mathbf{d} \cdot \mathbf{G}$ .
- Find the code word  $\mathbf{c}$  by the method of discrete convolution.
- Repeat the computation of the code sequence  $\mathbf{c}$  by using the encoding equation

$$c_{\lambda}^{(j)} = \sum_{\iota=0}^2 \sum_{i=1}^3 d_{\lambda-\iota}^{(i)} g_{\iota,\lambda}^{(j)}, \quad 0 \leq \lambda \leq 4, \quad 1 \leq j \leq 4$$

Compare this result to those of parts *d* and *e*.

**9.3** For the (2, 1, 3) nonsystematic convolutional encoder of Prob. 9.1:

- a* Find the generator polynomials.
- b* Find the generator-polynomial matrix  $G(D)$ .
- c* First find the set of output sequence  $c'(D)$ , and then determine the code sequence  $c(D)$  corresponding to the information sequence  $d(D) = 1 + D^2 + D^3 + D^4$ . Are the results identical to those obtained in Prob. 9.1, parts *b* and *c*?

**9.4** For the (4, 3) convolutional code of Prob. 9.2:

- a* Obtain all the generator polynomials  $g_i^{(j)}(D)$ ,  $1 \leq i \leq 3$ ,  $1 \leq j \leq 4$ .
- b* Find the generator-polynomial matrix  $G(D)$ .
- c* Find the code polynomial  $c(D)$  corresponding to the set of information sequences  $d(D) = [1 + D^2, 1 + D, D + D^2]$ .
- d* Find the code word  $\mathbf{c}$ , and compare it to the results obtained in Prob. 9.2, parts *d*, *e*, and *f*.

**9.5** For the (3, 2) convolutional code with  $m = 1$ , suppose that the generator polynomials are given as

$$\begin{aligned} g_1^{(1)}(D) &= 1 + D & g_1^{(2)}(D) &= D & g_1^{(3)}(D) &= 1 + D \\ g_2^{(1)}(D) &= D & g_2^{(2)}(D) &= 1 & g_2^{(3)}(D) &= 1 \end{aligned}$$

- a* Draw the encoder diagram of this code.
- b* Find the generator-polynomial matrix  $G(D)$ .
- c* Find  $c^{(1)}(D)$ ,  $c^{(2)}(D)$ , and  $c^{(3)}(D)$  from the encoding equation  $c(D) = d(D)$ . Matrix  $G(D)$  corresponds to the information sequence  $d(D) = [1 + D + D^3, 1 + D^2 + D^3]$ .
- d* Find the code word  $\mathbf{c}$  after computing the code polynomial  $c(D)$ .

**9.6** For the (3, 1) systematic convolutional code with  $m = 5$ , the generator sequences are given as  $\mathbf{g}^{(1)} = (1 \ 0 \ 0 \ 0 \ 0 \ 0)$ ,  $\mathbf{g}^{(2)} = (1 \ 0 \ 1 \ 1 \ 0 \ 1)$ , and  $\mathbf{g}^{(3)} = (1 \ 1 \ 0 \ 0 \ 1 \ 1)$ .

- a* Find the generator matrix  $\mathbf{G}$  of this code.
- b* Find the code word  $\mathbf{c}$  corresponding to the information sequence  $\mathbf{d} = (1 \ 1 \ 0 \ 1)$ .
- c* Discriminate the parity sequence from the code word  $\mathbf{c}$  by marking the information digits with an arrow (*a*).

**9.7** For the (2, 1, 3) systematic convolutional encoder where  $\mathbf{g}^{(1)} = (1 \ 0 \ 0 \ 0)$  and  $\mathbf{g}^{(2)} = (1 \ 1 \ 0 \ 1)$ :

- a* Find the generator matrix  $\mathbf{G}$  of this code.
- b* Find the generator-polynomial matrix  $G(D)$ .
- c* Devise the type I systematic convolutional encoder.
- d* Devise the type II systematic convolutional encoder.

**9.8** For the (2, 1, 3) convolutional code of Prob. 9.1:

- a* Draw the state diagram of this encoder.
- b* Draw the modified state diagram of this encoder.
- c* Find the code-generating function  $T(x)$ .
- d* Find the generating function  $T(x, y, z)$ .
- e* Find the minimum free distance  $d_{\text{free}}$ .

**9.9** For the (3, 2) convolutional code with  $m = 3$  and whose generator sequences are generally given by

$$\mathbf{g}_1^{(1)} = (g_{1,0}^{(1)} g_{1,1}^{(1)} g_{1,2}^{(1)} g_{1,3}^{(1)}) \quad \mathbf{g}_2^{(1)} = (g_{2,0}^{(1)} g_{2,1}^{(1)} g_{2,2}^{(1)} g_{2,3}^{(1)})$$

$$\mathbf{g}_1^{(2)} = (g_{1,0}^{(2)} g_{1,1}^{(2)} g_{1,2}^{(2)} g_{1,3}^{(2)}) \quad \mathbf{g}_2^{(2)} = (g_{2,0}^{(2)} g_{2,1}^{(2)} g_{2,2}^{(2)} g_{2,3}^{(2)})$$

$$\mathbf{g}_1^{(3)} = (g_{1,0}^{(3)} g_{1,1}^{(3)} g_{1,2}^{(3)} g_{1,3}^{(3)}) \quad \mathbf{g}_2^{(3)} = (g_{2,0}^{(3)} g_{2,1}^{(3)} g_{2,2}^{(3)} g_{2,3}^{(3)})$$

- a* Determine the  $2 \times 3$  submatrix  $\mathbf{G}_\lambda$ ,  $\lambda = 0, 1, 2, 3$ .
- b* Form the generator matrix  $\mathbf{G}$ .

**9.10** For the (2, 1, 3) convolutional code of Prob. 9.1, assume that  $\lambda = 5$ .

- a* Draw the tree diagram for this code. How many tree levels does it have?
- b* Draw the trellis diagram for this code. How many trellis levels does it have?
- c* Find the output code sequence  $\mathbf{c}$  for the input information sequence  $\mathbf{d} = (1 \ 1 \ 0 \ 0 \ 1)$  by utilizing the two diagrams in parts *a* and *b*.

## Maximum-Likelihood Decoding and Sequential Decoding for Convolutional Codes

In 1961, Wozencraft devised a decoding technique which is called sequential decoding. This technique has been the subject of research interest for over 20 years. In 1963, Fano introduced a new version of sequential decoding which is referred to as the Fano algorithm. Another version of sequential decoding, known as the stack or ZJ algorithm, was discovered independently by Zigangirov in 1966 and Jelinek in 1969. A major problem with sequential decoding is that the number of computations required to advance one node deeper into the code tree is a random variable. This characteristic strongly affects the complexity involved in achieving a given level of performance. The performance of sequential decoding is slightly suboptimum, but it is independent of code constraint length. This allows sequential decoding to be used with long constraint lengths. However its major drawback is that noise frames require large amounts of computation. Consequently, decoding times occasionally exceed some upper limit, causing data to be lost or erased.

In 1963, Massey introduced a less efficient but simpler-to-implement decoding method, called majority-logic or threshold decoding, which is applicable to convolutional codes. The structure of convolutional codes is not algebraic, but topological. Nevertheless an algebraic approach can also be used for decoding convolutional codes. Threshold decoding is conceptually and practically the closest to block decoding. Its conceptual simplicity stems from the fact that threshold decoding of convolutional codes begins with the calculation of a set of syndrome digits, the definition of which is very similar to the definition for block codes. Thus threshold decoding lacks the search aspects of Viterbi and

sequential decoding. In fact, threshold decoding is inferior in performance when compared to Viterbi or sequential decoding, but the implementation of the decoder is much simpler.

The Viterbi algorithm was proposed in 1967 as a convolutional decoding method. Since that time, it has been recognized as an attractive solution to maximum-likelihood decoding, which is characterized as the determination of the shortest path through a topological structure called a code trellis, and the Viterbi algorithm is an efficient solution. Thus, Viterbi decoding is known to be optimum in the maximum-likelihood sense. However, Viterbi decoding of convolutional codes is not practical for a long code with a large constraint length  $n_A$  because the probability of error increases exponentially with  $n_A$ . Viterbi decoding must be used with convolutional codes with small constraint lengths since there is a limit on the encoder memory  $\gamma = m$ . As a result, both complexity and decoding effort increase exponentially with  $n_A$ . Another point about the Viterbi algorithm is that  $2^\gamma$  computations must be performed per decoded data sequence even when the noise effect is negligible. This will be a wasted decoding effort.

Decoding techniques for convolutional codes are largely grouped into two classes, i.e., probabilistic decoding and algebraic decoding. Maximum-likelihood decoding by the Viterbi algorithm and sequential decoding by the Fano or the stack algorithm, are called probabilistic decoding. On the other hand, algebraic decoding implies majority-logic or threshold decoding, which is discussed in Chap. 11.

## 10.1 Concept of Probabilistic Decoding

Two of the most powerful techniques for decoding convolutional codes are (1) the optimal trellis search of Viterbi decoding and (2) the suboptimum tree search of sequential decoding. These two decoding techniques have developed independently, and they appear to be the best for determining the most likely information sequence.

The Viterbi algorithm uses the trellis structure of the code and examines all distinct paths at every trellis level, whereas a sequential decoder uses the tree structure of the code and follows only the path in the tree that appears to be the most likely. Sequential decoding involves a random motion in the tree, and hence the number of computations needed to decode a given information sequence is a random variable. This variability of the computational effort is one of the major drawbacks of sequential decoding. Compared with sequential decoding, the operations of the Viterbi decoder are less complex and the motion of the decoder is always forward with no backing up. However,



Viterbi decoding is not practical for long codes having large constraint lengths.

## 10.2 Optimum Likelihood Strategy

In a convolutional coded system,  $\mathbf{d}$  represents a sequence of  $kL$  information bits and  $\mathbf{c}$  represents a code sequence of length  $N = n(L + m)$ . The additional  $mn$  encoded digits due to the memory order  $m$  are produced after the last digit of the information sequence has entered the encoder. Suppose that the received sequence  $\mathbf{r}$  is a binary received  $N$ -tuple, and the decoder output  $\hat{\mathbf{d}}$  is a  $kL$ -bit estimate of the information sequence.

Consider the decoder that produces an estimate  $\hat{\mathbf{d}}$  of the information sequence  $\mathbf{d}$  based on the received sequence  $\mathbf{r}$ . Clearly, we see that  $\hat{\mathbf{d}} = \mathbf{d}$  if and only if  $\mathbf{r} = \mathbf{c}$ , and that a decoding error occurs if and only if  $\mathbf{r} \neq \mathbf{c}$ , or  $\hat{\mathbf{c}} \neq \mathbf{c}$ . Thus, the conditional error probability of the decoder, given that  $\mathbf{r}$  has been received, is defined as

$$P(\epsilon|\mathbf{r}) \triangleq P(\hat{\mathbf{c}} \neq \mathbf{c}|\mathbf{r}) \quad (10.1)$$

from which the probability of error is given by

$$P(\epsilon) = \sum_{\mathbf{r}} P(\epsilon|\mathbf{r})P(\mathbf{r}) \quad (10.2)$$

Therefore, an optimum likelihood strategy is nothing more than the minimization of  $P(\epsilon)$ . In other words, the conditional error probability  $P(\epsilon|\mathbf{r})$  must be minimized because  $P(\mathbf{r})$  is independent of the decoding process.

According to Bayes' law, the conditional probability of the code sequence  $\mathbf{c}$ , given that the received sequence  $\mathbf{r}$  has been received, is

$$P(\mathbf{c}|\mathbf{r}) = \frac{P(\mathbf{r}|\mathbf{c})P(\mathbf{c})}{P(\mathbf{r})} \quad (10.3)$$

For a given  $\mathbf{r}$ ,  $P(\epsilon|\mathbf{r})$  can be minimized by choosing the most likely code sequence  $\hat{\mathbf{c}}$ , which results in  $P(\hat{\mathbf{c}}|\mathbf{r})$  being maximized. If all code words are equally likely, maximizing  $P(\hat{\mathbf{c}}|\mathbf{r})$  is equivalent to maximizing  $P(\mathbf{r}|\mathbf{c})$ .

Now suppose that an information sequence  $\mathbf{d} = (d_0, d_1, \dots, d_{kL-1})$  is encoded into a code sequence  $\mathbf{c} = (c_0, c_1, \dots, c_{N-1})$  of length  $N = n(L + m)$ , and that a received sequence  $\mathbf{r} = (r_0, r_1, \dots, r_{N-1})$  is received over a discrete memoryless channel (DMC). A maximum-likelihood decoder (MLD) for a DMC must locate a code sequence in such a way that the conditional probability

$$P(\mathbf{r}|\mathbf{c}) = \prod_{j=0}^{N-1} P(r_j|c_j) \quad (10.4)$$

is as large as possible, where  $\mathbf{r}$  is a received sequence and  $P(r_j|c_j)$ ,  $j = 0, 1, \dots, N-1$ , are the channel's crossover probabilities. Since the logarithm is a monotonically increasing function of its argument, the MLD could also look for a code sequence  $\mathbf{c}$  for which

$$L(\mathbf{r}|\mathbf{c}) = \sum_{j=0}^{N-1} \log P(r_j|c_j) \quad (10.5)$$

is as large as possible.  $L(\mathbf{r}|\mathbf{c}) \triangleq \log P(\mathbf{r}|\mathbf{c})$  is called the log-likelihood function, or the metric, associated with the path  $\mathbf{c}$ . Each individual term  $\log P(r_j|c_j)$  is called a bit metric.

For a binary symmetric channel with transition probability  $p < 1/2$ , the received sequence  $\mathbf{r}$  is binary ( $q = 2$ ), and it may differ from the transmitted code sequence  $\mathbf{c}$  in some positions as a result of the channel noise. Let  $d(\mathbf{r}, \mathbf{c})$  be the Hamming distance between  $\mathbf{r}$  and  $\mathbf{c}$ . For a convolutional code of length  $N = n(L + m)$ , the log-likelihood function becomes

$$\begin{aligned} L(\mathbf{r}|\mathbf{c}) &= d(\mathbf{r}, \mathbf{c}) \log p + [N - d(\mathbf{r}, \mathbf{c})] \log (1 - p) \\ &= d(\mathbf{r}, \mathbf{c}) \log \frac{p}{1 - p} + N \log (1 - p) \end{aligned} \quad (10.6)$$

Since  $\log [p/(1 - p)] < 0$  for  $p < 1/2$ , the optimum decoding for a BSC chooses  $\hat{\mathbf{c}}$  as the code sequence which minimizes the distance  $d(\mathbf{r}, \mathbf{c})$  between  $\mathbf{r}$  and  $\mathbf{c}$ .

### 10.3 Sequential Decoding

Prior to the discovery of the optimum algorithm by Viterbi, a number of other algorithms were proposed for decoding convolutional codes. There are two principal algorithms for sequential decoding, that is, the Fano algorithm and the stack algorithm. The most widely used sequential decoding technique to date is based on the Fano algorithm. But another interesting sequential decoding technique is based on the stack algorithms which have been proposed and analyzed independently by Zigangirov and Jelinek. However, the stack algorithm has not found wide acceptance because the Fano algorithm is considered to be the most practical to implement. Our presentations focus first on the Fano algorithm and then on the stack algorithm.

The fundamental idea of sequential decoding is that only the path that appears to be the most probable should be extended. Because of

the limited search, the decoder is never completely certain that the current path is the best. This approach can be viewed as a trial-and-error search for the correct path in the code tree. It performs the search in a sequential manner, always operating on a single path. Each time the decoder moves forward, a tentative decision is made. However, the decoder is also allowed to back up and change previous decisions. If an incorrect decision is made, subsequent extensions of the path will be wrong. The decoder is eventually able to recognize a wrong decision by examining the path metric, but it may happen that an incorrect path is extended for many branches. In this unfavorable case, a substantial amount of computation is required to recover the correct path. This variability in computational effort is one of the major drawbacks of sequential decoding because the decoder must search back and forth, trying alternate paths until it finally decodes successfully.

### 10.3.1 The Fano algorithm

The Fano algorithm searches the code tree for the most probable path by examining one path at a time. The decoder examines a sequence of nodes in the code tree, starting with the origin node and ending with one of the terminal nodes. The decoder looks forward (or deeper) into the code tree. If the decoder sees a node it likes, it moves forward to that node. If not, the decoder moves backward and then tries to move forward along another path. It decides whether it likes a given node by comparing the metric  $\varphi$  of that node to a running threshold  $T$ . The decoder stops as soon as it reaches one of the terminal nodes.

The decoder always moves to an adjacent node, either forward or backward to that node with respect to the present node, depending on whether the metric values increase or decrease. Some nodes are visited more than once, and then their metric values must be recomputed. A fundamental idea of sequential decoding is to bias the metrics in such a way that backing up and going forward to unlikely paths are reduced to a minimum. The bit metric  $\varphi_i$  to be used for a DMC is

$$\varphi_i = \log_2 \frac{P(r_i|c_i)}{P(r_i)} - B \quad (10.7)$$

where  $B$  is an arbitrary bias term which should be selected so that the branch metrics are positive along the correct path and negative along incorrect paths. The partial path metric for the first  $l$  branches of a path  $\mathbf{c}$  is given by

$$\begin{aligned}
 \varphi_{l-1} &= \sum_{i=0}^{nl-1} \left[ \log_2 \frac{P(r_i|c_i)}{P(r_i)} - B \right] \\
 &= \sum_{i=0}^{nl-1} \log_2 P(r_i|c_i) - \sum_{i=0}^{nl-1} \log_2 P(r_i) - nlB
 \end{aligned} \tag{10.8}$$

This is the sum over the length of the path  $\mathbf{c}$  being compared to the corresponding  $\mathbf{r}$ . This biasing of the likelihood function is common to all sequential decoding algorithms, and the bias value that minimizes the average computational effort per decoded bit is equal to the code rate  $R$ . This bias value was first proposed by Fano, and the metric  $\varphi_i$  of Eq. (10.7) is usually called the Fano metric.

The details of Fano's algorithm are given in the flowchart of Fig. 10.1. Some explanatory notes for the flowchart are described below.

1. The decoder starts at the origin node  $s$  with the threshold  $T = 0$  and the metric value  $\varphi_s = 0$ . If the decoder looks backward from the origin node, we assume the metric  $\varphi$  to be  $-\infty$  there. This prevents a backward move from the origin node  $s$ .

2. It then looks forward to the best node with the largest metric. The threshold  $T$  changes throughout the algorithm by multiples of  $\Delta$ , a preselected threshold increment. At all times,  $T$  will be less than or equal to the metric of the current node. Ties in metric values can be arbitrarily resolved without affecting average decoder performance.

#### Loop I

3. Let  $\varphi_F$  denote the metric of the forward node to be examined. If  $\varphi_F \geq T$ , the decoder moves to the node that is one branch forward.

4. The "End of tree?" query asks whether the current node is a terminal node. The algorithm stops if the answer is Yes, but moves to the next step for a threshold tightening if it is No.

5. The  $T$  tight? query only occurs after a "Move forward" step. At a given node the threshold  $T$  is tight if increasing  $T$  by  $\Delta$  would make it larger than the metric of that node. Otherwise, the threshold is tightened by increasing it in increments of  $\Delta$ . Once  $T$  is tight, the decoder goes back to the step "Look forward to best node."

#### Loop II

6. If  $\varphi_F < T$ , the decoder then looks backward to the node one branch back.

7. Let  $\varphi_B$  denote the metric of the backward node being examined. If  $\varphi_B < T$ , then  $T$  is lowered by  $\Delta$  and the "Look forward to best node" step is repeated.



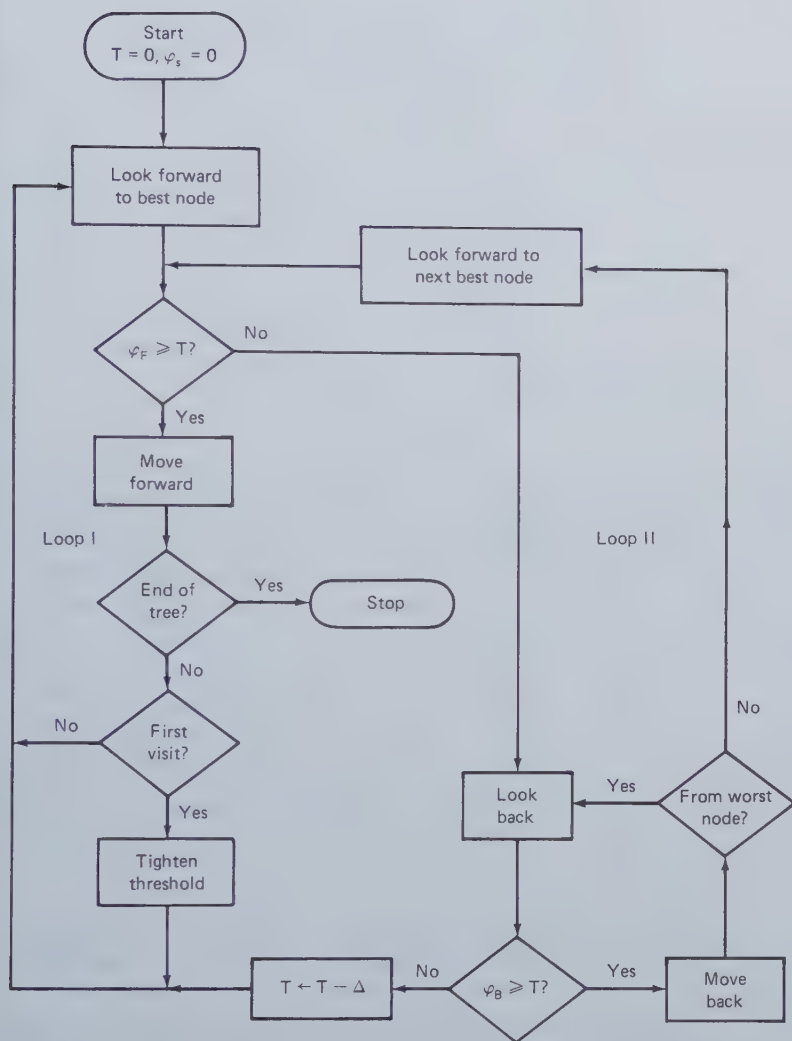


Figure 10.1 A flowchart describing the Fano sequential decoding algorithm.

8. If  $\varphi_B \geq T$ , the decoder moves back to node  $D$  one branch at a time.

9. If this backward move was from the worst node succeeding node  $D$ , the decoder again looks back to the node preceding node  $D$ .

10. If it is not, the decoder looks forward to the next best node succeeding node  $D$  and checks if  $\varphi_F \geq T$ .

To better understand the Fano algorithm, consider the  $R = 1/4$ ,  $m = 2$  convolutional code with the encoder shown in Fig. 10.2.



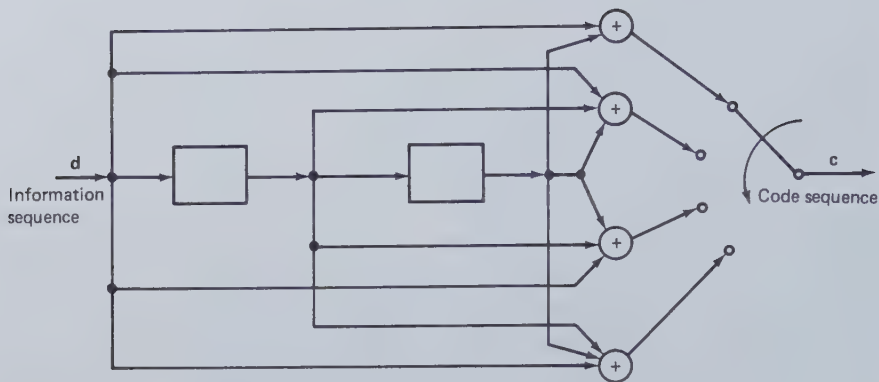


Figure 10.2 A (4, 1) convolutional encoder with  $m = 2$ .

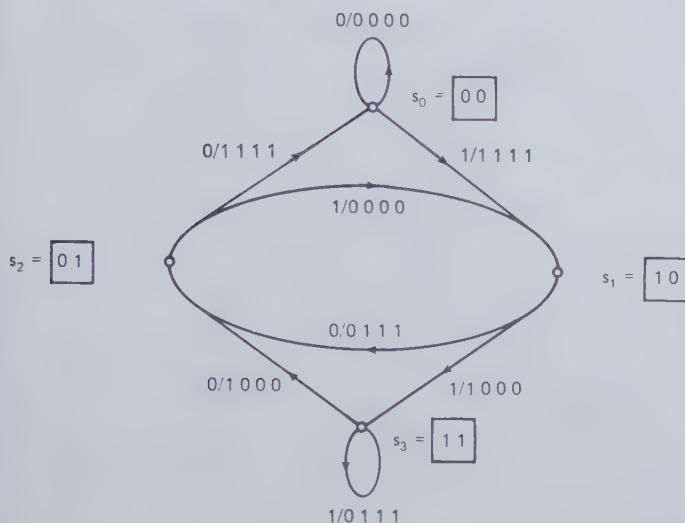
**Example 10.1** The generator sequences for the encoder of Fig. 10.2 are  $\mathbf{g}^{(1)} = (1\ 0\ 1)$ ,  $\mathbf{g}^{(2)} = (1\ 1\ 1)$ ,  $\mathbf{g}^{(3)} = (1\ 1\ 1)$ , and  $\mathbf{g}^{(4)} = (1\ 1\ 1)$ . The number of encoder states is four and the number of transitions is eight. The contents of the state table for a (4, 1, 2) convolutional encoder are tabulated in Table 10.1.

From the state table of Table 10.1, we can draw the state diagram and the code tree shown in Figs. 10.3 and 10.4, respectively. The information sequence dictates a path through the code tree, and the output sequence encountered along this path constitutes the resulting encoded sequence. The code sequence  $\mathbf{c} = (1\ 1\ 1\ 1, 0\ 1\ 1\ 1, 0\ 0\ 0\ 0, 1\ 0\ 0\ 0, 0\ 1\ 1\ 1, 1\ 0\ 0\ 0, 1\ 1\ 1\ 1)$  is the coded output corresponding to the information sequence  $\mathbf{d} = (1\ 0\ 1\ 1\ 1)$ . Assume that a code word  $\mathbf{c}$  is transmitted from this code and that the sequence  $\mathbf{r} = (1\ 0\ 1\ 1, 1\ 1\ 1\ 1, 0\ 0\ 0\ 0, 0\ 0\ 0\ 0, 0\ 1\ 1\ 1, 1\ 0\ 0\ 0, 1\ 1\ 1\ 1)$  is received. That is, there are single errors in the first, second, and fourth branches. The contents of each branch after each step of the Fano algorithm are shown in Table 10.2 for a value of  $\Delta = 1$ . The algorithm for the tree search terminates after 12 decoding steps, and the final decoded information is  $\mathbf{d} = (1\ 0\ 1\ 1\ 1)$ , as indicated in Table 10.2.

A Fano algorithm has to detect a decreasing metric and quickly find a path with an increasing metric. To do this, the running threshold  $T$  should be raised or lowered in multiples of  $\Delta$ . To find the most probable path, the threshold must be lowered below the minimum metric along the most probable path. The threshold is raised whenever possible so that the difference between the threshold and the path metric

TABLE 10.1 State Table for a (4, 1, 2) Convolutional Encoder

| State    | Input             |                   |
|----------|-------------------|-------------------|
|          | 0                 | 1                 |
|          | Transition/output | Transition/output |
| $s_0$ 00 | 00/0000           | 10/1111           |
| $s_1$ 10 | 01/0111           | 11/1000           |
| $s_2$ 01 | 00/1111           | 10/0000           |
| $s_3$ 11 | 01/1000           | 11/0111           |



**Figure 10.3** State diagram for the (4, 1, 2) convolutional encoder shown in Fig. 10.2.

along the correct path is less than  $\Delta$ . When the current metric falls below the running threshold, it is an indication that the path being followed may be wrong and that a search for a good path must be initiated. Normally, the decoder moves forward on the most probable branch at each node as long as the metric lies above the threshold. When the decoder cannot move forward, it attempts to move back and searches all paths that have not been visited using the current threshold. When no paths can be found that satisfy the threshold, it is lowered by  $\Delta$  and the decoder begins searching forward again. If the noise becomes more severe and the metric falls below the threshold after a forward move, the decoder moves backward in order to find a path that lies above the current threshold. If no path can be found whose metric stays above the threshold, the threshold is then lowered and the decoder attempts to move forward again with a lower threshold. However, even though the metric for the correct path may temporarily show large decreases as a result of channel noise, typical metric behavior shows that the metric should be an increasing function over a long period of time. Also, if a burst noise occurs, the metric on an incorrect path may increase temporarily, making the path look like a good path, but it will usually start decreasing when the noise subsides. The number of computations by the Fano algorithm depends on how the threshold increment  $\Delta$  is chosen. If  $\Delta$  is too small, a large number of computations will result. Raising  $\Delta$  properly may bring about a reduction in the number of computations; however, the threshold increment  $\Delta$  cannot be raised indefinitely. If  $\Delta$  is too large, it

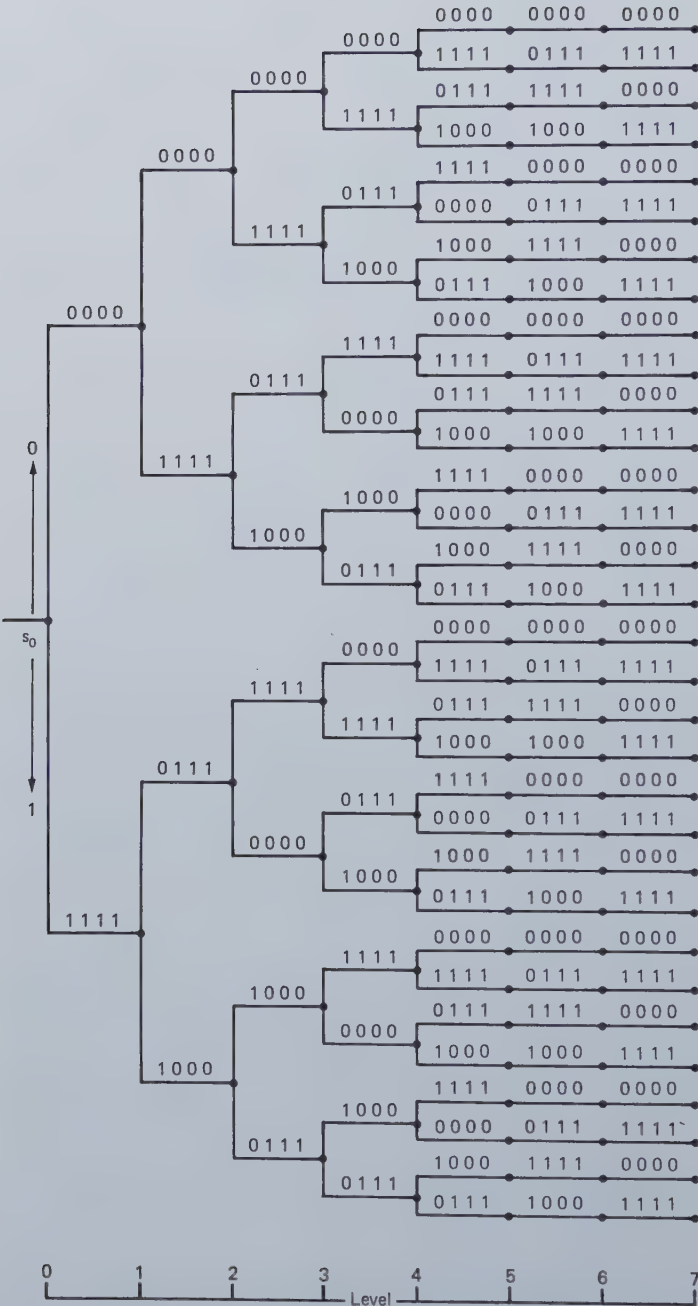


Figure 10.4 Code tree for the (4, 1, 2) convolutional code.

TABLE 10.2 Tree Search of the Fano Algorithm with  $\Delta = 1$  for the Example of Fig. 10.2

| Step | Look* | $\varphi_F$ | $\varphi_B$ | Node     | Metric | T    |
|------|-------|-------------|-------------|----------|--------|------|
| 0    | —     | —           | —           | <i>s</i> | 0      | 0    |
| 1    | FBN   | -1          | $-\infty$   | <i>s</i> | 0      | -1   |
| 2    | FBN   | -1          | —           | 1        | -1     | -1   |
| 3    | FBN   | -2          | 0           | <i>s</i> | 0      | -1   |
| 4    | FBNB  | -11         | $-\infty$   | <i>s</i> | 0      | -2   |
| 5    | FBN   | -1          | —           | 1        | -1     | -2   |
| 6    | FBN   | -2          | —           | 10       | -2     | -2   |
| 7    | FBN   | 2           | —           | 101      | 2      | 2    |
| 8    | FBN   | 1           | -2          | 101      | 2      | 1    |
| 9    | FBN   | 1           | —           | 1011     | 1      | 1    |
| 10   | FBN   | 5           | —           | 10111    | 5      | 5    |
| 11   | FBN   | 9           | —           | 101110   | 9      | 9    |
| 12   | FBN   | 13          | —           | 1011100† | 13     | Stop |

\*FBN, look forward to best node; FBNB, look forward to next best node.

†Final decoded information

may cause the number of computations to increase again. Experience has shown that the threshold increment  $\Delta$  should be between 2 and 10.

### 10.3.2 The stack algorithm

Another interesting class of sequential decoding algorithms, known collectively as the stack algorithm, has been developed to reduce the computational work required for the Fano algorithm. In comparison with the Fano algorithm, the stack algorithm is conceptually simpler because there is no retracing over the same path as in the Fano algorithm. However, the stack or ZJ algorithm has not found wide acceptance because the Fano algorithm is considered to be the most practical to implement.

In the stack algorithm, the decoder creates a stack of previously examined paths of varying lengths. Each stack entry contains a path along its metric, where the path with the largest metric is placed on top and the others are ordered according to their metric values. Of course, we use the same metrics and the same code tree as the Fano algorithm. At each decoding step, the path at the top of the stack is extended to its  $2^k$  successors by computing their branch metrics. The top path is then deleted from the stack and replaced by its  $2^k$  successors. Then the new stack is reordered according to decreasing metric values, and the same process is repeated. This decoding step continues until the path at the top of the stack is at the end of the tree or coded sequence. Thus, the stack algorithm will be terminated. A flow diagram for a typical stack algorithm is shown in Fig. 10.5.

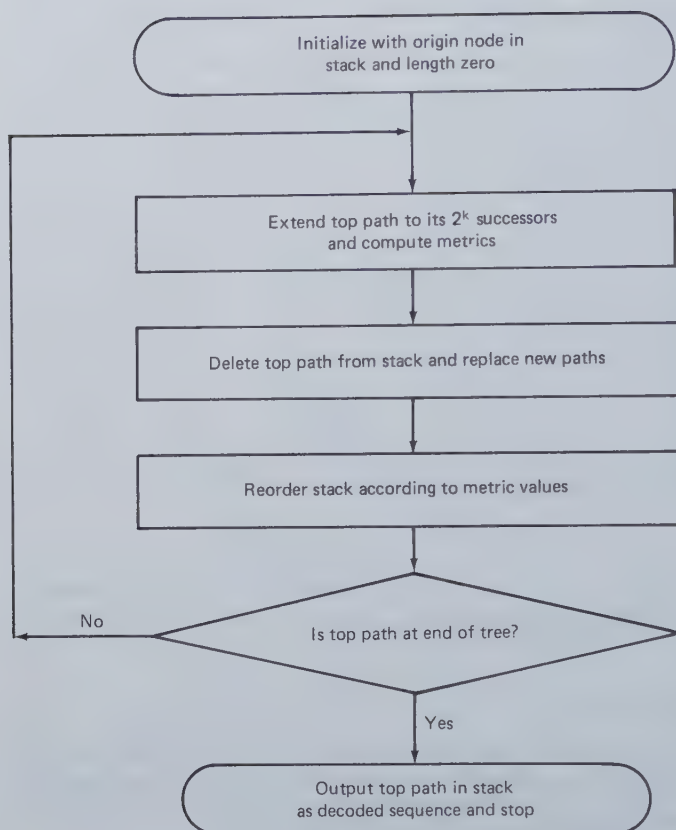


Figure 10.5 Stack decoding algorithm

A binary-input  $q$ -nary-output soft-decision channel is said to be symmetric if

$$P(j|0) = P(q-1-j|1) \quad j = 0, 1, \dots, q-1 \quad (10.9)$$

For a symmetric channel with equally likely input digits, the channel output digits are also equally likely. Thus, for all  $i$

$$P(r_i = j) = \frac{1}{q} \quad 0 \leq j \leq q-1 \quad (10.10)$$

Substituting Eq. (10.10) into Eq. (10.8), we have the partial path metric for the first  $l$  branches:

$$\varphi_{l-1} = \sum_{i=0}^{nl-1} \log_2 P(r_i|c_i) + nl(\log_2 q - B) \quad (10.11)$$



The second term in Eq. (10.11) shows a positive bias because  $q \geq 2$  and  $B \leq 1$ . Therefore, the longer paths are more likely to be part of the most probable path because the longer paths have a larger bias than the shorter paths.

For a BSC with equally likely digits, the bit metric  $\varphi_i$ , using Eq. (10.7), becomes

$$\varphi_i = \begin{cases} \log_2 2p - B & r_i \neq c_i \\ \log_2 2(1 - p) - B & r_i = c_i \end{cases} \quad (10.12)$$

where  $p$  is a transition probability of BSC and the bias term  $B$  is equal to the code rate  $R$ .

To best understand the stack algorithm, we will repeat Example 10.1 using the stack algorithm.

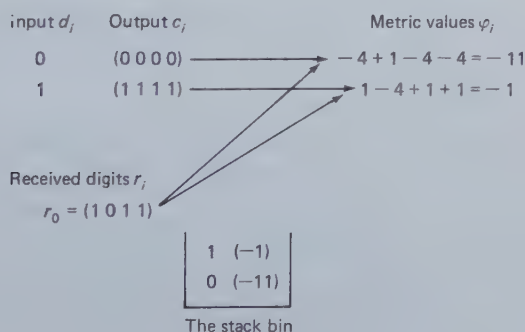
**Example 10.2** Consider the application of the stack algorithm to the  $R = 1/4$ ,  $m = 2$  convolutional code with the code tree of Fig. 10.4. For a BSC ( $q = 2$ ) with transition probability  $p = 0.12$ , the bit metrics of Eq. (10.12) are given as

$$\varphi_i = \begin{cases} \log_2 0.24 - 0.25 = -2.308 & r_i \neq c_i \\ \log_2 1.76 - 0.25 = 0.566 & r_i = c_i \end{cases}$$

Thus we have the metric table shown in Table 10.3a. It is common practice to scale the metrics, with the scale factor of  $1/0.566$ , by integers for ease of implementation. The integer metric table is shown in Table 10.3b.

Suppose that a code word  $\mathbf{c} = (1\ 1\ 1\ 1, 0\ 1\ 1\ 1, 0\ 0\ 0\ 0, 1\ 0\ 0\ 0, 0\ 1\ 1\ 1, 1\ 0\ 0\ 0, 1\ 1\ 1\ 1)$  is transmitted from this code over a BSC with  $p = 0.12$  and that the sequence  $\mathbf{r} = (1\ 0\ 1\ 1, 1\ 1\ 1\ 1, 0\ 0\ 0\ 0, 0\ 0\ 0\ 0, 0\ 1\ 1\ 1, 1\ 0\ 0\ 0, 1\ 1\ 1\ 1)$  is received. There exist single errors in the first, second, and fourth branches as assumed in Example 10.1. From the integer metric table of Table 10.3b, the contents of the stack after each step of the algorithm are shown below.

Step 1

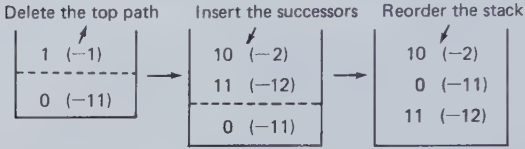


Step 2

$$1\ 0\ (1\ 1\ 1\ 1,\ 0\ 1\ 1\ 1) \rightarrow (-1) + (-1) = -2$$

$$1\ 1\ (1\ 1\ 1\ 1,\ 1\ 0\ 0\ 0) \rightarrow (-1) + (-11) = -12$$

$$r_0\ r_1 = (1\ 0\ 1\ 1,\ 1\ 1\ 1\ 1)$$

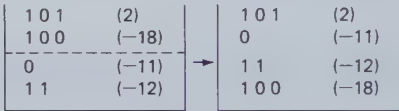


Step 3

$$1\ 0\ 0\ (1\ 1\ 1\ 1,\ 0\ 1\ 1\ 1,\ 1\ 1\ 1\ 1) \rightarrow (-2) + 4(-4) = -18$$

$$1\ 0\ 1\ (1\ 1\ 1\ 1,\ 0\ 1\ 1\ 1,\ 0\ 0\ 0\ 0) \rightarrow (-2) + 4 = 2$$

$$r_0\ r_1\ r_2 = (1\ 0\ 1\ 1,\ 1\ 1\ 1\ 1,\ 0\ 0\ 0\ 0)$$

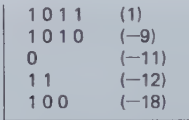


Step 4

$$1\ 0\ 1\ 0\ (1\ 1\ 1\ 1,\ 0\ 1\ 1\ 1,\ 0\ 0\ 0\ 0,\ 0\ 1\ 1\ 1) \rightarrow (2) + 1 + 3(-4) = -9$$

$$1\ 0\ 1\ 1\ (1\ 1\ 1\ 1,\ 0\ 1\ 1\ 1,\ 0\ 0\ 0\ 0,\ 1\ 0\ 0\ 0) \rightarrow (2) + (-4) + 3 = 1$$

$$r_0\ r_1\ r_2\ r_3 = (1\ 0\ 1\ 1,\ 1\ 1\ 1\ 1,\ 0\ 0\ 0\ 0,\ 0\ 0\ 0\ 0)$$

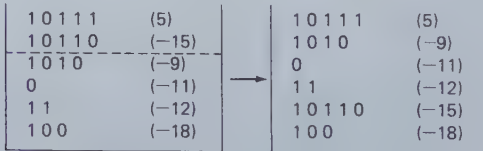


Step 5

$$1\ 0\ 1\ 1\ 0\ (1\ 1\ 1\ 1,\ 0\ 1\ 1\ 1,\ 0\ 0\ 0\ 0,\ 1\ 0\ 0\ 0,\ 1\ 0\ 0\ 0) \rightarrow (1) + 4(-4) = -15$$

$$1\ 0\ 1\ 1\ 1\ (1\ 1\ 1\ 1,\ 0\ 1\ 1\ 1,\ 0\ 0\ 0\ 0,\ 1\ 0\ 0\ 0,\ 0\ 1\ 1\ 1) \rightarrow (1) + 4 = 5$$

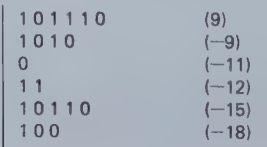
$$r_0\ r_1\ r_2\ r_3\ r_4 = (1\ 0\ 1\ 1,\ 1\ 1\ 1\ 1,\ 0\ 0\ 0\ 0,\ 0\ 0\ 0\ 0,\ 0\ 1\ 1\ 1)$$



Step 6

$$1\ 0\ 1\ 1\ 1\ 0\ (1\ 1\ 1\ 1,\ 0\ 1\ 1\ 1,\ 0\ 0\ 0\ 0,\ 1\ 0\ 0\ 0,\ 0\ 1\ 1\ 1,\ 1\ 0\ 0\ 0) \rightarrow 5 + 4 = 9$$

$$r_0\ r_1\ r_2\ r_3\ r_4\ r_5 = (1\ 0\ 1\ 1,\ 1\ 1\ 1\ 1,\ 0\ 0\ 0\ 0,\ 0\ 0\ 0\ 0,\ 0\ 1\ 1\ 1,\ 1\ 0\ 0\ 0)$$



Step 7

1011100(1111,0110,0000,1000,0111,1000,1111)  $\rightarrow 9 + 4 = 13$   
 $r = (1011, 1111, 0000, 0000, 0111, 1000, 1111)$

|                              |   |         |       |
|------------------------------|---|---------|-------|
| Decoded information sequence | d | 1011100 | (13)  |
|                              |   | 1010    | (-9)  |
|                              |   | 0       | (-11) |
|                              |   | 11      | (-12) |
|                              |   | 10110   | (-15) |
|                              |   | 100     | (-18) |

The interesting point is that the stack algorithm requires only seven decoding steps in order to reach the same final decoded path as the Fano algorithm, which required 12 steps. However, this comparison is somewhat deceiving because a stack algorithm computation is much more complex than a Fano algorithm computation. Particularly troublesome is the operation of reordering the stack.

The most undesirable feature of the stack algorithm is the necessity for reordering the stack, which becomes quite time-consuming when the number of entries in the stack is large and which places severe limitations on the decoding speed. There are several modifications for this stack algorithm which make it more practical to implement. The Jelinek algorithm partially alleviates this problem by quantizing the ordering. The contents of the stack do not have to be reordered after each decoding step. But this algorithm also has the disadvantage that the best path is not always extended, only a very good path. Nevertheless, the time saved by that result from the Jelinek algorithm is considerable. A second serious disadvantage of the stack algorithm is that the stack size required to advance the correct path by one branch is proportional to the number of computations. The required stack size is a Parleto-distributed random variable. This problem is partially alleviated by the Zigangirov algorithm, which can discard any path from the stack whenever its metric falls more than a fixed value below the metric of the top path. One advantage of the stack algorithm is its

TABLE 10.3 Metric Table for an  $R = 1/4$  Convolutional Code and a BSC with  $p = 0.12$

| $r_i$                    | $c_i$  |        |
|--------------------------|--------|--------|
|                          | 0      | 1      |
| (a) Metric Table         |        |        |
| 0                        | 0.566  | -2.308 |
| 1                        | -2.308 | 0.566  |
| (b) Integer Metric Table |        |        |
| 0                        | 1      | -4     |
| 1                        | -4     | 1      |

ability to use idle periods effectively. When the buffer is empty, the Fano algorithm must wait for the next branch to be received before further calculations can be made. But the stack algorithm can use its idle periods to extend some of paths further down in the stack. Thus the stack decoder need never be idle. Since the stack algorithm requires more storage than the Fano algorithm, the stack decoder will require considerably more memory.

### 10.3.3 Computational performance of the sequential decoder

In sequential decoding, the number of computations required to advance one node deeper into the code tree is a random variable. This characteristic strongly affects the complexity of achieving a given level of performance. Since the number of computations required to decode a frame of data is a random variable, its probability distribution must be computed in order to determine computational performance. The computational distribution of sequential decoding has been studied by many coding theorists for a number of years. One of the major important results of these studies is the lower bound to the distribution of computation proposed by Jacobs and Berlekamps, which is applicable to any sequential decoding algorithm. Without showing the derivation of this bound, a brief summary is given below.

Assume that we have an  $R = k/n$  convolutional code with  $2^k$  branches per node and  $n$  digits per branch. The probability distribution that  $C$  is greater than  $\theta$  is shown as

$$P(C > \theta) < \alpha \theta^{-\rho} \quad 0 < \rho < \infty \quad (10.13)$$

where  $C$  is the number of computations required to decode the first  $L$  information symbols, and  $\alpha$  and  $\rho$  are functions of the BSC crossover probability  $p$  and the code rate  $R$  only. Equation (10.13) is referred to as the Pareto distribution. It is important to note here that the Pareto distribution is independent of the encoder memory, and hence the computational behavior of sequential decoding does not depend on code constraint length. Unless  $\rho$  is very large, this distribution decreases slowly with  $\theta$ , which creates the necessity for a larger buffer. The computational behavior of the sequential decoder can also be described in terms of the Gallager function  $E_0(\rho)$ . The Pareto exponent  $\rho$  is an extremely important parameter for sequential decoding because it determines how rapidly  $P(C > \theta)$  decreases as a function of  $\rho$ . The Pareto exponent  $\rho$  is related to the code rate  $R$  by the Gallager function:

$$R = \frac{E_0(\rho)}{p} \quad 0 < R < C_0 \quad 0 < \rho < \infty \quad (10.14)$$

where  $C_0$  is the channel capacity. For a BSC with the transition probability  $p$ , the Gallager function is expressed by

$$E_0(\rho) = \rho - \log_2 [p^{1/1+\rho} + (1-p)^{1/1+\rho}]^{1+\rho} \quad (10.15)$$

and the code rate is

$$R_0 = E_0(1) = 1 - \log_2 [1 + 2\sqrt{p(1-p)}] \quad (10.16)$$

which is called the computational cutoff rate of the channel. This is a practical limit on the highest rate at which a sequential decoder can operate because a Pareto distribution with  $\rho = 1$  is a distribution with an infinite mean. For the average computational load of sequential decoding to be finite,  $R$  must be less than  $R_0$ . This requirement indicates that any sequential decoder operation where  $R > R_0$  will have a severe computational problem because of frequent buffer overflows. The minimum feasible value of  $E_b/N_0$  (dB) at which a sequential decoder can operate is usually predicted by calculating  $R_0$ . Since codes are usually chosen so that undetected-error rates are extremely small under typical operating conditions, the computational problem has the dominant effect on system performance. Thus,  $R_0$  is an important performance parameter.

## 10.4 Maximum-Likelihood Decoding

A practical technique for maximum-likelihood decoding is the Viterbi algorithm introduced by Viterbi in 1967. This decoding algorithm uses the trellis structure of the code and determines the maximum-likelihood estimate of the transmitted sequence that has the largest metric. The log-likelihood function represents the metric which can be computed for each path in the trellis and is additive over the received sequence for memoryless channels. The survivor is defined as the most probable path that has the largest accumulated metric. With these definitions, the Viterbi algorithm simply finds the path through the trellis with the largest accumulated metric in such a way that it processes a received sequence in an iterative manner. At each step, it compares the metric of all paths entering each state, stores only the survivor with the largest accumulated metric, and discards the unlikely paths at every state, which reduces the decoding effort. Therefore the Viterbi decoder must produce an estimate  $\hat{\mathbf{c}}$  of the code sequence  $\mathbf{c}$  based on the received sequence  $\mathbf{r}$ . In the code trellis, the upper branch leaving each state represents 0, while the lower branch



represents 1. Each branch is labeled with the  $n$  corresponding digits, and each of the  $2^{kL}$  code words of length  $N = n(L + m)$  is represented by a unique path through the trellis. A maximum likelihood decoder for a DMC chooses  $\hat{\mathbf{c}}$  as the code word  $\mathbf{c}$  which maximizes the log-likelihood function  $\log P(\mathbf{r}|\mathbf{c}) = L(\mathbf{r}|\mathbf{c})$ . Referring to Eq. (10.6), we see that maximizing the log-likelihood function (or metric)  $L(\mathbf{r}|\mathbf{c})$  for a BSC with the transition probability of  $p < 1/2$  is equivalent to minimizing the Hamming distance  $d(\mathbf{r}, \mathbf{c})$  between the two sequences  $\mathbf{c}$  and  $\mathbf{r}$ . Thus the maximum-likelihood decoder for a BSC chooses  $\hat{\mathbf{c}}$  as the path through the trellis which is closest to  $\mathbf{r}$  with respect to the Hamming distance.

A DMC is an idealized model of a noisy channel with a binary input and  $q$ -nary outputs. The input to a DMC is a sequence of binary digits, and the output is a sequence of soft-quantized digits. Assume that an information sequence  $\mathbf{d} = (d_0, d_1, \dots, d_{L-1})$  of length  $L$  is encoded into a code word  $\mathbf{c} = (c_0, c_1, \dots, c_{N-1})$  of length  $N = n(L + m)$ , and that a  $q$ -nary sequence  $\mathbf{r} = (r_0, r_1, \dots, r_{N-1})$  is received over the DMC. The received output  $j, j = 0, 1, \dots, q - 1$ , is assumed to be statistically dependent only on the corresponding input  $i, i = 0, 1$ , and determined by a conditional probability  $P(j|i)$ . If  $q = 2$ ,  $\mathbf{r}$  is a binary sequence, and the demodulator output for BSC is said to be hard quantized. If  $q > 2$ , the demodulator output is said to be soft quantized. It is well known that for additive white gaussian noise (AWGN), the digital channel can be modeled as a DMC and soft quantization yields a 2- to 3-dB performance improvement over hard quantization. In other words, a soft decoder gives the same error probability as a hard decoder with 2 to 3 dB less signal power at the transmitter.

As discussed above, if the decoding is performed over a BSC, the Hamming distance is used as the metric. However, for a DMC with soft quantization ( $q > 2$ ), it is more convenient to use positive integers as metrics rather than the actual bit metrics as a matter of implementation practice. The bit metric  $L(r_i|c_i) = \log P(r_i|c_i)$  can be replaced by  $\alpha[\log P(r_i|c_i) + \beta]$ , where  $\alpha$  is any real number and  $\beta$  is any positive real number. In practice it has been found that  $\alpha$  and  $\beta$  may be chosen in such a way that the smallest bit metric is zero and all other bit metrics are positive integers. There are many sets of integer metrics possible for a given DMC, depending on the choice of  $\alpha$  and  $\beta$ . The performance of the Viterbi decoder becomes slightly suboptimum because of the modified metric approximation by integers, but the performance degradation is negligible. In fact it has been verified that the use of 8-nary signaling results in a performance improvement of roughly 2 to 3 dB relative to binary signaling. But the cost of this improvement is increased demodulator complexity. Even when the number of quanti-

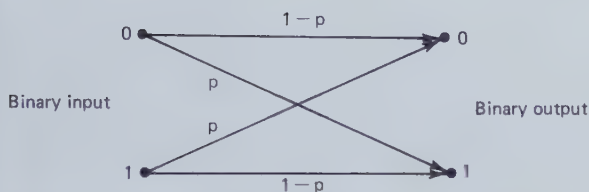


Figure 10.6 BSC representation (hard-quantized DMC).

zation levels is increased beyond eight ( $q > 8$ ), the performance is nearly as good as with eight quantization levels.

**Example 10.3** Consider the binary-input binary-output BSC with transition probability  $p < 1/2$  as shown in Fig. 10.6. Then we have  $P(1|0) = P(0|1) = p$  and  $P(0|0) = P(1|1) = 1 - p$ . Let  $\mathbf{c}$  be a transmitted code word of length  $N$  in a BSC with transition probability  $p$ , and let  $\mathbf{r}$  be the corresponding received sequence which is binary ( $q = 2$ ). From Eq. (10.6), the log-likelihood function becomes

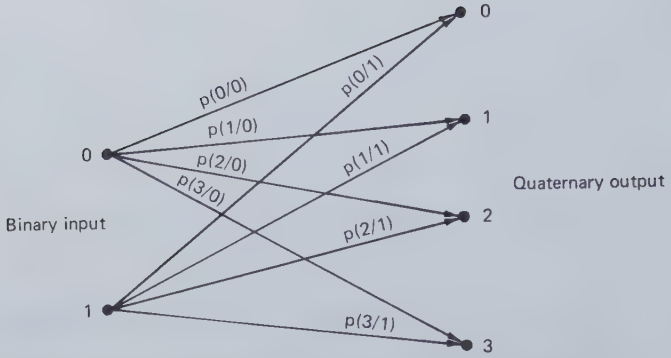
$$\begin{aligned} \log P(\mathbf{r}|\mathbf{c}) &= \log [p^t(1-p)^{N-t}] \\ &= -\log\left(\frac{1-p}{p}\right)^t + N \log(1-p) \\ &= -\mu_1 t - \mu_2 \end{aligned} \quad (10.17)$$

where  $t = d(\mathbf{r}, \mathbf{c})$  is the number of errors that occurred in the transmission of  $\mathbf{c}$  of length  $N$ . Hence,  $\mathbf{c}$  and  $\mathbf{r}$  differ in  $t$  positions, and their Hamming distance  $d(\mathbf{r}, \mathbf{c})$  is  $t$ . Since  $p < 1/2$ ,  $\mu_1$  and  $\mu_2$  in Eq. (10.17) are positive constants. Therefore, maximizing the log-likelihood function (or metric) is equivalent to minimizing the Hamming distance  $t$ . Consequently, in applying the Viterbi algorithm to the BSC, the Viterbi decoder must find the path closest to  $\mathbf{r}$  in the Hamming distance through the trellis.

**Example 10.4** Consider the binary-input quaternary-output ( $q = 4$ ) DMC shown in Fig. 10.7. Assume that the transition probabilities are

$$\begin{array}{llll} P(0|0) = 0.50 & P(1|0) = 0.25 & P(2|0) = 0.15 & P(3|0) = 0.10 \\ P(0|1) = 0.10 & P(1|1) = 0.15 & P(2|1) = 0.25 & P(3|1) = 0.50 \end{array}$$

The bit metrics for this channel  $\log_{10} P(r_j|c_i)$  are shown in Table 10.4a. By using the modified metric  $\alpha [\log_{10} P(r_j|c_i) + \beta]$  for  $\alpha = 10.1$  and  $\beta = 1$ , we obtain the integer metric table shown in Table 10.4b. Depending on the choice of  $\alpha$ , there are many sets of integer metrics possible for a given DMC. But the performance degradation will be very slight.



**Figure 10.7** Binary-input  $q$ -nary-output DMC (four-level soft-quantized representation).

**TABLE 10.4** Metric Tables for the DMC of Fig. 10.7

| $c_i$                                      | $r_j$ |       |       |      |
|--------------------------------------------|-------|-------|-------|------|
|                                            | 0     | 1     | 2     | 3    |
| (a) $\log_{10} P(r_j c_i)$                 |       |       |       |      |
| 0                                          | -0.3  | -0.6  | -0.82 | -1   |
| 1                                          | -1    | -0.82 | -0.6  | -0.3 |
| (b) $\alpha[\log_{10} P(r_j c_i) + \beta]$ |       |       |       |      |
| 0                                          | 7     | 4     | 2     | 0    |
| 1                                          | 0     | 2     | 4     | 7    |

**10.4.1 The Viterbi algorithm**

Maximum-likelihood decoding implies finding the path with largest metric through the trellis by comparing the metrics of all branch paths entering each state with the corresponding level elements of the received sequence  $\mathbf{r}$  in an iterative manner. In the decoding process, if at some level it is found that a path cannot possibly yield the largest metric, then that path would be discarded by the decoder. Thus, a decoder that compares the metrics of all paths entering a state and that keeps only the survivor at that state will yield a most likely path if the operation is repeated for all distinct states at each level. This simple iterative process is known as the Viterbi algorithm. There are  $2^\gamma$  states per level in the trellis; hence  $2^\gamma$  survivors must be determined for each level, yielding a decoding effort that would vary with  $2^\gamma$  for an encoder with memory order  $\gamma$ . Therefore, the exponential growth of

the decoding effort is related to the encoder memory order  $\gamma = m$ . The Viterbi decoder always moves forward; there is no backing up. The iterative steps for Viterbi decoding involve computing the branch metrics and the total accumulated metric, performing a pairwise comparison, discarding nonsurvivors, and selecting the proper path. Thus the Viterbi algorithm is explained by the following iterative process.

*Step 1:* Up to the level  $0 < j \leq m$ , compute the branch metric for the single path entering each state and store the accumulated metric of each state. For a BSC, compute the Hamming distance from  $\mathbf{r}$  to the branch entering each state.

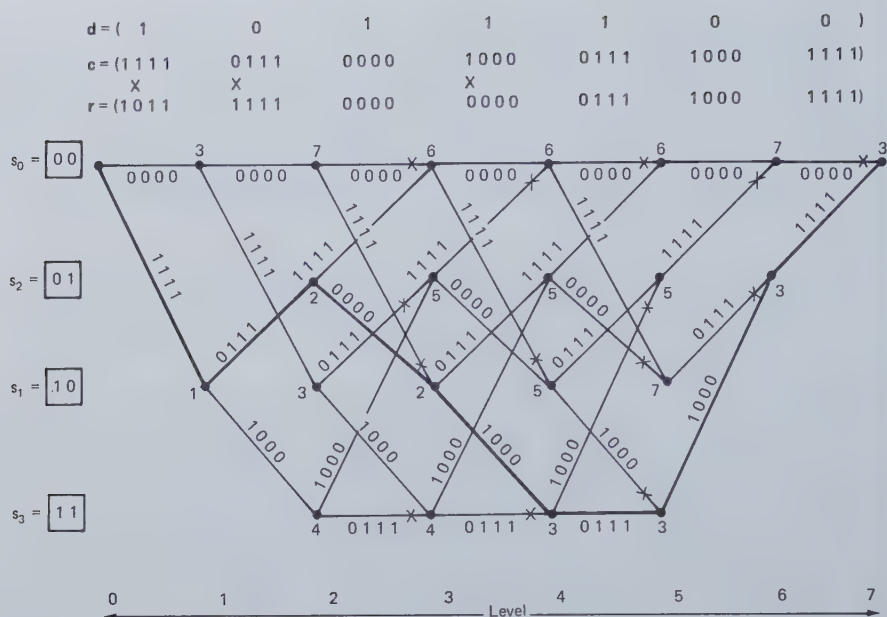
*Step 2:* For  $m < j \leq L$ , compute the accumulated path metric for all the paths entering a state by adding the branch metric entering that state to the path metric at the previous level. Store the path which is the survivor with the largest metric and discard all the nonsurvivors. For each state in this center portion of the trellis, there are  $2^\gamma$  survivors from the level  $m$  through the level  $L$ , one for each of the  $2^\gamma$  states. For a BSC, in lieu of the metric, the Hamming distance is computed by adding the distance of the survivor at the previous level to its distance from  $\mathbf{r}$  on the next branch.

*Step 3:* For  $L < j \leq m + L$  in the tail portion of the trellis, there are few states because the encoder is returning to the all-zero state. Repeat step 2 for each level. The metric for a path is computed by adding the metric of the survivor at the previous level to its branch metric. Stop at the level  $L + m$  where there is only one all-zero state. Thus the algorithm terminates.

The way in which the Viterbi algorithm corrects a specific error pattern is best explained by using the trellis diagram for an  $(n, k)$  convolutional code with memory order  $m$  as follows.

**Example 10.5** For the application of the Viterbi algorithm to a BSC where the demodulator provides only hard decisions, let us consider the  $R = 1/4$ ,  $m = 2$  convolutional encoder shown in Fig. 10.2. In this case the Hamming distance between the received sequence and the transmitted code words can be used as a metric. Let the 5-bit information sequence be  $\mathbf{d} = (1\ 0\ 1\ 1\ 0\ 0)$ , where the last 2 bits correspond to the tail of the information sequence. This tail portion is not considered part of the information sequence. As shown in Fig. 10.8, the code sequence  $\mathbf{c} = (1\ 1\ 1\ 1, 0\ 1\ 1\ 1, 0\ 0\ 0\ 0, 1\ 0\ 0\ 0, 0\ 1\ 1\ 1, 1\ 0\ 0\ 0, 1\ 1\ 1\ 1)$ , corresponding to  $\mathbf{d} = (1\ 0\ 1\ 1\ 0\ 0)$ , is transmitted and the received sequence is  $\mathbf{r} = (1\ 0\ 1\ 1, 1\ 1\ 1\ 1, 0\ 0\ 0\ 0, 0\ 0\ 0\ 0, 0\ 1\ 1\ 1, 1\ 0\ 0\ 0, 1\ 1\ 1\ 1)$ , where three single errors occur at positions 2, 5, and 13. Following the algorithm in order to deter-





**Figure 10.8** Viterbi decoding for a BSC.

mine the most likely transmitted path, the Hamming distance between  $\mathbf{r}$  and all distinct paths in the trellis are calculated and compared pairwise at each of the four states. At each state, the path having the smallest Hamming distance of the pair is retained. The numbers attached to each state represent the Hamming distance accumulated by the survivor at that state, and the paths discarded at each state are marked with an "x" on the trellis. The process is repeated for each succeeding level until level 5 is reached. The last two levels correspond to the tail of the information sequence. Since the tail consists of two 0s, only the branch corresponding to 0 will be extended by the decoder. At level 6 the number of survivors is reduced to two and to one at level 7. Thus this last survivor is the decoded sequence. The fact that the final survivor has a metric of 3 means that no other path through the trellis differs from  $\mathbf{r}$  in fewer than three positions. Tracing this survivor back to the origin node  $s_0 = (0\ 0)$  and deleting the tail, the decoded information is  $\mathbf{d} = (1\ 0\ 1\ 1\ 1)$ . Thus all three errors that occurred during the transmission have been corrected by the Viterbi algorithm. Note also that at some state [for example, the state  $s_2 = (0\ 1)$  at level 4] neither path is crossed out. This indicates a tie in the metric values of the two paths entering that state. Whenever a tie in metric value occurs, one path is arbitrarily selected as the survivor.

Soft-decision decoding uses the procedure discussed above except that the Hamming distance for a hard-decision metric is simply replaced by a soft-decision metric. All other decoding operations remain the same.



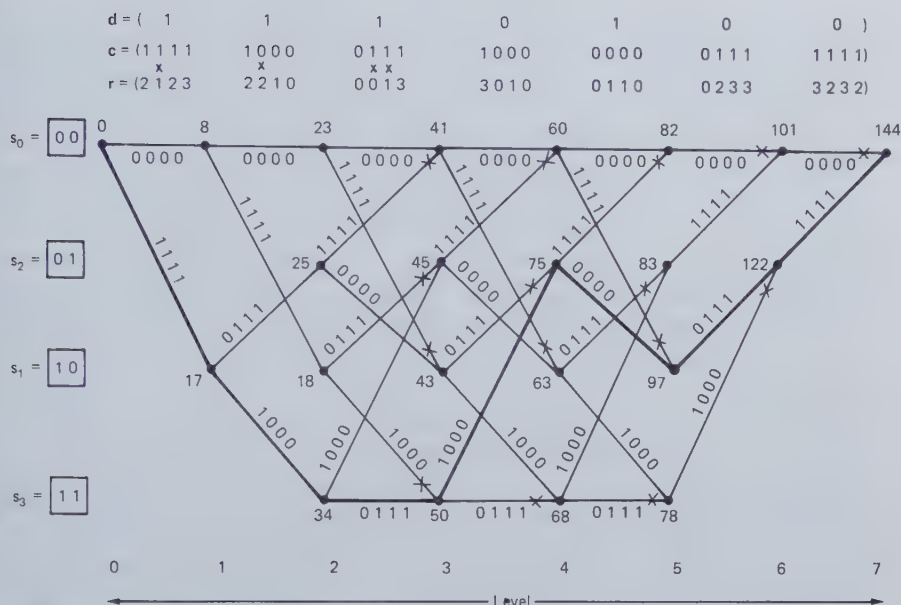


Figure 10.9 Viterbi decoding for a DMC.

**Example 10.6** Consider the binary-input quaternary-output DMC shown in Fig. 10.7. Assume that a code sequence  $\mathbf{c} = (1111, 1000, 0111, 1000, 0000, 0111, 1111)$  shown in the trellis of the  $R = 1/4$ ,  $m = 2$  convolutional code of Fig. 10.9 is transmitted over the DMC of Fig. 10.7, and that the  $q$ -nary received sequence is  $\mathbf{r} = (2123, 2210, 0013, 3010, 0110, 0233, 3232)$ . Using the metric values in Table 10.4b and applying the Viterbi algorithm to this received sequence  $\mathbf{r}$ , we have the final survivor, shown by the thick line,  $\mathbf{c} = (1111, 1000, 0111, 1000, 0000, 0111, 1111)$ , which corresponds to the decoded information sequence  $\mathbf{d} = (1110100)$ . The final  $m = 2$  branches of the tail part always correspond to 0 inputs that are not considered part of the information sequence.

Judging from the above examples, one might expect that the implementation complexity of a soft-decision decoder on a DMC is not significantly different from that of a hard-decision decoder on a BSC. This is a major advantage of the Viterbi decoding algorithm. We also know that the Viterbi algorithm is optimum in the sense that it minimizes the probability of error when decoding the entire sequence of information bits.

In summary, when a  $(n, k, m)$  binary convolutional code is decoded by means of the Viterbi algorithm, there are  $2^m$  states. Hence there

are  $2^m$  survivors at each level and  $2^m$  metrics, one for each survivor. Since each path that converges at a common state requires the computation of a metric, there are  $2^k$  metrics computed for each state. Of the  $2^k$  paths that merge at each state, only one survivor is the most probable path. In general, a convolutional encoder that consists of  $\gamma$  ( $k$ -bit) shift-register stages generates a trellis that has  $2^{k\gamma}$  states. Consequently convolutional decoding of such a code using the Viterbi algorithm requires keeping track of  $2^{k\gamma}$  survivors and  $2^{k\gamma}$  metrics. Thus the number of computations required to decode each level increases exponentially with  $k$  and  $\gamma$ . The exponential increase in computational burden limits the use of the Viterbi algorithm to relatively small values of  $\gamma$  and  $k$ .

Our study thus far has focused primarily on binary convolutional codes, which are particularly suitable for BSC or DMC over which binary or quaternary PSK modulation and coherent demodulation is possible. However, there are other modulation techniques in which  $M$ -ary FSK are employed in conjunction with noncoherent demodulation. The dual- $k$  codes, a class of nonbinary convolutional codes, are also decoded by means of the Viterbi algorithm using either hard-decision or soft-decision decoding. Any reader interested in this topic should consult the 1975 paper of Viterbi and Jacobs.

#### 10.4.2 Performance bounds for convolutional codes with Viterbi decoding

The most useful techniques for estimating the performance of convolutional coding are union bounds and computer simulation. The union-bound approach is often useful for predicting some specific performance in hardware implementation of the decoder. However, even though the usefulness of computer simulation is often limited by the amount of computer time required, computer simulation must sometimes be used for special coding applications where the union bound cannot provide a good performance estimate. The performance measures considered here are the first-event-error probability and the bit-error probability. We first obtain a bound on a first-event-error probability, and then the bit-error probability is derived using a union-bound argument. We begin with the performance analysis for a specific code on a BSC with the Viterbi algorithm. More general channel models will then be discussed briefly.

Consider the trellis of Fig. 9.11 and the augmented state diagram of Fig. 9.18 for the  $R = 1/2$ ,  $m = 2$  convolutional code whose generating function is

$$T(x, y, z) = x^5 y z^3 + x^6 y^2 z^4 (1 + z) + x^7 y^3 z^5 (1 + z)^2 \\ + \dots + x^{5+k} y^{1+k} z^{3+k} (1 + z)^k + \dots \quad (10.18)$$

For setting  $y = 1$  and  $z = 1$ , Eq. (10.18) becomes

$$T(x) = x^5 + 2x^6 + 4x^7 + \dots \\ = \sum_{\lambda=5}^{\infty} 2^{\lambda-5} x^{\lambda} \quad (10.19)$$

We see that one code word of weight 5, two code words of weight 6, four code words of weight 7, and so on result from this function. Thus we obtain the general form of the generating function

$$T(x) = \sum_{\lambda=0}^{\infty} c_{\lambda} x^{\lambda} \quad (10.20)$$

such that the coefficient  $c_{\lambda}$  of  $x^{\lambda}$  is equal to the number of code words of weight  $\lambda$ . This information can be used to estimate the event-error probability in a manner which is discussed later. Another interesting topic is the total information weight  $d_{\lambda}$  of all code words of weight  $\lambda$ , which is the coefficient of  $x^{\lambda}$  in the following function:

$$\frac{\partial T(x, y)}{\partial y} \Big|_{y=1} = \sum_{\lambda=0}^{\infty} d_{\lambda} x^{\lambda} \quad (10.21)$$

This information is useful for estimating the bit-error probability in a manner that is discussed later in this subsection.

**Example 10.7** Consider again the  $R = 1/2$ ,  $m = 2$  convolutional code. Substituting  $z = 1$  into Eq. (10.18) and then differentiating with respect to  $y$  yields

$$\frac{\partial T(x, y)}{\partial y} = x^5 + 4x^6 y + 12x^7 y^2 + \dots + (k+1)2^k x^{k+5} y^k + \dots$$

By applying Eq. (10.21) to this function, we obtain

$$\frac{\partial T(x, y)}{\partial y} \Big|_{y=1} = x^5 + 4x^6 + 12x^7 + \dots + (k+1)2^k x^{k+5} + \dots \\ = \sum_{\lambda=5}^{\infty} (\lambda - 4)2^{\lambda-5} x^{\lambda} \quad (10.22)$$

Thus the total information weight is expressed as  $d_{\lambda} = (\lambda - 4)2^{\lambda-5}$ . Hence  $d_{\lambda}$  for weight 5 code words is 1, for weight 6 code words is 4, for weight 7 code words is 12, and so on.

For convenience and without loss of generality, it can be assumed that an all-zero code word is transmitted because the codes are linear. A first-event error is made at an arbitrary trellis level  $j$  if the all-zero path (the correct path) is eliminated for the first time at level  $j$  by another path (the incorrect path) merging with it. That is, the correct path is not a survivor at  $s_0 = (0\ 0)$  at level  $j$ , and an incorrect path must be closer than the correct path to the received path. Now consider the case where the first-event-error probability is to be determined. Suppose that the path being compared with the all-zero path at some level  $j$  is at a distance  $d$  from the all-zero path. If  $d$  is odd, the all-zero path will be chosen correctly if the number of errors in the received sequence  $\mathbf{r}$  is less than  $(d + 1)/2$ ; otherwise the incorrect path will be selected. Thus the probability of selecting the incorrect path is

$$P_d = \sum_{e=(d+1)/2}^d \binom{d}{e} p^e (1-p)^{d-e} \quad d \text{ odd} \quad (10.23)$$

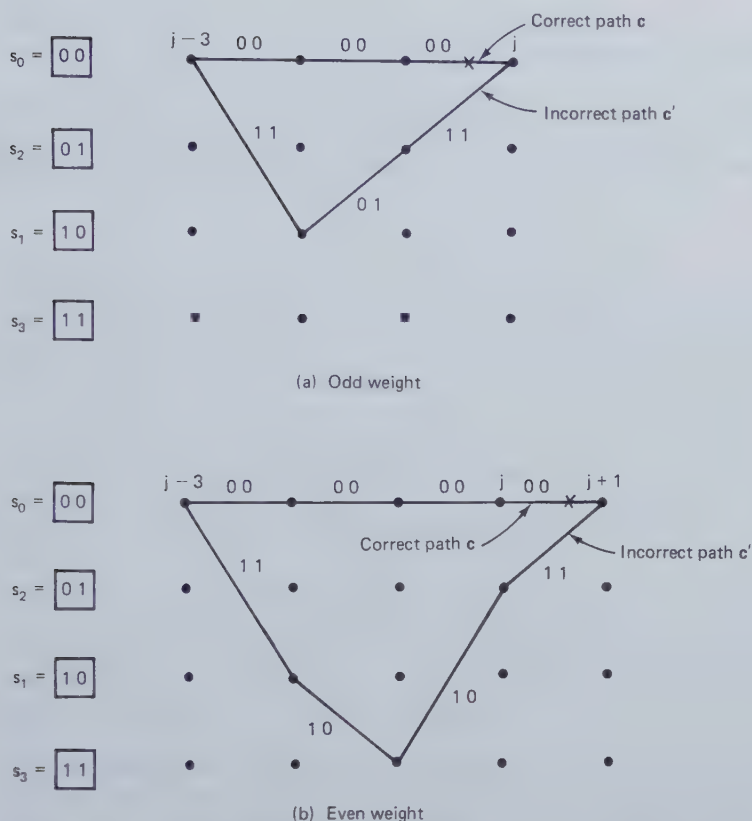
where  $p$  is the transition probability for the binary symmetric channel. This is the first-event-error probability when the distance  $d$  is odd. If  $d$  is even, the incorrect path is selected when the number of errors exceeds  $d/2$ . If the number of errors equals  $d/2$ , the metrics in the two paths are tied at the remerging node. Since the survivor is selected at random, an error is then made with probability  $1/2$ . Consequently, the first-event-error probability when  $d$  is even is

$$P_d = \frac{1}{2} \binom{d}{d/2} [p(1-p)]^{d/2} + \sum_{e=(d/2)+1}^d \binom{d}{e} p^e (1-p)^{d-e} \quad d \text{ even} \quad (10.24)$$

**Example 10.8** Consider the trellis diagram of Fig. 9.11 for the  $R = 1/2$ ,  $m = 2$  convolutional code. The incorrect path  $\mathbf{c}'$ , which is three branches long, is at distance 5 from the all-zero correct path  $\mathbf{c}$ , as illustrated in Fig. 10.10a. This incorrect path must be some path that had previously diverged from the all-zero state  $s_0 = (0\ 0)$  at level  $j - 3$  and is now remerging for the first time at level  $j$ . Since  $\mathbf{c}'$  is the distance 5 path, a first-event error will occur if in the five positions in which  $\mathbf{c}$  and  $\mathbf{c}'$  differ, the received sequence  $\mathbf{r}$  agrees with the incorrect path  $\mathbf{c}'$  in three or more positions. Thus the first-event-error probability for this distance 5 case is

$$P_5 = \sum_{e=3}^5 \binom{5}{e} p^e (1-p)^{5-e} \quad (10.25)$$

Next, consider the case where some incorrect paths may have an even weight, such as the distance 6 path, which is four branches long, shown in Fig. 10.10b. If the metrics of the correct and incorrect paths are tied at the remerging state,



**Figure 10.10** Typical first-error event for the  $R = 1/2$ ,  $m = 2$  convolutional code. (a) Odd weight in incorrect path. (b) Even weight in incorrect path.

the probability that an error will occur is  $1/2$ . For this incorrect path of distance 6, the first-event-error probability is given by

$$P_6 = \frac{1}{2} \binom{6}{3} p^3 (1-p)^3 + \sum_{e=4}^6 \binom{6}{e} p^e (1-p)^{6-e} \quad (10.26)$$

All incorrect paths of length  $j$  branches or less can cause a first-event error at level  $j$ . Hence, a union bound on the overall first-event-error probability  $P(E)$  can be obtained by summing the error probabilities for each of the possible incorrect paths which merge with the all-zero path at this level. If all incorrect paths of length greater than  $j$  branches are also included, the first-event-error probability at any level is overbounded by

$$P(E) < \sum_{\lambda=d_{\text{free}}}^{\infty} c_{\lambda} P_d \quad (10.27)$$



where the subscript  $\lambda$  denotes the distance quantity  $d$  such that  $c_\lambda$  is the number of code words of weight  $d$ .

**Example 10.9** Consider the  $R = 1/2$ ,  $m = 2$  convolutional code whose generating function is shown in Eq. (10.19). The coefficients  $c_\lambda = 2^{\lambda-5}$ ,  $\lambda = 5, 6, \dots, \infty$ , can be represented as the number of incorrect paths  $x^\lambda$ . Applying Eq. (10.27) to this problem, we see that the first-event-error probability is union-bounded by

$$P(E) < \sum_{\lambda=5}^{\infty} 2^{\lambda-5} P_d = P_5 + 2P_6 + 4P_7 + \dots \quad (10.28)$$

**Example 10.10** When the Hamming distance  $d$  between  $c$  and  $c'$  is odd, the first-event-error probability is given by Eq. (10.23). Since  $p \ll 1 - p$  for  $0 < p < 0.1$  and  $d - e < d/2$ , Eq. (10.23) can be bounded by the expression

$$P_d < \sum_{e=(d+1)/2}^d \binom{d}{e} [\sqrt{p(1-p)}]^d \quad (10.29)$$

where  $p$  is the transition probability of a BSC. If we know the inequality

$$\sum_{e=(d+1)/2}^d \binom{d}{e} = 2^{d-1}$$

Eq. (10.29) is further reduced to

$$P_d < [2\sqrt{p(1-p)}]^d \quad (10.30)$$

It can be shown that Eq. (10.30) is also an upper bound on  $P_d$  when  $d$  is even. Thus, substituting Eq. (10.30) into Eq. (10.27), the first-event-error probability at any arbitrary level  $j$  is bounded by

$$P(E) < \sum_{\lambda=d_{\text{free}}}^{\infty} c_\lambda [2\sqrt{p(1-p)}]^\lambda \quad (10.31)$$

On the other hand, referring to Eqs. (10.20), (10.27), and (10.30) for any convolutional code on a BSC with the generating function

$$T(x) = \sum_{\lambda=d_{\text{free}}}^{\infty} c_\lambda x^\lambda \quad \lambda = d \text{ (Hamming weight)}$$

the first-event-error probability is bounded by

$$P(E) < T(x)|_{x=2\sqrt{p(1-p)}} \quad (10.32)$$

where  $T(x)$  for  $x = 2\sqrt{p(1-p)}$  is

$$T(x)|_{x=2\sqrt{p(1-p)}} = \sum_{\lambda=d_{\text{free}}}^{\infty} c_\lambda [2\sqrt{p(1-p)}]^\lambda = d \quad (10.33)$$

which is equal to Eq. (10.31). For any ideal channel where  $p$  is small, i.e.,  $0 < p < 0.01$ , the bound of Eq. (10.32) is dominated by the first

term which corresponds to the path with the free distance  $d_{\text{free}}$  from the correct path, so it follows that

$$P(E) \simeq c_{d_{\text{free}}} [2\sqrt{p(1-p)}]^{d_{\text{free}}} \quad (10.34)$$

and since  $p \ll 1 - p$ , Eq. (10.34) is approximated by

$$P(E) \simeq c_{d_{\text{free}}} [2\sqrt{p}]^{d_{\text{free}}} \quad (10.35)$$

**Example 10.11** Consider the  $R = 1/2$ ,  $m = 2$  convolutional code whose generating function is given by Eq. (10.19). Thus we can easily derive the following expression:

$$T(x)|_x = 2\sqrt{p(1-p)} = \sum_{\lambda=5}^{\infty} 2^{2\lambda-5} [p(1-p)]^{\lambda/2} = \frac{2^5 [p(1-p)]^{5/2}}{1 - 4\sqrt{p(1-p)}}$$

Hence the first-event-error probability is found to be

$$P(E) < \frac{2^5 [p(1-p)]^{5/2}}{1 - 4\sqrt{p(1-p)}} \quad (10.36)$$

For a BSC with a transition probability of  $p = 0.002$ , and one code word of weight 5, i.e.,  $\lambda = d_{\text{free}} = 5$  and  $c_{d_{\text{free}}} = 1$ , the first-event-error probability resulting from Eq. (10.35) is  $P(E) \simeq 5.72 \times 10^{-6}$ , whereas using the bound of Eq. (10.36), we get  $P(E) < 6.94 \times 10^{-6}$ . Therefore we have the following measure of error performance for this particular code:  $5.72 \times 10^{-6} \leq P(E) < 6.94 \times 10^{-6}$ .

An important and practical performance measure is the number of bits decoded incorrectly. Turning now to the bit-error probability, we know that the expected number of bit errors, caused by any incorrect path which diverges from the correct path at level  $j$ , can be bounded by weighting each term of the union bound by the number of bit errors which occur on that incorrect path. The augmented generating function  $T(x, y)$  whose derivative with respect to  $y$  at  $y = 1$  is given by Eq. (10.21). Thus, for an arbitrary convolutional code with a generating function  $T(x, y)$ , the bit-error probability, defined as the expected number of bit errors per decoded bit, is bounded by

$$P_b(E) < \frac{1}{k} \sum_{\lambda=d_{\text{free}}}^{\infty} d_{\lambda} P_{\lambda} \quad (10.37)$$

When  $k > 1$ , Eq. (10.37) for  $P_b(E)$  should be divided by  $k$ . For small  $p$ , the bound is dominated by the first term, which is the path at free distance  $d_{\text{free}}$  from the correct path. We thus obtain

$$P_b(E) \simeq \frac{1}{k} d_{d_{\text{free}}} [2\sqrt{p(1-p)}]^{d_{\text{free}}} \simeq \frac{1}{k} d_{d_{\text{free}}} 2^{d_{\text{free}}} p^{d_{\text{free}}/2} \quad (10.38)$$

For the  $(2, 1, 2)$  convolutional code,  $k = 1$  and  $d_{\text{free}} = 5$ ; for  $p = 0.002$ , we obtain  $P_b(E) \simeq 2^5 p^{5/2} = 5.72 \times 10^{-6}$ , which is the same as the first-event-error probability  $P(E)$ .

**Example 10.12** The augmented generating function  $T(x, y)$  for the  $R = 1/2$ ,  $m = 2$  convolutional code is

$$T(x, y) = x^5y + 2x^6y^2 + 4x^7y^3 + \dots + 2^\lambda x^{\lambda+5}y^{\lambda+1} + \dots$$

Since  $k = 1$ , by using the union bound, we find that the bit-error probability is bounded by

$$P_b(E) < P_5 + 4P_6 + 12P_7 + \dots + (\lambda + 1)2^\lambda P_{\lambda+5} + \dots \quad (10.39)$$

where  $P_\lambda$  is given either by Eq. (10.23) or by the bound of Eq. (10.30). If we use the bound of Eq. (10.30), the bit-error probability becomes

$$\begin{aligned} P_b(E) &< \sum_{\lambda=5}^{\infty} (\lambda - 4)2^{\lambda-5} \cdot 2^\lambda [p(1-p)]^{1/2} \\ &= \frac{\partial T(x, y)}{\partial y} \Big|_{y=1, x=2\sqrt{p(1-p)}} \\ &= \frac{[2\sqrt{p(1-p)}]^5}{[1 - 4\sqrt{p(1-p)}]^2} \end{aligned} \quad (10.40)$$

It is sometimes instructive to consider the coherent PSK modulation over an additive white gaussian noise (AWGN) channel with hard quantization. For a binary case, the modulator must select one of two pulse signals, i.e.,  $s_0(t)$  for an encoded bit 0 and  $s_1(t)$  for an encoded bit 1. The waveform corresponding to each bit is arbitrary, but one typical form over each bit interval is

$$\begin{aligned} s_i(t) &= \sqrt{2P} \cos(\omega_c t + \phi_i) & \phi_i &= 0, \pi \\ &= \sqrt{2P} a_i \cos \omega_c t & i &= 0, 1 \end{aligned} \quad (10.41)$$

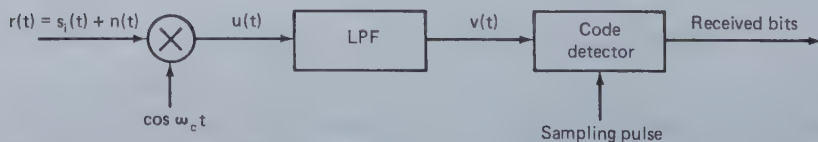
$E_b$  where  $a_i = \pm 1$  (antipodal)

$P = E_b/T$  is the modulated-signal power

$E_b$  = the energy per bit during  $T$  seconds

This antipodal signaling is called binary-phase-shift-keyed (BPSK) modulation.

Figure 10.11 depicts a coherent PSK detection scheme at the demodulator (receiver). The channel adds white gaussian noise  $n(t)$ , which is



**Figure 10.11** Coherent PSK detector.

flat over the signal spectrum with a one-sided noise power density spectrum  $N_0$  (W/Hz). The demodulator uses an established decision rule to estimate which transmitted bit was actually sent. Thus, the signal at the demodulator input is given by

$$r(t) = s_i(t) + n(t) \quad i = 0, 1 \quad (10.42)$$

where  $s_i(t)$ ,  $i = 0$  or  $1$ , is the transmitted waveform and  $n(t)$  is the noise added.

The autocorrelation of the noise is given by

$$\varphi(\tau) = \frac{N_0}{2} \delta(\tau) \quad (10.43)$$

When  $\tau = 0$ , the noise power is simply  $\varphi(0) = N_0/2$  (W/Hz). Gaussian noise  $n(t)$  with zero mean is generally expressed by

$$n(t) = x \cos w_c t - y \sin w_c t \quad (10.44)$$

The expected values of the inphase component  $x$  and the quadrature component  $y$  are  $E\{x\} = E\{y\} = 0$ , respectively. From Fig. 10.11,  $u(t)$  becomes

$$\begin{aligned} u(t) &= [s_i(t) + n(t)] \cos w_c t \\ &= [\sqrt{2P} \cos(w_c t + \phi_i) + x \cos w_c t - y \sin w_c t] \cos w_c t \\ &= \frac{1}{2}(\sqrt{2P} \cos \phi_i + x)(1 + \cos 2w_c t) - \frac{1}{2}(\sqrt{2P} \sin \phi_i + y) \sin 2w_c t \end{aligned} \quad (10.45)$$

The low-pass filter drops the double-frequency components but passes the zero-frequency terms without distortion. Therefore the detector output, ignoring the factor of  $\frac{1}{2}$ , becomes

$$v(t) = \sqrt{2P} \cos \phi_i + x(t) \quad (10.46)$$

When  $\phi_i = 0^\circ$ , the expected value of  $v(t)$  is

$$E\{v(t)\} = \sqrt{2P} + E\{x(t)\} = \sqrt{2P}$$

because  $x(t)$  is a zero-mean gaussian process. The variance of  $v(t)$  is

$$\sigma^2 = E\{[v - E\{v\}]^2\} = E\{x^2\} = N_0/2 \quad \text{W/Hz}$$

Thus, its probability density distribution is then

$$p(v) = \frac{1}{\sqrt{\pi N_0}} \exp \left[ -\frac{(v - \sqrt{2P})^2}{N_0} \right] \quad (10.47)$$

When  $\phi_i = 0^\circ$ , the waveform  $s_0(t)$ , which corresponds to the encoded bit 1, is transmitted. Then Eq. (10.46) yields

$$v(t) = \sqrt{2P} + x(t) > 0$$

When  $\phi_i = 180^\circ$ , the waveform  $s_1(t)$ , which corresponds to the encoded bit 0, is transmitted. Therefore

$$v(t) = -\sqrt{2P} + x(t) < 0$$

When  $v(t) < 0$  at a sampling instant, the decision is 0. But if  $v(t) < 0$  when 1 is transmitted, the probability of an error is then given by

$$P_e = P_r(v < 0) = \int_{-\infty}^0 p(v) dv \quad (10.48)$$

Substituting Eq. (10.47) into Eq. (10.48) and putting  $t = -(v - \sqrt{2P})/\sqrt{N_0}$ , we have

$$\begin{aligned} P_e &= \frac{1}{\sqrt{\pi}} \int_{\sqrt{2P}/N_0}^{\infty} e^{-t^2} dt \\ &= \frac{1}{2} \operatorname{erfc} \left( \sqrt{\frac{2P}{N_0}} \right) \end{aligned} \quad (10.49)$$

for an uncoded modulation scheme.

This is just equivalent to the BSC transition probability  $p$ . At high SNR, the error rate of Eq. (10.49) is well approximated by

$$P_e \approx \frac{1}{2} e^{-P/N_0} \quad (10.50)$$

The received-energy-per-bit-to-noise ratio  $E_b/N_0$  required to achieve a given bit-error probability serves as a figure of merit for different combinations of coding and modulation schemes. If we write  $E_b/N_0$  in terms of the received modulated-signal power  $P$  and information data rate  $R$ , we have

$$\frac{E_b}{N_0} = \frac{P}{N_0} \left( \frac{1}{R} \right)$$

Thus, a coding or modulation scheme that reduces the  $E_b/N_0$  required for a given bit-error probability leads to either an increase in the allowable information rate  $R$  in bits per second or a decrease in the necessary  $P/N_0$ , or a combination of both. When the BPSK modulation is used on an AWGN channel with coherent detection and hard quantization, the BSC crossover probability from Eq. (10.49) is

$$P_e = \frac{1}{2} \operatorname{erfc} \sqrt{\frac{2RE_b}{N_0}} \quad (10.51)$$



Next consider the first-event-error probability  $P(E)$  and the bit-error probability  $P_b(E)$  over an AWGN channel in terms of the free distance. Using the approximation for large values of  $E_b/N_0$ , the complementary error function of gaussian statistics is approximated by

$$\operatorname{erfc} \left( \sqrt{\frac{2RE_b}{N_0}} \right) \approx \exp \left[ -\frac{RE_b}{N_0} \right] \quad (10.52)$$

Therefore, using Eqs. (10.35) and (10.52), we see that the probability of a first-event error is approximated by

$$P(E) \approx C_{d_{\text{free}}} 2^{d_{\text{free}}} \exp \left[ -\left( \frac{RE_b}{N_0} \right) d_{\text{free}} \right] \quad (10.53)$$

By substituting  $p \approx \exp [-RE_b/N_0]$  into Eq. (10.38) for large values of  $E_b/N_0$ , we can simplify the bit-error probability for this case to

$$P_b(E) \approx \frac{1}{k} d_{d_{\text{free}}} 2^{d_{\text{free}}} \exp \left[ -\left( \frac{RE_b}{2N_0} \right) d_{\text{free}} \right] \quad (10.54)$$

Consider next the pairwise-error probability for two code words which differ in  $\lambda$  positions. That is,

$$\begin{aligned} P_\lambda &= \frac{1}{2} \operatorname{erfc} \left[ \sqrt{\frac{\lambda RE_b}{N_0}} \right] \\ &= \frac{1}{2} \operatorname{erfc} \left[ \sqrt{\frac{(d_{\text{free}} + l) RE_b}{N_0}} \right] \end{aligned} \quad (10.55)$$

where  $\lambda = (d_{\text{free}} + l)$ ,  $l = 0, 1, 2, \dots$ . For  $x \geq 0$  and  $y \geq 0$ , the following inequality holds:

$$\operatorname{erfc} (\sqrt{x+y}) \leq e^{-y} \operatorname{erfc} (\sqrt{x})$$

Therefore the bound for the pairwise-error probability becomes

$$P_\lambda \leq \exp \left[ -\left( \frac{\lambda - d_{\text{free}} RE_b}{N_0} \right) \right] \operatorname{erfc} \left[ \sqrt{\frac{RE_b d_{\text{free}}}{N_0}} \right] \quad (10.56)$$

Thus, substituting Eq. (10.56) into Eq. (10.27) yields the bound on the first-event-error probability:

$$\begin{aligned} P(E) &< \sum_{\lambda = d_{\text{free}}}^{\infty} c_\lambda P_\lambda \operatorname{erfc} \left[ \sqrt{\frac{RE_b d_{\text{free}}}{N_0}} \right] \\ &\quad \cdot \sum_{\lambda = d_{\text{free}}}^{\infty} c_\lambda \exp \left\{ -\left[ \frac{(\lambda - d_{\text{free}} R) E_b}{N_0} \right] \right\} \end{aligned} \quad (10.57)$$

or, it can also be obtained by using Eq. (10.32) as follows:

$$P(E) < \operatorname{erfc} \left[ \sqrt{\frac{RE_b d_{\text{free}}}{N_0}} \right] \exp \left( \frac{RE_b d_{\text{free}}}{N_0} \right) T(x)|_{x = \exp(-RE_b/N_0)} \quad (10.58)$$

Using Eqs. (10.21), (10.37), and (10.56), the bit-error probability for Viterbi decoding of binary convolutional codes over AWGN channels can easily be obtained as

$$P_b(E) < \operatorname{erfc} \left[ \sqrt{\frac{RE_b d_{\text{free}}}{N_0}} \right] \exp \left( \frac{RE_b d_{\text{free}}}{N_0} \right) \cdot \frac{\partial T(x, y)}{\partial y} \Big|_{y=1, x = \exp(-RE_b/N_0)} \quad (10.59)$$

Equation (10.59) has been used very effectively to obtain tight upper bounds for the bit-error probability on the binary-input AWGN channel for a variety of binary convolutional codes.

**Example 10.13** Consider the  $R = 1/2$ ,  $m = 2$  convolutional code with  $d_{\text{free}} = 5$ . The bound on the bit-error probability  $P_b(E)$  over a AWGN channel is

$$\begin{aligned} P_b(E) &< \operatorname{erfc} \left( \sqrt{\frac{5E_b}{2N_0}} \right) \exp \left( \frac{5E_b}{2N_0} \right) \cdot \frac{\exp(-5E_b/2N_0)}{[1 - 2 \exp(-E_b/2N_0)]^2} \\ &= \frac{\operatorname{erfc}(\sqrt{5E_b/2N_0})}{[1 - 2 \exp(-E_b/2N_0)]^2} \end{aligned} \quad (10.60)$$

The foregoing analysis is based on performance bounds for specific codes and is valid for large values of  $E_b/N_0$ . It is instructive for the reader to compare the approximate expression for  $P_b(E)$  given in Eq. (10.54) for a BSC with a similar expression obtained for a binary-input AWGN channel from Eq. (10.59). The analysis shows that there is a 3-dB power advantage for the AWGN channel over the BSC. Hence the transmitter needs an additional 3 dB of signal power on the BSC to achieve the same error probability. The actual coding gain experienced with a Viterbi decoder varies with the required bit-error probability  $P_b(E)$ , and it has a tendency to increase as  $P_b(E)$  decreases.

## PROBLEMS

**10.1** For a binary-input quaternary-output DMC with crossover probabilities  $P(r_i|c_i)$  given by

$$\begin{array}{cccc} P(0|0) = 0.55 & P(1|0) = 0.25 & P(2|0) = 0.15 & P(3|0) = 0.05 \\ P(0|1) = 0.05 & P(1|1) = 0.15 & P(2|1) = 0.25 & P(3|1) = 0.55 \end{array}$$

- a Find the metric table for this channel.
- b Find the integer metric table by choosing  $\alpha = 10.45$  and  $\beta = 1.30$ .

**10.2** For the (3, 1) convolutional code with  $m = 2$ , suppose that the generator sequences for this encoder are given by

$$\mathbf{g}^{(1)} = (1 \ 1 \ 0) \quad \mathbf{g}^{(2)} = (1 \ 0 \ 1) \quad \mathbf{g}^{(3)} = (1 \ 1 \ 1)$$

- a Draw the encoder diagram for this code.
- b Draw the trellis diagram for the information sequence of length  $L = 5$ .
- c Suppose that the code word  $\mathbf{c} = (1 \ 1 \ 1, 0 \ 1 \ 0, 1 \ 1 \ 0, 1 \ 0 \ 0, 0 \ 1 \ 0, 1 \ 1 \ 0, 0 \ 1 \ 1)$  is transmitted and  $\mathbf{r} = (1 \ 1 \ 1, 1 \ 1 \ 0, 0 \ 1 \ 0, 1 \ 1 \ 0, 0 \ 1 \ 0, 1 \ 1 \ 0, 0 \ 1 \ 1)$  is received. Decode  $\mathbf{r}$  using the Viterbi algorithm, and find the final survivor on the trellis.
- d What is the decoded information sequence?

**10.3** For the (3, 1) convolutional code with  $m = 2$  given in Prob. 10.2,

- a Construct the state table for this code.
- b Draw the state diagram for this encoder.
- c For a BSC with  $p = .05$ , find an integer metric table for the Fano metric.
- d Draw the tree diagram for an information sequence of length  $L = 5$ .
- e Decode the received sequence  $\mathbf{r} = (1 \ 1 \ 1, 1 \ 1 \ 0, 0 \ 1 \ 0, 1 \ 1 \ 0, 0 \ 1 \ 0, 1 \ 1 \ 0, 0 \ 1 \ 1)$  using the stack algorithm.

**10.4** For the (3, 1) convolutional code with  $m = 2$  of Prob. 10.2,

- a Decode the received sequence  $\mathbf{r} = (1 \ 1 \ 1, 1 \ 1 \ 0, 0 \ 1 \ 0, 1 \ 1 \ 0, 0 \ 1 \ 0, 1 \ 1 \ 0, 0 \ 1 \ 1)$  using the tree diagram drawn in Prob. 10.3d and the Fano algorithm of Fig. 10.1 with the threshold increment of  $\Delta = 1$ .
- b Repeat part a for  $\Delta = 2$  and  $\Delta = 3$ . The number of computations by the Fano algorithm depends on how the threshold increment  $\Delta$  is chosen. Does increasing  $\Delta$  bring about a reduction in the number of computations?
- c Compare the final decoded path with the results of Probs. 10.2 and 10.3, where the same received sequence  $\mathbf{r}$  is decoded.

**10.5** For the  $R = 1/3$ ,  $m = 2$  convolutional encoder of Example 9.16,

- a Find the minimum free distance  $d_{\text{free}}$  from the generating function.
- b Assume that  $c(x) = 0$  is transmitted. For a BSC with the transition probability  $p = .05$ , find the first-event-error probabilities  $P_d$  for the incorrect path at  $d = 7$  and  $d = 8$ , respectively.
- c Find the union bound for the first-event-error probability.
- d Find the upper bound on the bit-error probability  $P_b(E)$ .

**10.6** Prove that  $Q(\sqrt{2E_b/N_0}) = \frac{1}{2} \operatorname{erfc}(\sqrt{E_b/N_0})$ , where  $Q(x)$  and  $\operatorname{erfc}(x)$  are expressed by

$$Q(x) = \frac{1}{\sqrt{2\pi}} \int_x^{\infty} e^{-y^2/2} dy$$

and

$$\operatorname{erfc}(x) = \frac{2}{\sqrt{\pi}} \int_x^{\infty} e^{-y^2} dy$$

respectively.

## Threshold Decoding for Convolutional Codes

We considered both maximum-likelihood decoding and sequential decoding of convolutional codes in the previous chapter. The performance of Viterbi decoding is optimum despite the complexity of the decoder, which grows exponentially with code constraint length and number of encoder states. As a partial remedy for this difficulty, sequential decoding, which is essentially independent of constraint length, makes possible the use of very large encoder memory orders and has very low error probabilities associated with it. Notwithstanding this feature, the performance of sequential decoding is slightly suboptimum as a result of its long random searches.

Another class of decoding algorithms, which is known as threshold or majority-logic decoding, has received much attention for its simplicity and application to internal interleaving, which is easily accommodated in a number of communication systems. Threshold decoding is a practical technique, originally proposed by Massey, which can achieve coding gains of 1 to 3 dB with relatively simple implementations using simple threshold gates. The performance of threshold decoding is suboptimum compared to Viterbi decoding, but its algebraic approach is applicable to the decoding of convolutional codes in roughly the same manner as majority-logic decoding of block codes. The complexity of the feedback decoder for nonsystematic convolutional codes depends almost exclusively on the complexity of the syndrome logic and hardly at all on constraint length. Nonsystematic convolutional codes are superior because they achieve higher free distances for a given encoder memory. However, for correcting information errors, it appears that systematic convolutional codes are more advantageous than nonsystematic codes. It will become obvious later on that nonsystematic



codes offer few advantages simply because they are rarely used in syndrome decoders.

### 11.1 Basic Concepts for Syndrome Decoding

Let us consider an  $(n, k)$  systematic convolutional code of memory order  $m$ . The generator matrix for this systematic code is

$$\mathbf{G} = \begin{bmatrix} \mathbf{I}\mathbf{P}_0 & \mathbf{0}\mathbf{P}_1 & \mathbf{0}\mathbf{P}_2 & \cdots & \mathbf{0}\mathbf{P}_m \\ & \mathbf{I}\mathbf{P}_0 & \mathbf{0}\mathbf{P}_1 & \cdots & \mathbf{0}\mathbf{P}_{m-1} & \mathbf{0}\mathbf{P}_m \\ & & \mathbf{I}\mathbf{P}_0 & \cdots & \mathbf{0}\mathbf{P}_{m-2} & \mathbf{0}\mathbf{P}_{m-1} & \mathbf{0}\mathbf{P}_m \\ & & & \ddots & & & \ddots \end{bmatrix}$$

where the blank areas are all 0s.  $\mathbf{I}$  is a  $k \times k$  identity matrix,  $\mathbf{0}$  is a  $k \times k$  matrix of 0s, and  $\mathbf{P}_\lambda$ ,  $\lambda = 0, 1, \dots, m$ , are  $k \times (n - k)$  matrices and are given by

$$\mathbf{P}_\lambda = \begin{bmatrix} g_{1,\lambda}^{(k+1)} & g_{1,\lambda}^{(k+2)} & \cdots & g_{1,\lambda}^{(n)} \\ g_{2,\lambda}^{(k+1)} & g_{2,\lambda}^{(k+2)} & \cdots & g_{2,\lambda}^{(n)} \\ \vdots & \vdots & & \vdots \\ g_{k,\lambda}^{(k+1)} & g_{k,\lambda}^{(k+2)} & \cdots & g_{k,\lambda}^{(n)} \end{bmatrix} \quad 0 \leq \lambda \leq m \quad (11.2)$$

Notice that here the pattern repeats and is right-shifted in every row. We can interpret this matrix expression in terms of the shift-register description of the encoder. The parity-check matrix  $\mathbf{H}$ , which is a semi-infinite matrix, can be easily constructed by inspection of  $\mathbf{G}$ .

$$\mathbf{H} = \begin{bmatrix} \mathbf{P}_0^T \mathbf{I} \\ \mathbf{P}_1^T \mathbf{0} & \mathbf{P}_0^T \mathbf{I} \\ \mathbf{P}_2^T \mathbf{0} & \mathbf{P}_1^T \mathbf{0} & \mathbf{P}_0^T \mathbf{I} \\ \vdots & & \\ \mathbf{P}_m^T \mathbf{0} & \mathbf{P}_{m-1}^T \mathbf{0} & \mathbf{P}_{m-2}^T \mathbf{0} & \cdots & \mathbf{P}_0^T \mathbf{I} \\ & \mathbf{P}_m^T \mathbf{0} & \mathbf{P}_{m-1}^T \mathbf{0} & \cdots & \mathbf{P}_1^T \mathbf{0} & \mathbf{P}_0^T \mathbf{I} \\ & & \mathbf{P}_m^T \mathbf{0} & \cdots & \mathbf{P}_2^T \mathbf{0} & \mathbf{P}_1^T \mathbf{0} & \mathbf{P}_0^T \mathbf{I} \\ & & & \ddots & & & \ddots \end{bmatrix} \quad (11.3)$$

where the blank areas are all 0s.  $\mathbf{I}$  is an  $(n - k) \times (n - k)$  identity matrix,  $\mathbf{0}$  is an  $(n - k) \times (n - k)$  matrix of 0s, and  $\mathbf{P}_\lambda^T$ ,  $\lambda = 0, 1, \dots, m$ ,

$$\mathbf{P}_\lambda^T = \begin{bmatrix} g_{1,\lambda}^{(k+1)} & g_{2,\lambda}^{(k+1)} & \cdots & g_{k,\lambda}^{(k+1)} \\ g_{1,\lambda}^{(k+2)} & g_{2,\lambda}^{(k+2)} & \cdots & g_{k,\lambda}^{(k+2)} \\ \vdots & \vdots & \ddots & \vdots \\ g_{1,\lambda}^{(n)} & g_{2,\lambda}^{(n)} & \cdots & g_{k,\lambda}^{(n)} \end{bmatrix} \quad 0 \leq \lambda \leq m \quad (11.4)$$

are  $(n - k) \times k$  transpose matrices of  $\mathbf{P}_\lambda$ . As is the case with block codes,  $\mathbf{G} \cdot \mathbf{H}^T = \mathbf{0}$  and  $\mathbf{c}$  is a code sequence if and only if  $\mathbf{c} \cdot \mathbf{H}^T = \mathbf{0}$ , but  $\mathbf{d}$  and  $\mathbf{c}$  are of arbitrary length.

**Example 11.1** Consider the  $(2, 1, m)$  systematic convolutional code with generator sequences

$$\mathbf{g}^{(1)} = (g_0^{(1)} g_1^{(1)} g_2^{(1)} \cdots g_m^{(1)}) = (1 \ 0 \ 0 \cdots 0)$$

and

$$\mathbf{g}^{(2)} = (g_0^{(2)} g_1^{(2)} g_2^{(2)} \cdots g_m^{(2)})$$

Using Eq. (11.2), we see that  $\mathbf{P}_0 = g_{1,0}^{(2)}$ ,  $\mathbf{P}_1 = g_{1,1}^{(2)}$ ,  $\mathbf{P}_2 = g_{1,2}^{(2)}$ ,  $\dots$ ,  $\mathbf{P}_m = g_{1,m}^{(2)}$ . Therefore the generator matrix of Eq. (11.1) yields

$$\mathbf{G} = \begin{bmatrix} 1 & g_{1,0}^{(2)} & 0 & g_{1,1}^{(2)} & 0 & g_{1,2}^{(2)} & \cdots & 0 & g_{1,m}^{(2)} \\ & 1 & g_{1,0}^{(2)} & 0 & g_{1,1}^{(2)} & \cdots & 0 & g_{1,m-1}^{(2)} & 0 & g_{1,m}^{(2)} \\ & & 1 & g_{1,0}^{(2)} & \cdots & 0 & g_{1,m-2}^{(2)} & 0 & g_{1,m-1}^{(2)} & 0 & g_{1,m}^{(2)} \\ & & & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots \end{bmatrix} \quad (11.5)$$

On the other hand, since  $\mathbf{P}_0^T = g_{1,0}^{(2)}$ ,  $\mathbf{P}_1^T = g_{1,1}^{(2)}$ ,  $\mathbf{P}_2^T = g_{1,2}^{(2)}$ ,  $\dots$ ,  $\mathbf{P}_m^T = g_{1,m}^{(2)}$  from Eq. (11.4), the parity-check matrix of this code is

$$\mathbf{H} = \begin{bmatrix} g_{1,0}^{(2)} & 1 & & & & & & & & & \\ g_{1,1}^{(2)} & 0 & g_{1,0}^{(2)} & 1 & & & & & & & \\ g_{1,2}^{(2)} & 0 & g_{1,1}^{(2)} & 0 & g_{1,0}^{(2)} & 1 & & & & & \\ \vdots & & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \\ g_{1,m}^{(2)} & 0 & g_{1,m-1}^{(2)} & 0 & g_{1,m-2}^{(2)} & 0 & \cdots & g_{1,0}^{(2)} & 1 & & \\ & g_{1,m}^{(2)} & 0 & g_{1,m-1}^{(2)} & 0 & \cdots & g_{1,1}^{(2)} & 0 & g_{1,0}^{(2)} & 1 & \\ & & g_{1,m}^{(2)} & 0 & \cdots & g_{1,2}^{(2)} & 0 & g_{1,1}^{(2)} & 0 & g_{1,0}^{(2)} & 1 \\ & & & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots \end{bmatrix} \quad (11.6)$$

For an information sequence  $\mathbf{d}$ , the output sequences of an  $R = 1/2$  systematic encoder are

$$\mathbf{c}^{(1)} = \mathbf{d} \quad \mathbf{c}^{(2)} = \mathbf{d} * \mathbf{g}^{(2)} \quad (11.7)$$

where the information sequence is  $\mathbf{c}^{(1)} = \mathbf{d} = (d_0^{(1)}, d_1^{(1)}, d_2^{(1)}, \dots)$  and the parity sequence is  $\mathbf{c}^{(2)} = \mathbf{p} = (d_0^{(2)}, d_1^{(2)}, d_2^{(2)}, \dots)$ . Thus the transmitted code sequence through the P/S multiplexer becomes  $\mathbf{c} = (d_0^{(1)}d_0^{(2)}, d_1^{(1)}d_1^{(2)}, d_2^{(1)}d_2^{(2)}, \dots)$ . When  $\mathbf{c}$  is transmitted over a BSC, the received sequence  $\mathbf{r}$  becomes

$$\mathbf{r} = \mathbf{c} + \mathbf{e} = (r_0^{(1)}r_0^{(2)}, r_1^{(1)}r_1^{(2)}, r_2^{(1)}r_2^{(2)}, \dots) \quad (11.8)$$

as a result of the channel error sequence  $\mathbf{e} = (e_0^{(1)}e_0^{(2)}, e_1^{(1)}e_1^{(2)}, e_2^{(1)}e_2^{(2)}, \dots)$  if  $\mathbf{r} \neq \mathbf{c}$ .

A P/S multiplexer at the encoder output provides for consecutive interleaved transmission of information bit and parity bit. A demultiplexer inserted before the syndrome generator separates the error-corrupted information and the parity subsequences. However, since the channel is memoryless, we may treat the interleaved information and parity subsequences as if they were transmitted on separate BSCs with the same error statistics.

Let  $\mathbf{r}^{(1)}$  be the information subsequence and  $\mathbf{r}^{(2)}$  be the parity subsequence of the received sequence  $\mathbf{r}$ . Then it follows that

$$\begin{aligned} \mathbf{r} &= \mathbf{r}^{(1)} + \mathbf{r}^{(2)} = (\mathbf{d} + \mathbf{e}^{(1)}) + (\mathbf{p} + \mathbf{e}^{(2)}) \\ &= (r_0^{(1)}, r_1^{(1)}, r_2^{(1)}, \dots) + (r_0^{(2)}, r_1^{(2)}, r_2^{(2)}, \dots) \end{aligned} \quad (11.9)$$

where the error sequence  $\mathbf{e}$  is also divided into the information-error subsequence  $\mathbf{e}^{(1)} = (e_0^{(1)}, e_1^{(1)}, e_2^{(1)}, \dots)$  and the parity-error subsequence  $\mathbf{e}^{(2)} = (e_0^{(2)}, e_1^{(2)}, e_2^{(2)}, \dots)$ , i.e.,  $\mathbf{e} = \mathbf{e}^{(1)} + \mathbf{e}^{(2)}$ . Of course, each of these error subsequences equals 0 if the corresponding digits are received correctly and equals 1 if they are in error. Thus the syndrome digits are all 0 when no errors are made.

The syndrome can be generated by passing the received noise-corrupted information digits to a convolutional encoder and then adding the encoder's output to the received noise-corrupted parity digits, as shown in Fig. 11.1. The syndrome generator in the decoder operates in almost the same way as the encoder. Since the syndrome sequence  $\mathbf{s} = \mathbf{r} \cdot \mathbf{H}^T = \mathbf{e} \cdot \mathbf{H}^T$ , as it is in the case of block codes, we see that  $\mathbf{s}$  depends only on the channel error sequence. But the decoder can be designed to operate on  $\mathbf{s}$  rather than on  $\mathbf{r}$  because knowing  $\mathbf{s}$  is equivalent to knowing  $\mathbf{r}$ . Such a decoding process is called syndrome decoding.

Using the polynomial form introduced in Chap. 9, we see that the output coded sequence of an  $R = 1/2$  symmetric encoder is represented as

$$\mathbf{c}^{(1)}(D) = d(D) \quad \mathbf{c}^{(2)}(D) = d(D)g^{(2)}(D) \quad (11.10)$$

where  $g^{(2)}(D)$  is the code generator polynomial. If the channel error subsequences for the information and parity portions are denoted as  $e^{(1)}(D)$  and  $e^{(2)}(D)$ , respectively, the received sequences are expressed as

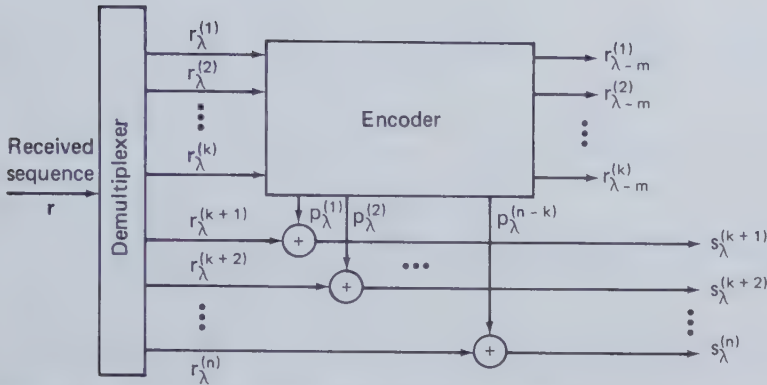


Figure 11.1 Syndrome generator for an  $(n, k, m)$  systematic convolutional code.

$$\begin{aligned} r^{(1)}(D) &= d(D) + e^{(1)}(D) \\ r^{(2)}(D) &= d(D)g^{(2)}(D) + e^{(2)}(D) \end{aligned} \quad (11.11)$$

The syndrome sequence is then formed by encoding  $r^{(1)}(D)$  and then adding it to  $r^{(2)}(D)$ . This gives the following syndrome polynomial:

$$\begin{aligned} s(D) &= r^{(1)}(D)g^{(2)}(D) + r^{(2)}(D) \\ &= e^{(1)}(D)g^{(2)}(D) + e^{(2)}(D) \end{aligned} \quad (11.12)$$

The information sequence estimated by the decoder is then

$$\begin{aligned} \hat{d}(D) &= r^{(1)}(D) + \hat{e}^{(1)}(D) \\ &= c^{(1)}(D) + [e^{(1)}(D) + \hat{e}^{(1)}(D)] \end{aligned} \quad (11.13)$$

Thus, correct decoding is accomplished as long as the estimated error sequence equals the actual error sequence. Extending Eq. (11.12) to an  $(n, k)$  systematic code of memory order  $m$ , we get the following syndrome in polynomial form:

$$s^{(j)}(D) = \sum_{i=1}^k e^{(i)}(D)g^{(j)}(D) + e^{(j)}(D) \quad j = k+1, k+2, \dots, n \quad (11.14)$$

## 11.2 Syndrome and Feedback Decoding

The received sequence of an  $R = k/n$ ,  $m$  systematic convolutional code is

$$\mathbf{r} = (r_0^{(1)} r_0^{(2)} \dots r_0^{(n)}, r_1^{(1)} r_1^{(2)} \dots r_1^{(n)}, \dots, r_\lambda^{(1)} r_\lambda^{(2)} \dots r_\lambda^{(n)}, \dots) \quad (11.15)$$

$$= \mathbf{c} + \mathbf{e}$$

where  $r_\lambda^{(i)} = c_\lambda^{(i)} + e_\lambda^{(i)}$ ,  $i = 1, 2, \dots, n$ . The syndrome sequence of this received word is given by

$$\mathbf{s} = \mathbf{r} \cdot \mathbf{H}^T \quad (11.16)$$

where  $\mathbf{H}^T$  is the transpose matrix of Eq. (11.3).

Applying Eqs. (11.2) and (11.15) to Eq. (11.16), we get

$$\begin{aligned} \mathbf{s} &= (s_0^{(k+1)}, s_0^{(k+2)}, \dots, s_0^{(n)}, s_1^{(k+1)}, s_1^{(k+2)}, \dots, \\ &\quad s_1^{(n)}, \dots, s_\lambda^{(k+1)}, s_\lambda^{(k+2)}, \dots, s_\lambda^{(n)}, \dots) \\ &= (r_0^{(1)} r_0^{(2)} \dots r_0^{(n)}, r_1^{(1)} r_1^{(2)} \dots r_1^{(n)}, \dots, r_\lambda^{(1)} r_\lambda^{(2)} \dots r_\lambda^{(n)}, \dots) \end{aligned}$$

$$\left[ \begin{array}{cccccccccc} g_{1,0}^{(k+1)} & g_{1,0}^{(k+2)} & \dots & g_{1,0}^{(n)} & g_{1,1}^{(k+1)} & g_{1,1}^{(k+2)} & \dots & g_{1,1}^{(n)} & g_{1,m}^{(k+1)} & g_{1,m}^{(k+2)} & g_{1,m}^{(n)} \\ \vdots & \vdots & & \vdots & \vdots & \vdots & & \vdots & \vdots & \vdots & \vdots \\ g_{k,0}^{(k+1)} & g_{k,0}^{(k+2)} & \dots & g_{k,0}^{(n)} & g_{k,1}^{(k+1)} & g_{k,1}^{(k+2)} & \dots & g_{k,1}^{(n)} & g_{k,m}^{(k+1)} & g_{k,m}^{(k+2)} & g_{k,m}^{(n)} \\ 1 & 0 & \dots & 0 & 0 & 0 & \dots & 0 & \dots & 0 & 0 \\ 0 & 1 & \dots & 0 & 0 & 0 & \dots & 0 & \dots & 0 & \dots & 0 \\ \vdots & \vdots & \dots & \vdots & \vdots & \vdots & & \vdots & \vdots & \vdots & \dots & \vdots \\ 0 & 0 & & 1 & 0 & & \dots & 0 & \dots & 0 & \dots & 0 \\ & & & & g_{1,0}^{(k+1)} & g_{1,0}^{(k+2)} & \dots & g_{1,0}^{(n)} & & & & \ddots \\ & & & & \vdots & \vdots & & \vdots & & & & \\ & & & & g_{k,0}^{(k+1)} & g_{k,0}^{(k+2)} & \dots & g_{k,0}^{(n)} & & & & \\ & & & & 1 & 0 & \dots & 0 & \dots & & & \\ & & & & 0 & 1 & \dots & 0 & \dots & & & \\ & & & & \vdots & \vdots & & \vdots & & & & \\ & & & & 0 & 0 & \dots & 1 & \dots & & & \end{array} \right] \quad (11.17)$$

The  $(n - k)$  syndrome components, each one corresponding to a received sequence, are expressed by



$$\begin{aligned}
 s_{\lambda}^{(k+j)} &= r_{\lambda}^{(k+j)} + \sum_{i=1}^k r_{\lambda}^{(i)} g_{i,0}^{(k+j)} + \sum_{i=1}^k r_{\lambda-1}^{(i)} g_{i,1}^{(k+j)} + \cdots + \sum_{i=1}^k r_{\lambda-m}^{(i)} g_{i,m}^{(k+j)} \\
 &= r_{\lambda}^{(k+j)} + \sum_{\mu=0}^m \sum_{i=1}^k r_{\lambda-\mu}^{(i)} g_{i,\mu}^{(k+j)} \quad j = 1, 2, \dots, n-k \\
 &\quad \lambda = 0, 1, 2, \dots, m
 \end{aligned} \tag{11.18}$$

On the other hand, since  $\mathbf{s} = \mathbf{e} \cdot \mathbf{H}^T$  also,  $(n-k)$  syndrome components, each one corresponding to each parity-error sequence, are given as

$$\begin{aligned}
 s_{\lambda}^{(k+j)} &= e_{\lambda}^{(k+j)} + \sum_{i=1}^k e_{\lambda}^{(i)} g_{i,0}^{(k+j)} + \sum_{i=1}^k e_{\lambda-1}^{(i)} g_{i,1}^{(k+j)} + \cdots + \sum_{i=1}^k e_{\lambda-m}^{(i)} g_{i,m}^{(k+j)} \\
 &= e_{\lambda}^{(k+j)} + \sum_{\mu=0}^m \sum_{i=1}^k e_{\lambda-\mu}^{(i)} g_{i,\mu}^{(k+j)} \quad j = 1, 2, \dots, n-k \\
 &\quad \lambda = 0, 1, 2, \dots, m
 \end{aligned} \tag{11.19}$$

The general structure of a feedback threshold decoder for an  $(n, k, m)$  systematic convolutional code, which is based on Eq. (11.18) or Eq. (11.19), is shown in Fig. 11.2. This syndrome-error-feedback decoder is called a threshold decoder because of the threshold logic (also called majority logic) involved in the error decisions.

Threshold decoding of a convolutional code is based on the concept of orthogonal parity-check sums. The threshold decoder is a device of majority-logic circuitry that operates on a syndrome to produce a likely estimate of some selected error digits. Any syndrome bit, or any sum of syndrome bits, represents a known sum of channel-error bits. We call such a sum a parity-check sum. Therefore, each check sum is a syndrome digit or a known sum of syndrome digits. If the received sequence is not a transmitted code word, some check sums will not be zero. A parity-check sum  $\xi_i$  is said to check an error bit  $e_0^{(1)}$  if  $e_0^{(1)}$  appears in  $\xi_i$ . A set of parity-check sums is said to be orthogonal on  $e_0^{(1)}$  if each  $\xi_i$  checks  $e_0^{(1)}$ , but no other error bit is checked by more than one check sum. If it is given a set of  $J$  orthogonal check sums on an error bit  $e_0^{(1)}$ , the majority vote can be used to estimate the value of  $e_0^{(1)}$ . The threshold gate will produce an output of 1 if and only if the majority vote of its inputs is 1. The output of the threshold gate is the estimate of  $e_0^{(1)}$ , that is,  $\hat{e}_0^{(1)}$ . If  $\hat{e}_0^{(1)} = 1$ ,  $r_0^{(1)}$  is assumed to be incorrect and hence must be corrected. If  $\hat{e}_0^{(1)} = 0$ ,  $r_0^{(1)}$  is assumed to be correct. The error correction is performed by adding  $\hat{e}_0^{(1)}$  to  $r_0^{(1)}$ . The output of the threshold gate  $\hat{e}_0^{(1)}$  is also fed back to modify the syndrome register before the next estimate can be made. This is the reason why such a decoding circuit is called a feedback decoder.

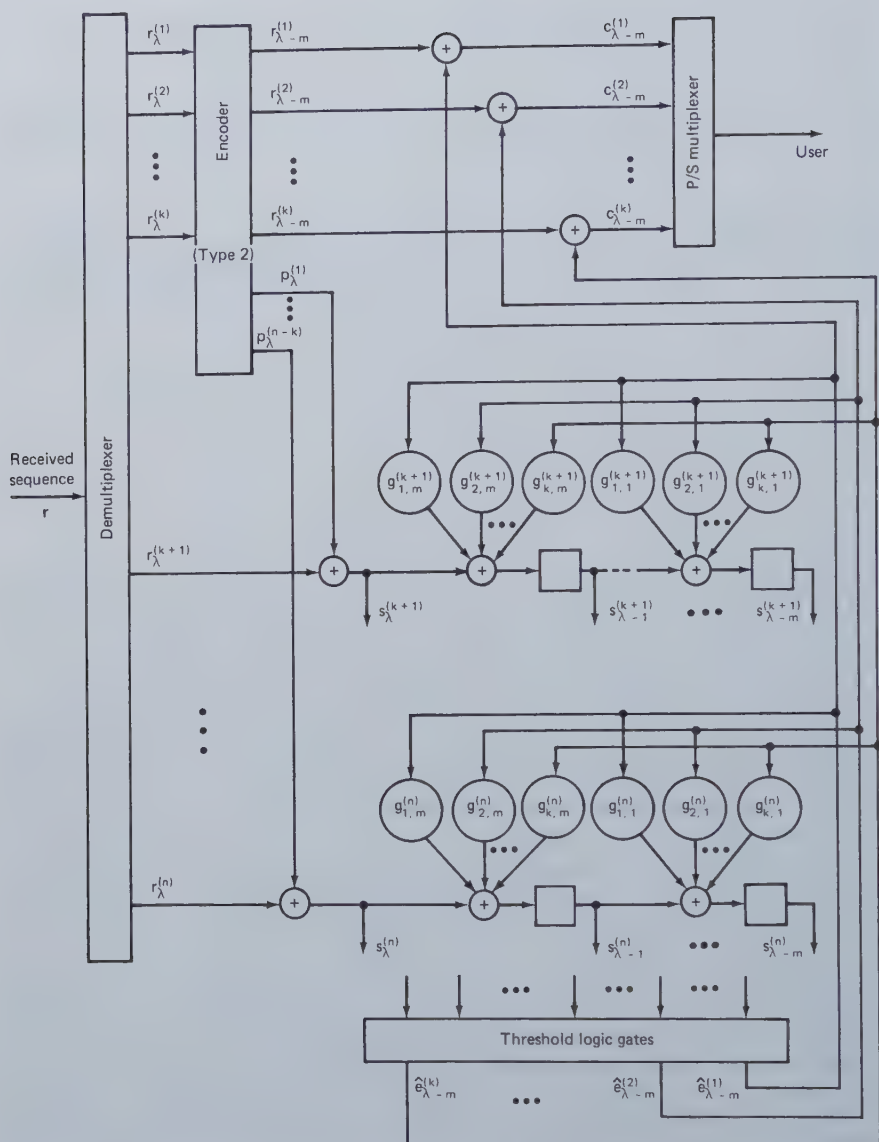


Figure 11.2 Complete threshold decoder for an  $(n, k, m)$  systematic convolutional code.

If an incorrect estimate  $\hat{e}_0^{(1)} \neq e_0^{(1)}$  is fed back to the syndrome, additional errors in subsequent decoding will occur. This is called the error propagation effect and is a serious flaw in threshold decoders using decision feedback. The error propagation effect has been the subject of much attention in the literature in the past. There are two approaches that are generally used to reduce the effects of error propagation. Er-

ror propagation can be either limited or unlimited. Limited error propagation cannot be entirely eliminated in a feedback decoder. One solution is to select codes that have the automatic recovery property which limits error propagation. For such codes, the decoder will recover from the error propagation by reducing the length of time that errors propagate. Unlimited error propagation is quite serious and is due to poor code selection. In selecting good codes, one should avoid the so-called catastrophic error-propagating codes because they will perform poorly no matter which decoding technique is used. However, even when good codes are used, improper decoder design can also cause unlimited error propagation. Systematic codes never have this problem. Massey and Sain have determined the properties of the code generator polynomials that cause this error propagation effect. The other approach to limiting error propagation is simply to disconnect feedback in the decoder. This is called definite decoding. However, the disadvantage is the increased probability of error. Consequently the error-correcting capability of the code will be reduced.

### 11.3 Some Codes for Threshold Decoding

Some classes of systematic convolutional codes can be used with threshold decoding if and only if these codes are completely orthogonalized. Convolutional codes that allow threshold decoding using simple-majority logic gates have a special structure which can best be explained from the several examples discussed below.

#### 11.3.1 Self-orthogonal codes

Let an  $(n, n-1)$  convolutional code with memory order  $m$  be a self-orthogonal code. The orthogonal parity checks of this code are syndrome bits themselves, not sums of syndrome bits. An  $(n, n-1, m)$  code is said to be self-orthogonal if the set of all syndrome bits that checks one error bit forms a set of check sums orthogonal on that error bit. Thus the set of orthogonal check sums consists of single syndrome bits rather than a linear combination of syndrome bits. Robinson and Bernstein (1967) have suggested a procedure for constructing  $(n, n-1, m)$  self-orthogonal codes using the concept of difference sets. The positive difference set associated with a set of integers is defined as the set of positive differences that are all distinct.

Consider an  $R = 1/2$ ,  $m$  systematic convolutional code. The syndrome vector of this code is expressed as

$$\begin{aligned} \mathbf{s} &= \mathbf{e}^{(1)} \mathbf{G}^{(2)} + \mathbf{e}^{(2)} \\ &= (s_0, s_1, s_2, \dots, s_m) \end{aligned} \quad (11.20)$$

where  $\mathbf{e}^{(1)}$  and  $\mathbf{e}^{(2)}$  are the information and parity error subsequences, respectively, and  $\mathbf{G}^{(2)}$  represents

$$\mathbf{G}^{(2)} = \begin{bmatrix} g_0^{(2)} & g_1^{(2)} & g_2^{(2)} & \cdots & g_{m-1}^{(2)} & g_m^{(2)} \\ & g_0^{(2)} & g_1^{(2)} & \cdots & g_{m-2}^{(2)} & g_{m-1}^{(2)} & g_m^{(2)} \\ & & g_0^{(2)} & \cdots & g_{m-3}^{(2)} & g_{m-2}^{(2)} & g_{m-1}^{(2)} & g_m^{(2)} \\ & & & & & & \ddots & \\ & & & & & & & g_0^{(2)} \end{bmatrix} \quad (11.21)$$

By taking the transpose of both sides of Eq. (11.20), we get

$$\mathbf{s}^T = [\mathbf{G}^{(2)}]^T \cdot [\mathbf{e}^{(1)}]^T + [\mathbf{e}^{(2)}]^T \quad (11.22)$$

where  $[\mathbf{G}^{(2)}]^T$ , which multiplies the information-error subsequence, is called the parity triangle of the code. The first column of the parity triangle is nothing but the generator sequence  $\mathbf{g}_1^{(2)}$ . The parity triangle of this code is used to select a set of check sums orthogonal on a given error bit. A list of some  $(n, n-1, m, t)$  self-orthogonal codes for various values of  $n$ , error-correcting capability  $t$ , and encoder memory  $m$  is given in Table 11.1.

**Example 11.2** Consider the  $R = 1/2$ ,  $m = 6$  systematic convolutional code with the generator sequences  $\mathbf{g}_1^{(1)} = (1 \ 0 \ 0 \ 0 \ 0 \ 0)$  and  $\mathbf{g}_1^{(2)} = (1 \ 0 \ 1 \ 0 \ 0 \ 1)$ . The generator polynomial  $g_1^{(2)}(D) = 1 + D^2 + D^5 + D^6$  corresponds to the set of integers  $g_1^{(2)} = \{0, 2, 5, 6\}$ . Thus the difference set associated with this generator polynomial is given by  $\{1, 2, 3, 4, 5, 6\}$ . Since this difference set contains all distinct positive integers, this code is self-orthogonal.

Let us obtain the parity triangle which is used for selecting a set of check sums orthogonal on  $e_0^{(1)}$ . Two different approaches to computing the syndrome bits are shown below.

#### Approach I

Using Eq. (11.19) for  $k = 1, j = n - k = 1, \lambda = 0, 1, 2, \dots, 6$ , and  $\mathbf{g}_1^{(2)} = (1 \ 0 \ 1 \ 0 \ 0 \ 1) = (g_{1,0}^{(2)}, g_{1,1}^{(2)}, g_{1,2}^{(2)}, g_{1,3}^{(2)}, g_{1,4}^{(2)}, g_{1,5}^{(2)}, g_{1,6}^{(2)})$ , all syndrome bits can be formed as shown below.

$$\begin{aligned} s_0 &= e_0^{(1)} && + e_0^{(2)} \\ s_1 &= e_1^{(1)} && + e_1^{(2)} \\ s_2 &= e_0^{(1)} + e_2^{(1)} && + e_2^{(2)} \\ s_3 &= e_1^{(1)} + e_3^{(1)} && + e_3^{(2)} \\ s_4 &= e_2^{(1)} + e_4^{(1)} && + e_4^{(2)} \\ s_5 &= e_0^{(1)} + e_3^{(1)} + e_5^{(1)} && + e_5^{(2)} \\ s_6 &= e_0^{(1)} + e_1^{(1)} + e_4^{(1)} + e_6^{(1)} + e_6^{(2)} \end{aligned} \quad (11.23)$$

TABLE 11.1  $(n, n-2, 1)$  Self-orthogonal Convolutional Codes

| (a) $(2, 1)$ Codes |     |                                                                                                                                                                                         |
|--------------------|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $t$                | $m$ | $g_1^{(2)}(D)$                                                                                                                                                                          |
| 1                  | 1   | $1 + D$                                                                                                                                                                                 |
| 2                  | 6   | $1 + D^2 + D^5 + D^6$                                                                                                                                                                   |
| 3                  | 17  | $1 + D^2 + D^7 + D^{13} + D^{16} + D^{17}$                                                                                                                                              |
| 4                  | 35  | $1 + D^7 + D^{10} + D^{16} + D^{18} + D^{30} + D^{31} + D^{35}$                                                                                                                         |
| 5                  | 55  | $1 + D^2 + D^{14} + D^{21} + D^{29} + D^{32} + D^{45} + D^{49} + D^{54} + D^{55}$                                                                                                       |
| 6                  | 85  | $1 + D^2 + D^6 + D^{24} + D^{29} + D^{40} + D^{43} + D^{55} + D^{68} + D^{75} + D^{78} + D^{85}$                                                                                        |
| (b) $(3, 2)$ Codes |     |                                                                                                                                                                                         |
| $t$                | $m$ | $g_1^{(3)}(D)$ $g_2^{(3)}(D)$                                                                                                                                                           |
| 1                  | 2   | $1 + D$ $1 + D^2$                                                                                                                                                                       |
| 2                  | 13  | $1 + D^8 + D^9 + D^{12}$ $1 + D^6 + D^{11} + D^{13}$                                                                                                                                    |
| 3                  | 40  | $1 + D^2 + D^6 + D^{24} + D^{29} + D^{40}$ $1 + D^3 + D^{15} + D^{28} + D^{35} + D^{36}$                                                                                                |
| 4                  | 86  | $1 + D + D^{27} + D^{30} + D^{61} + D^{73} + D^{81} + D^{83}$ $1 + D^{16} + D^{23} + D^{37} + D^{58} + D^{62} + D^{75} + D^{86}$                                                        |
| (c) $(4, 3)$ Codes |     |                                                                                                                                                                                         |
| $t$                | $m$ | $g_1^{(4)}(D)$ $g_2^{(4)}(D)$ $g_3^{(4)}(D)$                                                                                                                                            |
| 1                  | 3   | $1 + D$ $1 + D^2$ $1 + D^3$                                                                                                                                                             |
| 2                  | 19  | $1 + D^3 + D^{15} + D^{19}$ $1 + D^6 + D^{17} + D^{18}$ $1 + D^6 + D^{11} + D^{13}$                                                                                                     |
| 3                  | 67  | $1 + D^5 + D^{15} + D^{34} + D^{35} + D^{42}$ $1 + D^{31} + D^{33} + D^{44} + D^{47} + D^{56}$ $1 + D^{17} + D^{21} + D^{43} + D^{49} + D^{67}$                                         |
| (d) $(5, 4)$ Codes |     |                                                                                                                                                                                         |
| $t$                | $m$ | $g_1^{(5)}(D)$ $g_2^{(5)}(D)$ $g_3^{(5)}(D)$ $g_4^{(5)}(D)$                                                                                                                             |
| 1                  | 4   | $1 + D$ $1 + D^2$ $1 + D^3$ $1 + D^4$                                                                                                                                                   |
| 2                  | 26  | $1 + D^{16} + D^{20} + D^{21}$ $1 + D^2 + D^{10} + D^{25}$ $1 + D^{14} + D^{17} + D^{26}$ $1 + D^{11} + D^{18} + D^{24}$                                                                |
| 3                  | 78  | $1 + D^5 + D^{26} + D^{51} + D^{55} + D^{69}$ $1 + D^6 + D^7 + D^{11} + D^{60} + D^{72}$ $1 + D^8 + D^{11} + D^{24} + D^{44} + D^{78}$ $1 + D^{10} + D^{32} + D^{47} + D^{49} + D^{77}$ |



When expressed in matrix notation, Eq. (11.23) becomes

$$[\mathbf{s}^{(2)}]^T = \begin{bmatrix} s_0 \\ s_1 \\ s_2 \\ s_3 \\ s_4 \\ s_5 \\ s_6 \end{bmatrix} = \begin{bmatrix} 1 & & & & & & \\ & 0 & 1 & & & & \\ & 1 & 0 & 1 & & & \\ & 0 & 1 & 0 & 1 & & \\ & 0 & 0 & 1 & 0 & 1 & \\ & 1 & 0 & 0 & 0 & 1 & 0 & 1 \\ & 1 & 1 & 0 & & & & \end{bmatrix} \begin{bmatrix} e_0^{(1)} \\ e_1^{(1)} \\ e_2^{(1)} \\ e_3^{(1)} \\ e_4^{(1)} \\ e_5^{(1)} \\ e_6^{(1)} \end{bmatrix} + \begin{bmatrix} e_0^{(2)} \\ e_1^{(2)} \\ e_2^{(2)} \\ e_3^{(2)} \\ e_4^{(2)} \\ e_5^{(2)} \\ e_6^{(2)} \end{bmatrix} \quad (11.24)$$

The matrix in Eq. (11.24), which multiplies the information-error subsequence, is called the parity triangle of this code. The first column of the parity triangle is exactly identical to the generator sequence  $\mathbf{g}_1^{(2)}$ . We see that the parity triangle is formed by shifting down by one bit and truncating in column after column. This parity triangle is now used to select a set of check sums orthogonal on  $e_0^{(1)}$ . From these seven syndrome bits of Eqs. (11.23) or (11.24), four parity-check sums orthogonal on  $e_0^{(1)}$  can be selected

$$\begin{aligned} s_0^{(0)} &= e_0^{(1)} && && && + e_0^{(2)} \\ s_2^{(0)} &= e_0^{(1)} && + e_2^{(1)} && && + e_2^{(2)} \\ s_5^{(0)} &= e_0^{(1)} && & + e_3^{(1)} && + e_5^{(1)} & + e_5^{(2)} \\ s_6^{(0)} &= e_0^{(1)} + e_1^{(1)} && & & + e_4^{(1)} && + e_6^{(1)} + e_6^{(2)} \end{aligned} \quad (11.25)$$

Since  $e_0^{(1)}$  appears in each check sum and no other error bit appears more than once, this code is self-orthogonal. Thus we can obtain a correct estimate  $\hat{e}_0^{(1)}$  of  $e_0^{(1)}$  by using the majority-logic decoding rule whenever two or fewer errors occur among the 11 error bits checked by these four syndrome bits. As a result, the (2, 1, 6) self-orthogonal code has  $J = 4$  orthogonal check sums and is a double-error-correcting code because  $t = \lfloor J/2 \rfloor = 2$ . A threshold decoder for multiple-error correction must be able to estimate not only  $e_0^{(1)}$ , but also all other information error bits  $e_\lambda^{(1)}$ . Since  $t = 2$  in this example, a set of orthogonal check sums should be formed on another error bit, say,  $e_1^{(1)}$ . Once  $e_0^{(1)}$  is estimated correctly, i.e.,  $\hat{e}_0^{(1)} = e_0^{(1)}$ , it must be subtracted from each check sum it affects in order to form another set of check sums. Thus the new check sums orthogonal on  $e_1^{(1)}$  can be formed

$$\begin{aligned} s_1^{(1)} &= e_1^{(1)} && && && + e_1^{(2)} \\ s_3^{(1)} &= e_1^{(1)} && + e_3^{(1)} && && + e_3^{(2)} \\ s_6^{(1)} &= e_1^{(1)} && & + e_4^{(1)} && + e_6^{(1)} & + e_6^{(2)} \\ s_7^{(1)} &= e_1^{(1)} + e_2^{(1)} && & & + e_5^{(1)} && + e_7^{(1)} + e_7^{(2)} \end{aligned} \quad (11.26)$$

Thus, the exact same decoding rule that was used for  $e_0^{(1)}$  can be applied to estimate  $e_1^{(1)}$  because Eqs. (11.25) and (11.26) are identical, except that different error bits are checked. This means that there is no need to change the decoding circuitry in order to estimate  $e_1^{(1)}$ . A complete encoder-decoder pair for the (2, 1, 6) self-orthogonal code with threshold decoding is shown in Fig. 11.3.

### Approach II

A second approach to constructing orthogonal check sums utilizes Eq. (11.22). From Eq. (11.21), the transpose matrix of  $\mathbf{G}^{(2)}$ , truncated over the first constraint length  $n_A = m + 1$  is

$$[\mathbf{G}^{(2)}]^T = \begin{bmatrix} g_0^{(2)} & & & & & \\ g_1^{(2)} & g_0^{(2)} & & & & \\ g_2^{(2)} & g_1^{(2)} & g_0^{(2)} & & & \\ \vdots & & & & & \\ g_{m-1}^{(2)} & g_{m-2}^{(2)} & g_{m-3}^{(2)} & \cdots & g_0^{(2)} & \\ g_m^{(2)} & g_{m-1}^{(2)} & g_{m-2}^{(2)} & \cdots & g_1^{(2)} & g_0^{(2)} \end{bmatrix} \quad (11.27)$$

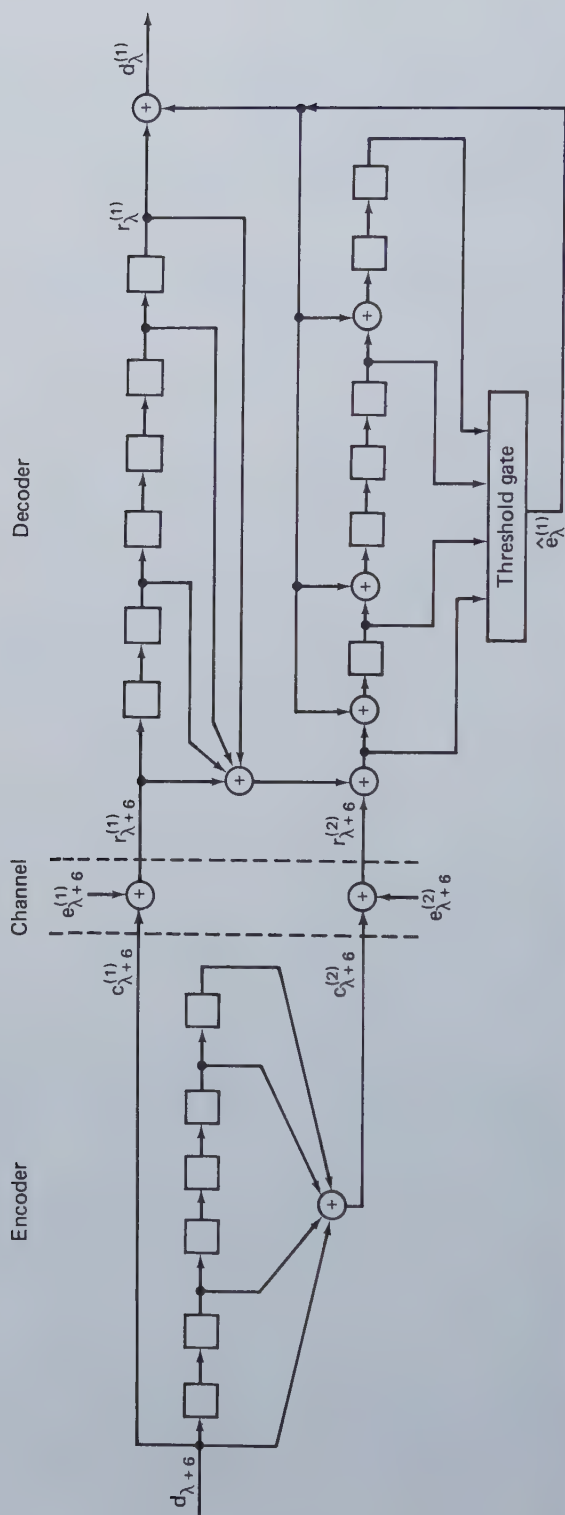
Since  $\mathbf{g}_1^{(2)} = (1 \ 0 \ 1 \ 0 \ 0 \ 1 \ 1) = (g_{1,0}^{(2)}, g_{1,1}^{(2)}, g_{1,2}^{(2)}, g_{1,3}^{(2)}, g_{1,4}^{(2)}, g_{1,5}^{(2)}, g_{1,6}^{(2)})$ , Eq. (11.27) becomes

$$[\mathbf{G}^{(2)}]^T = \begin{bmatrix} 1 & & & & & & \\ 0 & 1 & & & & & \\ 1 & 0 & 1 & & & & \\ 0 & 1 & 0 & 1 & & & \\ 0 & 0 & 1 & 0 & 1 & & \\ 1 & 0 & 0 & 1 & 0 & 1 & \\ 1 & 1 & 0 & 0 & 1 & 0 & 1 \end{bmatrix} \quad (11.28)$$

This is the same parity triangle as we obtained before. Substitution of Eq. (11.28) into Eq. (11.22) yields the same syndrome bits as shown in Eq. (11.24), and thus the orthogonal check sums on  $e_0^{(1)}$  and  $e_1^{(1)}$  are easily formed as shown in Eqs. (11.25) and (11.26), respectively.

Generally, as shown through this example, some of the syndrome bits  $s_\lambda^{(i)}$ ,  $\lambda = 0, 1, 2, \dots, m$ , orthogonal on any of the information-error bits  $e_i^{(1)}$ ,  $i = 0, 1, 2, \dots, m$ , can be easily formed for the first constraint length of syndrome bits.

**Example 11.3** Consider the (2, 1, 17) self-orthogonal code from Table 11.1 whose generator polynomial is  $g_1^{(2)}(D) = 1 + D^2 + D^7 + D^{13} + D^{16} + D^{17}$ . The set of integers associated with the powers of  $g_1^{(2)}(D)$  is  $g_1^{(2)} = \{0, 2, 7, 13, 16, 17\}$ . Therefore the positive difference set corresponding to  $g_1^{(2)}$  is given by  $\{17, 16,$



**Figure 11.3** A complete system diagram for a (2, 1, 6) self-orthogonal convolutional code with threshold decoding.

15, 14, 13, 11, 10, 9, 7, 6, 5, 4, 3, 2, 1} which contains all positive integers. Therefore this code is self-orthogonal. By applying  $k = 1, j = n - k = 1$ , and  $\lambda = 0, 1, 2, \dots, 16, 17$ , to Eq. (11.19), the first constraint length of syndrome bits can be constructed. Subsequently, the set of check sums orthogonal on the information error bit  $e_0^{(1)}$  taken from these 18 syndrome bits is  $\{s_0, s_2, s_7, s_{13}, s_{16}, s_{17}\}$ , which is identified by the integer set  $g_1^{(2)} = \{0, 2, 7, 13, 16, 17\}$ . Thus the orthogonal check sums on  $e_0^{(1)}$  are selected as follows:

$$s_0 = e_0^{(1)} + e_0^{(2)} \quad (11.29)$$

$$s_2 = e_0^{(1)} + e_2^{(1)} + e_2^{(2)}$$

$$s_7 = e_0^{(1)} + e_5^{(1)} + e_7^{(1)} + e_7^{(2)}$$

$$s_{13} = e_0^{(1)} + e_6^{(1)} + e_{11}^{(1)} + e_{13}^{(1)} + e_{13}^{(2)}$$

$$s_{16} = e_0^{(1)} + e_3^{(1)} + e_9^{(1)} + e_{14}^{(1)} + e_{16}^{(1)} + e_{16}^{(2)}$$

$$s_{17} = e_0^{(1)} + e_1^{(1)} + e_4^{(1)} + e_{10}^{(1)} + e_{15}^{(1)} + e_{17}^{(1)} + e_{17}^{(2)}$$

Since  $J = 6$ , this self-orthogonal code is a triple-error-correcting code. Recall that the first column of the parity triangle is the generator sequence  $g_1^{(2)} = \{0, 2, 7, 13, 16, 17\}$  and is shifted down one bit with truncated columns. Thus the syndrome bits of Eq. (11.29) can also be formed by using Eq. (11.22). The result is shown in Eq. (11.30) (see next page). The orthogonal check sums on  $e_0^{(1)}$  are indicated by numbered arrows in Eq. (11.30), whose set is identical to Eq. (11.29). The complete decoder diagram of this code is shown in Fig. 11.4. Note that a total of 22 different channel-error bits are checked by the six orthogonal check sums of Eq. (11.29), and the threshold-logic gates will correctly estimate  $e_0^{(1)}$  whenever  $t = 3$  or fewer error bits are 1s.

**Example 11.4** Consider the  $(3, 2, 2)$  self-orthogonal code from Table 11.1 whose generator polynomials are  $g_1^{(3)}(D) = 1 + D$  and  $g_2^{(3)}(D) = 1 + D^2$ . These generator polynomials are identified by the set of integers  $g_1^{(3)} = \{0, 1\}$  and  $g_2^{(3)} = \{0, 2\}$ , respectively. Since the difference sets associated with these generator polynomials are  $\{1\}$  and  $\{2\}$ , this code is proved to be self-orthogonal. Applying  $k = 2, j = n - k = 1$ , and  $\lambda = 0, 1, 2$ , to Eq. (11.19), the syndrome bits for the first constraint length are obtained as

$$s_0 = e_0^{(3)} + e_0^{(1)}g_{1,0}^{(3)} + e_0^{(2)}g_{2,0}^{(3)}$$

$$s_1 = e_1^{(3)} + e_1^{(1)}g_{1,0}^{(3)} + e_1^{(2)}g_{2,0}^{(3)} + e_0^{(1)}g_{1,1}^{(3)} + e_0^{(2)}g_{2,1}^{(3)} \quad (11.31)$$

$$s_2 = e_2^{(3)} + e_2^{(1)}g_{1,0}^{(3)} + e_2^{(2)}g_{2,0}^{(3)} + e_1^{(1)}g_{1,1}^{(3)} + e_1^{(2)}g_{2,1}^{(3)} + e_0^{(1)}g_{1,2}^{(3)} + e_0^{(2)}g_{2,2}^{(3)}$$

Using the generator sequences  $g_1^{(3)} = (1 \ 1 \ 0)$  and  $g_2^{(3)} = (1 \ 0 \ 1)$ , we see that Eq. (11.31) becomes

[illegible]



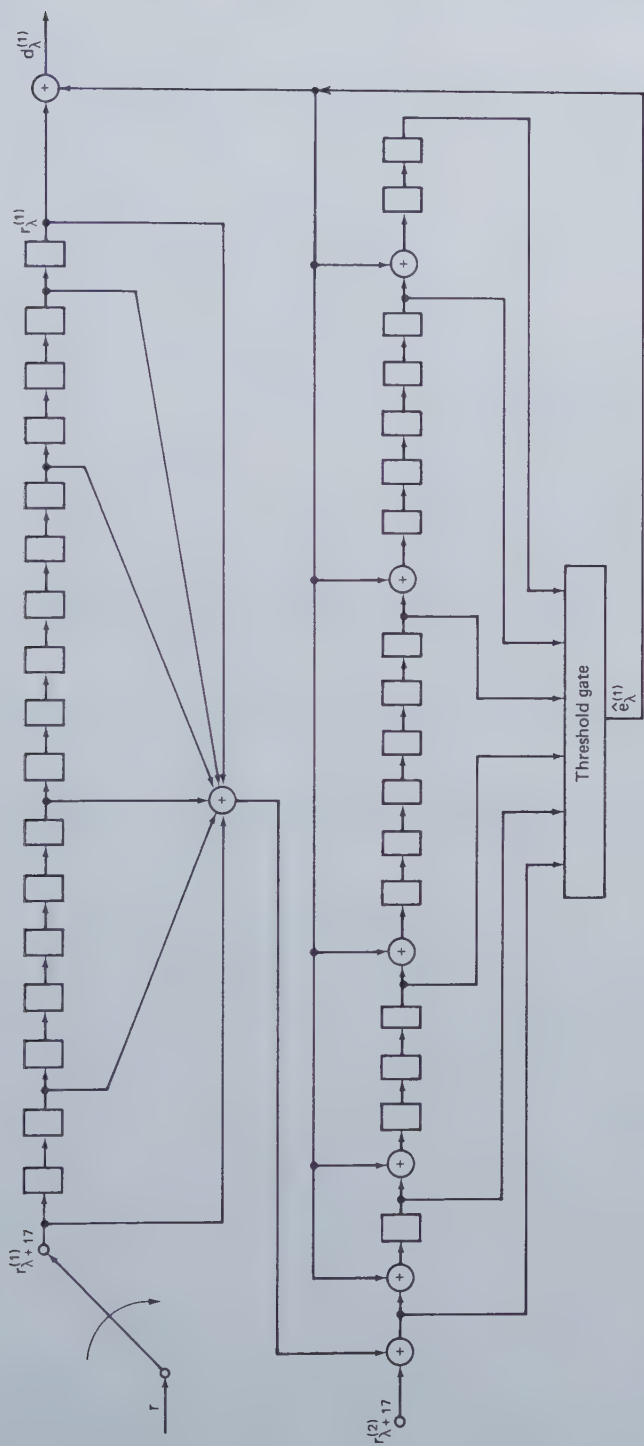


Figure 11.4 Complete threshold decoder for a  $(2, 1, 17)$  self-orthogonal code.

$$\begin{aligned}
s_0 &= e_0^{(1)} + e_0^{(2)} + e_0^{(3)} \\
s_1 &= e_0^{(1)} + e_1^{(1)} + e_1^{(2)} + e_1^{(3)} \\
s_2 &= e_0^{(2)} + e_1^{(1)} + e_2^{(1)} + e_2^{(2)} + e_2^{(3)}
\end{aligned} \tag{11.32}$$

from which the orthogonal check sums on  $e_0^{(1)}$  yield

$$\begin{aligned}
s_0 &= e_0^{(1)} + e_0^{(2)} + e_0^{(3)} \\
s_1 &= e_0^{(1)} + e_1^{(1)} + e_1^{(2)} + e_1^{(3)}
\end{aligned} \tag{11.33}$$

and the orthogonal check sums on  $e_0^{(2)}$  yield

$$\begin{aligned}
s_0 &= e_0^{(1)} + e_0^{(2)} + e_0^{(3)} \\
s_2 &= e_0^{(2)} + e_1^{(1)} + e_2^{(1)} + e_2^{(2)} + e_2^{(3)}
\end{aligned} \tag{11.34}$$

On the other hand, there are two parity triangles in this case: one corresponding to  $\mathbf{g}_1^{(3)} = (1 \ 1 \ 0)$  and one corresponding to  $\mathbf{g}_2^{(3)} = (1 \ 0 \ 1)$ . They are

$$\Delta_1 = \begin{bmatrix} 1 & & \\ 1 & 1 & \\ 0 & 1 & 1 \end{bmatrix} \quad \Delta_2 = \begin{bmatrix} 1 & & \\ 0 & 1 & \\ 1 & 0 & 1 \end{bmatrix}$$

These two triangles are used to form a set of three syndrome bits  $s_\lambda$ ,  $\lambda = 0, 1, 2$ , as follows:

$$\mathbf{s}^T = \begin{bmatrix} s_0 \\ s_1 \\ s_2 \end{bmatrix} = \begin{bmatrix} 1 & & \\ 1 & 1 & \\ 0 & 1 & 1 \end{bmatrix} \cdot \begin{bmatrix} e_0^{(1)} \\ e_1^{(1)} \\ e_2^{(1)} \end{bmatrix} + \begin{bmatrix} 1 & & \\ 0 & 1 & \\ 1 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} e_0^{(2)} \\ e_1^{(2)} \\ e_2^{(2)} \end{bmatrix} + \begin{bmatrix} e_0^{(3)} \\ e_1^{(3)} \\ e_2^{(3)} \end{bmatrix} \tag{11.35}$$

Thus, a set of two orthogonal check sums on the information-error bit  $e_0^{(1)}$  is

$$\begin{aligned}
s_0 &= e_0^{(1)} + e_0^{(2)} + e_0^{(3)} \\
s_1 &= e_0^{(1)} + e_1^{(1)} + e_1^{(2)} + e_1^{(3)}
\end{aligned} \tag{11.36}$$

Likewise, a set of two orthogonal check sums on the information-error bit  $e_0^{(2)}$  is

$$\begin{aligned}
s_0 &= e_0^{(1)} + e_0^{(2)} + e_0^{(3)} \\
s_2 &= e_1^{(1)} + e_2^{(1)} + e_0^{(2)} + e_2^{(2)} + e_2^{(3)}
\end{aligned} \tag{11.37}$$

Thus we see that Eqs. (11.36) and (11.37) are identical to Eqs. (11.33) and (11.34). Since each check sum comprises one syndrome bit, not sums of syndrome bits, this is a self-orthogonal code. Six information-error bits and three parity-error bits make a total of nine different channel-error bits that are checked by the two sets of orthogonal check sums. Therefore, the threshold de-

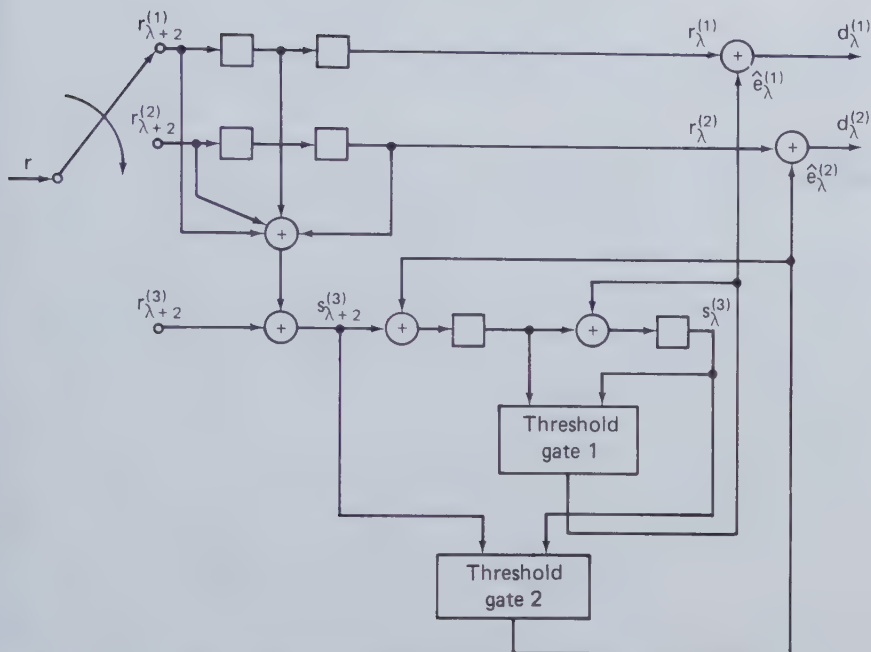


Figure 11.5 Complete threshold decoder for a (3, 2, 2) self-orthogonal code.

coding rule will correctly estimate both  $e_0^{(1)}$  and  $e_0^{(2)}$  if there are  $t = 2$  or fewer errors among these nine error bits, which are checked by Eqs. (11.36) and (11.37). A complete threshold decoder for a (3, 2, 2) self-orthogonal code is shown in Fig. 11.5.

### 11.3.2 Trial-and-error orthogonalizable codes

Another useful class of convolutional codes is the class of so-called trial-and-error codes introduced by Massey. Massey has constructed a large number of trial-and-error codes by forming the orthogonal check sums, which are composed of the syndrome bits or sums of syndrome bits. We call this class of trial-and-error codes the orthogonalizable codes, and they differ from the self-orthogonal codes in a few respects. If some of the orthogonal check sums are formed from sums of two or more syndrome bits, we can construct orthogonalizable codes. The estimates from the majority vote can also be used with feedback decoding to achieve a full error-correction capability. However, a disadvantage of orthogonalizable codes is that they do not possess the automatic resynchronization properties that limit error propagation with feedback. If definite decoding were used, a significant loss in performance would occur, and the error-correcting capability is reduced accordingly. For a given number of orthogonal check sums, trial-and-error codes require shorter constraint lengths than self-orthogonal

codes. A list of trial-and-error orthogonalizable codes, constructed by Massey, with the rules for forming the orthogonal check sums is given in Table 11.2.

**Example 11.5** Consider the (2, 1, 11) trial-and-error orthogonalizable code listed in Table 11.2. The generator polynomial  $g_1^{(2)}(D) = 1 + D^6 + D^7 + D^9 + D^{10} + D^{11}$  is obtained from the set of integers  $\{0, 6, 7, 9, 10, 11\}$ . According to Table 11.2, the orthogonalization rule for forming  $J = 6$  orthogonal check sums is given by the set  $\{0^2, 6^2, 7^2, 9^2, 1^2 3^2 10^2, 4^2 8^2 11^2\}$ , where  $\lambda^i$  denotes that the syndrome bit  $s_{\lambda^i}$  forms a check sum orthogonal on  $e_0^{(1)}$  and  $\lambda^i \theta^j$  denotes that the sum of syndrome bits  $s_{\lambda^i} + s_{\theta^j}$  forms an orthogonal check sum on  $e_0^{(1)}$ . Now, the syndrome sequence  $\mathbf{s}^T$  of this code can easily be obtained by forming the parity triangle by using the generator polynomial. Thus all syndrome bits are obtained as shown below.

$$\begin{aligned}
 s_0 &= e_0^{(1)} && + e_0^{(2)} \\
 s_1 &= e_1^{(1)} && + e_1^{(2)} \\
 s_2 &= e_2^{(1)} && + e_2^{(2)} \\
 s_3 &= e_3^{(1)} && + e_3^{(2)} \\
 s_4 &= e_4^{(1)} && + e_4^{(2)} \\
 s_5 &= e_5^{(1)} && + e_5^{(2)} \\
 s_6 &= e_0^{(1)} + e_6^{(1)} && + e_6^{(2)} \\
 s_7 &= e_0^{(1)} + e_1^{(1)} + e_7^{(1)} && + e_7^{(2)} \\
 s_8 &= e_1^{(1)} + e_2^{(1)} + e_8^{(1)} && + e_8^{(2)} \\
 s_9 &= e_0^{(1)} + e_2^{(1)} + e_3^{(1)} + e_9^{(1)} && + e_9^{(2)} \\
 s_{10} &= e_0^{(1)} + e_1^{(1)} + e_3^{(1)} + e_4^{(1)} + e_{10}^{(1)} && + e_{10}^{(2)} \\
 s_{11} &= e_0^{(1)} + e_1^{(1)} + e_2^{(1)} + e_4^{(1)} + e_5^{(1)} + e_{11}^{(1)} && + e_{11}^{(2)}
 \end{aligned} \tag{11.38}$$

If we apply the orthogonalization rule to Eq. (11.38), we obtain the  $J = 6$  orthogonal check sums on  $e_0^{(1)}$  as follows:

$$\begin{aligned}
 \xi_1 &= s_0 = e_0^{(1)} && + e_0^{(2)} \\
 \xi_2 &= s_6 = e_0^{(1)} + e_6^{(1)} && + e_6^{(2)} \\
 \xi_3 &= s_7 = e_0^{(1)} + e_1^{(1)} + e_7^{(1)} && + e_7^{(2)} \\
 \xi_4 &= s_9 = e_0^{(1)} + e_2^{(1)} + e_3^{(1)} + e_9^{(1)} && + e_9^{(2)} \\
 \xi_5 &= s_1 + s_3 + s_{10} = e_0^{(1)} + e_4^{(1)} + e_{10}^{(1)} + e_1^{(2)} + e_3^{(2)} && + e_{10}^{(2)} \\
 \xi_6 &= s_4 + s_8 + s_{11} = e_0^{(1)} + e_5^{(1)} + e_8^{(1)} + e_{11}^{(1)} + e_4^{(2)} + e_8^{(2)} + e_{11}^{(2)}
 \end{aligned} \tag{11.39}$$

Since  $J = d_{\min} - 1 = 6$ ,  $d_{\min} = 7$ . Thus this code has the error-correcting capability  $t = \lfloor J/2 \rfloor = 3$ . A threshold decoder for a (2, 1, 11) trial-and-error code is shown in Fig. 11.6.

TABLE 11.2 Trial-and-Error Orthogonalizable Codes

| (a) $R = 1/2$ codes |     |                                                 |                                                                                                                                                                                                |                                                                                                                                                       |
|---------------------|-----|-------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| $J$                 | $m$ | $g_1^{(2)}$                                     | Orthogonalization rules                                                                                                                                                                        |                                                                                                                                                       |
| 4                   | 5   | {0, 3, 4, 5}                                    | $(0^2) (3^2) (4^2) (1^{25})$                                                                                                                                                                   |                                                                                                                                                       |
| 6                   | 11  | {0, 6, 7, 9, 10, 11}                            | $(0^2) (6^3) (7^4) (9^2) (1^{32} 10^2) (4^2 8^2 11^2)$                                                                                                                                         |                                                                                                                                                       |
| 8                   | 21  | {0, 11, 13, 16, 17, 19, 20, 21}                 | $(0^2) (11^2) (13^2) (16^2) (17^2) (2^{23} 6^2 19^2) (4^2 14^2 20^2) (1^2 5^2 8^2 15^2 21^2)$                                                                                                  |                                                                                                                                                       |
| 10                  | 35  | {0, 18, 19, 27, 28, 29, 30, 32, 33, 35}         | $(0^2) (18^2) (19^3) (27^3) (1^2 9^2 28^2) (10^2 20^2 29^2) (11^2 30^2 31^2) (13^2 21^2 23^2 32^2) (14^2 33^2 34^2) (2^{23} 16^2 24^2 26^2 35^2)$                                              |                                                                                                                                                       |
| (b) $R = 1/3$ codes |     |                                                 |                                                                                                                                                                                                |                                                                                                                                                       |
| $J$                 | $m$ | $g_1^{(3)}$                                     | Orthogonalization rules                                                                                                                                                                        |                                                                                                                                                       |
| 6                   | 4   | {0, 2, 3, 4}                                    | $(0^2) (0^3) (1^2) (2^3) (1^{32}) (2^{24})$                                                                                                                                                    |                                                                                                                                                       |
| 8                   | 7   | {0, 2, 3, 4, 6}                                 | $(0^2) (0^3) (1^2) (2^3) (1^{32}) (2^{24}) (7^2) (3^{25} 6^2 6^3)$                                                                                                                             |                                                                                                                                                       |
| 10                  | 10  | {0, 1, 2, 3, 5, 8, 9}                           | $(0^2) (0^3) (0^3) (1^2) (2^{23}) (9^2) (3^3 4^3) (3^{25} 5^3) (1^3 4^2 6^2 6^3) (8^2 8^3) (7^3 9^3 10^3)$                                                                                     |                                                                                                                                                       |
| 12                  | 17  | {0, 1, 14, 15, 16}                              | $(0^2) (0^3) (1^{21}) (4^2) (5^2) (2^{36}) (14^3) (7^2 10^2 11^2 1^3) (3^{35} 3^3) (6^3 8^3 12^3) (3^3 16^3 17^3) (4^3 10^3 12^3 16^4)$                                                        |                                                                                                                                                       |
| 14                  | 22  | {0, 1, 20, 22}                                  | $(0^2) (0^3) (1^{21}) (4^2) (5^2) (2^{36}) (7^2 10^2 11^2 1^3) (3^{35} 3^2) (19^3 20^3) (22^3) (6^3 8^3 12^3) (4^3 10^3 12^3 16^2) (3^{27} 13^3 15^3 19^2) (9^3 13^2 14^3 18^2 20^2 21^2 1^3)$ |                                                                                                                                                       |
| (c) $R = 1/5$ codes |     |                                                 |                                                                                                                                                                                                |                                                                                                                                                       |
| $J$                 | $m$ | $g_1^{(2)}$ $g_1^{(3)}$ $g_1^{(4)}$ $g_1^{(5)}$ | Orthogonalization rules                                                                                                                                                                        |                                                                                                                                                       |
| 6                   | 1   | {0, 1}                                          | {0}                                                                                                                                                                                            | $(0^2) (0^3) (0^4) (0^5) (1^{214}) (1^{315})$                                                                                                         |
| 8                   | 2   | {0, 1, 2}                                       | {0, 2}                                                                                                                                                                                         | $(0^2) (0^3) (0^4) (0^5) (1^{214}) (1^{315}) (2^{223}) (2^{425})$                                                                                     |
| 10                  | 3   | {0, 1, 2, 3}                                    | {0, 3}                                                                                                                                                                                         | $(0^2) (0^3) (0^4) (0^5) (1^{214}) (1^{315}) (2^{223}) (2^{425}) (3^5) (3^2 3^3)$                                                                     |
| 13                  | 5   | {0, 1, 2, 3, 4}                                 | {0, 3, 5}                                                                                                                                                                                      | $(0^2) (0^3) (0^4) (0^5) (1^{214}) (1^{315}) (2^{223}) (2^{425}) (3^5) (3^2 3^3) (3^4 2^4 4^4) (5^5) (4^3 5^3 5^4)$                                   |
| 14                  | 6   | {0, 1, 2, 3, 4}                                 | {0, 3, 5}                                                                                                                                                                                      | $(0^2) (0^3) (0^4) (0^5) (1^{214}) (1^{315}) (2^{223}) (2^4 2^5) (3^5) (3^2 3^3) (3^4 2^4 4^4) (5^5) (4^3 5^2 5^4) (4^5 6^4)$                         |
| 16                  | 8   | {0, 1, 2, 3, 4}                                 | {0, 3, 5}                                                                                                                                                                                      | $(0^2) (0^3) (0^4) (0^5) (1^{214}) (1^{315}) (2^{223}) (2^{425}) (3^5) (3^2 3^3) (3^4 2^4 4^4) (5^5) (4^3 5^3 5^4) (4^5 6^4) (8^3) (5^2 6^3 7^2 7^4)$ |



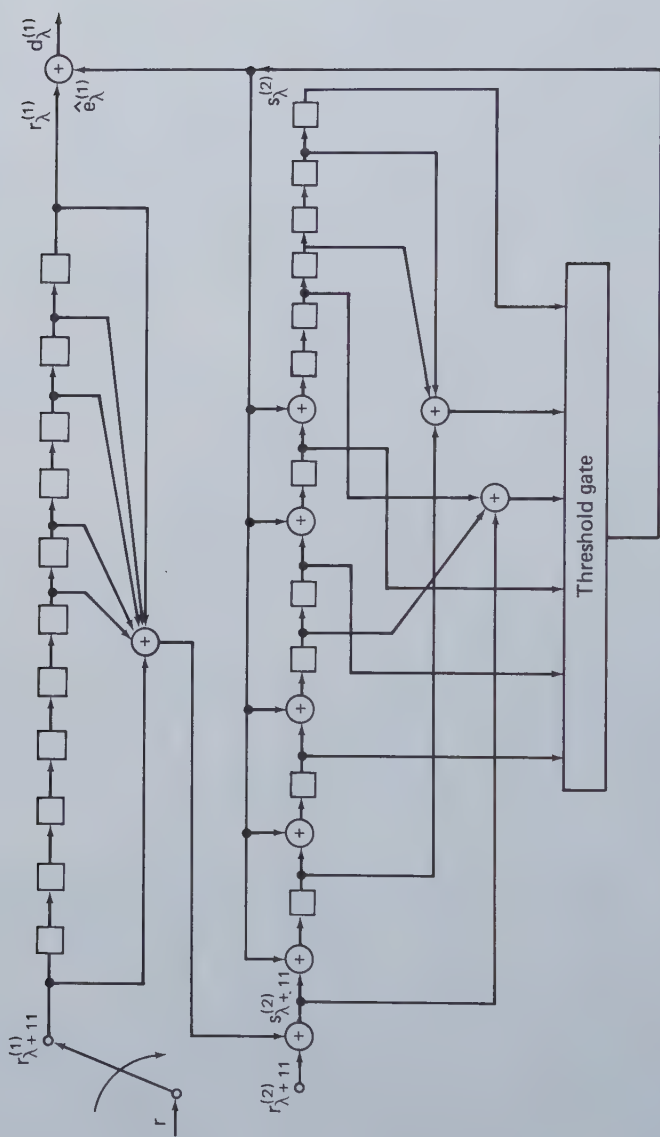


Figure 11.6 Threshold decoder for a  $(2, 1, 11)$  trial-and-error orthogonalizable code.

**Example 11.6** Consider the (3, 1) orthogonalizable code with  $m = 7$  whose generator polynomials are expressed by two sets of integers  $g_1^{(2)} = \{0, 1, 7\}$  and  $g_1^{(3)} = \{0, 2, 3, 4, 6\}$ . The orthogonalization rule for this code is given by  $0^2, 0^3, 1^2, 2^3, 1^3 3^3, 2^2 4^3, 7^2, 3^2 5^2 6^2 6^3$ . In this case, there are two parity triangles to be used, one corresponding to each generator sequence. Thus the two syndrome sequences  $s^{(2)} = (s_0^{(2)}, s_1^{(2)}, \dots, s_7^{(2)})$  and  $s^{(3)} = (s_0^{(3)}, s_1^{(3)}, \dots, s_7^{(3)})$  over the first constraint length can be written as

$$s^T = \begin{bmatrix} s_0^{(2)} \\ s_1^{(2)} \\ s_2^{(2)} \\ s_3^{(2)} \\ s_4^{(2)} \\ s_5^{(2)} \\ s_6^{(2)} \\ s_7^{(2)} \\ s_0^{(3)} \\ s_1^{(3)} \\ s_2^{(3)} \\ s_3^{(3)} \\ s_4^{(3)} \\ s_5^{(3)} \\ s_6^{(3)} \\ s_7^{(3)} \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \ 1 \\ 0 \ 1 \ 1 \\ 0 \ 0 \ 1 \ 1 \\ 0 \ 0 \ 0 \ 1 \ 1 \\ 0 \ 0 \ 0 \ 0 \ 1 \ 1 \\ 0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 1 \\ 1 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 1 \\ 1 \\ 0 \ 1 \\ 1 \ 0 \ 1 \\ 1 \ 1 \ 0 \ 1 \\ 1 \ 1 \ 1 \ 0 \ 1 \\ 0 \ 1 \ 1 \ 1 \ 0 \ 1 \\ 1 \ 0 \ 1 \ 1 \ 1 \ 0 \ 1 \\ 0 \ 1 \ 0 \ 1 \ 1 \ 1 \ 0 \ 1 \end{bmatrix} \begin{bmatrix} e_0^{(1)} \\ e_1^{(1)} \\ e_2^{(1)} \\ e_3^{(1)} \\ e_4^{(1)} \\ e_5^{(1)} \\ e_6^{(1)} \\ e_7^{(1)} \end{bmatrix} + \begin{bmatrix} e_0^{(2)} \\ e_1^{(2)} \\ e_2^{(2)} \\ e_3^{(2)} \\ e_4^{(2)} \\ e_5^{(2)} \\ e_6^{(2)} \\ e_7^{(2)} \\ e_0^{(3)} \\ e_1^{(3)} \\ e_2^{(3)} \\ e_3^{(3)} \\ e_4^{(3)} \\ e_5^{(3)} \\ e_6^{(3)} \\ e_7^{(3)} \end{bmatrix} \quad (11.40)$$

Applying Eq.(11.40) to the orthogonalization rule of this code, a set of eight orthogonal check sums on  $e_0^{(1)}$  can be formed as follows:

$$\begin{aligned} \xi_1 &= s_0^{(2)} &= e_0^{(1)} &+ e_0^{(2)} & \quad (11.41) \\ \xi_2 &= s_0^{(3)} &= e_0^{(1)} &+ e_0^{(3)} \\ \xi_3 &= s_1^{(2)} &= e_0^{(1)} + e_1^{(1)} &+ e_1^{(2)} \\ \xi_4 &= s_2^{(3)} &= e_0^{(1)} + e_2^{(1)} &+ e_2^{(3)} \\ \xi_5 &= s_1^{(3)} + s_3^{(3)} &= e_0^{(1)} + e_3^{(1)} &+ e_1^{(3)} + e_3^{(3)} \\ \xi_6 &= s_2^{(2)} + s_4^{(3)} &= e_0^{(1)} + e_4^{(1)} + e_2^{(2)} &+ e_4^{(3)} \\ \xi_7 &= s_7^{(2)} &= e_0^{(1)} + e_6^{(1)} + e_7^{(1)} &+ e_7^{(2)} \\ \xi_8 &= s_3^{(2)} + s_5^{(2)} + s_6^{(2)} + s_6^{(3)} &= e_0^{(1)} + e_3^{(2)} + e_5^{(2)} + e_6^{(2)} &+ e_6^{(3)} \end{aligned}$$

Since  $J = 8$ , the minimum distance of this code is  $d_{\min} = 9$ . This code has the error-correcting capability  $t = \lfloor J/2 \rfloor = 4$ . From the preceding discussion we can construct the threshold decoder of this code as shown in Fig. 11.7.

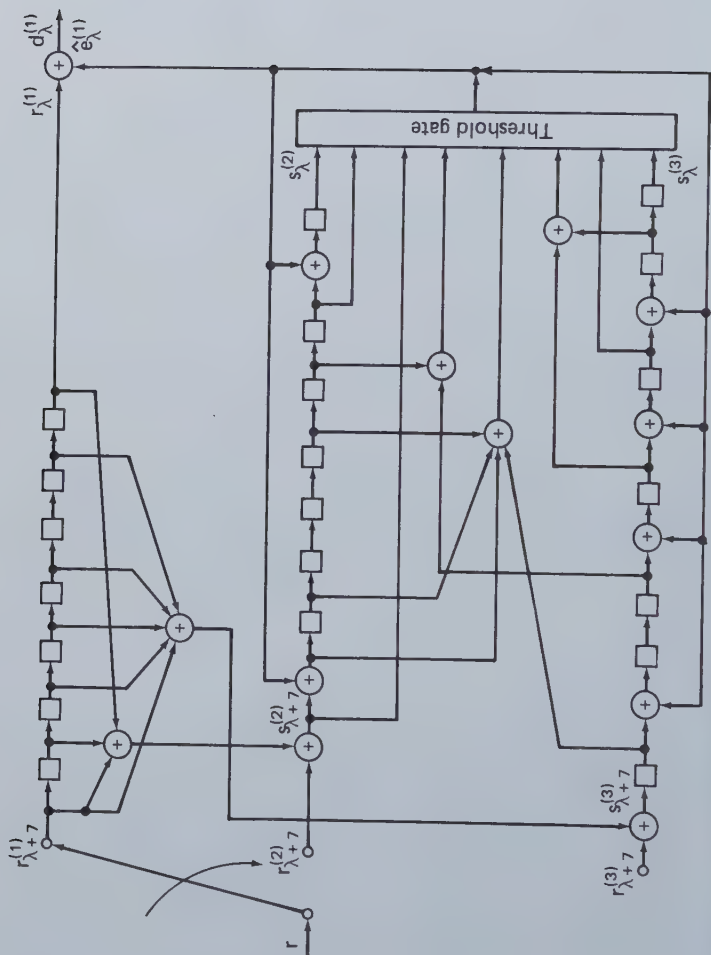
### 11.3.3 Uniform codes

One class of low-rate convolutional codes which can also be decoded by threshold decoding is the class of uniform codes. Massey presented his paper on uniform codes in 1966. Uniform codes are designated as  $(2^m, 1, m)$  codes of minimum distance  $d_{\min} = (m + 2)2^{m-1}$  and constraint length  $n_A = (m + 1)2^m$ . But, since the code rate is limited by  $R = 1/2^m$ , uniform codes comprise a small class. The generator polynomials of uniform codes are expressed as  $g^{(j)}(D) = g_0^{(j)} + g_1^{(j)}D + \dots + g_m^{(j)}D^m$ ,  $j = 1, 2, \dots, 2^m$ . Thus there are a total of  $2^m$  distinct polynomials, where  $g^{(1)}(D) = 1$  is customarily chosen. If a set of  $J = d_{\min} - 1$  orthogonal check sums on  $e_\lambda^{(i)}$  exists, the code is completely orthogonalizable. The syndrome components for time unit  $i$ ,  $\lambda$   $i \leq l-1$ , are formed as shown in (Eq. 11.42) (see facing page), where  $\hat{e}_{\lambda-m}^{(1)}, \hat{e}_{\lambda-m+1}^{(1)}, \dots, \hat{e}_{\lambda-1}^{(1)}$  are called the postdecoding errors. Notice from Eq. (11.42) that parity parallelograms are used instead of parity triangles because the postdecoding errors are included. Threshold decoding for a uniform code will be demonstrated through the following example.

**Example 11.7** Consider the  $(4, 1, 2)$  uniform code for  $m = 2$  whose generator polynomials are  $g^{(1)}(D) = 1$ ,  $g^{(2)}(D) = 1 + D$ ,  $g^{(3)}(D) = 1 + D^2$ , and  $g^{(4)}(D) = 1 + D + D^2$ . The constraint length of this code is  $n_A = (m + 1)2^m = 12$ , and the minimum distance is  $d_{\min} = (m + 2)2^{m-1} = 8$ . Using Eq. (11.42), the syndrome bits can be written as follows:

$$\begin{bmatrix} s_\lambda^{(2)} \\ s_{\lambda+1}^{(2)} \\ s_{\lambda+2}^{(2)} \\ s_\lambda^{(3)} \\ s_{\lambda+1}^{(3)} \\ s_{\lambda+2}^{(3)} \\ s_\lambda^{(4)} \\ s_{\lambda+1}^{(4)} \\ s_{\lambda+2}^{(4)} \end{bmatrix} = \begin{bmatrix} 0 & 1 & 1 \\ & 0 & 1 & 1 \\ & & 0 & 1 & 1 \\ 1 & 0 & 1 \\ & 1 & 0 & 1 \\ & & 1 & 0 & 1 \\ 1 & 1 & 1 \\ & 1 & 1 & 1 \\ & & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} \hat{e}_{\lambda-2}^{(1)} \\ \hat{e}_{\lambda-1}^{(1)} \\ e_\lambda^{(1)} \\ e_{\lambda+1}^{(1)} \\ e_{\lambda+2}^{(1)} \end{bmatrix} + \begin{bmatrix} e_\lambda^{(2)} \\ e_{\lambda+1}^{(2)} \\ e_{\lambda+2}^{(2)} \\ e_\lambda^{(3)} \\ e_{\lambda+1}^{(3)} \\ e_{\lambda+2}^{(3)} \\ e_\lambda^{(4)} \\ e_{\lambda+1}^{(4)} \\ e_{\lambda+2}^{(4)} \end{bmatrix} \quad (11.43)$$

$$\begin{aligned}
& \left[ \begin{array}{c} s_{\lambda}^{(2)} \\ s_{\lambda+1}^2 \\ \vdots \\ s_{\lambda+m}^{(2)} \\ s_{\lambda}^{(3)} \\ s_{\lambda+1}^{(3)} \\ \vdots \\ s_{\lambda+m}^{(3)} \\ s_{\lambda}^{(2m)} \\ s_{\lambda+1}^{(2m)} \\ \vdots \\ s_{\lambda+m}^{(2m)} \end{array} \right] = \left[ \begin{array}{c} g_m^{(2)} \dots g_1^{(2)} g_0^{(2)} \\ g_m^{(2)} \dots g_1^{(2)} g_0^{(2m)} \\ \ddots \vdots \ddots \\ g_m^{(2)} \dots g_1^{(2)} g_0^{(2)} g_{m-1}^{(2)} \dots g_0^{(2)} \\ g_m^{(3)} \dots g_1^{(3)} g_0^{(3)} \\ g_m^{(3)} \dots g_1^{(3)} g_0^{(3)} \\ \ddots \vdots \ddots \\ g_m^{(3)} \dots g_1^{(3)} g_{m-1}^{(3)} \dots g_0^{(3)} \\ \vdots \\ g_m^{(2m)} \dots g_1^{(2m)} g_0^{(2m)} \\ g_m^{(2m)} \dots g_1^{(2m)} g_0^{(2m)} \\ \ddots \vdots \ddots \\ g_m^{(2m)} \dots g_{m-1}^{(2m)} \dots g_0^{(2m)} \end{array} \right] \\
& + \left[ \begin{array}{c} e_{\lambda-m}^{(1)} \\ \vdots \\ e_{\lambda-1}^{(1)} \\ e_{\lambda}^{(1)} \\ e_{\lambda+1}^{(1)} \\ \vdots \\ e_{\lambda+m}^{(1)} \end{array} \right] \\
& \left[ \begin{array}{c} e_{\lambda}^{(2)} \\ e_{\lambda+1}^{(2)} \\ \vdots \\ e_{\lambda+m}^{(2)} \\ e_{\lambda}^{(3)} \\ e_{\lambda+1}^{(3)} \\ \vdots \\ e_{\lambda}^{(2m)} \\ e_{\lambda+1}^{(2m)} \\ \vdots \\ e_{\lambda+m}^{(2m)} \end{array} \right]
\end{aligned}
\tag{11.42}$$



**Figure 11.7** Threshold decoder for a  $(3, 1, 7)$  orthogonalizable code.



Thus, a set of  $J = d_{\min} - 1 = 7$  check sums orthogonal on  $e_{\lambda}^{(1)}$  are obtained as follows:

$$\begin{aligned}
 s_{\lambda}^{(2)} &= \hat{e}_{\lambda-1}^{(1)} + e_{\lambda}^{(1)} + e_{\lambda}^{(2)} \\
 s_{\lambda}^{(3)} &= \hat{e}_{\lambda-2}^{(1)} + e_{\lambda}^{(1)} + e_{\lambda}^{(3)} \\
 s_{\lambda}^{(4)} &= \hat{e}_{\lambda-2}^{(1)} + \hat{e}_{\lambda-1}^{(1)} + e_{\lambda}^{(1)} + e_{\lambda}^{(4)} \\
 s_{\lambda+1}^{(2)} &= e_{\lambda}^{(1)} + e_{\lambda+1}^{(1)} + e_{\lambda+1}^{(2)} \\
 s_{\lambda+2}^{(3)} &= e_{\lambda}^{(1)} + e_{\lambda+2}^{(1)} + e_{\lambda+2}^{(3)} \\
 s_{\lambda+1}^{(3)} + s_{\lambda+1}^{(4)} &= e_{\lambda}^{(1)} + e_{\lambda+1}^{(1)} + e_{\lambda+1}^{(4)} \\
 s_{\lambda+2}^{(2)} + s_{\lambda+2}^{(4)} &= e_{\lambda}^{(1)} + e_{\lambda+2}^{(2)} + e_{\lambda+2}^{(4)}
 \end{aligned} \tag{11.44}$$

under the assumption that the decisions on postdecoding errors are correct, i.e.,  $e_{\lambda-1}^{(1)} = 0$  and  $e_{\lambda-2}^{(1)} = 0$ . Hence this code can be completely orthogonalized. From Eq. (11.44) we also see that the postdecoding errors affect only the first  $2^m - 1 = 3$  check sums. Thus, any combination of these postdecoding errors will change exactly two check sums.

We have so far discussed threshold decoding with feedback. Even though definite decoding is not subject to error propagation, the error-correcting capability of the code is reduced and a certain degree of performance loss would occur. Research shows that feedback decoding with the error propagation effect generally outperforms definite decoding without feedback. One approach to reducing the effect of error propagation is to select codes with automatic recovery properties. Sullivan has studied the automatic recovery properties of several important classes of threshold decodable codes. Threshold decoding has the advantage that implementations at very high data rates (exceeding 50 Mb/s) are feasible. Note that this can be achieved if definite decoding with no feedback to the syndrome register is used with self-orthogonal codes.

### 11.3.4 Performance and comparison

The performance of an  $(n, k, m)$  convolutional code with threshold decoding on a BSC can be estimated from the distance properties of the code. In estimating any information block, the bit-error probability  $P_b(E)$  on a BSC with a feedback decoder can be upper-bounded by

$$P_b(E) \leq \sum_{\lambda=t+1}^{n_A} \binom{n_A}{\lambda} p^{\lambda} (1-p)^{n_A-\lambda} \tag{11.45}$$

because a decoding error can occur only when more than  $t =$

$\lfloor (d_{\min} - 1)/2 \rfloor$  channel errors occur within one constraint length  $n_A = n(m + 1)$ . When the channel-error rate  $p$  is small, i.e.,  $p \ll 1$ , Eq. (11.45) is dominated by its first term such that

$$P_b(E) \approx \binom{n_A}{t+1} p^{t+1} \quad (11.46)$$

If the channel-error rate  $p$  is not too large and if the code has good automatic recovery properties, the effect on bit-error probability becomes small, and Eq. (11.45) or Eq. (11.46) remains valid. The decisions of a feedback decoder are based on the minimum distance over one constraint length, while the decisions of a Viterbi decoder are based on the free distance between sequences. Since the maximum number of  $J$  orthogonal check sums on each information error bit is one less than the minimum distance of the code, it follows that

$$J \leq d_{\min} - 1 \quad (11.47)$$

If  $J = d_{\min} - 1$  orthogonal check sums can be formed on each information-error bit, the code is said to be completely orthogonalizable. If  $d_{\min}$  is odd, Eq. (11.46) reduces to

$$P_b(E) \approx \binom{n_A}{(d_{\min} + 1)/2} p^{(d_{\min} + 1)/2} \quad (11.48)$$

for completely orthogonalizable codes.

For antipodal BPSK signaling over AWGN channel, the error rate is approximated by  $p \approx \frac{1}{2} \exp[-RE_b/N_0]$ . Substitution of this channel-error rate  $p$  into Eq. (11.48) yields the bit-error probability for a feedback threshold decoder:

$$P_b(E) \approx \beta \exp[-(RE_b/2N_0)(d_{\min} + 1)] \quad (11.49)$$

when  $E_b/N_0$  is large. Equation (11.49) represents the performance criterion for threshold decoding if and only if the code is completely orthogonalizable. On the other hand, the bit-error probability of a Viterbi decoder on a BSC was approximated as shown below.

$$P_b(E) \approx \alpha \exp[-(RE_b/2N_0)d_{\text{free}}] \quad (11.50)$$

when  $E_b/N_0$  is large. Thus, comparing Eq. (11.49) for feedback threshold decoding to Eq. (11.50) for Viterbi decoding, we see that their performances are roughly equivalent only when  $d_{\min} + 1 = d_{\text{free}}$ . Therefore, we can conclude that their performances differ only in their distance measures.

Threshold decoders are simpler to implement than either Viterbi

decoders or sequential decoders. Therefore, threshold decoders look more attractive in terms of low-cost application, but long constraint lengths, which are required to achieve high performance, may increase implementation complexity. The time required to perform a computation is different depending on whether probabilistic or threshold decoding is used. Research has shown that threshold decoders are capable of higher operational speeds than probabilistic decoders (Viterbi or sequential decoders). Trellis or tree diagrams contain  $L + m + 1$  levels; thus the decoding delay is equal to one entire frame length  $L + m$ . However, a threshold decoder has a decoding delay of exactly the first constraint length  $m + 1$  with time levels between  $i$  and  $i + m$ . Therefore, for  $L \gg m$ , the decoding delay is substantial in Viterbi or sequential decoders, because probabilistic decoders delay making decisions until the entire received sequence is processed.

Finally, soft-decision threshold decoders have also been developed, though there is a significant increase in implementation complexity. The optimum technique for utilizing soft decision with threshold decoding is the APP decoding scheme discussed by Massey and others. A comparison of an APP decoder and a hard-decision decoder shows that storage for magnitude digits and provision for weighting syndrome digits have been added. Typical simulation results for both block and convolutional codes indicate that an APP decoder provides a performance improvement of about 1.7 dB relative to a hard-decision decoder when eight-level demodulation soft decisions are used. It was also reported that the  $R = 1/2$ ,  $J = 6$  trial-and-error code was implemented so that it would operate at a high data rate (5 Mb/s) utilizing four-level soft decisions with approximate APP decoding. This implementation provides a 2.5-dB coding gain at  $P_b(E) = 10^{-5}$  with PSK signaling.

## 11.4 Burst-Error-Correcting Codes

Since Hagelbarger (1959) introduced the idea of correcting burst errors of recurrent codes, many coding theorists have developed the techniques to provide more efficient codes designed for burst-error correction. The main contributors in this area are Wyner-Ash (1963), Berlekamp (1964), Preparata (1964), Massey (1965), Iwadare (1968), Kohlenberg and Forney (1968), Gallager (1968), Tong (1970), and Ferguson (1971). In this section, one class of convolutional codes for correcting burst errors, i.e., the Iwadare codes, are discussed.

Consider an  $(n, n - 1, m)$  systematic convolutional code designed for burst-error correction. For any positive integer  $\beta$ , an Iwadare code with  $m = (2n - 1)\beta + (2n - 3)$  is capable of correcting a burst of length  $b = \beta n$  relative to a guard space of length  $g = n(m + 1) - 1 =$

$\beta(2n^2 - n) + 2n^2 - 2n - 1$ . The generator polynomials of this code are specified by

$$g_i^{(n)}(D) = D^{a(i)} + D^{b(i)} \quad i = 1, 2, \dots, n-1 \quad (11.51)$$

where  $a(i) = (\beta + 1)(n - i) - 1$  and  $b(i) = (\beta + 1)(2n - i) + i - 3$ . Exponents  $a(i)$  and  $b(i)$  are first determined for finding  $g_i^{(n)}(D)$  as shown in Table 11.3.

TABLE 11.3 Determining  $a(i)$  and  $b(i)$  in Order to Find  $g_i^{(n)}(D)$

| $i$      | $a(i)$               | $b(i)$                   |
|----------|----------------------|--------------------------|
| 1        | $(n-1)\beta + (n-2)$ | $(\beta+1)(2n-1) - 2$    |
| 2        | $(n-2)\beta + (n-3)$ | $(\beta+1)(2n-2) - 1$    |
| 3        | $(n-3)\beta + (n-4)$ | $(\beta+1)(2n-3)$        |
| 4        | $(n-4)\beta + (n-5)$ | $(\beta+1)(2n-4) + 1$    |
| 5        | $(n-5)\beta + (n-6)$ | $(\beta+1)(2n-5) + 2$    |
| $\vdots$ | $\vdots$             | $\vdots$                 |
| $n-3$    | $3\beta + 2$         | $(\beta+1)(n+3) + (n-6)$ |
| $n-2$    | $2\beta + 1$         | $(\beta+1)(n+2) + (n-5)$ |
| $n-1$    | $\beta$              | $(\beta+1)(n+1) + (n-4)$ |

Using Table 11.3, the  $n-1$  generator polynomials are found to be:

$$\begin{aligned}
 g_1^{(n)}(D) &= D^{(n-1)\beta + (n-2)} + D^{(\beta+1)(2n-1)-2} \\
 g_2^{(n)}(D) &= D^{(n-2)\beta + (n-3)} + D^{(\beta+1)(2n-2)-1} \\
 g_3^{(n)}(D) &= D^{(n-3)\beta + (n-4)} + D^{(\beta+1)(2n-3)} \\
 &\vdots \\
 g_{n-2}^{(n)}(D) &= D^{2\beta+1} + D^{(\beta+1)(n+2)+(n-5)} \\
 g_{n-1}^{(n)}(D) &= D^\beta + D^{(\beta+1)(n+1)+(n-4)}
 \end{aligned} \quad (11.52)$$

which implies that their coefficients are

$$\begin{aligned}
 g_{1, (n-1)\beta + (n-2)}^{(n)} &= g_{1, (\beta+1)(2n-1)-2}^{(n)} = 1 \\
 g_{2, (n-2)\beta + (n-3)}^{(n)} &= g_{2, (\beta+1)(2n-2)-1}^{(n)} = 1 \\
 g_{3, (n-3)\beta + (n-4)}^{(n)} &= g_{3, (\beta+1)(2n-3)}^{(n)} = 1 \\
 &\vdots \\
 g_{n-2, 2\beta+1}^{(n)} &= g_{n-2, (\beta+1)(n+2)+(n-5)}^{(n)} = 1 \\
 g_{n-1, \beta}^{(n)} &= g_{n-1, (\beta+1)(n+1)+(n-4)}^{(n)} = 1
 \end{aligned} \quad (11.53)$$

Since  $k = n - 1$  and  $j = n - k = 1$ , the syndrome sequence from Eq. (11.19) becomes

$$s_\lambda = e_\lambda^{(n)} + \sum_{i=1}^{n-1} e_\lambda^{(i)} g_{i,0}^{(n)} + \sum_{i=1}^{n-1} e_{\lambda-1}^{(i)} g_{i,1}^{(n)} + \cdots + \sum_{i=1}^{n-1} e_{\lambda-m}^{(i)} g_{i,m}^{(n)} \quad (11.54)$$

We can also obtain the same syndrome sequence from Eq. (11.14).

Assume that a burst error of length  $b = \beta n$  starts at the first bit of block 0. The error sequence is then of the form

$$\mathbf{e} = (e_0^{(1)}, e_0^{(2)}, \dots, e_0^{(n)}, e_1^{(1)}, e_1^{(2)}, \dots, e_1^{(n)}, \dots, e_{\beta-1}^{(1)}, e_{\beta-1}^{(2)}, \dots, e_{\beta-1}^{(n)}, 0 \ 0 \cdots 0) \quad (11.55)$$

Therefore the syndrome sequence corresponding to  $\mathbf{e}$  can be found by the following procedure. When  $\lambda = 0$ , we have

$$s_0 = e_0^{(n)} + \sum_{i=1}^{n-1} e_0^{(i)} g_{i,0}^{(n)}$$

from Eq. (11.54), but  $s_0 = e_0^{(n)}$  because  $g_{i,0}^{(n)} = 0$ . When  $\lambda = 1$ ,

$$s_1 = e_1^{(n)} + \sum_{i=1}^{n-1} e_1^{(i)} g_{i,0}^{(n)} + \sum_{i=1}^{n-1} e_0^{(i)} \cdot g_{i,1}^{(n)}$$

but  $s_1 = e_1^{(n)}$  because  $g_{i,0}^{(n)} = 0$  and  $g_{i,1}^{(n)} = 0$  judging from the boundary conditions of Eq. (11.53). Repeating the same steps for  $\lambda = 2, 3, \dots$ , as before, we obtain the syndrome bits corresponding to the error bits listed in Table 11.4. From Table 11.4, we see that each information-error bit  $e_\lambda^{(i)}$ ,  $1 \leq i \leq n$ , and  $0 \leq \lambda \leq \beta - 1$ , in the syndrome sequence  $\mathbf{s}$  appears twice, and error bits in each block appear in descending order. In fact,  $e_\lambda^{(i)}$  repeats itself at the syndrome bits  $s_\lambda$  at  $\lambda = (\beta + 1)(n - i) + \lambda - 1$  and at  $\lambda = (\beta + 1)(2n - i) + \lambda + i - 3$ . Therefore the number of bits between these two occurrences of  $e_\lambda^{(i)}$  is  $(\beta + 1)n + (i - 3)$ . We call this the recurrence or repeat distance of  $e_\lambda^{(i)}$ . Accordingly, the two information-error bits in each block have different repeat distances. For example, for  $\lambda = 0$  and  $i = 1$  (see \* in Table 11.4),  $e_0^{(1)}$  occurs twice at the syndrome bit  $s_{(\beta+1)(n-1)+(n-2)} = s_{(\beta+1)(n-1)-1}$  and at  $s_{(\beta+1)(2n-1)-2}$ .

The result is essentially the same if the burst error starts at any arbitrary bit position in the block 0. If the burst of length  $b = \beta n$  starts at the  $j$ th bit of block 0, then the error sequence is

$$\begin{aligned} \mathbf{e} = & (0 \ 0 \cdots 0 \ e_0^{(j)} e_0^{(j+1)} \cdots e_0^{(n)} e_1^{(1)} e_1^{(2)} \cdots e_1^{(n)} \cdots e_{\beta-1}^{(1)} e_{\beta-1}^{(2)} \cdots \\ & \cdots e_{\beta-1}^{(n)} e_\beta^{(1)} e_\beta^{(2)} \cdots e_\beta^{(j-1)} 0 \ 0 \cdots 0) \\ & n(m+1) - 1 \text{ bits guard space} \end{aligned} \quad (11.56)$$



TABLE 11.4 Syndrome versus Error

| $s_\lambda$             | $e_\lambda^{(i)}$     | $s_\lambda$                | $e_\lambda^{(i)}$             |
|-------------------------|-----------------------|----------------------------|-------------------------------|
| $s_0$                   | $e_0^{(n)}$           | $s_{n\beta+(n-3)+1}$       | 0                             |
| $s_1$                   | $e_1^{(n)}$           | $\vdots$                   | $\vdots$ $n + \beta - 1$ bits |
| $\vdots$                | $\vdots$              | $s_{(\beta+1)(n+1)+(n-5)}$ | 0                             |
| $s_{\beta-1}$           | $e_{\beta-1}^{(n)}$   | $s_{(\beta+1)(n+1)+(n-4)}$ | $e_0^{(n-1)}$                 |
| $s_\beta$               | $e_0^{(n-1)}$         | $s_{(\beta+1)(n+1)+(n-3)}$ | $e_1^{(n-1)}$                 |
| $s_{\beta+1}$           | $e_1^{(n-1)}$         | $\vdots$                   | $\vdots$                      |
| $\vdots$                | $\vdots$              | $s_{(\beta+1)(n+2)+(n-6)}$ | $e^{(n-1)}_{\beta-1}$         |
| $s_{2\beta-1}$          | $e_{\beta-1}^{(n-1)}$ | $s_{(\beta+1)(n+2)+(n-5)}$ | $e_0^{(n-2)}$                 |
| $s_{2\beta}$            | 0                     | $s_{(\beta+1)(n+2)+(n-4)}$ | $e_1^{(n-2)}$                 |
| $s_{2\beta+1}$          | $e_0^{(n-2)}$         | $\vdots$                   | $\vdots$                      |
| $\vdots$                | $\vdots$              | $s_{(\beta+1)(n+3)+(n-7)}$ | $e^{(n-2)}_{\beta-1}$         |
| $s_{3\beta}$            | $e_{\beta-1}^{(n-2)}$ | $s_{(\beta+1)(n+3)+(n-6)}$ | $e_0^{(n-3)}$                 |
| $s_{3\beta+1}$          | 0                     | $s_{(\beta+1)(n+3)+(n-5)}$ | $e_1^{(n-3)}$                 |
| $s_{3\beta+2}$          | $e_0^{(n-3)}$         | $\vdots$                   | $\vdots$                      |
| $\vdots$                | $\vdots$              | $s_{(\beta+1)(n+4)+(n-8)}$ | $e^{(n-3)}_{\beta-1}$         |
| $s_{4\beta+1}$          | $e_{\beta-1}^{(n-3)}$ | $\vdots$                   | $\vdots$                      |
| $\vdots$                | $\vdots$              | $*s_{(\beta+1)(2n-1)-2}$   | $e_0^{(1)}$                   |
| $*s_{(n-1)\beta+(n-2)}$ | $e_0^{(1)}$           | $s_{(\beta+1)2n-1-1}$      | $e_1^{(1)}$                   |
| $s_{(n-1)\beta+(n-1)}$  | $e_1^{(1)}$           | $\vdots$                   | $\vdots$                      |
| $\vdots$                | $\vdots$              | $s_{m+\beta-1}$            | $e^{(1)}_{\beta-1}$           |
| $s_{n\beta+(n-3)}$      | $e_{\beta-1}^{(1)}$   | $s_{m+\beta}$              | 0                             |
|                         |                       | $s_{m+\beta+1}$            | 0                             |
|                         |                       | $\vdots$                   | $\vdots$                      |

Thus the syndrome sequence corresponding to  $\mathbf{e}$  is

$$\mathbf{s} = (s_0 \ s_1 \ \cdots \ s_{m+\beta-2} \ s_{m+\beta-1} \ \cdots) \quad (11.57)$$

whose component bits  $s_\lambda$  can be computed as explained through the following example.

**Example 11.8** Consider an  $(n, n-1, m)$  systematic convolutional code. Choosing  $n=3$  and  $\beta=3$ , we have the  $(3, 2, 18)$  Iwadare code because  $m = (2n-1)\beta + (2n-3) = 18$ . In this case,  $b=9$  and  $g=56$ . Using Table 11.3, we have

$$\begin{aligned} a(1) &= (n-1)\beta + (n-2) = 7 & b(1) &= (\beta+1)(2n-1) - 2 = 18 \\ a(2) &= (n-2)\beta + (n-3) = 3 & b(2) &= (\beta+1)(2n-2) - 1 = 15 \end{aligned}$$

Then from Eq. (11.52), the generator polynomials are

$$g_1^{(3)}(D) = D^7 + D^{18} \quad g_2^{(3)}(D) = D^3 + D^{15} \quad (11.58)$$

Suppose that a burst of length  $b=9$  starts with the first bit of block 0. Then the error pattern can be written as

$$\mathbf{e} = (e_0^{(1)} e_0^{(2)} e_0^{(3)}, e_1^{(1)} e_1^{(2)} e_1^{(3)}, e_2^{(1)} e_2^{(2)} e_2^{(3)}, 0 \ 0 \ \cdots \ 0) \quad (11.59)$$

56 bits

The syndrome sequence corresponding to  $\mathbf{e}$  is given by

$$\mathbf{s} = (s_0^{(3)} s_1^{(3)} s_2^{(3)} \dots s_{m+\beta-2}^{(3)} s_{m+\beta-1}^{(3)}) \quad (11.60)$$

Using Table 11.3, syndrome versus error is given as follows:

$$\begin{array}{lll} s_0 = e_0^{(3)} & s_7 = e_0^{(1)} & s_{14} = 0 \\ s_1 = e_1^{(3)} & s_8 = e_1^{(1)} & s_{15} = e_0^{(2)} \\ s_2 = e_2^{(3)} & s_9 = e_2^{(1)} & s_{16} = e_1^{(2)} \\ s_3 = e_0^{(2)} & s_{10} = 0 & s_{17} = e_2^{(2)} \\ s_4 = e_1^{(2)} & s_{11} = 0 & s_{18} = e_0^{(1)} \\ s_5 = e_2^{(2)} & s_{12} = 0 & s_{19} = e_1^{(1)} \\ s_6 = 0 & s_{13} = 0 & s_{20} = e_2^{(1)} \end{array}$$

Therefore the syndrome sequence expressed by Eq. (11.60) becomes

$$\mathbf{s} = (e_0^{(3)} e_1^{(3)} e_2^{(3)} e_0^{(2)} e_1^{(2)} e_2^{(2)} 0 e_0^{(1)} e_1^{(1)} e_2^{(1)} 0 0 0 0 0 e_0^{(2)} e_1^{(2)} e_2^{(2)} e_0^{(1)} e_1^{(1)} e_2^{(1)}) \quad (11.61)$$

We see that each information-error bit appears twice in  $\mathbf{s}$  and that the repeat distance of  $e_{\lambda}^{(i)}$  between the two occurrences is  $9 + i$  for  $i = 1, 2$ . This fact is useful in decoding. In general, the  $n - k$  syndrome sequences, one corresponding to each received sequence, are expressed by Eq. (11.17). However, in this case,  $n - k = 1$ . Therefore there exists only one syndrome sequence  $\mathbf{s} = (s_0, s_1, s_2, \dots, s_{19}, s_{20})$  that has exactly the same form as Eq. (11.61).

We can also utilize Eq. (11.14) to obtain the syndrome sequence. The syndrome polynomial for  $j = 3$  is

$$s(D) = \sum_{i=1}^2 e^{(i)}(D) g_i^{(3)}(D) + e^{(3)}(D) \quad (11.62)$$

Substituting Eq. (11.58) into (11.62) yields

$$\begin{aligned} s(D) &= e^{(1)}(D) g_1^{(3)}(D) + e^{(2)}(D) g_2^{(3)}(D) + e^{(3)}(D) \\ &= (e_0^{(1)} + e_1^{(1)} D + e_2^{(1)} D^2)(D^7 + D^{18}) + (e_0^{(2)} + e_1^{(2)} D + e_2^{(2)} D^2)(D^3 + D^{15}) \\ &\quad + (e_0^{(3)} + e_1^{(3)} D + e_2^{(3)} D^2) \\ &= e_0^{(3)} + e_1^{(3)} D + e_2^{(3)} D^2 + e_0^{(2)} D^3 + e_1^{(2)} D^4 + e_2^{(2)} D^5 + e_0^{(1)} D^7 + e_1^{(1)} D^8 \\ &\quad + e_2^{(1)} D^9 + e_0^{(2)} D^{15} + e_1^{(2)} D^{16} + e_2^{(2)} D^{17} + e_0^{(1)} D^{18} + e_1^{(1)} D^{19} + e_2^{(1)} D^{20} \end{aligned} \quad (11.63)$$

from which we get the same result as in Eq. (11.61). That is,

$$\mathbf{s} = (e_0^{(3)}e_1^{(3)}e_2^{(3)}e_0^{(2)}e_1^{(2)}e_2^{(2)}0e_0^{(1)}e_1^{(1)}e_2^{(1)}00000 \\ e_0^{(2)}e_1^{(2)}e_2^{(2)}e_0^{(1)}e_1^{(1)}e_2^{(1)}) \quad (11.64)$$

If the burst error begins with the third bit of block 0, then the error pattern is

$$\mathbf{e} = (00e_0^{(3)}e_1^{(1)}e_1^{(2)}e_1^{(3)}e_2^{(1)}e_2^{(2)}e_2^{(3)}e_3^{(1)}e_3^{(2)}000\cdots) \quad (11.65)$$

and it then follows that the syndrome polynomial  $s(D)$  will be

$$\begin{aligned} s(D) &= e^{(1)}(D)g_1^{(3)}(D) + e^{(2)}(D)g_2^{(3)}(D) + e^{(3)}(D) \\ &= (e_1^{(1)}D + e_2^{(1)}D^2 + e_3^{(1)}D^3)(D^7 + D^{18}) + (e_1^{(2)}D + e_2^{(2)}D^2 + e_3^{(2)}D^3)(D^3 \\ &\quad + D^{15}) + e_0^{(3)} + e_1^{(3)}D + e_2^{(3)}D^2 \\ &= e_0^{(3)} + e_1^{(3)}D + e_2^{(3)}D^2 + e_1^{(2)}D^4 + e_2^{(2)}D^5 + e_3^{(2)}D^6 + e_1^{(1)}D^8 + e_2^{(1)}D^9 \\ &\quad + e_3^{(1)}D^{10} + e_1^{(2)}D^{16} + e_2^{(2)}D^{17} + e_3^{(2)}D^{18} + e_1^{(1)}D^{19} + e_2^{(1)}D^{20} \\ &\quad + e_3^{(1)}D^{21} \end{aligned} \quad (11.66)$$

Thus the syndrome sequence of this code is

$$\mathbf{s} = (e_0^{(3)}e_1^{(3)}e_2^{(3)}0e_1^{(2)}e_2^{(2)}e_3^{(2)}0e_1^{(1)}e_2^{(1)}e_3^{(1)}00000 \\ e_1^{(2)}e_2^{(2)}e_3^{(2)}e_1^{(1)}e_2^{(1)}e_3^{(1)}) \quad (11.67)$$

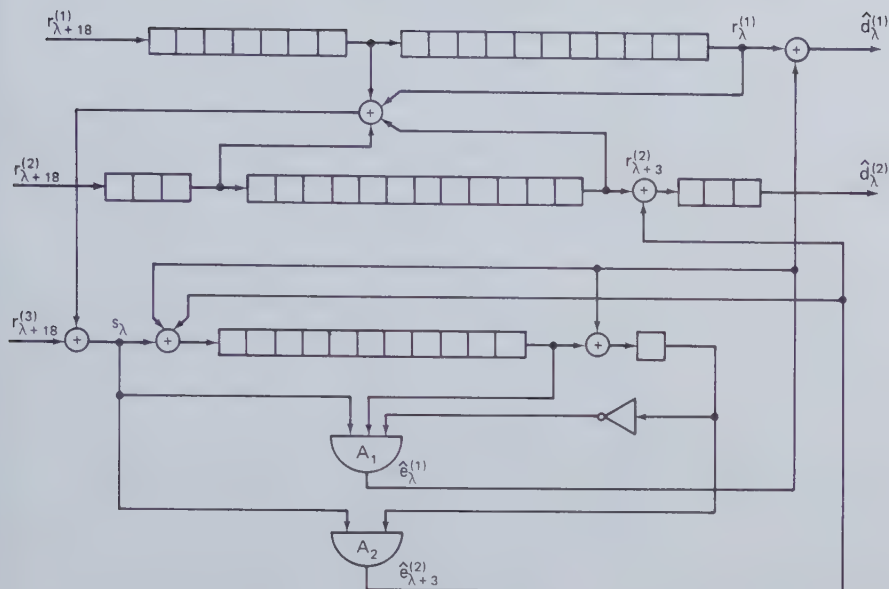
In this case, we also see that the repeat distance of  $e_\lambda^{(i)}$  is still  $9 + i$ . The decoder for the  $(3, 2, 18)$  Iwadare code, which consists of an encoder for syndrome calculation and an error-correction circuit, is shown in Fig. 11.8.

**Example 11.9** Consider an  $(n, n - 1, m)$  systematic code with  $\beta = 2$ . Then for  $n = 5$  we produce a  $(5, 4, 25)$  Iwadare code which is capable of correcting a burst of length  $b = 10$  relative to a guard space of length  $g = 129$  bits. In order to find the generator polynomials,  $a(i)$  and  $b(i)$ ,  $i = 1, 2, 3, 4$ , use the following values:

| $i$ | $a(i)$ | $b(i)$ |
|-----|--------|--------|
| 1   | 11     | 25     |
| 2   | 8      | 23     |
| 3   | 5      | 21     |
| 4   | 2      | 19     |

Thus, using Eq. (11.51), we can find the generator polynomials as follows:

$$\begin{aligned} g_1^{(5)}(D) &= D^{11} + D^{25} \\ g_2^{(5)}(D) &= D^8 + D^{23} \\ g_3^{(5)}(D) &= D^5 + D^{21} \\ g_4^{(5)}(D) &= D^2 + D^{19} \end{aligned} \quad (11.68)$$



**Figure 11.8** Decoding circuit for the (3, 2, 18) Iwadare code with  $\beta = 3$ .

A complete block diagram for this code based on Eq. (11.68) is shown in Fig. 11.9. Finally, the decoder for the (5, 4, 25) Iwadare code with  $\beta = 2$ , consisting of a reencoder, a syndrome generator, and an error-correction unit, is shown in Fig. 11.10.

The interleaving technique is used to obtain long burst-correcting codes from short burst-correcting codes. The principle of interleaving is simply that the outputs of  $\lambda$  separate encoders with constraint length  $n_A$  are multiplexed. The received bits are then demultiplexed and sent to  $\lambda$  separate decoders, where  $\lambda$  is the interleaving degree. A burst of length  $\lambda$  on the channel then looks like a simple error to each of the separate decoders. Hence, if each decoder with interleaving is capable of correcting single errors in a constraint length, then all bursts of length  $\lambda$  or less relative to a guard space of length  $(n_A - 1)\lambda$  will be corrected. Preparata-Massey codes with large values of burst-error-correcting capability can be constructed by interleaving. Iwadare codes can also be interleaved in the usual manner. The interleaved Iwadare codes are inferior to the Preparata-Massey codes with the same rate and burst-error-correcting capability. However, the Iwadare codes lend themselves to a nonuniform sort of interleaving, and this gives them certain advantages over the Preparata-Massey codes. The interleaved Preparata-Massey codes require smaller guard spaces if the interleaving degree is large enough.

Several coding techniques are available for correcting a combina-

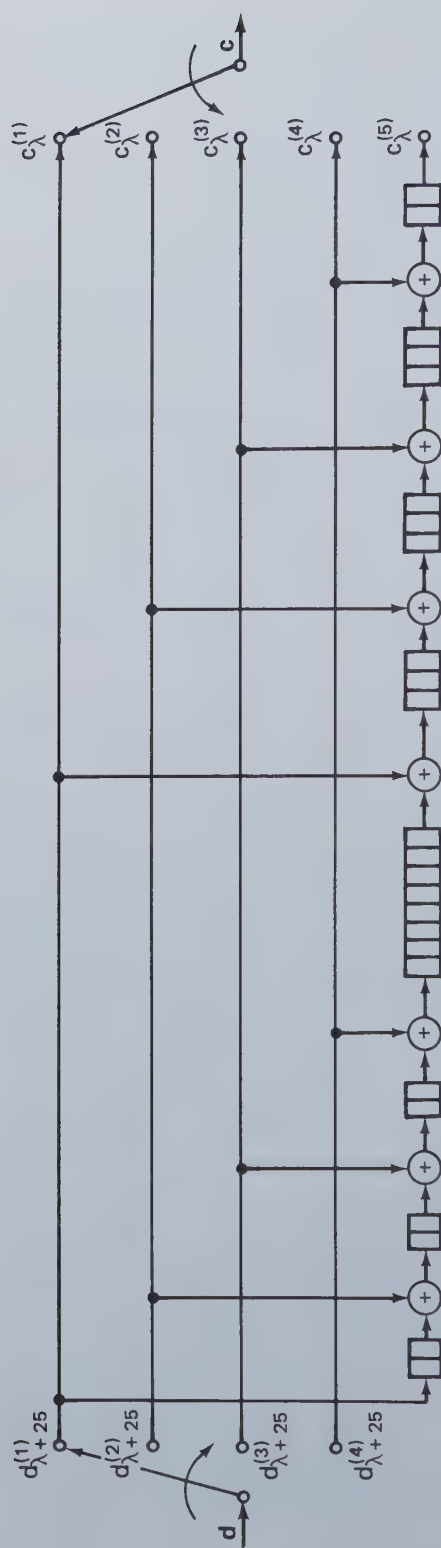


Figure 11.9 Encoder for the (5, 4, 25) Iwadare code with  $\beta = 2$ .



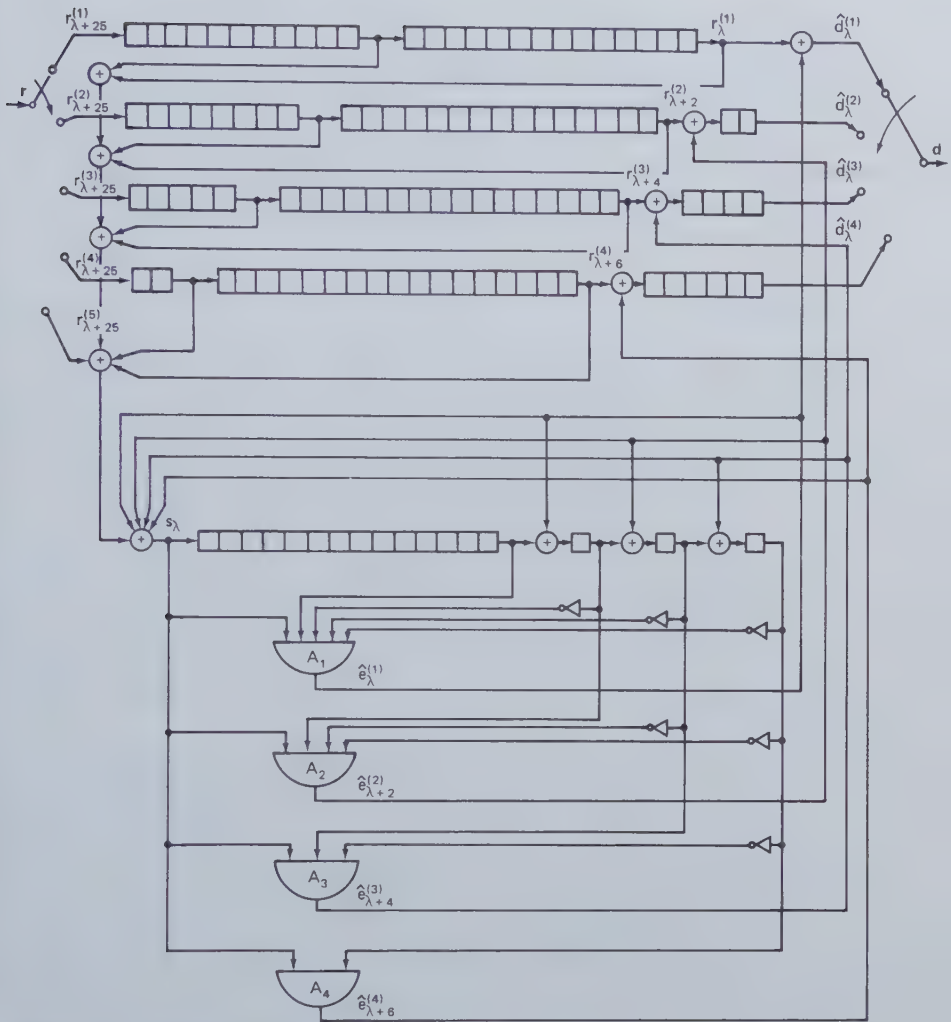


Figure 11.10 Decoder for the (5, 4, 25) Iwadare code with  $\beta = 2$ .

tion of random and burst errors. Therefore, codes can be constructed to correct a specific combination of burst and random errors. Diffuse convolutional codes are very effective on channels where both random and burst errors appear. The diffuse convolutional codes of Kohlenberg and Forney are an example of this type of construction. Ferguson has constructed a family of  $R = \frac{1}{2}$  diffuse threshold decodable codes. Tong has constructed a class of diffuse codes which are self-orthogonal. Iwadare and Massey have found a simple  $R \approx (n - 1)/n$  class of diffuse single-error-correcting codes, which are very attractive for use on telephone channels.

An adaptive decoding algorithm can also be employed to determine which type of error pattern has been received and then to switch to the appropriate correction circuit. Both Gallager's burst-finding system and Tong's burst-trapping system use an adaptive decoder. Kohlenberg and Forney have shown that Gallager's scheme performs better than diffuse coding schemes on dense-burst channels, but performs less well on diffuse-burst channels. Tong's burst-trapping system is sensitive to errors in the guard space.

## PROBLEMS

**11.1** For the  $m = 35$ ,  $(2, 1)$  convolutional code with the generator polynomial  $g_1^{(2)} = \{0, 7, 10, 16, 18, 30, 31, 35\}$

- Write the difference set associated with the generator polynomial  $g_1^{(2)}(D)$ . Is this code self-orthogonal?
- Form the parity triangle using the generator sequence  $\mathbf{g}_1^{(2)}$ .
- Form the syndrome bits by matrix notation in terms of the information-error bits and the parity-error bits.
- Find the parity-check sums orthogonal on the information-error bit  $e_0^{(1)}$ .
- Draw the block diagram of the threshold decoder for this code.

**11.2** For the  $m = 6$ ,  $(2, 1)$  systematic convolutional code with  $g^{(2)}(D) = 1 + D + D^4 + D^6$

- Find the generator matrix  $\mathbf{G}$  and the parity-check matrix  $\mathbf{H}$ .
- Form the parity triangle for the information-error bits.
- Find the syndrome bits for the first constraint length in terms of the channel-error bits.
- Find the set of check sums orthogonal on  $e_0^{(1)}$ .
- Find the orthogonal check sums on  $e_1^{(1)}$ . Is the parity triangle unchanged for  $e_1^{(1)}$ ?
- Devise the complete encoder-decoder block diagram for this code.

**11.3** For the  $m = 13$ ,  $(3, 2)$  self-orthogonal convolutional code with  $g_1^{(3)}(D) = 1 + D^8 + D^9 + D^{12}$  and  $g_2^{(3)}(D) = 1 + D^6 + D^{11} + D^{13}$

- Write the equations for the first constraint length of syndrome bits  $s_\lambda$ ,  $0 \leq \lambda \leq 13$ , by substituting  $k = 2$ ,  $j = n - k = 1$ , and  $\lambda = 0, 1, \dots, 13$ , into Eq. (11.19).
- Find two parity triangles corresponding to  $\mathbf{g}_1^{(3)}$  and  $\mathbf{g}_2^{(3)}$ , respectively.
- Form the two sets of check sums orthogonal on  $e_0^{(1)}$  and  $e_0^{(2)}$ .
- Devise the majority-logic decoder for this self-orthogonal code.

**11.4** For the  $m = 5$ ,  $(2, 1)$  orthogonalizable code with the generator polynomial  $g_1^{(2)}(D) = 1 + D^2 + D^4 + D^5$  and the orthogonalization rule  $\{0^2, 3^2, 4^2, 1^2 5^2\}$

- a Find the number of orthogonal check sums constructed by trial and error and the error-correcting capability  $t$ .
- b What is the minimum distance  $d_{\min}$ ?
- c Form the parity triangle, and express the syndrome sequence in matrix form.
- d Find a set of orthogonal check sums on the information-error bit  $e_0^{(1)}$  using the orthogonalization rule.
- e Design the majority-logic decoder for this code.

**11.5** For the  $m = 4$ ,  $(3, 1)$  orthogonalizable code with the generator polynomials  $g_1^{(2)}(D) = 1 + D$  and  $g_1^{(3)}(D) = 1 + D^2 + D^3 + D^4$  and the orthogonalization rule  $\{0^2, 0^3, 1^2, 2^3, 1^3 3^3, 2^2 4^3\}$

- a Find the syndrome sequence over the first constraint length after forming two parity triangles corresponding to  $g_1^{(2)}$  and  $g_1^{(3)}$ , respectively.
- b Form the orthogonal check sums on  $e_0^{(1)}$  by making use of the orthogonalization rule.
- c Devise the threshold decoder for this code.

**11.6** For the  $m = 3$ ,  $(8, 1)$  uniform code

- a Find the constraint length  $n_A$  and minimum distance  $d_{\min}$ .
- b How many generator polynomials does this code have?
- c Using the generator polynomials  $g^{(j)}(D) = g_0^{(j)} + g_1^{(j)}D + \dots + g_8^{(j)}D^8$ ,  $j = 1, 2, \dots, 8$ , form the parity parallelograms.

**11.7** For the  $\beta = 4$ ,  $(2, 1)$  Iwadare code designed for burst-error correction

- a Determine the memory order  $m$ , the burst length  $b$ , and a guard space  $g$ .
- b Find the generator polynomial  $g_1^{(2)}(D)$ .
- c Find the error pattern  $\mathbf{e}$  and the corresponding syndrome sequence  $\mathbf{s}$ .
- d What is the repeat distance of information-error bit  $e_\lambda^{(1)}$ ?
- e Devise the decoding circuit for this code.



# BIBLIOGRAPHY

- Abramson, N. M.: "Error Correcting Codes from Linear Sequential Networks," presented at the Fourth London Symposium on Information Theory, 1960.
- Bahl, L. R., C. D. Cullum, W. D. Frazer, and F. Jelinek: "An Efficient Algorithm for Computing Free Distance," *IEEE Trans. Inf. Theory*, **IT-18**:437-439 (1972).
- Berlekamp, E. R.: "Note on Recurrent Codes," *IEEE Trans. Inf. Theory*, **IT-10**:257-258 (1964).
- : "On Decoding Binary Bose-Chaudhuri-Hocquenghem Codes," *IEEE Trans. Inf. Theory*, **IT-11**:580-585 (1965).
- : "The Enumeration of Information Symbols in BCH Codes," *Bell System Tech. J.*, **46**:1861-1880 (1967).
- : *Algebraic Coding Theory*, McGraw-Hill, New York, 1968.
- : "The Technology of Error-Correcting Codes," *Proc. IEEE*, **68**:564-593 (1980).
- : "Bit-Serial Reed-Solomon Encoders," *IEEE Trans. Inf. Theory*, **IT-28**(6):869-874 (1982).
- Blahut, R. E.: "Transform Techniques for Error-Control Codes," *IBM J. Res. Dev.*, **23**:299-315 (1970).
- : *Theory and Practice of Error Control Codes*, Addison-Wesley, Reading, Mass., 1983.
- Blake, I. R., and R. C. Mullin: *An Introduction to Algebraic and Combinatorial Coding Theory*, Academic Press, New York, 1976.
- Bose, R. C., and D. K. Ray-Chaudhuri: "On a Class of Error-Correcting Binary Group Codes," *Inf. Control*, **3**:68-79 (1960a).
- , and ———: "Further Results on Error Correcting Binary Group Codes," *Inf. Control*, **3**:279-290 (1960b).
- Boyd, R. W., J. B. Cain, and G. C. Clark: "A Concatenated Coding Approach for High Data Rate Applications," *NTC'77 Conf. Rec.*, **36**:2-1 through 2-7 (1977).
- Burton, H. O.: "A Class of Asymptotically Optimal Burst Correcting Block Codes," presented at the ICC, Boulder, Colo., 1969.
- : "Inversionless Decoding of Binary BCH Codes," *IEEE Trans. Inf. Theory*, **IT-17**:464-466 (1971).
- , D. D. Sullivan, and S. Y. Tong: "Generalized Burst Trapping Codes," *IEEE Trans. Inf. Theory*, **IT-17**:736-742 (1971a).
- , ———, and ———: "Generalized Burst-Trapping Codes," *IEEE Trans. Inf. Theory*, **IT-17**:736-742 (1971b).
- , and E. J. Weldon, Jr.: "Cyclic Product Codes," *IEEE Trans. Inf. Theory*, **IT-11**:433-440 (1965).
- Busgang, J. J.: "Some Properties of Binary Convolutional Code Generators," *IEEE Trans. Inf. Theory*, **IT-11**:90-99 (1965).
- Cain, J. B., and R. W. Boyd: "Convolutional Code Performance with PSK Signaling in Nonstationary Gaussian Noise," *NTC'78 Conf. Rec.*, 2.5.1-2.5.6 (1978).
- Chen, C. L.: "On Majority-Logic Decoding of Finite Geometry Codes," *IEEE Trans. Inf. Theory*, **IT-17**(3):332-336 (1971).



- : "High-Speed Decoding of BCH Codes," *IEEE Trans. Inf. Theory*, **IT-27**(27):254–256 (1981).
- Chevillat, P. R., and D. J. Costello, Jr.: "Distance and Computation in Sequential Decoding," *IEEE Trans. Commun.*, **COM-24**:440–447 (1976).
- Chien, R. T.: "Cyclic Decoding Procedure for the BCH Codes," *IEEE Trans. Inf. Theory*, **IT-10**:357–363 (1964).
- Chow, D. K.: "On Threshold Decoding of Cyclic Codes," *Inf. Control*, **13**:471–483 (1968).
- Clark, C. C., and J. Bibb Cain: *Error-Correcting Coding for Digital Communications*, Plenum Press, New York, 1981.
- Costello, D. J., Jr.: "Free Distance Bounds for Convolutional Codes," *IEEE Trans. Inf. Theory*, **IT-20**:356–365 (1974).
- Delsarte, P.: "A Geometric Approach to a Class of Cyclic Codes," *J. Combinatorial Theory*, **6**:340–358 (1969).
- , J. M. Goethals, and J. MacWilliams: "On GRM and Related Codes," *Inf. Control*, **16**:403–442 (1970).
- Elias, P.: "Coding for Noise Channels," *IRE Conv. Rec.*, **Part 4**:37–47 (1955).
- Evans, T. A., and H. B. Mann: "On Simple Difference Sets," *Sankhya*, **11**:464–481 (1955).
- Fano, R. M.: "A Heuristic Discussion of Probabilistic Decoding," *IEEE Trans. Inf. Theory*, **IT-19**:64–73 (1963).
- Ferguson, M. J.: "Diffuse Threshold Decodable Rate  $\frac{1}{2}$  Convolutional Codes," *IEEE Trans. Inf. Theory*, **IT-17**:171–180 (1971).
- Fire, P.: "A Class of Multiple-Error-Correcting Binary Codes for Nonindependent Errors," Sylvania Rep. No. RSL-E-2, Sylvania Electronic Defense Laboratory, Reconnaissance Systems Division, Mountain View, Calif., 1959.
- Forney, G. D.: "On Decoding BCH Codes," *IEEE Trans. Inf. Theory*, **IT-11**:577–580 (1965).
- : "Review of Random Tree Codes, Appendix A: Study of Coding Systems Design for Advanced Solar Missions," NASA Contract NAS2-3637, Codex Corporation, 1967.
- : "Use of a Sequential Decoder to Analyze Convolutional Code Structure," *IEEE Trans. Inf. Theory*, **IT-16**:793–795 (1970).
- Forney, G. D., Jr.: *Concatenated Codes*, MIT Press, Cambridge, Mass., 1966a.
- : "Generalized Minimum Distance Decoding," *IEEE Trans. Inf. Theory*, **IT-12**:125–131 (1966b).
- : "Convolutional Codes I: Algebraic Structure," *IEEE Trans. Inf. Theory*, **IT-16**:720–738 (1970).
- : "Maximum Likelihood Sequence Estimation of Digital Sequences in the Presence of Intersymbol Interference," *IEEE Trans. Inf. Theory*, **IT-18**:363–378 (1972).
- : "The Viterbi Algorithm," *Proc. IEEE*, **61**:268–278 (1973).
- : "Convolutional Codes II: Maximum Likelihood Decoding," *Inf. Control*, **25**:222–266 (1974).
- , and E. K. Bower: "A High-Speed Sequential Decoder, Prototype Design and Test," *IEEE Trans. Commun. Technol.*, **COM-29**:821–835 (1971).
- Fraleigh, J. B.: *A First Course in Abstract Algebra*, 2d ed., Addison-Wesley, Reading, Mass., 1976.
- Gallager, R. G.: *Information Theory and Reliable Communication*, Wiley, New York, 1968.
- : "Tree Encoding for Symmetric Sources with a Distortion Measure," *IEEE Trans. Inf. Theory*, **IT-20**:65–76 (1974).
- Geist, J.: "An Empirical Comparison of Two Sequential Decoding Algorithms," *IEEE Trans. Commun. Technol.*, **COM-19**:415–419 (1971).
- Gilbert, E. N.: "A Comparison of Signalling Alphabets," *Bell System Tech. J.*, **31**:504–522 (1952).
- Gilhousen, K. S., J. A. Heller, I. M. Jacobs, and A. J. Viterbi: "Coding Study for High Data Rate Telemetry Links," Linkabit Corp., NASA CR-114278, Contract NAS2-6024, 1971.
- Goethals, J. M., and P. Delsarte: "On a Class of Majority-Logic Decodable Codes," *IEEE Trans. Inf. Theory*, **IT-14**:182–189 (1968).
- Golay, M. J. E.: "Notes on Digital Coding," *Proc. IRE*, **37**:657 (1949).

- Gore, W. C.: "Generalized Threshold Decoding and the Reed-Solomon Codes," *IEEE Trans. Inf. Theory*, **IT-15**:78-81 (1969a).
- : "The Equivalence of L-Step Orthogonalization and a Reed Decoding Procedure," *IEEE Trans. Inf. Theory*, **IT-15**:184-186 (1969b).
- Gorenstein, D. C., and W. W. Peterson: "Polynomial Codes over Certain Finite Fields," *J. Soc. Ind. Appl. Math.*, **8**:300-304 (1960).
- Gorenstein, D. C., and Zierler, N.: "A Class of Cyclic Linear Error-Correcting Codes," *J. Soc. Ind. Appl. Math.*, **9**:207-214 (1961).
- Gorog, E.: "Some New Classes of Cyclic Codes Used for Burst Error Correction," *IBM J. Res. Dev.*, **7**:102-111 (1963).
- Graham, F. L., and F. J. MacWilliams: "On the Number of Parity Checks in Difference-Sets Cyclic Codes," *Bell Systems Tech. J.*, **45**:1057-1070 (1966).
- Green, J. H., Jr., and R. L. San Soucie: "An Error-Correcting Encoder and Decoder of High Efficiency," *Proc. IRE*, **46**:1741-1744 (1958).
- Gross, A. J.: "Binary Group Codes which Correct in Bursts of Three or Less for Odd Redundancy," *IEEE Trans. Inf. Theory*, **IT-8**:356-359 (1962).
- : "Augmented BCH Codes which Correct Single Bursts of Errors," *IEEE Trans. Inf. Theory*, **IT-9**:121 (1963).
- Haccoun, D.: "Branching Processes and Sequential Decoding," *1977 IEEE Int. Symp. Inf. Theory*, Ithaca, N.Y., 1977, p. 68.
- , and M. Ferguson: "Generalized Stack Algorithms for Decoding Convolutional Codes," *IEEE Trans. Inf. Theory*, **IT-21**:638-651 (1975).
- Hagelbarger, D. W.: "Recurrent Codes: Easily Mechanized, Burst-Correcting, Binary Codes," *Bell System Tech. J.*, **38**:969-984 (1959).
- Hamming, R. W.: "Error Detecting and Error Correcting Codes," *Bell System Tech. J.*, **29**:147-160 (1950).
- Hartmann, C. R. P., J. B. Ducey, and L. D. Rudolph: "On the Structure of Generalized Finite Geometry Codes," *IEEE Trans. Inf. Theory*, **IT-20**(2):240-252 (1974).
- Heller, J. A.: "Sequential Decoding; Short Constraint Length Convolutional Codes," *JPL, Space Program Summary* 37-54(3):171-174 (1968).
- , and I. M. Jacobs: "Viterbi Decoding for Satellite and Space Communications," *IEEE Trans. Commun. Technol.*, **COM-19**:835-848 (1971).
- Hemmati, F., and D. J. Costello, Jr.: "Truncation Error Probability in Viterbi Decoding," *IEEE Trans. Commun.*, **COM-25**:530-532 (1977).
- , and ———: "Asymptotically Catastrophic Convolutional Codes," *IEEE Trans. Inf. Theory*, **IT-26**:298-304 (1980).
- Hocquenghem, A.: "Codes Correcteurs d'erreurs," *Chiffers*, **2**:144-156 (1959).
- Hsu, I. K., I. S. Reed, T. K. Truong, K. Wang, C. S. Yeh, and L. J. Deutch: "The VLSI Implementation of a Reed-Solomon Encoder Using Berlekamp's Bit-Serial Multiplier Algorithm," *IEEE Trans. Computer*, **C-33**(10):906-911 (1984).
- Iwaware, Y.: "On Type B-1 Burst-Error-Correcting Convolutional Codes," *IEEE Trans. Inf. Theory*, **IT-14**:557-583 (1968).
- : "A Class of High-Speed Decodable Burst-Error-Correcting Codes," *IEEE Trans. Inf. Theory*, **IT-18**:817-821 (1972).
- Jacobs, I. M., and E. R. Berlekamp: "A Lower Bound to the Distribution of Computation for Sequential Decoding," *IEEE Trans. Inf. Theory*, **IT-13**:167-174 (1967).
- Jelinek, F.: "A Fast Sequential Decoding Algorithm Using a Stack," *IBM J. Res. Dev.*, **13**:675-685 (1969).
- Johnnsson, R.: "Some Rate  $\frac{1}{2}$  and  $\frac{1}{4}$  Binary Convolutional Codes with an Optimum Distance Profile," *IEEE Trans. Inf. Theory*, **IT-23**:281-283 (1977).
- Kasami, T.: "A Decoding Procedure for Multiple Error-Correcting Cyclic Codes," *IEEE Trans. Inf. Theory*, **IT-19**:134-138 (1964).
- , and S. Lin: "On Majority-Logic Decoding for Duals of Primitive Polynomial Codes," *IEEE Trans. Inf. Theory*, **IT-17**(3):322-331 (1971).
- , ———, and W. W. Peterson: "Some Results on Weight Distributions of BCH Codes," *IEEE Trans. Inf. Theory*, **IT-12**(2):274 (1966).
- , and S. Matoba: "Some Efficient Shortened Cyclic Codes for Burst-Error-Correction," *IEEE Trans. Inf. Theory*, **IT-10**:252-253 (1964).
- Klieber, E. J.: "Some Difference Triangles for Constructing Self Orthogonal Codes,"

- IEEE Trans. Inf. Theory*, **IT-16**:237–238 (1970).
- Kohlenburg, A.: "Random and Burst-Error-Correction," *1st IEEE Annual Communications Convention*, Boulder, Colo., pp. 7–9. 1965.
- , and G. D. Forney, Jr.: "Convolutional Coding for Channels with Memory," *IEEE Trans. Inf. Theory*, **IT-14**:618–626 (1968).
- Larsen, K. J.: "Short Convolutional Codes with Maximal Free Distances for Rate  $\frac{1}{2}$ ,  $\frac{1}{3}$ , and  $\frac{1}{4}$ ," *IEEE Trans. Inf. Theory*, **IT-19**:371–372 (1973).
- , and R. J. McEliece: "An Upper Bound on the Free Distance of a Tree Code," *Jet Propulsion Laboratory, SPS 37–62*, vol. 3, 63–64 (1970).
- Layland, J. W., and W. A. Lusbaugh: "A Flexible High Speed Sequential Decoder for Deep Space Channels," *IEEE Trans. Comm. Tech.*, **COM-19**:813–820 (1971).
- Lin, S.: "Multifold Euclidean Geometry Codes," *IEEE Trans. Inf. Theory*, **IT-19**(4):537–548 (1973).
- , and D. J. Costello, Jr.: *Error Control Coding*, Prentice-Hall, Englewood Cliffs, N.J., 1983.
- , and H. Lyne: "Some Results of Binary Convolutional Code Generators," *IEEE Trans. Inf. Theory*, **IT-13**:133–139 (1967).
- , and G. Markowsky: "On a Class of One-Step-Majority-Logic Decodable Cyclic Codes," *IBM J. Res. Dev.* (1980).
- , and K. P. Yiu: "An Improvement to Multifold Euclidean Geometry Codes," *Inf. Control*, **28**(3), July 1975.
- Ma, H. H.: "The Multiple Stack Algorithm Implemented on a Zilog Z-80 Microcomputer," *IEEE Trans. Commun.*, **COM-28**:1876–1887 (1980).
- MacWilliams, F. J.: "Combinatorial Problems of Elementary Group Theory," Ph. D. Thesis, Dept. of Math., Harvard University, May, 1962.
- , "A Theorem on the Distribution of Weight in a Systematic Code," *Bell System Tech. J.*, **29**:144–160 (1963).
- , "Permutation Decoding of Systematic Codes," *Bell System Tech. J.*, **43**(Part I):485–505 (1964).
- , and H. B. Mann: "On the  $p$ -rank of the Design Matrix of a Difference Set," *Inf. Control*, **12**:474–499 (1968).
- , and J. J. Sloan: *The Theory of Error-Correcting Codes*, North-Holland, Amsterdam, 1977.
- Massey, J. L.: *Threshold Decoding*, MIT Press, Cambridge, Mass., 1963.
- , "Implementation of Burst-Correcting-Convolutional Codes," *IEEE Trans.*, **IT-11**:416–422 (1965a).
- , "Step-by-Step Decoding of the Bose-Chaudhuri-Hocquenghem Codes," *IEEE Trans. Inf. Theory*, **IT-11**:580–585 (1965b).
- , "Uniform Codes," *IEEE Trans. Inf. Theory*, **IT-12**:132–134 (1966).
- , "Some Algebraic and Distance Properties of Convolutional Codes," in H. B. Mann (Ed.), *Error Correcting Codes*, Wiley, New York, 1968.
- , "Advances in Threshold Decoding," in A. V. Balarkrishnan (Ed.), *Advances in Communications Systems*, vol. 2, Academic Press, New York, 1968.
- , "Shift-Register Synthesis and BCH Decoding," *IEEE Trans. Inf. Theory*, **IT-15**:122–127 (1969).
- , "Variable-Length Codes and the Fano Metric," *IEEE Trans. Inf. Theory*, **IT-18**:196–198 (1972).
- , and D. J. Costello, Jr.: "Nonsystematic Convolutional Codes for Sequential Decoding in Space Applications," *IEEE Trans. Commun. Technol.*, **Com-19**:806–813 (1971).
- , ———, and J. Justesen: "Polynomial Weights and Code Construction," *IEEE Trans. Inf. Theory*, **IT-19**:101–110 (1973).
- , and R. W. Liu: "Application of Lyapunov's Direct Method to the Error Propagation Effect in Convolutional Codes," *IEEE Trans. Inf. Theory*, **IT-10**:248–250 (1964).
- , M. K. Sain, and J. M. Geist: "Certain Infinite Markov Chains and Sequential Decoding," *Discrete Math.*, **3**:163–175 (1972).
- Mattson, H. F., and G. Solomon: "A New Treatment of Bose-Chaudhuri Codes," *J. Soc. Ind. Appl. Math.*, **9**:654–669 (1961).



- Mceliece, R. J.: *The Theory of Information and Coding*, Addison-Wesley, Reading, Mass., 1977.
- Meggit, J. E.: "Error Correcting Codes for Correcting Bursts of Errors," *IBM J. Res. Dev.*, **4**:329-334 (1960).
- : "Error Correcting Codes and Their Implementation," *IRE Trans. Inf. Theory*, **IT-7**:230-244 (1961a).
- : "Error Correcting Codes for Correcting Bursts of Errors," *Trans. AIEE*, **80**:708-711 (1961b).
- Melas, C. M.: "A Cyclic Code for Double Error Correction," *IBM J. Res. Dev.*, **4**:364-366 (1960a).
- : "A New Group of Codes for Correction of Dependent Errors in Data Transmission," *IBM J. Res. Dev.*, **4**:58-64 (1960b).
- , and E. Gorog: "A Note on Extending Certain Codes to Correct Error Bursts in Longer Messages," *IBM J. Res. & Dev.*, **7**:151-152 (1961).
- Mitchell, M. E.: "Error-Trap Decoding of Cyclic Codes," G.E. Rep. No. 62 MCD3, General Electric Military Communication Dept., Oklahoma City, Okla., 1962.
- Muller, D. E.: "Applications of Boolean Algebra to Switching Circuit Design and to Error Detection," *IRE Trans.*, **EC-3**:6-12 (1954).
- Odenwalder, J. P.: "Concatenated Reed-Solomon/Viterbi Channel Coding for Advanced Planetary Missions: Analysis, Simulations and Test," Linkabit Corporation Final Report on Contract No. 953866 (1974).
- , and A. J. Viterbi: "Overview of Existing and Projected Uses of Coding in Military Satellite Communications," *Proc. Natl. Telecommun. Conf.*, 1977, p. 36.
- Omura, J. K.: "On the Viterbi Decoding Algorithm," *IEEE Trans. Inf. Theory*, **IT-15**(5):177-179 (1969).
- Paaske, E.: "Short Binary Convolutional Codes with Maximal Free Distance for Rate  $\frac{2}{3}$  and  $\frac{3}{4}$ ," *IEEE Trans. Inf. Theory*, **IT-20**:683-689 (1974).
- Perry, K. M., and J. M. Wozencraft: "Seco: A Self-regulating Error-Correcting Coder-Decoder," *IRE Trans.*, **IT-8**:128-135 (1962).
- Peterson, W. W.: "Encoding and Error-Correction Procedures for the Bose-Chaudhuri Codes," *IRE Trans. Inf. Theory*, **IT-6**:459-470 (1960a).
- : "Binary Coding for Error Control," *Proc. Natl. Electronics Conf.*, **16**:15-21 (1960b).
- : *Error-Correcting Codes*, MIT Press, Cambridge, Mass., 1961.
- : "On the Weight Structure and Symmetry of BCH Codes," *J. Inst. Elect. Commun. Eng. Jap.*, **50**:1183-1190 (1967).
- , and D. T. Brown: "Cyclic Codes for Error Detection," *Proc. IRE*, **49**:228-235 (1961).
- , and E. J. Weldon, Jr.: *Error-Correcting Codes*, 2d ed., MIT Press, Cambridge, Mass., 1972.
- Posner, W.: "Simultaneous Error-Correction and Burst-Error Detection Binary Linear Cyclic Codes," *J. Soc. Ind. Appl. Math.*, **13**:1087-1095 (1965).
- Prange, E.: "The Use of Information Sets in Decoding Cyclic Codes," *IEEE Trans. Inf. Theory*, **IT-8**:85-89 (1962).
- Preparata, F. P.: "Systematic Construction of Optimal Linear Recurrent Codes for Burst-Error-Correction," *Calcolo*, **2**:1-7 (1964).
- : "A New Look at the Golay (23, 12) Code," *IEEE Trans. Inf. Theory*, **IT-16**:510-511 (1970).
- Proakis, J. G.: "Performance Capabilities of the Viterbi Algorithm for Combating Intersymbol Interference on Fading Multipath Channels," *Proc. 1977 NATO Adv. Stud. Inst.*, Darlington, U.K.
- : *Digital Communications*, McGraw-Hill, New York, 1983.
- Reddy, S. M., and J. P. Robinson: "Random Error and Burst Correction by Iterated Codes," *IEEE Trans. Inf. Theory*, **IT-18**:182-185 (1972).
- Reed, I. S.: "A Class of Multiple-Error-Correcting Codes and the Decoding Scheme," *IRE Trans.*, **IT-4**:38-49 (1954).
- , and G. Solomon: "Polynomial Codes over Certain Finite Fields," *J. Soc. Ind. Appl. Math.*, **8**:300-304 (1960).
- , T. K. Truong, and L. R. Welch: "The Fast Decoding of Reed-Solomon Codes Us-

- ing Fermat Transforms," *IEEE Trans. Inf. Theory*, **IT-24**:497–499 (1978).
- Reiger, S. H.: "Codes for the Correction of Cluster Errors," *IRE Trans. Inf. Theory*, **IT-16**:16–21 (1960).
- Rhee, M. Y.: *Coding Theory*, Hee-Jung Dang, Seoul, Korea, 1984 (in Korean).
- Robinson, J. P.: "Error Propagation and Definite Decoding of Convolutional Codes," *IEEE Trans. Inf. Theory*, **IT-14**:121–128 (1968).
- : "Punctured Uniform Codes," *IEEE Trans. Inf. Theory*, **IT-15**:149–152 (1969).
- , and A. J. Bernstein: "A Class of Binary Recurrent Codes with Limited Error Propagation," *IEEE Trans. Inf. Theory*, **IT-13**:106–113 (1967).
- Rudolph, L. D.: "Geometric Configuration and Majority Logic Decodable Codes," MEE Thesis, University of Oklahoma, Norman, 1964.
- : "A Class of Majority Logic Decodable Codes," *IEEE Trans. Inf. Theory*, **IT-13**:305–307 (1967).
- : "Threshold Decoding of Cyclic Codes," *IEEE Trans. Inf. Theory*, **IT-15**:414–418 (1969).
- : "Generalized Threshold Decoding of Convolutional Codes," *IEEE Trans. Inf. Theory*, **IT-16**:739–745 (1970).
- , and C. R. P. Hartmann: "Decoding by Sequential Code Reduction," *IEEE Trans. Inf. Theory*, **IT-19**(4):549–555 (1973).
- Savage, J. E.: "Sequential Decoding: The Computation Problem," *Bell System Tech. J.*, **45**:149–175 (1966a).
- : "The Distribution of the Sequential Decoding Computation Time," *IEEE Trans. Inf. Theory*, **IT-12**:143–147 (1966b).
- : "Progress in Sequential Decoding," in A. V. Balakrishnan (Ed.), *Advances in Communication Systems*, Academic Press, New York, 1968.
- Schalkwijk, J. P. M., A. J. Vinck, and K. A. Post: "Syndrome Decoding of Binary Rate  $k/n$  Convolutional Codes," *IEEE Trans. Inf. Theory*, **IT-24**:553–562 (1978).
- Shannon, C. E.: "A Mathematical Theory of Communication," *Bell System Tech. J.*, **27**:379–423 (Part I), 623–656 (Part II) (1948).
- Sidelnikov, V. M.: "Weight Spectrum of Binary Bose-Chaudhuri-Hocquenghem Code," *Probl. Inf. Transm.*, **7**(1):11–17 (1971).
- Singer, J.: "A Theorem in Finite Projective Geometry and Some Applications to Number Theory," *AMS Trans.*, **43**:377–385 (1938).
- Smith, K. J. C.: "Majority Decodable Codes Derived from Finite Geometries," Institute of Statistics Mimeo Series No. 561, University of North Carolina, Chapel Hill, 1967.
- Spence, E.: "A New Class of Hadamard Matrices," *Glasgow J.*, **8**:59–62 (1967).
- Stenbit, J. P.: "Table of Generators for BCH Codes," *IEEE Trans. Inf. Theory*, **IT-10**:390–391 (1964).
- Stern, T. E., and B. Friedland: "Application of Modular Sequential Circuits to Single-Error-Correcting  $P$ -nary Codes," *IRE Trans. Inf. Theory*, **IT-5**:114–123 (1959).
- Stone, J. J.: "Multiple Burst Error Correction," *Inf. Control*, **4**:324–331 (1961).
- Strang, G.: *Linear Algebra and Its Applications*, 2d ed., Academic Press, New York, 1980.
- Sullivan, D. D.: "Error-Propagation Properties of Uniform Codes," *IEEE Trans. Inf. Theory*, **IT-15**:152–161 (1969).
- : "A Generalization of Gallager's Adaptive Error Control Scheme," *IEEE Trans. Inf. Theory*, **IT-17**:727–735 (1971).
- Tong, S. Y.: "Burst-Trapping Techniques for a Compound Channel," *IEEE Trans. Inf. Theory*, **IT-15**:710–715 (1969).
- : "Systematic Construction of Self-orthogonal Diffuse Codes," *IEEE Trans. Inf. Theory*, **IT-16**:594–604 (1970).
- Townsend, R. L., and E. J. Weldon, Jr.: "Self-orthogonal Quasi-cyclic Codes," *IEEE Trans. Inf. Theory*, **IT-13**:183–195 (1967).
- Vinck, A. J., A. J. P. de Paape, and J. P. M. Schalkwijk: "A Class of Binary Rate  $\frac{1}{2}$  Convolutional Codes That Allows an Improved Stack Decoder," *IEEE Trans. Inf. Theory*, **IT-26**:389–392 (1980).
- Viterbi, A. J.: "Error Bounds for Convolutional Codes and Asymptotically Optimum Decoding Algorithm," *IEEE Trans. Inf. Theory*, **IT-13**:260–269 (1967).
- : "Convolutional Codes and Their Performance in Communication Systems,"



- IEEE Trans. Commun. Technol.*, **COM-19**:751-772 (1971).
- , and I. M. Jacobs: "Advances in Coding and Modulation for Noncoherent Channels Affected by Fading, Partial Band, and Multiple Access Interference," *Advances in Communications Systems*, Vol. 4, Academic Press, New York, 1975, pp. 279-308.
- , and J. K. Omura: *Principles of Digital Communication and Coding*, McGraw-Hill, New York, 1981.
- Weldon, E. J., Jr.: "Difference-Set Cyclic Codes," *Bell Systems Tech. J.*, **45**:1045-1055 (1966).
- : "Euclidean Geometry Cyclic Codes," *Proc. Symp. Combinatorial Mathematics at the University of North Carolina, Chapel Hill*, 1967.
- : "New Generations of the Reed-Muller Codes—Part: Non Primitive Codes," *IEEE Trans. Inf. Theory*, **IT-14**:199-205 (1968a).
- : "Some Results on Majority-Logic Decoding," in H. B. Mann (Ed.), *Error-Correcting Codes*, Wiley, New York, 1968, chap. 8.
- Weng, L. J.: "Soft and Hard Decoding Performance Comparisons for BCH Codes," *Proc. Int. Conf. Commun.*, 1979, pp. 25.5.1-25.5.5.
- Wozencraft, J. M., and R. S. Kennedy: "Modulation and Demodulation for Probabilistic Coding," *IEEE Trans. Inf. Theory*, **IT-12**:291-297 (1966).
- , and B. Reiffen: *Sequential Decoding*, MIT Press, Cambridge, Mass., 1961.
- Wyner, A. D., and R. B. Ash: "Analysis of Recurrent Codes," *IEEE Trans.*, **IT-9**:143-156 (1963).
- Zierler, N.: "On a Variation of the First Order Reed-Muller Codes," Lincoln Laboratory Report 34-80, Lincoln Labs., Mass. Inst. of Technol., Cambridge, Mass., 1958.
- : "A Complete Theory for Generalized BCH Codes," in H. B. Mann (Ed.), *Error Correcting Code*, Wiley, New York, 1968.
- : "A Complete Theory for Generalized BCH Codes," in H. B. Mann (Ed.), *Proc. 1968 Symp. on Error Correcting Codes*, Wiley, New York, 1969.
- Zigangirov, K. Sh.: "Some Sequential Decoding Procedures," *Problemu Peredachi Inf.*, **2**:13-25 (1966).



# List of Symbols

|                             |                                                                                                                                                               |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $A(z)$                      | Weight enumerator of a given block code $C$                                                                                                                   |
| $b$                         | Error burst length, or the burst-error-correcting capability                                                                                                  |
| $B$                         | Bias term in Fano metric which is equal to the code rate $R$                                                                                                  |
| $B(x)$                      | Error burst pattern                                                                                                                                           |
| $B(z)$                      | Weight enumerator of a dual code $C_D$                                                                                                                        |
| $c(x)$                      | Code polynomial                                                                                                                                               |
| $c^{(1)}(x)$                | Remainder polynomial resulting from dividing $xc(x)$ by $x^n + 1$ , or the $n$ -tuple code word obtained by cyclically shifting $c(x)$ one place to the right |
| $\mathbf{c}$                | Code word or code sequence                                                                                                                                    |
| $\mathbf{c}'$               | Shortened code word                                                                                                                                           |
| $\mathbf{c}^{(1)}$          | Code word cyclically shifted one place to the right                                                                                                           |
| $C$                         | $(n, k)$ cyclic code                                                                                                                                          |
| $C$                         | Number of computations required to decode the first $L$ information symbols                                                                                   |
| $C'$                        | $(n - i, k - i)$ shortened code                                                                                                                               |
| $C_D$                       | $(n, n - k)$ dual code                                                                                                                                        |
| $d(x)$                      | Information polynomial                                                                                                                                        |
| $d(\mathbf{r}, \mathbf{c})$ | Hamming distance between $\mathbf{r}$ and $\mathbf{c}$                                                                                                        |
| $d_{\text{EG}}$             | Minimum distance of EG codes                                                                                                                                  |
| $d_{\text{free}}$           | Free distance                                                                                                                                                 |
| $d_{\text{min}}$            | Minimum distance of the code                                                                                                                                  |
| $d_0$                       | Designed distance of a BCH code                                                                                                                               |
| $\mathbf{d}$                | Information data or information sequence                                                                                                                      |
| $e$                         | Period (or order) of a primitive polynomial $p(x)$ of degree $m$                                                                                              |
| $e(x)$                      | Error polynomial                                                                                                                                              |
| $e_I(x)$                    | Error confined to the information section of $r(x)$                                                                                                           |

|                            |                                                                                                       |
|----------------------------|-------------------------------------------------------------------------------------------------------|
| $e_{j_m}$                  | Error value or error magnitude                                                                        |
| $\hat{e}_0^{(1)}$          | Error estimate                                                                                        |
| $e_P(x)$                   | Error located in the parity-check section of $r(x)$                                                   |
| $e_{\lambda-m}^{(1)}$      | Postdecoding error                                                                                    |
| $\mathbf{e}$               | Error pattern or error vector                                                                         |
| $\mathbf{e}^{(1)}$         | Information error subsequence                                                                         |
| $\mathbf{e}^{(2)}$         | Parity-error subsequence                                                                              |
| $E$                        | Extension field of degree $m$ over a finite field $F$                                                 |
| $E_0(\rho)$                | Gallager function                                                                                     |
| $E_{\alpha_i}$             | Set of some properly selected error digits in the error pattern $\mathbf{e}$                          |
| $F$                        | Field                                                                                                 |
| $F^{(r)}$                  | $r$ -flat in $EG(m, 2^s)$ that does not pass through the origin $\alpha^\infty = 0$                   |
| $F^{(r-1)}$                | $(r-1)$ = flat in $PG(m, 2^s)$ that contains the point $(\alpha^{n-1})$                               |
| $g(x)$                     | Generator polynomial of a cyclic code                                                                 |
| $G$                        | Group                                                                                                 |
| $G_\lambda$                | Gain of the $\lambda$ th forward path                                                                 |
| $G/H$                      | Factor group                                                                                          |
| $[G^{(2)}]^T$              | Parity triangle of the self-orthogonal code                                                           |
| $\mathbf{G}$               | Generator matrix of the code                                                                          |
| $h(x)$                     | Parity polynomial of a cyclic code                                                                    |
| $h^*(x)$                   | Reciprocal polynomial of $h(x)$ , i.e., $h^*(x) = x^k h(x^{-1})$                                      |
| $H$                        | Subgroup                                                                                              |
| $[H]_n$                    | $n \times n$ Hadamard matrix                                                                          |
| $\mathbf{H}$               | Parity-check matrix of the code                                                                       |
| $i$                        | Error-starting symbol for the error burst in the block code word                                      |
| $I$                        | Ideal or subring of a ring $R$                                                                        |
| $J$                        | Maximum number of parity-check sums                                                                   |
| $k$                        | Number of information bits in a block code                                                            |
| $L(\mathbf{r} \mathbf{c})$ | Log-likelihood function or the metric associated with the path $\mathbf{c}$                           |
| $m(x)$                     | Minimal polynomial of a field element $\beta$ in $GF(2^m)$                                            |
| $m_i(x)$                   | Minimal polynomial of $\alpha^i$ , $1 \leq i \leq 2t$ , which are roots of the code polynomial $c(x)$ |
| $n$                        | Code length (number of coded digits) of a block code                                                  |
| $n_A$                      | Constraint length of a convolutional code affected by a single information bit                        |

|                          |                                                                                       |
|--------------------------|---------------------------------------------------------------------------------------|
| $N$                      | Normal subgroup                                                                       |
| $p$                      | Transition probability or crossover probability                                       |
| $p(x)$                   | Primitive polynomial over $GF(2)$                                                     |
| $p^*(x)$                 | Reciprocal polynomial by which the null space of the maximum-length code is generated |
| $P(c)$                   | Probability of correct decoding                                                       |
| $P(C > \theta)$          | Pareto computational distribution                                                     |
| $P(E)$                   | Overall first-event error probability                                                 |
| $P(r_j c_j)$             | Channel crossover probability                                                         |
| $P(\epsilon)$            | Probability of error                                                                  |
| $P(\epsilon \mathbf{r})$ | Conditional probability of error                                                      |
| $P_b(E)$                 | Bit-error probability                                                                 |
| $P_d$                    | First-event error probability for $d$ either odd or even                              |
| $P_{\text{ud}}(E)$       | Probability of an undetected error                                                    |
| $P_\lambda$              | Pairwise error probability                                                            |
| $q(x)$                   | Quotient polynomial                                                                   |
| $Q_j(x)$                 | Error sequence or covering polynomial for a specific code                             |
| $r(x)$                   | Received polynomial                                                                   |
| $\mathbf{r}$             | Received word, or received vector                                                     |
| $\mathbf{r}^{(1)}$       | Information subsequence of the received word $\mathbf{r}$                             |
| $\mathbf{r}^{(2)}$       | Parity subsequence of the received word $\mathbf{r}$                                  |
| $R$                      | Code rate                                                                             |
| $R$                      | Ring                                                                                  |
| $R_0$                    | Computational cutoff rate                                                             |
| $s(x)$                   | Syndrome polynomial or remainder polynomial or residue polynomial                     |
| $s^{(j)}(D)$             | Syndrome in polynomial form                                                           |
| $s_c(x)$                 | Syndrome polynomial or contents of error-pattern register                             |
| $s_p(x)$                 | Syndrome polynomial or contents of error-tracking register                            |
| $\mathbf{s}$             | Syndrome of the code                                                                  |
| $S$                      | State sequence                                                                        |
| $S(a, w, n)$             | Steiner system                                                                        |
| $t$                      | Error-correcting capability of the code                                               |
| $t_{\text{EG}}$          | Error-correcting capability of EG codes                                               |
| $t_{\text{PG}}$          | Error-correcting capability of the $(r, s)$ th-order PG codes                         |
| $T$                      | Running threshold                                                                     |



|                   |                                                                                                                    |
|-------------------|--------------------------------------------------------------------------------------------------------------------|
| $T(x)$            | Generating function                                                                                                |
| $U_{F^{(r)}}$     | Incidence vector of the $r$ -flat $F^{(r)}$ in $PG(m, 2^s)$                                                        |
| $V_{F^{(r)}}$     | Incidence vector of $F^{(r)}$ in $EG(m, 2^s)$                                                                      |
| $w(c)$            | Hamming weight                                                                                                     |
| $w[f(x)]$         | Weight of $f(x)$ , or the number of nonzero coefficients of a polynomial $f(x)$                                    |
| $w[s(x)]$         | Weight of syndrome                                                                                                 |
| $w[s^{(i)}(x)]$   | Weight of $i$ th shifted syndrome                                                                                  |
| $w_{\min}$        | Minimum weight                                                                                                     |
| $W_{2^s}(\theta)$ | Radix- $2^s$ weight of $\theta$                                                                                    |
| $x^j$             | Branch gain in the modified state diagram where the exponent $j$ denotes the Hamming weight of the $n$ coded bits. |
| $x^{n-k}Q_j(x)$   | Error sequence in the information section                                                                          |
| $x_\lambda$       | Transition at time $t = \lambda$                                                                                   |
| $\mathbf{x}$      | Transition sequence                                                                                                |
| $z$               | Burst-correcting efficiency                                                                                        |
| $Z$               | Integer ring                                                                                                       |
| $\alpha$          | Primitive element, i.e., a root of the primitive polynomial $p(x)$ of degree $m$                                   |
| $(\alpha^i)$      | Set consisting of $2^s - 1$ elements or points of $GF(2^{(m+1)s})$ which form $PG(m, 2^s)$ over $GF(2^s)$          |
| $\alpha_k^j$      | Error-location number at position index $j_k$                                                                      |
| $\gamma(x)$       | Check polynomial or remainder polynomial                                                                           |
| $\Delta$          | Preselected threshold increment                                                                                    |
| $\lambda(x)$      | Difference-set polynomial                                                                                          |
| $\lambda^*(x)$    | Reciprocal polynomial of $\lambda(x)$                                                                              |
| $\xi_i$           | Parity-check sums orthogonal on an error digit                                                                     |
| $\rho$            | Pareto exponent                                                                                                    |
| $\phi_B$          | Metric of the backward node                                                                                        |
| $\phi_F$          | Metric of the forward node                                                                                         |
| $\phi_i$          | Fano metric                                                                                                        |
| $\Omega(x)$       | Error-evaluator polynomial                                                                                         |
| BPSK              | Binary phase-shift keying                                                                                          |
| BSC               | Binary symmetric channel                                                                                           |
| DMC               | Discrete memoryless channel                                                                                        |
| MLD               | Maximum likelihood decoder                                                                                         |
| $EG(m, 2^s)$      | Euclidean geometry of dimension $m$ over $GF(2^s)$                                                                 |
| $F[x]$            | Ring of polynomials over the field $F$                                                                             |

|                                 |                                                                                                                            |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| $GCD(r, s)$                     | Greatest common divisor of two integers $r$ and $s$                                                                        |
| $GCD[r(x), s(x)]$               | Greatest common divisor of two polynomials $r(x)$ and $s(x)$                                                               |
| $GF(q)$                         | Galois field of $q$ elements.                                                                                              |
| $LCM(r, s)$                     | Least common multiple of two integers $r$ and $s$                                                                          |
| $LCM[r(x), s(x)]$               | Least common multiple of two polynomials $r(x)$ and $s(x)$                                                                 |
| $\log P(r_j c_j)$               | Bit metrics                                                                                                                |
| $PG(m, 2^s)$                    | $m$ -dimensional projective geometry over $GF(2^s)$                                                                        |
| $\mathbf{c}^{(1)} = \mathbf{d}$ | Information sequence                                                                                                       |
| $\mathbf{c}^{(2)} = \mathbf{p}$ | Parity sequence                                                                                                            |
| $r \equiv s \pmod{d}$           | $r$ is congruent to $s$ modulo $d$                                                                                         |
| $s(x) \equiv f(x) \pmod{g(x)}$  | $s(x)$ is congruent to $f(x)$ modulo $g(x)$                                                                                |
| $\Sigma E_{\alpha_i}$           | Equal to or close to                                                                                                       |
| $\lfloor J/2 \rfloor$           | Modulo-2 sum of error digits in $E_{\alpha_i}$                                                                             |
|                                 | Number of error digits in the correctable error pattern, i.e., the error-correcting capability for majority-logic decoding |



# Glossary of Terms

**Abelian group** A group that has a finite number of elements is called a finite group. The number of elements in a finite group  $G$  is the order of  $G$ . If  $a * b = b * a$  for all  $a, b \in G$ , the groups are called commutative or abelian.

**BCH bound** The minimum distance of the  $t$ -error-correcting BCH code is at least the designed distance  $d_0 = 2t + 1$ . The true minimum distance of a BCH code may be equal to or greater than the designed distance. The designed distance is only a lower bound on the true distance of the code. Thus the minimum distance of a BCH code with designed distance  $d_0$  is at least  $d_0$ . This is usually called the BCH bound.

**Bit-error probability** A more important and practical performance measure is the number of bits decoded in errors, that is, the bit-error probability  $P_b(E)$ . As for the first-event-error probability, the generating function approach will be used to derive useful bounds on the bit-error probability. The expected number of bit errors caused by any incorrect path which diverges from the correct path at level  $j$  can be bounded by weighting each term of the union bound by the number of bit errors which occur on that incorrect path. The bit-error probability is bounded by taking the derivative of the augmented generating function  $T(x, y)$  with respect to  $y$  and setting it to  $y = 1$ .

**Burst-error-correcting convolutional codes** Since Hagelbarger (1959) introduced the idea of correcting burst errors of recurrent codes, many coding theorists have developed the techniques to design more efficient codes for burst-error correction. The main contributors in this area are Wyner and Ash (1963), Berlekamp (1964), Preparata (1964), Massey (1965), Iwadare (1968), Kohlenburg and Forney (1968), Gallager (1968), Tong (1970), and Ferguson (1971).

**Burst-error-correcting cyclic codes** On memory channels, noise does not affect each transmitted digit independently. Thus transmission errors occur in bursts and memory channels are also called burst-error channels. Examples of burst-error channels are radio channels where error bursts are caused by signal fading, which results from multipath transmission; telephone lines, which are affected by impulsive switching noise and crosstalk from other lines; and magnetic recordings, which are subject to tape dropouts due to surface defects and dust particles.

**Characteristic of ring** For a ring  $R$ , if a positive integer  $n$  exists such that  $na = 0$  for all  $a \in R$ , then the least such positive integer is the characteristic of the ring. For example, an integral ring  $Z_n = \{0, 1, 2, \dots, n-1\}$  modulo  $n$  is of characteristic  $n$ .

**Coding bounds** There are essentially two kinds of coding bounds, that is, bounds on minimum distance and bounds on performance. The bounds on minimum distance are the Hamming bound and the Plotkin bound, which indicate the maximum possible minimum distance for a given code length and code rate, while the Gilbert-Varsharmov bound provides an achievable lower bound on the minimum distance of the best code. These bounds on minimum distance are often used to determine how close a new code is to the best possible code. The bounds on performance indicate that the average performance of all block codes exhibits a probability of error that decreases exponentially with code length. This bound implies the existence of specific codes which have better than average performance.

**Coding gain** The usual figure of merit for a coded system is the ratio of the energy per information bit to the noise spectral density  $E_b/N_0$  that is required to achieve a given probability of error. The coding gain is defined as the degree of improvement achieved when a particular coding scheme is used. The usual method of determining coding gain is to plot the probability of error  $P(E)$  versus  $E_b/N_0$  for both coded and uncoded operations and to read the difference in required  $E_b/N_0$  at a specified error rate.

**Commutative group** See Abelian group.

**Commutative ring** A ring  $R$  is an abelian group under addition because the additive operation is always commutative in a ring. But the multiplicative operation need not be commutative. A commutative ring is one in which multiplication is commutative such that  $ab = ba$  for all  $a, b \in R$ . An integral domain is a commutative ring with unity and contains no divisor of zero.

**Computational distribution** In sequential decoding, the number of computations required to advance one node deeper into the code tree is a random variable that describes the complexity involved in achieving a given level of performance. Since the number of computations required to decode a frame of data is a random variable, its probability distribution must be computed in order to determine computational performance. The computational distribution of sequential decoding has been studied for a number of years.

**Concatenated coding** Concatenated coding was introduced by Forney (1966) as a practical technique for implementing a code with a very long block length and a large error-correcting capability. A practical system of utilizing multiple levels of coding is to concatenate two or more codes. The most common approach uses two levels of coding, i.e., an inner code over  $GF(2)$  and an outer code with symbols from  $GF(2^m)$ . Reed-Solomon codes are commonly used as the outer code, whereas many different codes (the short-block code, the orthogonal code, the convolutional code, etc.) have been suggested for use as the inner code. Concatenated codes are effective against a mixture of random and burst errors.

**Constraint length** The constraint length of an  $(n, k, m)$  convolutional encoder, denoted by  $n_A = (m+1)n$ , is defined as the maximum number of encoder outputs that can be affected by a single information bit.



**Convolutional encoding** A convolutional encoder with constraint length  $n_A$  consists of an  $m$ -stage shift register together with  $n$  modulo-2 adders and a P/S multiplexer for serializing encoder outputs into a single code sequence for transmission. Thus, the data input to the encoder is shifted into and along the shift register  $k$  bits at a time. The encoder output sequences are obtained by taking the convolution of the information sequence with the generator sequences of the code. Since a convolutional encoder is a finite-state machine, its operation can be described by a state diagram.

**Cyclic code** An  $(n, k)$  linear block code  $C$  over  $GF(q)$  is called a cyclic code if whenever a code word  $\mathbf{c} = (c_0, c_1, \dots, c_{n-2}, c_{n-1})$  is in  $C$ ,  $c^{(1)} = (c_{n-1}, c_0, c_1, \dots, c_{n-3}, c_{n-2})$  is also in  $C$ . In general,  $i$  cyclic shifts to the right of any code word in  $C$  yields another code word in  $C$ .

**Cyclic group** A cyclic group  $G$  is a finite group in which every element can be written as a power of some element  $a \in G$ . Every cyclic group can then be expressed in the form  $G = \{a^0, a^1, a^2, \dots, a^{q-1}\}$ , where  $q$  is the order of  $G$ ,  $a^0 = e$  is the identity element,  $a$  is a generator of  $G$ , and the inverse of  $a^i$  is  $a^{q-i}$ . It is also true that  $a^q = a^0$ . A cyclic subgroup of  $G$ , i.e.,  $H = \{a^n | n \in \mathbb{Z}\}$ , must contain all elements  $a^n$  (or  $na$  for an additive group) for all  $n \in \mathbb{Z}$ . Thus, every cyclic group is abelian.

**Designed distance** The designed distance  $d_0$  is only a lower bound on the true distance of the  $t$ -error-correcting BCH code.

**Dual code** The dual code of an  $(n, k)$  cyclic code generated by  $g(x)$  is an  $(n, n - k)$  cyclic code generated by the parity-check polynomial  $h(x)$ .

**Elementary symmetric functions** The coefficients  $\sigma_i$ ,  $0 \leq i \leq n$ , of the error-locator polynomial  $\sigma(x)$  and the error-location numbers  $\alpha_i^j = \beta_\lambda$  are functionally related by the set of  $\nu$  equations. These coefficients  $\sigma_i$ 's are known as elementary symmetric functions of  $\beta_i$ .

**Encoding of cyclic code** The encoding of a  $k$ -bit information sequence involves computing the parity-check bits, which are the remainders  $\gamma(x)$  obtained from dividing  $x^{n-k}d(x)$  by  $g(x)$ . Thus the encoding is accomplished by a division circuit which is an  $(n - k)$ -stage shift register with feedback connections specified by generator polynomial  $g(x)$ .

**Error-correcting capability** An  $(n, k)$  block code with minimum distance  $d_{\min}$  guarantees that all error patterns of  $t = \lfloor (d_{\min} - 1)/2 \rfloor$  or fewer errors will be corrected. This  $t$  is called the error-correcting capability.

**Error-evaluator polynomial** The error-evaluator polynomial  $\Omega(x)$  is defined as the product of the error-locator polynomial  $\sigma(x)$  and the syndrome polynomial  $s(x)$  such that  $\Omega(x) = \sigma(x)s(x)$ . For nonbinary Reed-Solomon codes, the error-evaluator polynomial is used to find the error value (or error magnitude) at each error location.

**Error-locator polynomial** The error-locator polynomial plays an important role in multiple-error correction of the BCH and RS codes using the iterative algorithms proposed and developed by Berlekamp, Massey, and Peterson. The problem of finding the minimal-degree solution to the error-locator polynomial is best understood by the iterative operation. Application of this algorithm to obtain the error-locator polynomial is essential. At each step of the

iteration, the algorithm must retain both a minimal-degree polynomial whose coefficients satisfy Newton's identities and a possible correction term associated with the discrepancy.

**Error propagation effect** In threshold decoding, if an incorrect estimate of the error bit is fed to the syndrome, additional errors in subsequent decoding will occur. This is called the error propagation effect and is a serious flaw in threshold decoders that use decision feedback. There are two commonly used approaches for reducing the effects of error propagation. Error propagation can be either limited or unlimited. Limited error propagation cannot be entirely eliminated in a feedback decoder. One solution is to select good codes that have the automatic recovery property, which limits error propagation. Unlimited error propagation is quite serious when it results from poor code selection. The other approach for limiting error propagation is simply to disconnect feedback in the decoder. This is called definite decoding which is not feasible because of the increased probability of error.

**Error-trapping decoding** Consider a  $t$ -error-correcting cyclic code. Assume that all the errors lie in the  $n - k$  high-order position of the received word  $r(x)$ . If  $r(x)$  is cyclically shifted  $n - k$  times, the errors will move to the  $n - k$  parity-check section of  $r(x)$ . The errors are then trapped in the syndrome register if and only if the weight of the syndrome  $w[s(x)]$  is less than or equal to  $t$ . Thus the syndrome bits in the syndrome register are identical to the error bits in the  $n - k$  high-order positions. If the errors are not confined to the  $n - k$  high-order positions of  $r(x)$ , shift  $r(x)$  cyclically  $n - i$  times to the right in order to confine all the errors to the  $n - k$  parity-check section of  $r(x)$ . By doing so, the error pattern  $e(x)$  can be trapped in the syndrome register if  $w[s^{(i)}(x)]$  is less than or equal to  $t$ ,  $i = 1, 2, \dots$ , and error correction can be accomplished by simply adding the error pattern  $e(x)$  to the received word  $r(x)$ . This error-trapping concept makes it possible to implement the error-trapping decoder.

**Factor group** See Normal group.

**Fano algorithm** The Fano algorithm forms the backbone of sequential decoding for convolutional codes. The Fano algorithm searches the code tree for the most probable path by examining one path at a time. The decoder examines a sequence of nodes in the code tree, starting with the origin node and ending with one of the terminal nodes. If the decoder sees a node it likes, it moves forward to that node. If not, the decoder moves backward and then tries to move forward along another path, depending on whether the metric values are increasing or decreasing. Some nodes are visited more than once, and then their metric values must be recomputed. Thus, the decoder performs a trial-and-error search for the correct path in the code tree in a sequential manner. Because of this unfavorable search, a substantial amount of computation is required to recover the correct path. This variability of computational effort is one of the major drawbacks of sequential decoding by the Fano algorithm.

**First-event-error probability** For convenience and without loss of generality (because the codes are linear), we can assume that an all-zero code word is transmitted. A first-event error is made at an arbitrary trellis level  $j$  if the all-zero path (the correct path) is eliminated for the first time at level  $j$  be-

cause another path (the incorrect path) merges with it. That is, the correct path is not a survivor at the state  $s_0 = (0\ 0)$  at level  $j$ , and an incorrect path is closer to the received path than correct path. Suppose that the path being compared with the all-zero path at some level  $j$  has a distance  $d$  from the all-zero path. Then the probability of selecting the incorrect path is called the first-event-error probability, which is described by two different formulas, depending on whether distance  $d$  is odd or even.

**Gallager function** See Pareto exponent.

**Galois field** A field  $F$  with  $q$  elements is called a finite field, or a Galois field, and is denoted by  $GF(q)$ . The minimum number of elements a field can have is 2, i.e., 0 and 1. This is called the binary field, which is denoted by  $GF(2)$ .

**Generating function** The performance of a convolutional code depends on the decoding algorithm employed and the distance properties of the code. The distance properties and performance bounds for maximum-likelihood decoding can be obtained from the state diagram. The generating function provides a complete description of the weight distributions of nonzero code words by means of the augmented state diagram.

**Generator matrix** A linear  $(n, k)$  block code is completely specified by a  $k \times n$  matrix  $G$ , which is called a generator matrix. A block encoder transforms each  $k$ -tuple information block  $\mathbf{d}$  into a binary  $n$ -tuple code word  $\mathbf{c}$  by the  $k \times n$  generator matrix  $G$ . However, the generator matrix  $G$  of an  $(n, k)$  convolutional code is a semi-infinite matrix which has an infinite number of rows and columns. This corresponds to the fact that the information and code sequences of convolutional code can be arbitrarily very large.

**Greatest common divisor** Given two polynomials  $r(x)$  and  $s(x)$  of  $F[x]$ , their greatest common divisor  $GCD[r(x), s(x)]$  can be computed by an iterative application of the division algorithm.

**Hamming weight** The Hamming weight of a code word  $\mathbf{c}$ , denoted by  $w(\mathbf{c})$ , is defined as the number of nonzero components of  $\mathbf{c}$ .

**Ideal of ring** A subring  $I$  of a ring  $R$  satisfying  $rI \subseteq I$  and  $Ir \subseteq I$  for all  $r \in R$  is called an ideal of  $R$  under the operation of multiplication. If  $I$  is an ideal in a ring  $R$  under the operation of addition, then the ring of cosets  $r + I$  is variously called the quotient ring, the factor ring, or the residue class ring of modulo  $I$ , and is denoted by  $R/I$ . Since an ideal is a subgroup, the cosets can be formed. The ideal forms the first row of an array, with the 0 element in the leftmost column. This ideal is equivalent to the code word in coding.

**Interlaced, or interleaved, code** Given an  $(n, k)$  cyclic code, it is possible to construct an  $(ni, ki)$  cyclic code by interleaving. This can be done by arranging the  $i$  code words in the original code into  $i$  rows of an  $i \times n$  rectangular array and then transmitting the array column by column. This resulting code is called an interlaced, or interleaved, code. The parameter  $i$  is referred to as the interlacing degree. A burst of length  $i$  in any column will affect no more than one digit in each row. Thus if the original code corrects any single errors, the interlaced code corrects single bursts of length  $i$  or less. As a result, if the original code corrects any single burst of length  $b$  or less, the interlaced code will correct any single burst of length  $bi$  or less.



**Irreducible polynomial** A nonzero polynomial  $p(x)$  that is divisible only by  $p(x)$  is called an irreducible polynomial. A monic irreducible polynomial is called a primitive polynomial.

**Log-likelihood function** Suppose that an information sequence  $\mathbf{d} = (d_0, d_1, \dots, d_{kL-1})$  is encoded into a code sequence  $\mathbf{c} = (c_0, c_1, \dots, c_{N-1})$  of length  $N = n(L + m)$  and that a received sequence  $\mathbf{r} = (r_0, r_1, \dots, r_{N-1})$  is received over a discrete memoryless channel (DMC). A maximum-likelihood decoder (MLD) for a DMC must locate a code sequence in such a way that the conditional probability

$$P(\mathbf{r}|\mathbf{c}) = \prod_{j=0}^{N-1} P(r_j|c_j)$$

is as large as possible. Since the logarithm is a monotonically increasing function of its argument, the MLD may equivalently look for a code sequence  $\mathbf{c}$  for which

$$L(\mathbf{r}|\mathbf{c}) = \sum_{j=0}^{M-1} \log P(r_j|c_j)$$

is as large as possible.  $L(\mathbf{r}|\mathbf{c}) \triangleq \log P(\mathbf{r}|\mathbf{c})$  is called the log-likelihood function or the metric associated with the path  $\mathbf{c}$ . Each individual term  $\log P(r_j|c_j)$  is called a bit metric.

**Maximum-length codes** Maximum-length codes are the duals of the Hamming codes of length  $2^m - 1$ . Hence a  $(2^m - 1, m)$  code,  $m \geq 3$ , with minimum distance  $d_{\min} = 2^{m-1}$  is called the maximum-length code.

**Minimal polynomial** Let  $\alpha$  be a primitive element of  $GF(2^m)$ . The monic polynomial  $m(x)$  of smallest degree with coefficients in  $GF(2)$  such that  $m(\alpha) = 0$  is called the minimal polynomial of  $\alpha$ . If the transmitted code word  $c(x)$  has  $\alpha, \alpha^2, \dots, \alpha^{2^t}$  as its roots,  $c(x)$  is then divisible by the minimal polynomials  $m_1(x), m_2(x), \dots, m_{2^t}(x)$  of  $\alpha, \alpha^2, \dots, \alpha^{2^t}$ . The syndrome  $s(x)$  from the received polynomial  $r(x)$  can be computed by dividing  $r(x)$  by the minimal polynomial  $m_i(x)$  of  $\alpha^i$  if and only if  $r(x) \neq c(x)$ . In addition, the generator polynomial  $g(x)$  of the binary  $t$ -error-correcting BCH code of length  $2^m - 1$  can be obtained from  $g(x) = \text{LCM}\{m_1(x), m_2(x), \dots, m_{2^t}(x)\}$ . Therefore, the minimal polynomials play an important role in producing the generator polynomials and in generating the syndrome sequences in the BCH decoder.

**Minimum distance** The distance between the two code words is defined as the number of corresponding elements or positions in which they differ. This measure is called the Hamming distance. The smallest value in the set of Hamming distances for the  $2^k$  code words is called the minimum distance, denoted as  $d_{\min}$ .

**Monic polynomial** Let  $f(x)$  be a polynomial with coefficients in a ring  $R$ .

$$f(x) = f_0 + f_1x + \dots + f_{n-2}x^{n-2} + f_{n-1}x^{n-1}$$

The degree of  $f(x)$  is the largest power of  $x$  in a term with a nonzero coefficient  $f_{n-1}$ . A polynomial is called monic if the coefficient of the highest power of  $x$  is 1, that is,  $f_{n-1} = 1$ .

**Newton's identities** From power-sum symmetric functions and elementary symmetric functions, the coefficients  $\sigma_i$  of the error-locator polynomial  $\sigma(x)$  are related to the syndrome components  $s_i$  by Newton's identities.

**Nonsystematic convolutional codes** Good convolutional codes have the largest possible free distance. Since some nonsystematic convolutional codes have superior distance structures, nonsystematic codes are sometimes preferred over systematic codes because a larger free distance can be achieved for a given value of encoder memory order  $m$ . This leads to a performance advantage which increases with code rate. Nonsystematic convolutional codes are commonly used in Viterbi decoders.

**Normal subgroup** A subgroup  $H$  of a group  $G$  is normal if, for any element  $h$  of  $H$  and any element  $g$  of  $G$ ,  $g^{-1} * h * g$  is in  $H$ . If a subgroup  $H$  of a group  $G$  is normal, it is possible to define an operation on cosets to form a new group for which the cosets are the elements. This group is called the factor group and is denoted by  $G/H$ .

**Order of cyclic group** If  $G = \{a^q | q \in Z\}$  is a finite group in which every element can be written as a power of some element  $a$ ,  $a^q = a^0$ , where  $q$  is the order of  $G$ .

**Packing radius of the code** Suppose we place a small sphere of radius  $t$  around each of the possible transmitted code words. Assume that these spheres are increased in radius by integer amounts until they cannot be made larger without causing some spheres to intersect. That value of the radius is equal to the number of errors that can be corrected by the code. It is called the packing radius of the code. Therefore, the packing radius  $t$  is equal to the number of errors that can be corrected by the code.

**Pareto distribution** In sequential decoding a decoder computation is the process of advancing into the code tree by accepting or discarding the new path. If  $C$  is the number of computations required to decode the first  $L$  information bits, then for a sequential decoder, the probability that  $C$  exceeds some number  $\theta$  is given by  $P(C > \theta) < \alpha \theta^{-\rho}$  and is referred to as the Pareto distribution. The values of the Pareto exponent  $\rho$  and the constant  $\alpha$  depend on the channel and the code rate used. The  $\rho$  is an extremely important parameter for sequential decoding because  $P(C > \theta)$  is a function of  $\rho$ . The computational behavior of a sequential decoder can also be described in terms of the Gallager function  $E_0(\rho)$ , since it relates  $\rho$  and the code rate  $R$  as follows:  $R = E_0(\rho)/p$ .

**Parity-check bits** In an  $(n, k)$  block code for code rate  $R \leq 1$ ,  $n - k$  redundant bits are added to each  $k$ -bit information sequence to form a code word. These  $n - k$  redundant bits allow the code to combat channel noise and are called parity-check bits.

**Parity-check sum** The concept of orthogonal parity-check sums is central to majority-logic decoding. Majority-logic decoding is a simple method of detecting error digits from parity-check sums.

**Parity triangle** The matrix which multiplies the information-error subsequence is called the parity triangle of the code. The first column of the parity triangle is exactly identical to the generator sequence. The parity triangle is composed by shifting down by one bit and truncating in column after column.



This parity triangle is used to select a set of check sums orthogonal on an error bit.

**Perfect code** A perfect code is defined as one which satisfies the Hamming bound with equality. The only nontrivial perfect codes are (1) the  $(2^m - 1, 2^m - 1 - m)$  Hamming codes with  $d_{\min} = 3$ , (2) the binary repetition codes with two all-zeros and all-ones code words, and (3) two Golay codes: the (23, 12) binary code with  $d_{\min} = 7$  and the (11, 6) nonbinary ternary code with  $d_{\min} = 5$ . Both the (23, 12) binary Golay code and the (11, 6) ternary Golay code are quadratic-residue codes that have the largest minimum distance for any known code that has the same  $n$  and  $k$ ; this is what makes quadratic-residue codes so attractive.

**Power-sum symmetric functions** The error pattern is completely specified by the error-location number  $\alpha_x^j$ . A set of  $2t$  equations between the syndrome components and unknown error-location numbers are called power-sum symmetric functions. Consequently, in order to decode BCH codes these power-sum symmetric functions must be solved and the unknown error-location numbers must be determined from the syndrome components.

**Primitive polynomial** See Irreducible polynomial.

**Probabilistic decoding** Probabilistic decoding refers to the decoding of convolutional codes: maximum-likelihood decoding by the Viterbi algorithm and sequential decoding by the Fano algorithm or by the ZJ stack algorithm. The Viterbi decoding algorithm uses the trellis structure of the code and determines the maximum-likelihood estimate of the transmitted sequence that has the largest metric at every distinct path. The Fano sequential decoder uses the tree structure of the code and only follows the one path in the tree that appears to be the most likely. Sequential decoding involves a random motion in the tree, and hence the number of computations required to decode a given information sequence is a random variable. In the stack algorithm, the decoder creates a stack of previously examined paths, which vary in length. Each stack entry contains a path along its metric; the path with the largest metric is placed on top, and the others are ordered according to their metric values.

**Quasi-perfect code** The quasi-perfect code is defined as any code which has all vectors of weight  $t$  or less, some vectors of weight  $t + 1$ , and no vectors of weight greater than  $t + 1$  appearing as coset leaders. For example, the (24, 12) extended Golay code is self-dual and is quasi-perfect only because all spheres of radius  $t$  about each code vector are disjoint, but every vector is no further than  $t + 1$  from some code vector.

**Random-error-correcting cyclic codes** On memoryless channels, such as the BSC, noise affects each transmitted digit independently. Thus transmission errors occur randomly in the received word. The codes devised for correcting these random errors are called random-error-correcting codes. Typical examples of memoryless random-error channels are most line-of-sight transmission channels, many satellite channels, and deep-space channels.

**Self-orthogonal codes** Some classes of systematic convolutional codes that allow threshold decoding using simple majority-logic gates have a special structure. An  $(n, n - 1, m)$  convolutional code is said to be self-orthogonal if

the set of all syndrome bits that checks one error bit forms a set of check sums orthogonal on that error bit. A procedure was suggested by Robinson and Bernstein (1967) for constructing  $(n, n-1, m)$  self-orthogonal codes using the concept of all distinct positive difference sets.

**Shortened cyclic codes** An  $(n-i, k-i)$  shortened code can be formed by dropping  $i$  information bits from each code word of an  $(n, k)$  cyclic code. There are  $2^{k-i}$  such code words of length  $n-i$ . Shortening an  $(n, k)$  cyclic code by  $i$  bits corresponds to omitting the last  $i$  rows and columns of the generator matrix  $\mathbf{G}$ , or the last  $i$  columns from the parity-check matrix  $\mathbf{H}$ .

**Stack algorithm** The ZJ or stack algorithm is another interesting algorithm for sequential decoding of convolutional codes. It was developed to reduce the computational work required by the Fano algorithm. However, a stack algorithm computation is much more complex than a Fano algorithm computation: reordering the stack is troublesome and becomes quite time-consuming as the number of entries in the stack becomes large; it also places severe limitations on the decoding speed. In the stack algorithm, the decoder creates a stack of previously examined paths of varying lengths. Each stack entry contains a path along its metric; the path with the largest metric is placed on top, and the others are ordered according to their metric values. At each decoding step, the path at the top of the stack is extended to its  $2^k$  successors by computing their branch metrics. The top path is then deleted from the stack and replaced by its  $2^k$  successors. Then the new stack is reordered according to decreasing metric values. This decoding step continues until the path at the top of the stack is at the end of tree, which is the decoded information sequence.

**Standard array** The standard array is a way of tabulating all  $2^k$   $n$ -tuples over  $GF(2)$ . This  $2^{n-k} \times 2^k$  rectangular array forms the conceptual basis for decoding linear block codes. The standard array can be used as a decoding table by simply locating the received  $n$ -tuple word in the table.

**Syndrome** If the received word  $\mathbf{r}$  is exactly identical to the transmitted code word  $\mathbf{c}$ , there are no errors in the received word and no syndrome bits are generated by the syndrome register in the decoder. Otherwise (if  $\mathbf{r} \neq \mathbf{c}$ ), the syndrome register produces the syndrome sequence. The first step in decoding is to compute the syndrome  $\mathbf{s}$  which is functionally related with the error pattern  $\mathbf{e}$  such that  $\mathbf{s} = \mathbf{eH}^T$ . Thus, the function of a decoder is to estimate the error polynomial  $e(x)$  from the syndrome  $s(x)$ .

**Syndrome components of BCH codes** For decoding a  $t$ -error-correcting BCH code, the syndrome of  $2t$ -tuple is computed from the received polynomial  $r(x)$ . Each syndrome component  $s_k$  is computed by dividing  $r(x)$  by the minimal polynomial  $m_k(x)$ ,  $1 < k < 2t$ , of the field element  $\alpha^k$  of  $GF(2^m)$  such that  $s_k = r(\alpha^k) = \gamma_k(\alpha^k)$  because  $m_k(\alpha^k) = 0$ .

**Syndrome decoding** The syndrome can be generated by passing the received noise-corrupted information digits to a convolutional encoder and adding its output to the received noise-corrupted parity digits. Since the syndrome sequence  $\mathbf{s} = \mathbf{rH}^T = \mathbf{eH}^T$ , the syndrome sequence depends only on the channel error sequence  $\mathbf{e}$ . But the decoder can be designed to operate on  $\mathbf{s}$  rather than on  $\mathbf{r}$  because knowing  $\mathbf{s}$  is equivalent to knowing  $\mathbf{r}$ . Such a decoding process is called syndrome decoding.

**Systematic convolutional codes** An important subclass of convolutional codes is the class of systematic codes. In a systematic code, the first  $k$  output sequences are an exact replica of the  $k$  input sequences. Systematic codes have the advantage that they are simpler to encode than nonsystematic codes because less hardware is required. In addition, an inverting circuit is not needed to recover the information sequence from the code word.

**Threshold decoder** Threshold decoding of an  $(n, k, m)$  convolutional code is based on the concept of orthogonal parity-check sums. A feedback threshold decoder is a device of majority-logic circuitry that operates on a syndrome bit, or any sum of syndrome bits, to produce a likely estimate of some selected error digits. This syndrome-error-feedback decoder is called a threshold decoder because the threshold logic (also called majority logic) is involved in the error decisions. The output of the threshold gate is also fed back to modify the syndrome register before the next estimate can be made. This is also the reason why the decoding circuitry is called a feedback decoder.

**Trial-and-error orthogonalizable codes** Another useful class of convolutional codes is the class of so-called trial-and-error codes introduced by Massey. Massey has constructed a large number of trial-and-error codes by forming the orthogonal check sums composed of the syndrome bits or sums of syndrome bits. We call this class of trial-and-error codes the orthogonalizable codes; they differ from the self-orthogonal codes in a few respects. If some of the orthogonal check sums are formed from sums of two or more syndrome bits, we can construct orthogonalizable codes. The error estimates from the majority vote can also be used with feedback decoding to achieve a full error-correction capability. However, a disadvantage of orthogonalizable codes is that they do not possess the automatic resynchronization properties that limit error propagation with feedback.

**Uniform codes** One class of low-rate convolutional codes which can also be decoded by threshold decoding is the class of uniform codes. Massey (1966) designated uniform codes as  $(2^m, 1, m)$  codes with minimum distance  $d_{\min} = (m + 2)2^{m-1}$ , and constraint length  $n_A = (m + 1)2^m$ . Since the code rate is limited by  $R = 1/2^m$ , uniform codes comprise a small class.

**Union bound** The most useful technique for estimating the performance of convolutional codes are union bounds and computer simulation. The union-bound approach is often useful for predicting the performance of a specific hardware implementation of the Viterbi decoder. The performance measures considered are the bound on a first-event-error probability and the bit-error probability, which is derived using a union-bound argument.

**Viterbi algorithm** A practical technique for maximum-likelihood decoding of convolutional codes is the Viterbi algorithm introduced in 1967. This decoding algorithm uses the trellis structure of the code and determines the maximum-likelihood estimate of the transmitted code word that has the largest metric. The Viterbi algorithm simply finds the path with the largest accumulated metric through the trellis in such a way that it processes the received word in an iterative manner. At each step, it compares the metrics of all paths entering each state, stores only the survivor with the largest accumulated metric,

and discards the unlikely paths at every state, which reduces the decoding effort. Therefore the Viterbi decoder must base the estimate of the code sequence on the received sequence. However the principal limitation of the Viterbi algorithm is that the complexity of the decoding is proportional to the number of encoder states.





---

# Index

- Abelian (commutative) groups, 16, 18
  - Additive groups, 16
  - Additive white gaussian noise (AWGN), 368, 380–384
    - antipodal BPSK signaling and, 414
  - Algebraic decoding, 7
  - Algorithms:
    - Berlekamp iterative, 204, 205, 219, 233
    - Berlekamp-Massey, 198, 204–210, 215
    - euclidean, 22–23
      - for polynomials, 25–26
    - Fano, 8, 351, 352, 354–361
    - fast iterative, 210–213
    - Jelinek and Zigangirov, 365
    - Peterson-Berlekamp, 201–204
    - Peterson's iterative, 191, 198, 215
    - Reed, 81
    - stack ( $\mathbb{Z}_J$ ), 8, 9, 351, 354, 361–366
    - Viterbi, 7–9, 311–312, 352, 367, 370–374
      - performance bounds for convolutional codes with, 374–384
  - Antipodal BPSK signaling, 414
  - APP decoding scheme, 415
  - Applications:
    - of error-correcting code, 11–13
    - of error trapping, 139–143
    - of Reed-Solomon codes, 238–242
  - Ash, R. B., 415
  - Augmented Hamming codes, 109
  - Automorphisms, 19
  - Bayes' law, 353
  - BCH bounds, 173–178
  - BCH (Bose-Chaudhuri-Hocquenghem) codes, 9, 167–173
    - decoding algorithms for more than two errors, 195–196
  - BCH (Bose-Chaudhuri-Hocquenghem) codes: decoding algorithms for more than two errors (*Cont.*):
    - Berlekamp-Massey algorithm for, 204–210
    - checking of error pattern for, 213–214
    - computation of error-location numbers for, 199–201
    - decoder design architecture for, 214–219
    - fast iterative algorithm for, 210–213
    - initial conditions for, 197–198
    - Peterson-Berlekamp algorithm for, 201–204
  - decoding algorithms for two or fewer errors, 189–195
  - decoding procedures for:
    - error-locator polynomials for, 186–189
    - syndrome computation for, 178–186
  - euclidean geometry codes and, 295
  - generator polynomials for, 167–169, 249–254
  - majority-logic decodable codes and, 255
  - minimum distances and bounds for, 170, 173–178
- Berlekamp, E. R., 9, 10, 167, 415
  - bit-serial multiplier algorithm by, 223
  - iterative algorithm by, 204, 205, 219, 233
  - on performance of sequential decoder, 366
- Berlekamp-Massey algorithm, 198, 204–210
  - decoder design architecture and, 215
- Bernstein, A. J., 395
- Binary block codes, 4–5
  - linear, 35–36

- Binary block codes: linear (*Cont.*):
  - coding bounds for, 43–46
  - encoder implementation of, 41–43
  - Gilbert-Varshamov bound for, 49–51
  - Golay codes, 65–70
  - Hadamard codes, 70–71
  - Hamming bound for, 46–47
  - Hamming codes, 63–65
  - matrix description of, 36–43
  - parity-check matrices for, 39–41
  - Plotkin bound for, 47–49
  - product codes, 71–78
  - Reed-Muller codes, 78–86
  - standard array, 54–58
  - syndrome, 51–54
  - weight enumerators and probability of decoding error for, 58–62
- Binary codes, 5
- Binary convolutional codes, 6–7
- Binary difference-set codes, 9, 271, 274–279
- Binary narrow-sense BCH codes, 168
- Binary phase-shift keying (BPSK), 2
  - over additive white gaussian noise (AWGN) channels, 414
- Binary symmetric channels (BSC):
  - bit errors in, 46, 60
  - Hamming distances in, 368
  - optimum decoders for, 45
  - sequential decoders for, 366, 367
  - true error pattern in, 54
  - Viterbi algorithm and, 371
- Bit-error probability, 379
- Bits, 2
- Block codes, 4–5, 7
  - applications of, 12
  - burst-error correction for, 145–146
    - Fire codes for, 151–156
    - high-speed error-trapping decoders for, 156–164
    - theorems for, 146–151
  - convolutional codes and, 311, 332  
(*See also* Binary block codes; Linear binary block codes)
- Bose, R. C., 9, 167
- Bose-Chaudhuri-Hocquenghem codes (*see* BCH codes)
- Bounds:
  - for BCH codes, 173–178
  - for burst-error-correcting codes, 148
  - Elias, 43
  - for linear block codes:
    - Gilbert-Varshamov, 49–51
- Bounds: for linear block codes (*Cont.*):
  - Hamming, 46–47
  - minimum Hamming distances and weights for, 43–46
  - Plotkin, 47–49
  - Reiger, 164
  - types of, 6
  - union, 374, 377
- BPSK modulation, 382
- Buffer registers:
  - for cyclic Hamming codes, 122
  - for error trapping, 138
  - in Meggitt decoders, 116
- Burst-error channels, 3
- Burst-error correction, 145–146, 236–238
  - codes for, 9–10, 415–424
    - Fire codes, 151–156
  - high-speed error-trapping decoders for, 156–164
  - theorems for, 146–151
- Burst errors, 72, 75
  - Meggitt decoders for, 133
- Reed-Solomon codes and:
  - correction for, 236–238
  - expurgated, 228
- Burton, H. O., 9, 75, 150
- Catastrophic error-propagating codes, 395
- Cayley's theorem, 19
- Central processing units (CPUs), 214
- Channel code words, 2
- Channel decoders, 4
- Channel encoders, 4
- Channel-error rates, 414
- Channel noise, 4
- Channels:
  - additive white gaussian noise (AWGN), 380–384
  - burst noise in, 145
  - computational cutoff rates of, 367
  - mixed errors in, 71–72
  - super, 240
  - waveform, 3
- (*See also specific types of channels*)
- Check sums, 256
- Chien, R. T., 9, 167, 199–201, 214–215
- Chinese remainder theorem, 159–160
- Code trellis, 7
- Code words, 2, 4–5
  - in cyclic codes, 89–91
  - in majority-logic decodable codes, 263

Code words (*Cont.*):

- in product codes, 74
- weights of, 58–59

## Codes:

- BCH (Bose-Chaudhuri-Hocquenghem), 9, 167–173
  - decoding algorithms for more than two errors, 195–219
  - decoding algorithms for two or fewer errors, 189–195
  - decoding procedures for, 178–189
  - euclidean geometry codes and, 295
  - generator polynomials for, 167–169, 249–254
  - majority-logic decodable costs and, 255
  - minimum distances and bounds for, 173–178
- binary, 5
- block, 4–5, 7, 12
  - burst-error correction for, 145–164
- burst-error correcting, 9–10, 415–424
- catastrophic error-propagating, 395
- convolutional, 6–7, 9, 12–13, 311–312
  - decoding algorithms for, 354
  - encoding of, 312–318
  - generating function and distance properties for, 341–348
  - nonsystematic, 319, 325–332, 385–386
- Reed-Solomon, 219
  - structure of, 332–341
  - systematic, 319–325, 385–386
  - threshold (majority-logic) decoding for, 351, 387–424
- Viterbi algorithm and, 370–384
- cyclic, 35, 89–92
  - BCH codes, 168
  - encoders of, 100–104
  - generator and parity-check matrices for, 94–100
  - Hamming, 116–127
  - for long error bursts, 145
  - majority-logic, 255–308
  - Meggitt decoders for, 115–116
  - modifications of, 108–112, 127–132
  - syndrome and decoding of, 104–108
  - systematic structure of, 92–94
- difference-set, 9, 271, 274–279
- euclidean geometry, 78, 286, 288–298
- Fire, 9, 12, 151–164
- Golay, 46, 65–70
- Hadamard, 70–71

Codes (*Cont.*):

- Hamming, 8, 12, 43, 63–65, 116–127
  - cyclic, 116–127
  - cyclic codes modified into, 109–110
  - maximum-length codes and, 265–266, 270
  - modified, 127–132
- interleaved, 149
- Iwadare, 415–418, 421
- linear binary block, 35–36
  - coding bounds for, 43–46
  - encoder implementation of, 41–43
  - Gilbert-Varshamov bound for, 49–51
- Golay codes, 65–70
- Hadamard codes, 70–71
- Hamming bound for, 46–47
- Hamming codes, 63–65
  - matrix description of, 36–43
- Plotkin bound for, 47–49
- product codes, 71–78
- Reed-Muller codes, 78–86
  - standard array, 54–58
  - syndrome, 51–54
  - weight enumerators and probability of decoding error for, 58–62
- majority-logic decodable, 255
  - euclidean geometry, 286–298
  - multiple-step, 279–286
  - one-step, 257–279
- orthogonal parity-check sums for, 255–257
  - projective geometry, 298–308
- maximum-length, 9, 265–271
- perfect, 46–47, 63, 66–68
- product, 71–78
- projective geometry, 298–308
- Reed-Muller, 78–86
  - geometric interpretation of, 286, 291
- Reed-Solomon, 9, 12, 167, 219–242
  - application to concatenated coding of, 238–242
  - burst-error correction for, 236–238
  - encoding of, 222–224
  - for error bursts, 145–146
  - error-evaluator polynomials and error values for, 232–236
  - expurgated code for, 224, 226–229
  - syndrome computation for, 229, 232
- self-orthogonal, 395–405
- t*-error-correcting (see *t*-error-correcting code)

Codes (*Cont.*):

- trial-and-error orthogonalizable, 405–411
  - uniform, 411–413
- Coding, types of, 4–8
- Coding bounds (*see* Bounds)
- Coherent PSK modulation, 380
- Commutative (abelian) groups, 16, 18
- Commutative rings, 20
- Composite integers, 22
- Computational cutoff rates, 367
- Computer simulation, 374
- Computers:
  - burst errors in storage systems of, 145
  - decoder design architecture and, 214–215
- Concatenated coding, Reed-Solomon codes applied to, 238–242
- Conjugates:
  - in Galois fields, 30–32
  - of subgroups, 19
- Constant-weight codes, 5
- Convolutional codes, 6–7, 9, 311–312
  - applications of, 12–13
  - decoding algorithms for, 354
  - encoding of, 312–318
  - generating function and distance properties for, 341–348
  - nonsystematic, 319, 325–332, 387–388
  - Reed-Solomon, 219
  - structure of, 332–341
  - systematic, 319–325, 387–388
  - threshold (majority-logic) decoding for, 351, 387–388
  - burst-error correcting codes for, 415–424
  - performance of, 413–415
  - self-orthogonal codes for, 395–405
  - syndrome decoding, 388–391
  - syndrome and feedback decoding for, 391–395
  - trial-and-error orthogonalizable codes for, 405–411
  - uniform codes for, 411–413
  - Viterbi algorithm and, 370–374
  - performance bounds for, 374–384
- Coset decomposition, 17–18
- Coset leaders, 17, 54, 56, 57
- Cosets, 17–18, 54
- Cyclic codes, 35, 89–92
  - BCH codes, 168
  - encoders of, 100–104

Cyclic codes (*Cont.*):

- generator and parity-check matrices for, 94–100
  - Hamming, 116–127
  - for long error bursts, 145
  - majority-logic:
    - euclidean geometry, 286–298
    - multiple-step, 279–286
    - one-step, 257–279
    - orthogonal parity-check sums for, 255–257
    - projective geometry, 298–308
  - Meggitt decoders for, 115–116
  - modifications of, 108–110
    - shortened, 110–112
  - modified, decoding of, 127–132
  - syndrome and decoding of, 104–108
  - systematic structure of, 92–94
- Cyclic groups, 18
- Cyclic symbol-error-correcting codes, 215
- Decoders:
- for binary symmetric channels, 45
  - channel, 4
  - design architecture of, 214–219
  - high-speed error-trapping, 156–164
  - maximum-likelihood, 353–354
  - Meggitt (*see* Meggitt decoders)
  - for random errors, 74
  - sequential, performance of, 366–367
- Decoding:
- BCH codes:
    - error-locator polynomials for, 186–189
    - for more than two errors, 195–219
    - syndrome computation for, 178–186
    - for two or fewer errors, 189–195
  - concatenated codes, 240
  - cyclic codes, 104–108
    - modified, 127–132
  - error-trapping, 8–9, 115
    - of Fire codes, 151–153
    - high-speed decoders for, 156–164
    - improved version of, 135–139
  - maximum-likelihood, 7, 9, 333, 353–354, 367–384
  - Viterbi algorithm for, 352, 370–384
  - probabilistic, 7, 352–354, 415
  - probability of errors in, 58–62
  - of projective geometry codes, 303–308



Decoding (*Cont.*):

- sequential, 351–355
    - computational performance of, 366–367, 387
    - Fano algorithm for, 354–361
    - stack algorithm for, 361–366
    - threshold decoding and, 415
  - soft-decision, 372
  - threshold, 8, 311, 351–352
    - burst-error-correcting codes for, 415–424
    - performance and comparison of codes for, 413–415
    - self-orthogonal codes for, 395–405
    - syndrome, 388–391
    - syndrome and feedback, 391–395
    - trial-and-error orthogonalizable codes for, 405–411
    - uniform codes for, 411–413
  - Viterbi (*see* Viterbi decoding)
- Decoding errors, 58–62
- Deinterleavers, 145
- Delsarte, P., 308
- Demodulators, 3–4
- Demultiplexers, 390
- Difference-set codes, 9, 271, 274–279
- Difference sets, 395
- Discrete memoryless channels (DMCs), 353–354, 368
  - soft-decision decoding and, 372
- Distances:
  - in convolutional codes, 325, 341–348
  - Hamming (*see* Hamming distances)
  - minimum (*see* Minimum distances)
- Division algorithm, 22

EG codes (*see* Euclidean geometry codes)

Elias, P., 9, 311

Elias bound, 43

## Encoders, 4

- convolutional, 6–7
- of cyclic codes, 100–104
- of linear block codes, 41–43
- source, 2

## Encoding:

- of concatenated codes, 239
- of convolutional codes, 312–318
  - systematic, 319–321
- of Reed-Solomon codes, 222–224

Encoding equations for convolutional codes, 313, 315, 317

nonsystematic, 326–327

Energy per information bit, 6

## Error-correction coding, 4

- applications of, 11–13
- development of, 8–10

Error digits in majority-logic decodable codes, 256, 257

Error-evaluator polynomials, 232–236

Error-location numbers, 179–180

- computation of, 199–201

Error-locator polynomials, 186–189, 212  
for Reed-Solomon codes, 233

Error patterns in BCH codes, 213–214

Error polynomials, 105

Error propagation effects, 394–395

Error trapping, 133–135

- applications of, 139–143

Error-trapping decoding, 8–9, 115

- of Fire codes, 151–153

- high-speed decoders for, 156–164

- improved version of, 135–139

## Errors:

- in binary symmetric channels, 46, 60
- in channels, mixed, 71–72
- decoding, probability of, 58–62

Euclidean algorithm, 22–23

- for polynomials, 25–26

Euclidean geometry (EG) codes, 78, 286, 288–298

Evans, T. A., 271

Expurgated Hamming codes, 109

- decoding of, 127–129

Expurgated Reed-Solomon codes, 224, 226–229

Extended Hamming codes, 109

## Factor groups, 18

Fano, R. M., 7–9, 311, 351

Fano algorithm, 8, 351, 352, 354–361

Fano metric, 356

Fast iterative algorithm, 210–213

Feedback decoding, 391–395

Feedback shift registers, 115

Feedback threshold decoders, 393

Ferguson, M. J., 10, 415

Fields, 23–24

- Galois, 27–33, 117

- BCH codes and, 167

- euclidean geometry of, 289–290

- projective geometry codes and, 298–302

Finite-geometry codes, 9

Finite groups, 16



- Fire, P., 9, 149
- Fire codes, 9, 12, 151–156
  - high-speed error-trapping decoders for, 156–164
- Flats:
  - in euclidean geometry codes, 286, 288–290, 293–294
  - in projective geometry codes, 298, 304
- Forney, G. D., Jr., 10, 167, 415
  - concatenated coding introduced by, 238
  - on convolutional codes, 423–424
  - value of Viterbi algorithm proved by, 9
- Frequency shift keying (FSK), 2
- Gallager, R. G., 10, 415, 423–424
- Gallager function, 366–367
- Galois fields, 27–33, 117
  - BCH codes and, 167
  - euclidean geometry of, 289–290
  - projective geometry codes and, 298–302
- Generator matrices:
  - of block codes, 7
  - for convolutional codes, 312, 315
    - nonsystematic, 325–326
    - systematic, 319–320
  - for cyclic codes, 90–100
    - Hamming codes, 121–122
  - for linear codes, 37–39
- Generator polynomials:
  - for BCH codes, 167–169, 249–254
  - for convolutional codes, 315, 317–318
  - in decoders, 115
  - for maximum-length codes, 266–269
  - for Reed-Solomon codes, 222
    - expurgated, 224
  - for uniform codes, 411
- Geometric codes:
  - euclidean, 78, 286, 288–298
  - projective, 298–308
- Geothals, J. M., 308
- Gilbert-Varshamov bound, 6, 43, 49–51
- Golay codes, 65–70
  - as perfect code, 46, 66–68
- Gorenstein, D. C., 9, 167
- Gorog, E., 148
- Graham, F. L., 271, 308
- Ground fields, 27
- Groups, 15–19
- Hamming, Richard W., 116
- Hamming bound, 6, 46–47
  - minimum distance and weight for, 43–46
- Hamming codes, 8, 12, 43, 63–65
  - cyclic, 116–127
  - cyclic codes modified into, 109–110
  - maximum-length codes and, 265–266, 270
  - modified, decoding of, 127–132
- Hamming distances, 5, 43–46
  - in binary symmetric channels, 368
  - for soft-decision decoding, 372
  - Viterbi algorithm and, 368, 371
- Hamming weights, 43–46
  - in convolutional codes, 342
- Hardware (computer), decoder design
  - architecture and, 214
- High-speed error-trapping decoders, 156–164
- Hocquenghem, A., 9, 167
- Homomorphisms, 19
- IBM Digital Cypress, 219
- IBM 3370 disk storage system, 219
- Ideals, 21
- Improper subsets, 15
- Incidence vectors, 290, 293, 303
- Infinite groups, 16
- Information sequences, 4, 6
- Injections, 19
- Integer rings, 21–23
- Integral domains, 20–21
- Interleaved codes, 149
- Interleavers, 145
- Isomorphisms, 19
- Iwaware, Y., 10, 415, 423
- Iwaware codes, 415–418, 421
- Jacobs, I. M., 366, 374
- Jelinek, F., 8, 9, 351, 354
- Jelinek algorithm, 365
- Kasami, T., 9, 139, 141–143, 308
- Kernels, 19
- Kohlenberg, A., 10, 415, 424
- L*-step majority-logic decoders, 280–286
- Left cosets, 17
- Hadamard codes, 70–71
- Hagelbarger, D. W., 9, 415

- Lengthened Hamming codes, 109–110
- Lin, S., 9, 308
- Linear binary block codes, 35–36
  - coding bounds for:
    - Hamming bounds, 46–47
    - minimum Hamming distances and weights for, 43–46
    - Plotkin bounds, 47–49
  - encoder implementation of, 41–43
  - Gilbert-Varshamov bounds, 49–51
  - Golay codes, 65–70
  - Hadamard codes, 70–71
  - Hamming codes, 63–65
  - matrix description of, 36–43
  - parity-check matrices for, 39–41
  - product codes, 71–78
  - Reed-Muller codes, 78–86
  - standard array, 54–58
  - syndrome, 51–54
  - weight enumerators and probability of decoding error for, 58–62
- Linear block codes, 35
  - convolutional codes as, 312
  - cyclic codes, 89–92
    - encoders of, 100–104
    - generator and parity-check matrices for, 94–100
    - modifications of, 108–112
    - syndrome and decoding of, 104–108
    - systematic structure of, 92–94
  - Hamming codes as, 117
- Linear ( $n, k$ ) codes, 35
  - cyclic, 89
    - encoding of, 101–102
    - generator matrices for, 94–100
  - matrix description of, 36–41
  - Meggitt decoders for, 129
  - parity-check matrix for, 258
- MacWilliams, F. J., 59, 71, 271, 308
- MacWilliams identity, 59–62, 270
- Magnetic media, burst errors in, 145
- Majority (threshold) logic, 393
- Majority-logic decodable codes, 255
  - euclidean geometry, 286–298
  - multiple-step, 279–286
  - one-step, 257–265
    - difference-set, 271–279
    - maximum-length, 265–271
  - orthogonal parity-check sums for, 255–257
  - projective geometry, 298–308
- Majority-logic decoders, 9
- Majority-logic (threshold) decoding (*see* Threshold decoding)
- Mann, H. B., 271
- Markov processes, 333
- Mason's gain formula, 342
- Massey, J. L., 8, 10, 167
  - APP decoding scheme by, 415
  - Berlekamp-Massey algorithm by, 204
  - diffuse single-error-correcting codes by, 423
  - on error propagation effects, 395
  - majority-logic decoding by, 255, 351
  - systematic convolutional codes by, 321
  - threshold decoding proposed by, 9, 311, 387
  - trial-and-error orthogonalizable codes by, 405
  - on uniform codes, 411
- Matrices:
  - for cyclic codes, 94–100
  - generator (*see* Generator matrices)
  - Hadamard, 70–71
  - for linear codes, 36–39
  - parity-check (*see* Parity-check matrices)
- Maximum-length codes, 9, 265–271
- Maximum-likelihood decoders (MLDs), 353–354
- Maximum-likelihood decoding, 7, 9, 367–384
  - optimum likelihood strategy in, 353–354
  - tree diagrams and, 333
  - Viterbi algorithm for, 352, 370–374
  - performance bounds for convolutional codes with, 374–384
- Meggitt, J. E., 8–9
- Meggitt decoders, 115–116
  - for cyclic Hamming codes, 122
  - error trapping by, 133
  - majority-logic decodable codes and, 255
  - for shortened Hamming codes, 129
- Melas, C. M., 148
- Memories (computers):
  - burst errors in, 145
  - decoder design architecture and, 214
- Metrics:
  - Fano algorithm and, 355, 356, 358, 359
  - for soft-decision decoding, 372
  - stack (ZJ) algorithm and, 361–363
  - Viterbi algorithm and, 367, 370

- Minimum distances, 5
  - of BCH codes, 170, 173–178
  - bounds on, 6
  - Gilbert-Varshamov, 49–51
  - Hamming, 43–46
  - Plotkin, 47–49
- Mitchell, M. E., 9
- Modified cyclic codes, 109–110
  - decoding of, 127–132
- Modulators, 4
- Muller, D. E., 78, 255
- Multiple-step majority-logic decoding, 279–286
- Multiplexers, 390
- Multiplicative groups, 16
  
- Newton's identities in BCH codes:
  - for multiple errors, 195, 196, 198, 201, 205, 211
  - in syndrome decoding, 186, 187, 189
- Nilpotents, 20
- Nodes:
  - Fano algorithm's paths through, 355
  - in tree diagrams, 335–337
- Noise:
  - burst, 145
  - in waveform channels, 3
- Nonsystematic convolutional codes, 319, 325–332, 387–388
- Normal subgroups, 18
- $n$ -tuple error pattern, 51
- Null spaces, 5
  
- Omura, J. K., 9
- One-step majority-logic decoding, 257–265
  - difference-set codes for, 271–279
  - maximum-length codes for, 265–271
- Order of groups, 16
- Origin nodes, 337
- Orthogonal parity-check sums, 255–257, 279, 393
- Orthogonal polynomials, 266, 278
  
- Parallel correction for cyclic Hamming codes, 125
- Parleto distribution, 366, 367
- Parity-check bits, 5
  - for cyclic codes, 93, 100
  - in Hamming codes, 63
- Parity-check digits:
  - for difference-set codes, 274
  - in linear codes, 36–37
- Parity-check equations, 256
- Parity-check matrices, 39–41, 258
  - of BCH codes, 173–174
  - for cyclic codes, 94–100
  - for cyclic Hamming codes, 118–120
  - for linear codes, 39–41
  - orthogonal parity-check sums and, 255–256
- Parity-check polynomials for maximum-length codes, 266–269
- Parity-check sums, 256, 258–260, 263–264
  - orthogonal, 255–257, 279, 393
  - in projective geometry codes, 303
- Parity polynomials, 91
- Perfect codes, 46–47
  - Golay codes as, 46, 66–68
  - Hamming codes as, 63
- Perfect difference sets, 271, 275
- Performance:
  - of codes for threshold decoding, 413–415
  - of convolutional codes with Viterbi decoding, 374–384
  - of sequential decoders, 366–367
- Permutation groups, 19
- Peterson, W. W., 9, 146, 214, 308
  - decoding algorithm for binary BCH codes by, 167
- Peterson-Berlekamp algorithm, 201–204
- Peterson's iterative algorithm, 191, 198, 215
- Phase shift keying (PSK), 3
- Plotkin bound, 6, 43, 47–49
- Polynomial-generated codes, 35
- Polynomial rings, 24–26
- Polynomials:
  - error, 105
  - error-evaluator, 230–234
  - error-locator, 186–189, 212, 233
  - in Galois fields, 27–33
  - orthogonal, 266, 278
  - parity, 91
  - (See also Generator polynomials; Primitive polynomials)
- Power-sum symmetric functions, 180
- Preparata, F. P., 10, 415
- Preparata-Massey codes, 421
- Prime integers, 22
- Primitive elements of Galois fields, 27

- Primitive polynomials, 26
  - in BCH codes, 167–168
  - for expurgated Hamming codes, 128
  - in Galois fields, 29
  - in Hamming codes, 117
- Probabilistic decoding, 7, 352–353
  - optimum likelihood strategy in, 353–354
  - threshold decoding and, 415
- Probabilities:
  - bit-error, 379
  - of decoding errors, 58–62
- Processors, decoder design architecture and, 214
- Product codes, 71–78
- Projective geometry (PG) codes, 298–308
- Proper subsets, 15
- P/S multiplexers, 390
- Punctured Hamming codes, 109
  
- Quotient rings, 21
  
- Random-access memory (RAM), decoder
  - design architecture and, 214
- Random-burst-type errors, 71–75, 145
- Random-error correcting:
  - for burst errors, 145
  - code capabilities for, 45–46
  - cyclic Hamming codes for, 116–127
    - decoding of, 127–132
  - difference-set codes for, 271
  - error trapping for, 133–135
    - applications of, 139–143
    - improved decoding for, 135–139
  - majority-logic decodable codes for, 255
  - Meggitt decoders for, 115–116
- Ray-Chaudhuri, D. K., 9, 167
- Read-only memory (ROM), decoder
  - design architecture and, 214
- Received words, 3–4
- Reed, I. S., 9, 78, 167, 255
- Reed algorithm, 81
- Reed-Muller codes, 78–86
  - geometric interpretation of, 286, 291
- Reed-Solomon codes, 9, 12, 167, 219–242
  - application to concatenated coding of, 238–242
  - burst-error correction for, 236–238
  - encoding of, 222–224
  - for error bursts, 145–146
  - error-evaluator polynomials and error values for, 232–236
  - Reed-Solomon codes (*Cont.*):
    - expurgated code for, 224, 226–229
    - syndrome computation for, 229, 232
- Registers:
  - for cyclic Hamming codes, 122–123
  - in encoding Reed-Solomon codes, 223
  - for error trapping, 133, 135, 138, 140
  - for high-speed error-trapping decoders, 158–159, 163
  - in Meggitt decoders, 115–116
  - shift, 312, 313, 333
- Reiger, S. H., 146
- Reiger bound, 164
- Residues, 25
  - classes, 21
- Right cosets, 17–18
- Rings, 19–23
  - polynomial, 24–26
- Robinson, J. P., 395
- RS codes (*see* Reed-Solomon codes)
- Rudolph, I. D., 9, 271, 307
  
- Sain, M. K., 395
- Self-orthogonal codes, 395–405
- Semiconductors, 12
- Sequential decoding, 351–355
  - computational performance of, 366–367, 387
  - Fano algorithm for, 354–361
  - stack algorithm for, 361–366
  - threshold decoding and, 415
- Sets, 15
- Shannon, Claude, 8
- Shift registers, 312
  - for convolutional codes, 313, 333
- Shortened Hamming codes, 110
  - decoding of, 129–132
- Signal-to-noise ratios, 4
- Singer, J., 271
- Single-error-correcting perfect codes, 46
- Sloane, N. J. A., 71
- Soft-decision decoding, 372
- Soft-decision threshold decoders, 415
- Solomon, G., 9, 167
- Source encoders, 2
- Spaces, vector, 5, 36
- Stack (ZJ) algorithm, 8, 9, 351, 354, 361–366
- Standard array, 54–58
- State diagrams, 334–335
- Storage media, 3
- Super channel, 240



- Super decoders, 240
- Syndrome decoding, 388–391
  - feedback and, 391–395
- Syndrome-error-feedback decoders, 393
- Syndrome registers:
  - for cyclic Hamming codes, 122–123
  - for error trapping, 133, 135, 138, 140
  - in Meggitt decoders, 115–116
- Syndromes, 51–54
  - for BCH codes, 178–186
  - of coset leaders, 56
  - for cyclic codes, 104–108
  - in decoders, 115
  - for double errors, 57–58
  - for error trapping, 136–138
  - for expurgated Hamming codes, 128
  - for high-speed error-trapping decoders, 158, 163
  - for majority-logic decodable codes, 258–260
  - for Reed-Solomon codes, 229, 232
    - expurgated, 226, 228
  - for single-error corrections, 57
- Systematic convolutional codes, 319–325, 387–388
- Systematic groups, 19
- t*-error-correcting code, 46–47
  - BCH codes, 174, 179, 212
  - error trapping for, 135
  - perfect, 67
  - Reed-Solomon codes, 236
- Threshold decoders, 393
- Threshold (majority-logic) decoding, 8, 311, 351–352, 387–388
  - codes for:
    - burst-error-correcting, 415–424
    - performance of, 413–415
    - self-orthogonal, 395–405
    - trial-and-error orthogonalizable, 405–411
    - uniform, 411–413
  - syndrome decoding of, 388–391
    - feedback and, 391–395
- Threshold (majority) logic, 393
- Time-domain techniques, 214
- Tong, S. Y., 10, 415, 423, 424
- Tree diagrams, 332–333, 335–337
- Tree searches, 311, 352
- Trellis diagrams, 337, 338
- Trellis searches, 311, 352
- Trellis structures, 332–333
- Trial-and-error orthogonalizable codes, 405–411
- Uniform codes, 411–413
- Union bounds, 374, 377
- Van der Monde determinants, 175
- Varshamov-Gilbert-Sacks bound, 148
- Vector spaces, 5
  - in linear codes, 36
- Vectors, incidence, 290, 293, 303
- Viterbi, A. J., 9, 367, 374
- Viterbi algorithm, 7–9, 311–312, 352, 367, 370–374
- Viterbi decoding, 12, 219, 319, 325, 345, 367
  - complexity of, 387
  - performance bounds for convolutional codes with, 374–384
  - probabilistic decoding and, 352–353
  - threshold (majority-logic) decoding and, 351–352, 414–415
- Waveform channels, 3
- Weights, 5
  - enumerators of, 58–62
  - Hamming, 43–46, 342
  - of maximum-length codes, 269–270
- Weldon, E. J., Jr., 75, 271, 275, 308
- Wozencraft, J. M., 7, 9, 311, 351
- Wyner, A. D., 415
- Zierler, N., 9, 167
- Zigangirov, K., 8, 9, 351, 354
- Zigangirov algorithm, 365
- ZJ (stack) algorithm, 8, 9, 351, 354, 361–366



## ABOUT THE AUTHOR

Man Y. Rhee is professor of electrical engineering at Hanyang University, Seoul, Korea. At this same university he had also served as vice president for research. He received his B.S.E.E. degree from Seoul National University and his M.S.E.E. and Ph.D. degrees from the University of Colorado, Boulder, Colorado.

In the United States, Dr. Rhee taught at Virginia Polytechnic Institute and was employed at the Jet Propulsion Laboratory, California Institute of Technology. In Korea he was vice president of the Agency for Defense Development, Ministry of National Defense, Republic of Korea; president of the Korea Telecommunications Company (where the ESS telephone exchange system was first developed under contract with ITT/BTM); and president of Samsung Semiconductor and Telecommunications Company.

Dr. Rhee is the author of *Coding Theory* published in Korea, and has authored and coauthored many journal and conference papers on the theory and applications of error-correction coding.



















3 5282 00357 6397



|                                |                                                                                   |          |
|--------------------------------|-----------------------------------------------------------------------------------|----------|
| JA 10 '94                      | DATE DUE                                                                          |          |
| SE - 6 '93                     |                                                                                   |          |
| Rhee, Man Young                |                                                                                   | TK5102.5 |
|                                |                                                                                   | R46      |
|                                |                                                                                   | 1989     |
| Error-correcting coding theory |                                                                                   |          |
| DATE                           | ISSUED TO                                                                         |          |
| SE - 6 '93                     |  |          |
| JA 10 '94                      |                                                                                   |          |
| <i>renew</i>                   |                                                                                   |          |

TK5102.5  
R46  
1989



3 5282 00357 6397

# Error Correcting Coding Theory

Progressing systematically from block codes to convolutional codes, this authoritative book fully covers the basic structure, properties, code design, and implementation of various decoding methods for random error correction as well as burst error correction. Step by step it shows how to apply error-control techniques that permit communication links to function reliably in the presence of noise, distortion, and interference.

In addition, the book provides principles and methods for the use of error-protection codes to protect data in computer memories, on magnetic tape, or in disk storage systems. It also describes the use of these codes to protect against noise or circuit malfunction in digital logic circuits.

Clear, concise sections cover elements of abstract algebra for coding...linear binary block codes...cyclic codes...short random-error correction by Meggitt decoder or error-trapping decoding...burst error correction for block codes...BCH codes and Reed-Solomon codes...majority-logic decodable codes...convolutional codes and their structures...maximum-likelihood decoding and sequential decoding for convolutional codes...and threshold decoding for convolutional codes.

Cover Design: Eileen Kramer



9 780070 520615

90000



ISBN 0-07-052061-5



For more information about other McGraw-Hill materials, call 1-800-2-MCGRAW in the United States. In other countries, call your nearest McGraw-Hill office.

**McGraw-Hill Publishing Company**  
Serving the Need for Knowledge  
1221 Avenue of the Americas  
New York, NY 10020